

Week 0

All right, this is. OK. This is CS 50, Harvard University's introduction to the intellectual Enterprises of Computer Science and the arts of programming. My name is David Malan, and this is week 0, and by the end of today you'll know not only what these light bulbs here spell, but so much more. But why don't we start first with the, uh, the elephant or the elephant in the room, that is artificial intelligence, which is seemingly everywhere over the past few years, and it's been said that it's going to change programming, and that's absolutely the case. It's been that way actually for the past several years. It's only going to get to be the case all the more, but this is an incredibly exciting time. This is actually a good thing, I do think insofar as now using AI in any number of forms, you can ask the computer to help solve some problem for you. You can find some bug or mistake in your code. Better still, increasingly you can tell the AI what additional features you want to add to your software, and this is huge because even in the industry for years, humans have been programming in some form for decades, building products and solutions to problems. The is that you and I as humans have long been the bottleneck. There's only so many hours in the day. There's only so many people on your team or in your company, and there's so many more bugs that you want to solve and so many more features that you want to implement. But at the same time you still really need to understand the fundamentals. And indeed a class like this, CS 50, has never been about teaching you how to program. Like that's actually one of the side effects of taking a class like this, but the overarching goal is to teach you how to think, how to take input and produce. Correct output and how to master these and other tools. And so by the end of the semester, not only, not only will you be acquainted with languages like Scratch, which we'll touch on today if you've not seen it already, languages like C and Python and SQL, HTML CSS and JavaScript, you'll be able to teach yourself new things ultimately and ultimately be able to tell computers increasingly what it is you want it to do, but you'll still be in the driver's seat, so to speak. You'll be the pilot, you'll be the conductor, whatever your preferred metaphor. And that's what I think is so empowering still about learning introductory material, foundational material, because you'll know what you're ultimately talking about and what you can in fact solve. We've been through this before, like when calculators came out, it's still valuable, I dare say all these years later to still know how to do addition and subtraction and whatnot. And yet I think back on some of my own math classes, I remember learning so many darn ways in college how to take derivatives and integrals. And after like the 6th process of that, I sort of realized, OK, I get it. I get the idea. Do I really need to know this many ways? And here too, with AI and with code, can you increasingly sort of master the ideas and then lean on a copilot, assistant to actually help you solve those same problems. So let's do some of this ourselves here. In fact, just to give you a teaser of what you'll be able to do yourselves before long, let me go ahead and open up a little something called Visual Studio Code, AKA VS code for short. This is popular, largely open source or free software that's used by real world people. In industry to write code and it's essentially a text editor similar to notepad, if you're familiar with that or text edit, kind of like Google Docs, but no bold facing and underlining and things like that that you'd find in word processing programs. And this is CS 50's version thereof. We're going to introduce you to this all the more next week, but for now, let's just give you a taste of what you can do with an environment like this. So I'm going to switch

over to this program already running VS code, and in this bottom of the screen, you're going to see a so-called terminal. Window again, more on that next week, but it's in this terminal window that I can write commands that tells the computer what I want it to do. For instance, let's suppose just for the sake of discussion that I want to make my own chatbot, not Chat GPT or Gemini and Claw. Like, let's make our own in some sense. So I'm going to code up a program called chat.ie, and you might be familiar that I, I'm using a language here.ie is it's just called Python, and if unfamiliar, you're in good company. You'll learn that too within a few weeks. And at the top. File here I can write my code and at the bottom of the file of the window here I can run my code. So here's how relatively easy it is nowadays to write even your own chatbot using the AI technologies that we already have. I'm going to go ahead and type a command like import. I'm going to go ahead and type the following from OpenAI, import OpenAI. We'll learn what this means ultimately, but what I'm going to do is write my own program on top of. An API application programming interface that someone else provides a big company called OpenAI, and they're providing features and functionality that now I can write code against. I'm going to create a so-called client, which is to say a program of my own that's going to use this OpenAI software. And then I'm going to go ahead and ask this software for a response, and I'm going to set that equal to client.pos.create, whatever all that means, and then Inside of these parentheses, I'm going to say the following. The input I want to give to this underlying API is quote unquote something like in one sentence, what is CS 50? Much like I would ask Chachi PT itself. If you're familiar with things like Chat GPT and AI more generally nowadays, you know, there's this thing called models which are like statistical models that ultimately drive what the AI's can do. I'm gonna go ahead and say model equals quote unquote GPT 5, which is the latest and greatest version at least as of today. Now down in my terminal window I'm going to run a different command, Python of chat.ie and so long as I have made no typographical errors in this program, I should be able to ask OpenAI, not with Chat GPT.com, but with my own code for the answer to some question. But I want to know what the answer to that question is, so I actually want. Print out that response by saying print response output text. In other words, these 10 lines, and it's not even 10 lines because a few of them are blank, I've implemented my own chatbot that at the moment is hard coded, that is permanently configured to only answer one question for me. And let's see with the cross of the fingers. CS 50 is Harvard University's introductory computer science course, the intellectual enterprises of Computer Science and the Art of programming. Weirdly familiar, covering problem solving algorithms, data structures, and more using languages like CPython and SQL. OK, interesting, but let's make the program itself more dynamic. Suppose you wanted to write code that actually asks the human what their question is, because very quickly might we want to learn something more than just this one question. So up here, I'm going to go and change my code and type some. Like this type prompt equals input with parentheses. More on this another time too, but what I'm going to ask the user for is to give me an actual prompt. That is a question that I want this AI to answer. And down here, what you'll notice, even if you've never programmed before, is that I can do something somewhat intuitive insofar as line 5 is now asking the human for input. Let's just stipulate that this equal sign means store that answer in a variable called prompt, where variables just like in math. Y or Z. Let's go ahead and store that in prompt. So the input I want to give to OpenAI now is that actual prompt. So it's a placeholder containing whatever keystrokes the human typed in. If I now run that same command again, Python of chat.ie, hit enter, cross my fingers, I'll see now

dynamic prompting. So what's the question I might want to ask? Well, let's just say it again in one sentence, whoops, in one sentence, what is CS 50 mark? Enter. And now the answer comes back as probably roughly the same but a little bit different, a variant thereof, but maybe we can distill this even more succinctly. How about let's run it again? Python of chat.pi and let's say in one word what is CS 50 and see if the underlying AI obliges. And After a pause course in a word, so that's not all that incorrect, and maybe we can have a little fun with this now. How about in one word, which is, which is better? maybe Harvard or Stanford Question mark. Hope you picked right. Let's see. The answer is. Depends. OK, so would not in fact oblige, but notice what I keep doing in this code. I keep providing a prompt as the human like in one sentence, in one word. Well, if you want the AI to behave in a certain way, why don't we just tell the underlying system to behave in that way so why the human don't have to keep asking it in one sentence, in one sentence in one word, so we can actually introduce one other feature that you'll hear discussed in industry nowadays, which is not only a prompt from the user, which I'm gonna. Now temporarily renamed to user prompt just to make clear it's coming from the user. I'm going to also give our what's called a system prompt by setting this equal to some standardized instructions that I want the AI to respect, like limit your answer to one sentence, quote unquote. And now in addition to passing in as input, the user prompt, I'm going to actually tell OpenI. to use these instructions coming from this other variable called System prompt. So in other words, I'm still using the same underlying service, but I'm handing it now not only what the user typed in, but also this standardized text limit your answer to one sentence so the human like me doesn't have to do that anymore. Let's now go back to my terminal, Run Python of chat.ie once more, and this time we'll be prompted, but now I can just ask what is CS 50 question. And I'll likely get a correct and similar answer to before. And indeed it's Harvard University's flagship introductory computer science course. So it seems spot on too. But now we can have some fun with this too, and you might know that these GPTs nowadays have sort of personalities. You can make them obliged to behave in one way or another. Why don't we go into our system prompt here and say something silly like pretend you're a cat. And now let's go back to the prompt one final time. Run Python of chat.ie prompt again will be say what is CS 50 and with a final flourish of hitting enter. What do we get back? CS 50 is Harvard University's introductory computer science course teaching programming, algorithms, data structures, and problem solving, and it's available free online meow. So that was enough to coerce this particular behavior. So this is to say that with programming you have the ability in like 10 lines of text, not all of which you might understand yet, but that's the whole point of a class like this to build fairly powerful things, maybe silly things like this, but in fact it's using the same primitives that CS 50 has its own. Virtual rubber duck and we'll talk more about this in the weeks to come, but long story short, in the world of programming, it's kind of a thing to keep a rubber duck literally on your desk or really any inanimate cute object like this because when you are struggling with some problems, some bug or mistake in your code and you don't have a friend, a teaching assistant, a parent, or someone else who's more knowledgeable than you. About code where you literally are encouraged in programming circles to like talk to the rubber duck, and it's through that process of just verbalizing your confusion and organizing your thoughts enough to convey it to another person or duck in this case that so often that proverbial light bulb goes off and you realize, ah, I'm being an idiot now I hear in my own thoughts the illogic or the mistake I'm making. And you solve that problem as well. So CS 50, drawing inspiration from this, will

give to you a virtual duck in computer form. And in fact, among the other URLs you'll use over the course of the semester is that here, CS50.AI, which is also built into that previous URL, CS50.dev, whereby these are the AIs you can use in CS50 to solve problems and you are encouraged to do so. As you'll see in the course of syllabus, it is not reasonable. Not allowed to use AI-based software other than CS 50's own, be it Claw, Gemini, Chachi PT, or the like, but it is reasonable and very much encouraged along the way to turn not only to humans like me, your teaching assistant, and others in the class, but to CS 50's own AI-based software. And what you'll find is that this virtual duck is designed to behave as close to a good human tutor as you might expect from an actual human in the real world knows about CS. knows how to lead you to a solution, ideally without simply spoiling it and providing it outright. So with that said, that's sort of the end game to be able to write code like that and more. But let's really start back at the beginning and see how we can't get from zeros and ones that computers speak all the way back to artificial intelligence. So computer science is in the name of the course, Computer Science 50, but what is that? Well, it's really Just the study of information. How do you represent it? How do you process it? and very much germane to computer science is what the world calls computational thinking, which is just the application of ideas from computer science, or CS to problems generally in the real world. And in fact that's ultimately, I dare say what computer science really is. It's about problem solving and even though we use computers, you learn how to program along the way, these are really just tools and methodologies. that you can leverage to solve problems. Now what does that mean? Well, a problem is perhaps most easily distilled into a simple picture like this. We've got some input, which is like the problem we want to solve, and the output, which is the goal we want the solution there too. And then somewhere in the middle here is the proverbial black box, the sort of secret sauce that gets that input from output. So this then I would say is in essence is problem solving and thus computer science. But we have to agree, especially if we're going to use devices, Macs, PCs, phones, whatever, how do we all represent information, the inputs and the outputs in some standardized way? Is it with English? Is it with something else? Well, you all probably know, even if you're not computer people, that at the end of the day computers somehow use zeros in one entirely. That is their entire alphabet. And in fact you might be familiar already with certain such systems. So the unary notation, which means you essentially use single digits like fingers on your hand, for instance, unary AKA base one, is something you can do on your own human hand. So for instance, with one human hand, how high can I count? All right, so hopefully 12345, and if you want to count to 6 to 11 and 10 and so forth, you need to, you know, take out another hand or your toes or the like because it's fairly limiting. But if I think a little harder instead of just using unary, what if I use a different system instead? What about something like binary? Well, how high if you think a little harder, can you count on one human hand? So 31, says someone who studied computer science before, but why is that? It's kind of hard to imagine, right? Because 12345 seems to be the five possible patterns, but that's only when you're looking at the totality of fingers that are actually up 5 in total or 4 in total, or 1 or the like. But what if we take into account the pattern of fingers that are up and we just standardize what each of those fingers represent? So maybe we all agree. Like a good computer would too, that maybe no fingers up means the number 0. And if we want to count to 1, let's go with the obvious. This is now 1. But instead of 2 being this, which was my first instinct, maybe 2 can just be this a single second finger up like this, and that means we could now use 2 fingers up to represent 3. I propose we

can use just one middle finger up to offend everyone but represent 4. I could maybe use these two fingers with some difficulty to represent 567. I'm already up to 7 having used only 3 fingers. And in fact, if we keep going higher and higher, I bet I can get as high as 31 for 32 possible combinations, but the first one was. So that's as high as we can count. So we'll make this connection in just a moment, but what I started to do there is something called base 2. Instead of just having fingers up or fingers down, I'm taking into account the positions of those fingers and giving meaning to like this finger here, this finger here, this finger here, and so forth, different weights if you will. So the binary system is indeed all computers. And you might be familiar with some terminology here. Binary digit is not really something anyone really says, but the shorthand for that is going to be bit. So if you've heard of bits, and we'll soon see bytes and then kilobytes and megabytes and gigabytes and terabytes and more, this just refers to a bit meaning a single binary digit, either a 0 or a 1. A 0 is perhaps most simply represented by just like turning, maybe keeping a finger down, or in the world of computers which have access to electricity, be it from the wall or maybe a battery, you know what we could do? We could just decide sort of universally that when a light bulb is off, that thing represents a zero, and when the light bulb was on that thing's going to represent a 1 instead. Now why is this? Well, electricity is such a simple thing, right? It's either flowing or it's not, and we don't even have to therefore worry about how much of it is flowing. If you're vaguely remember a little bit about voltage, we can sort of be like zero volts. Nothing's there available for us, or maybe it's 5 volts or something else in between. But what's nice about binary only housing zeros and ones is that it maps really nicely to the real world by like throwing a light switch on and off. You can represent information by just using a little bit of electricity or the lack thereof. So what do I mean by this? Well, suppose we want to start counting using binary zeros and ones only. Well, let's think of them metaphorically as like akin to these light bulbs here and in fact let me grab a few of these light bulbs and let me propose that if we want to represent the number 0, well, it stands to reason that here single light bulb that is off can be agreed upon as representing 0. Now in practice, computers don't have little light bulbs inside, but they do have little switches inside, millions of tiny little things called transistors that have. One can allow it to capture a little bit of electricity and effectively turn on a metaphorical bulb or the switch can go off. The transistor can go off and therefore let the electricity dissipate and you have just now a 0. Unfortunately, even though I can let some electricity, here's the battery I mentioned is required, even though we might have some electricity available to us, I can therefore count to 1. But how do I go about counting? Hardware problem, how do I go about counting higher than one with just a light bulb? Yeah, so I need more of them. So let me grab another one here, and now I can put it next to it, and this too I'll claim is just still the number one. But if I want to turn 2 of them on, well, that would mean I could count to 2. And if I maybe grab another one now I can count as high as 3, but wait a minute, I'm doing something wrong because with 3 human fingers, how high was they able to count. So 7 in total starting at 0. So I've done something wrong here, but let me be a little more clever than about the pattern that I'm actually using. Perhaps this can still be 1, but just like my finger went up and only one finger in the second version of this, this can be what we represent as 2. Which one do I want to turn on as 3, your left or your right? So you're right, because now this matches what I was doing with my fingers a moment ago, and I claimed we could represent 3 like this. If we want to represent 4, that's fine. We have to turn that off, this off, and this on, and that's somehow 4 and let's go all the way up to 7. Which ones need to be on to represent the number

7? All right, so all of them here. Now if you're not among those who just sort of naturally said all of them, like what the heck is going on? How do half the people in this room know what these patterns are supposed to be? Well, maybe you're remembering what I did with my fingers, but it turns out you're already pretty familiar with systems like this, even if you might not have put a name to it. So in the human world, the real world, most of us deal every day with the so-called base 10 system, otherwise known as. deck implying 10 because in the decimal system you have 10 digits available to you 0 through 9. In the binary system we only had 2 by implying 2, so 0 and 1. And unary we had just 1, a single digit there or not. So in the decimal system we just have more of a vocabulary to play with, and yet you and I have been doing this since grade school. So this is obviously the number 123. But why? It's technically just three symbols 123, but most of us, your mind immediately goes, OK, 123, pretty obvious, pretty natural. But at some point you, like me, were probably taught that this is the one's place and this is the ten's place and this is the 100s place and so forth. And the reason that this pattern of symbols 123 is 123 is that we're all doing some quick mental math and. I, well, that's 100 times 1 plus 10 times 2 plus 1 times 3, OK, there's how we get 100 plus 20 + 3 gives us the number we all know mathematically is 123. Well, it turns out whether you're using decimal or binary or other base systems that we'll talk about later in the course, the system is still fundamentally the same. Let's kind of generalize this away. Here's a three digit number in some base system, specifically in decimal, and I know that only because of Placeholders that I've got on top of each of these numbers. But if we do a little bit of math here, 1, 10, 100, 1000, 10,000, and so forth, what's the pattern? Well, technically this is 100, 10 to 1, 10 to 2, and so forth, and we're using 10 because we can use as many as 10 digits under each of those columns. But if we take some of those digits away and go from decimal down to binary, the motivation being it's Way easier for a computer to distinguish electricity being on or off than coming up with like 10 unique levels of electricity to distinguish among. You could do it. It would be annoying and difficult to build and hardware you could do it so much simpler to just say on and off. It's a nice simple world that way. So let's change the base from 10 to 2, and what does this get us? Well, if we now do undo the math, that's 2 to 0. 121 is 222 is 4. So the the mental math is now about to be the same, but the columns represent something a little bit different. So for instance, if I turn all of these off again, such that I've got off, off, off, otherwise known as 000, it's 0 because it's 4 times 0 + 2 times 0 + 1 times 0 still gives me 0. By contrast, If I turn on maybe just this one all the way over on the left, well, that's 4 times 1 because on represents 1 and off represents 0, plus 2 times 0 plus 1 times 0, that gives me 4. And if I turn both of these on such that all three of them are now on, on, on, on, AKA 111, that's 4 times 1 plus 2 times 1 plus 1 times 1. That then gives me 7, and we can keep adding more and more bits to this. In fact, if we go all the way up numerically, here's how we would represent in binary the number you and I know as 0. Here's how we would represent 1. Here's how we would represent 2 and 3 and 4 and 5, and you can kind of see in your mind's eyes. Now because I only have zeros and ones and no 2s or threes, not to mention nines, I'm essentially going to be carrying a 1 in a moment if we were to be doing some math. So to go from 5 to 6, that's why the 1 ends up in the middle column to go to 7 here gives us now 111 or on on on, how do I represent 8? Using ones and zeros, yeah, yeah, so we're going to need to add another digit. We need to throw hardware at the problem using an additional digit so that we actually have a column representing 8. Now, as an aside, and we'll talk about this before long, if you don't have an additional digit available, if your computer doesn't have enough

memory, so to speak, you might accidentally count from 01234567. And then accidentally end up back at 0 because if there's no room to store the 4th bit, well, all you have is part of the number, and this is going to create all sorts of problems then ultimately in the real world. So let me go ahead and put these back and propose that we have a system now if you agree to sort of count numbers in this way via which we can represent information in some standard way and all the device underneath the hood. It is a bit of electricity to make this work. It's got to be able to turn things on, AKA use some transistors, and it's got to be able to turn those things off so as to represent zeros instead of ones. But the reality is like 2 bits, 3 bits, 4 bits aren't very useful in the real world because even with 3 bits you can count to 7, with 4 you can count to 15. These aren't very big numbers, so it tends to be more common to actually use units of measure of 8 bits at a time. A byte is just that. 1 byte is 8 bits. So if you've ever used the vernacular of kilobytes, megabytes, gigabytes, that's just referring to some number of bits, but 8 of them together compose one individual byte. So here for instance is a byte worth of bits, 8 of them total. I've added all the additional placeholders, and what number does this represent in decimal even though you're looking at 8 binary digits. It's just 0, because like literally every column is a 0. Now this is a bit more of mental math, but unless you know it already, what if I change all of the zeros to ones? I turn all 8 light bulbs on, what number is this? Yeah, so 255. Now some of those of you who didn't get that instantly, that's fine. You could certainly do the math manually. I dare say some of you have some prior knowledge of how to do this sort of system, but 255 means that if you start counting at 0 and you go all the way up to 255. OK, that's 256 total possibilities. Once you include 0 in the total number of patterns of zeros and ones, and this is just going to be one of these common numbers in computer science, 256. Why? Because it's referring to 8 of something, 2 to 8. Gives you 256 and so you're going to commonly see certain values like that. 256 back in the day, computers could only show 256 colors on the screen. Certain graphics formats nowadays that you might download can only use as many as 256 colors because, as we'll see, they're only using, for instance, 8 bits and therefore they can only represent so many colors of the rainbow as a result. So this then is how we might go from just zeros and ones, electricity inside of a computer to storing actual numbers with which we're familiar and honestly we can go higher than 255. What do you need to count higher than 255? 1/9 bit, a 10th bit, 11th bit and so forth and it turns out common conventions nowadays, and we'll see this in code 2 is to use as many as 32 bits at a time. So that's a good chunk of bits and anyone want to ballpark how high you can count if you've got 32 bits available to you. Oh, fewer people now, yeah, in the back. Yeah, so it's roughly 4 billion, and it's technically 2 billion if you also want to represent negative numbers, but we'll revisit that question, but 2 to 32nd power is roughly 4 billion. However, nowadays it's even more common with the Macs and PCs you might have on your laps and even your phones nowadays to use 64 bits, which is a big enough number that I'm not even sure offhand how to pronounce it. That's a lot of permutations that. 2 to 64 possible permutations, but that's increasingly commonplace. And as an aside, just to dovetail things with our discussion of AI, among the reasons that we're living through over these past few years, especially this crazy interesting time of AI, is because computers have been getting so much faster, exponentially so over time. They have so much more memory available to them. There's so much data out there on the internet in particular. to train these models that it's an interesting confluence of hardware now actually meeting the mathematics and statistics that we'll talk about later in the class that ultimately make tools like the cat we just built possible.

But of course computers are not all math, and in fact we'll use very little math per se in this class. And so let's move away pretty quickly from just zeros and ones and talk about letters of the alphabet, say in English. Here is the letter A. Suppose you want to use this letter in an email, a text message, or any other program. What is the computer doing underneath the hood? How can the computer store a capital letter A in English if at the end of the day all the computer has access to is a source of electricity from the wall or from a battery, and it has a lot of switches that it can turn on and off and treat the electricity in units of 8 or 32 or 64 or whatever. How might a computer represent a letter A? Yeah, we need to give it an identity, so to speak, as an integer. In other words, at the end of the day, if your entire canvas, so to speak, consists only of zeros and ones, like that is going to be the answer to every question today. You only have zeros and ones as the solution to these problems. We just need to agree what pattern of zeros and ones and therefore what integer, what number shall be used to represent the letter A. And hopefully, When we look at that pattern of zeros and ones in the right context, we'll indeed see it as an A. So if we look inside of a computer, so to speak, in the context of like a text messaging program or a word processor or anything like that, that pattern shall be interpreted hopefully as a capital letter A. But if I open up Mac OS's or Windows or my phone's calculator program, I would want that same pattern of zeros and ones to be interpreted instead as a. If I open up Photoshop, as we'll soon see, I want that same pattern of zeros and ones to be interpreted as a color presumably, not to mention videos and sound and so forth, but it's all just zeros and ones. And so even though I, when writing that chat program a few minutes ago, didn't have to worry about telling the computer, oh this is text, this is a number, this is something else, we'll see as we write code ourselves that you as the programmer will have control over telling the computer how to treat some pattern of zeros and ones telling it this is a number, this is a color, this is a letter or something else. Um, how do we represent the letter A? Well, it turns out a bunch of humans in a room years ago decided that this pattern of zeros and ones shall be known globally as a capital letter English A. What is that number if you do the quick mental math? So indeed 65 because we had a 1 in the 60s place and a 1 in the one's place. So 65, that's just sort of it. It would have been nice if it were just the number 1 or maybe the number 0, but at least after the capital letter A, they kept things consistent such that if you want to represent a letter B, it's going to be 66. C, it's going to be 67 Y because. The humans in this room, a bunch of Americans at the time, standardized on what's called ASCII, the American Standard Code for Information Interchange. It doesn't matter what the acronym represents, but it was just a mapping. Someone on a piece of paper essentially started writing down letters of the alphabet and corresponding numbers so that computers subsequently could all speak that same standard representation and Here's an excerpt thereof. In this case we're seeing 7 bits' worth, but eventually we ended up using 8 bits in total to represent letters, and some of these are fairly cryptic, maybe more on those other time, but down here if we highlight just one column, we'll see that indeed on this cheat sheet 65 is capital A, 66 is B, 67 is C, and so forth. So why don't we do a little exercise here. What pattern of zeros and ones do I see here? I've got 3 bytes, so 3 sets of 8 bits, and even though there's no placeholders now over the columns, what is this? Number It's 60. Yeah, so we've got the 1s, 2s, 4s, 8s, 16s, 32s, 64s column. So indeed this is going to be the number 72, 72. This is not what computer scientists spend their day doing. This is just to reinforce what it is we just looked at, and I'll spoil it. The rest of these numbers are 72, 73, 33, and anyone in this room could have done that if you took out a piece of paper, figured out what the columns are,

and just do a bit of quick or. Of mental or written math, but this is to say, suppose that you just got a text message or an email that if you had the ability to look underneath the hood of the computer and see what pattern of zeros and ones did you just receive over the internet, suppose that pattern of zeros and ones was 3 bytes of bits, which when you do the math are the numbers 72, 73, 33. Well, here's the cheat sheet again. What message did you just get? Yeah, so it's high. Why? Because 72 is H and 73 is I. Now some of you said hi fairly emphatically why. Well, 33 turns out you wouldn't know this unless you looked it up or someone told you is an exclamation point. So literally if you were to text someone like right now, if you haven't already, HI exclamation point in all caps. You would essentially be sending 3 bytes of information somehow over the internet to that recipient and because their phone similarly understands ASCII because it was programmed years ago to do so, it knows to show you HI exclamation point and not a number, 3 numbers no less, or colors or something else altogether. So here we then have high, 3 digits in a row here. What else is worth noting here? Well, there's some fun sort of trivia embedded even in this cheat sheet. So here again is A, B, C, D, E, F, G, and so forth, 65 on down. Let me just highlight over here the lowercase letters 97, 98, 99, and so forth. If I go back and forth, does anyone notice the consistent pattern between these two? Yeah, so the lowercase letters are 32 away from the uppercase letters. Well, how do we know that? Well, 97 minus 65 is, yeah, 32. 98 minus 66 is OK, 32, and that pattern continues. What does this mean? Well, computers know how to do this. Most normal humans don't need this information, but what it means is if you are representing in binary with your transistors on and off representing some pattern, and this is the pattern representing capital letter A, which is why we have a 1 in the 604 space and a 1 in the ones place, how does a computer go about lower casing this same letter? Yeah. Perfect. All the computer has to do is change this one bit in the 32's place to a 1, because that has the effect mathematically per our discussion of adding the number 32 to whatever it is. So it turns out you can force text from upper case to lower case or back by just changing a single bit. Inside of that pattern of 8 bits in total. All right, why don't we maybe reinforce this with another quick exercise? We have an opportunity perhaps here for um maybe to give you some stress balls right at the very start of class. Could we get 8 volunteers to come up on stage, maybe over here and over here and over here on the left. Let me go all the way on the right. Uh, let's see. OK, the high hand here, the, the hand that's highest there, yes, we're making eye contact. How about all the way, what, wait, let's see, let's go here in the crimson sweatshirt here, and how about in the, the white shirt here? Come on up. Did I count correctly? Let's say Come on down. The 8 of you. I, I didn't count right, did I? 123456. It's ironic that I'm not counting correctly here. How about on the left in gray? OK. Oh, OK, in black here. Come on down. All right, hopefully this is 81234567. 7. I'm pretty sure. OK, 8, there we go. All right. So let's go ahead and do the following exercise. I've got some sheets of paper preprinted here. If each of you indeed want to do exactly what you're doing and line up from left to right, each of you is going to represent a placeholder essentially. So we have over here the ones place all the way over here, and then we have the choose place. And the 4th place and the eights. 16 32, 64, 128, and we come bearing a microphone. If each of you want to say a quick hello, your name, maybe your dorm or house, and something besides computer science that you're studying or want to. Hi, I'm out. OK. I'm Alison. I'm a freshman in Mathews, and, um, I like climbing and I'm thinking of CS and Econ. 2. Hi, I'm Lily. I'm in Hurlbut this year and I'm thinking of doing CS in government. Nice to meet you. Hi, hi, I'm Sean. I'm in Candidate Hall and I'm thinking of doing

astrophysics and CS. Welcome. Hi, I'm Jordan. I'm doing applied math with a specialization in CS and Econ, and, um, I'm in Widsworth, and I like going to the gym. 16. Hi, I'm Shiv. I'm studying Mac and I'm in Canada. Nice. Hi, I'm Sophia. I'm in Thayer and I'm thinking of doing electrical engineering. Welcome. Hi, my name is Marie, and I'm in Canada B and I really like CS physics and astrophysics. Hi, I'm Alyssa. I'm in Holworthy. I'm also thinking of studying math or physics, and I also like to climb. Nice. Welcome to you all. So on the backs of their sheets of paper, they have a little cheat sheet that's describing what they should do in each of 3 rounds. We're going to spell out together a three letter word. You all as the audience have a cheat sheet above you that represents numbers, 2 letters. These folks don't necessarily know what they're spelling. They only know what they individually are spelling. So if your sheet of paper tells you to represent a zero in a given round, just kind of stand there awkwardly, no hands up. But if you're told on your sheet of paper to represent a 1, just raise a single hand to make obvious to the audience that you're representing a 1 and not a zero. And the goal here is to figure out what we are spelling using this system called ASCII. All right, round 1, execute. What number is this here? I'm hearing you can just shout it out. What number? 66 or B. So you're spelling B. All right, hands down. Round 2. More math. Feel free to shout it out Oh, I heard it, yeah, 79, which is, oh, OK, so we have BO. Hands down, 3rd and final round, execute. Number Yes, 87, which is the letter? W, which spells? Bow, if you want to take your bow now. Ah, OK, here we go. You guys can keep those. OK, thank you. All right, you guys see that back. Thank you to our volunteers here. Very nicely done. We indeed spelled out bow, and that's just because we all standardized on representing information in exactly the same way, which is why when you type BOW on your phone or your computer, the recipient sees the exact same thing. But what's noteworthy in this discussion is that you can't spell a huge number of words like, yeah, English, OK, we've got that covered. But odds are you're noticing, depending on your own background what human languages you read or speak yourself, that a whole bunch of symbols might be missing from your keyboard. For instance, we have accented characters here in a lot of Asian languages. There's so many more glyphs than we could have even fit in that cheat sheet of numbers and letters. And so ASCII is not the only system that the world uses. It was one of the earliest, but we've moved on in modern times to a super set of AI that's generally. Known as Unicode, and Unicode uses so many more bits than AI that we even have room for all of these little things that we seem to send constantly nowadays. These are obviously images that you might send with your phone or your computer, but they're technically characters. They're technically just patterns of zeros and ones that have similarly been standardized around the world to look a certain way, but they're this is an emoji keyboard in the sense that. You're sending characters. You're not sending images per se. The characters are displayed as images, obviously, but really these are just like characters in a different font, and that font happens to be very colorful and graphical as well. So Unicode, instead of using just 7 or 8 bits, which if you do the quick mental math, if ASCII only used 7 or let's say 8 bits, how many possible characters can you represent in ASCII alone? 256 because if we do that quick mental math, 2⁸, 256 possibilities like that's it. That is, that's enough for English because you can cram all the uppercase letters, the lower case letters, the numbers, and a whole bunch of punctuation as well, but it's not enough for certain other punctuation symbols, not to mention many other human languages. And so the Unicode Consortium. Its charge in life has been to come up with a digital representation of all human language, past, present, and hopefully future by using not just 7 or 8

bits but maybe 16 bits per character, 24 bits, or heck, even 32 bits per character. And per before, if you've got as many as 32 bits available to you, you can represent what, like 4 billion characters in total, and that's just one. One of the reasons why these emoji have kind of exploded in popularity and availability, there's just so many darn patterns like what else are we going to do with all of these zeros and ones. But more importantly, emoji have been designed to really represent people and places and things and emotions in a way that transcends human language. But even then they're somewhat open to interpretation. In fact, here's a pattern of I think 32 zeros and ones. Guessing no one's going to do the quick mental math here, but this represents what decimal number if we do in fact do out the math with that's being the ones place all the way over to the left, well, that's the number 4 billion, 36, 991,106. Who knows what that is? It's not a, and it's nothing near A upper case or lower case, but it is among the most popular emoji that you might send typically on your phone, laptop, or other device, namely. This thing here face with tears of joy, which odds are you've sent or received recently, but interestingly, even though many of you might have iPhones and see and send the same image, you'll notice that if you see a friend who's got Android or some other device, maybe you're using uh Meta's messenger program or Telegram or some other messaging service. Sometimes these emoji look a little bit different. Why? Because what a Unicode has done is they decided there shall exist an emoji known as excuse me, faced with tears of joy. Then Apple and Google and Microsoft and others, they're sort of free to interpret that as they see fit. So what you see on the screen here is a recent version from iOS. operating system. Google's version of the same looks a little something like this, and on Telegram if you have animations enabled, the same idea faced with tears of joy is actually animated, but it's the same pattern of zeros and ones in each case, but again they each essentially have different graphical fonts to present to you what each of those images actually is. All right, so those are each, excuse me, images. So those are each images. How is the computer representing them though? At the end of the day we've represented numbers, we've represented letters, but how about these things here colors. So how do we represent red or green or blue, not to mention every other color in between. At the end of the day, we only have one canvas at our disposal, yeah. So integers is the exact same answer as before. We just need to agree on what number do we use for red, what do we use for green, what do we use for blue, and we can come up with some standardized pattern for this. In fact, one of the most common techniques for doing this, and the one of the most common ways to do this in the real world is to use a combination of three colors together some amount of red, some amount of green, and some amount of blue and mix them together to get most any color of the rainbow that you might want. This is sort of a picture of something I grew up with back in the day where in like middle school when we'd watch movies or some kind of show and like in in class, we would kind of uh the projector screen would be over here. This is an old school projector with 3 different lenses, one of which projects some amount of green, some amount of red, some amount of blue, and so long as the lenses are correctly oriented to all point at the same circle or like rectangular region on the screen, you would see any number of colors coming to life in the old school. Video. I still remember all these years later we would kind of sit and lean up against it because it was super warm and you could feel it it's an easy way to fall asleep but back in grade school, but we use the same fundamental color system nowadays as well, including in modern programs like Photoshop. So let's abstract that away, focus on just three colors some amount of red, green, and blue, and let's propose for the sake of discussion

that we want to mix together like a medium amount of red, a medium amount of green, and just a little bit of blue. For instance, Let's propose that we'll use 72 amount of red, 23 amount of green, or 33 amount of blue RGB. Now why these numbers? Well, in the context of ASCII or Unicode, which is just a superset thereof, what is this spell? Hi, but again, if you were instead to open a file containing these 3 numbers or really these 3 bytes of bits in Photoshop, you would hope that they're going to be interpreted not as letters on the screen, but as some the color of a dot on the screen instead. So it turns out that in typically when you have three of these numbers together, each of them is using a single byte, so 8 bits. So you can have 0 red or 255 red, 0 green or 255 green or 0 to 255 of blue. So 0 is none, 255 is the max. So if we mix these together, imagine just like that projector consolidating these three colors into one central point. Anyone want to guess what you're gonna get if you mix some red, some green, some blue in those amounts and way back? Yeah, you're going to get a dark shade of yellow. I've brightened it up a little bit for the projector here, but you're going to get it roughly this shade of yellow, and we could play with these numbers all day long and get similar results if we want to represent different colors as well. And indeed, whether it's Photoshop or some other program, you can actually combine these amounts in all sorts of ratios to get different colors. So if you had 000, so no red, no green, no blue, take a guess as to what color that's going to be in the computer. So it's gonna be black, like the absence of all three of those colors, but if you mix the maximal amount of each of those 255 red and green and blue, that's gonna give you white. Now if any of you have made web pages before or use programs like Photoshop, you might have seen numbers like 00. Or FF. Long story short, that's just another base system for representing numbers between 0 and 255 as well, but we'll come back to that next semester when we make some of our own filters, uh, in sort of an Instagram-like way manipulating images of our own. So where are these colors coming from or where can we actually see them? Well, here's just a picture of that same emoji faced with tears of joy. If I kind of zoom in on that and maybe zoom in again, you can start to see if you blow it up enough or if you put your eyes close enough to the device sometimes, you can actually see individual dots or squares. These are generally known as pixels, and they're just the individual dots that collectively compose in which is to say that if each of these dots, which is part of the image, is going to be a distinct color like this one's yellow, this one's brown, and then there's a bunch in between, well, you're using some number of bits to represent each of those pixels' colors. So if you imagine using the RGB system, that's $8 + 8 + 8$ bits, so that's 24 bits or 3 bytes, just. To keep track of the color of each and every one of these dots. So now if you think about having downloaded a GIF at some point, a ping, PNG file, a JPEG, or any other file format, it's usually measured in what file size like megabytes typically that means millions of bytes. Why? Because if it's a pretty big photograph or pretty big image, each of those dots takes up at least 3 bytes, it would seem. And if you do out the math, if you've got thousands of dots, each of which uses 3 bytes, you're going to quickly get to megabytes, if not even larger, for things like, say, videos. But again, it's just patterns of zeros and ones and so long as the programmer knows what they're doing and tells the computer how to interpret those zeros and ones and equivalently, so long as the software knows, look at these zeros and ones and interpret them as numbers or letters or colors, we should see what we intended to represent. All right, so that's num that's uh colors and images. What about how many of you kind of played with these little flip books as a kid where they've got like 100 different little pictures and you flip through them really quickly and you see what looks like

animation in book form? Well, this is essentially a video, so therefore what is a video or how can you think of what a video is? It's just a whole bunch of like images flying across the screen either on paper or digitally nowadays on your phone or your laptop and that's kind of nice because we're sort of composing more interesting media now based on these lower level building blocks and this is going to be thematic. We literally started with zeros and ones. we worked our way up to letters. We then worked our way up to sort of images and colors and thus images. Now we're up at this level of hierarchy in terms of video because what's a video. It's like 30 images per second flying across the screen or maybe slightly fewer than that, that collectively tricks our mind into thinking we are seeing motion pictures, and that's the old school term for movies, but it literally is what it was. Motion pictures was this film was showing you 30 pictures per second, and it looks like motion, even though you're just looking at images, much like this flip book very quickly one after the other. What about music? Well, how could you go about representing musical notes if again your only ingredients are zeros and ones. Even if you're not a musician, how do you represent music like that on the screen here? Yeah. OK, so the frequency, like the tone that you're actually hearing from the device, what else might weigh in beside besides the frequency of the notes? Yeah. So the speed of the note, maybe the duration, like if you think about a physical piano, like how long you're holding the key down for or not, what else? So the amplitude may be how loud, like how hard did you hit the keyboard to generate that sound. So let me propose at the risk of simplifying, we could represent each of these notes using 3 numbers, maybe 0 to 255 or some other range that represents the frequency or the pitch of the note. Duration and the loudness and so long as the person receiving a file containing all of those zeros and ones knows how to interpret them three at a time, I bet you could share a musical file with someone else that they could hear in exactly the same way that you yourself intended. Let me pause here to see if there's any questions now because we've already built our way up from zeros and ones now to video and sound. Yeah, in front. What the letter like 65 would be and then the number 5. So how does the computer distinguish between the letter 65 and the number 65? It's context dependent. So put simply, and we'll see this as early as next week, the programmer tells the computer how to display the information either as a number or a letter or equivalently once. Programmed, the software knows that when it opens a dot GIF file or jpeg or something else to interpret those zeros and ones as colors instead of as like. DOCX for Microsoft Word file or the like. Other questions. On any of these representations, yeah, in front like the 1 thing like really briefly. Sure, so can we go on base 10 and base 2? So base 10 is like literally the numbers you and I use every day. It's base 10 in the sense that you have 10 digits at your disposal, 0 through 9, and any numbers you want to represent in the real world must be composed using 0 through 9. The binary system or base 2 is fundamentally the same. It's just the. doesn't have access to 2 through 9. It only has access to 0 and 1. But much like the light bulbs I was displaying here, you can simply ascribe different weights to each of the digits so that instead of it being as much as the one's place, the tens place, and the hundreds place, if we more modestly say the one's place, the two's place, the four's place, we can. The same system in binary you might need to use more digits to count as high because in 255 you can just write 255. That's 3 digits in decimal. But in binary we've seen you need to use 8 such digits, which is more, but it's still much better than unary, which would have had 255 light bulbs on. Instead. And they like. Is binary and base 2 the same thing? Yes, just like base 10 and decimal are the same thing as well, and unary and base 1 are the

same thing as well. All right, so let me just stipulate that even though we sort of took this tour quickly at the end of the day, computers only have zeros and ones at their disposal. So again, the answer to any question is to how can we represent. X is going to somehow involve permuting those zeros and ones into patterns or equivalently into the numbers that they represent. But if we now have a way to represent all inputs in the world, be it letters, numbers, images, videos, anything else, and get output from some problem solving process, like how do we actually solve problems? Well, the secret sauce in the middle here is another term that you've probably heard in the real world nowadays, which is that of algorithm. step by step instructions for solving some problem. So this ultimately is what computer science really is about too. It's not just representing information but somehow processing it, doing something interesting with it to actually solve the problem that you've been provided as input so you can output the correct answer. Now there's all sorts of algorithms implemented in our phones and in our Macs and PCs, and that's all software is. It's an implementation in. Code, be it C++ or Java or anything else, other languages exist too in code that the computer understands, but it's still just step by step instructions. And among the things we'll learn in CS 50 is how to express yourself in different ways to solve problems not only in different languages but using different methodologies as well, because as we'll see, among the reasons we introduced these several languages is you don't just learn more and more languages. They allow you to solve the same problems. Different languages will allow you to solve different problems and even save you time by being better tools for the job. So here, for instance, on an iPhone is maybe a bunch of contacts, which is presumably familiar where we might have a whole bunch of friends and family and whatnot alphabetized by first name or last name. And suppose we want to find one such person like John Harvard, whose number here might be + 1 94946. 82,750. Feel free to call or text him sometime. This is the goal of this problem. If we have our contacts app and I start typing in John's name by first name or last name, the autocomplete nowadays kicks in and it somehow filters the list down from my 10 or 100 friends or 1000 friends into just the single directory entry that matches. So here too, back in the days of RG&B. Projectors we had phone books like this here too. I'm pleased to say thanks to our friend Alexis, this is the largest phone book that we've used for this demonstration. This is an old school phone book that's essentially the same thing as our contacts app or address book nowadays, whereby I've got a whole bunch of names and numbers alphabetically sorted by first name or last name, whatever, and corresponding to each of those as a number. So back in the day, and frankly even nowadays in your phone. How do you go about finding someone in a phone book or your contacts app? Well, you could very naively just start at the beginning and look down and just turn one page at a time looking for John Harvard in this case. Now, so long as I'm paying attention, this step by step process will get me to John Harvard. Like this is a correct algorithm, even though you might kind of object to how I'm doing this, why, what's bad about this algorithm? It's just slow. I mean this is crazy slow. If there's like 1000 pages in the songbook, which looks like there are like this could take me as many as 1000 pages or maybe he's roughly in the middle, it's like 500 pages. Like that's crazy. That's really rather slow, especially if I'm going to do this again and again. Well, what if I do it a little smarter grade school, I sort of learned how to count 2 at a time, so 2468, 1012, 1416, 18. Again, if I'm paying attention, I'll get there twice as fast because I'm counting 2 at a time, but is that algorithm step by step correct? And I'm seeing no, but why? I might skip over John Harvard. So just by bad luck and kind of with 50/50 probability, he's going to be

sandwiched between two of the pages. Now I don't have to abort this algorithm altogether. I could just, as soon as I get past the J section if we're doing it by first name, I could just double back one page and just make sure that I haven't missed him. So it's recoverable and this algorithm therefore is sort of twice as fast. Mm, plus one extra step maybe to double back, but that's arguably otherwise a bug or a mistake in the algorithm if I don't fix it intelligently. But what did we do back in the day and what does your iPhone or Android phone do? What they typically do is they go roughly to the middle, look physically or virtually down, they see, oh, I'm in the M section, and so which side is John Harber to, to the left or to the right? So he's to the left, so I could literally now. Jesus Christ. We talked about this before class and this might be more, oh my God. There we go. We can tear the problem in half. Thank you. It's been a while. Uh, we can tear the problem in half. We know that John Harvard is to the left, so I can throw half of the problem away if, uh, dramatically, such that I'm now going to from a 1000 page problem to 500 pages instead. What now can I do? I can go roughly to the middle here and maybe I'm in the E section. So I went a little too far back to the left, but I kept it simple and I just divided so that I can conquer this problem, if you will. And if I'm in the E section now, is John Harvard to the left or to the right? To the right, so I can get, Jesus. Tear the problem in half, and now, thank you. So now John Harvard again is going to be in this half. I can throw this half away. So now I've gone from 1000 to 500 to 250, and I can repeat, repeat, repeat, down to 125, half of that, half of that, half of that, so I'm left with finally just a single page. And John Harvard is hopefully now on this page such that I can call him or not at all, at which point this is. All sort of for naught, but what's powerful about each of those algorithms is that the sort of good, better, and best. Like they all get the job done, conditional on the second one having that little fix just to make sure I don't miss John Harber between two pages, but they're fundamentally different in their efficiency and the quality of their design. And this is really representative of one of the emphases of a class like this. It's not just about writing correct code or getting the job done, but doing it well and doing it quickly, using the least amount of CPU or computing resources, using the minimal amount of RAM, using the fewest number of people using the least amount of money, whatever your constrained resource is, solving a problem better. So that first algorithm, step by step instructions, was all about doing something like this whereby. The first algorithm, if we plot things on a grid like this, we have on the x axis a representation of the size of the problem. So this would mean small problem like 0 pages. This would mean big problem like 1000 pages, and then the Y or vertical axis we have some measurement of time. So this is the number of seconds or the number of page turns, whatever you're. Metric actually is, so this would be not much time at all, so fast this would be a lot of time, so slow. So what's the relationship if we just roughly draw these three algorithms? Well, the first one is technically a straight line, and we'll describe that as n . The slope is N because if you think of N as a number for the number of pages, well, there's a 1 to 1 relationship. And the first algorithm as to how many times I have to turn the page based on how many pages there actually is, and you can think about this in the extreme. If I was looking for someone whose name started with Z, I might have to go through like 1000 darn pages to get to that person whose name started with Z unless again I do something hackish and just kind of cheat and go to the end. If we execute these algorithms again and again in the same way, that's going to be pretty slow. But the second algorithm was. Pretty much twice as fast plus that one extra step potentially, but it's still a straight line because if there's 1000 pages and I'm dividing the problem and doing two pages at a time, well, that's

like N divided by 2 steps plus 1, give or take. But it's still a straight line because, but it's still better. Notice if this is the size of the problem, 1000 pages for instance, we'll notice that the first algorithm took literally twice as much time. As the second algorithm, so we're doing better already, but the third algorithm fundamentally is going to look something like this. And if you remember your logarithms, so to speak, sort of the opposite of an exponential, this curve is so much lower and flatter, if you will, than either of these two. Mathematically, more on this another time, the slope is going to be like log based 2 of N or just logarithmic in nature, but what it means is that it's growing very, very, very slowly. It's still going up. It's never going to flat line and go perfectly horizontal, but it goes up very slowly. Why? Well, if you think about two towns nearby like Cambridge on this side of the river and the town of Alston on the other, suppose that they still have phone books like this one and they merge their phone books for whatever reason. So overnight we go from a 1000 page phone book to a 2000 page phone book. The first algorithm is going to take literally twice as long. As well the second one because we're only going through it 1 or 2 pages at a time, but if the phone book size doubles from this year, for instance, to next year, you can kind of in your mind's eye think about the green line. It's not going to go up that much higher. Why? Well, practically speaking, even if the phone book becomes 2000 pages long, well, how many more times do you have to tear or divide that problem in half? Just one because you're taking a 1000 page bye out of it or a 500 then a 250, you're taking much bigger bytes out of it than just one or two at a time. And so what computer science and what algorithms and about good design is about is figuring out what is the logic via which you can solve problems not only correctly but efficiently as well. And that then gives us these things called algorithms. And when it comes time to code, which we're about to do too. Code is just an implementation and a language the computer understands of an algorithm. Now this assumes that we've come up with some digital way, that is to say 0 and 1 based way to represent names and numbers, but honestly we already did that. We came up with the ASCII and then Unicode to represent the names, and representing numbers is even easier than that. That's really where we started. So code is just about taking it. Input some standardized representation of names and numbers and spitting out answers and that's truly what iOS and Android are doing. When you start doing autocomplete, they could be searching from the top to the bottom, which is fine if you've only got a few friends and family in the phone. But if you've got 1000 or if you've got 10,000 or if it's not a phone book anymore, it's some database with lots and lots of data. Well it's. to reason that it'd be nice maybe if the computer kept it all alphabetized just like that book and jumped to the middle, then the middle of the middle, then the middle of the middle of the middle and so forth. Why? Because the speed is going to be much, much faster, logarithmic in nature and not linear, so to speak, in nature, but we'll revisit those topics as well. But for now, before we get into actual code. Let's talk for a moment about pseudo code. So pseudo code is not one formal thing, but every human will come up with their own way of representing pseudo code. It's an English-like or human-like formulation of step by step instructions just using terse correct English or whatever human language. So for instance, if I want to translate what I did somewhat intuitively with that phone book by just dividing in half, dividing in half into step by step instructions, I could hand. Or nowadays like a robot or something like that. Well, step one was essentially to pick up the phone book, which I did. Step 2 was I opened to the middle of the phone book in the third and final algorithm. Step 3 was look at the page as I did. Step 4 got a little more interesting even though I didn't verbalize this.

Presumably I was asking myself a question. If the person I'm looking for, John Harvard, is on the page, then I would have called him right then. But if you weren't on the page, if you instead were earlier in the book, as did happen, well then I'm going to go to the left, so to speak, but more methodically, I'm going to open to the middle of the left half of the book. Then I'm going to go back to line 3. That's interesting. We'll come back to that in a moment, but else if the person is later in the book, well, I'm going to open to the middle of the right half of the book and then go back to line 3. Now, let's pause here. Why do I keep going back to line 3? This would seem to get me doing the same thing forever, endlessly. But not quite. Why? Yeah, so because I am dividing the problem in half, for instance, on line 6 or line 9 implicitly just based on how I've written this, the problem's getting smaller and smaller and smaller. So it's fine if I keep doing the same logic again and again because if the problem's getting smaller. Eventually it's going to bottom out and I'm gonna have just one person on that page that I want to call and so the algorithm's done. But there is a perverse corner case, if you will, and this is where it's ever more important to be precise when writing code and anticipate what could go wrong. I should probably ask one more question in this code, not just these three. What might that question. Yeah. Yeah, so if John Harvard is not in the book, there's this corner case where what if I'm just wasting my time entirely and I get to the end of the phone book and John Harvard's not there, what should the computer do? Well, as an aside, if you've ever been using your Mac or PC or phone and the thing just freezes or like the stupid little beach ball starts spinning or something like that and you're like, what is going on? Some human at Google or Microsoft or Apple or the like made a mistake. They forgot, for instance, that 4th uncommon but possible. Situation wherein if they don't tell the computer how to handle it, the computer's effectively going to freak out and do something undefined like just hang or reboot or do something else. So we do want to add this else quit altogether. So you have well defined behavior and truly think that the next time your computer or phone spontaneously reboots or dies or does something wrong, it's probably not your fault per se. It's some other human elsewhere did not. Correct code they didn't anticipate cases like these. But now let's use some terminology here. There's some salient ideas that we're going to see in scratch and C and Python and these other languages I alluded to earlier. Everything I've just highlighted here henceforth we're gonna think of as functions. Functions are verbs or actions that really get some small piece of work done for you. Functions are verbs or actions. Here though highlighted is the beginning of what we'll call. Conditions, conditionals like a fork in the road. Do I go this way? Do I go this way, or some other way altogether? How do you decide what road to go down? We're going to call these questions you ask yourself Boolean expressions, named after a mathematician Boole, and a boolean expression is just a question that has a yes or no answer or a true or false answer or a 1 or zero answer. Just it's a binary state, yes or no typically. Otherwise we have this go back to go back to, which is what we're generally kind of call a loop which somehow induces cyclical behavior again and again and those functions and those conditionals, boolean expressions and loops and a few other concepts are pretty much what will underlie all of the code that we write, whether it is in scratch, C, or something else altogether. But we need to get to that point and in fact, let's go and. Infer what this program here does. At the end of the day, computers only understand zeros and ones, so I claim here is a program of zeros and ones. What does it do? Anyone want to guess? I mean, we could spend all day converting all of these zeros and ones to numbers, but they're not gonna be numbers if it's code. What do you think? That's amazing. It does in fact

print hello world. All right, so no one except like maybe you and me and a few others in the room should know, and that's probably guess admittedly or advancing on the slide, but why is that? Well, it turns out that not only do computers standardize information, data like numbers and letters and colors and other things, they also standardize instructions. And so if you've heard of companies like Intel or AMD or Nvidia or others, among the things they do is they decide as a company what pattern of zeros and ones shall represent what. and it's very low level functionality. Those companies and others decide that some pattern of zeros and ones means add two numbers together or subtract or multiply. Another pattern might mean load information from the computer's hard drive into memory. Another might mean store it somewhere else. Another might mean print something out to the screen. So nested somewhere in here, and admittedly I have no idea which pattern I'll find because it's not interesting enough to go figure it out at this level, says print. And somewhere in there, like this gentleman proposed, I bet we could find the representation of H, which was 72 and E and L and L and O and everything that composes hello world because as it turns out in programming circles, the very first program that students typically write is that of hello world. Now this one here is written in a much more intelligible way, even if you're not a programmer, odds are if I asked you what does this program do, you would have said. Oh hello world, even though there's a lot of clutter here, like no idea what this is until next week in main void that looks cryptic. There's these weird curly braces which we rarely use in the real world, but at least I understand a few words like hello and world, and this is kind of familiar print F, but it's not print, but it's probably the same thing. So here too is an example of this hierarchy back in the day, in the earliest days of computers, humans were writing code by representing. Zeros and ones. If you've ever heard your parents talk about punch cards or the like, you're effectively representing patterns that tell the computer what to do or what to represent like literally holes and paper. Well, pretty quickly early on this got really tedious, only writing code at such a low level, so someone decided, you know what, I'm going to put in the effort. I'm gonna figure out what patterns of zeros and ones I can put together so as to be able to convert something more user friendly. To those zeros and ones and as a teaser for next week, that person invented the first compiler. A compiler is just a program that translates one language to another, and more modernly, this is a language called C which we'll spend a few weeks on together because it's so fundamental to how the computer works. Even this is going to get tedious by like week 6 of the class, and this is gonna get stupid, this is gonna get annoying, this is gonna get cryptic. We're just gonna write print hello on the screen in order to use a different language called Python. Why? Because someone wrote in C. A program that can convert Python, this is a white lie to C, which can then be converted to zeros and ones and so forth. So in computing there's this principle of abstraction where we start with the basics and thank God we can all trust that someone else solved these really hard problems way long ago. Then they wrote programs to make it easier. We wrote programs to make it easier. You can now write code like I did with the chatbot to make things even easier. Why? Because OpenAI and other companies have abstracted away a lot of the lower level. Implementation details and that's where I think this stuff gets really exciting. We can stand on the shoulders of others so long as we know how to use and assemble these kinds of building blocks. And speaking of building blocks, let's start here. Now odds are some of you might have started here in like grade school playing with scratch, and it's great for like after school programs, learning how to program, and you probably used it this language to make games and graphics and just maybe playful. Art or

the like, but in Scratch, which is a graphical programming language designed about 20 years ago from our friends down the road at MIT's Media Lab, it represents pretty much everything we're going to be doing fundamentally over the next several weeks in more modern languages like C and Python, more textual languages if you will. I bet I could ask the group here what does this program do when you click a green flag. Well, it says hello world on the screen, because with Scratch, you have the ability to express yourself with functions and loops and conditionals and all of this, but by using drag and drop puzzle pieces. So what we're about to do is this. We're going to go on my screen to scratch.mit.edu. It's a browser-based programming environment, and we're only going to spend one week, really a few days in CS 50 on this language, but the overarching goal is to one, make sure everyone's comfortable applying some of these building blocks and actually. Developing something that's interesting and visual and audio as well, but to also give us some visuals that we can rely on and fall back on when all of those curly braces and parentheses and sort of stupid syntax comes back that's necessary in many languages but can very quickly become a distraction early on from the interesting and useful ideas. So what we're about to see is this in a browser. This is the scratch programming environment and there's a few different parts of this world. This is the blocks palette, so to speak. Uh, that is to say there's a bunch of puzzle pieces or building blocks that represent functions and conditionals and and uh loops and other such constructs. There's going to be the programming area here where you can actually write your code by dragging and dropping these puzzle pieces. There's a whole. whole world of sprites here by default scratch is and is a cat by design, but you can make scratch look like a dog, a bird, a garbage can, or anything else, as we'll soon see. And then this is the world in which Scratch itself lives. So Scratch can go up, down, left, right, and generally be animated within that world. For the curious, kind of like high school geometry class, there's sort of this XY plane here, so 00 would be in the middle, 0 180 is here, 0. 180 is here. Uh, -240, 0 is here, and 240 0 is here. Generally you don't need to worry about the numbers, but they exist so that when you say up or down, you can actually tell the program go up one pixel or 10 pixels or 100 pixels so that you have some definition of what this world actually is. All right, so let's actually put this to the test. Let me go ahead here and flip over to in just a moment. The actual Scratch website whereby I'm gonna have on my screen in just a moment, that same user interface, once I've logged in. That via which I can actually write some code of my own. Let me go ahead and zoom in on the screen a little bit here and let's make the simplest of these programs first. Maybe a program that simply says he world. Now at a glance, it's kind of overwhelming how many puzzle pieces there are, and honestly, even over 20 years I've never used them all, and MIT occasionally adds to it. But the point is that they're color coded to resemble the type of functionality that they offer and also it's meant to be the sort of thing where you can. Kind of scroll through and get a visual sense of like what you could do and then figure out how you might assemble these puzzle pieces together. So I'm going to go under this yellow or orangeish category here to begin with. So there exists in the world of Scratch, not quite the same jargon that I'm using now functions and conditionals and loops that's more of the programmer's way. This is more of the child friendly way, but it's really the same idea under events. You have puzzle pieces that represent things that can happen while the world is running. So for instance, the first one here is sort of the canonical when the green flag is clicked. Why is that relevant? Well, in the two dimensional world that Scratch lives in, there's a stop sign which means stop, and there's a green flag which means go. So I can therefore drag one of

these puzzle pieces over here so that when I click that green flag, the cat will in fact do something for me. Doesn't really matter where I drop it so long as it's somewhere in the middle here. I'm gonna go ahead and let go. Now I want the look of the cat to change. I want to see like a cartoon speech bubble come out for now, so I'm going to go under looks here and there's a bunch of different ways to say things and think things. I'm gonna Keep it simple and just drag this one here. And now notice when I get close enough to that first puzzle piece, they're sort of magnetic and they want to snap together so I can just let go and boom because they're a similar shape, they will lock together automatically and notice too if I zoom in here. The white oval, which by default says hello, is actually editable by me because it turns out that some functions can take arguments or more generally inputs that influence their behavior. So if I kind of click or double click on this, I can change it to the more canonical Hello world or hello David or hello, whatever I want the message to be. I'm going to go ahead and zoom out and now over here at top right notice that I can very simply click the green flag and I'll have written. My first program in Scratch. I clicked the green flag. It said go and now notice it's sort of stuck on that because I never said stop saying go, but that's where I can click the red stop sign and sort of get the cat back to where I want it. So think about for just a moment what it is we just did. So at the one hand we have a very obvious puzzle piece that says say and it said something, but it really is a function, and that function does take an input represented by the white oval here. Otherwise known as an argument or a parameter, but what this really is is just an input to the function. And so we can map even this simple, simple scratch program onto our model of problem solving before with an addition of what we'll call moving forward a side effect. A side effect in a computer program is often something that happens visually on the screen or maybe audibly out of a speaker. It's something that just kind of happens as a result of you using a function like a speech bubble appearing on the screen. So here more generally is what we claimed it represents the solving of a problem and let's just consider what the input is. The input to this problem, say something on the screen, is this white oval here that I typed in Hello world, the algorithm, the step by step instruction. are not something really I wrote like our friends at MIT implemented that purple say block so someone there knows how to get the cat to say something out of its comical mouth. So the algorithm implemented in code is really equivalent to the say function. So a function is just a piece of functionality implemented in code. Which in turn implements an algorithm. So algorithm is sort of the concept and the function is actually the incarnation of it in code. What's the output? Well, hopefully it's the side effect, seeing the speech bubble come out of the cat's mouth like this. All right, so that's one such program, but it's always gonna play and look the same. What if I actually want to prompt the human for their actual name? Well, let me go back to the puzzle pieces here. Let me go ahead and throw this whole thing away, and if you want to delete blocks, you can either right click or control click and choose from a menu, or you can just drag them there and sort of let go, and they'll disappear. I'm going to go back in and get another, another event block, even though I could have reused that same one. I'm going to go ahead and go under sensing now, and if I zoom in over here, you'll see a whole bunch of things like I can sense distance and colors, but more pragmatically I can use this function in blue. Ask something and then wait for the answer. And what's different about this puzzle piece is that it too is just a function it too takes an argument, but instead of having an immediate side effect like displaying something on the screen, it's essentially inside of the computer going to hand me back the response. It's going to return a value, so to speak, and

a return value is something that the code can see, but the human can't. A side effect is something the human sees, but a return value is something only the computer sees. It's like the computer is handing me back the user's input. So how does this work? Well, notice, and this is a bit strange, this isn't usually how variables work, but scratch 2 supports variables, and that was a word I used quickly at the very start when we were making the chatbot, a variable like in math, X, Y, or Z just stores some value, but it doesn't have to store a number in code it can store like a human name. So what's going to happen when I use this puzzle piece is that once the human types in their name and hits enter, MIT, or really scratch, is going to store the answer, the so-called return value, in a variable that's designed to be called answer. But as we'll see, you can make your own variables down the line if you want and call them anything you want. But let me go ahead and zoom out. Let me drag this over here. I'm going to use the default question What's your name? But I could certainly change the text there and let me go under looks again. Let me go ahead and grab the say block and let me go ahead and say just for consistency like hello, comma, OK, and now let me go under maybe sensing. I want to say how do I wanna say this answer. Well, notice this the shapes are important. This too is an oval, even though it's not white, but that's just because it's not editable. It's gonna be handed to me by the As function. Let me zoom out and grab a second say block like this, and notice it will magnetically clip together. I don't want to say hello again so I could delete that, but now it's still the same shape even though it's a little smaller. Let me go back to sensing and notice what can happen here. When you have values like words inside of a so-called variable, you can use those instead of manual input at your keyboard and notice if you want the magnetic slap into place, it'll grow to fit that variable because the shape is the same. And now let's do this. Let me click the green flag at right. I'm seeing quote unquote, what's your name, I'm getting a text box this time, like on a web page, for instance, let me type in my name and watch closely what comes out of the cat's mouth as soon as I click the checkmark or hit enter. Huh, OK, I got my name right, but let me do it once more. Let me stop and start. D A V I D. Enter No, it didn't work. Let me try one other, maybe it's my name. Let's try Kelly. Enter what's missing obviously. So the, the hello, there's a bug, a mistake in this program, but is there like what explains this even if you've never programmed before, intuitively what could explain why I'm not seeing hello? Exactly, it's on two different lines, so it's doing one after the other, so it is happening. It's just you and I as the slowest things in the room are just not seeing it in time because it's happening so darn fast because my computer's so, you know, so new and so fast, it's happening but way too quickly. So how can we solve this? Well, can we solve this in a few different ways, and this is where in Scratch, at least for problems at zero when wherein you'll have an opportunity to play around with it. I can scroll around here and OK, under control I see something like weight, so I can just kind of slow things down and now notice too, if you hover over the middle of two blocks, if it's the right shape, it'll just snap into the middle too or you can just so you know, kind of drag things away to magnetically separate them, but this might solve this, so let me hit stop and then start. DAVID enter. Hello David. All right, that was a little, let's do like maybe 2 seconds to see it again. Green flag, D A V I D E. hello David. All right, it's working better. It's sort of more correct because I'm seeing the hello and the David, but kind of stupid, right, to see one and then the other. Wouldn't it be nice to say it all in one breath, so to speak? Well, here's where we can maybe compose some ideas. So let me get rid of this weight and the additional block. Let's confine our. in just one block, but let me go down to operations where we haven't been before, and this is interesting. There's this bigger oval here

that says join two things like apple and banana, and those are just random placeholder words that you can override with anything you want, but they're both ovals and white, which means I can edit them. So let me go ahead and do this. Let me drag this on top of the say block, and this is just gonna therefore uh override the hello I put there. Now I don't wanna say apple or banana, but I do want to say hello, comma. And I then want to say my name. OK, so now I can go back to sensing, go back to answer, drag and drop this here. That'll snap into place and let me zoom in. Now what I've done is take a function and on top of it I nested another function, the join function that takes two arguments or inputs and presumably joins them together as per its name. So let's see what this does for us. Let me click stop and start. I'll type in David, Enter. And it's so close. Now this is just kind of an aesthetic bug. What have I done wrong here? There's no space, so it looks a little wrong, but that's an easy fix. I just need to literally go into the hello block after the comma, hit the space bar so that now when I stop and start again and type in David, now I see something that's closer to the grammar we might typically expect syntactically here. All right, so let's model this after what we just saw earlier. We've now introduced a so-called return value. And this return value is something we can then use in the way we want. It's not happening immediately like the speech bubble. It's clearly being passed to me in some way that I can use to plug in somewhere else, like into that join block. So if we consider the role of these variables playing, let's consider the picture now as follows. If the input now to the first function, the ask block, is what's your name, quote unquote, that's indeed being fed into the ask. Block and the result this time is not a speech bubble. It's not some immediate visual side effect. It is the answer itself stored in a so-called variable as represented by this blue oval. Meanwhile, what I want to do is combine that answer with some text I came up with in advance by kind of stacking these things together. Now visually in scratch, you're stacking them on top, but it's really. You're passing one into the other into the other because much like math when you have the parentheses and you're supposed to do what's inside the parentheses and then work your way out, same idea here. You want to join hello and answer together and whatever that output is, that then becomes the input to the say block which like in math is outside of the joined block itself. So pictorially it might now look like this. There's two inputs to this story. Hello, space and the answer variable. The puzzle piece in question is joined. Its goal in life had better be to give me the full phrase that I want, hello David. Let's shift everything over now because that output is about to become the input to the say block, which itself will now have the so-called side effect. And so this too is what programming and in turn what computer science is about is. Posing with the solutions to smaller problems, solutions to bigger problems using those component pieces, and that's what each of these puzzle pieces represents is a smaller problem that someone else or maybe even you, has already solved. Now we can kind of spice things up here if I go back to scratch's interface, we don't have to use just the puzzle piece here. I can do something like this. Let me go ahead and drag these apart and get rid of the same block down here. Just for fun, there's all these extensions that you can add over the internet to your own scratch environment, and if I go to like text to speech down here, I can for instance do a speak block instead of a say block color here in green. I can now reconnect the join block in here and if we could raise the volume just a little bit, let me stop the old version, start the new version, type in my name and hear what Scratch actually sounds like. Hello David. OK, not very catlike, but we can kind of waste some time on this by like dragging the set voice to box, and I can put this anywhere I want above the speak block. So I'm just going to put it

here, even though I've already asked a question. Maybe kitten sounds appropriate. Let's try again. D A V I D, meow meow. OK, and then let's see, uh, giant, a little creepier, here we go, DAVID and lastly. Hello, David. All right, little ransom like instead. All right, so that's just some additional puzzle pieces, but really just the same idea, but I like that we've introduced some sounds. So let's do this. Let me go ahead and throw away a lot of those puzzle pieces, leave ourselves with just the one green flag clicked. And play around with some other building blocks that we've seen already thus far. Let me go ahead, for instance, under sound, and let's make the cow actually meow. So it turns out scratch, being a cat by default comes with some sounds by default like meowing. So if we go ahead and click the green flag after programming this program, let's hear what he sounds like now. OK, kind of cute, and if you want it scratch to meow twice, you can just play the game again. And a third time, Alright, but that's gonna get a little tedious, as cute as it is, so I can solve that. Let's just grab 3 of the puzzle pieces and just drag them together and let them connect, and now click the green flag. All right, doesn't, it gets less cute quickly, but maybe we can slow it down so that the cat doesn't sound so, so hungry maybe. Let me go under, uh, let's see, under control, let's grab one of those, wait one second, and maybe plop a couple of these in the middle here, that might help things, and now click the green flag. OK, still a little hungry, but let's see if we change it to 2, and then I change it to 2 down here in both places, let's play it again. OK, cuter, maybe. But now I'm venturing into badly programmed territory. This is correct. If my goal is to get the cat to meow 3 times, pausing in, sorry, 3 times, pausing in between, what is bad about this code, even if you've never programmed before though? Yeah, in the middle. Yeah, I literally had to repeat myself 3 times, essentially copy pasting, and frankly I could have been really lazy and I could right click or control click, and I could have chosen duplicate, but generally when you copy paste code or when you duplicate puzzle pieces, probably doing something wrong. Why? It's solving the problem correctly, but it's not well designed, even if for only because when I changed the number of seconds, now I had to change it in two places. So I had one. Initially, then I had to change it to 2, and if you just imagine in your mind's eye having not like 6 puzzle pieces, but 60 or 600 or 6000, you're going to screw up eventually if it's on you to remember to change something here and here and here and here, like you're going to mess up. It's better to keep things simple and ideally centralized by factoring out common functionality and clearly playing sound and waiting is something I'm doing at least twice, if not a third time here as well. So how can we do this better? Well, remember this thing. Loops. Maybe we can just do something a little more cyclically. So I tell the computer to do something once, but I tell it how many times to do that altogether. So notice here by coincidence under control I have a repeat block which doesn't say loop, but that's certainly the right semantics. Let me go ahead and drag the repeat block in and I'll change the 10 to 3 just for consistency here. I'm going to go back to sound. I'm going to go ahead and play sound. Meow until done just as before and just so it's not meowing too fast under control, I'm going to grab a wait one second and keep it inside the loop and notice that the loop here is sort of hugging these puzzle pieces by growing to fill however many pieces I actually cram in there. So now if I click play, the effect is going to be the same, but it's arguably not only correct but also well. Designed, because now if I wanna change the weight, change it in one place, if I want to change the total number of times, change it in one place, so I've modularized the code and made it better designed in this case, but now this is silly because, Even though I want the cat to meow, it feels like any program in which I want this cat to meow, I

have to make these same puzzle pieces and connect them together. Wouldn't it be nice to invent the notion of meowing once and then actually have a puzzle piece called meow? So when I want the cat to meow, it will just meow. Well, I can do that too. Let me scroll down to my blocks. Here in pink I'm going to click Make a block and I'm going to literally make a new puzzle piece that MIT didn't think of called meow and I'm going to go ahead and click OK. Now I have in my code area here a define block, which literally means define meow as follows. So how am I going to do this? Well, I'm going to propose that me. Ming just means to play the sound, meow until done, and then wait 1 2nd. And notice now I have nothing inside my actual program which begins when I click the green flag, but notice at top left because I made a block called meow, I now have access to one that I can drag and drop. So now I can drag meow into. This loop and per my comment about abstracting the lower level implementation details away, I'm going to sort of unnecessarily dramatically just move that out of the way. It still exists. I didn't delete it, but now out of sight, out of mind. Now if you agree with me that meow means for the cat to make a sound, we've abstracted away what it means. Mechanically for the cow to say that sound and so we now have our own puzzle piece that I can just now use forever because I invented the meow block already. Now I can do one better than this. It would be nice if I could just tell the meow block how many times I wanted to meow because then I don't need to waste time using loops either myself. So let me do this. Let me zoom out. And let me go back to my defined block. Let me right click or control click and just edit it, or I could delete it and start over, but I'll just edit it and specifically let me say, you know what, let's add an input, otherwise known as an argument to this meow block, and we'll call it maybe N from the number of times I want it to meow. And just to be super clear, I'm going to add a label which has no functional impact, but it just helps me remember what this does. So I'm going to say meow and time. So that when I see the puzzle piece, I know what the N actually represents. If I now click OK, my puzzle piece looks a little different at top left. Now it has the white oval into which I can type or drag input. Notice down here in the defined block, I now see that same input called N. So what I can do now is this. Let me go under control, drag the repeat block here, and I have to do a little switcheroo. Let me disconnect this. Plug it inside of the repeat block, reconnect all of this, and I don't want 10, and heck, I don't even want 3 down here anymore. I can drag this input because it's the right shape and now declare that meowing end times means to repeat the following end times play sound meow until done. Wait 1 2nd and keep doing that and total times. If I now zoom out and scroll up. Notice that my usage of this puzzle piece has changed such that I don't actually need the repeat block anymore. I can disconnect this and heck, I can actually right click and control click and delete it. Just use this under the green flag, change this to a 3, and now I have the essence of this meowing program. The implementation details are out of sight, out of mind. Once they're correct, I don't need to worry about them again, and this is exactly. Scratch itself works. I have no idea how MIT implemented the weight block or the repeat block. Heck, there's a forever block, and there's a few others, but I don't need to know or care because they've implemented those building blocks that I can then implement myself. I don't necessarily know how to build a whole chatbot, but on top of OpenAI's API, this web-based service, I can implement my own chatbot because they've done the heavy lift of actually. that for me. Well, let's do just a few more examples here. Let's bring the cat all the more to life. Let me throw away the meowing. Let me open up under when green flag clicked. How about that forever block that we just glimpsed? Let me go ahead and now add to the mix what we called earlier

conditionals which allow us to ask questions and decide whether or not we should do something. So under this, let me go ahead and under Forever, say if the following. Is true. Well, what boolean expression do I want to ask? Well, let's implement, how about this program and we'll figure out if it works. Uh, under sensing, I'm gonna grab this, uh, a very angled puzzle piece called Touching mouse pointer that is the cursor, and only if that question has a yes answer, do I want to play the sound meow until done. So let me zoom in here and in English. What is this going to implement really? Just describe what this program does less arcanelly as the code itself, yeah. Yeah, if you move the mouse over the cat, it will make noise, so it's kind of like implementing petting a cat, if you will. So let me zoom out, click the green flag, and notice nothing's happening yet, but notice my puzzle pieces are highlighted in yellow because it is in fact still running because it's doing something forever and it's constantly checking if I'm touching the mouse pointer, and if so. It's like I just pet the cat. Now it stopped until I moved the cursor again. Now it stopped. If I leave it there, it's gonna keep meowing because it's gonna be stuck in this loop forever, but it's correct insofar as I'm petting the cat. Let me do this though. Let me make a mistake this time. Let me forget about the forever and just do this, and you might think this is correct. Let me click the green flag. Now let me pet the cat. And like nothing's actually working here. Why though logically. Yeah. Yeah, the program's so darn fast it already ran through the sequence and at the moment in time when I clicked the rear flag, no, I was not touching the mouse pointer and so it was too late by the time I actually moved the cursor there. But by using the forever block, which I did correctly the first time, this ensures that Scratch is constantly checking the answer to that question. So if and when I do pet the cat, it will actually Detect as much. All right, about a few final examples before you're on your way building some of your own first programs with these building blocks. Let me go ahead and open up a program that I wrote in advance, in fact, about 20 years ago, whereby, let me pull this up. Whereby we have in this example a program I wrote called Oscar time, and this was the result of our first assignment in this class whereby when MIT was implementing Scratch for the very first time, we needed to implement our very own Scratch program as well. I'm gonna go ahead and full screen it here. The goal is to drag as much falling trash as you can to Oscar's trash can before his song ends, for which one volunteer would be handy here. OK, so. Hands go up quickly in blue. Yeah, come on up. All right, so you're playing for a stress ball here if we will. At one at some point I'm going to talk over what you're actually playing just so that we can point out what it is we're trying to glean from this program, and I'll stipulate this probably took me like 8, 12 hours, and as you'll soon see, the song starts to drive you nuts after a while because I was trying to synchronize everything in the game to a childhood song with which you might be familiar. Let me go ahead and say hello if you'd like to introduce yourself. Oh, hello. So I'm Han. And uh I'm a first year student. I'm pretty excited for this class. welcome. Well, here is Oscar time. If you want to go ahead and take control of the keyboard, all you'll need to do is drag and drop trash that falls from the sky into the trash can. Oh Anything score is 1. Nice score 2. All right, let's see what happens. You'll see a sneaker was perfectly timed. Thank you very much with the song after figuring out exactly how many seconds it had to wait. Now Han can drag not only the piece of trash, but the shoe as well. And it's around this point in the game where the novelty starts to wear off because it's like 3 more minutes of this game where more and more stuff starts to fall from the sky. So as Han, as you continue to play, I'm going to cut over. Here you keep playing. Let's consider how I implemented this whereby we'll start at the beginning. The very first thing I

did when implementing Oscar time honestly was the easy part. Like I found a lamppost that looked a little something like this, and I made the so-called costume for the whole stage, and that was it. The game didn't do anything. You couldn't play anything. You click the green flag, nothing happened. But then I figured out how to turn the scratch cat, otherwise known more generally as a sprite, into a trap. instead and so the trash can meanwhile is clearly animated because I realized that oh, I can give sprites like the cat different costumes so I can make the cat not only look like a trash can, but if I want its lid to go up, well, that's just another costume. And if I want to see Oscar popping out, that's just a third costume. And so I made my own simplistic animation and you can kind of see it. It's very jittery step by step by step by creating illusion of animation by really just having a few different images or costumes on Oscar. I hope you appreciate how much effort went involved into timing each of these pieces of trash with the specific mention of that type of piece of trash in the music. OK, 20 years later, still clinging. So you're doing amazing, by the way. How do we get the trash to fall in the first place? Well, at the very beginning of the game, the trash just started falling from some random location. What does it mean for trash to fall from the sky? Oh, big climax here. So you've got a lot of trash on the ground you should pick up. There we go, and your final score is. A big round of applause if we go for hot. OK. Thank you. So just to be clear, now let's decompose this fairly involved program that took me a lot of hours to make into its component parts. So this is just a sprite, and I figured out eventually how to change its costume, change its costume, change its costume to simulate some kind of animation. And I also realized that oh, I don't need to just have one Sprite or one cat or trash can. You can create a 2nd Sprite, a 3rd Sprite, and many more. So I just told the Sprite to go to a random location at Y equals 180 and X equals something. I think I restricted X to be in this region, which is why the trash never falls from over here. I just a little bit of math based on that Cartesian plane that we saw a slide of earlier. And then I probably had a loop that told the trash to move a pixel, move a pixel, move a pixel down, down, down, down until it eventually hits the bottom and therefore just stops. So we can actually see this step by step, and this is representative of how even for something like your first problem set in CS 50 and with Scratch specifically, you might build some of the same. So I'm gonna go back into. CS 50 Studio for today, which is linked on the course's website which has a few different versions of this and other programs called Oscar 0 through Oscar 4, where 0 is the simplest and truly I meant it when I look inside this program to see my code, like this was it. There was no code because all I did was put the Sprite on the screen and change it from a cat to a trash can and I Added a cost uh costume for the stage, so to speak, so that the lamp post would be fixated there. If I then go to the next version of code, version one, so to speak, then I had code that did this. Now notice there's a few things going on here. At bottom left, you'll see, of course, the trash can, and then at top right the trash. Here are the corresponding sprites down here. So When Oscar is clicked on here, the trash can, you see the code I wrote, the puzzle pieces I dragged for Oscar, and in a moment when we click on trash, you'll see the code I wrote or the puzzle pieces I wrote are dragged and dropped for the trash piece specifically. So what does Oscar do? Well, I first switch his costume to Oscar one, which I assume is this the closed trash can. Then forever, Oscar does the following. If Oscar's touching the mouse pointer, then change the costume to Oscar 2. Otherwise, that is, if not touching the mouse pointer, change the costume to Oscar one. Well, what's the implication? Any time I move the cursor over the trash can, the lid just pops up, which was exactly the animation I wanted to achieve. Meanwhile, if we

do this and click the green flag, you can see that in action, even for this simple. If I move the cursor over Oscar, we have the beginnings of a game, even though there's no score, there's no music or anything else, but I've solved one of my problems. Meanwhile, if I click on the trash piece here and then you'll see no code has been written for it yet. So we move on to Oscar version 2 and see inside it. And Oscar version 2, when I click on trash, ah, now there's some juicy stuff happening here and in fact this trash sprite has two programs or scripts associated with it, and that's fine. Each of them starts with when green flag clicked, which means the piece of trash will do two things at once, essentially in parallel. The first thing it will do is we'll set drag mode to draggable, and that's just a scratch thing that lets you actually move the sprites by clicking on them, making them. Dragable. Then it goes to a random X location between 0 and 240. So yeah, that must be what I did from the middle all the way to the right, and I said why always to 180, which is why the trash always comes from the sky from the very top. Then I said forever change your Y by -1, and here's where it's useful to know what 180 is, 240 is, and so forth, because if I want the trash to go down, so to speak, that's. Changing its Y by a pixel by pixel, by a pixel, and thankfully MIT implemented it such that if the trash tries to go off the screen, it will just stop automatically even if it's inside of a forever block lest you lose control over the sprites altogether. But in parallel what's happening is this also when the green flag is clicked, uh, the trash piece is doing this too forever if touching Oscar, what's it doing in blue here? Sort of teleporting away now to your eye, hopefully it looks like it's going into the trash can, but what does that mean to go into the trash can? Well, I just put it back into the sky as though a new piece of trash is falling. So even though you saw one piece of trash, 234, and so forth, it's the same sprite just acting that out again and again. So here if I click play. On this program you'll see that it starts falling one pixel at a time because it's draggable. I can sort of pull it away and move it over to the trash can like that, and as soon as I do, it seems to go in, but really it just teleported to a different X location still at Y equals 180. Again, it's not much of a game yet. There's no score, there's no music or anything, but let's go to Oscar 3 now. And in Oscar 3, if we scroll over to the trash, even more is happening here and so. As far as I realized, you know what, there was kind of inefficiency before. Previously I had these two programs or scripts synonym whereby they both went to the top by going to 0 to 240 for X and then 180 for Y. And if you noticed, I used that here and I used that down here in both programs. Now that was kind of stupid because I literally copied and pasted the same code. So if I ever want to change that design, I have to change it in two places and I already proposed that we frown upon that. So what did I do in this version? I just created my own block and I decided to call my own function go to top. What does it mean to go to the top, pick a random X between those values, and fixate on Y equals 180 initially. Now in both of those programs, which are otherwise identical, I just say what I mean. Go to top, go to top, and if I really wanted to, I could drag this out of the way and never think about it again because now that function. exists, so correct but arguably better designed. I've now factored out commonality so as to use and reuse my code as well. So let's go up to Oscar version 4 now, and in Oscar time version 4, the trashcan does a little something more whereby what have I added to this mix even though we haven't dragged this puzzle piece together before. Yeah, what's new? Yeah, so it turns out on the left here there's a variables category which goes beyond the answer variable that we just automatically get from the ask block. You can create your own variables X, Y, Z, but in computer programming it's best to name things, not silly simple words like X, Y, and Z, but full-fledged words that say what they

are like score. So I'm setting a score variable to 0, and then any time the trash is touching Oscar before it teleports away to the top, I change the score by 1. That is increment the score by 1. And what Scratch does automatically for me is it puts a little billboard up here showing me the current score. So if I now play this game once more, the score is going to start at 0. But if I drag this trash over here and even let it fall in as soon as it touches, the score goes to 1. And now if I click and drag again, the score is going to as soon as it touches Oscar, going to go to 2, and so forth. And you saw in the final flurs with Hunt playing that once you add The sound and other pieces of trash, which are just really other sprites, and I just had to wait like a minute, wait 2 minutes, so that the trumpet would fall at the right time. I've broken down a fairly involved program into these basic building blocks. And when you two write your own program, that's exactly how you should approach it, even if you have these grand aspirations to do this or that, start by the simple problems and figure out what bites can I, uh, bite off in order to make progress, baby steps, if you will, to the final. Solution. Well, let's look at one other set of examples before we have one final volunteer to come up and as you'll soon see, it's tradition in CS 50 to end the first class with cake. So in a moment, cake will be served out on the transept, and please feel free to come up and say hi and ask questions if you'd like to. Let me go ahead and open up though a series of building blocks here via which we can make so-called IV's hardest game, which is one implemented by one of your predecessors, a former classmate from CS 50. So here we have a whole bunch of puzzle pieces. Written by your classmates, but let me go ahead and zoom in on this screen. You'll see that this Harvard Crest is my sprite, so it's not a cat, it's not a trash can. It's a Harvard Crest, and it exists in a very simple two-dimensional world with two walls next to it. If I click on the green flag, notice that with my hands here, I can go up, I can go down, I can go left, and I can go right, but if I try going too far right, I get stuck on the wall. If I go too far left, I get stuck on the wall. Well, it's the sort of the beginning. Of any animation or game, but how do I do this? Well, let me go up here and propose that the first thing the Harvard Sprite is doing is it's going to the middle, 0.0, and it's then forever listening for the keyboard and feeling for walls. Now those are functions I implemented myself to kind of describe what I wanted the program to do. And let's do the shorter one first. What does it mean to feel for the walls? Just to ask the question, if you're touching the left wall, change your X by 1. If you're touching the right wall, change your X by -1. Why have I defined touching walls in this weirdly mathematical way? Yeah. Sure, yeah. otherwise you're like net. Exactly, because if I've gone so far right that I'm touching the right wall, well, I'm already kind of on top of the wall a little bit. So I effectively want the sprite to bounce off of it, and the easiest way to do that is just to say back up one pixel as though you can't go any further, and same for the left wall. Meanwhile, let me scroll over to the second script or program that's running in parallel. It's a little longer, but it's not more. Complicated. What does it mean to listen for keyboard? Well, just check if the key up arrow is pressed, change Y by 1. ergo go up, else if the key down arrow is pressed, then change Y by -1, key right arrow is pressed, change X by 1, and so forth. So again, this is where the math and the numbers are useful because it gives you a world in which to live up, down, left, right. Deconstructed into some simple arithmetic values. All right, so the net result is that we have a crest living in this world. Well, let's add a bit of competition here and then the second version of this game. Let me go ahead and full screen it again, click play, and now we'll see sort of an enemy bouncing back and forth autonomously. So there's no one playing except me. I'm controlling Harvard, Yale is bouncing on its own, and nothing bad's. Gonna happen if it hits me,

but it does seem to be autonomous. So how is this working? Well, if it's doing this forever, there's probably a forever loop involved. So let's see inside here. Let's click not on Harvard but on the Yale Sprite, and sure enough, if we focus on this for a moment, we'll see that the first thing Yale does is go to 0.0. It points in direction 90 degrees, which just gives you a sense of whether you're facing left or right or wherever. Then it forever does the following. If it's touching the left wall or touching the right wall, I was a little clever this time, if I may. I just kind of turn around 180 degrees, which effectively bounces me back in the opposite direction. Otherwise I go ahead and no matter what, just move one step. And this is why Yale is always moving back and forth. So a quick question, if I wanted to speed up Yale and make this beginning of a game harder, what would I do? Yeah. Yeah, so let's have it move like 10 steps at a time, right? This looks like a much harder game if you will, like level 10 now because it's just moving so much faster. All right, well, let's try a third version of this that adds another ingredient. Let me full screen this and click play, and now you'll see the even smarter MIT homing in on me by following my actual movements. So this is sort of like boss level material now. And it's just gonna follow me. So how is this working? Well, it's kind of a common game paradigm, but what does this mean? Well, let's see inside here, let's click on MIT Sprite, it's pretty darn easy. Go to some random position just to make it a little interesting lest MIT always start in the center and then forever point towards the Harvard logo outline, which is the name the former student gave to the costume that the Sprite is wearing that looks like a Harvard crest, and then move one step. So a core layer of the previous question how do we make the game harder and MIT even faster? Well, we can change this to be like 10 steps, and now you'll see MIT is a little twitchy because. This is kind of a visual bug. Let me make a full screen. Why is this visual glitch happening? It's literally doing what I told it to do, just looks stupid, yeah. say like again. Yeah, it's moving so fast that it's sort of going 10 pixels this way, but then I kind of, it kind of overshoot me, so then it's doubling back to follow me again and it's doubling back this way. And because these are such big footsteps, if you will, it just has this visual effect of twitching back and forth. So we might have to throttle that back a bit and make it 5 or 2 or 3 instead of 10 because that's clearly not desirable gaming behavior here. All right, well, let's go ahead and do this. Let's put them all together just as your former classmate did when submitting this actual homework. Uh, the game will conclude hopefully in an amazing climax where you've won the game, so we need someone, ideally with really good hand-eye coordination to play this final game here. Yeah, your hand went up first, I think. OK, come on up, big round of applause because it's a lot of pressure to end. All right, so if you win the game, cake will be served. If you don't win the game, there will be no cake. OK, but introduce yourself in the meantime. Hi, I'm Jenny Pan, freshman at Hollis, and I'm actually a CS major or concentration. Nice to meet you. Head to the keyboard here. This now is the combination of all of those building blocks and even more, AKA IB's hardest game, you will be in control just as I would of the Harvard Crest, and the goal is to make it to the exit, which is this gentleman on the right here, and you'll see there's multiple levels where each each level gets a little harder. All right. Here we go. I can't touch this. I can't touch this. Very good. I can't touch this. Nicely done. You can't touch this. So the loop has been added for Yale. Good. 2 Yales now. Alright, 3 yals now. Nice. You can't touch this. Here comes MIT. Very good, yeah. Two MITs. And get Nice, yes. That I Yes. Now Princeton. Another Princeton. 2nd to last level. I OK Have a Let's keep going, keep going. Yeah All right, this is CS 15, cake is now served.

Week 1

All right, this is CS 50, and this is week one, our second week together, and you'll recall that last week, week zero, we focused on scratch ultimately this graphical programming language by which you can drag and drop puzzle pieces that interlock together only if it makes logical sense to do so. And many of you had actually probably played with that in like middle school or even prior at some point, but for our purposes, the goals of Scratch were to give us sort of a mental model for some fundamental constructs that we're going to see again and again today in C in a few weeks in Python. and even thereafter, and those include things like functions and return variables and arguments and variables and loops and conditionals and more. And so even if today feels like a bit of a fire hose such as that pictured here, appreciate that a lot of today's ideas are exactly the same as last week's ideas. It's just that the syntax is going to change. It's going to look a little different. It's going to look a little scarier. It's going to be harder to sort of memorize except with practice will come that muscle memory, but the ideas ultimately are going to be the same. And indeed this is, if unfamiliar, MIT down the road has a tradition of hacks whereby students once a year do something fairly crazy and at this point they happen to connect an actual working uh drinking fountain to an actual fire hydrant, and the sign there, very pixelated, says Getting an education from MIT is like trying to drink from a fire hose. And that's indeed how computer science, how programming, how CS 50 will sometimes feel, but realize that what's going to be ultimately most important is not where you. Uh, feel you are day after day, but where 3 months from now you feel that you are relative to last week alone, so-called week zero. So let's look back at what week zero looked like. It looked a little something like this the simplest of programs by which we get get that cat to say Hello world. Today that same code is going to start to look a little like this, which was a glimpse we gave you last week. But this time I've deliberately color coded it to try to send the message that we're in scratch. We had this yellowish puzzle piece that sort of kicked things off. That didn't really do anything itself, but it got the program started, whereas the real work was done in purple here. Same is going to be true today, whereby I'm going to wave my hands for a little bit of time at this yellowish code on the screen, but what's really going to have the most effect is this same purple line here and the white text within. And we'll break down what all of these lines mean over the next couple of weeks, but sometimes we'll wave our hand at details if we feel it's a little unnecessary at this point in the story. And in fact, let me get rid of the color coding for now and we'll see that this is the kind. Of code in a language called C we're going to start playing with and using today and for the next several weeks. And indeed it's representative of what we're going to generally call source code. So source code is what programmers write. It's what you write. It's what you wrote, albeit by dragging and dropping puzzle pieces. This week onward you're going to start using your keyboard all the more and you're going to write source code. So this is code that we humans can understand with some training and with some practice, but of course per last week what language do computers ultimately understand? Only so binary zeros and ones and so you and I, yes, can write code starting today in a form that looks a little something like this, which admittedly might look a little arcane and cryptic, but it's certainly better than a whole bunch of zeros and ones. But we're going to write in source code, but the machines that we write code for ultimately only understand these here zeros and ones, which may very well say hello world, but we're going to call this moving forward machine code. So machine code is what the the. understand only the zeros and ones. Source code is what you and I understand and actually

write, so it stands to reason that we're going to have to somehow translate one to the other from source code to machine code. And I alluded to this ever so briefly last week, but we're going to use the same mental model whereby the source code we write might be the input to some problem. The output we want therefrom is going to be the machine code. So what we're going to equip you with today inside of this proverbial black box is a special piece of software that takes source code as input, produces machine code as output, and that type of program is called a compiler, and there's bunches of different compilers in the world. We're going to have you use one of the most popular ones, but it's simply a piece of software that someone else wrote that converts one language to another. Source code, for instance, in a language called C to machine code, the zeros and ones that our Macs PCs, phones, and other devices actually understand. So where are we going to do this and how are we going to do this? So I promised last week that we'd introduce you to this year tool, which I used briefly at the very start of class to whip up that chatbot. We're going to use it though not for Python this week, but indeed for a different language C. And indeed this tool, Visual Studio Code or VS code for short, is super popular in industry. This is what real programmers, so to speak, are using all of the time nowadays. There's absolutely alternatives. If some of you have programmed before, you might have used or experienced different tools, but this is a very common tool that you'll see even after CS 50, and in fact it's something that ultimately you can install for free on your own Macs and PCs so that by the end of the course you're completely independent of CS 50 and any CS50 related tools, but what we have done for the very start of the class is essentially provided you with a cloud-based version of this tool. So all you need is a web browser on any Mac or PC or the like, so that everything's pre-installed for you, pre-configured for you, and you don't have to deal with the stupid technical support headaches at the start of the term because it should just work. But by the end of the term, once you're a little more comfortable with technology and with code in particular, you can absolutely offboard yourself from this tool, install it, download it on your own Mac and PC, and have pretty much the exact same environment completely under your control. So starting today, you're going to see an interface that looks quite like this quite often, and we use this same interface last week ever so briefly. Moving forward, here's where we're going to write code at top right is where one or more codes. Tabs are going to appear similar to any tabbed environment that you might use. Here, for instance, is just a screenshot of the first file we'll create today called hello.c. The reason it's called hello.c is because it's in a language called C, as we soon shall see, no pun intended. Meanwhile, the code here happens to be color coded, not quite in the same way as you saw before, because I manually made it look more like scratch blocks. But among the features that VS code and other programming environments provide. Something called syntax highlighting whereby you don't worry about or even think about these colors, but as you write out code in a recognized language, tools like VS code will just color code different parts of your code for you just to make different features jump out, and we'll see what those features are over the course of today. But you'll also spend a good amount of time, as I briefly did last week, down here in the bottom right of your screen, the so-called terminal window, which is going to be where you run commands for compiling code and write in code. And in fact, as we'll see today, you're going to start using your mouse and clicking a little bit less. You're going to start using your keyboard and typing a bit more. And ultimately, even though if that first that might feel like a step backwards to sort of not use something that's so user friendly, the reality is most every programmer tends to find themselves ultimately much

more productive, much more powerful, using the keyboard more often, more quickly than, say, a traditional mouse or trackpad would allow. Meanwhile, we'll see some somewhat familiar features. Here at left like this is where you'll see the files and folders that we will create over time. At far left here is going to be an activity bar, which is essentially a modern form of a menu via which you can open and close things and access other features. For my purposes, I'll generally hide this part here. I'll generally hide this part here so that when we're together we're focusing almost entirely on code and commands, but I'm just typing some quick keyboard shortcuts to simplify my own user interface in that way. So with all that said, just some terminology. So this The whole collective environment that I'm describing here is generally what's known as a graphical user interface. Why? Well, it's an interface for users that's graphical in nature with icons and buttons and the like. Shorthand notation for this is GUIY GUI for short. But within this graphical user interface, as promised, is going to be that terminal window at bottom right where I promised we would be typing most of our commands. And just to give you a bit more jargon in computing, that's generally known as a command line interface, or CLI. For short, whereby you're typing commands into that interface instead, and the world of computing software is essentially divided into GUIs and CLIs and sometimes a piece of software might have one of each as well. But without further ado, why don't we go ahead and focus entirely first on this here program, which I dare say is the simplest program you can write in a language like C and see how we can actually compile and run it together. So I'm going to go over to VS code here. I've hidden my File Explorer with all the icons and I've hidden my activity bar so that only do I have room for tabs of code and the command prompt at the bottom. I'm calling this a command prompt because it's at this dollar sign where I'm going to run some of my commands, and it's a dollar signed by convention. It has nothing to do with currency. It's just a computing convention. Some systems will use a carrot symbol. Some systems will use a greater than symbol rather, or something else, but it just means type your commands here. The first such. And I'm going to type is this code `hello.c` with a single space in between. I've not used any spaces in the name of the file. I've not capitalized any aspect of the file just because this is convention, unlike your Mac or PC, where you might be in the habit of naming files with spaces and capitalization. Generally you'll make your life simpler by just using lowercase and no spaces at all. As soon as I hit enter, what you'll see is that a brand new tab appears called `hello.c` with a cursor blinking on line 1 and this is essentially VS code waiting for me now to type the first line of my code. Notice though that the command is complete thereby whereby I have another cursor here which I'd give if I click in the terminal window and give foreground to it, my cursor might blink there instead. That just means I can type another command when I am ready. So let's go ahead and whip up this code, and I've done this many times, so I can type it fairly quickly, but in this tab I'm going to do `include <stdio.h>`, so to speak, in `main void`. Then inside of so-called curly braces indenting therein by 4 spaces, I'm going to say `printf` quote `hello world` backslash and close semicolon, and voila, I've written my first program in C. In a class like this, no need to write down each and every line of code that I write. In fact, on the course's website will be copies of everything that we've done as well as excerpts therefrom in the course's notes, but you're welcome but not expected to follow along in real time with what I am typing here. So that's it. Like I've written my very first program and see if I had done this on an actual Mac or PC without a command line interface, I might have a new icon on my desktop, so to speak, called Hello, and ideally I could double click on that or tap on it and run the

program, but because I'm in this specific programming environment that has a mix of a GUI and a CLI. I actually need to click down in my terminal window, and I need to now compile this program first because at this point in time it exists only as source code. So to do this, I'm going to compile my code by very aptly saying `make space hello`, and I'm pronouncing the space, but literally I hit the space bar. `Make space hello`, is it sort of implies some. will make a program called `hello`. Notice I have not said `hello.c` again because the compiler, let's call it `Make` for now, even though that's a bit of a white lie, is going to infer that if I want to make a program called `hello`, it's going to automatically look for a file called `hello.c` in this case, so a bit of magic. Enter. And remarkably, any time you don't see any output at a command like this, that's probably a good thing. Generally speaking, when you see output when compiling your code, you have done something wrong or in this case I might have done something wrong, but no output is good because what I can now do, and this is a bit cryptic, I can run this program not by double clicking or tapping anywhere, but by doing `slash hello` with no spaces. And this is a bit weird, but what the slash means is that after having just made a program called `hello`, that program's going to end up in my current folder. It's somewhere in the cloud, yes, more on that in a bit, but the program called `hello` is just somewhere in my current folder. When I say slash, that's like saying go into the current folder and run the program therein called `hello` specifically. Now, as I often do, I'll cross my fingers, hope that I didn't mess this up in any way, and I should see in a second. `Hello world` indeed printed onto the screen. And so just to recap those in commands, one, I ran `code hello.c`, which is a VS code specific thing. `Code` short for VS code, just creates a new file called `hello.c` and then I'm on my way with my own keyboard. `Make hello` compiles that source code into machine. `Code` thereby creating a new file called `hello` and to run that program `hello`, I type this strange command. `hello`. But this is a paradigm no matter what you call your programs, we're going to see again and again and again. So even if you've not done something quite like this, it will very quickly get familiar. Yes, questions. How, when you say `make hello`, how like, how does, how do you, how does the computer know like what part of the code to what part of the code is ascribed to `hello`? Good question. When I say `make hello`, how does the computer know what part of the code is ascribed to this program, `hello`? It literally is going to take the entire contents of `hello.c` and turn them somehow into a program. And does it have to be like `hello`? Does it have to be named `hello`? No, I could have called it `goodbye` or anything more `myfirstprogram doc`, anything at all, so long as I change these words here accordingly. But it has to be like it needs to be like from the same thing like it. Yes. Exactly. If you change the name there, you need to change your commands accordingly. Other questions on these here, steps. No. All right, so let's tease apart what it is we just did and like why this code works in the way that it does. Well, to recap, in Scratch, we had a program like this. When the green flag was clicked, we wanted to say `hello world` onto the screen. The code that corresponds to that is roughly here and indeed notice that the yellowish or orangeish code lines up with the when green flag clicked, the purple code here lines up with the same block, and the white code inside of here roughly corresponds to what was in the white oval that we kept using again and again last week. So let's do more of a one. One correspondence and these slides are deliberately designed to give you again that sort of mental model of taking same ideas from last week and just changing the syntax this week onwards. So when we have a function like this thing here, and recall that a function is just an action or verb, it sort of accomplishes a small piece of work in code in C specifically, you're going to type, of course not a purple puzzle piece, but you're

going to say the word print while more technically print F, where the F as we'll soon see, means format the printed. Output because this is more powerful than just printing some raw text alone, then you can have parentheses open and closed left and right and notice that it's no accident that MIT chose an oval for their input to functions because it roughly looks like the start of a parenthesis and parenthesis on left and right. Meanwhile, what goes inside of the parentheses in the corresponding C code, well, at the end of the day, minimally hello, comma world because that's literally what we want to print to the screen. But in C, unlike in Scratch. A bit of overhead, a bit of additional syntax that you just got to deal with to make clear to the computer what you want to print. In particular, you're going to have to surround everything you want to print with double quotes to make clear that hello is not some special function or variable or something else. It's Hello world is the English phrase that you want to print. So double quote here, double quote there means He's the beginning and the end of what I want to print. You're also curiously going to put a backslash n. In most cases, at the end of the word or words you want to print, we'll take that away in a moment and see what it does and then lastly and perhaps most annoyingly in programming circles, you have to finish your thought with a semicolon, much like in English you would finish most sentences with a period instead. And the thing the thing about programming is with C in particular, if you mess up almost any of these details I just rattled off, something's going to go wrong. And so you're in good company. The very first program you try to write or try to compile, odds are it might not work correctly because you'll develop over time the muscle memory for spotting all of these seemingly minor and actually minor details but that do matter to the computer. All right, so if you're familiar, of course, with the notation in like mathematics of functions like a function in code, it's really the same idea as a function in math whereby the function F takes some input, for instance X, and generally produces some output. So if you're coming more from that background, realize that what we're really doing here is roughly the same. But in code, recall that we can have different types of output. So if this is our grand mental model and say we've got a function. As inside of this black box that takes arguments, that is to say is it's inputs, it can sometimes have side effects and recall that side effects are often visual things that happen as a result. They just play on the screen, maybe it comes out of the speaker. It's some thing generally ephemeral that just happens, but it's. Not necessarily useful in the same way as another type of function that we'll return to in just a bit, but last week recall that we got the cat with the speech bubble to manifest on the screen and say hello world in that speech bubble when the input was hello world and the corresponding function was instead say. So let's see if we can't now tease apart what the code we wrote is actually doing for us bit by bit. So let me go back to VS code here and let me propose to break this in a little way. Let me delete the backslash N, if only because at first glance, who knows or cares what that's doing. Let's just get rid of it if we don't understand it. I could now go back down to my terminal window and I could do hello enter again, but there's seemingly no change. Which is good, doesn't seem like I broke it, but I've kind of misled you here why? Why did nothing seem to change? I didn't recompile it, so recall that the compiler converts source code to machine code, but I already did that a couple of minutes ago. If I've changed the source code, it stands to reason that I need to recompile the code to actually see the effects of that. So let me do that again. Make hello E. Nothing seems to have gone wrong, but let me now do hello Enter. And it's subtle now and in fact let me go ahead and zoom in, it's really just an aesthetic bug insofar as functionally the program is still technically printing Hello world, but what's seemingly

wrong or put another way, what did the backslash and apparently do? Yeah. Yeah, so it's somehow giving me a new line, and that's essentially what the backslash N denotes is give me a new line there. And why was I doing that? Well, really just for the aesthetics. Like if this dollar sign represents my prompt where I type commands, if anything, it just looks kind of stupid that I finished a program over here and then the prompt is on the same line. It just looks wrong even though you could sort of argue that was my intent, even though in this case it wasn't. So what would the alternative be? Well, what you're seeing here is what's actually generally known as an escape sequence which are sort of special sequences of symbols like backslash and N in this case that do a little something unusual. And here's just a non-exhaustive list of some you'll encounter in the real world and including in CS 50 backslash N moves you to a new line. Backslash R is a so-called carriage return if you've ever seen or used an old school typewriter, this refers to the process of bringing the typing head back to the left end, so it sort of moves the cursor horizontally as opposed to vertically. This one's interesting. backslash double quote. Why does there exist this pattern? Backslash double quote, yeah. Exactly. So recall that phrase we tried to type, uh, print out like hello, comma world. If for some reason you don't want to say hello world, but you wanted to say some or like sort of snarkily like hello world or something like that, well, you can't put a quote, a quote, a quote at a quote and expect the computer to know which quote corresponds to what. It's just arguably ambiguous. So if inside of double quotes you actually want to print actual double quotes. This is an escape sequence that tells the computer this is not some quote delineating where my thought begins and ends. This is literally a double quote and we'll see other situations in which a single quote or apostrophe is the same. We'll see crazy situations in which you want to print a backslash, but backslash already has some special meaning, so there's solutions to all of these problems. But let's not get too far into the weeds here. But let me go back to the code and propose what the alternative. Otherwise might have been. If I didn't know about backslashn, my instinct to move the cursor to the next line might have been literally to just like hit enter or do something like this, like move the double quote, move the parenthesis, move the semicolon onto the next line. But this should start to rub you the wrong way. And indeed this violates a principle of most programming languages and that most programming languages are line based. You sort of start and finish your thought. Ideally on the same line, and this runs afoul of that. And 2, even if you're seeing code for the first time, assume that this just looks stupid as well to sort of move part of your thought to the next line. It just looks a little sloppy and it is. So C and many other languages, Python among them, solve this by giving you these so-called escape sequences. So if you want a new line there, you do backslash N and you will get your new line there. Now that's a bit of an Overstatement what I said in that sometimes lines of code will be so long that they do wrap onto multiple lines, but generally that's a convention that we're going to try to avoid. All right, what else could go wrong? Well, let's do this. Let me go ahead and clear my terminal window, which I can do by hitting control L, or I can literally type clear and I'm going to frequently do this just to keep the screen clear, even though it has no functional impact. It's just an aesthetic. Let me do something else. suppose I forgot to finish my thought and I omitted the semicolon, but otherwise the code is perfect. Let me do make hello now. Enter. Now we're going to see some output that's a little more arcane. Let me go ahead and scroll back up here to make clear that what's just happened is Iran make hello, but I didn't get back to another prompt. I don't see immediately a dollar sign because there's an error message here that is almost as long as the code I tried to write. Not to

worry. Let's see. Here is the name of the file in which the problem exists, stands to reason that it's in `hello.c`. Here is the line number in which the problem seems to exist, line 5, and that's helpful because it lines up with this. And then if you're you care to. This is the 29th character. So if I count from left to right around 29, something is wrong, something is missing. So it's a pretty decent error message. In fact, it even says expected semicolon after expression. There's a little green carrot symbol pointing me at the mistake. So this is again this is another value of the compiler. Not only does it know how to convert source code to machine code, it's also pretty good at finding mistakes in your code and trying to draw your attention to them. So how do I fix this? Well, assuming you've understood the error message at this. Point. Well, you just go back in, add the semicolon, let me go back down to my terminal window. I'm gonna clear it just to clean up the mess. Let me rerun make hello, and now we are back in business and indeed if I do slash hello, I've got hello world back on the screen. Well, let's make one other mistake. Suppose that I forgot, as you sometimes will, to include this line at the top, which will make more sense next week. But for now, let's just omit it and dive right into the code. You would think this is enough, just printing out Hello world. Well, here, let me go back down to my terminal window. Let me do make hello again now, and I'm going to get a whole different error message instead. So now the problem is still with `hello.c`. that makes sense. Line 3, OK, so somewhere in there `printf` is suddenly the problem even though the semicolon is back and the backslash N is back. So let's keep reading error call to undeclared library function `printf` with type. And then this is a whole mouthful. So here is an example of an error message that unless you're sort of conditioned to know what this means and you've seen it before, it's quite more cryptic and unclear like what the solution to the problem is, especially when the rest of your code is truly correct. I've just forgotten something stupid, but how can I sort of think about this problem? Well, it turns out. That another feature of C is that it comes with a bunch of header files, a bunch of files whose names don't end in `.doc` but end in `.h`, and these so-called header files, which end in `.h` contain code that other people wrote that you can use in your own programs. So for instance, in this particular case, a header file is giving us access to what's more generally. In computing called a library, a library is code someone else wrote that you can use, and I actually used the library last week when I did that import line and mentioned OpenAI, the company, I was actually using a library from that company that I automatically downloaded and installed into my programming environment in advance of class because I don't know how to implement a chatbot without standing on their shoulders and using a lot of the code they themselves wrote. Same idea here, even though `printf` is a feature of C, if you want to use it, you have to include that library by telling your program to include the header file. That defines that function and you only know this by being taught it or looking it up in a book or a reference, but in this case I wanted to use a header file called `stdio.h`. It is not `studio.h`. This is a very common bug online if you find yourself typing `studio`. Hypo, it's `stdio.h` and in that file then is defined the printer function. So if I go back to my code here, the solution to this problem truly is to just undo the deletion I made a moment ago because what line one is now doing for me is it's telling the compiler, oh by the way, I didn't write all of the code that I'm about to use. Please include the definition of `printf` from this other file called `stdio.h`. And again, you'd only know this by looking it up in a reference, attending a lecture or something like that. It's not obvious otherwise, but these are the kinds of things you very quickly look up. So where do you look them up? Well, it turns out the ecosystem of C. Has hundreds of books you can buy or download. Many, many,

many websites among them is one of CS 50's own. And in fact the conventional way to look stuff up for the programming language called C is to look at the official manual pages or man pages for sure, for the C language. Unfortunately, many of them were written decades ago when they were certainly written by fairly advanced programmers and not for a broad audience. And so what we have done is imported all of that freely available documentation. Uh, hosted it at our own URL here, manual.cs50.O, and we've essentially simplified it for those less comfortable, those of you who might be less familiar with, less comfortable with technology, and really for most people who aren't used to reading manual pages, it's just useful to have it written in teaching assistant-like language instead. So for instance, if you go to a URL like this, you'll see CS 50's documentation for this official library `Standardio.h` that comes with C itself. If you get a URL like this, you can look up the documentation for `printf` itself specifically. So for instance, let me go ahead and just give you a teaser for this. If I were to do the same. On my own computer, I might see the CS 50 manual pages here and you'll see header file by header file a bunch of frequently used functions in CS 50. We've also filtered the list down from a massive list to a much shorter list so that you can sort of see what's most likely useful to you. If you go to a specific page like `Standardio.H`, you'll see for instance here just over a half dozen functions that we won't touch on today beyond `printf`, but that we'll see in the class over time that does useful stuff for. For instance `printf` prints to the screen and we'll see other functions for opening files, closing files, and the like because all of that's related to standard IO input and output. If I go to a specific man page for this header file, you'll see the standard formatting for these pages. So here's the name of the function `printf`, and it prints to the screen. You'll see a synopsis, and this indeed indicates we're in less comfortable mode. If you want to see the original, more arcane documentation, just uncheck that. And you'll see the original official documentation, but you'll see a mention of like what header file this function is defined in so that you know what file to use in your own code. You'll see a so-called prototype, which is just the first line of code from that function. More on that in just a little bit. You'll see an English description. You'll see example code. Long story short, this is the authoritative answer. And even though you have access in this class, to the virtual rubber duck at CS50.AI and other forms of it that you'll soon see. You should also have the tendency and the instinct moving forward to check the official documentation, and all of today's AIs are trained on things like the official documentation. So that's the source material that any of these AI, the ducks the duck among them are actually relying on. But what we're also going to see is that besides these official functions, there's some that CS 50 itself has invented. We use these really as training wheels for just the first few weeks of the course, and then we take these training wheels off, but the reality is in a language like C, certain stuff is just really hard or annoying to do. Certainly if you're learning how to program for the very first time, or at least you are new to C, we'll eventually show you how to do it that way. But even if you just want to get input from the user like a string of text or a number of some. It's generally not that easy to do and see, at least in these early days. So for instance, at this URL here you can see documentation for CS 50's own library and CS 50's own header file `CS50.H`, and you'll see such functions in the documentation as these `get_string`, `get_int`, `get_char`, and a bunch of others as well, and we'll touch on those this week, but it will ultimately be a way of just getting useful work done quickly by standing on our shoulders and actually. Uh, using functions we wrote to then solve problems of interest to you. So let's focus, for instance, on one of these first. `get_string`. A string in programming speak

means text, zero or more characters of text like H E L L O W O R L D. That is a string of text in computer speak and it's obviously not a number like 50, it's actual text that you would type on the keyboard. We'll see then what other things we want to get, but with this. this function, we can start to replicate another program that we implemented pretty quickly last week in Scratch. So recall that in Scratch this one was a little more interactive. I used another blue puzzle piece ask to actually get input from the user and recall that unlike the printf function today and the same block last week, this time we still have the same input output model, but if we pass an arguments to a function that we're about to see. You can get back not just a side effect sometimes but a return value like a useful reusable value like the person's name, as we'll soon see. All right, so let's actually do this. If in scratch the equivalent was asking the user what's your name, asking them that, and then waiting for an answer that we can store in a variable, let me propose that in C side by side it's going to look a little something like this. Instead, at left we have the scratch block, the ask function. Here is the argument there too, and then it and wait just means it's going to wait till the user finishes typing. If I want to translate this to see now today moving forward, well, it looks a little something like this. The closest analog in C, thanks to CS 50's library is going to be a function called get string. So there's no C function called. Ask and we deliberately named this function get string just to make super clear what it is you are getting a string of text in this case, and we've got the parentheses ready to go indicative of this white oval for user input. If I want to prompt the user with that same phrase, what's your name? Well, I can just put it inside of those parentheses. But what next do I need to add around my user input? Yeah, I need the quotation marks just to make clear that these aren't special individual words. This is a whole phrase that I want to be displayed to the user, so I'm going to indeed put double quotes around everything, and this is just an aesthetic. I don't in this case want to bother moving the cursor to the next line. Like I want the user to see the question and I want the cursor to just stay there blinking, waiting for their prompt, but I don't want the cursor to be right next to the question mark. So I'm deliberately just leaving a single white space there just to kind of scooch it over a bit so it looks a little prettier, at least to my eye. Now we're not done yet. Because we need to do something with this value. The get string function, as we'll soon see, is going to prompt the user for me to type something in like my name, but where do I want to put that? Well, MIT has the answer put in a variable called answer, and you can't rename that in scratch. It's just defined as answer. But in C, what I'm going to need to do is something like this. If you want to keep return values around from a function, you literally use an equal sign. And then to the left of it you put the name of the variable into which you want to put that return value. So in mathematics we would use X, Y, and Z as our variables. Again in code, as in scratch, you can name your variables anything you want. By convention they should usually be lowercase. They should not have spaces therein, similar to file names, but this is a pretty good analog now of what's going on collectively. Here, but C is a little more precise. You can't just give the variable a name. You need to tell C or really the compiler, what type of value you want to put in this variable. So if it's a string of text, you put string. If it's a number, you're going to put something else. But for now it's a string per the function's name it's going to give me a string. Now we're so close to finishing this comparison. There's one detail missing. What's still missing from the code here? Yeah, yeah, so we have to finish the thought lastly with a semicolon. So if you're getting the sort of the point already, like this is one of the reasons why we start with scratch, you sort of you get the intuition pretty quickly and even though nothing on the right

hand side is particularly hard, there's just all these stupid little details that you have to ingrain in yourself over time. In this case, for C, but for many programming languages we're going to see the similar paradigm. But among the goals of the course too are to show you how ultimately languages have been evolving. And so one of the things we'll see in Python in a few weeks' time that some of this syntax actually goes away because over time humans. Gotten annoyed at older languages like this. Like why the heck do I have to keep putting a semicolon when it's clear that I'm at the end of the line? So we'll see among languages like Python we can get rid of some of these same features. But for now it's just a matter of remembering what goes where. All right, so let's go ahead now. And take that same idea of converting Scratch to see and actually do something with this code. Let me go back to VS code here. I'm going to keep my file name the same, but what you'll see on CS50's website is that we'll add version numbers to each of the examples that I'm typing out so you can actually see the progression of these programs, even though we're not changing the name. And what I'm going to go ahead and do here, for instance, in `hello.c` this time. Is the following. I'm going to go ahead and first get rid of the single `Hello world`. I'm going to go up here and include this time CS 50. H, so not one but two header files, and then inside of my curly braces inside the so-called main function, as we'll soon call it, I'm going to go ahead and do this exactly the same line of code as on the screen before. I'm going to get a string. Prompting the user for what's your name? question mark space close semicolon, and as an aside, this will will soon see print on the screen. What's your name? So that implies that the `get_string` function is actually using `printf` itself to print out that message. I do not need to use `printf` to display that message on the screen because I read the documentation for CS 50's `get_string` function and I just know that it is using `printf` for me to achieve that particular goal. Now let me do something intuitive but not quite correct. If I want to print out that answer so that the expression is going to be not `hello world` but `hello David` or `hello Kelly`, let me go ahead and say `hello answer` backslash and to move the cursor down as before, semicolon. So this is not quite right, and even if you've never programmed before, you can perhaps see where this is erroneously going. Let me remake the program because I've changed the source code and I need new machine code. Nothing seems to be wrong aesthetic logic rather syntactically, but if I do now `hello` and hit enter, you'll see I'm being prompt. What's your name? So I'm going to go ahead and type in `David` and then hit enter. But when I do, If you know where this is going, what am I going to see instead? `Hello answer` and the computer's just doing literally what I told it to do. I said, quote unquote, print out `hello answer`, but obviously that's not the goal that I have in mind. So how do I actually work around that? Well, what I really need to do is achieve the equivalent of this thing here, which we did by stacking blocks in scratch or nesting them, if you will, one inside of the other. So I want to join the expression `hello`, comma space and that answer, and it turns out in C you can't do it quite like this. Like there isn't an analog of the function at least that we'll see today. So we have to do this a little bit differently. We can do it though by maybe telling the computer, well go ahead and print out `hello` com space and then maybe we can give it like a placeholder to plug in the name once we know the name because when I'm writing my code I have no idea who's going to play this game, me or Kelly or someone else. So what if we use special syntax to indicate where I want the person's name actually to go? Let me propose that we now do this. Instead of printing out `hello` `hello answer`, quote unquote, let's go ahead and start printing out something, and I got my parentheses ready to go and I did my semicolon in advance this time. I want to somehow now say `hello` comma

placeholder, and you would only know this by someone having told you or a reference online percent S is the placeholder for a string that you don't know when you're writing the code, but when someone else is running the code, it will be filled in. And substituted for their input. So hello cents is the closest we can get to this. I still need those some other syntax. I still, I do need those quotes on the left and the right just to be aesthetically pleasing. I'm going to put a backslash N there at the end to move the cursor, but now I've left room in my parenthesis for one more thing, and you could perhaps guess where I'm going with this again, even if you've never programmed before, this is telling print F print out H E L L O comma space something. What should I probably pass in to these parentheses as a second input so that PRF knows what that something is. Yeah. The variable name, so the variable in which I have the user's name, and indeed the convention is to put a comma after the quotes and then the name of the variable that has the value you want to be substituted for that placeholder. Now notice there's a collision of syntax and grammar here. The comma inside of the quotes is just an English thing. Hello. so and so. The comma outside of the quotes is meaningful to see because it delineates which is the first input or argument to left and which now is the second, and we haven't seen this before in C up until now we've only been passing one input, but you can pass in 2 or 3 or 4. Completely depends on what the function is designed to expect. So let me put this all together now. Let me go back to VS code. Previously we were literally printing out answer, but I can change answer to percent S. I can move my cursor outside of those quotes, comma answer, because that's the name I gave to that variable. I can go back down to my terminal window and clear it just to reduce clutter. Let me do make hello one more time. Seems to work. hello, E DAVID, and now hello David is printed. OK, questions on any and all of that. Mhm. Good question. Where is it pulling these header files from? So what you are seeing here is a graphical user interface that's somewhere hosted in the cloud at cs50.dev, the URL I mentioned last week, and we're going to tease this apart in just a moment. That software is running on a computer and that computer's got a hard drive. or solid state drive like folders of storage, those files CS50. H and Standard IO.h and many more are pre-installed on the server to which I have connected, and they're stored in a standard place so that the compiler in particular knows where to look for them. And those are all things we did in advance for you, yeah. and not create a new like a new line. Why does the backslash N not create a new line? So it is backslash N is essentially being printed here, which has the effect of pushing the dollar sign to the next line, otherwise the dollar sign would stay on that second to last line. Other questions? Good, uh, why is there no backslash N over here? Good question. My choice as the programmer. I just wanted to see the sentence. What's your name? question mark and I wanted the user me to type my name immediately after it like this, but I didn't have to do it that way. I just wanted to show you the difference. Got you. And then also like just generally when we're like doing like the what actually always write the like first four lines. Should you always write the 1st 4? Oh, these? Yes, for today, trust me, do this, do this, do this, do this, and next week we'll understand even more what those lines do. However, slight caveat only use CS 50.H if you're using one of our functions. Clearly you don't need CS 50.H if you're just printing something out as in the first example. Other questions? dividing the first input and the second input. I understand that the second input is what I typed as the user, but the first input does not really feel like input for me because that's the question that you ask. Can you like. Correct. So to summarize the question on the right here, this input is effectively provided by the user. This first input though is provided by me. That's the way it is. So

these are both inputs because they're being provided as inputs to the function. The origins of those inputs though are entirely up to what I'm trying to achieve. The first one I know in advance, like I'm the programmer, I know I wanted to say hello someone. The second input I don't know in advance, so I'm using a place I'm using a variable to store the value that I'm going to get when the get string function is used later on, but they're both inputs even though they're used in different ways. Good question. Any others? Yeah, OK, so if we now have that done, well, let's just take a step back into the first question that was just asked about where are these files. Let's take a look back at actually what it is we're actually using here. So it turns out, even though most of you are using Mac OS or Windows, there's other operating systems out there in the world. Phones have iOS, iPads have iPad OS, Android devices have Android. Which is its own operating system. The operating systems in the world are the pieces of software that really just do the most fundamental operations on a device like booting it up, shutting it down, sending something to a printer, displaying something on the screen, managing windows and icons and all of that sort of commodity stuff that is used by other people's software as well. A very popular operating system in the programming world and in the world of servers in the cloud and on the internet at. It's called Linux, and it's a descendant of something called Unix, which has been around for quite some time, and it's what many programmers, most programmers use depending on their environments insofar as Linux is very highly performance, like you can support thousands of millions of users on servers running an operating system like this. It tends not to, but it can have a graphical user interface, which just means it can operate more quickly because it does. Need all of these graphics that are really just for humans' benefits, not necessarily for web browsers and other devices, and Linux, insofar as it's usually used or often used as a command line interface, comes with a whole bunch of commands that you'll start to use and see over time. Now I've used a bunch of commands already. I've used code, which is a VS code thing. I have used Make, which is for today's purposes our compiler, but that's a little white lie that we'll distill next week. And then I've used hello, which is a command I essentially invented as soon as I created a program called Hello. But there's a bunch of other ones as well. For instance, if I want to list the files in my current folder, I can type LS and hit enter for short. If I want to create a new folder, otherwise known as a directory, I can use MKDIR to make a directory. If I want to remove a directory, I can use RM. Director. If I want to remove a file, I can use RM. If I want to rename a file, I can use MV for move. If I want to copy a file, CP, if I want to change directories, change into a folder, I can use CD. Now these two just take a little bit of time and practice to memorize them, and they're all very terse insofar as the whole point of a command line interface is to let people navigate things quickly. So for instance, even though this will be a bit of a whirlwind, let me go back into VS code and let me propose that we play around with just a few of these commands so that you've seen me doing it, but generally speaking in CS 50's problem sets, we will tell you step by step what commands to type so that you can achieve the same results and then later in the term we'll stop bothering reminding you pedantically how to do this and that because it should come more naturally eventually. But for instance, let me go ahead and do this. Let me go ahead. And reopen my file explorer at left. Yours will look a little different. You'll have a different number as your unique ID, but generally you'll see whatever files and or folders you've created already. The first thing I created today was called hello.c. and then by using Make, I created a second file I claimed called hello. So the reasonhello works is because there is in fact a program called hello. In my current folder, ergot

that was created when I compiled my source code into machine code. Now suppose for the sake of discussion that this is going to get messy quickly because the more programs we create in class and for problem sets, you're just going to have a hot mess of files inside of this one main folder. Well, let's create subfolders like you might be inclined to do on your Mac or PC or Google Drive or whatnot. What we can do. This in a bunch of ways. I could right click or control click on my File Explorer and I'll see a somewhat familiar contextual menu and I can literally choose new folder or I can rename things or I can move things around by dragging and dropping them. But for today, let's focus more on the CLI, the command line interface, and again commands like this. So let me go back into VS code. And let me propose that we do a few things just because as a tour. First, let me delete the machine code. I'm done with this example. I don't really want to keep these bits around unnecessarily. I'm going to delete hello, not hello.c, but hello, the compiled program. When I type that, I'll be cautioned, remove the regular file, whatever that means, called hello. Here I'm being prompted for a yes no response. Y suffices. So I'm going to hit Y enter and watch what happens at top left as soon as I use my terminal window and this command. To remove that file, it disappears. I could have right clicked on it or control clicked on it, but this command line interface achieves the same thing. Now suppose that for problem set one and future problem sets, I want to keep like every program I write in its own folder just to keep myself organized, especially as the term progresses. Well, let me create a new folder called hello itself. So I don't want to create a program called hello. I want to create a folder called hello. One way I can do this per this here cheat sheet is to make a directory, which just means folder. So MK DIR, hello enter and you'll see a top left. Now I indeed have a folder and it even has an obvious folder icon next to it. Now I could cut some corners. I could click and drag on hello.c and just drop it into hello, but again, let's stick with the command line interface. Let me go ahead now and move MV for short. Hello.c into hello. So this is the first command where I'm passing in not one word after the command like code hello.c or make hello. Now I'm typing two words after the command because the way the move command is designed is to expect the origin as the first word and the destination as the second, so to speak, whereby if I want to rename hello. sorry, if I want to move hello.c into the hello folder. I should type like this. Now you can, just so you know, include a trailing slash, a forward slash at the end of the destination just to make clear that you want to put this into a folder and not just rename Hello.c to hello, but because the hello folder already exists, Linux knows what it's doing and it's just going to assume that when you do that, watch what happens at the top left. Hello.c seems to have disappeared, but if I click this little triangle, ah, there it is, it's now inside of that folder. But now I've created. Kind of a predict for myself. Let me clear my terminal window and now let me type LS. And when I type LS for list, you'll see only a folder called hello, and it's color coded just to call it out to your eyes and there's a trailing slash just to make obvious that it's a folder that's all done automatically for you by Linux, the operating system. But wait a minute, where did my hello program go? Like where is hello.c? Well, it's in that folder, so I need to change into that folder or directory. And here per the cheat sheet we have CD for. directory so I can do CD space hello, with or without the slash and hit enter, and now you'll see this, and it's admittedly a little cryptic, but my prompt has now changed to still be a dollar sign, but before it is just a constant reminder of where what folder I am in. We adopted this as a convention. Many systems do the same thing, though the formatting might be a little different. This is just to help you remember where the heck you are without having to type some other command to ask the operating

system what folder you are in. So now that I'm here, if I type `LS` and hit enter, what should I see? Just `hello.c` because that's the only thing in that their folder. So now let's do maybe one other thing. Let's do make `hello` inside of this folder. That is OK and notice at top left what just happened. Now I've got both files back. All right, suppose I want to get rid of one well I can do `RM hello` again. I can type `Y` for yes to confirm the deletion, and now I'm back to where I just was. Now suppose I want to do yet other things, suppose that I'm not really proud of this version of `hello.c`. Let me keep it, but. Rename it. Well, I can say how about `MV hello.c to old.c`. I just want to rename the file so `MV` can be used not only to physically move a file from one place to another. If you use it on two file names, it will just rename the file for you. So there's no rename command that you need to use instead. Uh, but you know what, nope, I regret that this program was fine. Let's name it back. So let's move `old.c` back to `hello.c` and watch it top left. It just renames the file again. Um, let me go ahead and make a backup though. So let me copy with `CP`, `hello.c` into a file called like `backup.c` just in case I screw this up. I want to have a spare around. Now you see at the top left, I've got both files. If I now type out less, you'll see both files. So what's happening in the GUI is the exact same thing is happening in the CLI, but you know what, this was just for demonstration's sake. I don't need any of this, so let me remove the backup, say yes for why. Let me go ahead and move `hello.c` out of this folder, which I could just kind of drag and drop it, but how do I move `hello.c` to the parent folder, so to speak? I want to move it out of this folder. Well, you would only know this by having been told. is special notation that means the so-called parent folder. So go back up in the hierarchy. And now if it's not obvious, a single dot, which we have seen before, means this folder, two dots means one step up. There's no triple dots or quadruple dots. You have to use different syntax, but more on that another time. So watch what happens when I do move `hello.c` up into the parent directory. Notice at top left that the indentation changed because it's no longer inside of that same folder and heck, now I'm going to go ahead and do this. I could go back to my main folder by doing `CD`. to back out of this folder, but when in doubt or if you ever get yourself into a confusing mess, just type `CD` enter alone and you'll be magically whisked away to your default folder, a home directory, so to speak, even though that too is a bit of a white lie, so that will lead you. Always where you're starting when logging in to CS50.dev AKA VS code, and now I can see the folder which happens to be empty, and the file. So let me go and do one last command or `hello` to really undo all of the work such that we're now back to where the story began. But the point here is just to demonstrate that with these basic fundamental commands you can do everything that you've taken for granted on Macs and PCs for years with a mouse instead. Questions. On any of these here Linux commands, yeah. Really good question. If you have 5 different files in a folder, how can you choose which one to open? Well, you can certainly do code, space, and the name of the file you want to open, or we're going to see other tricks like you can use an asterisk or a star for a so-called wildcard and say open everything in this folder, and you can even use more precise patterns than that. So over time, once we have more files at my disposal, I'll be able to do tricks like that as well too. Yeah. Yeah Uh-huh. So when you like delete the file, uh what was that? Hello How did Sure, so one of the things I did in my VS code a moment ago was once I was inside of the `Hello` folder into which I had put `hello.c` just for the sake of discussion, I then recompiled it by running `Make hello`, and this example is a little confusing, deliberately insofar as I've got a file called `hello.c` inside of a folder called `hello`, but because I compiled `Hello.C`, I then created a program called `Helloo` as well. But that program he was inside of.

folder called hello, which is only to say that you can totally do this. You can't have a file in a folder in the same place name the same thing because they would collide. Like you can't do that on a Mac or a PC as well. You have to have unique names, but you can certainly put something inside of another folder without collision. Good question. All right, so let's introduce a few more building blocks and a few more things we can do. So besides these Linux commands, which we'll now start taking for granted, we have a bunch of other features of programming languages that we saw in Scratch. Let's now translate them to C. So conditionals were sort of the proverbial fork in the road, enabling you to do this or this or some other thing based on the answer to a question, a so-called boolean expression. Here, for instance, in Scratch is how we might express if a variable X is less than a variable y. We'll go ahead and say x is less than y and out of context, I didn't include it in the slide, presumably we've created X and Y and somehow given them values, whatever they are, but this is just now the conditional part of the program. In C, the way you would do the same thing is you would say if and then a space, then parentheses, which have nothing to do with functions. If is not a function, it is a feature of C that implements conditionals just like this orange block is a feature of scratch. Inside of the parentheses you put your same Boolean expression. So here too out of context, if up here I have defined variables X and Y, well I can certainly use them in this conditional and I can use this less than operator just like in math class to ask this question. And the answer, even though it's a less than sign, is indeed if you think about it going to be true or false, yes or no, it's a boolean expression. It either is less than or it is not. All right, inside of the curly braces which are necessary here, I'm just going to literally put. Old friend print F, and there's nothing interesting here except the new phrase X is less than y with the backslash N, the semicolon, and the parentheses. This though is deliberate, just like in Scratch. The say is sort of indented and sort of hugged by the if orange puzzle piece. Similarly, do these curly braces, are they meant to sort of imply the same? It's sort of embracing these lines of code. As an aside, in C, they're not always necessary. If you have a single line of code, you can technically omit. However, what you'll see in C as well as in CS 150 in particular, we will generally preach a certain style like any company in the real world would do so that programmers who are collaborating on code all write code that looks the same so that it doesn't devolve into a mess because everyone has their own convention. So this is a convention to which you should indeed adhere and then I've indented 4 spaces to make clear logically that this line of code only executes if the answer to this question. Is true or yes. Meanwhile, in scratch, if we had an if E condition, so a two-way fork in the road, if X is less than Y, say so el say x is not less than Y. How can I do that in C? Well, if X less than Y, something else, something else. And what are the uh what goes in between those curly braces? Well, just two different print Fs. X is less than Y or X is not less than Y. The only new thing here is we've added Ls and another pair of curly. Races just like we've got sort of two orange shapes hugging those two purple puzzle pieces there. All right, how about something a little more involved and this looks like it's escalating quickly, but it's just because the scratch puzzle pieces are so big. If X is less than Y, then say X is less than Y. else if X is greater than Y, then say X is greater than Y. else if X equals Y, then say X is equal to y. How can we do this in C? Almost the same idea. If X less than Y, if X greater than Y, L X equals equals Y. Well, before we reveal what's in the curly braces, this is not a typo. Why have I presumably done this even if you've never used C before? Yeah. sit Exactly, the single equal sign, which we've used already when storing a value from get string into a variable like answer is technically the assignment operator.

So humans decades ago decided that when faced with a situation where they wanted to copy from the right to the left a return value into a variable, it made sort of visual sense to use an equal sign because you want those two things ultimately to be equal even though you kind of read the code from right to left. In that case, I can only imagine at some point the same people were in the room and they were coming up with the syntax for conditionals and like, oh shoot, we've already used equals for assignment. What do we now use for equality and the solution in C as well as in many other languages is literally this they use `==`. So this is the equality operator, whereas a single one is the assignment operator, and it's just because. Now Scratch is designed for kids, no sense of confus. using little kids with equal equal signs, so scratch uses a single equal sign, whereas C and most languages use double equal signs, so a minor divergence there. What goes in the curly braces, nothing all that interesting, just a bunch more print Fs. But here's an opportunity to distinguish not only the equivalence of this scratch code with C code, but a missed design opportunity that we sort of tripped over, if briefly last week. This is arguably not well designed, even though it is correct. Why, yeah. Yeah, we don't need to ask this third boolean expression. Is X equal equal toy, so to speak? Well, logically, if we're using sort of normal person numbers, it's either less than or greater than or by default equal to. So you're just wasting the computer's time and in turn the user's time by asking this third question. So slightly better here would be get Of the ELTS if just have a default case, an ETS block, so to speak, that looks like this. If it stands to reason that there's only 3 possibilities, you only really need to interrogate 2 of them out of the 3. So a minor optimization, but you can imagine doing that again and again and again in your code. You don't want to be wasting the computer or the user's time if you can improve things like that. All right, so now that we have these equivalences between scratch code and C code for these conditionals, well, what other things can we throw into the mix? Well, C has a whole bunch of operators, and just so that you've seen a list in one place, you've got not only assignments and less than and greater than in equality, but a few others here as well. Now, even though in like Microsoft Word and Google Docs you can kind of do a greater than or. to sign one over the other or less than or equal to in C, in most languages, you actually just hit the keyboard twice. You do the less than and an equal sign, or you do a greater than and the equal sign, and that's how you achieve the notion of greater than or equal to or less than or equal to this one we've seen. Anyone want to guess what exclamation point equals means, otherwise pronounced bang equals? Yeah. Not equal. So generally in programming you'll see an exclamation point implying the negation of something else, the opposite. So you don't want it to be equal to, you want it to be not equal to. Now you might think, shouldn't it be not equal equal? Yes, but they're trying to save keystrokes. So this is the negation of that, even though it doesn't quite look like it should be, just 2 characters instead of 3. Um, and there's many other operators that won't encounter in the wild over time. Um, but there's also worth noting in C, more than just strings, like strings we call were strings of text, and there's other types of Uh, data that you might get from a user or store we've seen string, but we'll actually see a whole bunch of others. So in C we're going to see boos themselves of a variable that can be true or false, and that's it. So very much interrelated with Boolean expressions. A variable itself can be true or false. We're going to see chars or characters, so not strings of text like multiple letters and words and the like, but just individual characters. See, unlike some languages. Does distinguish between single characters and multiple characters. Double, or rather let's jump to float. A float is otherwise known as a floating point value, which is

just a number that has a decimal point in it, a real number if you will, but a float generally uses nowadays 32 bits total to represent those numbers. The catch with that is that how many total values can you represent with 32 bits roughly per last week. It's one of the few numbers I propose you remember it's like roughly 4 billion, but how many real numbers are there in the world according to math class? An infinite number. So we seem to have a mismatch between what we can represent in code and how many actual numbers there are in the world. OK, so not to worry, if you need more precision, like more significant digits, well, you can upgrade your variables, so to speak, from a float to a double, which uses 64 bits, which is way more precise, twice as many bits, but it doesn't fundamentally solve the problem because really it's still finite and not infinite, and we'll end today with a look at what the real world implications of that are. But besides floating point values, there's just simple integers 012, and the negatives thereof, but those conventionally used 32 bits, which means the highest a computer can count using an in would be 4 billion. But if you want to do negative numbers, it's going to be roughly 2 billion, so you can go all the way to negative 2 billion. So that's not that large nowadays. Along uses 64 bits, which is a much bigger range of values, but there too still finite, and there's a bunch of. Others as well. So these are just the types of data that we can store and manipulate in our programs, but a couple of those now, do a couple of those, one in particular specifically come from CS50.H. So among the things you get by including CS50. H in your code is access to not only get string, but these other functions as well. And we'll start to use these in a little bit whereby you can get integers or chars. Doubles or floats. We don't have a get bol because it's not really useful to just get a true or false value typically, but we could have invented it. We just chose not to. But we'll frequently use these here functions that you can access by using that there header file. But where are we going to put these values and how are we going to display them? Well, turns out there's more than just percent S. So percents was a placeholder for a string, but if you want to print out something like a char. Single character you're actually going to use percent C. If you want to print out a floating point value, you're going to use percent F, an integer percent I, and a long integer that is a long, you're going to use LI instead. So in short, there are solutions to all of these problems. These are not intellectually interesting details, but they are useful, practical things to eventually absorb over time. So let's go ahead and do. Let's do just a few more examples together in a little bit we'll adjourn for a short break during which snacks will be served every week out in the transit. But before we get to that, let's focus on these here variables. So in Scratch we had the ability to store a bunch of values in variables that we could create ourselves by creating new puzzle pieces. In C you can essentially achieve the same. So for instance, suppose that in Scratch we wanted to keep track of someone's score. Using a counter, well, we might create a variable called counter and set it initially to 0 and then eventually add 1 to it, add 2 to it, and so forth as they drop trash into the trash can for instance. Well, in C you're going to do something almost the same. You can choose the name of your variable just like I did previously with answers. You can assign it a value like 0 initially, but per earlier, what more am I probably going to have to do in C on the right hand side here? Yeah. I got to give it a type and a counter insofar as it's numeric is not going to be a string of text, and I don't think I need to worry about decimal points if I'm just counting the equivalent on my fingers. So int will suffice and in is the go to number and at least if 2 billion plus values is more than enough for your case, which this is going to be still one minor thing missing. Yeah. The semicolon to finish the thought. So that on the right is the equivalent to doing this here on the

left. Suppose that in scratch you wanted to increment the counter and add 1 to the score, add 2 to the score, and so forth. It might look like this. Change counter by 1, implicitly going up unless you did negative, which would go down. In C you can do this actually in a few ways, and this looks a bit wrong at the moment. How can counter possibly equal counter plus 1? This does not mean equality per se. The single equal sign recalls. Assignment and it means take the value on the right and copy it to the value on the left or to the variable in this case on the left. So this takes whatever the current value of counter is 0, adds 1 to it, and then stores that 1 in the counter variable. So now the value is 1, and if you do it again, it goes to 2, it goes to 3, goes to 4, and so forth. But honestly, this incrementation technique is so common that there's more shorthand notation for it. You can also just do this. Looks a little weird at first glance, but counter plus equals one semicolon does the exact same thing. You can just type fewer keystrokes, and honestly, doing this is so down common in C that you can even do this. Counter plus plus does the exact same thing by adding one to the variable. There's no plus plus plus or plus plus plus or more pluses. It's only for incrementing individual values by one. So arguably this version and this version, albeit more verbose, are a little more. A tile because then you can add 2 or 3 or more at a time. And there are equivalents for you doing decrementation and doing minus or the minus symbol more generally in there. All right, so let's actually use this technique in some code. Let me go back into VS code here. Let me close my file explorer and let's go ahead and create maybe this time like a little calculator of sorts. Let me propose that we implement a very baby calculator, or rather, not even a. Calculator, yeah, let's just compare some few values. So let me do this code of compare.c to create a brand new program called Compare. And then in here I'm going to do a bit of boilerplates. I'm going to go ahead and include CS 50. H. I'm going to go ahead and include standard.io.H, and I'm going to go ahead and do in Main void, more on that next week. And then inside the curly braces, let's use these these new techniques. Let's give myself a variable called X and set it equal to. The return value of get in, that other function I promised exists, and let's prompt the user for a value for X with a sentence like What's X question mark, and then a space just to nudge the cursor over. Let's get another variable Y. Set it equal to get into again and ask the user this time what's Y, essentially using the same function twice but to get two different values. Now. Let's go ahead and do something pretty mindless. If X is less than Y, go ahead and print out with print FX is less than Y backslashn to move the cursor close semicolon. So it's not that interesting of a program, but it's at least dynamic in that now I'm prompting the user for two numbers. So let's do this Make compare enter seems to have worked, and in fact I can check that it worked by typing what command to list the files in my directory. LS for short. And now you'll see I've got hello.C, but no hello because I deleted that with RM a few minutes ago. I've got Compare.c which I just created, and then I've also got a program called Compare and the asterisk there is just a visual indicator that this is executable. It's a program you can run. It's not just a simple old file, even though I didn't type. As previously with hello, it would have similarly had an asterisk next to it in this context, but you don't see that in the File Explorer. If I now do compare, well, let's do something silly like 1 for X, 2 for Y. OK, X is less than Y. Let's do it again. compare, 2 for x, 1 for Y. OK and I see nothing. Well, why am I seeing nothing? Well, logically, I didn't have a condition for checking for greater than, let alone equal to. So let's enhance this a little bit. Let me go ahead and minimally say, right, else if x is not less than Y, let's go ahead and print out X is not less than Y backslash and close semicolon. So I'm at least handling that situation too. Let me clear my terminal window, do make compare

again, compare 1 and 2 works exactly the same. Now let me go ahead and do 2 and 1. There we have better output. Of course it's not really complete yet because if I do slash compare again and do 1 and 1, it'd be nice to be a little more specific than X is not less than Y. It's not wrong, but it's not very precise. So I can add in to the mix what we did earlier and I can say, OK, well, else if X is greater than Y. Say X is greater than Y else if X equals equals y, go ahead and print out X is equal toys. But here too someone observed that this is sort of stupidly inefficient. What line of code should I actually improve here? To tighten this up, yeah. if you could Yeah, so line 17, I think I can just get rid of that unnecessary question because logically that's going to be the case at this point and now I can go ahead and recompile this with make compare compare again enter 1 and 1, and now we're back in business, catching all three of those situations, uh, those scenarios there. Questions on any of these things here. Why have I deliberately not done this? Let me rewind just a moment and let me hide my terminal window just to keep the emphasis on the code here. Why not do this and keep my code arguably simpler? Like, why not just ask three questions Step 9, step 13, and step 17 here? Yeah, what don't you like? Because then it would check each and every condition, um, even though for example the first one might be fulfilled, it would check the 2nd and 3rd to that way. Exactly, it's another example of bad design because now no matter what you were asking three questions on lines 9, 13, and 17. Even if X ends up being less than Y from the get-go, you're still wasting everyone's time by saying, well, well, is X greater than Y? You already might know that it's not. Is X equal to Y? You already might know that it's not. And so these three conditionals at the moment are mutually exclusive, whereby you're checking all three of them no matter what, even though logically that should. Be necessary. So our first approach was actually quite better. And in fact just to show you the density difference here, let me go back to this very first version here whereby I was only checking that one condition is X less than y. Well, if you're more of a visual learner, you can actually draw out what code looks like in flow chart form. So here is a drawing of a program that starts here and ideally stops down here, and each of these figures in the middle sort of represent logical components of the code. Uh, here in the in the diamond here is my boolean expression, which represents the start of the conditional. So if X is less than Y, I have a decision to make. Yes or no, true or false. Well, if it is less than Y, true, well, let's go ahead and print out quote unquote X is less than Y and then stop. However, the first version of that program recall just said nothing if it were not the case that X were less than Y. That's because false just led to the stop of the program. There's no keyword stop. There's just. No no code to handle that situation, but the second version of the code, when I actually added in ES looked fundamentally a little different. So now 2nd version of that code asked is X less than Y and if true, behavior is exactly the same. But if it weren't true, it were instead false, that's when I got the message X is not less than Y. But in the third version of the code where I added the if else if L if, then the picture gets a little more complicated and let me zoom in top to bottom. Here we have. Longer flow chart, but the questions are really the same. When I start this program, I ask, is is X less than Y? If so, I print out X is less than Y. However, in that, in that last version of the program, I was still foolishly asking the same question. Well, wait a minute, is X greater than Y? Wait a minute, is X equal toy? And that's the version in which again I had all of that unnecessary code which I just undeleted here asking three questions at a time. Ideally, Don't want to make that mistake by doing it again and again and again. So if I instead revert that code to ETS if and Es if, then my flow chart looks a little bit different because notice the sort of shortcuts now. If X is

less than Y, woo, true, we do this and we're done super quick. If x is not less than Y, fine, we do ask one more question. X is greater than Y. Well, if so, boom, we make our way to the end of the program by just printing that only if it's the. Reverse case where x equals equals y. Do we check this condition? No. This condition, no. This condition and then OK, now we can print out X is equal toy because it must be logically. Of course it's been observed multiple times. This is a waste of everyone's time, so we can prune this chart more and just have one question, two questions, and that alone tightens up the program. So again, if you're more of a visual learner, most any block of code you can translate to this sort of pictorial form, but it really just captures the same. Logical flow that the indentation and the syntax in the code itself is meant to imply. All right, how about a final exercise with one other type here? Recall that this is our available types to us. Actually two final examples here before we have a bit of a break. Here we have a list of types that we can use, and here we have a list of functions that we can use. Let's go ahead and make a program that's representative of something we do quite often nowadays, but using a different type. So let me go back into VS code. Let me close compare.c. Let me reopen my terminal window and clear it just so we have a new prompt. And let's go ahead and create a program. Called agree.c. It's all too often nowadays that we have to like agree to terms and conditions. To be fair, it's usually in the form of like a pop up and a button that we click, but we can do this in code at the command line as well. Let me go ahead and include to start CS 50.H and include to start standardio.h. Let me again for today's purposes do in Main void, but we'll reveal next week what we why we keep doing that. And now for a yes no answer, it suffices just to ask for a single char or character, not a whole string. So let's do this char. C equals get char and let's ask the user, quote unquote, do you agree question mark for instance. And now I can actually compare that value for quality with some known answers. For instance, I could say if C equals equalsy, then go ahead and print out, for instance, agreed period backslash semicolon, else if C equals equals n. In quotes, let's go ahead and print out, for instance, not agreed backslash and semicolon. Now there's still room for improvement here, but notice we're just now using the same building blocks and C um in different ways to solve different problems. But notice on lines 8 and 12 I've used single quotes, which I alluded to earlier. Why is that the case? Why single in this case here? Yeah, it's a single character and this is just the way you do it in C. When you want to compare a single character, you use chars and you use single quotes. When you want to use strings of text like multiple characters, multiple words, multiple sentences or paragraphs, you use strings. So this would seem to work, but are. I could be a little more efficient if the user doesn't type Y. I mean, frankly, I could just chop off this L if and make it an E and just assume if you don't give me ay answer, then at least I'm going to assume the worst and you don't agree. But even here, the program's not all that great. Let me go ahead and do make agree. And then do slash agree, and do I agree? Sure, I'm gonna go ahead and type Y. Meanwhile, if I type anything else like N or uh even emphatically no, that would seem to, whoops, why did that not work? Yeah. Exactly. So among the features of CS 50's functions like Getchar is that it will enforce what type of data you're getting. So even though I, because I use Getchar, if the user doesn't cooperate and types in multiple characters, GetAR, like some of our other functions, is just designed to prompt them again. Again and again until they cooperate, that's useful so that you don't have to deal with that kind of error checking. But here I could type N in upppercase, and that seems to now work but that only works because of the Ls. Let me go ahead and do this, which is very reasonable. I'm going to go ahead and type Y Y, which you

would hope works. That feels like a bug at this point. Like it's fine if we don't want to support yes and no. We just want to support Y and N, but it's kind of obnoxious not to support the upper case version thereof. So how can we fix this? Well, let me hide my terminal window and I could go in and fix this as follows. I can say, well, else if C equals equals quote unquote. Capital Y in single quotes and then I can do print out agreed period backslash and semicolon, and then I can do that that would work that would work there, but what rubs you the wrong way perhaps about this solution, even if you've never programmed before. Just applying some of the lessons from last week, yeah. It's redundant. I mean, I didn't technically copy and paste, but like line 14 is identical to the line 10, so I might as well have copied and paste, and that's generally bad practice. Why? Well, if I want to change the English language to say something else in that case, now I have to change it twice and it's just I'm repeating myself, which is just bad design. So there are ways to address this through other types of operators that we haven't yet seen. If I want to ask two questions at once, that's fine. I can do something like this. Well, if C equals equals Y or C equals equals Y, I can tighten things up using so-called logical operators whereby I am now taking a boolean expression and composing it from two smaller boolean expressions, and I care about the answer to one of those questions being true. So whether it's lowercase y or uppercasey, this code now will work and if it's anything else, we're going to default to not agreed. So the two vertical bars, which is probably not a character you type that often, and it varies where it is on your keyboard depending whether it's American English or something else, just means logical or. This is not relevant here, but you could also in some contexts use two ampersands to connote and, but this does not make sense why. Why is it clearly not correct to say and in between these two clauses, yeah. Exactly, the variable can't both be lower case and uppercase. That just makes no sense. So this would be a bug, but using a vertical two vertical bars here is in fact correct. All right, well, let's do one final flourish here besides conditionals. We had these now loops. We call that a loop is just something that does something again and again and again. Here, for instance, to scratch how we might meow 3 times. In C there's going to be a few different ways to do this. Here is one. You can in C. Declare a variable like I for integer or whatever you want to call it and set it equal to 3, the number you care about. You can then use a loop, and the closest to the repeat block is arguably a y loop. There is no repeat keyword in C, so we can't translate this verbatim, but we could say while I is greater than 0. Why? Because that's sort of logically what I want to do. If I start counting at 3, maybe I can just sort of decrement one at a time and get down to 0, at which point I can stop doing this thing. So I'm going to initialize a variable to I, a variable I to 3, and then I'm going to say while I is greater than 0, go ahead and do the following. And at the end of that loop before whipping around again, I'm going to use this line of code, which we haven't seen, but you can infer I minus minus just means subtract 1 from I. So this is going to have the effect of starting at 3, going to 2, going to 1, going to 0, and as soon as it goes to 0, this boolean expression will no longer be true. And so the loop will just implicitly stop because that's it. So what are we going to put inside of the curly braces besides this decrementation? Well, I think I can get away with just saying meow, and that will now print 123 times. And yet that's interesting. I sort of counted instinctively 123, even though I'm proposing that we count 321. So can we implement the logic in the other direction whereby we count up from 0 instead of down from 3? Well, sure, we just have to. A few changes. We can set i equal to 0 initially. We can change our boolean expression to check that I is less than 3 again, again and again. And on each iteration of this loop, let's just keep

incrementing `i` with `i++`. And at this point it will have the effect of doing `i = i + 1` is not less than 3, so I won't put any more fingers up. I will meow in total 3 total times. And again, if you're a visual person, here's how we might start counting at 0. Initially, check that `i` is less than 3, which it is initially, and if so, we print out meow. Then we increment `i` and we get whisked around again to the Boolean expression because that's how while loops work. You constantly have the condition being checked again and again. That's just how C works. As soon as I've incremented `i` from 0 to 1 to 2 to 3, `i < 3` will eventually not equal, not be less than 3, so the answer will be false, so the loop will just. So that has the effect of achieving the same, but it turns out that looping some amount of times is so darn common that you don't strictly have to use a `while` loop. A `for` loop, so to speak, is another alternative there too, whereby the syntax is a little weird, it's a little harder to memorize, but it allows you to write slightly less code because you write more code on a single line. So the way you read a `for` loop is exactly the same in spirit. You initialize the variable, everything to the left of this first semicolon. You then check the condition and the computer does all this for you. If `i < 3`, if so, you execute what's inside of the curly braces, and then automatically the thing to the right of the second semicolon happens, so `i` gets incremented from 0 to 1 in this case. The condition is checked is `1 < 3`. It is, so we print meow again. And C increments 1 to 2. Is `2 < 3`? Yes. So we now again `i` gets incremented to 3. Is `3 < 3`? No, so the `for` loop stops. So it's exactly the same, but just more magic is happening in this first line of code here more than you yourselves have to actually write, and it's just arguably more common convention, but both of them. Are perfectly correct if you'd like to do that yourself. So let's go ahead and actually implement now this this beginning of a CAT in VS code. Let me go back to VS code and close `agree.c`. Let me reopen my terminal window and create an actual cat in `cat.c`. And let's go ahead and do this initially the wrong way, include `stdio.h` in `main` void, and then inside of `main`, let's go ahead and print out `printf("meow\n");`. And then heck, let me just copy paste. So this is obviously the wrong way, the bad way to do this because I'm literally copying and pasting, but it is correct if I want the cat to meow 3 times, I can make this cat. I can do `cat`, and I get my meow, meow, meow. But let's now actually use some of those new building blocks whereby we converted scratch to. See and let me go back into this code and I'll do the while loop first. So I could instead have done in `i = 3` if we count down initially while `i` is greater than 0, then go ahead and print out `printf("meow\n");` and then do `i--` or `i = i - 1`. I minus minus because we're starting at 3. Now let me go back to my terminal window and clear it. Do `make` and we get 3 meows and this is now arguably better implemented. What if I want to flip things around? Well, I could now change, uh, maybe do it the normal person way. I could start counting at 0, and I can do this so long as `i` is less than 3, and I can do this so long as I increment `i` on each iteration. Now I can do `make` again, and that too works. But there's another way I could do this if I want to count like a normal person, like start counting from 1 and count up to 2 and through 3. I could do this, but this is arguably this is correct. It would iterate 3 times, but it's a little confusing because now I have to think about what it means to be less than 4. OK, that means equal to 3. I could be a little more explicit and say, we'll do this while `i` is less than or equal to 3 using yet another one of those operators. So I can make a cat yet again, and that too would work. Now which of these is correct or best? The convention truthfully is in general in code to start counting from zero, start counting up to but not through the value. You that you want so at least you see the starting point and the ending point on the screen if you will

at the same time. But of course I can condense all of this a bit more and turn this whole thing into a 4 loop and I instead could do 4 in 1 equals 0, 1 less than 3, 1 + plus, and then down here I could do print out quote unquote meow and if only because I typed fewer keystrokes that time, like this feels a little nicer, it's a little tighter and more efficient to create even though the effect is the same. Indeed, when I make this cat. And do a final time this year to gives me the three meows. So what could go wrong? Well, sometimes you might be inclined to do something forever, and we might have done that in scratch, and indeed we did when we had some things bouncing back and forth off of walls and so forth. You can achieve the same thing in code. In fact, in C we could use a while loop, but there is no forever block, so while suffices. But recall that the while loop expects a boolean expression, and if I want to do something. Forever I essentially need an expression here that's always true. So I could do something stupid and arbitrary like while 2 is greater than 3 or while 1 is less than 2. I mean, make a statement of fact that never changes ergo it's just going to run forever. But if the whole goal here is to do something forever and to get this boolean expression to be true, the convention in programming is just to literally say while true, and that implies. And functionally means that you will do this thing forever unless you somehow prematurely break out of those curly braces. More on that before long. So if I want to meow forever, I could now just do this, and this would be an infinite deliberate loop, but unlike a game where you might want it to keep going and going and going for some time, I'm not sure this is going to be the best thing for us. Let's go ahead and try this. So let me go ahead here and Include for good measure uh CS 50's library if only because um it too is giving us features like uh bulls. Uh, here I'm gonna go ahead and say while true, and then inside of my curly braces I'm just gonna print out meow. Let's go ahead backslash N semicolon. Let's go ahead here and make cat one final time. Let me go ahead here and do cat and. This is like the annoying cat game just like meowing meowing meowing endlessly. Like I've now kind of lost control over my terminal window and mark my words, at some point you might do this too, but let's go ahead and take a juicy 10 minute break here. Uh, we have some delicious blueberry muffins out in the transep. Come back in 10, and we'll figure out how to stop this here, cat. Alright, so it's been about 10 minutes and like VS code is freaking out with high code space CPU utilization detected. Consider stopping some processes for the best experience. So this is what happens when you have intentionally or otherwise an infinite loop insofar as I've been printing out meow endlessly, and I was warned by my colleague that I probably shouldn't let this run too. Long because we might lose control over the environment altogether, but the answer to how to solve this is going to be control C. So there's a few cryptic keystrokes that you can use to generally interrupt things as in this way. And in fact, if I go back and you'll see, you know, I kind of lost control over my code space here, I'm going to go ahead and try to reload the window altogether, but had I hit control C in time, let's hope this doesn't now go off the rails. Control C would have been our friend. There we go, and we're back. OK, so now that we've got control over our so-called code space again, how can we go about making our meowing program a little more dynamic in so far as let's like start asking the user how many times they want the cat to meow, certainly rather than do it an infinite number of times and even rather than do it 3 times alone. I think we have all. These building blocks thus far. So let me go ahead and stay in cat.c here and go ahead and delete the body of the contents of my main function. And let's go ahead and do this. Let's give myself an in and I'll go ahead and call it N for number, though I could be more verbose than that if I wanted. I'm going to set it equal to the so-called return value of get

in, which we call is going to get an integer from the user and quote unquote, let's ask the user what's and just. Like I asked earlier, what's X and what's Y, where N is the number of times I want the cat to meow. Now how can I use this variable? Well, we have that building block too. I could use a Y loop or a 4 loop, and if I use a 4 loop, I could do this. I could initialize a variable I for integer, set it equal to 0 initially. I could then do I less than not 3 this time, but N, so I can use that. Variable is a placeholder inside of the loop to indicate that I want to do this n times instead of 3, and on each iteration through this loop I can do I + plus. Of course I could be counting down if I prefer by using decrementation, but logically I would say this is canonical. Start at 0 and go up 2, but not through the value that you actually care about. And I'll go ahead now and print out quote unquote meow with a backslash N semicolon. Back down to my terminal, make this catgaca enter. I'm prompted this time for N. I can still give it 3, and I'm going to get 3 meows this time. However, if I run it again with cat and a different input like 4, of course I'm going to get 4 meows instead. Now what is get in doing for me? Well, it does a few things similar to get char doing a few things for me. For instance, suppose that instead of answering this question correctly with a number. and I say something random like dog that is not an integer and so the get in function is designed to reject the user's input implicitly and just reprompt again, again and again. I can try bird and it's going to do this again. So somewhere in the implementation of get in there's a loop that we wrote that does this kind of error checking for you, but it doesn't do everything because an integer is a fairly broad category of numbers. It's like negative infinity. Through positive infinity and that's a lot of possibilities, but suppose I don't want some of those possibilities. Suppose that it makes no sense to ask the cat to meow like -1 time, and yet the program accepts that. It doesn't do anything or anything wrong, but I feel like a better designed program would say no, no, no, like -1 makes no sense. Let's meow 0 or 1 or 2 or more times instead. So how can I begin to add some of my own error checking and coerce the user. To give me the type of input I want. Well, let me clear my terminal window and go back up into my code, and why don't I do something like this? After getting N, let's just check if N is less than 0, because if so, I want to prompt the user again, and I can prompt the user again by doing N equals get in quote unquote what's N question mark semicolon. Now what's going on here? Well, on line 6 I'm doing two things. I'm getting an integer from the user and I'm not only storing it in the variable N, I'm also technically creating the variable N. So I didn't call this out earlier, but on line 6 when you specify the type of a variable and the name of the variable, you are creating the variable somewhere in the computer's memory, and that's necessary in C to specify the type. If the variable already exists though, and you just want to reuse it and change it later on, it suffices as in line. 9 just to reference it by name. It would be sort of stupid to specify the type again because C already knows what type it is because you told C what it is on line 6. So that's why line 6 and 9 are a little bit different. So let's see how this now works. Let me go back to my terminal window and remake this cat. Let me doca again. Let me not cooperate and type in like -1 again and notice I am reprompted this time. Fine, fine, fine. Let's type in 3. And now it works, but you can perhaps logically see where this is going. Let me go ahead and run this againca, type in -1, type in -1, and huh, it didn't prompt me again, but that's consistent with the code. If I hide my terminal window here, you'll notice that I've got 1, maybe 2 tries to get this question right, and after that there's no more prompting of me. Now you can kind of imagine that this is probably not the best way to do this. If I were to go inside of line 9 and then move the cursor down and say, OK, well if N still doesn't it still is less than 0, well, let's just do get into again and

ask what's N question mark and heck, OK, if it's still less than 0, well, let's just keep asking the same, right? Why is this bad? I'm repeating myself. I'm essentially copying and pasting even though I'm retyping. I mean this just never ends, right? Like how many chances are you going to give the user? In spirit you'd hope that they don't not cooperate this many times, but really to do this the right way we should probably prompt them potentially as many times as it takes to get the correct input. So this is not the right path for us to be going down. But of course we have already now this notion of like a loop whereby we could just do this in a loop. Ask the question once and maybe just repeat the question again, again, but the same question. So how might I do this? Well, let me go ahead and delete all of this and let me just try to spell this out logically. So I want to get a variable N from the user and let's go ahead as follows. While true, I know how to do infinite loops now, and even though that created a problem for me with the cat, I bet we can sort of terminate the loop prematurely. I proposed earlier as follows. I could do this in N equals get in and ask the user again what's N question mark. And then I could do something like this. If N is less than 0, well then, you know what, go ahead and just continue on with the same loop. Else if it is not the case that N is less than 0, what do I want to do? I want to break out of this loop. So this is new syntax. This is something you can do and see whereby if N is. Less than 0, fine.in means go back to the start of the loop and do the same exact thing again. Otherwise if you instead say break, it means break out of the loop and go to below whatever curly brace is associated with that loop. So continue essentially brings you to the top, brake brings you to the bottom if you will. So logically I think this is right, but this code curiously isn't quite going to work and get me a value for N. Let me go ahead and open my terminal window again. Let's make this cat and huh cat.c line 1925 is an error use of undeclared identifier N. Well, what does that mean again cat.C line 19. Let me hide my terminal window highlight line 19 N is being used in line 19, but I created it in line 8 and so what's the problem? Why is it? Not declared seemingly, yeah. Like and Yeah, this is a subtlety, but I'm using. I'm creating N inside of this loop. I mean literally between the curly braces on line 7 and 17. The implication of which because of how C works is that that variable only exists inside of that 4 loop. This is a problem of what's known as scope, the. and only exists inside of the scope of the while loop in which it was declared. So how do I actually fix this? Well, I need to logically somehow declare that variable and outside of the loop so that it exists later on in the program as well. And there's a few different ways I can fix this, but the best. The way is probably to move the declaration of N, so to speak, the creation of N outside of the curly braces, and maybe kind of squeeze it in here below line 5, so still inside of Main, whatever that is, more on that next week, but in the same curly braces as everything else. So I can in fact do this, and this is where the syntax gets a little bit different. I can solve this quite simply as follows. I can go down to a new line 6 and just say in and semicolon, and that's it. This declares a variable called N. It creates a variable called in and initially it doesn't give it any value, so who knows what's in there. More on that another time. But now on line 9, I don't need to recreate it. I just need to assign it a value. And because now N has been declared on line 6 and between the curly braces on line 5 and all the way down on 24, now N is in scope, so to speak, for the entirety of this code that I've written. So let me reopen my terminal window and clear that old error. Let me do make cat again. Now the error messages is gone. Let me go ahead and do slashca. What's in. Now I'm back in business and I can do 3 for meow meow meow. But better yet, because I'm inside of a loop now, watch that I can do -1, -1, 1123, 50. Finally I can cooperate with something like 3, and because I'm in a loop

that by design may very well go infinitely many times until the user actually cooperates and lets me break out of. That exact loop. Now I strictly speaking, don't need both continue and break. I wanted to demonstrate that both exist, but this is like twice as much code than I actually need. If logically I just want to break out of this loop if and only if n is greater than or equal to 0, because I'm sort of comfortable with the idea of zero mouses, but negative makes no sense, well I can just flip the logic. I can say if N is greater than or equal to 0, then go ahead and break and I've tightened up. The code further. I could technically do something else. I could say something like if N is less than 0, but wait a minute, I want to negate that. You can start to do tricks like this an exclamation point with some additional parentheses so you can invert the logic. It's arguably a little hard to read even though that would be logically correct. So I'm just going to say more explicitly as before, if N is greater than or equal to 0, break out of this here loop. All right, so this is one way to use an infinite loop, but it turns out there's another construct that you can do altogether that is in a feature of C. Instead of using a while loop and forcing it to be infinite by using while true and then eventually manually breaking out of it, there exists another type of loop altogether, and that's called a do while loop, and you can literally say the word do, which means do the following, then you can do. Exactly what we did before N equals get in quote unquote what's end question mark, so exactly like before, but then after those curly braces you use a while keyword, so at the end of the loop instead of the beginning, and that's where you put your boolean expression. I want to do all of that while N is less than 0, so you can kind of invert the logic. And now kind of tighten things up further by just telling the computer do the following. What's the following? Everything in between those curly braces while N is less than 0, and this implicitly handles all of the continuation and all of the breaking by just doing what you've said do this while this is true, but the difference between this do while loop and a normal while loop is literally that the condition is checked at the bottom. Instead of the top, so when you say while parentheses something, that question is asked first and then you proceed maybe this condition is only asked at the very end and why is this useful? Well, oftentimes when writing programs where you want to do something at least once, like you obviously want to ask the user this question at least once, there's no point in asking a question. And like while true or while anything else you should just do it and then you should do it again if the expression evaluates the true and tells you to do something. Now you haven't played with these loops yet most likely unless you have programmed before. Uh there's a fun sort of meme that's apropos of this moment, so let's see if this maybe causes a few chuckles if you remember Looney Tunes here. It's funny for people in the know. There we go, thank you. OK, this doesn't make sense. It eventually will, and it still might not be funny, but it will at least make sense, and it illustrates the difference between do while loop, like the roadrunner, is stopping because he's checking the condition while not on edge, he'll run, but if he is on the edge, he's not going to proceed further. But of course the coyote here, he's going to do running no matter what and then only too late does he check. Ha ha, he's still on the edge. All right, so, ah, thank you. All right, now you're cool. All right, so many more memes will now make sense as a result, but let's go ahead and revisit this code and maybe do something a little bit different here whereby we no longer want to just futz around with some of these, uh, conditionals and these loops. Let's actually make the software a little better designed. And to do this, we'll revisit an idea that we touched on last week and he had to do with problems. 0, which was like create your own function like C does not come with everything you might want. CS 50 library is not going to come with everything you

might want. And at the end of the day, a lot of programming is about abstracting away your ideas, so you solve a problem once and then reuse it, reuse it, reuse it. And heck, you can package it up in a so-called library like we have and let other people use it as well. So here for instance in Scratch is how we could have implemented the notion of meowing as by getting The cat to play the sound meow until done. We abstracted it away and then we had a magical new puzzle piece called meow. In C, this is going to be a little weird today, but next week these details will start to make more sense. You would instead do the following literally type void, the name of the function you want to create, and then void again in parentheses. For now, know that this is the return value of the function. So void means it returns nothing. This is the input to or the arguments to the function, void means it takes no inputs, and that makes sense because literally meow doesn't return anything, it doesn't take anything, it just meows. It has a so-called side effect audibly last week. So this means, hey, see, invent a function called meow that takes no input, produces no output, but does have a side effect of printing meow on the screen. Meanwhile, if I wanted to do something like this in code last week where I meowed 3 times, well, that's fine. We have the building blocks for this, and here's where inventing your own functions starts to get more compelling. I can abstract away the notion of meow. Now this doesn't come with C, it doesn't come with the CS 50 library. I just created in the previous code this meow function so I can encode with a 4 loop and that new function meow 3 times. But I can abstract this away further. Recall that the refinement and scratch last time was this. I could edit the new function and I can say it actually does take an input, otherwise known as an argument, called N. And I clarified that this means to me meow some number of times and then inside of those scratch blocks I repeated N times the meowing act. Well, in C I can achieve the exact same thing even though it's going to look a little more cryptic, but meow still returns nothing. It has an audible or visual side effect, but it doesn't return a value. But this version does take an input, and this might look a little weird, but just like before, when you create a variable in C, you specify the type and the name. When you invent your own function in C and it takes one or more inputs, AKA arguments, you specify the type and the name of those as well. No semicolons up there, just inside of the parentheses, and you'll get used to with practice this convention, but the rest of this code is exactly the same except instead of 3. I'm now using N. So again, I'm just composing the exact same ideas as last week, even though it looks way more cryptic this week, but it will come more and more familiar with more and more practice. So how can I go about implementing this myself? Well, let me propose that. We do something like this. Let me go back to VS code here and let me go ahead and let's really delete most of the code that I've written inside of Maine and let me just suppose for the moment that meowing exists, and I'm going to go ahead and say for the first version for into I equals. less than 3, so we're not going to take input yet. I plus plus, and then I'm going to go ahead here and say meow is what I want this function to do. Now if I scroll back up, you'll see there's no definition of meow yet, so I'm going to invent that too. I'm going to go up here and say void, meow, void, and again this first version means no input, no output, just a side effect, and that side effect super simply is going to be to say just quote unquote meow with the backslash n. And now if I go and open my terminal window, clear it from before, do make cat so far so good, scat, we're back in business, but I've abstracted the function away. Now much like last week where I sort of dramatically dragged the meow definition way down to the bottom of the screen just to make the point that you don't need to see it anymore, out of sight, out of mind, let me sort of try to do the same

here. Let me highlight and delete that. And like go way, way, way down arbitrarily just to be dramatic and paste it near like the 100th line of code and scroll back up. Now out of sight, out of mind. I've already implemented the idea of meowing. We don't need to see or talk about it again. But there is a caveat in C. When I now clear my terminal and make this cat, now I've introduced a problem, and there's like more problems it seems than code. Let me scroll back up to the first such error and you'll see this on line 9 of cat. 9, there's an error. Call to undeclared function meow and then something fairly arcane. But that means that meow is no longer recognized as an actual function. I know that it doesn't come from CSST.H, and I know it doesn't come from standard IO.H. It's just down there. But why is the compiler being kind of dumb here? Uh, yeah. defined the Yeah, because insofar as the first version worked, like logically it would seem that putting it at the bottom was just a bad idea because C compilers are fairly simplistic. Like they won't proactively do you the favor of like checking all the way down to the bottom of the file. They're going to take you literally. So if meow doesn't exist as of line 9, that's on you. Like that is an error. So I could fix this by just undoing what I did. Move it way back up to the top, but let me argue that in general when writing C programs, the main function which I keep using and we'll talk more about next week is literally meant to be the main part of your code. And so it kind of stands to reason that it should be at the top because when you open the file, it'd be nice to see the main program that you care about, the main function. So there's an argument to be made that it's a little annoying to have to put my functions all at the top, which is just going to push main further and further down. So there is a solution, and this is dare say the only time copying and pasting is appropriate. Let me delete most of these blank lines, which is unnecessarily dramatic, and just move it below main as over here. The way I can uh the solution here though is to do this to copy the first line of the main function. It's so called signature and then just put that one line and only that one line with a semicolon above main and this is what's known as a prototype. So a prototype is just a bit of a hint to the compiler, a promise if you will, that hey, compiler, there will exist a function called meow. It takes no input and it returns no output semicolon, and it's on the honor system that it will eventually exist later in the file. We'll talk more about this next week, why that works, but this is sort of a promise to the compiler that it will eventually be defined. Now what I've done here on line 4 as an aside is what's generally known as a comment. I just wanted to put on the screen exactly what I was verbalizing. Anything in C that starts with slash slash is a note to self, like a sticky note in scratch, which is just for the human, not for the computer, and it's a way of reminding yourself or someone else what's going on on that line or those lines of code. But I'll go ahead and delete it for now is unnecessary because now if I go back into my terminal and clear those errors, make this cat again now. It does work because the the meow function has been defined exactly where it should be, and now I can make the new version of this cat even better. I could change the function meow to take a variable n as input for the number of times, and then in here I could do something like my 4 loop 4 into i equals 01 less than Ni plus plus, and then in this. For loop, I can print out quote unquote meow, and then I'm gonna have to change this too because I have to copy and repaste it if you will, or just manually fix that. But now I can get rid of all of this and do meow 3, for instance, and this now will be the second version of the scratch code if you will, makeA still gonna work exactly the same meow meow meow, but now I've implemented my own function that does take input even though it doesn't happen to return any outputs. All right, questions. On any of these examples just yet. Confusion All right, let me add one other feature

to this to demonstrate that we can take not only input but actually produce output if we want. If I go back into this code here, let me propose that it's a little silly to be hard coding that is fixating 3. It'd be nice to get input from the user, so I could do this. I could use `in` `N` equals `get in` and say something like. Like what's question mark and then I could pass `N` in if only to demonstrate a couple of things. So one, now the program's dynamic, I'm going to ask the user how many times to meow and I'm going to pass in that value `n`. Now this deliberately is confusing at the moment because wait a minute, I got `N` defined here, used here, but then redefined here and then reused here. So it turns out that even if you create `n` up here and use the name `N`, no other functions can see it for that same issue of scope. So for instance, suppose I didn't quite remember this, and I sort of naively just said `void`, `meow` doesn't need to take any inputs because heck, `N` is already defined in `Main`. Let me go ahead and open my terminal and clear it, make `CA` and see what error comes out here. Well, error oh sorry. I made two mistakes here. Let me, I also have to change the prototype up here to say `void`, which means again `meow` takes no inputs. Let me go ahead now and rerun make `CAT`, and there we have an undeclared identifier again `N`. So in `cat`. C line 14, which is here, it doesn't like that I'm using `N`. But wait a minute, I created `N` here, but for the same logic as earlier, that's fine. You create an `N` on line A, but where does `N` exist? In what scope? Yeah, only between the curly braces, which is lines 7 and 10. So by the time you get down to 14, it's out of scope, so to speak, so it just doesn't work. So the solution is exactly what I did the first time. I can pass it into `meow` as input, and I have to tell C to expect that input, and I can use the same name, but arguably that's going to get confusing sometimes. But let me do this. Let me go back into my code. Let me undo this change such that now `meow` does take an input, but instead of just calling it `N` and using `N` everywhere for number, this is crazy, let's just call this like `times`. So `meow` takes some number of times and then it uses that value, and now I'm passing in on line 9 `N`, but in the context of the `meow` function on lines 12 onward, that same variable `n`. Is now referred to as `times` because you're passing it in as input and giving it its own name and that's totally your prerogative. It's just a matter of scope. I mean I could have called it `M` or some other letter of the alphabet, but `times` is even more clear because that's the number of times I want the cat to meow. But again, the whole point here is just this matter of scope. All right, so let's take a higher level look now at some of the things we've been thinking about, and then we'll do a final deep dive too on some of the corner some of the problems that we can solve with all of these building blocks and some of the problems that we're sort of ignoring for now. So when it comes to writing good code, CS 50 and really the world in general tends to focus on these kinds of axes correctness, design, and style. What does this mean? Correctness just means does the code work the way it's supposed to in the context of a class, it should do exactly what the homework assignment, AKA problem set, tells you. To do in the real world, it should do exactly what someone decided the software should do the product manager, the CEO, or the like. Correctness just means it behaves as it should. That's different though from how well designed the code might be, and we've seen that a few times. I've had some simplistic examples in Scratch and C that were 100% correct, like it did the right thing logically, but I was wasting the computer's time. I was wasting the human's time by asking more boolean expressions than I needed to and so forth. So design is more about like in the in the world. English like not only saying things that are correct but doing it well, like and making a good cogent argument, not just one that happens to be correct. Style meanwhile is the third axis on which we might evaluate the quality of

someone's code, and that's more of the aesthetics like is everything pretty printed, that is nicely indented, are variables well named and not just called X Y Z arbitrarily or something like that. So style matters really to other humans, not to the computer, but to other humans. And to illustrate these. And you'll see that in problem set one onward you'll be given a number of tools that you can use. So one of those tools is called Check 50, and in each problem set, problem in C and Python and other languages, you'll be showed how you can test your own code and you can literally run a command that CS50 created called Check 50. You'll then specify what's called a slug, which just means a unique identifier for that homework problem, and you'll get quick feedback on whether or not your code is correct. It doesn't mean it's well implemented or well designed or. that is well stylized, but at least that's the first gauntlet in getting good code submitted. Design though is much more subjective. Design is something you get feedback on from a human, for instance, in section or a teaching assistant or in software. You can actually see at top VS code there's a couple of buttons that I haven't yet used but could. Design 50 is built on top of the CS50 duck, whereby if you have a program open in a tab, you click Design 50, you will get chat GPT like advice on how you can. Prove not the correctness of that code, but the design of that code, the quality thereof, which is a bit more subjective and modeled after what a good teaching assistant might say. Style 50 meanwhile is a third tool that will provide you with feedback on the style of your code. It will show you on the left what your code looks like and on the right what your code really should look like insofar as it should be consistent with what we've taught in class and consistent with CS 50's so-called style guide. And those of you who have some prior programming experience undoubtedly won't. Like some of CS 50's stylistic choices, and that's going to be the case in the real world too, but as I alluded to earlier, in typical companies you would have an official style guide or tool to which everyone adheres so that everyone's code actually looks the same as everyone else's, even though people have contributed different solutions to problems. So correctness, design, style is not only how we, but really the world writ large tends to evaluate the quality of code, and we do it by way of the CS 50 specific tools here. All right, how about one final flourish then to this here program back in VS code, I've got a correct solution right now. Um, it's well styled. I'll stipulate even though it could stand to have some more comments. So for instance, I could do something like this, like meow uh some number of times, a comment to myself or up here I could say something like, uh, get. A number from user just to remind myself and my TA or my colleague what it is this code is doing, but what more could I do in the way of design? Well, this function here get in will indeed get me an integer, but not just positive or zero, but negative, and I could go in and add a bunch of code like before, like I could actually do instead of this line, I could do something like in and. Semicolon do the following N equals get in and then I can say what's N question mark and then after that I can do something like while N is less than 0, keep doing that so I can have a pretty verbose implementation of getting user input or I can implement another function of my own that only gets a positive integer or non-negative integer from the user. For instance, I might do something like this. Uh, I could, uh, declare at the, uh, maybe below my main function a function like this, uh, in, uh, how about get N and then inside of this I might say void because I'm not going to pass in any input. Then inside of this function is where I'm going to do into and do Y. N equals get quote unquote what's n question mark and then down here I'm going to do while N is less than 0, but rather than do something immediately with N because I'm no longer inside of my so-called main function, what I'm going to do, which is new, is return this value n. And notice

that this notion of returning a value, which is the first time I've done this explicitly, is consistent with this little hint here on my 19, which implies that this get end function which I'm inventing is going to return, not void, which means nothing but an integer, and that's the whole purpose of this function in life now. If I scroll back down here, I can get rid of this whole block of code. And just say get in from the user and then I can immediately call meow with that value. I need to do one other thing. I need to highlight this line of code here, and I'm going to go ahead and add another prototype up top, which is the only time again for now that copy paste is encouraged and best to do. So I've invented my own function, get in, the whole point being now I have this sort of abstraction here of a function whose whole sole purpose in life is to get me not just an integer, but one that is zero or positive and not negative. If I open my terminal window, clear the mess from before, make this catca what's N3, I'm now back in business, and again we've essentially translated from scratch last time into C this time. Exactly how we might modularize now the code, abstract away these lower level details, and ultimately create my own function that as before takes not only arguments but in this case has not only side effects, it doesn't have side effects but rather a return value this time. All right, so as you walked in, we had a little walkthrough of Super Mario Brothers playing from yesteryear, which was a side scrolling game in which Mario would jump down and go up, down, left, right and try to collect coins and make it to the end of the level. There's a lot of obstacles throughout this kind of game whereby the world might look a little something like this, like there's a pit that Mario's got to jump over, and then there's these coins hidden typically behind these question marks that he can jump up and hit his head with and actually accrue points. Now we're not going to do anything graphical just yet. We're leaving. Graphics behind for now in the form of scratch, but with C we can implement some of these ideas. For instance, if I were to write code to generate just this row of 4 question marks, I dare say there's a bunch of ways we can do this. In other words, let's see if we can't use all of today's building blocks to start implementing our own tiny version of Super Mario Brothers in a file, say, called Mario.c. So let me open and clear my terminal window. Let me run codemario.c. And let's just try to do something super simple like print 4 question marks in a row. Well, to do this, I need printf, so I'm going to include standard IO.H. I'm then going to do in Main void. More on that next time. And inside of Main, my default function that just automatically as before gets called for me, I'm going to print out the simplest possible implementation, just print out for. Or question marks like that. So no need per se for a loop just yet, but I think we can go down that rabbit hole too. Let me go down into my terminal window, make this version of Mario Mario enter, and voila. We have a very black and white version, textual version of 4 question marks in the sky. Now I'm kind of cheating. Here by just hard coding 4 question marks, what if I wanted not 4 but 3 or 5 or some other number? Well, we could do that with a loop too. So let me change this code here and do something like this 4 I equals say 0 I less than say 4 for now, I plus plus. Then inside of this loop I can print out one question mark at a time, semicolon. Now let me go back to the bottom, make this version of Mario Mario enter and hmm. It's not actually correct this time. So why am I getting a column instead of a row with this here change? Yeah. Yeah, so I've got, I foolishly included the backslash N after each question mark. OK, so that seems like an easy fix. Let me get rid of that. Let me now recompile Mario, rerun Mario, and now so close. Now I've just done something stupid. All right, I need the backslash N, so I think I do want this here, or What do you propose instead? Yeah, I should really put the backslash in outside of the loop so once I'm done printing all of the question marks, then I get the backslash

in and that's fine, even though we haven't seen this before. Backslash N is an escape sequence that you can certainly print by itself, so I do quote unquote backslash N outside of the loop below those curly braces. Now if I do make Mario Mario, now I get the 4 question marks in a row. As well as the new line at the very end. So again, kind of a little baby exercise, but demonstrative of how you can just take a few different techniques, a few different building blocks we've used to compose a correct solution to what a moment ago was a brand new problem. Well, let's try another. So later on in Super Mario Brothers, when you go into sort of the underground world, you see, or rather it's still above ground, you see a column of bricks. Like this that he has to jump over. So those here, how might we make a column? Well, we kind of had that solution already. And in fact, if I go back to VS code here and just change this version of Mario, I think we can design this thing to be pretty simply the same. I is less than 3 though, and I do want to put the backslash N at the end there, make Mario Mario and albeit textual. I've got my column. Of 3, of let's see, I don't want question marks. Let's make this a little better. Maybe we'll use the hash symbol because that kind of sort of looks like a square. So make Mario Mario. OK, now we're back in business. But let's make it more interesting by going into Mario's underground now and here's the third and final Mario problem whereby we want to implement like this 3 by 3 grid of bricks circled here. So this one's interesting because we've never done something in two dimensions. I did horizontal, I did vertical, but we haven't really composed those ideas into the same. So let me now think a little harder this time about how I can print out row, row, row, and this is where if you have in your mind's eye any familiarity with. Old school typewriters, it's kind of the same idea where you want to print a row of bricks, then go back to the beginning, a row of bricks, then go back to the beginning and a row of bricks, and that's kind of what printf has always been doing for us. It's printing line by line by line of text. It's not jumping around, so we can leverage that perhaps as follows. Let me go into my main function here. And if I want to print out something 2 dimensional, let me kind of think about it as rows and columns. So maybe I could do this for in I equals 01 less than 3, I + plus. Why? Well, I want to do something 3 times, even if I have no idea where I'm going with this solution. I at least want to do something 3 times like 3 rows of text. But how about this on each row, what do I want to do? I want to print out 3 things. So I could steal this idea like I equals 01 less than 3, I plus plus, and then inside of this loop, let me just print out one brick at a time, no new lines yet, one brick at a time, but there is a bit of a problem here. This is correct to nest loops in this way. Totally fine to have an outer loop, totally fine to have an inner loop, but I probably don't want the inner loops variable competing with the outer loops variable by giving them the same name. And that's fine. It is pretty conventional in code when you want another integer, and it's not I because you've used it already, fine. You can use J. So using I and J and K is generally fine. If you're using L, M, and O, like at that point you're probably doing something wrong. There's no hard line, but at some point it gets ridiculous and you should be coming up with better variable names. But I and J, maybe K is fine. So now what's really happening? Let me propose that this is my Uh, for each row, this is my for each column. I want to print one brick. Now this isn't quite correct, but let me go ahead and make this version of Mario Mario, and now there's what, 123, there's 9 bricks there. So I'm close, right? It's supposed to be 3 by 39 total. What do I want to do though to get this just right? Yeah, up on the left Yeah, what, on what line number would you or after, uh, where would I put the new line? Cause I think I don't wanna put it here because I'm gonna get myself into trouble as before, how about in back? After the what? After 13, yeah, so

after I finished printing each brick in the column from left to right, I'm gonna go ahead and print out, I think, a single new line here, nothing else, and now if I open my terminal, run Mike Mario again. I amario, now we've got it, and it's not a perfect square because like this. One is because like the hashtags are kind of more vertical than they are horizontal, but it's pretty darn close. The takeaway here being you can certainly nest these kinds of ideas and compose them, and honestly I and J is maybe making this more confusing than necessary. I could just give these better names like row row. Row and then maybe call for column or column I can spell it out if that's clear column column just to make clearer to myself to my TA, to my colleagues what exactly these variables represent and indeed like an old school typewriter, the outer loop is. Handling row by row by row, but each time you're on a row you first want to do column, column, column, column, column, column, and that's what logically the nesting is achieving. And again, if I do make Mario Mario, all I've done is change variable names, it has no functional effect beyond that. Now this is a little more subtle, but there is a bit of duplication in this program. There's a bit of magic, and this is subtle, but does anyone want to conjecture what still could be improved here? What is maybe rubbing you the wrong way? Yeah, I've hard coded the 3 here and here. It's not a big deal. It's like an in-class exercise. Like who really cares if I'm just manually typing 3? But if I want to make this square bigger and bigger and bigger over time, I'm going to have to change it in two different places and I've conjectured last time and today eventually that's going to come back and bite you. You're going to do something stupid or a colleague isn't going to realize you hard coded 3 in multiple places, like just bad design. So how could we fix this? Well, we could just. Declare a variable like n, set it equal to 3, and then use n in both places, and that's pretty darn good. That's better because now we are reusing the value, but we can do 1 better than this. It turns out in C and in many languages too, there's the notion of a constant whereby if you want to store something in a variable but you want to signal to the compiler that this value should never change and better still, you want to prevent yourself, a human, or Not to mention a colleague from accidentally changing this value, you can declare it to be constant or constant for sure. So if I go back into VS code on line 5 now and say constant, that means that N is an integer that has a constant value. So if I do something stupid later in my code and I try to set N equal to something else, the compiler won't let me do that. It will protect me from myself, so it's just a slightly better design as well. All right, questions on any of these here, Mario. Examples the first of our sort of real world problems, albeit simplified textually. All right, let's focus lastly on things we can't really do well with computers, uh, namely some of the limitations thereof. So here is a cheat sheet of some of the operators we've seen thus far. We played with these with comparison and, uh, doing some, uh, addition or the like, but here we have addition, subtraction multiplication, division, and the modulo operator which is essentially the remain. operator, which you can do with a single command with a single operator like this. Let's use some of these to make our own calculator and see what this calculator can and can't do for us. So back here in VS code, let me open my terminal. Let's go ahead and create a program called Calculator.c. And in this program, let's do something super simple initially that just like adds two numbers together. So let's include first. Uh, CS 50.h so we can use our get functions. Then let's go ahead and include standard IO.h so we can use print F. Let's just copy paste our usual in main void, and inside of Main, let's do this declare a variable X, set it equal to get in, and let's ask the user what's X question mark. Then let's declare another variable Y, set it equal to get in and ask the user what's Y question mark. Let's do something super simple like

give me a third variable, heck, we'll call it Z, set it equal to $x + y$, and then lastly, let's just print out the sum of $X + Y$. So this is a super simple calculator for addition of two numbers. Print F quote unquote. What's the answer going to be? Well, it's not pers. This was quick earlier. What's the placeholder to use for an integer? Percent I backslash N and what do I want to substitute for that placeholder? Just Z in this case we haven't quite done this before, but again it's just the composition of some of our earlier ideas. I can go ahead and make this calculator enter calculator enter what's X is 1, what's Y is 2, and indeed I get 3. So not a bad calculator. It seems to be working correctly, but it's maybe not the best design. Like it's generally frowned upon to create a variable like Z if you're only going to use it. A moment later in one place, like why are you wasting my time creating a variable just to use it once and only once? Sometimes it's fine if it makes your code more readable or clearer, and in fact it might if I called it sum. Like that's arguably a net positive because I'm making clear to the reader that it's the sum of two variables. But even then I'm quibbling, I could just get rid of that third variable altogether and heck, I could just do $X + Y$ right here. That's totally fine and reasonable, especially since it's still a pretty short line of code. It's not. Hard for anyone to read feels like a reasonable call, but this hints at again my comment on design being subjective. There's no steadfast rules here. Some of the TAs might disagree with me, but like, ah, this feels fine. It's readable, which is probably the most important thing ultimately. Let's make this calculator calculator. E 12, and we still get 3. So the code now is still working. As an aside, if you're starting to wonder how I type so fast, sometimes I'm kind of cheating. With autocomplete. So if I know I want to create a program called Calculator, and Calculator.c exists, I can start typing CAL tab and you can hit tab to sort of autocomplete the rest of the file name if it happens to exist there. Better still, if I want to go back to previous commands I've typed, I can actually use my up and down arrows to go through my history. So if I go up, up, up, you'll see all of the recent commands I typed, and that saves me time. 2 so just little keyboard shortcuts that speed things along. All right, well, let's do something like this, not just addition. Why don't we use some multiplication? So how about we prompt the user, not for two numbers, but how about just 1 initially X, and let's go ahead and multiply X by 2, and I would do X, asterisk 2, which is the multiplication operator in C. Let's make this version of the calculator calculator, and now what's X? Let's do 1. So 1 times 2 is 2. Let's do this again. Let's type in 22 times 2 is 4. Let's do this again 33 times 2 is 6, and so forth. That's fine. It seems to work, but maybe let's implement like a recent meme from the past year or two. How about this? Let's, uh, let's see if you recognize it as we go. So I'm going to get rid of this code altogether and inside of my calculator I'm gonna do something like in dollars equals \$1 by default. Then I'm going to deliberately induce an infinite loop just for demonstration's sake. Then I'm gonna do a character from the user and say something like this using get char, which gets a single character. Uh, how about I'll tell the user, here's this many dollars percent I with the US dollar sign before it. Double it and give to next person question mark if familiar with that one, and I'm going to prompt them for a yes no answer, but I'm going to plug in the current number of dollars so they know what they're wagering on. Then below this I'm going to say if the character the human typed in equals equals Y for yes, then I'm gonna go ahead and do dollars times equals 2, which recall was our shorthand notation for doubling something. Uh, in this case, I could more pedantically say equals dollars times 2, but again I can say some keystrokes and dollar equals 2 instead. There's no plus plus. There's no star star trick, asterisk trick, you have to do it in this way minimally. However, if the user does not want to double it and give it to the

next person, then let's do an L and just break out of this infinite loop altogether. But notice what I've deliberately done in Getchar. Similar to Print F, I have included a placeholder. Why we implemented Getchar and get in and get string just like Print F and that you can pass in placeholders and plug in values. Why? Well, again, for the meme's sake, I want to be able to tell the user how much money I'm about to hand them when I ask them the question, Do you want to double it and give it to the next person? I want to see the number and the dollar sign is just because we're talking about dollars. The percent I is because we're talking about integers. All right, if I didn't mess this up, let's make this version of a calculator or meme. So far, so good. calculator, enter. Here's \$1 which was the initial value of my dollars variable on line 6. Double it and give it to the next person. All right, why? Here's \$2. Double it and give it to the next person. OK. OK. OK. OK. OK. I'm going to do it faster. It's getting pretty good. You can see the power of exponentiation. It's getting pretty high. Let's keep going. Keep going, lot of dull, mm mm, too far. That does not happen in the memes. What happened here? What's going on? Yeah, what do you think? Exactly good intuition because the computer only has a finite number of bits allocated to each integer. I hypothesized earlier that it's usually 32 bits, maybe 64 bits, but it's finite, which means you can only count so high and it's roughly 4 billion or again an integer by default can be negative or positive. So it's roughly 2 billion, and that's pretty close to what we were getting here. In fact, we overflowed the integer in memory. In fact, integer overflow is a term of art. Whereby you can overflow an integer by trying to store too big of a value in it, and the reason for this is again to make this clear, this is a piece of memory inside of a laptop or a desktop or some other device, and in these little black chips is a whole bunch of bits or really bytes that can store information electronically, but they allocate those bits in units of 8, maybe 16, maybe 32, maybe 64, but finitely many per value. Whether we're using 32 or 64, you can only count so high if you have a finite number of bits, and we've seen this problem even on a small scale with our flat light bulbs last week. If we have a 3 digit number as represented by like 3 physical light bulbs or 3 tiny transistors in the computer, I can count from 0 to 1 to 2 to 3 to 4 to 5 to 6 to 7. If I want to count to 8 though, I need a 4th bit. But as the red suggests, if you don't have a 4th bit, for all intents and purposes, that number is just 0. Or as an aside, depending on how you're representing your numbers, sometimes a leading one indicates that the number itself is negative, which is why in VS code we actually saw both symptoms. First we went negative because we wrapped around logically, much like that 1 resulted in our getting back effectively to zero, and then we did. Indeed end up on 0 ultimately. So how can we chip away at this? Well, a couple of solutions perhaps. Let me close my terminal window here and instead of using an in, well, let's just kick the can down the road. Let's use a long, which is 64 bits. So at least we can give away even more money in this scenario. I can't use percent I I need to use LI now for a long integer, but I think at this point, if I go back to VS Code's terminal window here, or when I quit that program by hitting control C quickly. Uh, now I'm going to go ahead and do make calculator again calculator, and I'm just going to keep hitting Y. But because I'm using a long in now and thus 64 bits, if I do this long enough, it's going to get crazy high and much, much higher than before, high enough that I'm not going to keep clicking Y enter because we're never going to hit the boundary, but eventually, especially if I did this in a loop automatically, it would certainly, 00. OK, I guess exponentiation works fast. OK, so it did work. I didn't think I was going to hit it enough times, but the same problem happened again. We overflowed this long integer, even using that many bits because I was talking so long I kept hitting Y enough times to

overflow even that long integer. So that too was a problem, and this happens truly in the real world. So picture here is a Boeing 787 from a few years back, long before there were all the more recent problems with planes whereby after 248 days of continuous power, which is kind of a thing in the aviation industry like time is money, and generally they want the planes in the air as much as possible, which means they want them powered on as much as possible, which means they don't like turn them off at night. They keep them going and flying. After 248 days, the New York Times reported a few years back that a Model 787 airplane. That has been powered continuously for 248 days can lose all alternating current electrical power due to the generator control unit simultaneously going into fail-safe mode. This condition is caused by a software counter internal to the GCUs that will overflow after 248 days of continuous power. Boeing is in the process at the time of developing a GCU software upgrade. Will remedy the unsafe condition. So literally what this means is that the power to these planes would just shut off if the planes were on for more than 248 days at a time, and this was a common thing for planes to be maximal power. Why was this actually happening or what was the solution? Well, the short term fix, because it took a while for Boeing to fix this, was what? What would you do if the symptom is that the plane shuts off midflight after 248 days, yeah. Literally turn it off and back on again, much like you've probably been taught with your phones and computers and any other electronic devices that somehow freak out on occasion. Reboot the plane. Now why is that? Well, any time you reboot a phone or a laptop or a plane, all of those variables get reset to their default values, which, if it's the first line of code, like in some of my examples, get set back to zero again. For instance, the first line of code is executed from top to bottom. So this effectively solved the problem. But when they finally rolled out a fix, then you didn't have to do that anymore. But the source of the problem is essentially that they were probably using 32 bit integers, but also negative value, so they had 31 bits at their disposal to count to positive numbers, and 248 days is roughly how many tens of a second there are, which means once you count in tens of a second for 248 days you would overflow an integer and the power would shut off effectively because. Something ended up going to zero, so there was a lot of sort of marketing speak or technical speak in that, but it boiled down to just a simple integer overflow. There's a historical bug in Pac-Man if you've ever played this in any of its forms whereby you can play up to level 255, but because there was a missing if condition that checked what level you were on, you could accidentally garble the screen if you were amazing at Pac-Man because they too would overflow an integer and just random character would end up appearing on the screen, so it's sort of like a badge of honor to actually hit level 256 in this way because of this bug. But there's yet other issues we can see here, and if you don't mind, we might go a couple minutes over, but let me just demonstrate what these examples can do for us here. If I were to revamp my calculator here as follows by clearing my terminal window after hitting control C to kill that, let me go ahead and get rid of all of this meme code here, scrolling down to the inside of main. And let's just do a couple of things like this in X equals quote unquote uh what's X question mark. Then let's go ahead and do in Y equals get in quote unquote what's y question mark. Then let's go ahead and print out just X divided by Y. So here's a percent I backslash N X divided by Y. This would seem to be a calculator now for division, which I can make as before. And actually, sorry, I don't want to do the missing terminating, oh, sorry, missing a double quote there was an unintended bug. So, if I make this here calculator, do slash calculator, type in 1, type in 3. I get 0, which is weird. What if I do instead maybe 2 and 3? It's 0 instead of 0.66. What

if I do 3 and 3? Well, that curiously works, but if I do something like 4 and 3, which would be 1.33, that 2 doesn't seem to work. So there's this other issue in computing when you have finite numbers of bits known as truncation whereby even when you're trying to do floating point math like with a decimal point, if you are using an integer, you're going to. Throw away everything after the decimal point unless you're explicitly using the right data type, and we saw an allusion to this earlier. If I actually go in now and change my values from integers to floats and change the percent I to a percent F and remake this calculator, now I can do 1 divided by 3 and I actually get back that their response. But there's another issue latent here which happens to in the real world whereby I'm going to tweak this percent F to be a little arcane. It turns out. You can see how many digits you want to show how many significant digits you want, if you will, by just using a dot and then a number like 50 arbitrarily. And contrary to what you might have learned in grade school, this calculator would seem to think that calculator 1 divided by 3 is not 0.33333333 infinitely many times. There's all this random stuff happening at the end. Long story short, this is because computers one only use finitely many bits even. To represent floating point numbers and if there's an infinite number of those, you can't possibly represent every possible floating point value. So we're essentially seeing an approximation of $1/3$ precisely, but this too happens quite a bit in the wild. There's really no solution to this other than by throwing more bits at the problem using a double instead of a float, or at least somehow trying to detect this and catch this. That then is what we'd call floating point imprecision. But to tie this together and sort of induce a bit of fear and for the coming years. These things happen all of the time. Back when I was finishing school, there was the so-called Y2K problem or year 2000 problem whereby for decades computers had been using not 4 digits to represent years but just two because it was convenient, it was more efficient because you use half as much memory to represent maybe the year 1999 just using two digits instead of 4. Of course when the year rolled around from 1999 to 2000, if you didn't have these numbers even in memory, you might confuse. 2000 with 1900, which was the presumption if you're only storing two digits. So we screwed that up and thankfully the world scrambled, and if you read up on Wikipedia and news articles from the time everyone thought the world might very well end, but it didn't, so you'd think we'd learned our lesson. Unfortunately, another such problem is coming up in the year 2038, whereby historically since the 70s and prior, computers have generally used 32 bit integers to keep track of time, the date and the time. By means of counting how many seconds have passed since January 1st, 1970, and all of the math is just relative to that date because that's when computers were really starting to come onto the scene, if you will. Unfortunately, there's only 4 billion values you can count to or 2 billion if you're doing negatives from January 1st, 1970. And so, um, on the date January 19th, 2038, we will overflow a 32 bit counter and suddenly if this problem is not. Fixed by you or other people before the year 2038, our computers and phones and other devices may very well think it's December 13, 1901. So there are solutions to these problems. CS 50 is all about empowering you with solutions to these problems, but if you'd like to scan this here code, um, this will add that date to your Google calendar or your Outlook calendar. Keep an eye on it. That though is week one for CS 50. Problem set one will be in your hands soon. We'll see you next time.

Week 2

1 fish, 2 fish, red fish, blue fish. Congratulations. Today is your day. You're off to great places, you're off and away. It was a bright cold day in April, and the clocks were striking 13. Winston Smith, his chin nuzzled into his breast in an effort to escape the vile wind, slipped quickly through the glass doors of victory mansions, though not quickly enough to prevent a swirl of gritty dust from entering along with him. Alright, this is CS 150, and this is week 2, and if we could after this dramatic reading, a round of applause for our volunteers. So we can now take for granted from week 1 that we now have a new way to express some of the ideas that we first explored in week 0 like functions and conditionals and variables and the like, and now we're doing in see what we used to do in Scratch. Today what we're going to start to focus on is some real world problems so that we can take for granted that we have that expressiveness. We have some tools in our toolkit and actually start to solve some real world problems. Problems if representative thereof. In particular, the real world problem that we're going to start today and this week with is that of reading levels. Odds are when growing up you read at a certain level based on the age at which you were at. Maybe it was 1st grade level or 5th grade level or 10th grade level or the like, and that was a function of just how comfortable you were with the words in the book or words on the screen that you were reading. What you've just. Thanks to our volunteers are 3 different reading levels that each of these 3 volunteers reads at. And in fact, why don't we go ahead and hear them again and be a little more thoughtful this time as to assess at what reading level your classmate is reading. So let's start with Leah if you'd like to introduce yourself first. Hi, I'm Leah. I'm a 1st year in Holworthy, and here's my little thing. 1 fish, 2 fish, red fish, blue fish. So what reading level would you say Leah reads based on her recitation thereof, yeah, in the front. Kindergarten, OK, so a fairly young age, and what makes you say kindergarten? She is speaking in very short phrases without much complexity. OK, very short phrases without much complexity, and indeed, according to one scientific measure that we'll explore in this week's problem set, indeed, we would say that Lea reads before grade one, so kindergarten. Indeed be apt, but welcome to the stage here. Let's move on now to Maria if you'd like to introduce yourself. Yeah, hi, I'm Maria. I'm in Stoughton thinking of applied math. Um, congratulations. Today is your day. You're off to great places. You're off and away. Another familiar phrase perhaps at what reading level would you say Maria is? Yeah, over here. And what makes you say 2nd or 3rd grade? OK. So now we're starting to introduce complexities like rhyming and a bit more substance to the quote. And indeed based on that reading that same measure that I described earlier, which will involve a mathematical function that somehow analyzes what it is Maria just said, indeed we would conclude that she read at a 3rd grade level or grade 3. Finally, Omar, if you'd like to introduce yourself and read once more yours. OK, um, so hi everyone, I'm Omar. Um, I'm a freshman at Hurlbut thinking of doing Cosci, and this is my reading. Um, it was a bright cold day in April, and the clocks were striking 13. Winston Smith, his chin nuzzled into his breast in an effort to escape the vile wind, slipped quickly through the glass doors of victory mansions, though not quickly enough to prevent the swirl of gritty dust from entering along with him. All right, sort of escalated quickly. What reading level is Omar at, would you say? Someone else. What might you say or estimate? Yes, right here in the front. OK, 8th grade, and what made you say that? Uh More complex sentences, more complex words, and indeed, according to that same measure, this full paragraph of text now, which indeed has even more grammar when you see it there on the screen, would be said to be at grade 10 because of that added complexity. So with that said, we're going to need to be able to

somehow sort of crunch these numbers to determine, given a body of text at what reading level someone is, but in order to do that and apply any metrics to a body of text, we're going to need to represent that text in memory using something like strings. Last week, but last week with strings, we could really just print them out or display them wholesale on the screen. But I think we're going to need to break down these various texts and others like it at a finer grade level. And indeed among the goals for today is to explore exactly that and also to take the proverbial hood off of the car to take a look underneath and how the computer is actually working, how these things like strings are actually functioning. So if you could join me one last time in a round of applause for our volunteers, thank you so much for helping out. Thank you guys. Thank you. Thank you to Maria as well. So among the goals for today beyond exploring a representative problem like this of reading levels is going to be another one which is even more important and more omnipresent than reading levels, namely cryptography, the art of scrambling information or specifically encrypting it so you can send secure communications. Now you sort of take this for granted increasingly nowadays that when you send a text message or perhaps an email or check out online with a credit card that somehow Other your information is secure and over the coming weeks we're going to explore to what extent that is actually true and why or why not. Now with cryptography, similarly too, if we want to be able to send messages securely such that if I want to send a message to you, I don't want anyone else in the room to be able to figure out what it is I have said, even if they physically intercept that message, which is all too possible in the digital world, we're going to need to come up with metrics. And mechanisms for actually scrambling information in a reversible way so that I can write my message, somehow scramble it. You can receive that message even if after it's passed through many other hands, and you can descramble or decrypt that same message. So for instance, here on the screen is a message, a fairly simplistic one, that has somehow been encrypted, and we'll see by the end of today and by the end of this week that this encrypted message, and there's a bit of a tell on the end there, actually will be said to decrypt too. This is CS 50, but why is going to be the underlying question and what additional tools do we need on our toolkit in order to do that. Another word on tools. So up until now you've probably experienced some bugs, whether it was in scratch or ever more so in C. In fact, don't feel too bad if like the very first program you wrote in C like didn't even work. You couldn't even make it or compile it until you went back and fixed some of the code that you had written. Well, it turns out that bugs, mistakes, and Programs are ever so commonplace and even though we've already provided you with tools like the virtual rubber duck at CS50.AI also embedded into VS code at cs50.dev, of whom you can ask questions along the way. Among the goals today are just to give you some lifelong tools at how you can actually debug software yourself when you don't have a duck nearby, when you don't have a TA nearby, let alone any humans at all. So with debugging, there's going to be a number of techniques that we can use all toward an end of like finding and removing. Bugs or mistakes from our software and perhaps the person best known for having popularized this term of bugs is that of Dr. Grace Hopper, pictured here, who was a rear admiral in the navy and was one of the original programmers of the so-called Harvard Mark 1, a very early mainframe computer that if you wander across the Charles River over to the science and engineering complex here at Harvard, you can actually see part of this on display still in the lobby. It was succeeded by the Harvard Mark II, and on the Harvard Mark 2. Hopper and her team were known for having put this note in their logbook after having done some number crunching on the system there. And if

we zoom in, they have found a problem with the computer this one day whereby there was literally a bug, a moth inside of the circuitry of the computer, and as was written here, first actual case of bug being found. And ever since then do we say ever more so the phrase bug and debugging when it comes to finding and eliminating problems in our code. So let's start. With just that, in fact, let me go over to VS code and let's deliberately make some mistakes together that might very well be reminiscent of some of the mistakes you've accidentally made thus far, but along the way give you all the more tools for solving those problems as opposed to sort of having to ask someone else, be it virtual or physical, for help, and actually find these mistakes in your own code. Let me go ahead and consciously in VS code create a program known to be buggy called buggy.c. And in this program, let's go ahead and do some fairly Familiar code initially. I'm going to go ahead and start just like we did last week with In Main Void. More on that today before long inside of my curly braces, I'm going to say print F, hello, comma, world. That's it. Now I'm going to go back to my terminal window here. I'm going to go ahead and do make buggy to make a program from that source code, but before I do, odds are even after just a week of this stuff, you can probably spot a few mistakes I've made, a few bugs. What do you see wrong already, yeah. I didn't include standard IO. H, that so-called header file, which is important because it tells the compiler that I plan to use functions therein like printf, which clearly I'm doing. So let me go in and include stdio.h. What else seems to be wrong here? Yeah. I'm missing a semicolon at the end of line 5 here, so I'm gonna go ahead and add that in, and this is subtle and arguably not a bug, but maybe an aesthetic detail. What else have I done arguably wrong, yeah, and back. Yeah, I forgot my backslash and the new line character just to move the cursor to the next line so that when I get a new prompt, it's on a fresh line of its own. Again, more of an aesthetic, but certainly a pretty reasonable thing to do. So let me go ahead now and actually in my terminal window run Make buggy and indeed compiled. But up until then had I not fixed those mistakes, I would have triggered a whole bunch of bugs, a whole bunch of error messages as a result. In fact, let's rewind in time and undo the fixes. Just made and go back to the original form here and try running again, make buggy enter and we'll see some scary looking messages up here. Let me scroll up to the top of the output here where we see buggy.c:3, which means line 3. That's where the problem is right now. error. Call to undeclared library function, printf with type, and then it starts to get a little more complicated, but I do see clearly that it's calling my attention to printf. So hopefully at some point, if not last week, hopefully this week onward, your instinct will be, oh, all right, I'm an idiot. I forgot the header file in which printf is actually declared. It's not a huge deal. It's going to come with practice, so that's how I might know in more intuitively what in fact the solution here might be. Now here's another common mistake that I've just gone in and fixed, but I did do something wrong and hopefully none of you actually did this because it's an annual FAQ. What did I just do accidentally wrong? So it's not stdio.h, it's stdio.h. So do kind of ingrain that one for standard input output. The next though bug that I haven't yet fixed is that semicolon. So let me clear my screen and rerun Make Buggy. I should no longer see that first error message anymore, but I now do see another error message online. 5: expected semicolon after expression. All right, that one's pretty explicit, so I'm going to go ahead and fix this, but notice that up until now my code wouldn't have been able to compile because of those two error messages. It stopped showing me by showing me these errors, but at this point, if I run make buggy enter. It did in fact compile and yet it's arguably still buggy because when I run buggy I

get my prompt on the wrong line. So this is a distinction now between a syntax error, something that or a programming error that outright stops my program from compiling. It's sort of a deal breaker versus something that's maybe more of a logical error. I actually meant to move the cursor to the next line and so there's different. Types of errors in the world as we're seeing here of course if I rerun Make Buggy Against slash buggy, now we're back in business hopefully with the intention of having this, uh, display exactly that. All right, well, let's modify to look a little more like something else from last week. Recall that last week I started to get someone's name more dynamically, so I said something like name equals get string. And that was a function we introduced, and I might have said something like this what's your name? question mark with a space just to move the cursor over. I know now I definitely need to end my thought with a semicolon. I could try and compile this. Make buggy now and I'm seeing a different error message altogether that you might not have seen yet. So on buggy.C line 5 error use of undeclared identifier name. What now is the mistake that I've made? Why does it not know? Yeah, I forgot to declare the type of this variable, which for those of you with the prior programming experience is not something you have to do in some languages like Python, for instance, but in languages like C, C++, Java, and others you do in fact need to explicitly tell the compiler that you want to instantiate a vari. create a variable in the computer's memory by telling it it's type, and it's not going to be an in because I don't want an integer, of course, in this case I want text which we now know to be called string instead. All right, I think this fixes that bug, so let me do make buggy again and hopefully. Huh, a fatal error this time again indicating that my code did not recompile on line 5 still, I have an error, but this time it says use of undeclared identifier string. Did I mean standard in. So this is a bit of a red herring. The compiler's trying to be helpful in saying did I mean standard in, but I don't think I actually do. That just is the most similar looking word in the compiler's own memory. What's the actual mistake that I've made here? Yeah. Yeah, I didn't include the CS50 header file because string recall is a feature of the CS 50 library, as is get string and get into and others. So the solution here is indeed to go up here and just to be nitpicky, I tend to alphabetize my header files. It's not strictly required technically, but stylistically. I find it nice to be able to skim the header files alphabetically to see if something is there. Not I can include CS50. H in addition to standardio.h and it's in that file CS50.h that not only is get string declared so that the compiler knows that it exists, it turns out so is the word string. So this is a bit of a white lie, and this is something we do in the early weeks of the class. We dug up these old training wheels from a bicycle, the whole idea being to sort of keep you up and avoid you having to do. Too much complexity early on. The point of these training wheels in the form of the CS 50 library is to let us kind of ignore what a string really is for just another week or two, after which we will then peel back that layer, take off those training wheels, and reveal to you what is actually going on. So for now, strings exist, but they exist because of the CS 50 library. In a couple of weeks. They're still going to exist, but we're going to call them by a different name, as we'll eventually see. But everyone in the real world, every software developer uses the phrase string. So this is a concept that exists. It is not CS 50 specific at all. It's just that in C, the word string doesn't typically exist unless you make it so as we have. All right, so I think now if I clear my terminal window. And rerun make buggy. Now it should in fact compile and if I runs buggy enter, I should be able to type in my name and now voila, hello. So this is now not a syntax error because I didn't screw up my code per se like it compiled. Everything is grammatically correct, so to speak, but logically, intellectually, this is

not what I wanted, right? I wanted it presumably to say hello, David, so let's fix one final bug here. How do I fix this? On what line? How do I get it to say yeah hello David. Yeah, on line 7 I need to do the string placeholder, the format code, so to speak, percents, and then one more thing, someone else. What do I do after this? Yeah, and back Yeah, comma and then add the variable name that contains the value I want to substitute in there, which is indeed name, though I could have called it anything I want. All right, so now make buggy enter seems to have compiled again buggy. Now I type in my name once more, and now we're back in business. So over the course of these few exercises, clearly I meant to make most of all of these bugs, these mistakes, but they demonstrate not only syntax error. which are just going to stop the compiler in its tracks. Like you won't even be able to compile your code until you fix those things. But even after that there could be these latent bugs that seem to not be there until you actually provide input and see what's actually happening at so-called run time when you're running the actual code. And so here's where it's no longer as easy as just reading the error message and figuring out what it means because there There is no error message that appeared on the screen when it said hello, comma world. We had to use our own human intellect and realize, OK, that's clearly not what I wanted. Had you run CS 50's own Check 50 program on something like that, we could have told you that that's not correct by automatically assessing the correctness of it, but the compiler has no idea what you are trying to achieve logically. It only knows about the language C itself and the requisite syntax for actually. Uh, writing and compiling code. So how could we go about solving logical problems in code? So I would propose that we start to consider this here list whereby when you want to find a logical problem in your code and better understand what's going on or really what's going wrong, printf is going to be your friend. Up until now we've used printf to literally print on the screen. Hello David. Hello Kelly or anything else on the screen, but you can certainly use Printf temporarily to just print stuff out inside of your program that you might want to better understand. And then once you understand it and once you've solved some problem, fine, then you can delete those temporary lines of code, recompile, and move on. So let's use printf as a debugging tool in that sense. Let me go back over to VS code here and let me in this same program. Buggy.c sort of delete everything and start over with a different sort of bug. I'm going to include standard IO. H at the top. I'm going to do in Main void after that. And then inside Maine I'm going to do a simple for loop that just prints out like a stack of 3 bricks like we saw in the world of Mario when Mario needed to, we claimed sort of jump over a stack of bricks. We want to print out just 3 of those at the moment. So I'm going to go ahead and say 4 in. I equals 0. I is less than or equal to 3 because I want 3 of these I plus plus. Then inside of this 4 loop I'm going to go ahead and quite simply do print F hash symbol to represent the brick followed by a new line to move the cursor to the next line, semicolon to complete the thought. Now I've deliberately made a stupid mistake here, but in the context of a simple enough program that we can focus on the debugging technique on not on the obscurity of the bug in question. Hopefully you'll spot the bug in just a moment if not already. When I do make buggy now and do buggy, I don't get 3 bricks. I of course get 1234 total. So there's a logical bug in this program, and odds are you can already spot what it is. But let me propose that this program is representative of a type of problem that you can solve a little more diagnostically by poking around and really asking the computer via print up to show you what's really going on, and I would. that one of the most helpful techniques in a situation like this, if you're trying to wrap your mind around why are there 4 bricks instead of

3, well, clearly this is related to the loop somehow. So let's look a little more thoughtfully at what the value of `i` is before we print out each of those bricks, and I might literally do something like this temporarily `printf("i is %d\n", i);` and then I could just print. Right here and now, the value of `i` just so that I can actually see it. Let me now go down into my terminal window, make buggy again buggy, and now, and I'll full screen my terminal, I'll get some diagnostic information at the same time. So when `i` is 1, I get a brick. When `i` is 0, I get a brick. When `i` is 1, I get another brick. When `i` is 2, I get another brick. When `i` is 3, I get a 4th brick. So now I can kind of see that, OK, my loop is working, but I'm. Going too far. I'm going too long now. I can do this even more succinctly for what it's worth. I don't need a whole new `printf` statement. I could just go into my existing `printf`, put my `%d` there, and then maybe just space just to scooch things over, and then print out `i` in that same line. If I now do make buggy buggy, OK, now I'm seeing that I'm printing a hash, a brick for each value of `i` from `i` equals 0 to 3. So the solution of course is that I shouldn't be starting at 0 and iterating less than or equal to 3. The solution is like, oh, I'm an idiot. I should have said less than 3, or if I prefer to count starting at 1 like a normal person, I could have said `i` equal to 1 and then go up 2 and through 3. But as I claimed last week, the canonical way, the most common way to do this is start counting at 0 and go up 2, but not through the total value that you have in mind. But there's going to be another technique that's worth knowing here. Let me go ahead and sort of abstract this away by whipping up a slightly better variant of this as follows. Let me go ahead and delete this `for` loop. Let me assume for the moment that inside of `Main`, I'm going to ask the user now for the height of a pyramid, and I'm going to do something like this. `int H; get_in(&H);` and let's prompt the user. for the height value of this pyramid or this wall, and then let's go ahead and assume there exists a function called `print_column` who takes as input a number `H`, which is how many bricks you want to print. Now this function does not exist yet `print_column` get in does exist, but I don't have access to it, so let me not make the same mistake twice. What do I need to add? At the top of this file, yeah. I need the CS 50 header file because I'm using the `get_in` function now, which again comes from our library, not C. So let me go ahead and include `CS50.h`. But now `print_column`, I can invent this function myself. So let me go ahead and say `void print_column(int height);` More on that in just a moment. And then I'm going to recreate the loop from before `for (i = 0; i <= height; i++)`. I is less than or equal to the height, so I'm going to deliberately for now make that same mistake as before `i++`, and then inside of this `for` loop I'm going to go ahead and print out a single hash and a new line to represent that there brick. So now `Main` can use a function called `print_column`. It's going to pass in the value of `H` and then this `for` loop in the `print_column` function is going to take care of printing this thing for me. So let me do this again. Make buggy enter. So far so good. buggy. Let's put it in `height`. I'm going to say manually height of 3, and I should see 3 bricks, but of course I'm still seeing 4. Now before we move on, let me hide my terminal and propose that this is just kind of stylistically bad to put anything other than your main function at the top, but recall that if I move my helper function `print_column`, and it's a helper function insofar as I made it to help me solve another problem, I can't recompile and run my code now. Why? The compiler won't let me, yeah. Exactly when the compiler gets to line 7 of my code, it's going to abort compilation because it doesn't know what `print_column` is. Why? Because I don't tell it what it is until line 10, and this was the only time I proposed that copy paste is reasonable is to highlight and copy the very first line of that function, paste it above main with a semicolon, and that's a so-called function prototype. It

specifies what the name of it is, what its inputs are, if any, and what its output is, if any. And more on these inputs and outputs later on. But now this is just a more complicated but more modularized version of the same program. Let me do make buggy, still compiles buggy, type in 3, and I still have that same bug. But the catch now is that my code has gotten more complicated, and the point of my having abstracted away this idea of printing a column into a new function is that there's just more code now to debug. I could certainly go in there and start adding print. But at some point printf is going to be a very primitive tool, and you're going to waste more time adding printf's, recompiling your code, running your code, changing the print F, recompiling your code, running your code. It's going to get very tedious quickly when you have lots of lines of code on the screen. So can I actually step through my code line by line, maybe like your TA would in a section or a small class, line by line walking through the code? You can because another tool that you have access to. Is that called debug 50. So this is a CS 50 command that will start an industry standard debugger, and a debugger is a piece of software that is used in the real world that literally lets you do that debug your code by letting you slow down or even pause execution and walk through execution of your code line by line. The only reason we call it debug 50 is because in VS code it's a little annoying. To start the debugger and so we automated the process of starting the debugger, but everything thereafter has nothing to do with CS 50 and everything to do with real world software engineering techniques. So how do we use this? Let me go back to VS code here and let me propose that I want to step through this code line by line just like we might at a whiteboard in a smaller class to figure out why I'm getting 4 instead of 3 hashes. Well, in my terminal window. What I'm going to go ahead and do is this debug 50 space buggy. So debug 50 is the command. It needs to know what program I want to debug. So I'm specifying buggy, which is the name of the program I just compiled. I'm going to get an error though with the first time I run this, as will you if you make the same mistake. I'm about to see this message here. Looks like you haven't set any break points. Set at least one break point by clicking to the left of a line number and then rerun debug 50. So what is this really telling me? Well, the debugger has no idea when and where I want to pause execution so as to start walking through my code line by line. It wants me to tell it where to break, that is where to pause by clicking on a line number. So let me hide my terminal for just a moment, and you've probably never done this intentionally. But if you hover over the space to the left of your program's line numbers, you'll see a little red dot, a little stop sign of sorts. If you actually click on a line number, that red dot will stay there, and you can see the hover here saying click to add breakpoint. What I'm going to go ahead and do is say click to add a breakpoint at main. Main is the entry point to my program. It's the default function that gets called. Let's break right away so I can step through this code line by line. All right, let me reopen my terminal window. And clear it and then run debug 50 again with slash buggy enter and now a whole bunch of stuff is going to happen quickly on the screen and then it's going to clean itself up because once the debugger is running and ready to go, it's going to allow me to start stepping through my code line by line. So what is going on? Well, notice nothing has happened in the terminal yet. Why? Because my code has been paused inside of main. In particular, it's been paused in the first real line of code. So the curly brace is uninteresting. The first line is just the function's name essentially. So line 8 is the first juicy. Of code that could possibly do anything useful. It's been highlighted here in yellow, and that the fact that this cursor is here means that we have broken execution on this line, but we have not yet executed this line, which

is why in the terminal I don't see anything yet. I definitely don't see heights followed by colon. Notice what else has happened here. All of a sudden in the left hand side of the screen where your File Explorer typically is or where the CS 50 duct typically is, we see mention of variables. You can actually see inside of the debugger what the value of any variable in the computer's memory happens to be. Now I don't quite understand this right now. We'll come back to this over time, but weirdly, before line 8 even executes, it seems that H has a default value of 32,764, which seems to have come from nowhere. As an aside, this is going to be what's called a garbage value, and this is actually why we have Oscar so omnipresently here. A garbage value tends to be a default value inside of a variable that's the result of that memory having been used previously for something else. Inside of your computer, you've got all of this memory, random access memory or RAM. More on that today, and it stands to reason that my computer or whatever cloud server we're using has been running for some time. So the bits that H is going to use might already have some random switches on and off, some random pattern of bits that happens to give me 32,764. But the moment this line of code executes, that value is going to get changed to what I actually want it to be, which is what the human is going to type in. Meanwhile, at the bottom here you'll see a so-called call stack. More on this too in the weeks to come, but you'll see that we've paused on the function called main in the file called buggy.c. So how do I do something useful? Well, at the very top of the debugger, you'll see a whole bunch of color coded icons. One looks like a play button, and if I click that, it's just going to continue execution of my code as though I don't want to step through it anymore. So I'm not going to click that just yet. The second arrow, which is a little curved arrow over a dot, is the so-called step over line, which will mean step over this line and execute it, but only one line at a time. Let's go ahead and do exactly that. So I'm going to click the step over icon, the second one, which is the curved arrow with the dot under it. Click. Now I see in my terminal window height being prompted. All right, let's go ahead and type in 3 just like I did before and hit enter. Now notice what happens. Execution has paused on line 9 instead of 8. And you'll see that my variable, a so-called local variable, has the value of 3 as intended. All right, so far this isn't all that enlightening other than demonstrative of the fact that I can pause execution of my program anytime I want. So let's now click that step over button again so that we actually print this column. Click. And there we have it, 4 hashes at the bottom of the screen. Now execution is paused at the end of the function. This is just my opportunity to either stop or restart or continue. I'm just going to go ahead and click the play button and let it finish executing. Unfortunately that wasn't really at all in in. Lightning except to confirm for me that I typed in 3 and 3 is what is in the computer's memory. Not that interesting though yet. So let's do this. Let's leave the break point on line 6 as before. Let's rerun the debugger by running `debug 50buggy`. Let's let it do its startup thing, which looks a little messy at first, but now we've highlighted line 8 again. I'm going to go ahead and step over this line because I do want to get an N. I'm going to type in 3 again enter, but this time instead of stepping over line 9 and just letting print column happen, this is where the debugger gets powerful. Let me step into line 9 and walk through the print column function itself line by line. So let me go ahead and click not this button, which is the curved arrow over the dot, but the next one, which is the step into button. Click. And now you'll see that execution has jumped inside the print column and paused on line 14, at which point I can see at top left what the default value of I is, and this is some crazy garbage value because whatever bits are being used to store I's value have some random garbage from some previous use of

that memory. But as soon as line 14 executes once, I bet I is going to take on a value of 0. So let's do that. I'm going to go ahead and click step over. Because I don't need to step into this because there's no other functions there. Step over it and immediately at top left I is now 0. Now line 16 is highlighted. Let's step over this, OK? Not in the terminal window what do you see? The first of our hashes. Let's step over, step over, second hash, and I is now 1. Step over, step over. Now we see a third hash and I is now 2. Step over, step over. OK, there's the symptom of the bug. 4 hashes and yet I is 3, but wait a minute, this is going to draw my attention now to line 14 before I continue on or wait a minute. 3 is of course less than or equal to 3, which is why I got that 4th hash on the screen. So at the end of the day, like you still need to exercise some of your own human intellect to figure out. And understand what's going on, but the value of this here debugger is that you can pause and work through things at your own pace and poke around inside of your own code and better understand what's happening as opposed to compiling the program, running it, and just now having to infer from the symptoms alone what the source of the problem might be. So that was a lot. Let me go ahead here and just let it continue to the end because I know what the problem is now. I need to change the less than or equal to sign to a simple less than instead. Questions though on debug 50 or any of these steps, yeah. Sure, um, could you go over what the break point. And then my second. Mhm. Correct. So in order of your questions, what again are these break points, the break point or the little red stop sign here just tells the debugger where to pause execution. So frankly, I didn't have to break pause execution at Maine. If I really care about debugging print column, I could have clicked down here instead, and then it would have just run main automatically and only paused once print column gets called. So a break point is where your code will break, the point at which it will break. As for the garbage values, I'm it's I'm oversimplifying exactly what's going on inside of the computer's memory, and it's not necessarily using exactly the same memory as before, but the operating system will govern exactly how the memory is laid out. This is actually a significant problem, long story short in a lot of today's systems because it's not that interesting to me to know that there was 32,000, whatever that number is, or. Negative number, but suppose that that revealed the password of someone, another program or function that had some information there. It seems all too easy with the debugger, let alone see, to actually poke around the computer's memory, and we're going to come back to that in a couple of weeks. But for now it's a garbage value insofar as you didn't put the value there, it somehow got there on its own for now. Other questions. Some like when it's 0 goes to 1 at the end of the 4, but when it goes the 4 the next time because. After you Correct. So the question is about the order of operations for a 4 loop. So the first time you go through a 4 loop, the initialization happens, the stuff before the first semicolon, and the condition is actually checked, the Boolean expression. Then everything inside of the curly braces is executed, then the incrementation or update happens. Which in this case is I + plus and then the condition is again checked, the Boolean expression, the code is executed, the update happens, the condition again, the code is updated, and so it starts to loop like this. The debugger's graphics are fairly simplistic and it just highlights the whole line without making super clear what's happening, but that's just the definition of a for loop. Good question. Others about debug 50 or print death? All right, yeah. the And uh Can you change the position of I plus and height? Short answer no. The first thing is the initialization, the variable you want to create and initialize. The second thing is the actual condition, the so-called boolean expression. The third thing is always the update. So it must come in this order. What

you're not seeing is that you can actually have multiple boolean expressions, you can have multiple initializations, you can have multiple updates, but we're keeping it simple for now. And this is canonical. All right, so to make clear, assuming that either `printf` or `Debug 50` helped me figure out where the illogic was in my thoughts, I now know that the fix here is to just go and change the less than or equal to to a simple less than, and if I run the program again, of course it's going to give me the 3 bricks that I always wanted instead. But there's other techniques we can use too. So besides `printf` and `debug`, you might wonder why we have a 7 ft duck behind me here, all of these little rubber ducks on the floor. So rubber duck debugging per week zero is actually a thing. This was popularized in a book some years ago, and the idea is that when you are facing some bugs, some mistake in your program, or you're just confused on some concept, there is anecdotal evidence to suggest that just talking out the problem with an. An inanimate object like a rubber duck on your desk is enough often for that proverbial light bulb to go off over your head because you hear in your own words what confusion you're having, what illogical thoughts you're having, and you don't even need another human or TA or AI in the room to answer the problem for you. So in fact, on the way out today at the end of class, we've got hundreds of ducks and enough for everyone to take home with you if you'd like to use that as another debugging technique, whether in 50 or something else, but of course now in the age of AI, you also have the AI powered virtual duck at CS50.AI and also in VS code at CS50.dev, which really is a mechanism for asking questions that you don't think you can solve on your own. So it might be reasonable to ask the duck what does this error message mean if you're having trouble wrapping your mind around it, but it's less reasonable to say copy paste your code into the duck and say. What's wrong with my code? You should really be meeting the AI halfway. After all, what's the point of actually doing this or any other class is to develop that muscle memory, develop those mental models, get some practical skills. So try hard to walk that line between asking the duck too much versus deploying some of these same tools yourself, `printf` `Debug 50`, even a physical rubber duck on your desk before you resort to sort of escalating it to human-like or duck help. All right, so with those tools added to one's tool kit, let's actually consider and reveal what's been going on underneath the hood since last week. So this was the mental model we proposed for last week, whereby when you write source code in a language like C, it's not something that the computer itself understands natively because computers we saw only understand zeros and ones, AKA machine code. So the compiler is the program that we use to convert your source code. To the machine's code from C to zeros in 1 in this case. More generally, a compiler is just a program that translates one language to another, and in this case we're going from source code to machine code. So let's consider what's really happening. And indeed this is among the goals of this week is to take a look at a lower level so that when you encounter more interesting, more challenging problems, you'll understand from so-called first principles what the computer is actually doing and supposed to do. So you can deductively figure things out for yourself and generally not view computers as like magic or I don't know how this works. You'll have a fairly bottom up sense of how everything works by term's end inside of any computer, laptop, desktop, phone, or the like these days. So here's the simplest of programs that we wrote last week, even though there's a lot of syntactic complexity as we've seen. The goal is to get it to machine code. These here zero. And ones. So how has that been happening when you just run `Make` since last week? Well, these are the two commands that we've typically run after creating a file like `hello.c`. We then compile with `Make hello` and then we

run it with hello. So let's give ourselves this starting point real quick just so that we have an example in mind of exactly what it is we're compiling. So let me go back to VS code here. Close out buggy.c. And let's create a new file just like last week called hello.c inside of which is our old friend Standardio.h in Main void, and then inside of this we'll keep it simple, just printing out hello world, which again is my source code in C. How do I now actually compile that? Well, of course I can go down to my terminal window, make hello hello, and we're off and running. So it was a bit of a white lie for me to let you think though that last week the compiler itself is called Make. Make is a command that literally makes your program. It makes it by compiling it, but make is not technically the compiler. If we really want to get nitpicky, the compiler you've been using is actually called Clang for C language, and this is a very popular compiler freely. All open source, so to speak, you can even look at the code other humans wrote to create the compiler online, and what Make is really doing for us is essentially automating this command. So all this time I could have just run clang space hello.c, but the default file name from Clang, the compiler, we and for historical reasons it's not going to be hello, as you would hope it's going to be a do out for assembler output and we don't do this in the first in week one of the class because like this just makes things unnecessarily complex that we're adding some random name that you just have to know to type. However, we can do this now as follows. Let me go back to VS code here. Let me clear my terminal and type LS and we'll see everything we've created thus far buggy.c, which when I compiled it, I got buggy, and hello.c, which I just wrote, and when I compiled it, I got hello. Let's do this command now manually though. Let's use clang on hello.c and hit enter. That too seems to work, but if I now type LS you'll see a third program specifically called A.out, which happens to be the same as hello, it just is using the default name instead of my custom name hello, but if I do slash A. out, indeed that too will work. But the reason we don't do that certainly in the first week of the course is that things get a little annoying or sort of escalate quickly thereafter. So let me go ahead and change this program as we've done a few times already. Let me include CS50. H so that we get access to like get string. Let me do string name equals get string, quote unquote. What's your name? question mark, close quote. And then down here just like before, let me add my percent S and add in my name. So I did that super quickly, but it's the same program we wrote a few minutes ago and it's the same one we wrote last week. What happens now though is as follows if I now try to do clang, hello.c enter, I actually get. An error message, this one perhaps more cryptic than most. Somehow or other I have this error Linker command failed with exit code one because of an undefined reference to get string. Now in the past when we've seen undefined or really undeclared mentions of get string, the problem was just with missing this line. This line is clearly here. But the catch is I'm getting this error message now because when I run clang of hello.c, I'm just assuming that clang knows where to find the CS50 version of Get string, and that is not the case. Technically, if I want the compiler to compile this code for me, what I'm actually going to have to do is this. Let me go back to uh my terminal window here and I'm gonna say clang. Hello.c, but I'm then going to specify LCS 50, which is cryptic at first glance, but this is telling the compiler to link in the CS 50 library so that it knows what the zeros and ones are that belong to the get string function. Long story short, if I hit enter now, the error message has gone away. If I type LS, I've still got a. out, but it's. New version thereof, and if I do a out now I see the new behavior where I can type in my name and see hello David. Now this is getting a little stupid that I keep using A. out. We can change that as well. In fact, these commands, as we're starting to see, support what are

called command line arguments, and a lot of the programs we've run already take command line arguments when we run code space hello.c, the so-called command line argument. To code is hello.c. When I run make hello, the command line argument to make is hello. In other words, the command line arguments to a program are all of the words you're typing in your terminal after the name of the program itself, whether it's make or whether it's code or any. Else, so this is to say what I just ran clang of hello.c CS50. I was passing in two command line arguments hello.c, which is the code I want to compile, and dash LCS 50, which means use the CS 50 library, please. But I can add another to the mix. I can actually do something like this whereby I do clang oh hello, then I can do hello.c. And then dash LCS50 enter. Now that too seems to work, and if I type LS, well, I've got all the same programs as before. So let's go ahead and get rid of those to make clear what's going on. I'm going to remove a.out. I'm going to remove hello, and just for good measure, I'll remove buggy as well, so that all I have left in this folder is source code. So if I type LS, there's my two files. Let's do this again, clang oh hello. Hello.c-LCS 50. Enter. Now if I type LS, I don't see a.out anymore because apparently according to the documentation for Klango, the actual compiler, if you pass 0 as a command line argument followed by another word of your choice, you can name the program anything you want without having to resort to MV or clicking on it and typing a new name manually. So if I now do hello, I see the exact same version where it's just asking me for my name and then printing it out. But long story short, the whole point of this exercise is that like running commands like this quickly gets very tedious. You have to remember like the order in which to do it, what the command line argument. I mean this is just stupid waste of time, typically, certainly in week one of the course they have to memorize these kinds of magical commands to get things working. But for now, no, that when you run make. It's essentially automating all of that for you and making it as simple semantically as make hello or make buggy, but what's really happening is the make command, because of the way we've configured CS50.dev for you, is doing all of this behind the scenes. It's not that magical. This just means change the file name to hello when you compile it. This just means compile this code, and this just means use the CS 50 library. Like that's all But that message about linking something in, there's there's something juicy going on there such that make is in fact helping us sort of solve a whole bunch of problems when we compile. And in fact, let me propose that if we take a step back and look at some of the actual code that we're compiling, let's consider like what we actually mean by compiling. Yes, it's the case that the compiler code means to go from source code. To machine code, but technically there's a few more steps involved. Technically, when you compile your code, that's sort of become the industry term of art that really is referring to 4 separate processes, all of which are happening in succession automatically, but each of which is doing a different thing. So just once, let's walk through these these several steps. So what is this pre-processing step? So consider this program here which we wrote in brief last week. We've got include standardio.h, which is there because we want to be able to use printf. Ultimately we've then got a prototype for this meow function, and the meow function does this. All it does is printf quote unquote meow followed by a new line, takes no input, returns no return values. The main function now has a for loop, iterates 3 times each time calling the meow function, and we saw this already earlier today. This line of code here, the so-called prototype, is necessary because we need to tell the compiler that meow exists before we actually use it here, especially if I don't get around to implementing it. Until later, so this copy paste of that first line of code, a so-called prototype, solved that

problem. This is what the header files are essentially doing for us. Before I use `printf` down here, the compiler needs to know what it is, what its inputs are, what its outputs are. Turns out the prototype for `printf` is going to be in `stdio.h`, and that's what that line of code has been doing for us all this time. In fact, let's take a simpler example that we keep using here whereby I'm in. Including `CS50.h` and `stdio.h` and I'm using the `CS50` `get_string` function to get someone's name and put it in a variable called `name` and then I'm printing out hello, such and such. What's going on now when I preprocess this file by running `make`, which in turn runs `clang`? Well, the compiler finds on the server's hard drive the file called `CS50.h` goes inside and essentially copies and pastes its contents into my own code. Meanwhile, Such that we get the prototype there for `get_string`, and we haven't seen this yet, but it stands to reason that all this time using `printf` we've been passing in a prompt like what's your name? and we've been getting back a string. What's inside the parentheses we call is the input, what's before the function name is the output, the so-called return value. What about `stdio.h`? It's in that file that `printf`'s prototype is. So essentially what the compiler does when preprocessing this file is it finds `stdio.h` somewhere on the server's hard drive goes inside and copy and pastes those relevant lines of code into my code as well. It's to avoid me having to do all of that myself, find the file, copy, paste it, or manually type out the prototype. These preprocessor directives just automate all of that TDM. So what this effectively has at the top of my code after the file has been preprocessed is all of those hash symbols followed by `Inclu` are changed to contain the actual contents of those. Header files. Now the compiler knows what `GetString` is all about and what `printf` is all about. That then is the preprocessing step. What does compiling technically mean? Compiling means taking that pre-processed code, which again looks a little something like this, and convert it into something called assembly code, and we won't spend much time in this class on assembly code, but this is how programmers used to write code before there was C, before there was Python. And Java and all of these other modern languages, programmers were writing code like this. Before this existed, they were programming zeros and ones into the earliest of mainframe computers using punch cards and other technologies like literally sheets of paper with holes in them. Not very fun, very tedious, so the world invented this. Also not very fun, very tedious, so the world invented C. Not that much fun, so the world invented Python and so forth that we continue to sort of evolve as a species with code, but The compiler technically takes your preprocessed source code and converts it into something that looks like this cryptic, and that's to be expected, but there are some familiar phrases. There's mention of `main`, there's mention of `get_string`, there's mention of `printf`. and there's a bunch of other things `move` and `push` and `XO` and `call` and these other commands here. These are the assembly instructions. Those are the lowest level instructions that the CPU inside of a computer understands. CPU is the central processing unit, the thing by Intel or AMD or Apple or other companies. Those are the lowest level commands that the actual hardware inside of the computer understand. It's just nice to be able to write words like `main` and `for` and `printf` than it would be to run these much more arcane commands that you'd have to look up in a manual. So compiling just takes C code and makes it a lower. Level type of code called assembly. When I said `a.out` means assembler output. That's why inside of that file is essentially the output of an assembler. All right, we're almost there. What does it mean to assemble a program, which is step 3 of the compilation process? That means converting assembly code to the actual zeros and ones we keep talking about. So if the file is called `hello.c`, when that file is `a.out`. Assembled the assembly code becomes

the zeros and ones for your code in hello. C, but your code is not everything that composes your final program. Your code from hello.c has to be combined with code from CS50's library, from the standard IO library that other humans wrote. I and the team wrote the CS 50 code. Other humans in the world wrote the printf code and standard IO. So essentially. Fourth and final step is to link all of those zeros and ones together. Somewhere on the server there is not just the header file CS50. H and Standardio. H, but your code, hello.c, our codecs50.c, and the code that contains ride's own implementation. bit of a white lie. It's technically not called standardio.C, but the point remains ultimately the same. So these files have already been compiled for you in advance. This is your code. What the assembly process does is it combines all of that into zeros and 1s, and then all three chunks of zeros and 1s are linked together. So if you think back to when I tried compiling the code without LCS 50, there was some mention of link. linking just means the computer did not know how to link your code with CS 50's code because we were missing LCS 50, which tells the compiler to go find it somewhere on the hard drive. And the final step then of linking is to combine all of those zeros and ones into one bigger blob of zeros and ones, and that's What's inside your Hello program that you can execute. So long story short, these 4 steps are what's been happening ever since the start of last week pre-processing, compiling, assembling, and linking. But thankfully the world of programmers generally just treats all four of these steps as what we know now as. Compiling, it's just a lot easier to say compile and not worry about those lower level details, but that might reveal better to you what all these error messages mean when you see hints of this kind of terminology. Questions On any and all of that, from here on out, we're gonna go higher level than lower, yeah. I, I, I don't get the part with the like when we're talking about um well I think it's the assembly process when you basically convert into zeros and ones um doesn't like across the multiple like the three different ones so the zeros and ones signify different things like one can signify text and the other can signify something else how does the computer know like what part what 8 bit corresponds to like which part? Really good question. How does the computer know which of those zeros and ones corresponds to data like numbers or strings of text or. commands. We're going to come back to that in week 4 of the class, but long story short, what we just saw on the screen is a big blob of zeros and ones actually follows some pattern where the bits up top represent a certain functionality. The bits on the bottom represent something else, and they're organized into patterns. So long story short, we'll come back to that, but they follow conventions. It's not just a hot mess of like zeros and ones. Other questions. Correct, the pre-processing step goes into the header file and essentially copies and paste the contents of it into your own code so you don't have to waste time doing that manually yourself. Other questions? Just curiosity when you're talking about the um how it convert it to assembly code and you're saying that the CPUs. Commands is the CPU that That into the binary Uh, no, the, so when you compile your code, you're going from the uh assembly code to the zeros and ones. That, sorry, uh, when you compile, let me pull up the, the chart again. When you compile your code, you're going from the C code to the assembly code, and the patterns you get when you see the assembly code are specific to a certain CPU. So long story short, if you're designing software for iPhones or for Android devices or Macs or PCs, you're going to necessarily use a different compiler because given the same C code, you will get different assembly. Instructions in the output and this is why you can't just take back in the day like a CD containing a program from a Mac and run it on a PC or vice versa because it's the wrong patterns of instructions. But the reason why we have all

of these annoying layers of complexity is because 14 different people can now implement the notion of compiling. Someone can implement the preprocessor. Someone can implement the compiler, the assembler, the linker, and you can actually collaborate. By breaking things down into these quantized steps, but also you can do this step, this step, and then two different people can write compilers to actually write to output assembly code for like iPhones over here and Android devices over here, but all of us can still enjoy using the same language up here. So there's a lot of reasons for this complexity. Just understanding it is useful, but you're not going to need to use this sort of knowledge day to day. But it's what enables so much of today's complexity nonetheless. All right, so a bit of a flourish now as to what we've been doing with compiling. Well, compiling is going ultimately from source code to machine code. Couldn't you just kind of reverse the process, right? If someone wrote really interesting software like Microsoft Word or Excel or something like that, well, when I buy it or download it, like I literally have a copy of all of those zeros and ones, couldn't I just kind of reverse this process and reverse engineer someone else's code by Decompiling it and this is genuinely a threat, and this comes up in matters of law and intellectual property because the zeros and ones have to be accessible to you and to your computer so it's not a great feeling if someone with enough time and enough savvy could sort of reinvent Microsoft Word by just figuring out what all those zeros and ones mean. However, it's sort of easier said than done to reverse engineer code from these zeros and ones. For instance, this pattern of bits on the screen here did what did we say last week? Silly quote, no normal person should be able to answer this, but I did say it before. These zeros and ones print what? It just prints out hello world, and I cannot glance at that and figure it out like off the top of my head, but if I know what architecture, what CPU this code has been compiled into and I pay attention in week 4 and know what the various layout of the zeros and ones are, I could painstakingly figure out what each of those patterns of zeros and 1 means by breaking them into chunks of 8 or 16 or 30. 64, which are common units of measure that I alluded to last week. Now that's going to take a crazy amount of time and the sort of presumption is that if you are smart enough and capable enough and have enough free time to do that, it would probably take you less time to just implement Microsoft Word the normal way and just rebuild the software. It's going to take you more time to go in reverse than it would in the so-called forward direction, but there's other subtleties as well. Inside of this code is not only commands like printf functions like printf, but suppose that it contained a loop, for instance, to print meow, meow, meow. Well, we know already that you can use a for loop sometimes, or you can use a while loop, but they're functionally equivalent. It's sort of a stylistic decision which one you use, whichever one you're more comfortable with, or maybe feels a little better design. But you can't figure out from the zeros and ones whether or not it was a Y loop or a 4 loop because it just results in the same pattern of zeros and ones. It's just a programmer's choice, which is to say you can't even perfectly reverse engineer everything because it's not going to be obvious from the zeros and ones what the source code originally looked like. But again, the bigger deal breaker is if you have that much time and energy and savvy, just like. Implement Microsoft Word itself. Don't try to reverse the whole process, which is going to be much more painstaking and time consuming instead. Now this is not true for all languages, and just as a teaser, in a few weeks' time when we talk about web programming in another language called JavaScript, it turns out that JavaScript source code is actually sent from web servers to web browsers, and you can look at the source code of any website on the internet, Harvard.edu. Book.com,

gmail.com, it's going to be there, so not all languages, it turns out, are even compiled typically. Sometimes the source code is just executed by the underlying computer. So we're just scratching the surface of some of the implications of all of this in a little bit of time, let's take a look further under the hood at the actual memory, solve some other problems, but I think it's now time for Cheez It. So let's go ahead and take a 10 minute break. Snacks are now served. See you in 10. All right, we are back and up until now when we've been writing code, recall that we have to specify like what type of value you want to put in a variable. Like that's why I had to go in and add string before the word name in my first bug today. But it turns out C, as we've kind of seen already, has a whole bunch of these data types. I rattled these off last week, Bull into long float double char string, but we'll consider for a moment just how much space each of these things takes up and see if we can help you see what the debugger was seeing earlier. That is what is. In memory. So a bull, it turns out, actually takes up one bite, which is kind of stupid because technically a bull, true or false, really only needs one bit. It just turns out that it's more efficient and easier to just use a whole bite, 8 bits, even though 7 of them are effectively unused. So a bull will take up 1 bite even though it's just true and false. An int recall uses 4 bytes. So if you want to count really high with an int, the highest you can go is roughly 4 billion we've claimed, unless you want to represent negative numbers, in which case the highest is like 2 billion, because if Want to be able to count all the way down to -2 billion. You've got to kind of split the difference. A long meanwhile is twice that. It uses 8 bytes, which is roughly 9 quadrillion possibilities, which is quite a few more than 4 billion. That is if you want to include negative numbers as well. Then we had floats which were real numbers with decimal points which speak to just how precise you can be with significant digits. A float is 4 bytes by default, but a double gives you twice as many bits to play with, which gets you get lets you be more precise. Even though at the end of the day whether you're using floats or doubles, floating point imprecision as we've seen is a fundamental problem for scientific, financial, and other types of computing where precision is ever so important. A char meanwhile, at least as we've seen it, is a single byte using Asy characters specifically. And then string I'll put as a question mark because a string totally depends on its length. If you're storing high, that's like 12 bytes. If you storing hello, that's like 5 bytes and so forth. So string. Things depend on how many characters you actually want to store inside of them. So where does this go? Well, here's a picture of a stick of memory, a dim, so to speak, whereby on this stick of memory, which is slid into your computer, your laptop, your desktop, or some other device, there's all these little black chips that essentially contain lots of room for zeros and ones. It's somehow electronic, but inside of there are all of the zeros and ones that we can store data in. So if we kind of zoom in on this, it stands to reason. That for the sake of discussion, if this one chip represents like 1 gigabyte, 1 billion bytes, it stands to reason that we could slap some addresses on these bytes whereby we could say this is the first byte and this is the last bite, or more precisely this is by 0123 by 1 billion, and it doesn't matter if it's top down, left, right, or any other order we're just talking about this conceptually at the moment. So in fact, let's go ahead and draw this really as a grid of memory, a sort of canvas that we. Just use to store types of data like bows and ins and jars and floats and everything else. If we are going to use 1 bye to store like a char, well, you might use just these 8 bits up here, 1 bye up here. If you want to store an in, well, that's 4, you might use all 4 of these bytes necessarily contiguous. You can't just choose random bits all over the place when you have a 4 byte value like an in, they're all going to be contiguous back to back to back

in memory like this. But if you've got a long or a double, you might. Use 8 bytes instead. So truly when you store a value in memory, whether it's a little number or a big number, all you're doing is using some of the zeros and ones physically in the computer's hardware somewhere and letting it permute them, turn them on and off to represent that value you're trying to store. All right, so let's go ahead and abstract away from the hardware though, and let's just start to think of this grid of memory sort of in zoomed in form and consider at a lower level what is actually being stored inside. of here. For instance, suppose that we've got some code like this containing 3 scores on like problem sets. You've got a 72 on one of them, a 73 on another, and a 33 on the third. I've deliberately chosen our old friends 72, 73, 33, which we call spell high or together in the context of colors is like a shade of yellow just so that we're not adding some new random numbers to the mix. These are our old friends three integers. Well, let's use these in a program. Let me go over to BS code here and let me create with code. A program called scores.c that's just going to let me quickly calculate my average score on my problem sets. I'm going to go ahead and include, as we often do, standard IO. H at the top. I'm going to do in Main, void after that, and then inside of my curly braces, I'm going to do exactly those sample lines of code. My first. was let's say a 72. My second score was 73, and my third score was 33. So I've declared three variables, one for each of my problem set scores. Now let's calculate the average. So print F quote unquote, average colon, just so I know what I'm printing. And now I'm going to go ahead and use maybe percent I backslash N and then what I'm going to pass in is a bit of math. So to compute an average it's just score 1 plus score 2 plus score 3 divided by 3, and I put the scores, the numerator. Parentheses just like in grade school like I need to do that operation first before doing the division so just like math class, semicolon at the end to finish my thought. Let's see how this goes. Make scores enterslash scores and it would seem that my average across these three problem sets is 72, which I, which is great, but I don't think that's actually what I want here. What have I done wrong? It's unintentional, yeah. Yeah, I'm kind of being a little generous with myself here. I didn't really factor in my worst score, so that was accidental. So now let me do this correctly. Make scores slash scores, and now, OK, my average is 59, but I, I beg to differ. I'd like to quibble. My score technically, I think mathematically should really be 59.33. I'm kind of being cheated that third of a point. So what's going on here? Why am I only seeing 59 and not my full grade? So when Perfect, because I'm using integers. When I divide by 3, it's going to truncate everything after the decimal point, which we touched on at the very end of week one, which is an issue with just truncation in general. So one approach to fix this, I could change my percent I to F, which is the format code it turns out for a float, and that is what I want to print. So let's see if that fix alone is enough. Make scores, oops, it's not. I got ahead of myself there and let me scroll up to the error. Format specifies double, but the argument has type in. Turns out you can use percent F for doubles as well, so that's why I'm seeing double, even though I intended to float in this case. So there's a problem here. I, the argument has Type in even though I'm passing in percent, you're seeing mention of percent D here, which is an alternative to percent I. We typically encourage you to use percent I because I for integer, but there is a that is not the solution to this problem because I want my third of a point back. So how could I go about fixing this? Well, the fundamental problem here is that I'm trying to format an integer as a float or even as a double. Well, I need to convert these scores to floats instead, so I could go in. And change this to float, this to float, this to float, and heck, just to be super precise, I could add a 0.0 on the end of each of them, just to make super clear these are floats, but

there's another way, I could, for instance, Uh, simply convert my denominator to 3.0 because it turns out so long as you involve like one float in your math, the whole thing is going to get promoted, so to speak, to floating point values instead of integers. I don't have to convert all of them. So I think now if I do make scores, scores now, ah, there's my third of a percent, the third of a point back. There's another way to do this just as an aside, and we'll see. This again down the line if you really want to stick with 3 because it's a little weird just semantically to divide by 3.0, like that's an implementation detail, but you're truly computing an average of 3 things. You can technically cast the 3 to a float in parentheses you can specify the data type that you want to convert another data type to, and this too should make the compiler happy. A ha, scores. I get. Roughly the same answer. We're seeing some floating point in precision though nonetheless, but that too would achieve the goal here. But a short, that's all just a function of floating point arithmetic there. So what's going on now actually in the computer's memory? Let me revert back to the simpler one with just 00 there and let me propose that we consider where these three things are in memory. Well, if we treat this as my grid or canvas of memory, who knows where they're End up, but for the sake of discussion, let's assume that 72 ended up in the top left of my computer's memory. I've drawn it to scale, so to speak, and that this score 1 variable is clearly taking up 4 bytes of memory, and it's an inch, and that's typically how many bytes are used on systems. Technically it depends on the exact system you're using, but nowadays it's pretty reasonable to assume that an integer will be 32 bits on most modern systems. Score 2 is probably over there so. Score 3 is probably over there, so I'm using 12 bytes total, 4 bytes for each of these values. All right, so that's really all that's going on underneath the hood. I don't have to worry about this. The compiler essentially figured out for me where to put all of these things in memory, but what really is in memory, well, technically each of these variables, if it's, if it's composed of 32 bits, is really just a pattern of literally 320s and 1s, and I figured out the pattern here. I crammed them all into the. Space there, but you see here 3 patterns of 32 bits which collectively compose those numbers there. But let's consider design now. In terms of my code, this gets the job done. It's not that bad or big of a deal for just calculating the average of 3 scores, but this should also start to rub you the wrong way this week onward when it comes to design. Like this is correct, especially now that I clamored back my third of a point. But this is bad design using the variables in this way. Why might you think? Yeah. Yeah, I'm going to have to type in each score manually with each passing week when I get the 4th problem set in the 5th. I mean, surely people who came before us came up with a better way to solve this problem than like manually create 10 variables, 20 variables, whatever it is by the end of the semester. It just feels a little sloppy and indeed that's often the way to think about the quality of something is designed. Think about the extreme. If you don't have 3. Scores, but 30 or 300, is this really going to be the best way to do it? And if you feel like, no, no, there's got to be a better way, odds are there are certainly if the language itself is well designed. So let's consider how else we might go about solving this. Well, it turns out we can treat our canvas of memory, that grid of bytes, into chunks of memory known as arrays. An array is a chunk of contiguous memory back to back to back, whereby if you want to store 3 things, you ask the computer for a chunk of memory for 3 things. If you want 30, you ask for one chunk of size 30. If you want even more, you ask for a chunk of size 300. Chunk is not a term of art. I'm just using it to colloquially explain what an array actually is. It's a chunk or a block of memory. That is back to back to back to back. So what does this mean in practice? Well, it means that we

can introduce a little bit of new syntax and see. If I want to create one variable instead of 3, and certainly 1 variable instead of 30, I can use syntax like this. Hey compiler, give me a variable called scores plural. Give me room for 3 integers therein. So it's a little bit of a weird syntax, but you specify the type of all. Of the values in the array you specify the name of the array scores in this case, and I pluralized it just semantically because it makes more sense than calling it score now and then in square brackets, so to speak, you specify how many integers you want to put into that chunk of memory. So this one line of code now will essentially give me 12 bytes automatically, but they'll all be referable by the name scores plural. So let's go ahead and weave this into some code. As follows. Let me go back to VS code here, clear my terminal, and now let's just whip up the same kind of program but get rid of these three independent variables. And instead let's go ahead and just say in scores plural bracket 3. Now I need a way to initialize the three values, but this I can do too. It turns out that if I want to put 3 values in this, I just need slightly new syntax. I can say scores 0 equals 72 scores 1 equals 73 scores 2 equals 33. So it's not all that different from having three variables, but now I technically have one variable and I am indexing into it at different locations locations 01, and 2, and it's a 0 because we always in computing start counting from 0. So I do scores bracket 0 is going to be my 72 problem set. Scores 1 is my 73 problem set, and scores 2. Is my weakest my 33P sets. Now my syntax down here has to change because there are no more score 12 score 3 variables, but there are scores 0 plus scores 1 plus and notice what VS Code is trying to do for me. It's saving me some keystrokes as I type in scores and type 1 single bracket. Notice it finishes my thought for me and magically puts the cursor where I want it so I can put the two. Right there and generally save on keystrokes, but that has nothing to do with C. It just has to do with VS code trying to be now helpful. So I think now if I go down here and do make scores slash scores, we get the same answer, but it's arguably better designed because I now have one variable instead of 3, let alone many more. And in fact, if I wanted to change the total number of scores, I can just change what's in that initial square bracket. So if we consider what's going on now, if we look at the computer's memory, it's the same exact layout, but there's no more 3 variable names. There's one scores 0, scores 1, and scores 2. And notice here ever more important in arrays values are indeed contiguous back to. Back to back. Now the screen is only so wide, so they kind of wrap around to the next row of bytes, but the computer has no notion of up, down, left, right. I mean it's just a piece of hardware that's got lots of bytes available that can be addressed from the first bite all the way down to the last bite. The wrapping is just a visual artifact on this here screen. All right, so if I've done this now, maybe we can make this program a little more dynamic than just hard coding in my scores. Let me go in and add the CS 50 header library so that we could also use, for instance, like get in and start getting these scores dynamically. So I could do get in and I could prompt the user for a score. I could use get in again and I can prompt the user for another PET score. I can use get into a third time. And prompt the user for a third such score and then pretty much the rest of my code can stay the same. Let's do make scores again, scores 72, 73, 33, and now my program's a little more interactive. Like this doesn't work for just my 3 scores. It can work for anyone's scores in the class. Now this too hints of bad design. I like my introduction of the array because I now have one variable instead of 3, but what now might rub you the wrong way among lines 78, and 9. Yeah. It's repetitive. I mean, I typed it manually, but I might as well have just copied and pasted like literally the same thing. So what's a candidate for fixing this? Like what programming construct might clean this up, yeah.

Yeah, we could use a 4 loop or a Y loop or whatever, but a 4 loop would get the job done, and that's often my go to. So let's do that instead. Let's go under my declaration of the array and do 4 in I equals 0, I less than 3, I + plus, which we keep seeing again and again. Uh, now how do I index into the array at the right location? Well, here's where the square brackets are kind of powerful. I can just say my scores array at the location I should get an in. From the user as follows. So now I'm using get it once inside of a loop, but because I keeps getting incremented, as we've done many a time now for meowing and other goals, I'm putting the first one at location 0. Why? Because I is initialized to 0. I'm putting the second one at location 1. Why? Because I'm going to + + or increment I on the next iteration, then the next iteration. So this has the ultimate effect of putting these three scores at location 0, 1, and 2. Instead of me having to type all of that out manually, now I don't love how I've done this still. If we really want to nitpick, this solves the problem correctly, but it's kind of got a poor design decision still. It's got a magic number, as people say. What is the magic number here and why is it bad? Yeah, over here. Yeah, it was a little soft, but I think the number 3 is hard coded in two places. We've got it on line 6, which is the size of the array, and then again on line 7, which is how many times I want to integrate. But those are the exact same concepts, but it's on the honor system that I typed the number 3 correctly both times. So I think we can fix this a little better. I could do something like in N equals 3, and then I could use N here and then I could use N here so that now I only change it in. One place if your eyes are wandering to the bottom of the program, there's still a problem here because I've still hard coded 0, 1 and 2, but we'll come back to that. But this is arguably a little better. But let's talk a little bit about style. Typically when you have a con, typically when you've got a variable that should not change its value, we saw last week that we should declare it as constant, and the trick there is to literally just write constant for short in front of the type of the variable, and now it should not be changeable by you, by a colleague, a collaborator, or the like. But typically too by convention stylistically to make visually clear to another programmer that this is a constant, it's convention also to capitalize constant, so to actually use like a capital N here in all places just to make clear visually that there's something interesting about this variable and indeed it is a constant that cannot be changed. All right, with that refinement, I don't think we've really improved the program fundamentally. I think we're going to need to do a bit more work to do this really well. So I'm going to do this a little quickly, but mostly to make the point that we can make this indeed more dynamic. So let me hide my terminal window there. Let me go ahead now and get the scores, as I already am as follows here, and let me go ahead and uh assume for the sake of time. That we have a function that exists already called average and I simply want to pass in to that average function the scores whose average I want to calculate. So average does not exist off the shelf. Like I can't just use an existing library for it. I'm going to have to implement this thing myself, but how? All right, well, let's go ahead and do this at the top of my file. I'm going to go. Head and compute or define a function called average uh that takes in what an array of numbers. So this syntax is going to be a bit new, but the way I do this is in say array bracket 0 or array sounds a little too generic. Let's just call it numbers for instance here. So that says my average function is going to take as an argument in array of numbers. This average function though should return a value too, and it should return what type of value from what we've seen thus far. A number of floats specifically. It could be in, but then I'm going to get short changed by 3 of a point potentially. So I think I wanted to return a float or if you really want precision, then you could return a double just to be

really nitpicky, but that seems excessive here. All right, well now inside of my average function, how can I calculate the average? Well, this is just kind of like a math thing, so I could declare a variable called sum and set it equal to 0. I could then have a loop inside of this function for in I gets 0, I less than. Huh, uh, I'm gonna come back to this. The number of numbers in the array, and then I'm gonna do I plus plus, and then on each iteration I'm gonna do sum equals whatever the current sum is plus whatever is in the numbers array at that location. So I'm going a little quickly, but again I'm just applying the same lesson learned numbers is my array. Numbers 1 means go to the I location in there, but if my loop starts at 0, that means go to location 0 and then 1 and then 2, and heck, if there's more scores in this array, it's just going to keep going on up from there because of the plus plus. But I hesitated here for a couple of reasons, so I put it to do here, which is not a thing that's a note to self. How far do I iterate? Well, if you've come into CS 150 with programming before, you can usually just ask an array, AKA a vector, what its length is in Java and in Python and the like. You can't do that in C. So if I want to know what the length is of this array, I've got to have the function tell me. So I'm gonna to additionally propose that this average function can't just take the array, it's also going to have to take another argument, a second input for instance called length that tells me how long it is. And then down here, which is where we started the story when I used this so-called average function. I'm gonna have to tell the average function by passing in N how many numbers are in that array just because this is annoying that you have to pass in not only the array but also its size separately. That's the way it's done in C. More recent languages have improved upon this so you can just figure out what the length of the array. is as we'll see in a few weeks in Python. All right, back to the average function at hand. I think we're almost there. This is a little unnecessarily verbose. Recall that we can tighten this up by just doing plus equals whatever is in numbers 1. That's just tightening it up. It's syntactic sugar, so to speak. And then the last thing I'm going to do in my average function is what actually calculate the average. So what is the average? It's just the numerator, like the sum of all of the scores divided by the total number of all of the scores. Well, I've got the sum, so I think I just want to do some divided by what to get the actual average now. Yeah. Exactly sum divided by length will give me the average because the sum is the numerator effectively all of the scores added together and the denominator is the length. How many numbers were there actually? Now I can't just write this math expression here. If this is going to be my function's return value, and we've done this once or twice before, I literally say. And my average function return this value so it hands back the work. I could use printf and just print it on the screen, but I don't want that visual side effect. I want to hand it back so that on line 23 I can simply calculate the average of those end scores and let printf use it as the value of that format code F. All right, uh, I think we are in reasonably good shape. Let me cross my fingers now and hope I didn't screw this up. Make scores. OK, slash scores, how many do we want to do? So we'll do 72, 73, 33, enter, and there is, oh, so close, average. I've had a regression. I've made the same mistake again just in a different way. I think I saw your hand go up. Why am I getting 59 and I'm not getting my third of a point? Yeah, I in this return line on line 11 right now, I'm again stupidly doing integer divided by integer that will make us suffer from integer truncation because if you're returning an integer, there's no room for the decimal point or any numbers thereafter. So how do we fix this? Well, I could change the sum to float like that would. Reasonable. So then I do a float divided by the length. I could do my casting trick like convert the float, the length to a float just for the sake of floating point

arithmetic. There's a bunch of ways to solve this, but I think I'll go with this one now. Let me now do make scores again 72, 73, 33. And now I've got, albeit with some imprecision, I think enough precision certainly for like a college grade in this case 59.333 and so forth. OK, so what are the things to actually care about here? So there's a decent amount of code here. Most of it is sort of stuff we've seen before, but the interesting parts I would propose are this. When you create your own function that takes an array as input, you have to take as input the length of the array. You're not going to be able to figure it out correctly, as in modern newer languages. You also need, of course, to pass in the array itself. How do you pass in an array? Well, when you're defining the function, you specify the type of values in the array. Whatever you want to name the array inside of this function, and then you use empty square brackets like this. You don't have to put in or some other number there. All you need to tell the compiler is that my average function is going to take some array of values, specifically this many. You don't put it inside the square brackets there. Then when I use it now it's just. The now familiar syntax when you want to index into your array, that is go to location 0 or 1 or 2, you just use square bracket notation here. But the array itself, recall, was actually created in Maine when I did this line of code here where I said give me an array called scores, each of whose values it's going to be an in and I want this many of them. And so maybe the final flourish that I'll add here just to be sort of nitpicky is I keep saying that main should really go at the top, fine, no big deal. Let me highlight my average function, move it to the bottom of my file just because, and then And only then I'll copy and paste that first line, the so-called prototype, so that lang doesn't freak out by not knowing what the average function is. So in short, there's seemingly a bunch of complexity here, but all were the only thing that's really new in this one example is this is how you pass to a function in a array that already exists elsewhere, not by its name, but by the square brackets there. OK. Questions on arrays or any of this new syntax, yeah. When did the whole like OK. Said that we could store as a float and instead of saying 3.0 is a float, you just said because 3.0 is a float. How does it know it's not a double? Oh, how does it know it's not a double? So by default, if you just type a number like 3.0 into your code, it will be assumed to be a double just because, um, raw values, literal numbers with a decimal point will be treated by the compiler as doubles and be allocated 64 bits. percentage Uh, uh, just because, like the world did not need to create a new format code like percent D is not double. percent D is decimal integer, but don't worry about that. We tend not to talk about it too much in class. percent I is integer, percent F is float, but percent F is also double. And this is not consistent because what's a long percent LLI what did I say last week? percent LLI gives you a long integer. It's just a mess that's, there's no good reason for this other than historical baggage. Sure, I'm not sure if that's reassuring, but all right, so, um, OK, let's use this knowledge for like something useful now and actually tease apart what is how we can use these these skills for good and to better understand what's going on inside of the computer as follows. Let me go over to our grid of memory and this time let's not store some numbers, but let's store like these three lines of code. These three variables, so 3 chars, even though you know where this is going, like this is not good design because I've got 3 stupidly named variables C1, C2, C3. But let's make a point first. The first variable's value is quote unquote h. Second is I. Third is exclamation point. Why though am I using single quotes suddenly instead of double quotes? It's a character.chars are single quotes, strings are double quotes, and we'll see the distinction why in a moment. So for instance, if this is my grid of memory and this program contains just 3 variables, each of them a char, odds are they'll end up

like this in memory C1, C2, C3, H. assuming there's nothing else going on in my program, they're just going to end up being back to back to back in this way, even though it might not, uh, in this way. So what does this really mean? is going on. Well, let's go ahead and poke around. Let me go back to BS code here. Let's close scores.c, reopen my terminal, and let's create a new program called Hi.c and just do something playful. So let me include standard IO. H at the top. Let me do in main void after that, and inside of my curly braces, let's just repeat this C1 equals H in caps, char C2 equals I in caps, and then char C3 equals. In uh in exclamation point, that's all. Now let's actually poke around and see what's inside the computer's memory. So I could do something like this. I could print F, for instance, C, C, C backslash N and C turns out means character. So what do I want to plug in C1, C2, and C3 semicolon. So let's go ahead and do this. Make hi enter high and voila, there's my HI exclamation point. There's no magic here. Like I'm literally just printing out three HR variables. I can, I don't need the spaces. If I want to get rid of those spaces between the word, I can remake this, make hi hi, and now we're back in business. But here's where an understanding of types can give you a bit of power and sort. Sate some curiosity. What if I change my percent C to peri peri peri. So into into into. Well, turns out that a char is really just a number because it's an ASCII value from 0 to 255. So there's nothing stopping me from telling the compiler, don't print these as chars, print them as integers. So let's do make high high, E. And that's a little cryptic. It looks like it's saying 727,333, but no, let me add those spaces back in between each of those placeholders. Ma high again. There are our old friends 72, 73, 33. It is not necessary in this case to say in. In in because the compiler is smart enough and printf is smart enough that if you hand it a value that happens to be a char, it knows already it's going to be an integer essentially, so you don't even need to bother explicitly casting it this way. We're essentially implicitly casting it to an integer by using those format codes as such. All right. So that just proves that what I've claimed is the case that there is this equivalence between characters and numbers is actually the case inside of the computer's memory. So even though you're storing HI exclamation point, technically you're storing three patterns of 8 bits each that give you these decimal numbers 72, 73, and 33 or specifically these patterns here. All right then, what is a string? And this is where things get a little more interesting. A string, as we've used it, is like a whole word or a phrase or when we started class today, like a whole paragraph of text. So that's multiple values. Now why is that interesting for us potentially? Well, let's go ahead and write one line of code as a string. So here for instance is one line of code with a string. Let's go ahead and put that into my program. So I'm going to go back to VS code here and clear my terminal, and I'm going to go ahead and delete all of this code here for a moment. And I'm going to do something like this strings equals quote unquote high with with double quotes now. And now just like in week one, I'm going to print out pers backslash N and print out the value of S. Per earlier, because string is technically one of our training wheels for just a few weeks, I'm going to additionally include CS 50. H at the top so that the compiler knows about what this word is string. All right, let's go into the terminal. Make high high enter, and we're back in business, printing that out now as an entire string. Well, what's going on inside of the computer's memory this time? Well, I still have HI exclamation point, but it's a string now. Well, it turns out the way that's going to be laid out in the computer's memory is exactly like before. There's no mention of C1, C2, C3 because those variables don't exist. There's just one variables, but it's referring to 3 bytes of memory, it would seem HI point, and you can kind of see where this is going, like a string as a spoiler turns out it's actually just what?

It's just going to be an array of characters, hence the dots we're trying to connect today. So at the moment though, this is a single variable a string, the value of which is HI. But you know what, if it is in fact an array, I bet we can start playing around with our new square bracket notation and see as much in our. Actual code. So in fact, let me go ahead and do this. In VS code now. Let's not use percent S. Let's use C, C, and C 3 times. Then instead of just S, let's print it out like it is an array 0, S1, S2. Let's go back to VS code, uh, my terminal and BS code, make high high and nothing has changed, but I'm printing it out now one character at a time because I understand what's going on underneath the hood in this case. I can actually see these values now. Let's go ahead and change the percent C to I and at a space just so it's easier to read percent I space I space. Don't need my casts in parentheses because printf is smart enough to do this for me. Make high again. lahi. There again is my 72 73 33. However, that came from the mere fact that I put in double quotes. So what's really happening here is it seems that a string is indeed just an array of characters. But how does the computer know when doing percents know what to actually print? In other words, it stands to reason that eventually if I've got more variables, more code, there's going to be other stuff in the computer's memory. Why does print F know when using percent S to Stop here and not just keep printing characters that are over here, especially if I did have more variables and more stuff in memory. Well, let's take a look at what's just past the end of this array. Let's go back to VS code and now let's get a little crazy and add in a 4th I. And even though this shouldn't exist, let's do S3, which even though it's the number 3, it's the 4th location, but H is only 3 values. So let's look 1 location past the end of this array. Make. Interesting. It seems, and maybe it's just luck, good or bad, that the 4th byte in the computer's memory seems to be a 0. Well, that's actually very much by design. It turns out if we look a little further by convention, what the compiler will do for us automatically is terminate, that is end any string we put in double quotes with a pattern of 8. er bits more succinctly it's just the number 0 because if you do the math where you've got 80s, it gives you zero and decimal or more technically the way it's typically written is this because it's not like the number 0 that we want to see on the screen. Backslash 0 similar to backslash N is sort of a special escape character. This just means literally 80 bits, not the number 0 that you might see in a phone number or something like that. So even though we said strings equals high with an exclamation point, seemingly 3 characters, how many bytes does a string of length 3 actually seem to take up in memory? It's actually gonna be 4, and this happens automatically. That's what the double quotes are doing for you. They're telling the compiler this is not just a single character, this is a sequence of characters. Please be sure to terminate it for me automatically with a special pattern of 80 bits, and that special pattern of 80 bits actually has a name. It's the so called. Null character or NUL for short. The null character is just a bite of zero bits, and it represents the end of a string. You've actually seen it before, if super briefly, two weeks ago. He was our AI chart, and we focus mostly on like this column here and this column here, and then we looked at the exclamation point over here, but all this time over here, Ask character 0 is null, NUL, which just means that's how you pronounce all 80 bits. It's been there this whole time. So why is it done this way? Well, how is the computer actually printing something out in memory? Well, it needs to know where to stop. Printf is pretty stupid. Odds are inside a print up there's just a loop that starts printing the first character, the next character, the next character, and it's looking for the end of the string. Why? Well, consider what might happen. Suppose you've got a program that has not just one string, but 2, for instance, two

strings like this. So in fact, let me go back to. code here, clear my terminal, and let's just make this program a little more interesting for a moment. String T equals by for instance and then down here let's do two print Fs percents, backslash N and print outs. Print F pers backslash N print out T. Now to be clear, percents means string placeholder. T and S are just also the names of the variables. There's no percent T that we want to use here. All right, let me go down to my terminal, make high high, and voila, I get high and by just like you would have expected last week. But what's going on inside of the computer's memory? Well, insofar as I've asked it to create two variables S and T like this. Odds are what's happening in the computer's memory is high is ending up here, a. A S T, because there's nothing else in this program, it's probably going to end up here, BE, but it wraps on this particular screen. T is taking up 12345 bytes total, just as high it's taking up 4 bytes total because the compiler is automatically adding for me the backslash zero, the null character to make clear to other functions where this string ends. So what does this mean in real terms and why is it 0? Why is it 0? Like, uh, just because like at the end of the day all we have is bits. We've got 8 bits to work with for chars. You've got to pick some pattern. We could have chosen all ones. We could have chosen all zeros. We could have chosen something arbitrary. A bunch of humans in a room years ago decided. 8 zeros will mean the null character. That's the special character we will use to terminate strings in this way. Well, what does that mean with our new syntax? Well, it means we can poke around with strings as well. So even though that first variable is S and that second one is T, you could technically poke around and access S 0 and 1 and 2 and 3, T 0123 and 4, and so forth. So in fact, if I wanted to dive in deeply there. And actually see that. Well, let me go ahead and do this back in VS code here. Let me make a refinement here. I've now got my two strings here. I could go and for instance down here just like before, C, C, C, C, C, C, C, C, and if I then do S 0. Uh, S1, S2, oops, 2, and then down here T 0 T1, T2, T3, and I'm doing that only because the word by is longer than the word hi. If I do make hi.i. Same principles work even in this context here. But let's add an interesting twist just because if I have these values in memory here. As follows. Well, it's kind of if I've got two words in memory, I could use them in an array too instead of having like S and T or word 1 and 2. I can actually put strings in an array too. So let's go ahead and do this. Let me go back to VS code and just for fun now, let's go ahead and do this. Give me an array called Words that's going to fit two strings, then. In the first words, words 0 put high, then in words 1 put by. The only thing new here is that I'm making an array of strings now instead of an array of ins, but all of the syntax is exactly the same. How can I go about printing these things? Well, just as before, I can do print Fs backslash N and print out words 0. Then I can do print Fos. Backslash N words 1. Again, I'm just sort of applying the same simple syntax that we saw before I again of the 6th version of this program, right? I'm just sort of jumping through hoops and tactically to demonstrate that these are just different lenses through which to look at the exact. Same idea. And while a normal person would not do this, we could think about what's really going on in memory with arrays of words when those words themselves are arrays of characters because a word is just a string. So this code here gives us something like this in memory in that program a moment ago this is words 0, this is words 1. The only thing that's different is I'm not calling them SNT. I've given them one name. With two locations, 0 and 1. Well, if each of these values is itself a string, well, you said earlier that a string is just an array, so we can actually think of these two strings, even though the syntax is getting a little crazy. Using two sets of square bracket notation where I can index into my array of words and then

index into the individual letters of that word by just using more square brackets. And again this is just to demonstrate a point not because a normal person would do this, but if I go back to VS code instead of printing out these two strings, why don't I do something like this? `Print F C C C` `backslash N`. Then let's print out the first word, but the first character they're in. Let's print out the first word, but the second character they're in, the first word, but the third character they're in, and even though I'm saying 3rd and 2nd and 1st, it's 21 and 0 respectively because we start counting at 0. And then lastly here we can print out the second word `C, C, C, C, backslash N` then words bracket, how do I get to the second word in this array? Words bracket 1, the first character they're in. Words bracket 1, the second character they're in. Words bracket 1, the third character they're in words bracket 1. The last character therein and again I'm this is just to demonstrate a point, but if I do make high now high, we have full control over everything that's going on. If you now do agree and understand that an array can be indexed into with square bracket notation, as can a string because a string is itself just an array. Strings are arrays for today's purposes then. Questions on any and all of these tricks. No? All right, yeah, in front. about OK. Like, with like the full average in in in in in number, is that, is that establishing in the right? like how are you? How do you like establish? How do you establish or create an array? Well, in the context of this program, if I go back to VS code, line 6 here gives me an array of size 2. An array of two strings if you will. The previous example we were playing with, which was my scores, oops, wrong program, wrong file. If I open up scores.c as before, this line here, line 9, gives me an array of N integers. So that is what establishes or creates the array in memory. You specify a name, the size, and the type. That's all. And the only thing that's new today again is the square bracket notation, which in this context creates an array of that size, but once it exists, you can then access that chunk of memory by using square brackets as well. Other questions on a raise, yeah, in front. Or do you need to go in index index. Good question. Do you need to go index by index to put things inside of an array? Short answer no. So let me open up again scores.c from before, and what I could have done in an earlier version of my program would be something like this. I could have done 72, 73, 33, and I deliberately didn't show this because I didn't wanna add too much complexity, but. You can use curly braces in this new way and initialize the array in one line, and in that case you don't even need to specify the size because the compiler is not an idiot. It can figure out that if you've got 3 numbers on the right, it knows that it only needs 3 elements on the left to put them into. But let me undo that and leave it just as I did, but short answer yes, you can statically initialize an array if you know all of the values up front and not when using get in in that case. All right, so if you're on board with the idea that all a string is, is an array and that array is always null terminated, we can now. Use that knowledge to solve some simple problems and problems that others have already solved before us. So let me go ahead and close that file in VS code. Let me go ahead and open up another program here called length.C and let's just play around with the length of strings as follows. Let me include the CS 50 library at the top. Let me include standard IO after that. Let me do into Main void after that, and then inside of Main, let's prompt the user for their name. By using get string and just say name Colin today and then after that, let's go ahead and figure out the length of the person's name like D A V I D. I should get the answer of 5 and K E L L Y, we should get the answer of 5 and hopefully for a longer or shorter name, we'll get the correct answer as well. So how can I go about counting. The number of characters in a string. Well, the string is just an array, and that array ends with the null character. There's a bunch of ways we can do this, but

let me go ahead and do this. Let me create a variable called N which eventually will contain the length of the name, and I'm going to set it equal to 0 because I don't know anything yet about the length. Then I can do this with a 4 loop, but I prefer this time to use a while loop. I'm gonna say the following while the person's name. At that location does not equal backslash zero, go ahead and add 1 to the value of N. And then after all of this, go ahead and print out with percent I backslash N the value of N. So what's going on here, this is easier said when you know already where you want to go with it, but with practice, you too can bang this out pretty quickly. N is going to contain the length of my string. I have in my loop here a boolean expression that's just asking the question, Does name at the current value of N not equal the null character? In other words, you're asking yourself, Is this character null? Is this character null? Is this character null, is this character null? And if not, you keep going, you keep going. And this is kind of a clever trick because I'm using N. And incrementing it inside the loop. So when I look at D, that's not equal to backslash zero, so I increment N. Now N is 1, so I look at name bracket 1. What's name bracket 1? If it's my name, A A does not equals 0, so it increments N. What's at location? D A V I D V V does not equal backslash n. So we repeat with I, we repeat with D, and then we get to the end of my name, which is the null character because the get string function and C put it there automatically for me. The null character does equals zero and does not get incremented any more time. So at this point in the story on line 13, N is still 5 because I have not counted the new the null character. So I hope I will see 5 on the screen. This is just kind of a very mechanical way of checking, checking, checking, checking, trying to figure out through inference how long the string is because it's as long as it takes to get to that backslash zero, the null character. So let's do make length enters length. Type in my name David and I indeed get 5. Let's go ahead and do length. Kelly, I indeed get 5 and hopefully for shorter and longer names I'm going to get the exact same thing too. In fact, we can try a corner case length enter. Let's not give it a name at all. If I just hit enter here, what should the length of the person's name be? 0, which is not incorrect. It's literally true, but that's because we're going to get back essentially quote unquote, but even though it's quote unquote in the computer's memory, it's still going to take up one bite because the get string function will still put null at the end of the string even if it's got no characters therein. So it turns out. This is not something you need to do frequently like initializing a variable, using a loop like this. It turns out there are better solutions to this problem. You do not need to reinvent this wheel yourself because it turns out in addition to standard IO.H and CS 50. H and as you probably saw on Problemset one, math. H and perhaps others, there are other libraries out there, namely the string library itself. In fact, if you go into the CS50 manual, you can look up the documentation for a header file called string.H which contains. for that is prototypes for a whole bunch of helpful functions. In fact, the manual pages for it are at this URL here. The most important function, and the one we're going to use so often for the next few weeks is wonderfully called Stirlang for string length. Someone else literally decades ago wrote the code that essentially looks quite like this but packaged it up in a function that you and I can use so we don't have to jump through these stupid hoops just to count the length of a string. We can just ask the string length function. What the length of a string is, but odds are, if we looked at the C code that someone wrote decades ago, it would look indeed quite like this. So how can I simplify this program? Well, I can get rid of all of this code here. I can include string. H at the top of my file, and then I quite simply. Do something like this in length equals stir lang of name. That's going to put in the variable length, actually, let's be

consistent in `N` equals `strlen` of `name`, and then on line 9, let's print it out. Let's try this. Make `length` `length` `David`. OK, Kelly, OK, and no one. And 0, it seems to now be working. So this is a wheel we do not need to reinvent and frankly now in a matter of design I don't really need the variable `N` anymore. Recall that we can nest our functions just like we did with `average` before. So let me get rid of that line and just say `strlen` of `name` is actually perfectly reasonable here. All right, well, what more can we do with this? Well, let's consider some other matters of design. Let me close out `length.c` and let's create another program of our own called `String.c` in which we'll play around now with this library and others. Let me go ahead and include `CS50.h`. Let me go ahead and include `stdio.h`. Let me go ahead and include also `string.h`. All right, what do I want to now do? Well in `Main` void and inside of `Main`? Let's go ahead and write a program that prints a string character by character just to demonstrate these mechanics. So `strings` equals `get_string`, and I'm going to ask the user for some input because I just want to play around with any old string. I'm going to go ahead and proactively say output here, and I'm going to go ahead and not use a new line character there deliberately. Below this now I'm going to have a `for` loop, though I could use a `while` loop that says in `i` equals 0, `i` is less than `strlen`. Of the string I just got from the human and increment `i` on each iteration, and on each iteration print out just one character in that string. Specifically at `S` location `i` and then at the very bottom of this program, let's just print a single backslash end to move the character onto a new line. Long story short, what have I done? I wrote a stupid little program that prompts the user for a string, prints the word output thereafter, and then just prints the word that they typed in character by character by character by character until it reaches the end of the string based on the length returned by `strlen`. So let's go ahead and run this in my terminal window. I'm going to do make `strings` `string`, and I'll type in my own name it before. This was a subtlety. I deliberately wrote two spaces here because I just, um, to be nitpicky, I wanted input and output to line up perfectly so you can see what's happening. Indeed, if I do enter here, now I see input is `David`, the output is `David` as well. So that was just a formatting trick that I foresaw. Why is this program correct but not arguably well designed? It's pretty good in that it's using the `strlen` function. I didn't reinvent the wheel unnecessarily, but there's an inefficiency that's kind of subtle. And it relates to how a `for` loop works. Any thoughts? This program I claim is doing unnecessary work somewhere. Yeah. OK, that's definitely stupid. Um, you don't have to output a character by character, that's just my pedagogical decision here. So correct but not the question we're fishing for. There's a second stupid thing, yeah. Che Yes, every time through this loop, and this isn't so much my uh conscious choice but my mistake. I'm checking the length of `S` again and again. Why? Because recall how a `for` loop works. The initialization happens once at the very beginning. Then you check the Boolean expression. Then if it's true, you do the code. Then you do the update, then you check the boolean expression, then you do the code. Update Boolean expression, you do the code. But every time you evaluate this Boolean expression, you're asking, does `i` is less than the `strlen` of `S`? But this is a function call like you are literally using `strlen` again and again and again and like a crazy person you're asking the computer what's the length of `S`, what's the length of `S`? What's the length of `S`? It's not going to change. It's going to be the same no matter what. So how can we fix this? Well, I could solve this in a couple of ways like I could, for instance, down here dot and equals `strlen` of `S` and store it in a variable `n`. And just do that. I think that eliminates the inefficiency because now I calculate the length of `s` once. It's not going to change, nor is my variable, so I can now use and reuse that variable. It's just saving

me a little bit of time, you know, microseconds maybe, but when you're writing bigger programs and you're doing things in loops, if that loop is running not 3 times or 5 times, but a million times, millions of times, all of those microseconds, milliseconds might very well add up. But it turns out there's some syntactic tricks we can do too. I alluded to this earlier. If you want to initialize not one variable, but 2, you can actually do it all before the first semicolon like that. So now on line 9 I'm declaring a variable called `I` and setting equal to 0, and I'm declaring a second variable called `N`, also the same type in, and setting it equal to the length of `S`. And now I can use that again and again. Now, as an aside, this is a little bit of a white lie because smart compilers nowadays are so advanced that they will notice that you're calling `Sterling` again and again inside of a loop, and they will just fix this for you unbeknownst to you, but it's representative of a class of problems that you should be able to spot with your own human eyes and avoid altogether so that you don't waste more time and more compute and more money in some sense than you might otherwise need to in this case. Any questions on that there, optimization, yeah. You do not say it again. The constraint is that you have to use the same data type for all of your initialization, so you better hope that you only want ints in this case. Otherwise, you got to pull it out and do what I did earlier. Good question. Others on this. Yeah. When does it account for spaces? Uh, a space is just uh ask a character number 32, so there's nothing special about it. It's sort of invisible, but it is there. It is treated like any other character. There's no special accounting whatsoever. The null character, which is also invisible, is special because `Pri F` and `Stirling` know to look for the end of that variable, the end of that value as such. All right, let's try one other demonstration of some of these ideas here. Let me go into Uh, another file that we'll create called how about `uppercase.c`. Let's write a super simple program that like uppercases a string that the human types in and see how we can do this sort of good, better, and best. So I'm going to call this file `uppercase.c`. Inside of this file, let's use our now friends, include `CS50.h`. Let's do include `stdio.h`. Let's then include lastly, how about `string`. H? And the goal here inside of `Main` is going to be to Get a string from the user, so `string S equals get string`, and we're gonna ask the user for a before string, representing what it is they typed before we uppercase everything. Then I'm going to go ahead after that and print out just as a placeholder after and two spaces just to be nitpicky so that the text lines up vertically on the screen. Now I'm going to do the following `4 I equals 0, N equals slang of S, semicolon I less than N just like before I ++`. So I'm just kicking off a loop that's going to iterate over the string the human typed in. Now if my goal in life is to Change the user's input from lower case, if indeed in lower case to upper case, let's just express that literally if the current character in the string, so `SI` is greater than or equal to `A` and `S` bra `I` is less than or equal to `z` using single quotes, this is arguably a very clever way of expressing the question is it lower. Case we know from our `AI` chart from week 0 that the `AI` chart has not only numbers representing all the uppercase letters but also numbers representing all the lower case letters. Lowercase `a` for instance, is 97, and they are all contiguous thereafter. So we can actually treat just like we did before chars as ints and ints as chars and sort of ask mathematical questions about these chars and say is `S I` between `A` and `Z` inclusive. So if it is lowercase, and I'll add a comment here for clarity, if `SI` is lowercase, what do we want to do? We want to force it to uppercase. So this is a little trick I can do as follows. Print `F`. The current character, but let's do some math on it. Let's changes `I` by subtracting some value. Well, what might that value be? We'll recall from week ze our `AI` chart here and let's focus, for instance, on the lower case letters here and the uppercase letters here. What's the

distance between all upper and lower case letters? It's 32, right, and the lowercase letters are bigger, so it stands to reason. If I just subtract 32 from the lower case letter, it's going to immediately get me to the uppercase version thereof. So this is kind of cool. So I can actually go back to VS code and I can literally subtract the number 32 in this case because AI is a standard, it's not going to change. Else if the letter is not lowercase, I'm just going to go ahead and print it out unchanged without doing. Any mathematics at all to it, and I'll make clear with the comment, uh, else if not lower case makes clear what's going on there. All right, let me go ahead and make upper case in my terminal window. uppercase. Let's type in my name all lower case and I get back, David, uh, minor bug, couple bugs actually. Let me fix my spacing. I think I want another space after the word after, and at the very bottom of my program, I think I want a backslash n. Now let's rerun uh make on uppercase uppercase enter DAVID and now it's forcing it all to uppercase. Meanwhile, if I do it once more and type in name capitalized, it's still gonna force everything else to uppercase Questions. Oh, I'm an idiot. OK, thank you. Yes, uh, I misspelled after, otherwise my aligning, my alignment would have worked. So let's do this again, make uppercase, if only so that we can prove it's the same, DAVID and all lower case, and there we go. That was thank you, the intent. Alright, so it's kind of a little trick, but this is kind of tedious, right? Like Microsoft Word, Google Docs all have the ability to toggle case from uppercase to lower case or lower case to uppercase. It's kind of annoying that you have to write this much code to achieve something so simple, seemingly and so commonplace. Well, it turns out there's a better approach here. In addition to there being the string library, there's also the C-type library. In Ctype.H, another header file, there's a whole bunch of other functions that are useful that relate to characters, uh, characters, uh, in AI. So for instance, if we go ahead and use this. As follows, I'm going to go ahead at the top of my file here and include now C. H. It turns out there's going to be functions via which I can actually ask these questions myself. For instance, in this next version of the program, I don't need to do any of this clever but pretty verbose math. I can just say if the is lower function, which comes from the CType library passing in SI, returns true, will then convert. The letter to lower uppercase by subtracting 32, but you know I don't even need to do this mental math or math in code. I can also from the CT type library use a function called 2 upper which takes as input a character like SI and let someone else's function do the work for me. So let me go back down to my terminal window here. Let me make uppercase now uppercase enter before DAVID. This now works too. But if I really dig into the documentation. And for the CType library, you'll see that you can just use the is lower function on any character and it will very intelligently only uppercase it if it is actually lowercase. So someone else years ago wrote the conditional code that checks if it's between little a and little Z. So knowing this, and you would see that indeed in the documentation, I don't even need this else. I can instead just get rid of this whole conditional, tighten my code up significantly here, and simply say. Print F using percent C, the two upper version of that same letter and let the function itself realize if it's uppercase, pass it through unchanged, if it's lowercase, change it first and then return it. So now if I open my terminal window again and clear it, make uppercase uppercase enter DAVID, and we're back in business. So again, demonstrative of how if you find that coding is becoming tedious or you're solving a problem that like surely someone else has solved, odds are there is in fact. A library function for whether it's from CS 50 or from the standard library that you yourselves can use. And unlike the CS 50 library, which is indeed CS50 specific, which is why Klang needed to know about L CS 50, many of these libraries just

automatically work. You don't need to link in the C-type library. You don't need to link in other libraries, but non-standard libraries like CS 50's training wheels for the first few weeks, we do need to do that. But `main` is configured to do all of that. Automatically for you. All right, in our final minutes together, let's go ahead now and reveal some of the details we've been sweeping under the rug about `main`. I asked on week one that you just sort of take on faith that you got to do the void, you got to do the end, you got to do the void, and all of that. Well, let's see why that actually is. So `main` is special insofar as in C, it is the function that will be called automatically after you've compiled and then run your code just. Because not all languages standardize the name of the function, but C and C++ and Java and certain other ones do in this case. Here is the most canonical, simple form of `main`. We know that including `stdio.h` just gives us access to the prototypes for functions like `printf`. But what's going on with `int` and what's going on with `void`? Well, `void` in parenthesis here just means that `main`, and in turn all of the programs we've written up until this moment. Do not take command line arguments. Literally every program we've written out, hello, scores, everything else, I have never once typed another word after the name of our programs that we've written in class. That is because every program has `void` inside of these parentheses telling the computer this program does not take command line arguments, words after the programs. Name that is different from `make` and `code` and `CD` and other commands that you've typed with words after them their names at the prompt, but it turns out the other supported syntax for the `main` function in C can look like this too, which at a glance looks like kind of a mouthful, but it just means that `main` can take zero arguments or it can take 2. If it takes 2, the first is an integer and the second is an array of strings. By convention those inputs are called `RC` and `RV`. `RC` is the count of arguments that are typed after the after the program's name. `RV` is the argument vector, AKA array of actual words. In other words, now that we have the ability to use arrays, we can get 0 or 1 or 2 or 3 or more words from users. The prompt when they run our own programs. So what do I mean by this? We can now write programs that actually have command line arguments as follows. Let me go into BS code here and close our old program `uppercase`. Let's write a new simpler program here in my terminal called `greet.c` and just greet the user in a couple of different ways. So I'm going to include initially `CS50.h`, and then I'm going to include `stdio.h` here. Then I'm going to say in `main` `void` without introducing anything new just yet. I'm going to ask the user like we did last week for a return value from `GetString`, asking them what's your name, as we've done so many times. Then I'm going to say `printf` hello com persn spitting out their answer as follows. Same program as last week again. I'm going to make `greet`. I'm going to say `greet`, and I'm prompted now for my name. I hit enter. Notice that I did not take any command line arguments. The only command I ran was `greet`, no other words. Let's now use this new trick and actually let the user type their name when they're running my program rather than waste their time by using `GetString` and prompting them. Let me go into my editor here. Let's get rid of the `CS50` library. Let's get rid of my use of `GetString`. And let's simply change `void` to `int RGV`, then `string RGV` open open bracket close bracket. That's all. Down here, let's simply print out `RGV[1]` for reasons we'll soon see. The only change that I'm making really is changing the prototype for `main` from the first version, which we've been using for like a week and a bit now, to the second version, which is the only other version supported. I'm gonna go back to my terminal window now, make `greet`, and darn it, I shouldn't so close. Why did I make, uh, how do I fix the mistake I accidentally made? Yeah, and back. Oh, no, in front. Yes, I should have kept the `CS50` library

because it's in the CS 50 library that string is defined, so includes CS50. H in week 4 we will delete that line for real and actually show you what string actually is. I promised at the start of class that string is a term of art, but it's not a keyword in C, but it will see what it means in a couple of weeks' time. OK, let me fix this. Make greet.greet, but now I'm going to type before. I even hit enter my actual name, and when I hit enter now, I see hello David. If I instead do greet Kelly, enter, now I see hello Kelly. If I do nothing like greet enter, I just see hello null, which is not the same null as before NUL. This is NULL for reasons we'll come back to before long, but clearly Pri F knows something's going on. There's no actual word there. Why though, did I do RV1? Well, it turns out that just as a feature of C, if I recompile this program and do slash greet and type in nothing else, I'm gonna see something kind of curious. Hello. greet because automatically the zero location in the RVR variable will automatically contain the program's own name. Why is this useful? If you ever want to do something self-referential like thanks for running my program or you want to show documentation for your program and the name of your program, that it depends on whatever the file itself is called, you can use RV 0, which will always contain the program's name no matter what the file. Has been named or renamed to, but we can fix that null issue now in a couple of ways. So RC is the other input that I said now can exist, which is the count of arguments at the prompt. So if I want to check if the user actually typed their name, I could say something like if R C equals equals 2, well then and only then go ahead and print out their name else let's just do some clever default like print F, quote unquote hello world or heck, nothing at all. This. Version of the program now is a little smarter because when I run Make greet and do greed of my name, it works exactly as intended. But if I forget and only do greed, it's going to say hello world. Moreover, if I don't quite cooperate and I say David Malan, enter, it similarly just ignores me because our count is not 2 anymore. It's now 3. So RC contains the total numbers of words at the prompt, but the first one is always the program's name. Question. Why do you Sorry, can you say that once a little louder? Why is it information that we just have, or? Oh, so the short answer is just because, like the definition of C, if you look at the documentation for C, you can either define Maine as taking no arguments with the word void, or you can specify that Ma can take two arguments, and the compiler and the operating system will just ensure that if you provide two, those two variables, RC and RV will be filled with those two values automatically. Someone else decided that though that's just the way it works. You can't come up, you can't put 3 there, you can't put 4 there, you can change the names of those variables, but not the types. Because of this convention. So there's one last feature of Maine then. It's the actual value with returns. Up until now, every program I've written starts with in Ma something. I mean something. What is that in? We have yet to use it. Technically, the value that Maine returns is going to be called a so-called exit status, which is a numeric status that indicates success or failure. Numbers are everywhere in the world of computing. So for instance, here's a screenshot from Zoom whereby Something goes wrong with Zoom, like you have bad internet connectivity or something like that, you might see an error code like 1132. That means nothing to normal people unless you Google it, look up the documentation, but it means something very much to the software engineers who wrote this code because they know, oh shoot, 1132 means this error, and they probably have a spreadsheet or a cheat sheet somewhere that converts those codes to actually useful error messages and frankly, in a better world. They would just tell you what the problem is rather than just say report the problem and mention this number. That said on the web, odds are you're familiar with this number 404, which

is also a weird thing for so many normal people to know, but this generally means file not found. It's a numeric code that signifies that something has gone wrong. Exit status isn't quite this, but it's similar in spirit. In Maine, you can return a value like 0 or 1 or 2 or something else. to indicate whether something was successful or not. By convention, a program, a function like main returns 0 on success if all is well, and that leaves you then with like several 100 possible things that can go wrong because you could return 1 to signify one thing, 2 to return another, 3 to signify another, and so long as you have a spreadsheet or a cheat sheet or something, you can just keep track as the programmer as to what error means what. So what does this mean in real terms? Well, if I go over to VS code here, let me implement a relatively simple program, our last called Status.c. So in status.c, I'm going to go ahead and use the CS 50 library at the top, the standard IO library at the top, and then inside of Int Main and with our new format. RC string arg square brackets inside of main I'm going to now do the following. If RGC does not equal 2, then I'm going to go ahead and print out this time a warning. I'm not going to have some silly default like he world. Let's tell the user that they didn't use my program correct, and I'm going to say print F missing command line argument. And we'll assume they know what that means. Then to signify an error, I'm going to say return 1. It could be 2, it could be 3, but this is the first possible error, so I'm going to start simple with 1. Otherwise, if RXC does equal 2, and I get to this part of my code, I'm going to say hello SN and pass in RV 1 just like before. And just to be super specific, I'm going Return 0 to tell the computer, the operating system, that this is success. 0 signifies success. Any other value signifies error. Let's make status now. Let's do status, and this is a little magical, but let me go ahead and cooperate initially. I'm going to type in my name David and I'm going to see hello David. Most people wouldn't know this, but among the commands you can type at your Terminal are this one here and the TFs and the TAs and I would do something like this. We, after running your code, can do echo, space, dollar sign question mark, and we can see secretly the return value that your program returned 0 in this case. Meanwhile, if we do this again, status status, and let me not type my name this time. When I do this, I see missing command line argument. What value should the code have returned then? One, so let's see, echo dollar sign question mark, there's the one. So even after just one week of CS 50, if you've ever wondered how Check 50 knows if your code was correct or not, among the ways we check for that is by checking the semi-secret status code, this exit status, which is. really a secret. It's just not displayed to normal people because it's not all that enlightening unless you're the software developer who wrote the code in question. But this means we could return one in some cases or 2 in other cases or 3 or 4 and yet others. And these command line arguments are sort of everywhere and in fact. A program I skipped over a moment ago was going to be this. There's no academic value to what you're about to see, but another program that takes command line arguments is known as CAOSay, and this is sort of very famous in computing circles because it's been on systems for many years. CASay is a program that allows you to type in a word after the prompt like moo, and it will print out what's called Askki Art, an adorable little cow with a speech bubble that says moo, so kind of evocative of like scratch. But it takes other command line arguments, not just the words that you want to come out of its mouth, but even the appearance that you want it to have. So for instance, I can say dash F duck and run it again enter, and now I have a little cute duck saying mu, which is a bit of a bug. So let me change that to quack for instance instead. And again, no academic value here. It's just fun to now play with the various options. But if we really want to have fun with this,

we can do another one, so. I say dragon and we can say something like R and now we have this crazy dragon appearing on the screen, which is to say again, no value here. It's just fun to play with command line arguments sometimes and how is CO say doing this? Well, someone wrote code maybe in C or some other language using RGC and RV and poking around at their values and maybe a conditional that says if the F value is dragon, then print this graphic. Else if the value is duck, then print this other one, it all boils down to the same fundamentals of week zero of functions and conditionals and loops and boolean expressions and the like. It's just being composed into more and more interesting things. And indeed, in closing among the other interesting things we'll play with this week to come full circle is that of cryptography, the art of scrambling information so as to have secure communication. So important nowadays with passwords and credit card numbers and personal messages that you might want to send. And we'll have you explore through code some of the algorithms via which you yourselves can encrypt information. And there's a number of ways we can do this form of encryption, and they all boil down to this mental model. You've got some input like the message you want to send and you want to encipher it somehow, encrypt it somehow so that No one knows what message you've sent, so you want your plain text, which is the human readable version in English or any other language, to become ciphertext ultimately. So the code you'll be writing this week is inside of this black box some kind of cipher, an algorithm that encrypts information. So that you can do exactly this. Now the catch is that you can't just give it plain text and run it through an algorithm and get ciphertext because you need to somehow have a secret typically for encryption to work. Like if I'm going to send a message to someone in back, well, I could just randomize the letters that I'm writing down, but how would they know how to reverse that process? Probably what we need to do is agree in advance that, you know what, I'm going to change every A to a B. And every B to a C and a C to a D and a Z to an A. I'll wrap back around at the end of the alphabet. It's not very sophisticated, but who no middle school teacher if they intercept two kids passing notes in class, are going to waste time trying to figure out this cipher, but it does presuppose that there's a secret between them, the number one in that case, because I'm changing every letter by one place. So how might this work? Well, if I want to encrypt the word. Him and my secret key with someone that I've come up with in advance is one, I should send the cipher text IJ. Now this is a simple cipher, so I'm not really encrypting the punctuation, which may or may not be a good thing, but I am encrypting at least the alphabetical letters. But what does the recipient then have to do to decrypt this message when they see on paper IJ, how do they know what I said? Well, they use that same key, but subtract, so B becomes A, C becomes B, A becomes Z, and so forth, essentially inverting the key from positive one. To -1, of course, slightly more secure than a cipher of 1. A key of one would be 13. And in fact in computing circles, 13 has special significance. Rot 13 ROT 13 is an algorithm that's been used for many years online just to sort of avoid spoilers like Reddit might do this or other websites where they want you to have to do some effort to see what the message says, but it's not all that. Hard, you just have to click a button or write the code that actually does this, but if you use 13 instead, you wouldn't get you wouldn't get IJ, you'd get UV because you and V are 13 places away from H and I respectively. But again, we're not touching the punctuation or we could send something more personal like I love you and the message comes out like that. Slightly more secure than that would be rot 26, no. No, why? Because it's the same thing. It literally rotates all the way around A becomes A, B becomes B. So there's a limit to this. But more seriously, that speaks to just

how strong this encryption is or is not, because if you think about this now from an adversary's perspective, like the teacher in the room intercepting the slip of paper, how much work do they need to do? Well, they just try all possibilities key of 1, key of 2, key of 3, 25, and at some point. will see clearly that they guessed the key, which means that cipher is not very secure.

Nonetheless, what we're talking about is historically known as the Caesar cipher because back in the day when Caesar was communicating by by legends with his generals, if you're the first human on Earth to come up with encryption or come up with this specific cipher, it doesn't really matter how complex it is if no one else knows what's going on nowadays it's not hard at all to write some C code or any other language. Which they could just brute force their way through this. So there are much more sophisticated algorithms nowadays than simple rotations of letters of the alphabet, as we'll soon see. But when it comes to decryption, it really is just a matter of reversing that process. So this message here, if we rotate all the letters in the opposite direction by subtracting one, will be our final letters for today. There's a bit of a hint there which will reveal that this message and our final words for us as the clock strikes 4:15 is going to be the you becomes. T and the I becomes H. I'm, I'm the only one this is amusing. H I I S W A S C S 50 and this was CS 50. We'll see you next time.

Week 3

Why All right, this is CS 50. This is week 3, and this was an artist's rendition of what various sorting algorithms look and sound like. Recall from week 0 that an algorithm is just step by step instructions for solving some problem to sort information, as in the real world just means to order it from like smallest to largest or alphabetically or some other heuristic. And it's among the algorithms that we're going to focus on today in addition to searching, which of course is looking for. Information as we did in week 02. Among the goals for today are to give you a sense of certain computer science building blocks like there's a lot of canonical algorithms out there that most anyone who studied computer science would know, anyone who leads a tech interview would ask. But more importantly, the goal is to give you different mental models for and methodologies for actually solving problems by giving you a sense of how these, uh, real world algorithms can be translated to actual computers. You and I can control. We thought we'd begin today with an actual algorithm for sort of taking attendance. We of course do this with scanners outside, but we can do it old school whereby I just use my hand or my mind and start doing 123456789, 1011, 12, and so forth. That's going to take quite a few steps because I've got to point at and recite a number for everyone in the room so I could kind of do what my like grade school teachers taught me, which is count by 2s, which would seem to be faster. So like 2468, 1012, 1416, 1820, and clearly that sounds and is actually faster, but I think with a little more intuition and a little more thought back to week zero, I dare say we could actually do much better than that. So if you won't mind, I'd like you to humor us by all standing up in place and think of the number one, if you could, and join us in this here algorithm. So stand up in place and think of the number 1. So at this point in the story, everyone should be thinking of the number 1. Step 2 of this algorithm for you is going to be this pair off with someone standing, add their number to yours, and remember the sum. Go OK. At this point in the story, everyone, except maybe one lone person if we've got an odd number of people in the room, is thinking of what number? 2, OK, so next step, one of you in each pair should sit down. OK. Good. Never seen some people sit down so fast. So, those of you who are still standing, the algorithm's still

going. So the next step for those of you still standing is this, if still standing, go back to step 2. Ergo repeat or loop if you could. Um And notice if you've gone back to step 2, that leads you to step 3, that leads some of you to step 4, which leads you back to step 2. So this is a loop. Keep going. If still standing, pair off with someone else, still standing, add together, and then one of you sit down. So with each passing second, more and more people should be sitting down. And fewer and fewer are standing. OK. Almost everyone is sitting down. You're getting farther and farther away from each other. That's OK. I can help with some of the math at the end here. All right. I see a few of you still standing, so I'll help out and I'll I'll join you together. So I see you in the middle here. What's your number? 32. OK, go ahead and sit down and I'll pair you off with what's your number? 20. OK, you can go ahead and sit down. Uh, who's still Then you're still standing. 27? OK, you can sit down. You guys are still adding together, who's gonna stay standing? OK, what's your number? The worst part is doing like arithmetic across a crowded room, but 27 also? 47. OK, you can sit down. Is anyone still standing? Yeah. Nice, 15. OK, you can sit down. Anyone still standing? OK, so all I've done is sort of automate the process of pairing people up at the end here. When I hit enter, we should hopefully see, oh, the numbers are a little, what's going on there? There we go. When I hit enter, we'll add together all of the numbers that were left. And if you think about the algorithm that we just executed, each of you started with the number 1 and then half of you handed off your number. Then half of you handed off your number, then half of you handed off your number. So theoretically all of these ones with which we started should be aggregated into the final count, which if this room weren't so big, would just be in one person's mind, and they would have declared what the total number of people in the room is. I'm going to speed that up by hitting enter on the keyboard, and if your execution of this algorithm is correct, there should be. 141 people in the room. According to our old school human though, Kelly, who did this manually one at a time, the total number of people in the room, according to Kelly, if you want to come on up and shouted into the microphone is of course going to be, I don't know, something around 160, I think 160, so not quite the same. OK, but that's pretty good. OK, round of applause for your, your accuracy. OK, so ideally counting one at a time would have been perfectly correct. So we're only off by a little bit. Now, presumably that's just because of some bugs in execution of the algorithm, maybe some mental math didn't quite go according to plan, but theoretically your third and final algorithm wherein you all participated, should have been much faster than my algorithm or Kelly's algorithm, whether or not we were counting one at a time or two at a time. Why? We'll think back to week zero when we did the whole phone book example, which was especially fast in it. Final form because we were dividing and conquering, tearing half of the problem away, half of the problem away. And even though it's hard to see in a room like this, it stands to reason that when all of you were standing up, we took a big bite out of the first problem and half of you sat down, half of you sat down, half of you sat down, and theoretically there would have been if you were closer in space, one single person with the final count. So let's see if we can analyze this just a little bit by considering We did. So here's that same algorithm. He recall is how we motivated Week zero's demonstration of the phonebook in either digital form, as you might see in an iPhone or Android device, looking for someone, for instance, like John Harvard who might be at the beginning, middle, or end of said phonebook, but we analyzed that algorithm just as we can now this one. So in my very first verbalized algorithm, 1234, you could draw that as a straight line because the relationship between the number of people in the room and the of time it takes is

linear. It's a straight line with each additional person in the room. It takes me one more step. So if you think to sort of high school math, there's sort of a slope of one there. And so this N number denoting number of people in the room is indeed a straight line. And on the x axis, as in week 0, we have the size of the problem in people and the time to solve in steps or seconds or whatever your unit of measure is. If and when I started counting 2 at a time, 2468, 10. And so forth. That still is a straight line because I'm taking two bytes consistently out of the problem until maybe the very end where there's just 1 person left, but it's still a straight line, but it's strictly faster. No matter the size of the problem, if you sort of draw a line vertically, you'll see that you hit the yellow line well before you hit the red line because it's moving essentially twice as fast. But that third and final algorithm, even though in reality it felt like it took a while and I had to kind of bring us to the exciting conclusion by doing some of the math. That looked much more like our 3rd and final phone book example because if you think about it from an opposite perspective, suppose there were twice as many people in the room. Well, it would have taken you all theoretically just one more step. You know, granted one more loop and there might be some substeps in there if you will, but it's really just fundamentally one more step. If the number of people in the room quadrupled, 4 times as many people, well, that's 2 more steps. equivalently, the amount of time it takes to solve the Attendance problem using that 3rd and final algorithm grows very slowly because it takes a huge number of more people in the room before you even begin to feel the impacts of that growth. And so today indeed, as we talk about not only the correctness of algorithms, we're going to talk about the design of algorithms as well, just as we have code because the smarter you are with your design, the more efficient your algorithms ultimately are going to be and the slow. Where their cost is going to grow and by cost I mean time like here maybe it's money, maybe it's the amount of storage space that you need. Any limited resource is something that we can ultimately measure, and we're not going to do it very precisely. Indeed, we're going to use some broad strokes and some standard mechanisms for describing ultimately the running time, the amount of time it takes for an algorithm or in turn code to actually run. So how can we do this? Well, last week we're called, we set the stage. for talking about something called arrays which were the simplest of data structures inside of a computer where you just take the memory in your computer and you break it up into chunks and you can store a bunch of integers, a bunch of strings, whatever, back to back to back to back, and that's the key characteristic for an array. It is a chunk of memory wherein all of the values they're in are back to back to back, so right next to each other in memory. So we drew this fairly abstractly by drawing a grid like this and I said, Well, maybe this is byte 0 and this is by. 1 billion, whatever the total amount of memory is that you have, we zoomed in and looked at a little something like this a canvas of memory. We talked about what and where you can put things, but today let's just assume that we want 1234567 chunks of memory for the moment, and inside of them we might put something like these numbers here. Well, the interesting thing about computers is that even though if I were to ask you all find the number 50 in this array, I mean, our eyes quickly. See where it is because we sort of have this bird's eye view of the whole screen and it's obvious where 50 is, but the catch with computers and with code that we write is that really these arrays, these chunks of memory, are equivalent to a whole bunch of closed doors and the computer can't just have this bird's eye view of everything. If the computer wants to see what value is at a certain location, it has to do the metaphorical equivalent of going to that location, opening the door, and looking. Then closing it and moving on to the next. That is to

say, a computer can only look at or access one value at a time. Now that's in the simplest form. You can build fancier computers that theoretically can do more than that, but all of the code we write generally is going to assume that model. You can't just see everything at once. You have to go to each location in these here lockers, if you will. Starting today too, when we talk about the locations in memory, we're going to use our old zero indexing vernacular. That is to say we start counting from 0 instead of 1. So this will be locker 0, locker 1, locker 2, all the way up to locker 6. So just ingrained in your mind that if you hear something like location 6, that's actually implying that there's at least 7 total locations because we started counting at 0. So that's intentional. We don't have in the real world yellow lockers, so we're going to make this metaphor red instead. We do have these lockers here and suppose that within these 7 lockers physically. On stage we've put a whole bunch of money, monopoly money, if you will, but the goal initially here is going to be to search for some specific denomination of interest and use these physical lockers as a metaphor for what your computer is going to do and what your code ultimately is going to do if we're searching for the solution to a problem like this. The input to the problem at hand is 7 lockers, all of whose doors are metaphorically closed, the output of which we want to be a bull, true or false answer. Yes or no, that number is there or no, it is not. So inside of this black box today is going to be the first of our algorithms, step by step instructions for solving some problem where the problem here is defined among all of these dollar bills, specifically the \$50 bill. If we could get two volunteers to come on up who are ideally really good at monopoly. OK, how about over here in front and uh how about let me look a little farther and back. OK, over here, there and back. Come on down. All right, as these volunteers kindly come down to the stage, we're going to ask them in turn to search for specifically the \$50 bill that we've hit in advance. And if my colleague Kelly could come on up too because we're going to do this twice, once searching in one with one algorithm and a second time with another. Let me go ahead and say hello if you'd like to introduce yourselves to the group. Hey, I'm Jose Garcia. Hi, I'm Caitlin Cow. All right, Jose and Caitlin, nice to meet you both. Come on over and let me go ahead and propose that Jose, um, the first algorithm that I'd like you to do is to find the number 50 and let's keep it simple. Just start from the left and work your way to the right. And with each time you open the door, stand over to the side so people can see what's inside and just hold the dollar amount up for the world to see. All right? The floor is yours. Find us the \$50 bill. 20 No, that's good. That's good acting too. Thank you. No, you can shut it just like the computer, all right. No, very clear, thank you. Mm, still no \$10 bill. Next locker. \$5 bill, not going well. \$100 bill, but not the one we want. This 1 \$1 bill, still no \$50. Of course, you've been sort of set up to fail, but here, amazing, a round of applause, Jose found the \$50 bill. All right, so let me ask you, Jose, you found the \$50 bill. Um, it clearly took you a long time. Just describe in your own words what was your algorithm, even though I nudged you. Yeah, so my algorithm was basically walk up to the first door available, open it, check if the dollar bill was the dollar bill that I was looking for, and then put it back and then go. The next one. OK, so it's very reasonable because if the \$50 bill were there, Jose was absolutely going to find it eventually, if slowly. In the meantime, Kelly's going to kindly reshuffle the numbers behind these doors here. And even though Jose took a long time here, I mean, what if Jose like wouldn't have been smart to start from the other end instead, do you think? Um, not necessarily because we don't know if the 50 is going to be at that end. Exactly. So he could have gotten lucky if he sort of flaunted my advice and didn't start on the left, but instead started on the right, boom, he would have solved this in one step. But in

general, that's not really going to work out. Maybe half the time it will, you'll get lucky, half the time it won't, but that. Not really a fundamental change in the algorithm whether you go left to right, right to left, to Jose's point, if you don't know anything a priori about the numbers, the best you can probably do is just go through linearly left to right or right to left, so long as you're consistent. Now, could you have jumped around randomly? Uh, I guess I could have, but if again, if they weren't in any like specified order, I don't think it would have helped either. Yeah, so and additionally, if he just jumped around to random order, then you might get lucky and it might be in the very first one, might have taken fewer steps ultimately, but presumably you're going to have to then keep track of like which locker doors have you opened. So that's going to take some memory or space. Not a big deal with 7 lockers, but if 70 lockers, 700 lockers, even random probably isn't going to be the best job. So let me go ahead and take the mic away and hand it over to Caitlin. You can stay on the stage with us. Caitlin, what I'd like you to do is approach this a little more intelligently by dividing and conquering the problem, but we're going to give you an advantage over Jose. Kelly has kindly sorted the numbers from smallest to largest from left to right. So accordingly, what's your strategy going to be? Start in the middle, OK, please. And go ahead as before and reveal to the audience what you found, not the 50, the 20, but what do you know, Caitlin, at this point? Correct. So the 20 is going to be to the left. So where might you go next with this 3 locker problem? Let me propose that you maybe go to the middle of the 3. There we go, the middle of the middle. Like that would have been good, but let's. Oh no, oh no, it's up 100 instead. You failed, but what do you know? It's in the middle that I should have just let you, but. Now we have a big round of applause for Kaitlin for having found the 50 as well. OK. So the one catch with this particular demo is that because they know presumably what monopoly money denominations are, because we just did this exercise and we had the whole cheat sheet on the board, you probably had some intuition as to where the 50 was going to be, even though I was trying to get you to play along. But in the general case, if you don't know what the numbers are and that they are the specific denominations, but you do know that they're going from smallest to largest, going to the middle, then the middle of the. Middle, then the middle of the middle again and again would have the effect of starting with a big problem and having it, having it, having it just like the phone book as well. So thanks to you both, we have these wonderful parting gifts that we found in Harvard Square. If you like Monopoly, you'll love the Cambridge edition filled with Harvard Square name spots. So thank you to you both, and a round of applause for our volunteers here. All right, so let's see if we can't formalize a little bit these two algorithms known as linear search insofar as Jose was searching essentially along a line left to right, and binary search by implying 2 because we were having that problem in 2 again and again and again. So for instance. Linear search from left to right or equivalently right to left, we could document our pseudo code as follows. For each door from left to right, if the 50 is behind the door, well then we're done. Just return true. That's the boolean value which was the goal of this exercise to say yes, here is the 50. Otherwise, at the very bottom of this pseudo code, we could just say return false because if you get all the way through the lockers and you have never once declared true by finding the 50, you might as well default at the very end to saying false, I did not find it. But notice here, just like in week zero when we talked about pseudo code for. Searching the phone book, my indentation of all things is actually very intentional. This version of this code would be wrong if I instead used our old friend if else and made this conditional decision. Why is this code now in red wrong in terms of

correctness? Yeah. Exactly, because if the number 50 is not behind the first door, the E is telling you right then and there return false. But as we've seen in C code, whenever you return a value like that's it for the function, it is done doing its work. And so if you return false right away, not having looked at the other six lockers, you may very well get the answer wrong. So the first version of the code where there wasn't an L but rather this implicit line of code at the or this explicit line of code at the very end that just says if you. Reach this line of code returned false, that addresses that problem. And to be clear, even though it's right after an indented return true when you return a value as in C, that's it like execution stops at that point, at least for the function or in this case the pseudo code in question. All right, so here's a more computer sciencey way of describing the same algorithm and even though it starts to look a little more arcane, the reality is when you start using variables and sort of standard notation you can actually. Express yourself much more clearly and precisely, even though it might take a little bit of practice to get used to. Here is how a computer scientist would express that exact same ideas instead of saying for each door from left to right we might throw some numbers on the table. So for I, a variable apparently from the value zero on up through the value and minus 1 is what this shorthand notation means. If 50 is behind doors bracket I, so to speak, so now. Sort of treating the notion of doors as an array using our notation from last week. If 50 is behind doors bracket I return true. Otherwise, if you get through the entirety of that array of doors, you can still return false. Now notice here N minus 1 seems a little weird because aren't there end doors? Why do I want to go from 0 to N minus 1 instead of 0 to end? Yeah. Because um 0 is the first. Exactly, if you start counting at 0 and you have n elements, the last one is going to be addressed as n minus 1, not N, because if it were N, then you actually have N + 1 elements, which is not what we're talking about. So again, just a standard notation and it's a little terser this way. It's a little more succinct and frankly it's a little more adaptable to code. And so what you're going to find is that as our problem sets and programming challenges. We assign sort of get a little more involved. It's often helpful to write out pseudo code like this using an amalgam of English, and C, and eventually Python code, because then it's way easier after to just translate your pseudo code into actual code if you're operating at this level of detail. All right, so in the second algorithm where Caitlin kindly searched for 50 again, but Kelly gave her the advantage of sorting the numbers. Advance now she doesn't have to just resort to brute force, so to speak, trying all possible doors from left to right. She can be a little more intelligent about it and pick and choose the locker she opens. And so with binary search as we call that, we could implement the same pseudo code or we could implement pseudo code for it as follows. We might say if 50 is behind the middle door, then go ahead and return true. Else if it's not behind the middle door, but 50 is less than that number behind the middle door, we want to go and search the left half so that didn't happen in Caitlin's sense because we ended up going right, so that's just another branch here. LA 50 is greater than what was at the middle door. We want to search the right half, but there's going to be one other condition here that we should probably consider, which is what? Is it here, is it to the left, or is it to the right? But there's another a corner case that we'd better. Keep track of what else could happen if it's not in the ray or really like we're out of doors so we can implement this in a different way. I left myself some space at the top because I shouldn't do any of this if there are no doors to search for, so I should have this sort of sanity check whereby if there's no doors left or no doors to begin with, let's just immediately return false. And why is that? Well, notice that when I say search left half and

search right half, this is implicitly telling me just do. This again, just do this again, but with fewer and fewer doors, and this is a technique for solving problems and implementing algorithms that we're going to end today's discussion on because what seems very colloquial and very straightforward, OK, search the left half, search the right half, is actually a very powerful programming technique that's going to enable us to write more elegant code, sometimes less code, to solve problems such as this and more on that in just a little bit. But how can we Now formalize this using some of our array notation. Well, it looks a little more complicated, but it isn't really. Instead of asking questions in English alone, I might say if 50 is behind doors bracket middle. This soda code presupposes that I did some math and figured out what the numeric address, the numeric index is of the middle element and how can I do that? Well, if I've got 7 doors and I divide by 2, what's that? 7 divide by 2. 3.5, 3.5 makes no sense if I'm using integers to address this, so maybe we just round down, so 3, so that would be locker number 0123, which indeed if you look at the seven lockers is in fact the middle. So this is to say using some relatively simple arithmetic, I can figure out what the address is, the index is of the middle door. How many there are and I divide by 2 and round down. Meanwhile, if I don't find 50 behind the middle door, let's ask the question. If 50 is less than the value at the middle door, then let's search, not the left half per se in the general sense, more specifically search doors 0 through doors bracket middle minus 1. Otherwise, if 50 is greater than the value at the middle door, go ahead and search doors bracket middle plus 1 through doors bracket and minus 1. Now let's consider these in turn. So searching the left half, as we described this earlier, seems to line up with this idea like start start searching from doors 0, the very first one, but why are we searching doors bracket middle minus 1 instead of doors bracket middle? Yeah. Yeah, exactly. We already checked the middle door by asking this previous question, so you're just wasting everyone's time if you divide the half and still consider that door as checkable again. And same thing here. We check middle + 1 through the end of the locker's array because we already checked the middle one. So same reason, even though it just kind of complicates the look of the math, but it's really just using variables and arithmetic to describe the locations of these same lockers. But let's consider. Now what we mean by running time, the amount of time it takes for an algorithm to run, and consider which and why one of these algorithms is better than the other. So in general, when talking about running time, we can actually use pictures like this. This is not going to be some like very low level mathematical analysis where we count up lots of values. It's going to be broad strokes so that we can communicate to colleagues, to other humans, generally, whether an algorithm is better than another and how you might compare the two. So here for instance is a pictorial analysis of two different algorithms the phone book from week zero and then the attendance taking from today itself. And let's generally, as we've done before, sort of label these things. So the very first algorithm took end steps in the very worst case if I had to search the whole phone book or if I had to count everyone in the room. So the first algorithm took indeed end steps. The second algorithm took half as many plus 1 maybe, but we'll keep it simple. So we'll call that $N/2$. The third and final algorithm both in week 0 with the phone book and today with attendance is technically $\log_2 N$. And if you're a little rusty in your logarithms, that's fine. Just take on faith that $\log_2$ alludes to taking a problem of size N and dividing it in half and half and half as many times as you can until you're left with one person standing or one page in the phone book. That's how many times you can divide in half a problem of size N . Well, it turns out that we're a little more detailed than most computer

scientists care to get when describing the efficiency of algorithms. So in fact we're going to start to use some notation. Instead of worrying precisely mathematically about how many steps today's and the future's algorithms take, we're going to talk in broader strokes about how many steps there are on the order of, and we're going to use what's called big O notation, which literally is like a big O and then some parentheses, and you pronounce it big. of such and such. So the first algorithm seems to be in big O of n , which means it's on the order of N steps, give or take some. This algorithm here, you might be inclined to do something similar. It's on the order of N divided by 2 steps, and this one's on the order of log based 2 of N steps, but it turns out what we really care about with algorithms is how the time grows as the problem it. Self grows in size. So the bigger end gets, the more concerned we are over how efficient our algorithm is. If only because today's computers are so darn fast, whether you're crunching 1000 numbers or 2000 numbers, like it's going to take like a split second no matter what. But if you're crunching 1000 numbers versus a million numbers versus a billion numbers, like that's where things start to actually be noticeable by us humans and we really start to care about these values. So in general, when using big O. like this, you ignore lower order terms or equivalently you only worry about the dominant term in whatever mathematical expression is in question. So big event remains big event. big O of N over 2, it's the same thing really as like big O over n . Like it's not really, but they're both linear in nature. One grows at this rate, one grows at this rate instead, but it's for all intents and purposes the same. They're both growing at a constant rate. This one too uh it's on the order of log of n where the base is who cares? In short, what does this really mean? Well, imagine in your mind's eye that we were about to zoom out on this graph such that instead of going from 0 to like a million, maybe now the x axis is 0 to a billion, and same thing for the y axis 0 to a million. Let's zoom out so you're seeing. 0 to a billion. Well, in your mind's eye, you might imagine that as you zoom out, essentially things just get more and more compressed visually because you're zooming out and out and out, but these things still look like straight lines. This thing still looks like curved lines, which is to say as n gets large, clearly this green algorithm, whatever it is, is more appealing, it would seem than either of these two. Algorithms and if we keep zooming out like at some point the ink is going to be so close together that they all are for all intents and purposes pretty much the same algorithm. So this is to say computer scientists don't care about lower order terms like divide by 2 or base 2 or anything like that. We look at the most dominant term that really matters is N gets bigger and bigger. So that then is big O notation. And it's something we'll start to use pretty much recurringly any time we analyze or speak to how good or how bad some algorithm is. So here's a little cheat sheet of common running times. So for instance, here's our friend big O of N , which means the algorithm takes on the order of end steps. Here is one that takes on the order of log N steps. Here are some others we haven't seen yet. Some algorithms take n times log n steps. Some algorithms take squared steps, and some algorithms just take one step, maybe, or maybe 2 steps or 4 steps or 10, but a constant number of steps. So let me ask, if the algorithms we've looked at thus far, for instance, linear search being the very first today, what is the running time of linear search in big O notation? That is to say, if there's people, or if there's N lockers on the stage, how many steps might it take us to find a number among those N lockers? They go of, yeah. Big O of N in fact is exactly where I would put linear search. Why? Well, if you're using linear search in the very worst case, for instance, the number you're looking for, as with Jose, might be all the way at the end. So you might get lucky. It might not be at the very end, but

generally it's useful to use this big O notation in the context of worst case scenarios because that really gives you a sense of how badly this algorithm could perform if you just get really unlucky with your data set. So even though BigO really just refers to an upper bound, like how many steps might it take, it's generally useful to think about it in the context of like the worst case scenario, like, ah, the number I care about is actually way over here. But what about binary search? Even in the worst case, so long as the data is sorted, how many steps might binary search take by contrast? Big O of log in. So binary search we're going to put here, which is to say that in general, and especially as N gets large, binary search is much faster. It takes much less time. Why? Because assuming the numbers are sorted, you will be dividing in half and half and half, just like with the phone book in week 0, that problem, and you will get to your solution much faster. Why should you not use binary search though on an unsorted array of lockers? Like a random set of numbers, yeah. Exactly, you're making these decisions based on inequalities less than or greater than, but based on like no rhyme or reason. You're going left going right, but there's no reason to believe that smaller numbers are this way and bigger numbers are that way. So you're just making incorrect decision after incorrect decision, so you're probably going to miss the number altogether. So binary search on an unsorted array is just incorrect, incorrect usage of the algorithm. But like Kelly did, if you sort the data in advance or you're handed sorted data, well then you can in fact apply binary search perfectly and much more efficiently. Sure there is more efficient. Absolutely. Is Linear search sometimes more efficient if it's going to take you more time to sort the data and then use binary search? Absolutely. And that's going to be one of the design decisions that underlies any implementation of an algorithm because if it's going to take you some crazy long time, not to sort like 7 numbers, but 70, 700, 7000, 7 million, but you only need to search the data once, then what the heck are you doing? Like why are you wasting time sorting. Data, if you only care about getting an answer once, you might as well just use linear search or heck, do it even randomly and hope you get lucky if you don't care about reproducing the same result. Now, in general, that's not how much of the world works. For instance, Google's working really hard to make faster and faster algorithms because we are not searching Google once and then never again doing it. We're doing it again and again and again so they can amortize. So to speak, the cost of sorting data over lots and lots of searches, but sometimes it's going to be the opposite. And I think back to graduate school, where I was often writing code to analyze large sets of data, and I could have done it the right way, sort of the CS 50 way, by fine tuning my algorithm and thinking really hard about my code, but honestly, sometimes it was easier to just write really bad but correct code, go to sleep for 7 hours, and then my computer would have the answer by morning. The downside, as admittedly happened more than once, is if you have a bug in your code and you go to sleep and then 7 hours later you find out that there was a bug, you've just wasted the entire evening, so there too a trade off sometimes when making those resource decisions. But that's entirely what today is about making informed decisions and sometimes maybe it's smarter and wiser to make the more expensive decision, but not unknowingly, at least knowingly. All right, so there we have our first two algorithms, but let's consider another way of describing the efficiency of an algorithm. Big O is an upper bound, sort of how bad can it get in these cases where maybe the data is really not working to our advantage. Omega, a capital Omega symbol here is used for lower bounds. So maybe how lucky might we get in the best case, if you will, how few steps might an algorithm take? Well, in this case here, here's just a cheat sheet of

common run times, even though there's an infinite number of others, but we'll generally focus on functions like these. Let's consider those same algorithms. So with linear search from left to right, how few steps might that algorithm take? For instance, in like the best case scenario. Yeah, is this uh hand about to go up? Yeah, so one step why because maybe Jose could have gotten lucky and opened the door and voila, that was the 50. It didn't play out that way, but it could have. In the general case, the number you're looking for could very well be at the beginning. So we're going to put linear search at omega of 1. So one step, and maybe it's technically a few more than that, but it's a fixed number of steps that Nothing to do with the number of lockers. Case in point, if I gave you not 7, but 70 lockers, you could still get lucky and still take just one step. So Omega is our lower bound, Big O is our upper bound. Ah, spoiler, what is binary search is lower bound. Well, apparently it's also omega of 1, but why? That is in fact correct, yeah. Same reason you could get lucky in the best case, and it's just smack dab in the middle of all of the data. So the fewest number of steps binary search might take is also actually one. So this is why we talk about upper bound and lower bound because you get kind of a, a sense of the range of performance. Sometimes it's going to be super fast, which is great, but something tells me in the general case we're not going to get lucky every time we use an algorithm, so it's probably going to be closer to those upper bounds, the big. Now, as an aside, there's a 3rd and final symbol that we use in computer science to describe algorithms, that of a capital theta. Capital theta is jargon you can use when big O and Omega happen to be the same, and we'll see that today, not always, but here's a similar cheat sheet. None of the algorithms thus far can be described in this way with theta notation because they are not all the same. With their big O and Omega, they differed in both of our analyses, but we'll see at least one example of one where it's like, OK, we can describe this in theta, and that's like saying twice as much information with your words to another computer scientist rather than giving them both the upper and the lower bounds. The fancy way of describing all of what we're talking about here, big O, omega, and theta is asymptotic notation, and asymptotic notation refer or. Asymptotic refers to a value getting bigger and bigger and bigger and bigger but not necessarily ever hitting some boundary as N gets very large. In short, is what we mean when we deploy this here asymptotic notation. All right, so with the first of these things like linear search, let's actually kind of make this a bit more real. Let me actually go over to in just a moment. Uh my other screen here, OK, in VS code, let me go ahead and create a program called Search.c. and in search.c, let's go ahead and implement a fairly simple version of Linear search initially. So let me go ahead and include, for instance, CS50.h. Let me go ahead and include stdio.h. Then let me go ahead and do main void, so we're not going to bother with any command line arguments for now. And then let me go ahead and just give myself an array of numbers to play with. And we did this briefly last week in answer to a question, but I'm going to do it now concretely rather than use something more manual to get all of these numbers into the array. I'm going to say give me an array called numbers, and the numbers I want to put in this array initially are going to be the exact same denominations we've been playing with 20, 500, 10, 5, 100, 1, and 50. Again, this is a notation that I alluded to in answer to a question last week whereby if you want. To statically initialize an array that is give it all of your values up front without having the human type them all in manually. You can use curly braces like this, and the compiler is pretty smart. You don't have to bother telling the compiler how many numbers you want 1234567 because it can obviously just count how many numbers are in the curly braces,

but you could explicitly say 7 there so long as you're counting is in fact correct. So on line 6 this gives me an array of 7 numbers. Initialize to precisely that list of numbers from left to right. All right, let's ask the human now what number they want to search for, just as I did our two volunteers and say in N equals get in, then let's just ask the user for the number that they want to search for. Then let's implement linear search. And if I want to implement linear search in terms of the programming constructs we've seen thus far, like what type, what keyword in C should I use? What programming technique, yeah. Yeah, so maybe a 4 loop or a Y loop, but 4 loop is kind of my go to lately. So let's do 4 in I equals 0 because we'll start counting from the left. I is less than 7, which isn't great to hard code, but I'm not going to use the 7 again, so I think it's OK in one place for this demo. Then I + plus, then inside of this array. Let's go ahead and ask a question just like Jose was by opening each of the doors by saying if numbers 1 equals equals the number we asked about N, well then let's go ahead and print out some informative message like found backslashn. And then for good measure, like last week, let's return 0 to signify success. It's sort of equivalent to returning true, but in main recall you have to return an in. That's why we revealed at the end of week 2 the return type of main is an in because that is what gives the computer its so-called exit status, which is zero if all is well, or anything other. 0 if something went wrong, but I think finding the number counts is all is well. But if we get through that whole loop and we still haven't printed found or returned 0, I think we can go ahead and safely say not found backslashn, and then let's just return 1 as our exit status to indicate that we didn't find the actual number. So in short, I think in See this is linear search. Let me open up my terminal window again. Let me make search enter. Let me do search enter, and I'll search for, as I asked Jose, the number 50, and we indeed found it at the end. Let me go ahead and rerunear and let's search for the other number at the beginning 20. That then works. And just to get crazy, let's search for a number we know not to be there like 1000. And that in fact is not found. So I think we have an implementation then of linear search, but let me pause here and ask if there's any questions with this here code and the translation of algorithm to. See, yeah, in the back. Why I did not specify the length of the array. So it is not necessary when declaring an array and setting it equal to some known values in advance to specify in the square brackets how many you have because like the compiler is not an idiot. It can literally count the numbers inside. Of the curly braces and just infer that value you could put it there, but arguably you're opening up the possibility that you're going to miscount and you're going to put 7 here but 8 numbers over there or 6 numbers there so it's best not to tempt fate and just let the compiler do its thing instead. A good question. Other questions on this code so far. All right, if none, let's go ahead and maybe convert this linear search to one that's maybe a little more interesting that involves like searching for strings of text. After all, we started the class in week zero by searching for names in a phone book like John Harbert. Let's see if we can adapt our code for searching for strings instead of integers. So in my code here, let's go ahead and delete everything inside of Maine just to give myself a clean canvas. Let me go ahead and give me another array, this one called, let's just call it strings because that's the goal of this exercise, and set them equal to some familiar pieces from the game of Monopoly if you might have played. So there's like a battleship piece in there. There's a boot in there, there's a cannon in there, an iron, a thimble, and a top hat, though it does vary nowadays based on the addition that you have. So kind of a long array, but I have 123456 total values in this array of strings. Now let's ask the user for a string. We'll call it S for short and say with get string, what string are you looking for among

those 6. Then I think we can do a 4 loop again for in I equals 0, I less than 6, I plus plus, and then inside of this loop, let's do the same thing if Uh, let's say strings bracket i equals equals the strings that the human typed in. I think we can go ahead and say print found backslashn and then as before return. 0 to signify success, and if we don't, after that whole 4 loop, let's print print F not found backslash and down here and return 1 to signify error. So it's really the same thing at the moment, except that I'm actually using strings instead. Integers. All right, let me go ahead and open up my terminal window again and clear it. Let me go ahead and recompile this code. MakeSearch.c. Seems to compile OK. Let me do search and let's go ahead and search for the first one. How about battleship enter. Huh, not found. All right. Well, let's maybe a typo, maybe. Let me search for something easier to spell, boot. Not found. That's weird. Both of those are at the very start of the array. Let's do slashes again and search for top hat. Enter. Not found. What is going on? Well, this isn't actually that obvious as to what I'm doing wrong, but it turns out that when we actually compare strings instead of integers and see, we're actually going to have to use this other library, at least today, that we saw briefly last week. Last week we introduced it because of a function called Stirling, which gives us the length of a string. Turns out that string. H also comes per its documentation with another useful function called Stircom for string. Compare and its purpose in life is to actually compare two strings left and right to make sure they are in fact the same. So for today's purposes, suffice it to say you cannot use equals equals apparently to compare two strings. Intuitively, why is that? Well, for a computer it's super easy to compare two integers because they're either there or they're not in memory. But with a string, it's not just a character and another character, it's like 7. Characters over here and a few characters over here. Maybe it's a few, maybe it's more, you have to compare each and every character in a string to make sure they're in fact the same. So Stir Compare does exactly that, probably in the implementation of Stir Co from like years ago someone wrote a wild loop or a floor loop that looks at each string left to right and compares each and every one of the characters they're in and then gives us back an answer. So how do we go about using this? Well, to use stir compare, what I can actually do in VS code here is go and change my code as follows. Instead of using equals equals, I'm going to actually use this function per its documentation. I'm going to call stir compare, then I'm going to pass in one of the strings, which is in strings bracket I. Then I'm going to pass in the second string, which is S. However, having read the documentation, and this is a little nonobvious, it turns out that Comp will return 0 if the strings are equal. Otherwise it's going to return a positive number or a negative number. So what I care about for now is does the return value of stir comp when given those two strings, give me back zero? If so, they are equal and I'm going to say unquote found. So let's go ahead and open the terminal again. Let me go ahead and clear it and do make search to recompile my code and huh. I've done something wrong. Let's see. Let me scroll up to the very first line in line 11 error call to undeclared library function stir comp would type in and something something which gets more complicated after that. Why is line 11 not working despite what I just preached? Yeah. Yeah, I just did something stupid. I didn't include the string.h header library, so all clang our compiler is doing when invoked by Make is it's encountering literally the word stir comp and not knowing what it is because we haven't taught it what it is by simply saying include string. H at the top. OK, let me reopen my terminal window, clear that message. Way to make search again. Now it's compiling search enter. Now I'm going to go ahead and search as I did before for Battleship. Ah, now it's finding it. Let me run ear again. Search for boot. Ah, OK, that's found.

Let me go ahead and search for top hat. That too is in there. Let me go ahead and search for something that's not there like the number 50. Not in fact found, so I think we've actually fixed that their problem. But if we go back to this code for a moment, it's indeed the case for the documentation that equals equals 0 is what I want to do. Why in the world would Stir comp be designed to return a positive or a negative number 2? It's not returning true or false, it's returning one of three possible values zero, negative or positive. Why might it be useful? Yeah. Um, you could kind of like compare which of the ring is like. Yeah, super clever. So if you're passing in two strings, it's great to know if they're equal, but wouldn't it be nice if this same function could also help us sort these strings ultimately and tell me which one comes first alphabetically? And technically it's not going to be alphabetically, it's going to be a cute phrase askibetically because it's actually going to look at the asy values of the characters and do some quick arithmetic and tell you which one comes first and which one comes later, which is enough. We'll eventually see for actually sorting these strings as well. So in short, the documentation will tell me that I should check not only for zero if I care about equality, but if I care about inequality, that's checking if one comes first or last, I should check whether something is less than 0 or greater than. But for this demonstration, implementing linear search, I don't care about comparing them for inequality. All I care about is that they are in fact the same or Not in this case. All right, well, let's go ahead and do one other example of sort of linear search, but let's make the problem more like that actually in week zero of searching a phone book. So let me go back to VS code here. Close search.c and let's make an actual phone book. So I'm going to say code of phonebook.c and then inside of phonebook.c let's use our same header file. So include CS50. H, includes standard IO. Dot and let's include an advance string.h. Then let's as before do in main void, no command line arguments today. Then inside of here, let me give myself first an array of strings. How about some names in the phone book? So I'm going to say string, names, equals, and then three names just to make a demonstration. Kelly and David and say John Harvard here. But if it's a phone book, I need more than just names. So let me go ahead and give myself another. Array string of numbers open bracket close bracket equals and now the same phone numbers we used in week zero for the three of us plus 1 617495 1000 same for both Kelly and me, so plus 1, 617495 1000. And then as before, if you'd like to text or call John directly, you can do so at plus 1-949-4682750 and semicolon. So one question first, I obviously declared our names to be an array of strings because that's what text is. Why have I also declared phone numbers to be strings and not integers because a phone number is like literally a number in the name of it, yeah. Yeah, so even though we have phone numbers in the US, even though we have Social Security numbers and a bunch of other things that we call numbers, if you have other non-digits in those, in those values, you have to actually use strings because if it's not an actual integer, but it does have things like pluses or dashes or parentheses or any other form of punctuation, as is common in the US and other countries for phone numbers in particular, you're going to actually want to use strings and not. as well as for corner cases like if there are if you're in the habit back home, if you're not from say the US and you actually have to dial 0 1st to make like a local regional call, you don't want to have a leading zero in an integer because mathematically, as we know from grade school like leading zeros, number 0s that come first have no mathematical meaning, they're going to disappear effectively from the computer's memory unless we store them in fact as characters in strings in this way. OK, with that said, let's go ahead and ask the human now after having declared those two

arrays for the name they want to look up the number of. So let's say string name equals get string, and let's go ahead and ask the human for the name for which to search. Then let's use a 4. As before, for in i equals 0, i less than 3, which again for demonstration purposes I'm just hard coding today i + plus and then in the 4 loop I'm going to use our new friend strcmp if the return value of strcmp passing in names bracket i. And the name the human typed in equals equals zero signifying that they are in fact the same. Well, that means we found the location i where the person's name is. So let's go ahead and print out found, but just to be fun, let's print out whom we found. So percent F. Backslash N and then output there the number which is going to be in the corresponding numbers array at that same location i will return 0 and at the very end of this program, let's go ahead and print out not found if we get that far and return 1. All right, so a little more complexity this time, but notice I'm comparing the names just like a normal person would in your iOS app or your Android app when looking for someone's name, but what I care about is getting back the number. So that's why two lines later, I'm printing out the number that I found at location i, not the name, because I already know the name. All right. In my terminal window, let's go ahead and make this phone book. phonebook. Let's go ahead and search for John, whose number is hopefully indeed exactly that number. So suffice it to say this code 2 does work. This is a linear search because I'm searching left to right. These aren't actually sorted alphabetically by name or let alone number. So I think we're doing well here but I don't necessarily love this implementation even if you're new to programming, what might you not like about how I've implemented a phone book in the computer's memory? Why is this maybe not the best design, yeah. OK, yeah, and I would say, so you're pointing out that we have this duality. We've got two arrays. They're the exact same length, and it just so happens that location zero's name lines up with location 0's number and location 1 and location 2, but we're kind of on the honor system here whereby the onus is on us to make sure we don't screw this up and we make sure we always have the same number of names and the same number of numbers and better and moreover that we may get the order exactly. right, we are just trusting that when we print out the IT number, so to speak, that it lines up with the IT name. So that's fine. And honestly for 3 people who really cares, it's fine, but if you think about 30 people, 300, 3 million, well we're not going to hard code them all here, but even in some database that will store them in later in the course feels like just trusting that we're not going to screw this up is asking for trouble. And indeed a lot of programming is just that, like not trusting yourself. And definitely not trusting your colleague not to mess something up, but programming a bit more defensively and trying to encapsulate related information a little more tightly together and not just assume, as on the honor system that these two independent arrays will line up. But at this point we have no means of solving this problem unless we give ourselves just a bit new functionality and syntax. So I used this phrase earlier to kick things off. Data structures. It's like how you structure your data in the computer's memory. Arrays are the simplest of data structures. They just store data back to back to back from left to right contiguously in memory, but they all have to be, as we've seen, the same kinds of values in, in, in, or string, string string. There's no mechanism yet for storing an in and a string together, and then another in and another string together, or let alone two strings, 2 strings, 2 strings that are somehow a little bit different. But it would be nice if C gave us an actual data type to store people in a phone book such that we could create an array called people inside of which is going to be a whole bunch of persons, if you will, back to back to back, and I want two of them. So wouldn't it be nice if I could

literally use this code and see? Well, decades ago when C was invented, they didn't give us a person data type. All we have is int and float and char and bool and string and so forth. Person was not among the available data types, but we can invent our own data types it turns out. So in see what we can do if we want persons to exist and every person in the world shall have a name and a phone number for now, we can do this. Name string number. Now that's a decent start, but it's going to be kind of a stupid implementation if I then just do name string name 1, string name 2, string name 3, string name 4. We've already started down that road last week and decided arrays were a better solution. But here's an alternative when you want to just store related data together. I can use these two keywords and see type struct, which albeit terse, just means define a new type that is a data structure. So multiple things together inside the curly braces you literally put the two things you want to relate together. Name string number and then outside the curly braces you specify the name you want to give to this brand new custom type that you have invented. Technically, stylistically, you'll see that C50 prefers that the name actually be on the same line as the last curly brace, which looks a little weird to me, but that's what industry tends to do so so be it. But these several lines together tell C invent for me a new data type called person and assume that every person in the world has a string called name and a string called number. And now I can use this new data type in my own code to solve this problem a little bit better. So in fact, let me go ahead and do this as follows. I'm going to go back to VS code here and at the very top of my code above main just to make this. Available to not only main but maybe any future functions I write. I'm going to say type struct, as we saw on the screen. Inside of my curly braces, I'm going to say string name and string number, and then I'm going to name this thing person. Now I'm going to go about using this and I'm going to go ahead and delete my previous hash system approach of having names and numbers in separate arrays, and I'm instead going to give myself an array of people. We can call it persons, but I'm trying to be somewhat grammatically correct, so I'm going to say people bracket 3 to give myself an array, call people inside of which is room for 3 persons, inside of which is room for a name and number each. So how do I now initialize these values? Well, I'm going to hard code them, that is type them manually, but you can imagine using get_string or get or some other function to get this data from the human themselves. I'm going to say go to the people array at location 0. And access the name field, and this is syntax we haven't seen yet, but it's not that hard. You literally use a dot, a single period, to say go inside of that structure and access the name field, the name attribute, so to speak, and let's set that equal to Kelly. Then let's go into that same array location, people bracket 0 and set the number for the zeroth person to be plus 1617495 1000. Then let's go ahead and do the same thing for people 1. Set that person's name to, for instance, mine, David. Then let's do people bracket 1.n number equals quote unquote, same as Kelly because we're both in the directory. So plus 1 617495 1000. And then lastly, people bracket 2.name equals quote unquote for John Harvard, people 2.n number equals plus 1. 949-4682750 in this case. And now the rest of the code is almost the same. I'm going to now on the new line 24 still ask the user what name they want. I'm going to still iterate from 0 to 3 because there's still 3 elements in this array, even though each has two values within, and I'm Compare now not names, but people bracket i.name to go access the name of that person and compare it to the name that the human has typed in. And when I find that person, I'm going to go into the people array at location i but print out the number instead. So all we've done here is add this dot notation which allows you to access the inside of a data

structure, and all we've done is introduce up here some new C keywords that let you invent your own data types inside of which you can put most anything you want. I have chosen a string name and a string number. All right, let me go ahead and open my terminal window and clear it from before. Let me do make Phonebook to make this version. So far so good. Make Phonebook enter. I'm going to go ahead now and search for say John. And I have again found his number, so this is still correct, but even though this took more minutes in terms of the voiceover and it took more lines of code, it's arguably better designed now because at people bracket 0 is an actual person in everything about them. At people bracket 1 is another person in everything about them. And so forth. This is what we mean by encapsulate, you can think of these curly braces as sort of hugging these data types inside of the data structure together so as to keep them together in the computer's memory as well. All right, well, just to set the stage, uh, literally, as we'll strike the lockers and put something else up, the efficiency of binary search as implemented by Caitlin was predicated on Kelly having in advance sorted the values up front. But of course we've only considered now the running time of searching for information using two algorithms, and there can be many others. In the real world, but those are two of the most canonical. We found that binary search was faster than linear search, but it required that we sort the data. So to your question earlier, maybe we should consider just how expensive it is in terms of time, money, space, humans to sort data, especially a lot of data, and then decide whether or not it's worth using something like binary search or perhaps even something else. So. The next problem we'll solve today ultimately is given generic input and output. The input to our next problem is going to be unsorted data, so like numbers out of order, the output of which should be sorted data. So for instance, if we pass in 72541, 603, I want whatever black box is implementing my sorting algorithm to spit out 01234567. So that's going to be the question we answer. But first I think it's time for some delightful Hello Pandass chocolate biscuits. Uh, let's take a 10 minute break and snacks are now served. All right, we are back and recall that the cliffhanger on which we left was that how do we go about sorting numbers? Well, here are some numbers, 8 of them in fact, from 0 to 7, but currently unsorted. Um, we don't quite have enough monopoly boards for everyone, but we do have some delightful, uh. For Mario Brothers Pez dispensers, if I could get 8 volunteers for this final demo up here, oh, and not a lot of hands. OK, all right, 123456, and let's go farther back 7 and 8. How about alright, come on up hopefully I counted properly. Come on over. Upon arrival at the stage, go ahead and grab your favorite illuminated number and stand in that same order at the front of the stage if you all could. Welcome to the stage. All right, grab your favorite number, stand in that same order. All right. Good. And 123456. I definitely said 1 through 8. Who is the number 8 then? OK, we need an 8. Come on down. All right. Well, technically we need a 4, but come on down. Yeah. All right, grab the 4 and let me start from this end first if you want to give a quick hello and a little something about you. Uh, hi, my name is Cameron. I'm a first year and I want to study mechanical engineering. Welcome. Hi, I'm Charlotte. I'm also first year and I'm in Canada F. Welcome. Hi, I'm Ella. I'm also a first year and I'm in Thayer. Hi, I'm Precious. I'm also a first year. I'm in the. Hi, I'm Michael. I'm just an Eventbrite guest. Hi, I'm Marie. I'm a first year and I'm in Canada. Welcome. Hi, I'm Rick. I'm a first year and I'm in Holworthy. Nice. I'm Jay. I'm a 1st year in Holworthy and I really like free stuff. OK, well, let's see then, uh, if we can't award all these supermarket, but there's Pez dispensers. The first notice, of course, that all 8 of our volunteers are completely out of order, but in an ideal world we would have the smallest number over here.

So go over there, number 0. Wait a minute. 7, take over here. 2, OK, OK, make yourselves look like that. No pez. It's OK. All right, so 72541603. OK, we won't do the introductions again, but now we have a list of numbers completely out of order and wouldn't it be nice if zero were eventually over here? 7 were all the way over there and everything else was sorted from smallest to largest. Well, if you all could go ahead and sort yourselves from smallest to largest, go. Alright, and uh Jayden, what was your algorithm for doing that? Um, I I, I, I know that I have the least number because I don't think anybody has the number less than 0, so I put myself at the last bottom line. OK, and I assume Precious, what was your algorithm? I knew I had the largest numbers, so I just had to be at the end of the. OK, fair. So you guys got the easy ones. Uh, number 4. How about You knew 3 was before me and 5 was after me. Nice. And number 4 didn't actually have to move coincidentally, but as for 5 and 3 and 2 and 1 and 6, they probably had to take into account some additional information. Who's to their left, who's to their right, and it just kind of worked, but it didn't look very algorithmic, if you will. It looked very organic and obviously correct, but I'm not sure that same approach would work well if we had not 8 but 80 or 800 or 8000 pieces of data. So let's see if we can't form. I this a little bit. Let me take the mic and if you guys could reset yourselves to those same original positions from 7 on the left to 3 on the right, let me propose a couple of algorithms, canonical ones if you will, but see if maybe we can't formalize step by step what to do. So the first one I'm going to do, given all of these numbers, is just try to select the smallest number. Why? To Jayden's point earlier, I just want to put the smallest number over here. At least that's a problem I can solve. Well defined. It's a nice bite out of the problem. So 7, OK, it's the smallest so far. 2, that's that's smaller. So I'm going to remember that 2 is the now smallest number I've seen, not 5, not 41 is even smaller. So I'm going to remember 1, not 60. That's pretty good, but I'm going to check the whole list. Maybe there's -1 or something like that, but no, 3. So I'm going to remember that 0 was the smallest element I found. Let's select Jaden and put Jayden over here, but Before Precious or anyone else moves, we don't really have room for you. Like Precious is in the way because if this is an array of 8 values for integers, well, we can't just kind of make room over here because if you think back to last week, we might have some garbage values there or something else is going on. We don't want to change data that doesn't belong to us. So what to do with? Well, maybe precious, maybe you can go over there. So you just take Jade in spots and we'll swap these two values. Accordingly, now though, Jayden is in the right space, which is good because now I can move on to the second problem. What's the next smallest element that's presumably greater than 0? Well, at the moment 2 is the next smallest element, not 5, not 401 is the next smallest element. I'm going to remember that not 6, not 7, not 3. OK, so number 1, if you could go to the right location, but I'm afraid we're going to have to evict number 2 to make room. All right, let's do this again. 0 and 1 are in good shape, so now I think I can ignore them as complete. 5 is the current smallest. No, 4 now is no. 2 now is 67, no, 3, no. OK, so 2 is the next smallest. So let's swap 2 and 5, and now I've solved 3 out of the 8 problems. Let's do this again. 4 is at the moment the smallest, not 5, not 6, not 3 is the now smallest. So let's swap 34 and 3. Which unfortunately is making the 4 problem a little worse, like he belongs there, it would see, but I think we can fix that later. So now half of the list is sorted. 5 is the next smallest, 6 and 74, and then we've got to fix the 4, so 4 goes back there. Now I messed up the 5, but it will come back to that. All right, 67, OK, 5. Let's put you where 6 is, and One more mistake to fix, so 7, OK, 6 and 7 need to swap. And now I've solved 8 problems in the aggregate, so it's complete. Now to be

fair, my approach is clearly way slower than your approach, but you all were working in parallel, whereas I was doing it more methodically step by step, and I dare say my algorithm is probably going to be more translatable to code. And indeed what I just acted out is what the world would call selection sort whereby on each iteration, each pass in front of the humans, I was selecting the smallest element I could find. All right, what, how else could I do this though? Let's do something that's maybe a little more organic like your approach where you were actually comparing who was next to you. Go ahead and reset yourselves one final time to this arrangement. 7 on the left, 3 on the right, and let me propose again to walk through the list again and again, but let me focus more narrowly on the problem right in front of me because I felt like I was taking a lot of steps back and forth, back and forth. Maybe we can chip away at some of that wasted time. Let's compare 7 and 2. They're obviously out of order, so let's just immediately swap you two if we could. All right, now 7 and 5, clearly out of order. Let's swap these two. 7 and 4 out of order. Let's swap these 7 and 1 out of order, let's swap these 7 and 6 out of order. Let's swap these 7 and 0 out of order, swap these two. 73 out of order, swap these two. So a lot of work for Precious there, but I've now indeed solved one of the eight problems. Moreover, I don't need to keep addressing the 7 problem because notice that Precious has essentially bubbled her way up to the end of the list. And indeed that's going to be the operative term here. Another algorithm that computer scientists everywhere know is called bubble sort, whereby the goal is to get the biggest elements to just bubble their way up to the top of or the end of the list one at a time. Now, am I done? No, clearly not. There's still stuff out of order except for Precious. Indeed, I have solved one of these eight problems and now fine, I'll go back and I'm just going to try this same logic again 2 and 5, good, 5 and 4, nope, swap those 5 and 1, nope, swap those 5 and 6 are good. 6 and 0, nope, swap those 6 and 3, nope, swap those. And I already know that. Precious is where she needs to be, so I think I'm done with the second of 8 problems, and I'll do this a little faster now. 2 and 4 and 1 swap, 4 and 5 are good, 5 and 0 swap, 5 and 3 swap, and now we've solved 3 problems. Let me reset 2 and 1 swap, 2 and 4 are good, 4 and 0 swap, 4 and 3 swap, and now I've solved half of the problems. 4 out of 8, we're almost done. 1 and 2 are good, 2 and 0 swap. 2 and 3 are good, OK, and now we're done with 5 out of the 8 problems, 1 and 0 swap. Uh, 1 and 2 are good. Those are all good. And let me just do a final sanity check. Everything now is sorted, so now I'm done solving all eight of those problems. So you all were wonderful. We need the numbers back, but Kelly has some delightful Pez dispensers for you on the way out if you want to head that way, just leave the numbers on the shelves and a round of applause for our 8 volunteers for helping to act this out. Thank you. So let's see if we can't formalize what these volunteers kindly just did with us, starting with the first of those algorithms, thank you, namely selection sort. Let's see if we can't slap some pseudo code on this, thinking of our humans now as more generically in array. So we had the first person at location 0 and we had the last person at location N minus 1. And just for clarity so that you've kind of seen the symbology, this obviously is going to be location N minus 2. Location and minus 3 and so forth until it's sort of you hit the other end that we've already written out. So that's just how we refer to all of our 8 volunteers' locations or in this case 1234567 locations, but in the middle connoting that this can be a much, much larger array. So here's some pseudo code for the first algorithm selection sort for i from 0 to n minus 1. So from the first element to the last element, find the smallest number between. The numbers bracket i and numbers and minus 1. In other words, if you're starting i at 0, look at specifically every

lighted number between 0 and location N minus 1. When you have found that smallest element, swap it with the number at location l , which starts again at 0. That's how we got, I think, Jaden into place at the very beginning. Then l , by nature of how 4 loops work, gets updated from 0 to 1 so that we do the same thing. Find the smallest number between numbers 1, so the second element through the 8th element because this number is unchanged and is the total number of values, so the end point there is not changing. Once we found the second smallest person, we swap them with. Location l aka 1, and that's how we got the number 1 into position and then the number 2 and then the number 3 and number 4. So this then was selection sort in pseudo code form and that allowed us to actually go through this list again and again and again in order to find the next smallest element. So what was happening a little more methodically if it helps just to map that symbology of the bracket notation in the eyes, if this is where we started with location l and we did everything between location and minus 1. Essentially I traversed this whole list from left to right, literally walking in front of our volunteers looking at each element, and the first element I saw was 7. At the moment that was the smallest element I had found, and who knows, in a different list. Maybe 7 would be the smallest element, so I kind of stored it in a variable in my mind, but I checked then 2 and remembered, no, no, no, 2 is clearly less than now I'm going to remember 2. OK, now I'm going to remember one when I find it, and then I'm going to remember 0 when I find it, and then what I did once I found jade in it with the value of 0, light it up. I moved that location to here and then evicted Precious recall and moved Precious over to that location that we had freed up. Why? Why all this sort of back and forth? Well, you have to assume with an array that you're not entitled to the memory over here. You're not entitled to the memory over here if you've already decided that you have 7 lockers or 8 people, you have to commit to the computer in advance. That's why we put the number typically in the square brackets or the compiler infers from the curly brackets how big the array actually is. All right, and suffice it to say when I went through this again and again and again, I did the same thing over and over. Now you might have thought me sort of dumb for having asked the same. Questions again and again like I was surprised to discover the number 1. I was surprised to discover the number 2, even though on my very first pass I literally looked at all eight of those numbers. But you have to think about what memory I'm actually using now. I certainly could have memorized all of the numbers and where they are, but I proposed that just very simply I was using like a single variable in my brain just to keep track of the then smallest element. And once I'm done finding that and solving that problem, I moved on to do it again and again. But that's going to be a trade off, and this is going to be thematic in the coming weeks whereby, well, sure you could use more memory and I could have been smarter about it and maybe that would have improved or hurt the running time of the algorithm. There's often going to be a trade-off between how much memory or how much time you actually use. So we'll discover that over time. So how fast or slow is selection sort? Well, consider when I had 8 humans on stage, I first went through. Uh, all of them, but how many comparisons did I make? Really, I was doing N minus 1 comparisons because if I've got N people, I've got to compare the smallest number I found against everyone else, and you compare N people left to right, N minus 1 times total. So the first pass I was making, I was asking N minus 1 questions. Is this the smallest? Is this the smallest? Is this the smallest N minus 1 times? Once I solved one problem when we got Jayden into Jayden's right place, then I had one fewer problem, then. One fewer fewer problem and so forth. So it was like n minus 1 steps plus N minus 2 steps plus N minus 3 steps plus one final

step once I got to the final of the eight problems. Now if you remember kind of the cheat sheet at the back of your math books, say growing up, you'll note that this series here can be more simply written as N times N minus 1, all divided by 2. And if you've not seen that before, just take on faith that this is identical to this series of numbers up here. So now we can just kind of multiply this out. So that's technically n^2 minus N all divided by 2, which is great if we multiply that out, that's n^2 over 2 minus n over 2. We're getting 2 into the weeds. Let's whip out our big O notation now whereby we can wave our hands at the lower order terms, only care about the biggest, most dominant term which mathematically in this expression, if you plug in a really big value of n , which is going to matter more the N squared, the 2, the N , or the 2. Like the N squared, like the others absolutely contribute to the total value, but if you plug in a really big value, the dominant force is going to be this n squared because that's really going to blow up the total value. So we can say that selection sort when analyzed in this way, ah, it's on the order of n squared steps because I'm doing so many comparisons so many times. So if that's the case, the question then is, Um, what is indeed not just its upper bound but maybe it's lower bound as we'll eventually see. So for selection sort for now, let's stipulate that it's indeed in big O of N^2 , and that's actually the worst of the algorithms we've seen. Like that's way slower than linear search because at least linear search was big O of N . Selection sort is n square, which of course is n times n , which is and will feel much, much slower than that. So what if though we consider the lower bound of selection sort? All right, maybe it's bad in the worst case, but maybe it's really good when the numbers are mostly sorted. Unfortunately this is the same pseudo code for selection sort. We make no allowance for checking the list to make sure it's already sorted, and in fact that's kind of a perverse case to consider for any algorithm. What if the problem's already solved? How's your algorithm going to perform? All of my volunteers, as they kind of almost did accidentally, they started lining up roughly in order. Suppose they literally had been in order from 0 to 7. Well, my stupid algorithm would still have me walking back and forth, back and forth, back and forth. Why? Because the code literally tells me do this this many times and every time I do that find the smallest element. So it's going to be sort of a stupid output because the list is not going to be any change, any any at all changed, but my code is not taking into account in any way. The original order of the numbers. So no matter what, this is to say that if we consider whether the lockers or the humans, the omega notation for this algorithm, even in the best case where the data is already sorted, is crazily also n squared. Now I could certainly change the pseudo code, but selection sort as the world knows it is more of a. Constrative algorithm or sort of a quick and dirty one, its running time is going to be in omega of N^2 , and now we can actually deploy our fader notation because the big O notation is N^2 and the omega notation is n^2 1 and the same. We can also say that selection sort is in theta of N^2 , which is not great because that's annoyingly slow. So maybe the solution here is don't do that. Let's use bubble sort instead. The second algorithm where I just compared everyone side by side again and again. Well, here's some pseudo code for bubble sort, which you can assume applies to the same kind of array from 0 on up to n minus 1. Here's one way to write bubble sort. Repeat the following N times. For i , from 0 to n minus 2, if the number at location i and the number at location $i + 1$ are out of order, swap them. And there's kind of an elegance to this algorithm and that like that's it and you just assume that when you go through the list, this is how from i from 0 to n minus 2, this is how I was effectively comparing elements 0 and 1 and 2 and 3 and 4 and 5 and 6 and 7, but notice I didn't say 8, there were 8 total people. Why do we go from 0

ton minus 2 instead of from 0 ton minus 1? Uh, yeah. Yeah You already checked the glass. Not quite. So it's not that we've already checked the last one. I'm saying with this line of code here, we never even go to N minus 1 technically. If we have N minus 345, and it's going to compare against N minus 1 because that's. Exactly, because we're doing this simple arithmetic here, we're checking current location I plus 1. You can think of these as my left and right hand. Left hand is pointing at 0, right hand's pointing at 1. I don't want to do something stupid and have my left hand point at N minus 1 because then my right hand arithmetically when you add 1 is going to point at N, which does not exist. That's beyond the boundary of the array because the array goes from 0 ton minus 1. So just a live. A bit of a safety check there to make sure we don't walk right off the end of the array, but we do this end times because recall that Precious ended up being where 7 needed to be at the very end of the list, but that didn't mean there weren't 77 more problems still to solve 0 through 6. So I did it again and I did it again and per its name bubble sort, the biggest element bubbled up first, then the next biggest, then the next biggest, then. biggest that is 7, then 6, then 5, then 4, and we got lucky on some of them, but eventually we finished with 0. So how do we analyze this thing? Well, we could also technically do this N minus 1 times as an aside if you're thinking through that I'm wasting some time because we get one for free. Once we get to solving 7ve problems, you get the 8th 1 for free because that person is obviously where they need to go. So when we had these numbers initially, And we were comparing them with bubbleword again, left hand right hand. It's like treat this as I, this is I plus 1, and we just kept swapping pair wise numbers if in fact they were out of order. So all this is saying is what our humans were doing for us organically. So how do we actually analyze the running time of this? Last time I just kind of spitballed that it was minus 1 steps plus N minus 2 steps. Well, you can actually look at pseudocode sometimes and if it's neatly written, you can actually Infer from the pseudo code how many steps each line is going to take. For instance, how many steps does this first line take? I mean, like literally n minus 1. The answer is right there because it's saying to the computer or to me acting it out, repeat the following N minus 1 times. All right, so that's helpful. How many lines, how many steps does this inner loop induce? Well, you're going from 1 ton minus 2, so that's actually N minus 1, total steps, not n. And then this question here, if numbers bracket 1 and numbers 1 are out of order, it's a single question. It's like our Boolean expression, we'll call it one. I mean, maybe you need to do a bit of more work than that, but it's a constant number of steps. Doesn't matter how big the list is, comparing two numbers is always going to take the same amount of time. And then swapping them, oh, I don't know, it's going to take like 1 or 2 or 3 steps, but constant doesn't matter which the numbers are, takes the same amount of work. So let's stipulate, let me rewind, stipulate that the real things that matter are the loops, these constant number of steps, who really cares? But the loops are what are going to add up as N gets large. So this really then is. If this is the outer loop and this is the inner loop, think about our two dimensional Mario square from week one. We did something on the outside and then something on the inside to get our rows and columns. This is equivalent to N minus 1 times N minus 1. If we do our little foil method N^2 minus $2N$ plus 1 , who cares? This is ultimately going to be on the order of big O of. N squared only because again if you ask yourself when I plug in a really big value for N, which of these is really going to contribute most to the answer, it's obviously going to be n^2 again and we can ignore the lower order terms. So this doesn't seem to have made any progress. Like selection sort was on the order of big events was on the order of N^2 d. Bubble

sort, based on this analysis is. Also on the order of N^2 , maybe we're getting lucky in the lower bound. So on the upper bound for bubble sort, it's indeed N^2 , as was selection sort. But with this pseudo code for bubble sort, unfortunately we, or rather unfortunately we were not doing anything clever to catch that perverse case where maybe the list was already sorted. After all, consider if the list was sorted from 0 to. I was still asking all the same darn questions. Even if I did no work, I was going to repeat that n minus 1 times back and forth, making no swaps, but making all of those comparisons. But here's an enhancement to Bobble So that we can add that selection So didn't really have room for. I can say after one pass of this inner loop walking from left to right, if I made no swaps, quit. So put another way, if I traversed the list from left to right, I make no swaps. I might as well just terminate the algorithm then. Because there's no more work clearly to be done. All right, so based on that modification, the lower bound of bubble sorts running time would be said to be an omega then of. And because I'm minimally going to need to make one pass through the list. You can't possibly claim that the list is sorted unless you actually check it once. And if there's n elements, you're going to have to look at all of them to make sure that it's in order. But after that, if you've done no work and made no swaps, no reason to traverse the list again and again and again. So a bubble sort can be said to be an omega of N because indeed we can just terminate. After that single pass if we've done no work, we can't say anything about theta because they're not one and the same big O and Omega, but that does seem to have given us some savings. Unfortunately, it really only saves us time when the list is already or mostly sorted, but in the average case and in the worst case, odds are they're both going to perform just as bad on the order of n^2 . In fact, let's take a look at a visualization that will make this a little clearer than our own humans and voices might have. Explained here is a bunch of vertical purple bars made by a friend of ours in the real world, and this is an animation that has a bunch of buttons that lets us execute certain algorithms. A small bar represents a small number. A big bar represents a big number, and the goal is to get them from small numbers or small bars to big numbers or big bars left to right. So I'm going to go ahead and click on selection sort initially, and what you'll see from left to right is in pink the current smallest element. That's been discovered, but also in pink, the equivalent of my walking across the stage left to right again and again and again trying to find the next smallest element and you'll see clearly just like when we put Jayden at the far left, the smallest element ended up over here, but it might take some time for Precious, for instance, or number 7 to end up all the way over on the right because with each pass we're really just fixing one problem at a time and there's N problems total, which is giving us on the order of. N squared steps and now the list is getting shorter, so we're at least doing some work that you don't have to keep touching the elements you already sorted with just like I was. So now selection sort is complete. Let's visualize instead bubble sort. So let me re-randomize the array just so we're starting with a random order. Now let's click on bubble sort and you'll see the pink bars work a little differently. It connotes which two numbers are being compared at that moment in time, just like my left hand and right hand going left to right, and you'll see that even though it's not quite as pretty. A selection sort where I was getting at least the smallest elements all the way to the left. Here we're just fixing pair wise problems, but the biggest elements like Precious's number 7 are indeed bubbling their way up to the top one after the other. But as you can see, and this is where n^2 is sort of visual visualizable, we're touching these elements or looking at them so many times again and again. We are making so many darn comparisons. This is taking

frustratingly long, and this is only what, a few dozen bars or numbers. You can imagine how long this might take with hundreds, thousands or millions of values. I dare say we're gonna have to do better than bubble sort and Selection sort because we're not stun even yet, just trying to give the satisfaction of getting to the end and now we are, but neither of those algorithms seems incredibly performing because it's still taking us quite a bit of time to actually get to that there solution. So how can we actually do better than that? Well, we can try taking a fundamentally different approach, and this is one technique that you might have encountered in math or even in the real world, even if you haven't sort of applied this name to it. Recursion is a technique in mathematics and in programming that allows you to take sort of a fundamentally different approach to a problem and in short, a recursive function is one that's defined in terms of itself. So if you had like F of X equals F of something on the right hand side of a mathematical expression, that would be recursive in that the function is dependent on itself. More practically in the world of programming, a recursive function is a function that calls it. Self. So if you are writing some function in C and in that function you call yourself. You actually have a line of code that says call that same function by the same name, that function is recursive. Now this might feel a little weird because if a function is calling itself, it feels like this is the easiest way to get into an infinite loop because why would it ever stop if the function is calling itself, calling itself, calling itself, calling itself? We're going to have to actually address that kind of problem. But in the real world. We've actually, or rather in this class already, we've actually seen implicitly an example of this, including today as well as in week 0. So here's that algorithm for searching the doors of the lockers and recall that after we did this check at the very top if there are any doors left, returned false, if not, we did these conditions. We said if the numbers behind the middle door returned true because we found it, but things got interesting here. I said if else if the number is less than the middle door, then search the left half. Else if the number is greater than the middle door, then search the right half. Well, at that point in time, you should be asking me or yourself, well, how do I search the left half? How do I search the right half? Well, here you go. Like on the screen right now is a search algorithm, and even though it says down here search the left half or search the right half, which is like, well, how do I do? That we'll just use the same algorithm again and this is how in terms of my voiceover you end up searching the left half of the left half or the right half of the left half or any such combination. This line here search left half, this line here search right half is representative of a recursive call. This is an algorithm or a function that calls itself, but why does it not induce an infinite loop? Like why is it important that this line and this line are written exactly as they are, so as to avoid this thing just forever searching aimlessly, yeah. We do have this condition at which it stops, but more importantly, what is happening before I make these recursive calls? Exactly, I'm recursing, that is calling myself, but I'm handing myself a smaller problem, a smaller problem, a smaller problem. It would be bad if I just handed myself the exact same number of doors and just kept saying search these, search these, search these because you would never make any progress. But just like our volunteers earlier, so long as we did divide and conquer and we searched smaller and smaller numbers of doors, eventually indeed we're going to bottom out and either find the number we're looking for or we're not. So. we're going to call these kinds of conditions that sort of just ask a very obvious question and want an immediate answer base cases. Base cases are generally conditionals that ask a question to which the answer is going to be yes or no right then and there. A recursive case, by contrast, these two down here is when you actually need to do a

bit more work to get to your final answer. You call yourself, but with a smaller version of the problem. So we could have in fact in week 0 have written this. Sort of similarly, if you go back to in your mind to week 0, we had more of a procedural approach, so to speak. When we were searching the phone book, I proposed that this induced what we called loops on line 8 and line 11, which is literally said go back to line 3, and that was more of a mechanical way of sort of inducing a loop structure. But if I really wanted to be elegant, I could have said, well, you know what, 7 and 8 together really just means search the left half and 10 and 11 together really mean just search the right half. So let's condense these. Pairs of lines into shorter instructions. Search the left half of the book, search the right half of the book. I can then delete two blank lines, and now I have a recursive algorithm for searching a phone book. It's a little less obvious because you have to ask yourself when you get to line 7 or 9, Wait a minute, how do I search the left half or the right half? And that's when you need to realize you start the same algorithm again but with a problem that's half as large. In week 0 we do the procedural approach where we literally tell you what line of code to go to. But today we're offering a different formulation, a recursive approach where it's more implicit what you should do. And we'll see now a couple of examples from the real world, so to speak. So here's a screenshot from Super Mario Brothers 1 on the original Nintendo Entertainment System. Let me go ahead and get rid of some of the distraction, like the ground and the mountains there, and here we have a sort of half pyramid, not unlike that you implemented in Problems set 1, but this is an interesting real world physical structure in that you can define it recursively, like what is a pyramid of height. 4, if you will. Well, just to be a little, a little difficult, a pyramid of height 4 is really just a pyramid of height 3 plus one more row. OK, well, what is a pyramid of height 3? Well, a pyramid of height 3 is really just a pyramid of height 2 plus one more row. Well, what's a pyramid of height 2? Well, a pyramid of height 2 is really just a pyramid of height 1 plus one more row. Well, what's a pyramid of height 1? A single brick on the screen. And I sort of changed my tone with that last remark to convey that this could then be our base case whereby I just tell you what the thing is without sort of kicking the can and inviting you to think through what a smaller structure is plus one more row, whereas every other definition I gave you then of a pyramid of some height was defined in terms of that same structure, albeit a smaller version thereof. So we can actually see this in the real world. Let me go ahead and pull up one thing here. I'm going to go to. Give me one sec before I flip over. Here I am on Google.com, if you'd like a little computer science humor here. Uh, if you ever Google search for recursion and hit enter, you'll see uh a joke that computer scientists at Google find funny. Ha ha, 12, lasts. Does anyone see the joke? I did not make a typo, but Google's asking me, did I mean recursion? And if I click on that, I just get the same ha ha page. OK, all right, that didn't go over well. Anyhow, so there are these Easter eggs in the wild everywhere because computer scientists are the ones that implement these things. But let's go ahead and actually implement, for instance, a version of this in code. Let me go back over here in a moment to VS code, and in VS Code, let me propose that in my terminal window. Let me create one of two final programs. This one's. Going to be called iteration.c just to make clear that this is the iterative that is loop-based version of a program whose purpose in life is to print out a simple Mario pyramid. I'm going to go ahead and include CS50. H at the top as well as standardio.h. I'm not going to need string.h. I don't need any command line arguments today, so this is going to start off with main void. And now I'm going to go ahead and ask a question like, uh, give me a variable called height of type integer and ask the human for the height of this Mario like pyramid.

And then let's assume for the moment that I've already implemented a function called Draw whose purpose in life is to draw a pyramid of that height semicolon. So I've abstracted away for the moment the notion of drawing that pyramid. Now let's actually implement Dra, whose purpose in life again is to print out a pyramid akin to the one we saw a moment ago like this here on the screen. Well, in order to print out a pyramid of a given height, I think I need to say void. Uh, drawn for instance because I'm not going to bother returning a value. I just want this thing to print something on the screen, so void is the return type, but I do want to take as input an integer like the height of the thing I want to print. I can call this argument or parameter anything I want. I'll call it N for number. So how can I print out a pyramid that again looks like this? Well, I'll do this quicker than you might have in problem set 1, but it seems obvious that like on the first row I want 1 brick. On the second row I want 2. On the 3rd I want 3. On the 4th, I want 4. So it's actually a little easier than problem set 1 in that it's sloped in a different direction. So let me go ahead and do exactly this in code. Let me say $4 \text{ N} \leq n$, the height, I plus plus. So this is going to be really for each row of the pyramid. Let me go ahead now and in an inner loop for $J = 0$, let's do $J < I + 1$ for reasons we'll see in a moment and then $J++$ and then inside of this loop, let's just print out a single hash, no new line, but at the end of the row, let's print out a single new line to move the cursor to the next line. Now why am I doing this? Well, this represents for each column of pyramid, and if you think about it on the first row, which is row 0, I actually want to print not zero bricks but one brick. That's why I want to go ahead here and go from 0 to $I + 1$, because if I is 0, $0 + 1$ is 1, so my inner loop is going to go from 0 to 1, which is going to give me 1 brick. It's a little annoying to think about the math, but this just makes sure that I'm actually getting bricks in the order I want them. And then it's going to give me 2 bricks and then 3 and then 4, and between each of those rows it's going to print a new line. So let's go ahead and do make iteration to compile this code. Ah, I messed up. Why do I have a mistake online? 8 of this code. Let me hide my terminal and scroll back up. It seems clang. My compiler does not like my draw function. Yeah. Yeah, I forgot the prototype. So this is the one and only time where it seems reasonable to copy paste. Let's grab the prototype of that function up here and go ahead and teach the compiler from the get-go what this function is going to look like, even though I'm not defining it now until line 13 onward. All right, let's go ahead and make iteration again. Ah, iteration enter. Let's do a height of say 4 and voila. Now I've got that there pyramid. So I did it a little quickly and it's certainly to be expected if it took you hours on problems set one to get the other type of pyramid printed, but the point for today is really to demonstrate how we can print a pyramid like this using indeed what I'd call iteration. Iteration just means using loops to solve some problem, but we can. Alternatively use recursion by reimplementing our draw function in a way that's defined in terms of itself. So let me go into my code here and I'm actually going to leave the prototype the same. I'm going to leave main the same, but what I'm going to go ahead and do is delete all of this iterative code that's doing things very procedurally, step by step by step with loops, and I'm instead going to do something like this. Well, if I want to print a pyramid of height N. What did I say earlier? Well, a pyramid of height N is really just a pyramid of height N minus 1 plus 1 more row. So how do I implement in code that idea? Well, let me go back in code here and say, well, if a pyramid of height N first requires drawing a pyramid of height N minus 1, I think I can just write this, which is kind of crazy to look at, but because you're calling yourself in yourself, but let's see where this takes us. Once I have drawn a pyramid of height and minus 1. That is a height 3 for

instance. What remains for me to do is to myself print one more row. And so to print one more row, I think I can do that really easily with fewer loops. I can do $4 - i$, i equals 0 less than N plus plus, and then very simply in this loop I can. Print out a single hash one at a time. At the end of this loop, I can print out a new line, but no more nesting of loops. What I've done is print one more row, and here I've done print a pyramid of height and minus 1. I'm not quite done yet, but I think this is consistent with my verbal definition that a pyramid of height 3 is a pyramid of height 4 is a pyramid of height 3, which I can implement per line 16. Just draw me a pyramid of height and minus 1, and then I myself will take the trouble to print the 4th and final row. But something's missing in this code. Let me go ahead and try running it. Let's see what happens. Make, oh darn it. I meant to call this something else, so I'm going to do this. I'm going to close this version here. I'm going to rename iteration.c to recursion.c to make clear that this version is completely different. Let me now go ahead and make the recursion version and huh, Klang is noticing that I have screwed up on line 14, it says error. All paths through this function will call itself, and Klang doesn't even want to let me compile this code because that would mean literally just forever. Loop effectively by calling yourself. So what am I missing in my code here if I open up what we're now calling recursion.c. In my editor What's missing here over here? Yeah, I'm missing a base case, and I can express this in a few different ways, but I would propose that before I do any drawing of anything at all, let's just ask ourselves if there is anything to draw. So how about if N equals equals 0? Well then don't do anything, just return. You don't return a value when your return value is void, it means you don't return anything, so you just return period or return semicolon. Or just to be super safe, I could actually do something like this, which is arguably better practice just in case I get into this perverse scenario where someone hands me a negative number, I want to be able to handle that and not print anything either. So just to be safe, I might say less than or equal to 0. I'm not doing one because if I did do one, then I would want to at least myself print out one brick, which is fine, but I have to like change all of my code. Little bit, so I think it's safer if my base case is just if n is less than or equal to 0, you're done. Don't do anything. And this then ensures that even though thereafter, I keep calling draw again and again and again and the problem's getting smaller and smaller from 4 to 3 to 2 to 1, as soon as I hit 0, the function will finally return. So let's go ahead and open up my terminal, rerun make recursion to make this version did compile this time recursion enter. Let's type in 4 cross my fingers, and this too prints the exact same thing. And even though it doesn't look like fewer lines of code, I would offer that there's an elegance to what I've just done, whereas with the iterative version with all the loops, it was very clunky, like step by step, just print this and print that and have a nested loop inside of another. But with this, especially if we distill it into its essence. By getting rid of my comments like this and frankly I can get rid of the unnecessary curly braces only because for single lines in conditionals you don't need them. Like this is arguably like a very beautiful implementation of drawing Mario's pyramid even though it's calling itself and arguably because it is calling itself. Questions then on this idea of recursion or this implementation of Mario, yeah. Call Good question. Are there any scope issues involved? Short answer No. However, the current value of i , for instance, will not be visible to the next time the function is called. It will have its own copy of i if that's what you mean. And we'll next week talk in more detail about what's going on here. And in fact, I probably can't break this in class very easily, but it turns out if I use a very large version for heights, let's just hit a lot of zeros and see what happens. That was too many. Let's see what happens. That's also too many. Let's see

what happens there. That's the first time at least I and class have encountered this error. You might have encountered this weird bug in office hours or in your problem set, and that's fine if you did. We'll talk about what this means next week too, but this is bad. Like this clearly hints at a problem in my code. However, the iterative version of this program would not have that same error. So this relates to something involving memory because it turns out as a little teaser for next week. Each time I call Draw, I'm using a little more memory, a little more memory, a little more memory, a little more memory, and my computer only has so much memory. This program in its current form is using too much memory. There are workarounds to this, but that is a trade off to the elegance we're gaining in this solution. So what's the point of all this and how do we get sidetracked by Mario? There's another sorting algorithm, the 3rd and final one that we'll consider today that actually is recursion to solve the problem not only elegantly arguably, but also way faster somehow than bubble sort and selection sort. And in essence it does so by making far fewer comparisons and wasting a lot less work. It doesn't keep comparing the same numbers again and again. Here in its essence is the pseudo code for merge sort. Sort the left half of the numbers. So the right half of the numbers, then merge the sorted halves, and this is kind of a weird implementation of an algorithm because I'm not really telling you anything. It seems like you're asking me how do I sort numbers, and I say, we'll sort the left half, sort the right half. It's like someone being difficult and yet implicit in this third line is apparently some magic, this notion of merging halves that are somehow already. is actually going to yield a successful result. As an aside, we're actually going to need one base case here too. So if you're only given one number, you might as well quit right away because there's nothing to do. So we'll toss that in there as well. And base cases are often for 0 or 1 or some small num sized problem. In this case it's a little easier to express it as one because if you have one element, it's indeed already sorted. So what does it mean to merge two sorted halves? Well, let's actually consider this. I'm going to reuse some of these same numbers here. I'm going to put my 1, my 3, my 4, and my 6 on the left, and these together represent a list that is indeed sorted of size 4. And then I'm going to put 4 other numbers on the right there that are similarly sorted. As well, and by merging these two lists, I mean start at the left end of this list, start at the left end of this list, and just decide one step at a time which number is the next smallest, and then I'm going to put it on the top shelf to make clear what is sorted. So if my left hand's pointing at this list, my right hand's pointing at there, which hand is obviously pointing to the smaller element, left or right. Like the right, so I'm going to grab this and I'm going to use a little more space up top here and put the 0 in place and then I'm going to point to the next element there. So my left hand has not moved yet. It's still pointing at the 1. My right hand is pointing at the 2, which number comes next clearly? Left, so I'm going to grab the 1 and put it up there and update where my left hand is pointing. So now I'm pointing at the 3 here and the 2 there. What comes next? Obviously the 2. What comes next? Obviously the 3. What comes next, obviously the 4. What comes next, obviously the 5, but notice my hands are not going back and forth, back and forth, back and forth like any of the algorithms thus far. I'm just taking baby steps, moving them only to the right, effectively pointing at for a final time each number once and only once. What comes next, 6, and now my left hand is done. What comes last, the number 7. What I just did is what I mean by merge the sorted halves. If you can somehow get into a scenario where you've got a small list sorted and another small list sorted, it's super easy now to merge them together using that left right approach, which I'll claim only takes end steps. Why? Because every time I asked you a

question, I was taking one. Out of the problem, there's 8 bytes total. I asked you 8 questions or I would have if I verbalized them all. So it's N steps total to merge lists of that size. So what then is merge sort? Mergesort is really all three of these steps together, only one of which we've acted out, 2 of which are sort of cyclical in nature. They're recursive by design. So what does this mean? Well, let's start with this list of eight numbers, which is clearly out of order 63, 4152, 70 and let's apply mergesort to this set of numbers and I'll do it digitally here because it'll take forever to keep moving the numbers up and down physically. So let's move it to the top just to give ourselves a little bit more room and let me propose that we apply mergesort. What was the very first step in mergesort, at least that we highlighted, the juicy steps. What's the first step in mergesort? Sort the left half, yeah. And then the second step was going to be sort the right half and then the third step was going to be merge the sorted halves. So let's see what this means by actually acting it out on these numbers. So here's my 8 numbers. Let's go ahead and sort the left half. Well, the left half is obviously going to be the 4 numbers on the left, and I'm just going to pull them out just to draw our attention to them over here. Now I have a list of size 4, and the goal is to sort the left half. How do I sort a list of size 4? Uh, be what was yes, but just be more pedantic. Like how do I sort any list using merge sort? Sort the left half. So let's do just that. So of the list of size 4, how do I sort this? Well, I'm going to sort the left half. How do I sort a list of size 2? Sort the left half, right, well, I'm just gonna write the 6 here. How do I sort a list of size 1? I just don't. I'm done. That was the so-called base case where I just said return, like I'm done sorting the list. OK, so here I here's the story recap. So the left half, sort the left half, sort the left half, and I just finished sorting this. So what comes next? Sort the right half, which is this, and now I've sorted the left half of the left half of the left half, which is a big mouthful, but what do I do as a third and final step when sorting this list of size 2. Merge them. This part we know how to do. I point left and right, and I now take the smallest element first, which is the 3. Then I take the 6 and now this list of size 2 is sorted. So if you remind in your mind's eye what step are we on, well, we have now sorted the left half of the left half. So what comes after the left half is sorted? We sort the right half. So we're sort of rewinding in time, but that's OK. I'm keeping track of the steps in my mind. I want to now sort this list of size 2. How do you sort a list of size 2? Well, you divide it into a list of size 1. How do you sort this? You're done. You then take the other right half and you sort it. Done. Now you merge the two sorted halves. So I point at the 4 and the 1, obviously the one comes first, then the 4. Now I have sorted the right half of the uh the right half of the left half of the original numbers. What's the next step now that I have the left and right halves of this list of 4 sorted? Merge those, so same idea but fewer elements. I'm pointing at the 3 and the 1. Obviously the 1 comes now I'm pointing at the 3 and the 4. Obviously the 3 comes next, pointing at the 6 and the 4, the 4 comes next, and now the 6 comes last. Now I have sorted the left half and it's intentional that 1346 is the original arrangement of the lighted numbers I had on the shelves a moment ago. All right, it's a long story it seems, but what comes after you sorting the left half of the original list? You sort the right half. So let's put some, uh, put those numbers over here. How do I sort a list of size 4? Well, you sort the left half. How do you sort this thing of size 2? You sort the left half, you sort the right half, and now you merge those together. How do I now sort the right half of the right half? Well, I sort the left half, I sort the right half, and then I merge those together. Now, I have sorted the left half and the right half of the right half of the original elements. What's next? The merging, 025. And 7. Now we're exactly where we were originally with the lighted numbers. I've got 1346, the

left half sorted, 02 57. The right half sorted. What's the third and final step? Merge those two halves, of course, 0123456. And 7 And hopefully even though there's a lot of words that come out of my mouth as acting this out, there wasn't a lot of back and forth. Like I definitely wasn't like walking back and forth physically and I also wasn't comparing the same numbers again and again. I was doing sort of different work at different conceptual levels, but that was like only what, like 3 levels total. It wasn't N levels on the board visually. So where does this get us with merge sort? Well, with mergesort, it would seem that we have an algorithm that I claim is doing a lot less work. The catch. Is that merge sort requires twice as much space just as we saw when I needed two shelves in order to merge those two lists. So how much less work is actually going to be possible? Well, let's consider sort of the analysis of the original list and how we might describe its running time in terms of this big O notation. Hopefully it's not going to be as bad as n^2 ultimately. So here are some like bread crumbs that if I hadn't kept updating the screen and deleting numbers once we move them around, here's sort of like traces of every. Bit of work that we did, we started up here, we did the left half, the left half of the left half, the right half of the right half, and then everything else in between, and you'll see that essentially I took a list of size 8 and I did 3 different passes through it at this conceptual level, at this conceptual level, and at this one. And each time I did that I had to merge elements together and if you kind of think about it here, I pointed at 4 elements here and 4 elements here and in total I pointed at 8 elements. So there was end steps here for merging and if you. Trust me, I'll claim that on this level conceptually there were also 8 steps. I wasn't merging lists of size 4, but I was merging 2 lists of size 2 over here and 2 more lists of size 2 over there. So if you add those up, those are in total steps or or merges if you will. And then down here this was kind of silly. I was, but I was merging ultimately 8 single lists altogether into the higher level of conceptually. So from a list of size 8 we sort of had 3 levels of work. And on each level we did n steps, the merging. So whereas 3? Well, it turns out if you have 8 elements up here, the relationship between 8 and 3 is actually something formulaic, and we can describe it as $\log$ base 2 of n . Why? Because if n is 8, if you don't mind doing some logarithms here, $\log_2$ of 8 is the same thing as $\log_2$ of 2^3 power. The $\log_2$ and the 2 cancel itself out, which gives you exactly the number 3. That I sort of visualized with those traces on the screen, which is to say, irrespective of the specific value of N , the big O running time of merge sort is apparently not n squared, but it's $\log n$ times n or more conventionally $n \times \log n$. Because you're doing things, $\log n$ times technically based 2, but we don't care about that generally for big O notation. And indeed in big O notation we would say that merge sort is on the order of $N \log N$. That's its big O running time sort of at the upper bound. What about the lower order bound? Well, there's no clever optimization in our current implementation as there was for bubble sort, and so it turns out the lower bound would be in ω of $N \log N$ and in θ , therefore, O and $\log$ in as well because Big O and Ω are in fact in this case one and the same. And if we actually go back to our visualization from earlier, give me just a moment to pull that up here in our earlier implementation or an earlier demonstration of these algorithms, we had a side by side comparison of all of the comparisons, but here if I go ahead and randomize it and click Merge sort, you'll see a very different and clearly faster algorithm, even though the computer speed has not changed. But it's touching these elements so many fewer times it's wasting a lot less time because of this cleverness where it's instead dividing and conquering the problem into smaller and smaller and smaller pieces. And to give this a final flourish, since that was, yes, faster, but not necessarily obviously

faster than other things that we've done, how might we actually compare these things side by side by side? Well, in our final moments together, let's go ahead and. Automatically and for no real reason, just dim the lights so that I'll hit play on a visualization that at the top is going to show you selection sort with a bunch of random data. On the bottom it's going to show you bubble sort with a bunch of random data and in the middle is going to show you merge sort and the takeaway ultimately for today is the appreciable feel of difference between big O of N^2 and now the big O of $N \log N$. All right, the music just makes sorting more fun, but that's it for today. We will see you next time.

All right, this is CS 50, and this is week 5 already, wherein we will focus today on data structures, which is a topic we touched on a little bit in in simple form, but today we'll dive all the more deeply and for better or for worse, this is our last week on C. Uh, next week, of course, we transition to Python, which is a so-called higher level programming language, which is really, frankly, just going to make our lives a lot easier. We're going to be able to solve a lot of the same problems, but so much more quickly as Humans, but not necessarily as we'll see as fast when we run the code as the computer might have if we were still using a lower level language like C. So indeed thematic over this week and next is going to be the theme that we've seen before of trade-offs. But before we get there, why don't we focus on a couple of data structures that you might encounter in the real world, namely stacks and queues. Let's learn some facts about both of these if we could dim the lights dramatically. Once upon a time there was a guy named Jack. When it came to making friends, Jack did not have the knack. So Jack went to talk to the most popular guy he knew. He went up to Lou and asked, What do I do? Lou saw that his friend was really distressed. Well, Lou began, Just look how you're dressed. Don't you have any clothes with a different look? Yes, said Jack, I sure do. Come to my house and I'll show them to you. So they went off the jacks, and Jack showed Lou the box where he kept all his shirts and his pants and his socks. Lou said, I see you have all your clothes in a pile. Why don't you wear some others once in a while? Jack said, Well, when I remove clothes and socks, I washed them and put them away in the box. Then comes the next morning and up I hop. I go to the box and get my clothes off the top. Lou quickly realized the problem with Jack. He kept clothes, CDs, and books in a stack. When he reached for something to read or to wear, he chose the top book or underwear. Then when he was done, he would put it right back. Back it would go on top of the stack. I know the solution, said a triumphant Lou. You need to learn to start using a queue. Lou took Jack. clothes and hung them in a closet, and when he had emptied the box, he just tossed it. Then he said, Now, Jack, at the end of the day, put your clothes on the left when you put them away. Then tomorrow morning when you see the sun shine, get your clothes from the right, from the end of the line. Don't you see, said Lou, it will be so nice. You'll wear everything once before you wear something twice. And with everything in queues in his closet and shelf, Jack started to feel quite sure of himself, all thanks to Lou and his wonderful Q. All right, our thanks to Professor Shannon Duvall at Elon University who kindly put together that animation, and it's meant to paint a picture of a couple of things that we've all encountered in the real world, but more technically what we just saw were what are known as abstract data types, whereby there are data structures in some sense, but it's really about the design thereof, what characteristics or features or functionality these structures offer irrespective of how they are implemented in terms of lower level implementation details, which is to say you can implement, as we'll see,

cues and stacks in any number. Ways which are going to have real world implications for how you can actually use them and what kinds of problems you can solve with them. So let's consider for instance queues in the first place. So a queue is something you sort of experience all the time. Any time you go to a store, uh, go to some event for which you have to line up in a so-called queue, you'd ideally like there to be some fairness property about that queue, such that if you got in line first, you get into the store first. You get to check out first or some other such goal. Meanwhile, the person who got there last actually is at the end of the line. is at the end of the line and therefore gets served or enters in at the end. So queues have what computer scientists would say is a FIFO property first in, first out. That is, if you're the first person in line, you're the first person to get out of line. And for many problems that is a good solution, certainly if you're concerned with fairness. But more technically, AQ has what we'll call two operations NQ, which is a fancy way of saying getting in line, and DQ, a fancy way of saying getting out of the line from the front of it. But those two operations, if Think about it in code. Could it be implemented with different actual details? And by that I mean this. Here is one way that we could go about implementing in C code, a queue for a bunch of people or persons who want to line up for something. So for instance, we'll decree that this queue can hold no more than 50 people, like that's the physical capacity, and then we define a structure, which we've done a couple of times in the past whereby this structure has not only an array of persons that we'll call people and that will be. As big as is the capacity, so this is an array of size 50 for 50 such persons. And then we're going to propose that we also keep track in this implementation of a queue of the current size of the queue. So we're going to make a distinction between the capacity, like how many total people can be there and the size, like actually how many people are in line at that moment in time so that you know which of the spots in the array are effectively empty, and we're going to call that whole structure a queue. Now the catch with this particular implementation and code of a queue. Is what? There is inherent in it a limitation, something you just kind of have to deal with and I see you nodding. What what's your instinct for this? For example, like if you have like. OK, well, I think you hit the nail on the head in that it's only for 50 students or 50 people, which means if a 51st person wants to get into line, you literally have no means of remembering them in this data structure. So how do you solve that? Well, we could just recompile our code after changing the 50 to like 51 or maybe 500 or 5000, but there there's this trade-off because you could still be undershooting the total number of people trying to get into maybe a big concert in the case of an extreme, but at the same time, If you over allocate memory using 5000 locations in memory, what if only a few people show up? Now you're just wasting memory. And certainly at the end of the day you only have a finite amount of memory in the computer, so you kind of have to decide a priori like before compiling your code, how big is this structure going to be, how much space you're going to waste, and in the end it's all sort of stupid. It would be ideal if instead we could just grow the queue as needed and shrink it, essentially asking the operating system as we started doing last week for more memory. Giving it back if we don't actually need that memory, which is to say can't really do an array in this static sense, and by static I mean we're literally deciding in advance at compilation time how big this thing is going to be. As an aside, this is also a bit annoying for implementing a queue because you have to somehow keep track of who is at the head of the queue, the front of the queue, because as you start plucking people off, you need to remember who's the next person effectively, but there are ways in code that we could solve this. So let's consider an alternative to

a cue which gives us very different properties, namely a stack, and we saw that in the animation whereby Jack used a stack to put his clothes into a box so that every time he got dressed, he sort of took the sweater from the top, from the top, from the top, and might never wear anything other than black as a result. If he does a wash before he actually reaches the blue in the red sweater there. So a stack, as we've just seen, has a LIFO property to it last in, first out. So if I do a load of law. and I plop some more sweaters on this stack. Well, I'm presumably going to use the last sweater that went in first as opposed to trying to create a mess and like you know pull the bottommost sweater out, which is just going to be a little more effort than it would be otherwise from just taking it from the top. So sometimes last in first out doesn't give you maybe this fairness property you might want for other problems, but it does give you an efficiency, a convenience, certainly. So maybe that might be compelling. And stacks are actually everywhere too. If you've checked your Gmail. Recently, odds are you've opened up gmail.com or Outlook.com, and you've looked at your inbox, and where does the new mail by default end up at the top, at the top, at the top. And I dare say all of us are guilty of sort of neglecting emails that fall below the breaker onto the next page and sort of focusing only on the last in and therefore replying to it first out, which isn't great maybe for the senders of those emails, but it's just how those user interfaces are implemented quite often unless you override those default settings. So how might we implement a stack? Well, we need to implement more technically two fundamental operations. The analogs of NQ and DQ in the world of stacks are called push, which means push something onto the top of the stack, and pop, which means remove something from the top of the stack also. And the team in the cafeterias and dining halls on campus do this all day long. Any of the cafeterias or dining halls that have stacks of trays, of course, you put the first tray at the bottom and then the next tray and the next tray in the next tray, and which tray do all of you. Pick up well, presumably the one on the very top, because it's even harder to grab the bottommost tray than it would be for something like a sweater. As a result, there's maybe undesirable properties like maybe no one ever gets to the nasty tray at the very bottom of the stack because we're constantly replenishing the top ones. But thanks to gravity like that just happens to be the most appropriate data structure in the real world for distributing things like trays in a cafeteria. So how might we implement that idea in code? Well, funny enough, we can pretty much use the exact. Same structure we could just rename Q to stack because at the end of the day we need to keep track of some number of people and maybe people's a weird sort of analog here, but we kept everything else the same so why not that? But the size is also something we still need to remember and it turns out it's a little easier to implement a stack in this way because you can always remove it from the end of the array, end of the array, and the first thing that went into the stack, the first in, can always stay at location 0, for instance. But ultimately we could implement it in this way, but we have the same darn limitation. You can still only put 50 sweaters, 50 trays, 50 people into that stack data structure. So this is just one implementation approach. But that doesn't mean that's necessarily a limitation of stacks and cues. They're abstract in the sense that we could do better. We could maybe start to manage our own memory, move away from statically defining the total size of this array, and just start allocating and de-allocating. That is growing and shrinking. The data structure instead, which is to say we can make these abstract data types much less abstract with actual implementations. Let's consider a data structure that we saw an abstract data type that we saw early on that we didn't necessarily give this name. A dictionary is yet another

abstract data type that's sort of everywhere in the world, literally in the world of dictionaries containing words and their definitions, and you can think of a dictionary really in the abstract if you were to draw this on the chalkboard as really just a two column table. Whereby on the left is the word and on the right is the definition. If it's a physical book, it's essentially the same thing with lots of columns of words on the left, often bold faced, and then the definitions right next to them. You can also see this in the context of like a phone book, which is where we began the course in week zero, where it's essentially a dictionary of names and numbers instead of words and definitions, and a computer scientist would generalize the notion of a dictionary further and just call the thing on the left a key and the thing on the right a value. You and these things are omnipresent in computing and you're going to start to see them all the more today and next week and beyond in that if you just want to associate some piece of data with another piece of data, a so-called key value pair, a dictionary is going to be your go to data type. But even these two we can implement in different ways for reasons that we've already seen, like maybe there's only a finite size to this dictionary if we're using an array. Maybe we can do better than that, and maybe a The dictionary, if implemented one way, is going to be fast. Maybe if implemented in another way it's going to be slow. So we'll consider these other design possibilities today too in the context of phone books and other data structures as well. After all, if you have an iPhone or an Android phone and Apple or Google only decided that you can have 50 friends because they implemented the Contacts app in an array, I mean that would be an annoying limitation. So presumably they've done things a little more dynamically as we'll do today. So let's focus on the first of the data structures we saw back in week 2, that is an array which we recall was just a chunk of memory where you can store values in it back to back to back, and that was the fundamental definition. The values are back to back to back or contiguous in memory, and as we've seen, we generally have to decide in advance the size of an array. So for instance, if we want to store 3 values like 12, and 3, it might look pictorially like this or in code, let's go ahead and implement this same idea and take a moment. Whip up our very first program here and we'll call it say Li.c. And in this program, let's just do something demonstrative of how you could use arrays to store 3 things in memory. It's quite simply the numbers 123, but you can imagine it being 3 people's names, 3 sweaters, 3 people, or any other piece of data as well. So I'm going to go ahead and at the top of lists.c include `stdio.h`. I'm going to then do in main void, so no command line arguments. Then I'm going to go ahead and give myself in a red. A of integers of size 3 called list, and that's how we've done that from week 2 onward. Then just for the sake of discussion, I'm going to hard code some representative values. So the first value will be at location 0 because the rays are 0 indexed. Then I'm going to do the second value, which will be 2, and then the third value which will be at location 2, but the value will be 3. Now just to prove that we've stored this correctly in memory, let's just do a quick 4 loop 4 in to I equals equals 0. Uh, I is less than 3, I plus plus. And then inside of this for loop, I'm just going to do a quick print F of percent I backslash N printing out the value of list at location I. So it's not a useful program per se, but it gives us an array to play with. It prints out what's in it, so hopefully we will see 12. And 3 on the screen. So let me make this list program list enter and voila, we're on our way going. All right, but what if now we actually want to uh change that design and be like, oh shoot, I now have a 4th number that I want to store or just bought a 4th sweater or a 4th person wants to get in line or I want to add a 4th brand to my contacts, whatever the scenario might be, it stands to reason that ideally you would plop that 4th value right here in memory. That everything

remains contiguous. You're still using an array. Your code doesn't really have to change except for the length. All for all intents and purposes, it's the same implementation using just a bit more memory. But recall that when you declare an array of a fixed size, you only are getting promised that chunk of memory, not necessarily more memory to the right, to the left, above or below conceptually, because recall in the context of your whole computer, you've got this canvas of memory, all of which represent here bytes and there could be a whole bunch of actual values or garbage values in memory. So in a more complicated program that 123 sure might end up here, but if I also had created a string in this program, H E L L O world might have also ended up right next to it in memory, which means I can't just plop the four here because then if I'm still using that string elsewhere in my program, now it's going to say hello world instead of hello world because you're just claiming. The H that byes as your own, which does not in fact belong to your array. Of course it looks like there's plenty of other memory I could use here because these garbage values represented by Oscar are not being used. They've been used in the past, but we treat garbage values as memory we could reuse certainly. So wouldn't it be nice to maybe just plop the 123, and 4 in this chunk of memory over here? And I can totally do that. But of course if I want to do that, I've got to copy the first 3 values over and then put the 4th 1 there and then presumably give back to the operating system the memory I no longer need. So that in fact when using arrays is a perfectly valid solution and I think we can go ahead and do this in our same program. So let me go back to VS code here and instead of statically allocating memory for this array, and by static I mean literally hard coding, coding the number 3 here in a way that is permanent. Effectively, let me go ahead and do this instead. At the top of my code, let me delete the static allocation of that that array before, and now let me leverage my understanding, if still preliminary of pointers and memory management from this past week for to just dynamically allocate a guess at how much memory I need initially. So I'm going to go ahead and Use Malloc and allocate space for 3 integers, but integers take up a few bytes, and it usually is 4, but just for good measure, I'm going to say times whatever the size of an int is is the total number of bytes I want. So presumably it's going to be 3 times 4 equals 12, but I'm generalizing it. But then recall that Malloc returns the address of that chunk of memory, the address of the first byte. So if I want to create an array effectively called list, I can't just do in list like this yet, but what I could say is that all right, now my list variable is actually going to be the address of an integer and set Malloc's return value equal to that. So in code here, what I've done is I'm asking on the right hand side, the operating system, please give me 12 contiguous bytes in memory. All of those. Of course it can be numerically addressed like 0x123, 124, 125. We've had that story before. Malloc by definition, returns the address of the first such byte, and it's on me to remember that I allocated 12 if need be. So I'm just storing the address of that first byte in a pointer called List. But recall from last week there's this functional equivalence we saw between treating a pointer as an array and Sometimes even treating an array like a pointer, the C language sort of lets us do this, this conversion, if you will. So what I could do here now is quite the same syntax as before. I could say List 0 gets 1, List 1 gets 2, List 2 gets 3. And even though I have this fancy new line inspired by week 4, the syntax thereafter can be exactly the same. Why we'll recall that these three lines here using square bracket notation is just syntactic sugar for the stuff we learned last week. Specifically, I could instead of doing list 0, I could much more arcanelly say go to that address in list and put the number 1 there, please. I can say go to the address list plus 1. And put the value 2 there. I could then say finally go to the

address at list + 2 and put the number 3 there, but this looks ridiculous and even sort of an experienced programmer might not be inclined to do this if with using fewer keystrokes and more readable code they could just do instead what I did the first time around, which is functionally the same, and just treat that chunk of memory as though it's an array. And the computer will essentially do the requisite pointer arithmetic to figure out where to put 12, and 3. So even though this is still kind of fresh hot off the press from last week, it's exactly the same as we tinkered with last week. So suppose now that some time passes and I realize for the sake of the story that oh shoot, I need more than 3 integers. I need space for 4 so as to achieve this picture in memory. Well, I could of course just like delete all that code, change the 3 to a 4, redo the whole thing, recompile the code, rerun it, but let me propose that we write our code in a way that allows us to change our mind while the program is running how much memory we actually need. And case in point, if you meet someone new, you want to add them to your phone, well, you. Obviously don't want to have to wait for Apple to recompile the Contacts app, reboot your phone just to add one more person. You want the program just to ask the operating system for more memory for that new person. So in this case, let's just pretend that some time passes and now I want to go ahead and actually change my mind and instead allocate space for 4 integers instead. While I could do something like this, I could just say literally list equals Mao of 4 times size of in. Semicolon. I don't need to redeclare list on line 13 because it already exists from line 5, but this is bad. Because what have I done wrong here in line 13? I've made a poor decision, yeah, in front. Yeah, I'm wasting all of the memory I had from line 5 because I'm essentially forgetting where it is. If the list pointer is literally a pointer, like a foam finger pointing somewhere in memory, what I'm really doing is saying point it over here now, but I've completely lost track of those other 3 integers in memory, and that's what we described last week as a memory leak which you could find with Valgrind and if you didn't find it or fix it in code, eventually the computer and the program would slow down over time. So this is probably bad. It's not good. To just unilaterally change your mind and say no, no, no, forget about that memory. Give me a new chunk of memory, especially if you want to copy the old memory into the new just like I did a bit ago when trying to get the 123 into the bigger chunk of memory that can fit 1234. So how might I do this? Well, a temporary variable is kind of our go to solution. Anytime we need to remember something in addition to something we already have in mind. So let me just give myself a temporary variable called TMP by convention for short. And set the return value of this mala call to that, and then what I could do is something like this, much like my print statement earlier, I could do another 4 loop and say 4 into I equals 0. I is less than 3, I plus plus, and then in this 4 loop I could say treat that new chunk of memory as an array like we can set the I location equal to the IF location in list. So these lines here. Copy old list into new list. It copies those first three values, and then what I bet I could do at the bottom here is then just manually, I can say go to the 4th location, which when you zero index is technically bracket 3 and set that equal to the number 4. So these lines here. Copy the 1, the 2, and the 3 using a loop and then line 20 here at the moment just adds the fourth value. And again this is a stupid sort of way to write code in that if you want to put the 4 there, you should have just done it earlier. I'm just pretending that some time has indeed passed in the program and I've changed my mind along the way and I want to let the user add some value to memory. OK, but before we proceed further, I dare say that there are some Other mistakes we should clean up. One of the lessons I preached last week was that any time you use Malloc, what should you do or check for? You should always

what? You should always free. So here I'm clearly not freeing any memory, so I should definitely do that. And there was one other rule of thumb with memory. What should you always do when using malloc? Yeah. Check to see if malloc came back, which just means something is wrong, like it's out of memory or something else went wrong, and if you don't do that, your program may very well crash with one of those segmentation faults that we saw, uh, briefly in the past. So it makes the code a lot more bloated, but it is good practice. So let's just check if the list pointer I get back contains null, there's no point continuing on. Let's just go ahead and immediately return one because something has indeed gone wrong and then down here under malloc again, let's do the same. If the temporary pointer also contains null, now let's go ahead and similarly return one or any other non-zero value. But here's the subtlety and let me combine your two ideas. If I immediately return one on line 20 after the second malloc call fails, what should I still go back and do first? Yeah, you want to elaborate on your first instinct? Yeah, I want to still free the first chunk of memory because if we execute line 5 and all is well, which means that line 678, and 9 don't apply like it's not in fact nullified. We got back a legitimate value, that means we have a chunk of memory given to us for 3 integers, which means it still exists down here at line 19 and 20. So if I'm ready now to abort this program and return one to signify error, I first want to free. That original list and say to the operating system, Here's your memory back. Now, as an aside, strictly speaking, this is not necessary because the moment the program itself quits, the computer is just going to give back the memory to the operating system. So when programs quit, the memory leaks sort of go away, but your code is still buggy and generally we're running software that doesn't run for a split second but for minutes, hours, days, continually, in which case it's best practice to squash these memory bugs now check for null free any memory so that you never indeed encounter these kinds of leaks. All right, so let's forge ahead a little bit more and let me propose that after we have done the copy, we now want to similarly free the original list. However, what I think we're going to want to do first is, after freeing the original list is remember that the new list is effectively. That which we allocated the second time around. So even though this program's getting a little long, notice that what I've just done is I've said, OK, store in the list variable the address of this new chunk of memory so that list now with a foam finger is effectively pointing here instead of up here. But before that I made sure to free what my finger was pointing at originally the list pointer. All right, lastly, let's just scroll down to the bottom of the code here. I can manually change the 3 to a 4 just to demonstrate that I've stored all 4 values in here and then at the very end of the program, I think I have to free the list again because now the list is pointing all the foam finger to the bigger chunk of memory, the one. 234 and then I can go ahead and return 0 at the very end because all is hopefully well at this point. Let me go ahead and open my terminal window again and make this version of list. I made a lot of mistakes here, it seems. Let's scroll up to the very first call to undeclared library function malloc. What have I apparently done wrong? Or forgotten What have I done wrong? Yeah, you back. Yeah, so in standard lib.h is where malloc is actually declared. So let's just add that quickly. Let's go ahead and include standard lib.h. In addition to stdio.h. Let me clear my terminal window, rerun, make list. Enter now we're good.list and phew, we see 1234. OK, so at this point in the story, all we've done is write a dopey little program that allocates memory for 3 integers 12, and 3, then changes our mind and allocates more memory. 4 integers, freeing the original chunk of memory after copying the 1st 3 integers into the new memory and adding that 4th value. But this is kind of a lot of hoops to jump through and let me propose one refinement here.

So if back in VS code, we go back into list.c here, it turns out that at least this loop isn't strictly necessary, not to mention the fact that we already have another loop for just printing the list. If I want to more cleverly reallocate memory, it turns out. That there's another function that we didn't talk about last week but is in Stanlib.H2 called Realloc, which, as the name kind of suggests it reallocates memory but a little smarter in that it will try to grow your existing chunk of memory if it can, which is going to be super efficient because then you can just plop the 4 at the very end. Or if there just isn't room there because maybe someone else put Hello world right there in memory elsewhere in your program, it's going to do all of the copying for you. So what you get back ultimately. A pointer to the new chunk of memory containing all of the original data as well. However, we're still going to have to check for null. We're still going to want to free the original list if something goes wrong and then return one. We're still going to want to add the 4th value because Realloc has no idea what more we want to put in the list, but I can in fact delete my other 4 loop whose purpose in life was just to copy all of those integers from old into new. All right, that was a lot. Let me pause for any questions. How does realloc know that it should reallocate the memory in least? shouldn't you tell like if you have a lot of mas before? Very good question. That's because I wrote a bug that we didn't trip over because I didn't compile this version of the code. So the question is how does Realloc know what to reallocate? Well, according to the documentation, which I forgot to read, you need to tell Realloc what the address is of the chunk of memory that you do want to realloc. So the first argument to re-alloc, which I did admit. forget until a moment ago is to put the address of the chunk of memory that you already malloced earlier so that it knows to go there, see if there's indeed some garbage values it can reclaim at the end of that chunk of memory or if it has to wholesale move things elsewhere in memory to give you 4 times the size of the end this time instead of just 3. But still, things can go. Wrong. Like you still want to check for this null value because realloc might not be able to give you enough memory or your memory could just be so fragmented that even though you want 4 bytes, maybe there's 3 bytes over here, 2 bytes over here, 1 byte over here. If there aren't 4 contiguous bytes, realloc could fail and it will return null to signify as much. Other questions on any of this? Why do we still need the temp variable for the same reasons as before? Because if we just say list equals realloc and something does go wrong, realloc by definition will return null but not touch the original memory, which case we have now lost track of where that original chunk of memory is, so we can never go back to it to print it, to change it, to free it. So we have to use this temporary variable here. Good question. Other questions, yeah. Is there a reason that we free list instead of temp? Uh, so let me, so down here or further down? OK, so further down, let me scroll down to where we came from. So here, after we've added this 4th value to temp, I've gone ahead and freed a list which at this point in the story is still pointing to the original chunk of memory, the 123. Then I am updating. List as a variable to point to the new chunk of memory. Then I'm doing my thing by printing out all of the integers therein. Then I am freeing what list is then pointing to. So I'm not technically freeing the same address in memory multiple times because I'm in the intervening time moving what list is pointing to. Absolutely, yes, it would be correct to go ahead down here and just say temp because temp is still in scope. It's still pointing at the same thing. I would just argue that that's semantically wrong because at this point in the code really list is the variable you care about. Temp was really meant to be a throwaway temporary variable, and you're asking for trouble if you use a temporary variable later than you, the programmer, intended. And if a colleague did that too, who knows what

you've done with the temporary bolt in the meantime. Good questions. Yeah, in front. Always goes I like memories Correct. Realloc will try to give you more memory in the same location as before if there's room at the end. So realloc will do two potential things for you. So if the computer's memory looks like this, you're sort of out of luck because Realloc can't give you this bye. However, if it finds 4 bytes down here, for instance, Realloc will not only allocate those 4 bytes for you, it will then copy the data for you over to it, which is wonderful because it just means we don't need an extra 4 loop all the time we do this. Yeah, in front. How does it know how much data to, uh, because how much, how does, how does Realloc know how much data to copy because the operating system, and you can think of it as the standard library, `STDLIB.H` keeps track of what memory has been allocated for you in the past. So when you pass in that same address, it knows it has essentially a lookup table, a dictionary if you will, that tells it what memory has been allocated already, so you don't have to worry about that. Yeah, in front. Good question. In other programming languages, you don't always have to declare the length of an array. Case in point Python coming next week. That is because someone else who invented that programming language wrote all of this kind of code for you. And indeed that's one of the goals with our transition between weeks 5 and 6 is to demonstrate that all of these problems are still being solved just not by you and not by me anymore. We're standing on the shoulder. Of other smart people who have invented not just new code but like a new language and a new compiler or as we'll see an interpreter for it so that we can hide all of these lower level details because honestly as you can see already, like this is an annoying number of lines of code just to have a conversation about the numbers 1234 in Python we could reduce this code to like two lines of code, one line of code. It's going to be fine. All right, so with that said, among the goals here was to demonstrate that there are a bunch of ways in which we can implement these data types, but let's talk more concretely about what we'll call data structures, which are concrete definitions of how you use the computer's memory to lay stuff out in memory, and using data structures, you can implement stacks and queues and dictionaries and all of these other things. So we're going to put into your toolkit today a whole bunch of canonical data structures that like every computer scientist does and should know that you want. Necessarily implement all of the time yourself, but when you use some feature of Python or Java or C++ or some other language, you are choosing among typically implementations of these data structures that someone else has written the code for so that you can just benefit from the functionality and the features thereof like that FIFO property we talked about or LIFO without having to get into the weeds too much yourself. So when it comes to data structures, let's consider that. We have at our disposal now a few new pieces of syntax and C, and we're going to add just one more today. We saw last week that we have the `struct` keyword, and we've seen that for a few weeks now. Whenever we want to invent our own data structure, we can use literally `struct`. We saw in the past that you can use the dot operator to actually go inside of a structure to get at a person's name or their number, and we saw last week the star operator. For dereferencing a pointer, de-referencing an address to actually go somewhere like inside of a structure. Wonderfully today we're going to see that you can actually in some cases combine the dot and the asterisk into a single operator with two characters that literally looks like an arrow, and that will help reflect the yellow and black drawings that we've done over the past couple of weeks where we have an arrow on the screen pointing somewhere. This literal arrow. Code is going to line up with that same concept. So let's introduce the first of our alternatives to arrays. An array again is

a contiguous chunk of memory where the values are back to back to back. Among the upsides so fast because like all of the data is right there. We've seen since week 0 you can do binary search and just jump around randomly by just doing simple arithmetic to go to the middle, the middle of the middle, by just dividing by 2 a couple of times and rounding as needed. The problem with the arrays, to be clear, is that they are statically, they are statically allocated to be a specific size, maybe 3, maybe 4, but it is a finite value, which is problematic because look at all of the code we had to write just to resize these things again and again. Well, what if we sort of try to preempt that kind of pain and try to just build up a list by linking it together no matter where. the values actually are in memory and move away from this constraint that everything has to be contiguous. After all, as I said a moment ago, if the computer has plenty of memory here, here, here, here, that collectively is more than enough memory, but none of those individual chunks is quite as big as you need for an array, well, heck, let's at least try to leverage all of the available memory and stitch together the data structure as opposed to really holding firm this constraint that the array. Be back to back to back and contiguous. So a linked list is something you can now build using that syntax from last week and a bit more today in your same canvas of memory so that for the sake of discussion, suppose that we want to store first in our list the number one. Well, we all know already that it might very well exist at an address like OX 123 for the sake of discussion, but it's somewhere there. Suppose that you want to store a second value in memory, but you didn't think about it initially and so you weren't smart enough to put it like right next to the 1 and then the next value next to that. But you know somehow from Mao or similar functions that you could put the number 2 over here at address OX 456 for the sake of discussion. And similarly, there's room for the number 3 over here at say address OX 789. So already we have a list of values in memory, but because they are not continuous, you can't just do some trivial plus plus trick to go from one to the other because they're differing numbers of bytes apart. They're not just back to back one byte. So what if we try to solve that problem? In the following way, instead of just using one byte for each of these values, let me waste a little bit of memory or spend a little bit of memory and have some metadata associated with our data. So data is value or values you care about. Metadata is data that helps you maintain the data you care about. So let me propose that we use two chunks of memory for every value, such that the top of each of those chunks represents. Actual Verro you we care about 12, and 3 respectively, and you can perhaps see where this is going. The second chunk of memory that I've allocated to each of these values could perhaps be a pointer to the next one, a pointer to the next one. And if this is the end, we can put our old friend OX 0, aka null, and just treat that as the end of the list implicitly. So even though these things could be any in memory by just storing with each value the address of the next value in memory, creating effectively a treasure map or bread crumbs, however you want to think of it metaphorically, we can get from one node to the other. And indeed that's going to be a term of art we start using a node, it's just a generic structure that contains data and metadata usually like the number you care about and a pointer to the next such node. Um, these are not to scale as an aside. This is typically 4 bytes. A pointer, as we've discussed, is technically 8 bytes, but it just looks prettier to draw them as simple squares on the screen. So what does this really mean? Well, who really cares about OX1, 23456789? We can really think of this actually as being more of a picture with arrows, but to keep track of this list of three values, I do propose that we're going to need one additional value over here, and it's deliberately just a single square. Because to keep track of

this list of three values, I'm going to use just one variable called say list and store in that variable a pointer as we defined it last week, the address of the first node. Why? Because the first node can then get me to the second. The second node can then get me to the 3rd, and so forth. So what's the upside now? If I want a 4th value somewhere on the screen, I could put it here, here, here, here, wherever there's enough room and just make sure that I update the arrow to point to that next chunk, update the arrow to point to the next chunk. There's no copying of data. 12, and 3 can stay there now forever until the program quits, and we do actually free it, but we can just keep adding, adding, adding, or growing this data structure in memory. So that is what the world knows as a linked list in Python to which you were essentially alluding a list in Python is indeed a linked list. Other languages call these vectors, but they are essentially arrays that can be grown and shrunken automatically, effectively without you having to worry quite as much about it. So how does the code for implementing something? This work. Well, let me propose that we have this familiar friend of a person which we claimed in past weeks has a name and a number associated with them. We know from last week that strings are not technically a thing in C as a keyword, so that's technically just char, star, name and number, but same idea otherwise, and this is what we defined in the past as a person. So this is a structure we've seen before. I now need to implement the code equivalent of these rectangles, each of which has an integer, and then a pointer to the next such value. So let me propose that we delete what's inside this structure, change the name from person to node, which again is a generic term for a container of values, and let me propose that inside of this new node structure we put literally an in for the number we care about. There's going to be my 123 or 4, and then, and this is a little bit new. Let's include in this structure a pointer to the next such node. It's a pointer in the sense that it's an arrow. It's the address of the next node, so that's why we say node star. I could call it anything I want, but semantically calling it next makes perfect sense because it's the next such node. But this isn't quite right for annoying technical reasons. I need to do one other thing here. I need to technically, and we've not done this before, put the name, give a temporary name to the structure, if you will. So literally say struct node here, even though I've already said node here. Why? Because I technically need to change this line to say struct node star. Long story short, why is this necessary? We'll recall in the past C and the compiler reads your code top to bottom, left to right. Well, if we in a previous version of this code, we use the word node here, but the compiler never sees the word node until down here. Like it's just not going to compile because the word literally doesn't exist. We saw this with functions in the past, so with the solution. To that was to put the prototype higher up in the file and then it would compile OK. You can think of this as somewhat analogous whereby if I give this structure a name on this first line, even if it's redundant to this one, then I can say struct node inside of these curly braces because the compiler has already seen the word node there. So just you have to do it this way. So now that we have this in code, we can kind of start playing around with actually storing these things in memory. So let me propose that we go ahead and do this by transitioning back to BS code here and let's instead of using our array-based implementation, let's implement the first of our link lists, and I'm going to be a bit extreme and pretty much everything inside of Maine. I am for convenience now going to include the CS 50 library, not so much for the Charar thing, but because, as we discussed last week, it's still useful for getting ins and getting strings and other things which unless you use ScanF are much harder and more annoying to get and see. So let's go ahead and do this outside of Maine, let's go ahead and invent. this node called struct node

here. Then inside of my curly braces we'll give every such node a number and every such node a pointer to the next such node, and we'll call this whole thing node by convention. Then inside of Main, let's go ahead and do this one step at a time. Let me propose that to create a linked list initially it's empty. So how do I represent an empty linked list? Well, I could call the variable list and set it equal to null, but what is the data type for a linked list? Well, per the picture that we had up earlier, insofar as all we need is a single pointer. At far left here to represent the address of the first node in the list. I dare say all we need to say is that our list is of type node star. That is to say, what is the link list? Well, it's by definition, the address of the first node in the list. So that's the first subtlety here. So that gives me a picture with no other nodes. It just gives me a single pointer initialized to null. Now let's go ahead and for parity with the previous example, just do something 3 times. So in this 4 loop structured exactly as before, let's go ahead and allocate. New node. Ask the user for a number to put inside of it, and then start stitching things together so as to achieve a picture in memory quite like this. So how am I going to do this? Well, first I need to allocate a new node. How do I do that? Well, I can use our new friend Malloc and allocate the size of a node. I want to store the address of this chunk of memory somewhere, and what I'm going to propose is that we have a temporary variable, and I'll call this in. Which whose type is that of a node star. So what am I doing here? I'm trying to build up this list in memory so that I first have a pointer to the list. I first have a pointer that is null pointing nowhere. No list exists. I then want to go ahead and create one new node, store value in it. And then point my list at that node. Then I want to do it again and again a total of 3 times. So how do we do this? We allocate space for the size of a node, however many bytes that's going to be. It's probably going to be 12 because it's 4 for the end and 8 for the pointer, but who cares? size of will answer that question for me. I'm going to store the address of this chunk of memory inside of a temporary variable called N for node, and that's why it has to be node star, because it's going to be pointing to. An actual node. I'm gonna do my quick sanity check. So if N equals equals null, we can't proceed further. I'm going to go ahead and just return one right now. So that's just sort of boilerplate code you should be in the habit of doing any time you're using Malloc. But if all goes well, let's do this. Let's go to the address in N. And then go inside of that node and change its number to be whatever the human wants it to be by using get in and just prompt the human for their favorite number. Then let's go to that same node and update the next field to equal for now null because all I want to do is allocate one new node with that number. That's it. Then I'm gonna need to stitch this together further. So I'll propose that all we need to do, and let's clean this up first, is now make sure that we string these nodes together. This syntax isn't quite right because technically, because of precedents I need to oops I need to. Uh, de-reference N and then go inside of it. I need to de-reference N and then go inside of it. However, this syntax, if it's looking a little overwhelming and you have no idea now what's going on, thankfully in C there's much simpler syntax, which is this go to the node and go inside it to get the number. Go to the node and go inside it to get next. So the. The arrow notation that I promised we would now have is the same thing as using the star operator, the Dereference operator, parenthesizing it, then the dot operator, which is just a pain in the neck to write out all the time. I dare say narrow number and narrow next is just much simpler. It says go to N and point at the number field or the next field respectively. All right, so the last thing I'm going to propose we do, and then we'll make this much more clear in picture form as this, let's go ahead and prepended. The node to the list and by prepent I mean insert it at the beginning, insert it at the beginning, insert it at the beginning

again and again. I'm going to say `n` next equals `list`, then update the `list` to set equal to `n`. And then after all of this mess, I'm going to return 0. OK, this was a huge amount of code, but let me give you a quick recap, then we'll paint a picture. Here is my `list` initially, so the foam finger is pointing to null, which means the list is of size 0. There's nothing there. Then I asked the computer to do this 3 times. Give me enough memory for a new node. Then, after checking that it's not null, put the user's favorite number in it and update the next field for the moment to null. Then lastly, go ahead and prepend this brand new node to the existing list, and by prepended I mean put it at the front. So `N` at this moment is pointing to that new node, and I'm saying, you know what, whatever the current list is, empty or otherwise, set the next pointer equal to the list, whatever that list is, and then change the list to point at this new node. So now let's do this more carefully step by step in picture form. So I'm going to propose that we go through some of these representative lines as follows. Here is the first line of code, even without the assignment. If you just allocate a variable called `list` that's a pointer to a node, what you essentially have is a box of memory that looks like this. It's a Garbage value though because there's no assignment operator, so who knows what's inside of this pointer. That is why in my actual code I set it equal to null, which effectively creates in memory the same box but gets rid of Oscar the Grouch and puts the null value there. So we know it's not a garbage value, it's a pointer known as null. So that's what that very first line of code did in the computer's memory. The next thing I wanted to do was allocate enough memory for a node, not a node star for a whole node. I want that whole chunk of a rectangle given to me in memory. That's going to return to me the address of the first byte thereof, and I'm going to store that in a temporary variable called `N`. So at this point in the story, `N` is going to be a pointer of its own, another box that initially. It is going to be a garbage value, but because I am using the assignment operator, it's going to point to that chunk of memory which Mao, if successful, presumably allocated for me in the computer's memory. So `N` for all intents and purposes, points at that same chunk. These values are still garbage values because it's just a chunk of memory. Who knows what it's been used before. But that's why after this line of code I took care to get an in from the user and then initialize the next pointer to null. So for instance, for the sake of discussion, let's get rid of get in for the picture and just say the human typed in the number 1 initially. Well, that's equivalent to putting the 1 in the number field by first going to the address of `inn`. And then dereferencing it using the star and the dot notation respectively. So that means follow the arrow and then change number to the value 1. Then the next line of code. Or rather, or equivalently you can just do the same thing and thankfully now C syntax lines up with what the pictures look like we've been drawing. Go to `N`, follow the arrow to the number field. That's literally what the syntax is telling me. Meanwhile, if I use that same syntax again for `narrow` next, set it equal to null, that's like saying go to `N`, follow the arrow, and change the next field in this case to null, or we'll just blank it out to be clear. So at this point in the story we have allocated the node, we have stored 1 and null there. `list` is still null. `N` is pointing to this, but the whole point of this exercise is to add this node to the list. So we need to somehow update this value, which is why ultimately I'm going to do something like `list` equals `n`. Now that seems a little weird semantically, but recall that `N` is a point. That is the address pointing at OX 123 or wherever that is. So to point `list` at the same node, it's equivalent to setting `list` equal to `N` because then we'll effectively have an arrow identical from `list` pointing at that new node. And at this point I don't even care what `N` is anymore. It was always meant to be a temporary value. This now is my list. So even

though I did it in code already preemptively in a loop, the first iteration for that loop literally created this in memory. Let me pause before we go through numbers 2 and 3 for any questions. Because the VS code version looks scary, this is perhaps a little more bite sized. No, OK, so how about we do this twice more for 2 and 3 respectively. So again, inside of our loop we're back to this line which asks the operating system for enough memory for the size of a node, stores that address temporarily in a variable called N. So here's our friend Oscar brought back onto the screen. Maybe the new chunk of memory is over there. This effectively points N at that chunk of memory. The next line of code inside of that loop that's relevant is this, and we'll get rid of get in to just pretend that I literally typed in 2. We're going to go to this version of N, follow the arrow, go to the number field, and set that equal to 2. The next line of code we start at the end, follow the arrow, change the next field to null, and then same lines as before, we now need to update. equaling n, but something's about to go wrong here. If I update list to point to the same node that N is pointing at, watch what happens. I set list equal to that, and because it's temporary, might as well go away at this point, but. What have I done wrong logically here? Yeah. Yeah, I lost the arrow to the original node. I have orphaned the first node because now nothing in my code is actually pointing at it. I've got in duplication two pointers pointing at this chunk of memory, so this thing, even though we obviously as humans can still see it, we have lost track and code of where it is, which means that is the definition of a memory leak. I can never get that back or give it back to the operating system until the program itself finally quits. So I think I need to be a little smarter and not do this line quite like this yet. I think what I want to do, and I've rewound, so list is still pointing to the original list and is pointing to only the new node. What I think we need to do is something like this. And this is why the code was fairly non-obvious in VS code at first. Go to N, follow the arrow, go to the next field, and here's the cleverness point this pointer to the existing list's value. So if the existing list is pointing here, that just means, hey, point this to the exact same thing because now I can safely update the list. To point at the same thing as n, so its arrow now points here, but even when I get rid of N, I wonderfully have the whole thing stitched together. And the metaphor I often think of is like around Christmas time and olden times when people would like stitch popcorn together. That's what you're kind of doing with a thread here. You're trying to stitch together these nodes or popcorn kernels, if you will, such that one can lead you to the next, can lead you to the next, can lead you to the next, but you can never let go of part of that strand. In the process. So here now we have a list which is great because notice we haven't touched the 1, but we've added the 2. We can go ahead in a moment and add the 3, but you can perhaps see where this is going. I'm kind of doing it backwards by accident, but we'll get there soon. So now let's allocate a new node, run through in our mind's eye all of those same steps. I'm going to hopefully end up with a list that now looks like this. And even though it's kind of long and stringy, these values could be anywhere in memory, but because of these various pointers, I can jump from one to the other, making more efficient use of everything inside of the computer's own memory. All right, but of course we've got this symptom that didn't really intend whereby the whole darn thing is backwards, but I think that's kind of OK for now. But I'd like to propose that we consider how we can now maybe traverse this thing and actually print out the values in memory. So let me go ahead and do this. Let's go ahead and how about Let's say, let's go back to VS code here. So at this point in the story, we've got the same code that implements that same idea, except I'm using Get in just so that I can dynamically type in the 1, the 2, and the 3 without having to hard

code it into the actual code. Suppose that after doing this exercise, I want to do something interesting like print the numbers. Well, we don't have that code yet in this version of my program. So let's bring that back. Last time I did this just using a for loop and array notation, and I think I can do that, but let me propose first that I implement this idea pictorially. Here's the same diagram. This is what exists in the computer's memory. If I want to go ahead and print out these numbers, albeit in reverse order, let me propose that we can do this by giving ourselves another temporary variable. We'll call it PTR pointer for short, and that's like having another foam finger that points at The start of the list. So it's not pointing at list, it points at whatever list is pointing at, which means here. Then I can print out the 3 pretty easily so long as I next update pointer to point to the 2, print it out, then point it to the 1, print it out, and eventually I'm going to realize, oh, I'm out of nodes because the end of this. List is null. So that's the idea I want to implement now logically in code, create a temporary variable called pointer, set it equal to whatever the list itself is. Print out the value, update the pointer, print out the value, update the pointer. Print out the value, update the pointer, realize it's null, and stop. So in code it's a relatively small loop even though the syntax is still pretty new since we've only just started playing with memory since last week. But what I'm going to do is exactly what I proposed. I'm going to create a new pointer called PTR and set it equal to the list itself. That's like having another foam finger temporarily pointing at the first element in the list. Then what I'm going to do is say while that temporary variable is not null, go ahead. And traverse the list. What do I mean by that? Well, let's go ahead and print out the current element in the list by using percent I backslash N and printing out whatever the pointer is pointing at, specifically its number field. So that is follow the arrow and print out the number. Then inside of this loop I'm going to update after doing that my temporary variable called pointer to be equal to pointer arrow next. And that will have the effect with just those few lines of code of implementing precisely this idea. I first set pointer equal to the list, which happens to point here first. I then do my print F and then I update the next field. I rather, I update pointer to be the value of pointer. Follow the arrow next. So if this is OX123, for instance, that is what is now in, sorry, if this is OX 456, that is what's now in. So the arrow effectively looks there in my for loop. I print out with percent I this number, and then I go to the next field, follow the arrow, and then set it equal to rather whatever this pointer is here, OX 789, set it equal to the pointer there. So I effectively move the arrow there. Then lastly I update PTR to point to the value of this next field, which is null, which means effectively pointer itself is null, which means the 4 loop cleverly. Stops now because I was supposed to do this whole loop while pointer is not null, but pointer is now null. And just as an aside, if you prefer the semantics of a 4 loop, there's nothing new here per se. I can do this exact same thing using a 4 loop simply as follows, and it's a little tighter to implement less follows. I can say 4. Instead of I equals 0 in that old approach, I can actually use pointers in a 4 loop like this for node star pointer equals the start of the list. Keep doing something so long as pointer does not equal null, and on each iteration of this loop, update the pointer to equal whatever the pointer's own next field is. And then inside of this 4 loop, print out using percent I backslash N, the current pointer's number. Field semicolon. So here is where again we see the equivalent of 4 loops and wild loops. What you can do with one, you can do with the other. This is a little more elegant in that you can express a whole lot of logic in one line of the 4 loop. Frankly, I do think the first version is nonetheless more readable. So let me undo, undo, undo, undo everything I just did on the course's website. You'll see both of these versions. This one's a little more pedantic as to what

it's doing step by step. OK, that too is a lot. Let me pause here to see if there are. Any questions? And if you're feeling like that fire hose, like this is why we transitioned to Python where all of this now gets swept under the rug, but it's still happening, just not by us in a week. Questions, yeah. Yeah, really good question. So we, here I've been preaching like we don't want to lose memory. We don't want to leak memory, and here I am fairly extravagantly now spending twice as much memory to maintain this data structure, that's going to be among the themes with all of the data structures we talk about. If we want to gain some benefit like dynamic growth and shrinking of the data structure, you've got to give me something, and what you've got to give me in this case is the ability to use more space. In a bit today and after a break in particular we're going to we'd really like these algorithms to be faster. Well, that's fine, but you're going to have to give me something in return. You're going to have to spend more space to make the code faster. And so time and space and financial cost and human time and any number of other resources are all things that you need to evaluate as a programmer or a manager and decide which is least and or most important to you. And right now I don't care about space as much as I care about the dynamism that I'm trying to solve first. Other questions on here, yeah. this point. Yes, why am I using Pointer instead of N? I, well, yes, I could reusen at this point. I deliberately chose to use Pointer for two reasons. One, I'm using it for different reasons here. Two, it's not necessarily the best idea to use one variable here for a specific purpose and then reuse the name down here besides, it's out of scope at this point anyway. So it just makes me feel better that I have different variables doing different things, but it would not break if I did it your way. Other questions. Yeah, and back. Are pointers temporary, not necessarily like the linked list we are building up in memory exists because we are using pointers to build this data structure and to keep it intact for as long as the program is running. My temporary variables N and pointer PTR in this case, those are ephemeral, and I'm only using them to kind of stitch things together temporarily. A good question. All right, so let's now motivate why we're spending so much time sort of stitching these things together so carefully. Well, here's our little cheat sheet of common but not exhaustive running times. Let's consider what the running time is for some fairly basic stick operations like inserting a number into a linked list, maybe searching for a number in the link list or traversing it, and also deleting ultimately numbers in a linked list. So here is my list initially, completely empty, and suppose I go ahead and insert the one. Then I insert the 2, then I insert the 3 using code like we just wrote. I love this approach because even though it looks a little scary at first, this is probably the simplest way to implement insertion into a linked list. Why? Because I'm just constantly prepending the next element, prepending, prepending, which means All of my hard work is just here at the beginning of the list. So even if this thing has 1000 elements in it, I'm only manipulating some pointers all the way over here pictorialally at the left, which means it's pretty darn fast. So given that definition in this picture, what would you say the big O running time is of insertion into a link list when using my current implementation? Big goal of one. Why? Well, it's not literally one step, but it is a constant number of steps because if we literally counted the lines of code I was executing, it's a few steps to sort of 0.1 thing up here, point the other thing down here, then update the third and boom, we're done. In particular, what my current code does not care about is the whole length of this list. Why? Because I'm never traversing the whole thing for the insertion part. I am obviously for the printing part, but for the insertion I'm just prepending again and again. The downside though of this approach is that the whole darn thing is coming out backwards. I'm not doing anything with regard to the ordering of

these elements, which means what's the running time of search going to be, for instance? If I tell you search for like the number one, find it for me. What's the running time gonna be there in Big O? They go of, yeah. Big event because in the worst case it's going to be all the way at the end and we've seen this scenario before, so it's big O of end for searching. It's definitely big O event for traversing or printing, but that goes without saying if you want to print every element, obviously you have to touch every one of the N elements. But what about deletion? Suppose I want to delete an element that's going to be in big O of. Also, and why? Because again, in the worst case it could be all the way at the end. So only insertion as currently implemented is big O of one because we are exercising full control over where the new elements go irrespective of what the actual values are. So things could escalate quickly here if we do actually want to start keeping things, say, in sordid order because we. No longer just naively plop things at the very beginning of the list. I think we need to start being a little more careful as to where we put things. So in fact, even though we're doing OK on Insert right now, we still have big O of N for the searching and for the deletion, which we won't do in code, as well as of course for traversal. So how else might we go about building this list? Well, let me propose that we could maybe append to the end of the list. Let's try that and see if it gets us anywhere better. So here's my list initially, completely empty, aka null. I go ahead and insert the number 1 as before, but now in this algorithm I'm going to insert the number 2 and the number 3. So this is great because now by chance it ended up beautifully in order, but that's because I chose the numbers 123, but we'll come back to that detail. Let's consider now what the running time is of this algorithm of insertion using a pending to the list. What's the big O not big running time of insertion now? Big old even end. So it's sort of strictly worse because now it's always going at the end. Now I could be a little smart about it. I could just allocate another pointer and just always have another pointer pointing at the end of the list, just as I have a pointer pointing to the start of the list. That's totally fine if you're willing to spend one more pointer, which is a drop in the bucket, a legitimate solution, but where I'd like to go with this. Let's maintain sorted order no matter the order in which the numbers are inserted, whether it's 1233212 13, 3312, whatever order the human types in the numbers, I want to build the structure out such that they always end up in sorted order just so that my contacts in my iPhone or my Android phone for instance are sorted as intended. So how do we go about doing that? Well, here we're still dealing with some big O. Let's try this. Here's my list initially empty. Now we, the user inserts person number 2 1st, so it ends up there. Then they insert number 1. I'd like it to go there. Person number 4, it goes over there, and then person number 3, it ends up here. Even though it's sort of obvious with a piece of paper and pencil how to stitch this together, this is now an annoying number of logical steps because there are so many opportunities where I could screw up and orphan one or more of these nodes. But let's consider the scenarios that we encounter we might encounter. Maybe we get lucky and it's like an empty list and we just have to insert one new node that is trivial. We've done that already. The 2 was super easy to implement. The one could be really easy to implement too because that involves the pre-pending scenario and we've seen that prepending is super simple. So there's only two other scenarios to consider app pending, if it's a really big number and ends up at the end, and we've talked about but haven't seen code for that. The annoying one, I dare say is going to be when the new number belongs in the middle. But I propose to think through it this way because now you just have 4 problems to solve, not just one massive. Ill-defined problem. You've got scenarios in which you want to insert a new node into

an empty list. You want to prepend the new node into the beginning of the list, append it to the end of the list, or somewhere in the middle. So that's like 4 blocks of code in my program. I can now sort of take the proverbial baby steps and implement this bit by bit. And to do this, let me propose that in a moment I'll switch over to VS code, but Uh, sort of Julia Child style, I'm going to open up a pre-made version of the program that actually gives us a working solution, albeit initially with some bugs. So here we have out of the oven, this version of List.c at the. Top of the file. I've got my same includes as before. I've got my same structure as before here. I've again got in main void. I've got the beginning of my list here, setting it equal to null, and then for the sake of discussion, I'm going to insert 3 values for this example, 12, and 3. By allocating enough room for a node, setting it equal to N, then I'm going to make sure a sanity check that N is not null, and then I'm going to actually populate this with the human's first choice of values. So let me scroll down, but as such, there's nothing too new just yet. Here we have the lines of code in which I'm getting in it from the user, setting next equal to null, and then I'm pre-pending no matter what per our earlier version that we did on the fly. This new node to the list and then updating the list to point to it and then down here I'm printing the numbers. So this is where we left off, but this is a pre-made version that's nicely commented. It's on the course's website for reference. What I'm not doing now is intelligently prepending, appending, or plopping the code in the middle. So how do we do that? Let's take a look at this version of the code. So everything thus far is the same. And if I scroll down besides the new comments, you'll see that now I'm starting to make some decisions after I have allocated the new node and populated its number and next field. As an aside, I don't strictly need to initialize the next field to null because eventually, as we've done in every past example, I've updated that next field anyway. However, because this one might now end up at the end of the list and I just want to program defensively, initializing pointers to null before you're ready to assign their value. is a good thing in general. So here's the first of the questions I'm going to ask myself. If the list into which I am inserting this new node is empty, so it's the beginning of the story, super easy, just set the list equal to the address of that new node, and we're done. That's what happened when we inserted a bit ago, the number 2 for the. Very first time. So indeed what has just happened here is that now the list previously empty contains only a node containing 2. However, thereafter there was another scenario. So when we moved on in our story and added the number 1 to the list, well that happened to end up at the beginning, but it could also end up at the end or in the middle. So let's break down those scenarios here too. So here, if it is not the case that the list is empty in that if condition, we're going to end up here now in the Eps. What do I want to do here? Well, let's go ahead and for now in this simplified version append it to the end of the list so we can see that code. How do I do this? Well, I'm using a 4 loop, much like the one I had before, which just allows me to traverse the existing list whether it has one node or many, and I'm going to ask a question. If following the current nodes pointer field next field leads me to null. AKA the end of the list. OK, let's go ahead and update the end of the list to actually equal the new node. So in other words, if I'm sort of following, following, following all of the arrows and I reach a node whose next field is null, no problem, update that next field to point to the new node I want to insert, irrespective of the values. I just want to append this node no matter what, and then I want to break out of the code. Then at the bottom of this version of the program, it's all quite the same printing out the numbers using the 4 loop version of my code from before instead of the Y loop, but they're equivalent. But what I did do in advance in baking this version of the program is also

go through the motions of freeing every one of the nodes afterward, but we'll come back to that. So this version of the code, just to be clear, only appends nodes to the list. It's still not treating things in order, but we've now seen two of the scenarios plucked off. The list is empty or it has numbers, and we want to put something at the end. So let me propose now that I take out of our distribution code another version of this program that does that and a bit more. I'm going to go ahead and open up in just a moment, a new and improved version of lists.c and now. It looks almost the same at the top, scrolling down, scrolling down, scrolling down. Here's some now familiar code. If the list is empty, do that simple thing as before and just prepend it, or rather just set it equal to the list. But here is now where we're adding some inequality. So if the number in question belongs at the beginning of the list, so if the number in the new node N is less than. The number in the current list, which is presumed to be the first node at the moment, then go ahead and update the new node's next field to point at the existing list and then update the list to point at this new node, thereby giving us from 2 in the list to 1 and 2 in the list. To be clear, if I go back to VS code here, what's happened here is because 1 is less than 2. Of course I'm going to update the new nodes next field to point to the list. What does this mean? Well, the new node at this point in the story is the new node for the number one because that's the second thing we're inserting. I'm going to update its next field to be whatever the list a moment ago was already pointing at. So this is the after effect, but a moment ago list was pointing at only the 2. So now the next field of the 1 points at the 2. And then lastly here in this line, I update the list pointer to be the address of that new node. And here's where I'll wave my hand a little bit today because it starts to escalate quickly. It's useful and it might very well be useful for problems at 5 in particular, but I think more healthily reviewed step by step at a slower pace. Here is where I'm asking myself, all right, if it's not the only element in the list and it doesn't belong at the beginning of the list, well, it belongs somewhere later in the list, which gives me two final scenarios. Let's figure out which scenario we're in. Let's use this 4 loop to iterate over all of the as as many of the nodes in the list as we need to. If we get all the way to the end, because our pointer variable now equals null, it's like following the arrows, following the arrows, and maybe we're trying to insert the number 5. I've already hit the number 4. I've hit null. 5 belongs at the end. So here we have our promised append code, which is exactly the same as before, but now I'm doing it conditionally if I've indeed found my way to the end of the list. And then lastly, let me scroll down just a little bit. If it's not the case that the list is empty and it's not the case that the new node belongs at the beginning and it's not the case that the new node belongs at the end, I'm just somewhere in the middle of the list because the new number I'm inserting is less than the one I'm looking at here, and it's OK to use two arrows, but I'll wave my hands at that for now. These 3 lines, 2 pointer manipulations and a break is what's going to stitch together that 3 in between the two. And the 4. And let me propose for lecture's sake take this on faith that this collectively does stitch things together properly, but I do think, as you'll see in Problem set 5, it's a much better exercise to think through a little more carefully step by step because there's just a lot of fine tuning of these pointers together and the order of operations does matter. But at the very end of this program, notice this is Kind of mindless, even though the syntax is undoubtedly less familiar. Here is how just like traversing the whole list to print it out, we can similarly do one more pass over the linked list and free every one of the nodes. But notice it's not quite as simple as just saying free the whole list. Free is not that smart. Mao is not that smart, and Even though you have called Malloc 123 times, you have to really call free. You have to call free 123 times.

You can't just pass at the beginning of the link list and say you figure out what to delete because it has no idea what a linked list is or what your data structure actually is. So the reason that this loop is a little complicated is that what I'm doing with these three lines is essentially traversing my list. And making sure that I have a pointer that when I'm ready to delete the 31, I have a pointer pointing at the 2, and then I free the 1. I update my pointer to point at the 3, and then I delete the 2. I update my pointer to point at the 4, then I delete the 3, and then I delete the 4. So there's a bit of trickery involved in making sure you don't orphan things step by step. OK, that was a lot. Let me pause here. To see if there are in fact any questions, even though we're deliberately waving our hands at some of those details. Questions on this. Now, let me add one final flourish. If we were to really quibble over this, I mean, my God, we're up to 80 lines of code already just to implement the numbers 1234, but there are some subtle bugs in here at the moment. So for instance, suppose that something goes wrong with Mao inside of this for loop here, and suppose that it's not your first iteration, something goes wrong on maybe the second or the third iteration. Why is this error check suddenly bad as I've implemented it? Yeah. I didn't free the memory from the previous iteration, so this is where like, oh, like memory management starts to get really annoying because if you do want to practice what I've been preaching, which is free any memory you've allocated and you've already allocated one, maybe two nodes because Mao is again failing maybe at the last iteration here, you have to somehow go back and free all of that, and that's fine. Like we have code at the bottom of my file here which could traverse through the existing list and just free it all. So I could just copy paste that code, put it in. To my if condition and then run that code 2 to delete the whole list, but at this point, if you're copying and pasting, you're probably doing something wrong. And so let me propose as a final version of this just for your reference later in the 9th and final in version 9 of this file here, 0 indexed what we have. Give me one second to just make a quick copy and copy it over in list 9.c. Our last version of this, we have the following whereby now in my function. Uh, in my main function, I have the exact same code as before, but I've taken the liberty of implementing an unload function so that I can call it here as well as at the bottom of this main function so I can unload it here or unload the list there, and all I've done now is in good form in terms of design, just implement the notion of deleting a linked list in its own function so I can call any number of times from any number of places. But just so you've seen how I might do that there. Alright, so let's ask the question, after all this, what is the running time of inserting into a linked list? They go of Say a lot, big of. And damn it, like that's no better. All right, what's the running time of searching a linked list? Big old event, damn it, uh, what's the running time of deleting from a linked list? Big O of end. So like everything is literally big O of ends. So there's the price we've suddenly paid. We have an hour after we started with arrays gotten to the point where we can dynamically grow in a linked lists, and I dare say, even though we've not done it and won't do it today, shrink the linked list by freeing things that we don't need. So we have the dynamism and we can make more efficient use of memory even if it's very fragmented and there's a few bytes here, a few bytes there, but we've paid this price because with the rays recall, even our phone book example, we at least had binary search, the running time for. Which was big O of log and so my God, not only are we spending more space, the darn thing is slower. Surely this is not how our phone contacts are implemented. Surely this is not how stacks and cues are always implemented. And indeed it's not. This is just going to be a stepping stone to now doing a sort of mash up of data structures whereby we take the best features of arrays, the best features of

linked lists, mash them together to get new and improved data structures, but for that we're going to have to have some cookies first and we'll come back in 10 minutes. Cookies are now served. All right, we are back. So let's recap how we got here and why. So we started with our old friend's arrays, which we introduced in week 2, and recalled that the whole appeal of arrays was that one, as all things go, like relatively simple, certainly now in retrospect, but more importantly, they were really darned fast. Like arrays insofar as they are stored back to back contiguous in memory means that we could do very simple arithmetic recall to like figure out the length of it and then divide by 2 to get the middle, divide by 2 again to get the. of the middle and so forth. And even though we might have to deal with a little bit of rounding, arrays lent themselves to binary search and thus logarithmic time, so big O of $\log n$. But today I claim that the downside of arrays that that you have to decide in advance how big you want it to be and if you guess wrong and it's too small, how much memory you ask for, you then have to reallocate memory and that's fine. It's solvable with `Malloc` or `realloc`, but it's going to take some amount of time to copy all of the old memory into the new memory. Whether you do it with a for loop or `memcpy` does it for you. Meanwhile, we only did it with like 3 values, maybe 4, but imagine it being 3 million values that you now need to allocate more space for. You're going to waste a huge amount of time copying 3 million values from the old location to the new. And so that's just generally not very appealing. And so that motivated a whole discussion of linked lists where by now we can create a more dynamic data structure whereby we only allocate memory as we need it, so we don't. To worry about underestimating or overestimating and therefore wasting memory, we can just go bit by bit for each new value. We allocate another node, another chunk of memory, and the thing just grows and grows and grows. But as we saw just before break, the downside is, even though we're avoiding the inefficiency of having to move stuff around in memory, once allocated to nodes can stay where they are and we just update our pointers, all of our running times for searching, inserting new elements, deleting old elements would seem to be big O of N . But why was that? Well, in the context of a linked list, recall that it might look a little something like this whereby we have a pointer called `list` pointing to maybe 4 values like this, and suppose that we do want to search for a value. Now it's nice because in our latest version of this linked list, it was sorted from smallest to largest, and that was always a precondition of doing binary search, but even though it's obvious to our human eyes where the middle is, it's like roughly over there. How is the computer going to figure that out, or how is your code that you write? Well, unfortunately, the way we've stitched a link list together with these pointers is if you want to find the middle, you can, but you've got to start at the beginning, traverse the whole thing to figure out how long it is, then do it again and stop halfway through once you know what the halfway point roughly is. Then if you want to search the middle of the middle, you've got to do that whole process again. And so now just to use binary search, you need to spend big O of n steps just to even find the middle. Now if your mind is kind of spinning and you're like, Well, maybe I could just kind of cheat and use a pointer to always point to the middle of the list, totally fine. You can spend in some additional space to remember the beginning in the middle of the list, the end of the list, but where does that stop? What if with binary search you go not just to the middle, but the middle of the middle, the middle of the middle of the middle, are you going to keep around a pointer to every element because if you do, you're essentially back to an array if you've got one location for every other location. So it just kind of devolves into a mess even though there's some minor optimizations

we could in fact make. In fact, we didn't talk about it yet, but one common alternative to a singly linked list, which ours is, it's. With a single pointer from node to node, computer scientists also like to talk about doubly linked lists where there's arrows going both directions, which actually would have simplified some of the last code that we looked at because I don't have to look ahead to figure out what I want to free or what and where I want to insert some value, but that too doesn't fundamentally change the speed, it just makes your code a little easier to write. So in short, with Linklis we get dynamism. We can now grow and shrink things without wasting time copying, but we've lost hold of our binary search, and that was very appealing as far back as week zero when we wanted to do something quite quickly. So let's see if we can't make some mashups now. Take some arrays, take some link lists, literally mash them together into a sort of Frankenstein data structure and see if we can't get some of the speed of arrays, but the dynamism of link lists. And so I give you trees. If you think about it in your mind's eye what a family tree looks like, where you typically have some parents and then some children and some grandchildren and so forth, it's this sort of tree-like structure, even though by convention it's drawn top down instead of bottom up like trees in the real world. But the top of that family tree we're going to call the root of the tree. It just so happens to indeed grow down. But a tree is a very common data structure and it's interesting vis a vis arrays and linked lists in that it's the first of our two dimensional data structures. An array is effectively just a Single dimension along from left to right. A link list is essentially the same, even though in reality it might be up, down, left and right in memory. It's still just one thing stitched together in a single dimension. A tree adds now a second dimension and specifically useful for us is what we're going to call binary search trees, which is spoiler, going to give us back the ability to use binary search, but we're going to store the data a little more cleverly than in arrays alone. Instead of storing our data in one dimension in a binary search tree, we're going to Store in effect in two different dimensions and that's going to gain us some speed. So here for instance is an array of 7 numbers as we might have seen it back in week 2 when we first introduced arrays. Let me draw our attention to the middle element and then to the middle of the middles, and then the middles of the middles of the middles just by color coding them slightly differently. If I were to run binary search on these numbers or the lockers that we had on the stage a few weeks back, I would jump to the middle, then the middle of the middle, and so forth. The catch though is that implemented. is an array, it's not going to be very easy to add new values. Why? Because if I want to add the number 8 or 9 or 10, I might get lucky and there might be room in memory here, but I might get unlucky, in which case then we've got to start jumping through those hoops of Mao or reo and and copying all of this memory to a new location, which is doable. We solved it in code, but it's going to be slow for larger data sets. So can we avoid that? Well, maybe I deliberately color coded things like this because let me propose that instead of storing these seven values in an array. Let's store them in a family tree-like structure like this where I just kind of exploded them vertically on the y axis here. So now the middle element. The 4s at the top of this tree. The 4, the 2 and the 6, which were the middle elements after the middle, are going to be to the left and right of the 4, and then these leaf nodes, so to speak, we borrow a lot of vernacular from the world of actual trees. These are leaves in the sense that they themselves have no children. They're at the edge of the data structure, are going to be the middles of the middles of the middles, but all of the data is still there. I've just exploded it from 1 to 2 dimensions, and let me propose that now that we have this technique of using pointers, which we use with. C code, but you can depict them

pictorially with arrows. Let me propose that we stitch together these seven values in memory using a bunch of pointers whereby now each of these nodes drawn as a single square for simplicity is going to have not only an integer associated with it and not just one pointer but per these arrows as many as 2 arrows associated with it. So our nodes are about to go from data structures with 2 things, a number and a pointer to 3 things a number. And 2 pointers for the left and right child respectively. And I dare say now that we have a two dimensional tree data structure, consider how you might find a number therein. Suppose I'm searching for the number 5. Well, I start at the root of the data structure and even though our human eyes obviously know where we're going, notice what's important about this binary search tree. If I go to the root of the node of the tree, I see the 44 is obviously less than 5. What does this mean? This means I can divide and. the problem right off the bat. I know that 5 is going to be to the right of this node, which means effectively if you think in your mind's eye about snipping the branch there, I have just halved the problem essentially like dividing the phone book in half. Why? Because I don't even waste time looking at this subtree, the left child of the 4 element. Meanwhile, if I go from the root to its right child here, I see the number 6.5, of course, is less than 6, so this is effectively like snipping off that child. because I don't need to go further there because I know a smaller element is going to be in this direction, and that's the key property of a binary search tree. It's not just a family tree with numbers all over the place. They follow a certain pattern. Every element is going to be greater than its left child and less than its right child, assuming you don't have identical values, and that property is actually a recursive one to borrow terminology from a couple of weeks back, recall that a recursive function is one. That calls itself a recursive data structure like the pyramid in Mario is a data structure that can be defined in terms of itself. Well, a binary search tree is a recursive property insofar as if it applies to this node, it also applies to this node. Case in 0.2 is greater than 1, but it's also less than 3. It's true over here. 6 is greater than 5, but less than 7, and it's technically. True of the leaf nodes because the definition is at least not violated there because they don't even have children themselves. So this is a binary search tree because of that pattern. So this then invites the question, Well, how long does it take us to search for a value in a binary search tree? Well, if the number is 5, it's going to take me 12 steps. But if there's an elements here, can someone want to generalize that either mathematically or just instinctively big go of. Logn and even if you're not quite sure how the math works out any time you take a data set and you have it, have it, have it, we're talking about log based 2 of N again and indeed that's going to describe the height of this tree. The height of this tree is essentially log based 2 of N because if N is 7, it's going to give me. essentially 2 when we round appropriately. If we round up if we've got 8 elements log based 2 of 8 to the 3, so that means 3, so 123, it kind of works out even if I'm doing that a bit quickly. The height of this tree is log base 2 of N, AKA big O of log N. How long does it take to insert? I think it's going to take log in because I can insert over here or over here or over here depending on where the number goes. How long does it take to delete? I'll claim it's going to take about the same. So wow, we're back in business. I've got now the ability to grow and shrink my data structure because if I want to insert the number 8, it's going to go right there. If I want to insert the number like 5.5, I can see where I would put it. It's going to be easy to add new nodes by just updating the pointers without copying everything in memory like we had to for arrays, but there is a downside here. I gotta concede something. What am I, what price am I paying? What's the trade off here to gain that dynamism and that speed, but. Yeah, I'm literally using 3 times as much

memory now because even though it's not depicted here explicitly, each of these squares represents an integer and a pointer and another pointer, so that's like 16, that's like 20 bytes at this point of memory instead of just 4 bytes for each of the integers in an array. Nowadays though. Space is pretty cheap. We all have very large Dropbox folders, iCloud folders, and the like, so it's not really a big deal to use that many more bytes. Certainly not a big deal for 7 numbers, but if it's 7 million numbers, maybe this isn't the best data structure to use even if speed is important. You've got to decide ultimately based on your actual use case what matters more. So in short, a binary search tree you can kind of think of as an amalgam of, or rather a variant of a linked list, except that every node has as many as 2 pointers instead of one, which is what gives us now this second dimension. And in fact this translates pretty nicely to code. In fact, if we consider how we implemented in a linked list, a node, recall that it looked like this where you got a number in each node and a pointer to the next element in the linked list. Well, I. Think for a binary search tree we can sort of borrow this as inspiration, make a little more room because we need two pointers instead of 1, and I'm just going to call the left child the left pointer and the right pointer, but here is the 3 times as much space, give or take, because I now have 3 elements associated 2 pieces of metadata and 1 piece of data that I actually care about to stitch this thing here together. All right, well, if this is the data structure there, how could I implement this in code? Well, here's where recursion again comes into. The fact that a binary search tree is recursive in nature in that what you say about this node, about it being greater than the left child and less than the right child can be said of this node and this node and this node and this node, you can leverage that beautifully in code like this. So suppose I'm implementing a search function in C whose purpose in life is just to say yes or no, true or false, the number you're looking for is in this tree, which might be a useful thing to check in in an algorithm. Search is going to take two arguments I propose the number you're searching for and a pointer to the tree, that is the root of the tree initially. So how do you actually traverse this thing in C code? Well, we can pluck off the easy case first, the base case. If the tree itself is null, like if you hand me nothing, I'll give you your answer right now, false, like there's no number here if the tree is empty. So that's easy. Otherwise, if the number you're looking for is less than. The number in the current node. So tree is what's passed in a pointer to the root. So if you follow the arrow, you can get inside of that value and see its number. If the number you're looking for is less than that, OK, you want to what snip off the right tree and dive down the left subtree. So you search the tree's left child for the same number. Else if the number you're looking for is greater than that number, you search. For the trees, right, child, for that same number. And the 4th and final scenario is what? Well, if the number you're looking for equals the number in the current node, you got it return true. And if you're uh recall some of our past design discussions, this is sort of a waste of everyone's time to ask this question explicitly. Let me tighten this up design wise because there's only 4 possible scenarios. Either there's nothing there, it's to the left, it's to the right, or you found it. It's right there. So whether or not you agree at this point in your programming career, like there is a beauty to this code that most programmers would claim is here and that it's so relatively elegant whereby you've defined what the function is. You've got this base case which is arguably one of the clunkiest parts, but the fact that you can just check a value here and then traverse the exact same structure, but a subset of it by traversing the left subtree or the right subtree is like a beautiful application of recursion. And it allows you to uh search for this thing no matter where it is in the computer's memory. Questions then on this idea

of a binary search tree or this actual. Code thereof. Uh, nope. If the number is not there, we recall. So if we get all the way to the bottom of the tree, such that now I'm at one of those leaf nodes, and that's not the number I'm looking for, such that there's no left child left, no right child left, this conditional is going to kick in and I'm going to return false. But if I find it along the way, whether it's at the top of the tree or somewhere in the middle or among the leaves, I will eventually return true. Good question. And to be clear, even though I'm calling this a tree, that's true certainly for the first time I call this function because I'm passing in a pointer to the whole tree structure. But if you think about it, what's the left subtree and the right subtree, it's just a smaller tree. It's like a baby tree that's attached to this parent node, so to speak, so it's perfectly reasonable to just call the search function with that child because it in turn has a whole subtree below it or the right child which has the whole subtree below it instead. All right, so I like this direction. We've now kind of improved upon link list. We've gained back some of our performance because we can now find something with big O of log and time. I don't love the fact that I'm using 3 times as much memory roughly. That feels like kind of a high price to pay just to speed things back up. But let's consider whether or not this thing is actually going to work as the data structure gets bigger and bigger as well. So it looks beautiful here as written, and that's. Deliberate because I drew the picture like this and it's got 7 elements in it. But how did we get to 7 elements? Let's start from the beginning. Suppose that the tree is initially empty and suppose that a human using get into or some other technique inserts the first element into the list like the number 2, and the goal is to maintain the binary search tree property, which means you've got to have it greater than the left child, less than the right child. So suppose the human using get into or some other. Technique next gives me the number 1. No big deal. I plop it right there. Is the left child. Suppose they give me the number 3 next, no big deal. It goes right there. I have very deliberately manipulated this story to work out beautifully such that the tree is smaller, but it's still a binary search tree and nicely balanced, so to speak. But what if the user, for whatever reason just gives me a more perverse sequence of inputs like the worst case scenario. to give me 3 elements and suppose they give me 1 1st. OK, that's the root. Then they give me 2. OK, that's cool. That's like the right child. But what if they then give me 3? Well, to maintain that binary search property, the 3 has to go over here. Suppose perversely then they didn't give me 4, then 5, then 6. Imagine in your mind's eye where this story is going. What have I accidentally created in memory then? A link list which is like bad for all the reasons we discussed before the break because even though we're getting the dynamism, it's devolving into big O of N. So I've kind of manipulated the situation here with their original example with 7 elements and then 3 elements by making sure that they were inserted in just the right order because unless you are clever about how you build the tree in memory, it could very well devolve from a tree. In two dimensions into actually a linked list in one dimension. And now this is just a long and stringy tree that does not violate the binary search tree definition, but it is surely not balanced in this case. Now as an aside, if you take higher level languages and data structures and algorithms, there's many different alternatives to binary search trees that actually have baked into the algorithms, a little bit of rejiggering of the structure so that really as soon as you insert this 3. You spend a little bit more time and clean the situation up and essentially what you do is like pivot the thing around this way so that 2 becomes the new root and then 1 hangs off of it and 3 still hangs off of it. So with each insertion or deletion, you rebalance the tree as needed, which does cost you a bit more time, but it avoids the thing devolving into big O of N again and we

won't do that in code, so this is recoverable, but not if you implement it naively as I did at least verbally. In this story, all right, well, can we do better than that? Well, why might we want to? Well, at this point in the story, it certainly could devolve into big old event, and that's not great. Certainly for large data sets. It's nice that we're back to log in, at least if you take on faith that we could kind of rebalance this thing as needed and maintain a logarithmic height for it. But really the holy grail of data structures is to achieve something that is big O of 1, like constant time whereby no matter how many numbers or names or sweaters are in the data structure, it will take just 1 step or maybe 3 steps, right, even 100 steps, but a number of steps that is completely independent of how many actual pieces of data are in the data structure. That is to say over time it doesn't get any slower even if you've got tens, hundreds, thousands, millions of elements in there already. So how do we gain something like big O of one constant. Time, the appeal of which is reminiscent of our early picture from week one. Like this was our early algorithm for finding someone in a phone book or counting students in the room, something linear, literally straight lines. This was the logarithmic curve, which especially as you zoom out, starts to get very, very appealing time wise. Something that's constant time looks even prettier. It is a straight line at like the one step mark or the two step marks, whatever the constant number of step marks is. And even though Algorithmic will still grow in perpetuity. Constant time by definition never changes, and this is what we'd really like. So when you're searching for someone in your phone, you're searching for something on Google, you're asking a question of ChatGBT, you get an answer like that in constant time, independent of how much data is actually in there. Well, let's see how we can do this. To do this, we're going to at least need a new building block, a term of art known as hashing. Hashing sort of formally takes an infinite domain of value. Use and maps it to a finite range of values. So from high school math class, domain is the input, range is the output. So an infinite domain to a finite range is the goal here of hashing. And we might see this actually in the real world when you're playing, you know, games or whatnot, or you're cleaning up after a game. Like here is, here are some super jumbo playing cards that we got online and suppose that you want to just get these into sorted order. You could do this very painstakingly. There's 52 cards here. You could kind of lay them all. Out and start sifting through them and put the 2 over here and the 4 over here and the hearts and the clubs and so forth, or you can start to look at the cards and bucketize them first to take a 52 size problem and maybe shrink it down into 413 byte problems. So here for instance is where the first diamond might go, the club here, spade over here, diamond over here, and I can kind of just do this again and again, bucketizing literally. All of these values so that I've got a very simple heuristic that allows me to move the cards into these buckets, each of which is going to have a subset of the values, and then I've got smaller problems I can deal with. So assume that I bucketize all 52 of these values, then I've just got 4 problems remaining, and I dare say it's a little easier then because they're all of the same. So and so I can pretty easily sort it from ace to king or whatnot because those are effectively just numbers at that point. So hashing refers to again taking values from an infinite range. In this case it can be finite and it is in this case, but if you were doing it more generally with numbers, you just have to map it to a finite range like 1234 finite. Number of buckets of values at which point then you can solve the problem a little differently or a little more efficiently. So why is this germane? Well, I would propose that if we want to start organizing our data in memory toward an idealistic goal of achieving constant time, hashing might be one ingredient for the solution there too. And generally we're going to describe

the process by which you decide what input goes to what output is namely what's called a hash function. It's a mathematical function or a function code that takes as input a card from a deck or maybe a word from a dictionary and outputs a value that represents the bucket into which it should go. So in the case of our contacts app, for instance, of course in the GUI of it you have all of your friends and family top to bottom alphabetically presumably, and you might want to ideally find someone quite quickly, ideally in constant time. The naive implementation that Apple or Google could implement is just use linear search, search through all of your contacts top to bottom, and eventually. You will correctly find the person, but wouldn't it be nice if they instead use an array and then they can use binary search and get you the person in logarithmic time? That's great. But if you have a lot of friends and family in there or a much larger data set, wouldn't it be nice to just jump to the answer in one step instead of even log of end step? So that's our goal. Can we get close to or actually at constant time? So with the hash function, we essentially have our old friend problem solving here, the inside of which the algorithm is known as a hash function. And for instance, if I'm looking at Mario's number. I might now want to look for Mario, not top to bottom or not divide and conquer jumping around to the the middle of the middle of the middle. Let me just figure out what bucket Mario is in. And in the English alphabet there's 26 letters of the alphabet A through Z, either uppercase or lowercase. And suppose that I want to find what bucket Mario is in. Well, much like these cards and the suits thereof, wouldn't it make sense that anyone whose names starts with A goes into the first bucket and maybe the Bs go into the second bucket and the Zs go into the last bucket. So it stands to. that if I pass in Mario to a hash function implemented in C or some other language, I would like to get back to the number 12 because M is the 13th letter of the alphabet, but if we start counting at 0 with our buckets, which are essentially an array, then it's index location 12 instead of 13. Similarly, if Luigi is the input, I'd like to get back to the number 11. So my hash function somehow takes his input in this story, a string, and gives me an integer. I claim there's theoretically an infinite number of names in the world in the English language. But there's only gonna be 26 possible answers from this hash function, 0 through 25, so that's our infinite domain to our finite range instead of 4, it's now 26. All right, so what. Should we do with the computer's memory to leverage the fact that we can very easily bucketize names based on the first letter of someone's name? Well, let me propose that the hash function part of this, arcane as it looks, is actually pretty straightforward. So if you wanted to translate this idea into C, you can include `Cy.H`, which we've used a few times to get access to like functions like `2 upper`, and this is just to make sure you can be case insensitive. Here's my hash function. And it's going to return an `int`, which is the goal. It takes a string as input. We'll call it `name`, and what does this function do? Well, it's kind of some clever asy arithmetic. It first converts to uppercase, the first letter of that person's name. So if it's all lower case, forces it to uppercase `Y` because I want to subtract no matter what 65, aka the `Asky` value of capital A from this, and I don't want to screw up the math if I'm doing like a lowercase letter minus a capital. I want minus is all. So this will return to me a number between 0 and 25 inclusive, because if it is a letter, a name that starts with a, I'm only looking at the first letter. I'm subtracting off A, that gives me 0, and I'm going to return 0 as a result, if it's Z, I'm going to return 25 instead. Now there's no error checking in here. If you type in non-English symbols, it's going to break. So let's just assume for simplicity this is indeed an English name that's coming in. I can refine this a little bit. I'm going to propose moving forward in our final week here of C there are some added defenses you can put in place when writing code. Like if you know that

you're receiving a name as input that is you're passing something in by reference, there's a danger now per last week because now the caller of this function, whoever's using this function, is telling you where to find Mario and where to find Luigi's name. The problem with that is that you could go to that address and actually change their name in memory, even if you're not supposed to. You're supposed to just use the name. So you can do something like const which says you should not be able to change this value even though I'm not giving you a copy of it by value. I'm giving you a reference there too. Another refinement here is that a hash function for an array is the goal. Should return a value that's 0 or 1 or 2 on up, never negative, so we can even more protectively say it's not just an in, it's an unsigned in, and we talked briefly about that last week, albeit in the context of CARS. These are just like minor improvements that makes your code arguably better designed because you're opening yourself up to fewer possible mistakes or issues. All right, so with that said, let's now assume that we've got this kind of function implemented and we can now use it to decide what bucket to put. These people's names into, well, let's give you what are called hash tables, which are sort of the Swiss Army knives of data structures, the kind of thing that some computer scientists have been quoted as saying if they were stuck on a desert island with only one data structure, this is probably the one they would want. Why? It's just really generally useful because it allows you quite powerfully to associate keys with values, which is to say to come full circle today, hash tables are often how you would implement. at a lower level, the thing we began class with talking about dictionaries, collections of key value pairs that after all is what a phone book is if we call it, you know, names and numbers, but it's keys and values. That's what an actual English dictionary is the Oxford English Dictionary. It's a bunch of words and definitions or keys and values, so useful in general to be able to associate one piece of data with another ergo hash tables. So here's how you might implement and see a hash table. You want it to be a size 26, for instance, so 26 buckets from A to Z, hence the 26. You want this to be an array, and that's fine. This is an array of 4 buckets. I'm going to use an array of 26 buckets because a hash table 2 is going to be an evolution of our linked list mashed together with an array. So a hash table in short, is going to be an array with linked lists, as we'll soon see. Here's the array 26 pointers to nodes. So I'm going to give myself an array of pointers that is going to store ultimately a whole bunch of person objects like this. So for instance, here's a char name, Chatar number as we've discussed in the past, representing a person. These are the pieces of data I might want to store in this data structure. However, let's simplify it. Let's not worry about the phone number because we're not going to call anyone today, but for a linked list of persons, I'm going to need to store, let's say the person's name. But also a pointer to the next such name, to the next such name to the next such name. So again, I'm just deleting number as being unnecessary detail, but if we're gonna have an array of linked lists, this is our new definition of node for this part of class whereby it's not for a tree, it's now for a hash table, and we'll see this in action now. Here's my array of size 2. 6. I drew it vertically, but who cares? These have always been artists' renditions thereof. It just fits nicely on the screen this way. This is location 0. This is location 25. So any A name should end up over here. Any Z name should end up down here, and so forth. Let's just generalize this away as letters of the alphabet for clarity. That's where all the names are going to go. So hopefully Mario here, Luigi here, and everyone else. So what are each of these squares? They're just pointers to nodes initially all null, all claim, but as soon as I insert Mario into this so-called hash table, I'm not going to put him literally here. I'm going to create a new

node in memory, put Mario there, and then stitch it together because if I get another M name, I'm going to stitch it together and together and together again. So for instance, here comes Mario into this data structure. So this is a pointer to a person structure. Here's Luigi and here's a third character. As well, Peach. That's all working out great. There's a whole bunch of characters in the Nintendo universe. Here's a lot of them. Unfortunately, especially if you're a fan, there's also other names that do start with M and L and other letters of the alphabet. So we're poised to have what we're going to call collisions, which is a downside of using a hash function. If you're going from something infinite to something finite, by definition you're going to have a heck of a lot of potential collisions somehow, multiple M names, multiple L names, and so forth. So we've got a mini. This somehow. Well, if you meet someone in the real world whose name happens to start with M and you already are friends with Mario, well, you could delete Mario from your phone and put that new person there, but that's kind of dumb. You could clobber the value, that is, or maybe you put the M friend here, and when that fills up, you put the M friend here, and then when you meet someone else whose name starts with M, you put it here. But then it just evolves into this mess at which point now there's no rhyme or reason as to who is where. It devolves back into something linear. If you have to search the whole darn thing looking for M friends just because you ran out of space where you want. So here's the beauty of mashing together in array with a linked list. You hash the name to the intended location like box 12 here, and then you just start stringing them together in a linked list and hopefully you don't have too many of those collisions, but at least now you don't have to delete or make a mess of the data structure. So here's another bunch of names, 3 starting with L. Here's a bunch for the other letters of the alphabet, and it's just a linked, it's an array now of linked lists. This then is a hash table. So the question to consider now, is this better than an array? Is this better than a linked list? Well, I dare say it's better than a linked list because if it were a linked list from A to Z, what would be the running time of searching for anyone? Well, I'll spoil it, big old event because even if it's alphabetically sorted, you've got to start at the beginning and go all the way through the list potentially to find someone like Zelda whose name starts with, of course, Z. But here, we have an array of linked lists. So what's really the running time here? It's not quite as bad as N steps, because if you assume a uniform distribution of names such that the world of Nintendo maybe has as many M names as L names as A names as B names, you could assume that there's a bunch of chains, a bunch of linked lists here chained together, but they're all roughly the same. So maybe you have end names in your phone book this way, but there are these lists are only of size, they're only 126 of that length because you've got that many names there. So what's the running time? Well, ideally we'd move away from link lists with big O of N and achieve our cons. In time, but we have these collisions to worry about here. Just to be clear, we want to get from big O of N to something constant time, but we're not going to get to constant time if we've got collisions. If we've got 3 L names and a few B names and a few A names, we can't just jump to that location. And find the person we're looking for. So what's the fundamental goal? Well, I think we want to maybe use a smarter hash function and here depicted is an excerpt from a bigger hash table that is a much bigger array that assumes that you're not looking at the first letter of everyone's name, but apparently what instead. The first three letters of the person's name, which just decreases the probability of collisions because in this model, I dare say there's no one else's name in the Nintendo universe that starts with LIN. So now Link has its own location in memory and similarly for Luigi LUI I believe is unique in the the Nintendo

universe, so we don't have a collision. Unfortunately, while this does seem to eliminate collisions based on this tiny example, what's the tradeoff? Or what's the catch? Uh yeah. This is a lot more memory. I mean, kind of hinted at the fact that I didn't even fit most of it on the screen anymore. Here's LAA. Here's LUG, but what about all of the other letters of the alphabet and the other combinations of all possibilities. Moreover, some of these just don't make much sense, at least in English or in the Nintendo world. I don't think there's anyone whose name is going to start with AAA or AAB or AAC. AA or AA and so forth. We're wasting a huge amount of space to reduce the probability of collision. So that's fine. We might get constant time now, but at what cost? Well, a heck of a lot more memory. And so this is one of the tensions when using a hash table is you want to come up with a good hash function that's maybe a little more sophisticated than the first letter, but not so wasteful that you need a crazy number of buckets and therefore a huge amount more memory. So really, even with collisions, it's not quite as bad as N steps because technically if you have k buckets where k is like 26 buckets or 4 in this case, technically if you do assume that the names are uniformly distributed over A through Z, the English alphabet, well, each of those linked lists is going to be hopefully no bigger than N divided by K . So N divided by 26. But what do we know about higher order terms when doing big O notation? Big O of N divided by K . Yes, it's faster, but asymptotically, that is theoretically, you're still talking about big O of N . So here's the tension though, like it's absolutely going to be faster. It will be like 26 times faster than a linked list, but it's still just big O of N because it's going to take an amount of time that's still linear in. The size of the data set, so we seem to have strayed yet again away from our constant time search. So can we find this holy grail? Well, we kind of can if you let me spend just like a lot more space. There are tries in the world which weirdly is short for retrieval, even though we don't say retrieval, but a try. Is a tree made out of arrays, right? So at some point computer scientists were just like mashing things together Frankenstein style like linked lists and arrays, and now we've got trees and arrays. You two can mash something together and come up with your own, but let's look at what a try actually is because it is going to get us that constant time grail. So here is the root of a try. You can think of each node in a try as really being an array of values A through Z in the case of an English problem like we've been playing with here. And what you do is you treat this array as being indexed from 0 to 25 or equivalently A through Z, and you treat each of those elements as a pointer to another such node in the try. And what you do is implicitly store the names that you're storing in this data structure by going to an appropriate location based on the first letter in their name and then adding a pointer that represents the second letter in their name, adding a pointer that represents the third letter of their name, and so forth. So what do I mean by this? Suppose we want to insert toad, one of the characters from the Nintendo universe, first. If we count up where T is in the alphabet, this. A pointer here will be changed from null to a pointer to a new node that represents the second letter in Toad's name, which is going to be of course O. Then to insert TOA we're going to need another node. A is going to lead me to D and for uh depiction's sake, I'm going to draw it in green even though this would actually be a boolean or something like that in memory. That indicates that toad's name stops here. So in other words, this try and memory has 4 nodes now. Each of those nodes is essentially an array of size 26, but the word toad is not actually stored in the data structure explicitly. There's no character toad, but implicitly because the T pointer is non-null, the O pointer is non-null, the A pointer is non-null, and the D pointer is in fact null at this point is the common technique here. This allows me to to insert

other names from Nintendo's universe like Todet because I can continue from here to go to the E node to the T node, uh, to the T node again, and then E node which I'll again mark in green. So you can even have names that are substrings or equivalently super strings of each other by just having all of these various bread crumbs along the way, where again a non-null pointer here to a non-null to a. Non-null to null pointer here indicates that or it can't be null at this point. This is where we have to use a boolean indicates that there is a name in this data structure that ends here and there's another name that ends here. Meanwhile, if there's a third name from the universe like Tom, same idea, but eventually we can start reusing some of these arrays whereby non-null, non-null null, or there's a boolean flag here that says True, a name ends here. Now we're reusing that same array. So each of the nodes represents the lth letter of the word or the name you're trying to store in the data structure and by playing around with null and non-null and some booleans, you can implicitly store names in this structure. Now it's way too pictorially difficult to depict lots and lots of names in this form. So just imagine in your mind's eye that there's dozens, hundreds, thousands of names. Now in this data structure but just more arrows and more arrays. How do you actually look someone up in this data structure? Well, if you want to ask a question like I toad in this data structure or is to in this data structure or anyone else, you can simply start at the root node as we would do for any tree and you hash on the first letter of toad's name, which gives you this location and you check, is it null. If not, T is implicitly there, so you follow that pointer here and then you hash the second letter of toad's name and O and check this pointer and you follow that arrow. Then you check the third, you hash on the third letter of toad's name A and you follow that arrow. Then the fourth letter of toad's name D, and you see, ah, there's a boolean here represented in green that means toad is in this data structure. And notice what's. Subtle here it doesn't matter if there's 3 names in this try or 3 million names in this try, how many steps did it take me to confirm or deny that Toad is in this try? 1234, which is arguably constant, even though the names can vary at some point there's no Nintendo name longer than what, like 10 characters, 20 characters, maybe 30. I mean there's some reasonable bound that is finite where there's never going to be a name longer than that because Nintendo's never going to come up with a crazy long name for a game. And so you effectively have constant time for looking up TOAD Todet, Tom, Mario, Luigi, Peach, any of the other names we've looked at. So this is to say a try allows you to ask questions like is Toad in this data set or equivalently what is Toad's phone number in this data set, because if you assume now that each of these pointers ultimately is not just a bull saying yes or no, but maybe it's an actual person structure with a name and a number, you can store even data like that your key value pairs where your names are your keys and your phone numbers are your values. To make this more clear, then here is a data structure, how. Might represent in each of these nodes. It's not quite technically just an array. It's an array of size 26. We'll call it children because it represents the children of that node of type struct node star. And then here, for instance, for simplicity is that person's number if we reintroduce numbers and want to store in this data structure someone's phone number as well. So using that data structure and that kind of code, you can implement a try using something as simple as this. Initially your try is just a pointer to a node, one such struct. We can of course initialize it to null to make clear that there's no names in here, but each time we allocate a node, we can then add another node, another node, hashing on the 1st, the 2nd, the 3rd, the 4th, the last character in the person's name, allocating a node as needed, flipping that boolean true or false, or adding their phone number

as a star to indicate that we have then found them. And so of all the data structures we've looked at today. Big O of 1 is actually achieved with tries, and yet curiously for problem set 5, you're not gonna implement tries, you're gonna implement hash tables, that's sort of Swiss Army knife of data structures that like every programmer everywhere knows about. Why? Like why not use tries very often in practice perhaps. Certainly can But what's the trade-off perhaps? Yeah, it's a huge amount of memory. Things have escalated since the start of class. We added, we started with 1 inch, then we added an in and a pointer and in in 2 pointers. Now I'm proposing 26 pointers plus a boolean or a data structure called person. I mean, it's escalating significantly, and the biggest catch with a try, as you might have imagined with Toad and To E and Tom on the screen, there's a huge amount of wasted memory just as we saw with the hash function. But that can be reined in as you'll explore on the problem set with a try most of the pointers in those arrays are just null and unused, and it just tends to result in you're using way more memory to solve the problem correctly, but in a way that tends to slow the computer down and just waste more memory than is useful. That said, just as we started today, there are stacks in the real world, there's cues in the real world. There are even Hash tables in the real world, which you'll indeed implement in code for problem set 5. Has anyone here ever had a salad from a restaurant called Sweetgreen in Harvard Square also elsewhere in the US, like 12, like 2 of us, 3 of us. OK, so not hard to imagine going to such a store, getting in a queue and staring at a shelf like this because what Sweetgreen and similar restaurants do when you order for pickup is they. Hash your salad into a shelf like this. And so literally in sweet green might you see some wooden shelves like this. This is the A through E bucket, the F through J bucket, the K through N bucket, and the O through Z bucket whereby if your name like Maly happens to be in one of those ranges, they will hash my salad and put it here. But of course, even in the real world, there are some constraints and what can go wrong with this here hash table system. Someone who's been there, maybe. What can go wrong? Imagine like the extreme, lots of values here, yeah. So there's no more space, right? So, and this has happened to me in the past, especially since Sure before adopting this system. They used to put the A's here, the B's here, the C's here, the D's here, and so forth. And then someone at some point realized that they were very frequently overflowing the A's to the B's and the B's to the C's, that no one was using Q or Z with any frequency, and so they were sort of wasting space and running out of space. So at some point they decided to like literally remove most of the letters of the alphabet, make the buckets bigger, and. You are, so now it's very unlikely that you're going to have so many K's through ends that you overflow the shelf, but this is in the real world a data structure like we've seen today. And so therefore among the goals, even as arcane as things seem to be getting with all the pointer notation and dereferencing this and that, really all we're doing in code is implementing real world solutions that other people have already come up with and translating them to a new domain. And the very last thing you'll do. See this week is indeed implement your very own spell checker whereby we'll give you a very large file of all 100,000 plus English words. You'll have to come up with a clever and efficient way to load it up into memory, and we'll give you tools that will actually measure how fast or how slow your code is, how much memory or how little memory your code is, so as to actually compare it against not just your own but perhaps others as well. So with that said, we'll end a bit early today. We'll see you next time.

All right, this is CS 50, and this is already week 6, wherein we transition away from C to a programming language called Python. And that's not to say that the past several weeks haven't been among the goals of the course. Indeed, in learning C, I very much think that you'll have at the end of this class so much more of a bottom up understanding of how computers work, of how programming languages work, and in particular you'll appreciate and understand better how Python and Java and C++ and Swift and so many other languages. are actually doing their thing nowadays. But recall that we started with Scratch some weeks ago, and in Scratch what was nice was that the first program we wrote, Hello World, was just all too accessible. All you had to do was interlock two puzzle pieces in order to make the cat in that case say Hello world. Well, thereafter, of course, we transitioned to C and recall that in week one we asked you to take on faith that you can sort of ignore that first line in a lot of these parentheses and the curly braces and really just focus on the essence of the program, which Clearly is still about hello world and printing it, albeit using a different function and a bit new syntax. Today, very excitingly, all of that is truly going to go away and be distilled into a single line of code when you indeed want to have the computer say something like Hello world. And this is what we mean by Python being a higher level language. So humans over the decades learned from earlier designs, earlier programming languages, what worked well, what did not. Computers got faster. Computers had more memory, and so you were able to start. Spending more of those resources in order to have the computer do more for you. And so you don't need to be as pedantic syntactically anymore. You don't need to write as much code anymore and frankly you can just start solving problems of interest to you, building products of interest to you, so much more readily by choosing the right tool for the job. And so in the real world, if you continue coding after CS 50, like sometimes C will be the right tool for the job, sometimes Python will be the right tool for the job, and sometimes it's going to be a different language. Together that you'll never have studied in school. And in fact what's compelling, I think, about this week 6, much like when I took the class back in the day, is that after CS 50 you'll have a taste of 12, maybe a few different programming languages, and that's going to be enough to bootstrap yourself and teach yourself new languages because you're going to start to recognize in the real world similarities with past languages that you've seen, programming paradigms that are still sort of with us, and the syntax, yeah, that's invariably going to change, but that's the. that you are going to Google or ask Chat GPT or some other AI about down the line so long as you know enough of it to sort of get real work done, you'll focus mostly ultimately on the ideas and the problems you want to solve and less on the syntax. And so among the goals for this week and this week's problem set and really the rest of the course is to get you more comfortable feeling uncomfortable in front of your keyboard because we're not going to give you and tell you everything you need to know for a language like Python. You're going to turn to the documentation. You're going to turn to the duck, and you're going to learn to to. Yourself ultimately a new language. So let's actually write our first program and compare and contrast with how we might do that in C. So recall that in C. We were in the habit for the first couple of weeks of doing make hello and make this build utility just kind of magically new to look for a file called hello.c and magically to create a program called hello, and then you could run it with hello. And then a week or so later we revealed that make is really just automating compilation of your. And with the actual compiler clang in this case and passing it command line arguments like 0 to get a specific output like the file name hello instead of the default, which recall was a out passing in the name of the file you want to

compile and turning on any libraries that you might want to compile into your program, link into your program beyond the standard ones. But then you could still run it in exactly the same way. Starting today, when you write Python code and then want to run it, you're simply going to run. The Python program itself. So just as KLang is a C compiler, Python is itself not only a programming language but a program as well. And with the Python program, which understands the Python programming language, can you run code that you'll have written in a file called hello.c? And what this program is doing is a little bit different from what Clang is doing, but we'll see that difference before long. But first, let me go over to VS code and let's write our simplest, our first of Python programs by doing code hello.c, and then in this file without any includes, any in main voids, I'm simply going to say print quote unquote, hello world, close quote. All right, now I'm not going to. Make. I'm instead just going to do Python of hello.c, cross my fingers as always, and voila, my first program in Python. So it's sort of obvious that we got rid of the hash and clue. We got rid of the Imain void, no curly braces, only a couple of parentheses here, but what else is different to your eyes? That's a little more subtle here versus C. Yeah, yeah, so there's no F, so the print function is a little more human friendly. It's print instead of print F where the F did mean formatted, but we'll see that we still have that functionality. So no need for the line breaks, specifically the backslash n, and yet here's my cursor on the next line. So I dare say humans over the years realized we are more commonly wanting a new line than we don't want it. And so they made the default to actually give it to you automatically. And there's one more detail, yeah. So there's no semicolon, so I finished my thought at the end of the line, but I didn't need to explicitly terminate it with a semicolon. This is just with one program, all of these salient differences, but I'd argue that we got rid of all of the annoying stuff thus far anyway, so we can really focus on what this program itself is doing. But what's exciting with Python 2 is just how quickly you can solve certain problems, and this isn't true of just Python. It's really any higher level language than C. In fact, just for fun, let me go ahead and implement. Problem set 5, wherein you're challenged with implementing the fastest spell checker possible. So let me go back here to VS code. Let's close out hello.c and clear my terminal window, and let me go ahead and do this. Let me first split my terminal by clicking this rectangular icon over here, and that's going to give me two terminal windows now, left and right, because in the first one at left, I'm going to CD into a directory I came with today, which is the staff's solution to. Problemset 5's spell checker in C. And on the right hand side here I'm going to CD into another directory I brought with me today called Python, inside of which is a translation of Problemset 5 into Python. In particular, I've implemented in advance a speller.pi file, which is the analog in Python of speller.cnc, and I've also prepared a dictionary. Pi file. Unfortunately, if we open up dictionary.i. You'll see that it's not actually implemented yet. So in dictionary.c, let's implement in Python problem set 5 and see how long it takes. Well, the first thing I'm gonna do is declare a global variable, we'll call it words, and set that equal to the return value of a Python function called set, which essentially gives me a set object wherein I can store a whole bunch of words without duplicates. Python's gonna manage all of that for me. In effect, it's going to implement what I needed to implement myself in Problems at 5 a hash table. Now down here I'm going to go ahead and define a function called check, pass in as input a parameter called Word because of course that's how it was implemented in C. But notice the difference already in Python we use a new keyword called def to define a function, and we don't have to specify the. Type of the variable being passed in word in this case, and we also don't have to specify a return type for

the function. Now inside of this checks function, it suffices to do this I'm going to return word.
lower in words, which is effectively a boolean expression asking is the lower case version of this
word in the set? If so, returned true, otherwise returned false. Done with the check function.
Now let's go ahead and define another function called load, which recall took an argument of the
dictionary that you want to load into memory. And let's go ahead now and do this with open
dictionary as file, which effectively opens the dictionary, as in C we used FO in Python we use
open. And it gives it a variable name of file. Then once that file is open, I'm gonna go ahead and
update that entire set of words which starts out empty by taking the file, reading the entire
contents top to bottom, left to right, and splitting all of the lines therein on the. New lines that
terminate each of the strings, effectively updating the set with every word in that their dictionary.
Then I'm gonna assume that it all just worked because there's a lot less effort for me to uh to
perform myself in Python, and I'm just gonna go ahead and return true T in Python. Done. Next,
let's go ahead and define that other function from Palm set 5 size, whose purpose in life was to
tell me the size of the dictionary I had loaded. Well, in Python, that's pretty easy. I can just return
the length, or LEN for short, of the set in which I've stored all those words. Done. And then lastly
I'm gonna go ahead and define an unload function which recall was responsible for freeing any
memory I myself had allocated. I don't seem to have done any of that in Python. In fact, that's
managed for me now, so I'm gonna go ahead and simply say return true because there's no
work to be done. And that's it in like 19 lines of code in Python, most of which are blank lines. I
claim I have reimplemented Problem set 5 in Python. Well, let's take a look now at the
difference. I'm going to go ahead and reopen my terminal window, and I'm gonna go ahead and
maximize it so we can see more output. And now I'm gonna go. And run Python, which is going
to be not only the name of the language but the name of the program we use today to start
running our Python code, and I'm going to run it on speller.ie, which I brought with me today,
specifically on the largest of problems that finds files, homes.text. Enter. And as with Problem
set 5 itself, we'll see a whole bunch of misspelled words being printed to the screen, some of
which might very well be misspelled, some of which are just not in the dictionary, some of which
are simply possessives of words that are in the dictionary, but at the very end of this output I
should see not only how many words were found, but the total time involved, which appears to
be 1.87 seconds. Not bad, seeing as it only took me like what, a minute or two to write the
actual code. But there is going to be a trade-off, we'll see, even though it took me much less
human time and arguably was a lot easier to implement this this spell checker in Python than I
dare say it was for most everyone in C, let's see what that tradeoff might be. Over in my left
hand terminal window in which I'm in the C directory which I brought with me as the staff
solution in C to problem set 5, let's go ahead and make that spell checker. Then let's go ahead
and do slash speller and run it on the same file, uh, Holmes.text, and see how long the C
implementation takes. Enter. And we see some of the same output might be slower sometimes
just because of the cloud and there total time spent in the CPU, not necessarily printing
everything to the screen, which might take longer, is only 1.32 seconds versus the 1.87 seconds
in Python. Now, while only half a second, that's a decent percentage of the total amount of time
spent running this spell checker in each of the windows, and so that it alone seems to be one of
the trade-offs, even though it seems to be much faster. They're easier to implement a problem in
Python, there's going to be trade-offs insofar as the code might very well run slower. And as
we'll see today, that's in large part because whereas C is of course compiled. That's why Iran

make and in turn clang, and then the zeros and ones, the so-called machine code is what you're running in Python generally, the the computer is interpreting your code, essentially reading it top to bottom, left to right, much like a human in between two. Other humans might slowly translate one spoken language to the other if those two people don't in fact speak the same language themselves. So there's a bit of overhead when using Python, but I will say that the Python community has been working on this problem for some time and so in general it's not necessarily going to be as significant a trade off because there's certain tricks we can do. And in fact underneath the hood, what the Python language can do for you and the specific interpreter you're using is technically. Semi-secretly compile your code for you into something called bytecode and then run that bytecode, which is more efficient than actually reinterpreting it again and again, but we'll see more of this over time. For now, let's take a look at maybe two other problems that we might solve, dare say more easily, more quickly than we could have in C for problem at 4. Let me go ahead and shrink down my terminal window here, close out dictionary.ie, close one of my terminal windows and CD back to my main directory, and let's go. Go ahead and open up that bridge bitmap photograph that we used in Problem set 4 and how to apply a number of Instagram-like filters there too. Well, now let's go ahead and implement maybe one of those filters, the blur filter, whose purpose in life is just to blur this image. Well, let's see how long this takes. Let me go ahead and open up, say, uh, blur.ie, which is now going to be a Python program for blurring images. It's empty initially, but I can pretty much write this. Quite quickly now let me go ahead and at the top of this file write the Python keyword from PIL for Python image library, import an object called Image and another one called Image filter in particular two features of the Python image library that's gonna make this so much easier to actually solve. And then let's go ahead and define a variable we'll call it before, representing the before version of this image and set that equal to image. open. bridge.bmp where that is of course is the name of the file we want to blur. Then let's go ahead and create a variable called after representing the after version of this same filter and set that equal to before. filter parenthesis, image filter.bo blur, and then just to be a little dramatic, I'm going to blur it more so than you needed to in problem set 4, but we'll see it more visibly now on the screen. Let's do an argument of 10. And then at the very end of this process, let's do after.save and save it in a file called say out.bmp. Done. So in just 4 lines of code, I claim I've implemented the blur function now in Python of what we did previously in C. Let me open my terminal window. Let me run the Python command this time on blur.ie. Cross my fingers as always. And indeed I've made a mistake. Perhaps even if you've never written Python before, you can see it and in fact we'll see a number of these errors, some intentional, some unintentional, but on line 4 what I intended to do was set equal to before. filter, that variable I created called after. All right, that's all right. Let's go back down to my terminal window, clear it to get rid of all that, and rerun Python of Blur.ie.ross my fingers even harder this time. Nothing bad seems to be happening. Indeed, now let's go ahead and open up out. BMP and before we reveal that, let's go back to the original, which is bridge. BMP and now dramatically, let's see the blurred version thereof. Voila, hopefully to your eyes too. It looks quite a bit blurry. Well, how about one more flourish? Those of you who are feeling more comfortable last week and implemented perhaps, uh, edge edge detection in see. Well, let's see if we can whip that up quite quickly too. Let's go ahead and write a file called edges.ie using that same bridge. BMP file, and in this file, let's go ahead and do the following. As before from the Python image library, let's import, uh, the image feature and the image filter

feature. Then as before, let's create a variable called `before`. Set it equal to `image.open('bridge.bmp')`. So so far the same as before. Now let's create a variable called `after`. Set it equal to `before.filter(ImageFilter.FIND_EDGES)`, which is different from box blur, and by definition it's going to find the image the edges of this here image, and then after, as before, let's do `after.save('out.bmp')` and just clobber the version of the blurred file that we just created. All right, that's it. Let's go ahead and open up my terminal window now. Let's go ahead and again run Python, but this time on Edges.ie, cross my fingers real hard. So far so good, and that was quite fast. Recall that the bridge. BMP image looked like this, but now when we open up this new and improved version of Out. BMP thanks to Python and just 4 lines of code, we now have all of our edges detected. So what can we then learn from C itself? Well, C had of course functions, and functions were those actions or verbs that simply got worked on. And let's go ahead and compare side by side, much like we did with scratch and C. The idea is that today onward are still going to be the same. And how they could translate to Python. So on the left here we'll now have our friend scratch. This of course was one of the first puzzle pieces we saw. It's a purple puzzle piece saying say, and it was a function insofar as it said the value of its argument, which in this case is hello world. What we've already seen in Python what this looks like. It looks similar to the version in C, but it's no longer `printf`. There's no longer a semicolon, and there's no longer an explicit new line. So in Python it's quite simply this. Meanwhile in Python there are a whole bunch of libraries as well. Now in C we had simply header files, and those header files gave you access to the prototypes of that is the signatures of the functions that you want to use from those libraries. Python uses somewhat different vernacular whereby Python has what are called modules and packages, and a package is just a collection of modules. But a lot, a module is just a library using Python Speak, so to speak. So anytime you hear someone discussing a module or a package in Python, they're just talking about using a library, and that library might come with the language itself just built in as standard, or it might be a third party library that you might download and install yourself much like I did a few weeks back when we installed the cow sayy program so that I could actually have a cow or other animals on the screen. Display text. So in C recall we had something like this `#include CS50.h`, which was the header file preinstalled for you somewhere, but we will have for at least this week an analog of the CS50 library in C also in Python just to make this transition from C to Python a bit easier. These two though are meant to be training wheels that you can take off and should take off, you know, even within a week or so. It's just meant to smooth that transition. Clear what's the same and what's different. So in the CS 50 library for Python, we also have a function called `get_string` whose purpose in life is to get a string. To access it though, you don't use `#include CS50.h`. That's a C thing. In Python you would say `from CS50 import get_string`. It's a little more verbose, but it's also a little more precise as to what you want from the library, especially if you don't want the whole thing loaded into memory. So here for instance is now a scratch program that was a little more interesting than just printing out HelloWorld. This was the first program we wrote that actually got some user input. So in fact, let me go back to VS Code and let's see if we can't resurrect this C program real quickly in the form of a new `hello.c`. So I'm going to run code of `hello.c`. And then in my, in my code tab, I'm going to do `#include CS50.h`, `#include <stdio.h>`, and then below that, I'm going to go ahead and whip up our familiar version of this, I made `void`, and then inside of the curly braces we'll bring back `string`. Even though we now know it's `char*`, we'll call our variable `answer`, set it equal to `get_string`, ask

the user, quote unquote, what's your name with a space just to move the cursor over. Still need my semicolon and C. And then after that, recall back in week one we did hello slash n. And then plugged in the variable answer so as to see Hello David, hello Kelly, or something else. Just to be safe, let me do make hello. All is well so far. hello, type my name, and this version in C seems to be working OK. So in C, these lines of code here translate pretty literally to what we just saw, although we got the answer variable in. for free. That blue puzzle piece just existed without our having to create it, but it's a decent number of hoops to jump through in order to just get user input and print it out. Well, in Python, this is going to get a little more succinct in that the Python version of this code is now going to look like this. Printf is now print. The semicolons are gone, and what else seems a little bit different. Yeah. I don't need any placeholders. Yeah, so we don't need the percents anymore. In fact, I'm curiously using a plus, which if some of you studied Java or some other language, you might have actually seen this before. Even if you've never seen Python before, you've only seen C in CS 50, you can probably guess what the plus is doing, even if you don't know the technical vocab. What is the plus probably doing here? Yeah, so it's concatenating or joining together the thing on the left with the thing on the right, and we actually had that vernacular in the world of Scratch. We had the joint puzzle piece that joins hello, comma space and the value inside of answer. A plus in Python can do exactly the same thing. So it's a little more user friendly. Having to anticipate, let's put the placeholder here and then come back later and plug in the variable. Humans over time just realize that it's a lot easier to sort of do this in this way than bother with placeholders, though you can still use placeholders for other purposes. Another subtle difference between the C and Python version of these two lines. More subtle Than that What's missing? Yeah, I'm back. Uh, so the backslash N is again gone for Python, so that sort of happens for free, indeed, and one more difference. Yeah, we don't need to declare the type of answer. Recall that if you rewind in the C version, you needed to tell the compiler that this is a string, and last week we could have changed string to char array, but we still had to tell the compiler what data type we're putting into that variable. In Python we can now get rid of that data type, and Python will just figure it out from context. If get_string returns a string, well then obviously the variable should store a string. If a function returns an int, well then obviously the variable should store an int, and the length. It's just doing more of that decision making for you just to save you time and save you thought. There's a subtlety here though where we can make this program a little bit different. In fact, let's whip it up first in Python. Let me go back to VS code here, clear my terminal, and let's go ahead and create a program that again called Hello.py. That'll open up my previous version thereof. And just so we can see these things side by side, I'm going to drag that tab over to the right of VS code and let go. And now you can see the C version still on the left and the Python. Version at the right. What I'm going to do here now in my Python version is change it to be quite like the version in C now at left. So as promised, I'm going to do from CS 50 import get_string. Then below that I'm going to say simply answer equals get_string, quote unquote, what's your name? question mark space no semicolon, but then on the next line, what I'm doing, but uh parenthesis, then on the next line I'm going to do print, quote unquote. space close quote plus answer. Down here I'm going to go ahead and run Python if Hello.py again, no compilation step. I'm just going to interpret it line by line. What's my name David? And it seems now to work exactly the same. Now it turns out in Python there's even more ways to solve problems like this, even trivial problems like this. So here we're using the plus sign, not as addition per se, but as the

concatenation operator, the joint operation. If you want though, you can take it. Advantage of the fact that print in Python can take more than one argument. It can take 2 or 3 or 4 or even 0 by simply changing the plus to a comma, getting rid of that seemingly superfluous space, and just give print two things to print because it turns out per the documentation of print, which we'll eventually see, it knows that if it takes +12 arguments by default, separate them for you by a single space, and that's something we can override as well. Which one is better? Like, uh, like I don't know, like they're sort of equivalent. It's such a trivial difference, but it speaks to the flexibility that you'll start to have whereby the language is a little less rigid than C was certainly when it comes to printing strings. So in fact, if I go back to VS code here and I go ahead and change that plus to a comma and get rid of the space inside of the quotes, I can rerun Python of hello.ie, type in my name, and we see exactly the same results there. But we can take this one step further, even though it's going to look a little cryptic. This is sort of the more pythonic way to do things, and that too is actually a term of art. To do something pythonically is to do it the way that most Python programmers would do it. It's not the only way. It's not necessarily the right way, but it's sort of the recommended way in the community. So here we have that latest version where I'm passing two arguments to print. The first is quote unquote hello, comma, and then the second of which is the value of answer. I could similarly write this same program. With this crazy syntax, it takes a little getting used to, but it turns out it's actually kind of nice overall. What's obviously different? Well, one, there's these weird curly braces are back. They're not part of the logic of the program. They're literally inside of the double quotes, but you can probably guess how this, what this does for me because there's one other crucial difference. What else has changed between before and after? Yeah, there's this weird F which is not part of print F. It's actually inside of the parenthesis and next to the double quotes. And even this one this came out was a little weird looking to people, but this is how you get this thing to be a formatted string, AKA an F string, as opposed to it being just a literal string of text. Now you can probably guess what it means to put the variable's name inside of the curly braces. It means the value of that variable is going to be substituted right there, similar in spirit to the percentage. S in C, but a little more explicit with the percents, you had to remember that that percents corresponds to this variable's value or something like that, which was just annoying if anything else, if anything, but this time you have a placeholder in curly braces that just says what you want there, that particular value, and what this means more technically is that the answer variable will be interpolated by the interpreter, which means its value will be plugged in right there. So let's try this. Let me go back over to VS code. And quite simply, on my last line of code here, let's change the input to print to be quote unquote, hello, comma, and then Curly brace, answer. Then close curly brace, close quote, and I've done this, this is intentional, but let's see. Let me go ahead and rerun Python of hello.ie DAVID. What are we about to see? Hello comma. Answer, so this is a bug, but just to demonstrate like what is going on and what's therefore missing, what, what did I forget? Yeah. Yeah, I didn't declare that this is a so-called F string or format string. The fix for this weirdly is just to put an F right there. And now if I rerun Python of hello.ie, type in my name again, cross my fingers, now I see that the variable has indeed been interpolated and its value plugged in where I wanted it. All right, turns out we can take off one of these training wheels already. I propose that Get string just exists in the library just to smooth the transition, but honestly it's not really doing anything all that interesting. So let's take this first training wheel off. It turns out that Python comes with a function appropriately named input such

that if you want to get input from the human via their keyboard, you can just use the input function. So we can already for this program get rid of the CS 50. Library because input essentially behaves just like the get string function. So if I go back to my Python version here, I can change get get string to input, and I can even go and delete this training wheel up there, rerun Python hello. ie in my terminal, DAVID enter, and we're still in business as well. So input is generally going to be the way you go about getting input now from the user. All right, let me pause here and see if there's any questions as we try to bridge these two worlds from C to Python. Yeah. uh, so in Python, we don't need a name. Good question. In Python, why don't we need the main function anymore because clearly that's been omnipresent in like every program we've written thus far, and here we have it in all of our Python programs thus far absent. It turns out that humans realize it's just so common that you want the file you're editing to be the main part of your program. Like, why bother adding the additional syntax of saying in main void or something analogous? It's just easier if you want to write two lines of code to get some work done. Why do you have to waste my time adding all of these this boilerplate code which we've been doing. Up until now, now that said, we're going to bring back Maine in a little bit because it will solve a problem, but generally speaking, what I'm doing here is indeed a program, but people in the real world would also call these scripts where a script is like a lightweight program that pretty much just reads top to bottom, left to right. It might be fairly lightweight. It's really synonymous with writing a program, but this is again one of the appeals of a language like Python. You can just get right in and get out and get the job done. Even Java has moved to this in recent years where you don't have to put everything in a class, uh, public static void main for those familiar, you can just write, uh, system.out.print line and get some work done. Yeah. Good question. Is input only for a string? Yes, right now, it will get input from the user via their keyboard and you'll get back a string just like get string, and we'll come back to why that's maybe not a good thing. All right, so what's more what we want to do at this point? Well, let's tease apart some differences now with C. So up until now, every argument we've ever passed into a function in C, and scratch for that matter is a so-called positional parameter, and a parameter is the same thing as an argument, but generally when you're looking at the function from the functions perspective, it's a parameter that it accepts, but when you're Calling the function and passing in an input, you call it typically an argument, but they refer to essentially the same thing, and all of the parameters we've been passing into functions thus far have been positional in the sense that the order matters. The first thing, then the second thing, then the third thing, and so forth. For instance, with print F, the first thing has to be the quoted string, maybe with the placeholder, and then if there's another argument after the comma, that can be the 2nd argument, the 3rd argument, and. So forth, but it turns out Python additionally supports what are called named parameters whereby you don't have to rely only on the order in which you're enumerating the arguments to a function, and that's helpful because some functions, especially in the real world, when you start using other people's libraries that have lots of functionality, they might not take just one or two arguments. They might take 4 arguments, 10 arguments, maybe even more, and it can just be. Wielydy to have to remember the precise order of all those arguments, you're just asking for trouble if you're going to screw up or a colleague is going to get the order out of out of whack. So with name parameters you can actually be explicit with Python and tell it what argument you are trying to pass in by giving it an actual name. So let me go over to BS code here and propose that we use this for really the simplest of programs in

order to Override that default new line that we seem to be getting for free just by calling print. In other words, let me go ahead here and clear my terminal window. Let me close hello.c and focus only on hello.ie for just a moment and let's make it much simpler, like the very first version, and just print out using Python's print function, not print F, quote unquote hello. close quote. And now here I'm going to do Python hello.ie. Enter and we still see that the cursor moves to the next line, the dollar sign moves to the next line, because I'm automatically getting a new line. Well, what if you don't want that? How can you override that behavior? Well, you can actually use a named parameter in Python, and I can go up here and add a second argument. That if it were just something like this that would literally print out the word this because it's just another string, but if I give it a name like end equals quote unquote, I can override the default behavior of the Python print function by changing the value of its end parameter to be the so-called empty string, quote unquote, which means literally there's nothing there. Watch what happens now if I run Python of hello.ie and hit enter, the dollar sign is weirdly and sort of in the ugly way on the same line just like it was when I made the mistake in C in week one of omitting the backslash N. That is to say, what the default value of this end parameter really is is quote unquote backslashn, and I can make it explicit by changing my code as such. I'm going to go ahead and rerun Python hello.ie, and now the cursor back on the next line. And not that this is that useful other than overriding that default, but you could do fun things like, if you really want print to be excited to print some. Things for you. And if I now run Python of hello.ie a third time, now you see that it's ending with exclamation looks a little stupid with the dollar sign, so you could even toss in a new line there, run it yet again, and now we sort of get both of those there. But I would say the common case is to use that end named parameter simply to override it. So how do you learn more about these kinds of things? Well, if you go to The official documentation for Python, which is a thing more so than with C, like if you want to learn more about Python and the functions it offers and the arguments it takes, you go to the official documentation docs.python.org. This is essentially analogous to the so-called manual pages or man pages that CS50 has a version of, but there is no one de facto source for those man pages. Several different versions of them exist in the wild, whereas Python itself. A community maintains its own official documentation. So for instance, if you go to a specific URL like this ending in functions.html, you'll see an exhaustive list of all of the functions that come with Python besides just the print function, and we'll see a bunch of more today. If specifically you scroll down to the print documentation, you'll see something that's a little arcane that looks like this, but this is represented. of a Python prototype, if you will, also called a signature that just tells you the name of a function and then how many and what type of arguments it takes. So how to read this? Well, the print function takes some number of objects. So in Python specifically, this syntax of star objects just means zero or more objects, whatever that is, like a number or a string or something else. The stuff you want to. Print out after that, if you start using named parameters, you can specify what the default separator is the separator between arguments to print. So recall that when I did quote unquote hello, quote unquote or quote unquote hello, comma answer it was separated automatically for us by a single space even without my hitting the space bar inside of my quotes. That's because. As the default value here is in fact a single space. The default value for and as promised is indeed backslashn. And then there's some other stuff related to file IO that print can also deal with, but more on that perhaps another time. There's one curiosity here in Python, it turns out that you can use double quotes or single

quotes around strings, where in C it was much more regimented. Double quotes are for strings and single quotes are for. Chars characters only single characters. It doesn't matter in Python which one you use so long as you're consistent and stylistically you should really pick one and go with it. And the only time you should really alternate between the two is maybe if you want to put like an apostrophe for some human's name inside of double quote inside of single quotes or something like that, but generally you have a little more flexibility. in Python and you'll see in different languages Python community tends to use single quotes at least in the documentation. The JavaScript world tends to use single quotes. Um, we in CS 50 often use double quotes just for consistency with what we do and see, but any, uh, community or company would typically have its own style guide that dictates which one you should use if only for consistency.

Questions then on this here print function. As just representative of all of the docks that you'll see. All right, well, let's take a quick look at variables. We've used these a few times already, but let's focus in a little more detail on what's actually different in Scratch, if you wanted to create a variable called counter and set it equal to 0, you would use this orange puzzle piece here. In C, you would do something like this the type of the variable, the name of the variable, and then set it equal to the initial value semicolon. In Python, it's going to be a little similar, but you can probably guess where we're going with this. How is this line of code probably about to change? Yeah, Good, we're not going to bother with in or the data type more generally, we're just gonna say counter because obviously like a smart interpreter can just figure it out from context that you're putting a 0 in there. It's obviously an integer, and what else is about to go away? The semicolon. So this is the C version and voila, this now is the Python version and this is as silly as this example is, it's kind of representative of how languages like Python just tend to be a little more programmer friendly because you just type less and get the same work done. All right, so if we wanted to do something now and scratch like. Increment the counter by 1. You would use this puzzle piece here. In C we could do something like this in Python it's going to be almost exactly the same, except of course no semicolon. In C we could alternatively do this, and you can also do this in Python. In C though, You could also do what other technique? Plus plus, I'm sorry, but Python has taken that away from us. So if you got into the habit of using plus plus or minus minus, that's great. Use them and see all you want in Python, they just don't exist, so you'll see this more commonly instead as the heuristic. All right, what about the various types that exist in Python? Because even though you don't have to specify the types when declaring your variables, they do in fact actually exist underneath the hood, and it's worth knowing a little something about them because Not knowing will lead to some form of bug. So in C we had types like this bull, char, double, float in long, and string, the last of which was thanks to the CS 50 library. But last week we would have started calling a string chartar instead, which it still is a data type, the address of some char. In Python, we're going to whittle this list down to a subset of those essentially whereby we still have bulls, we still have floats, we still have ints, and we do have strings, but they're literally. Called ss STR so it's not a CS 50 thing. The Python community calls strings STRs, but absent from this list is any mention of star, not to mention chartar, there are no pointers in Python, and indeed as powerful as I'd hope you found weeks 4 and 5 to be, I dare say you also found them incredibly frustrating and challenging and want to yield bugs in your code because with that. Power of memory management comes a whole slew of potential mistakes that you can make, and that's true not just for CS 50 students but for programmers, adult programmers, full time programmers around the world. And so among the other features

of languages like Python is they try to take away certain features of languages like C that were just too dangerous in the first place. It might be wonderfully powerful. It might help you solve problems more quickly, more precisely, but if they tend to do more damage than they're worth, some. it's worth just abstracting those details away. Similarly, Java has references, as some of you might know, but does not have pointers per se. You can't go poking around arbitrary locations in memory in the same way that you can with C. So let's take some of these data types out for a spin and see what's the same and what's different. Let me go back to VS code here and let me propose that we bring back one of our old calculators from a while back. So let me clear my terminal, close hello.ie. And let me go ahead and open up a version of this program that I brought in advance, which was our calculator version 0 from back then. So just to remind you, one of the first versions of our calculator had the CS 50 library as well as the standard IO library, and then we simply got an IT using GetIT in week 1. We got another in in week one using GetIT, and then we simply performed some additions. It was a very Trivial calculator that we did very early on just to demonstrate some of the operators and syntax of C. Well, let's go ahead and try converting this to Python by creating our own program, calculator.i. So in my terminal window I'm going to write code of calculator.i. It's going to open another tab which I'm just going to drag over to the right just so we can see both side by side. I won't bother with, say, well, Let's do it for parity here. Let me copy the C code into the Python file, even though this will not work in the same way. But let's keep what we need and get rid of what we don't. So instead of the slash slash for comments in Python, it turns out the convention is to use a single hash symbol like this. So it's a minor difference. It's half as many keystrokes, so that's nice, but we're not going to include anything like this. But we are going to do from CS 50. Let's import a function that I promised would exist called get in, but we'll soon get rid of that training wheel as well. We don't need main or this curly brace. We don't need this curly brace, and we don't need all of this indentation as a result. So I'm going to move all of that over to the left. I'm going to fix all of the comments to be Python comments by changing the slash slash to hash symbols. And now I'm going to change each of these three lines of code, as you might expect to the Python version. So you probably can guess already. We can get rid of the in there and the in there. We can get rid of the semicolon here and the semicolon here. We can get rid of the F in print F here and we can get rid of the semicolon here. And there's a few different ways we could do this, but I dare say the simplest is going to be to get rid of the format code altogether in that first argument and just tell Python to print X plus Y. So there's a few different ways we can do this, but that's probably the most literal translation of the program at left. To the program at right. Let's reopen the terminal window and run Python of calculator.ie and hit enter. Let's do something like X is 1, Y is 2, and hopefully, we do in fact get 3. All right, so that's all fine and good, but let's take off one of our training wheels now. So let me get rid of our C version here and focus just for the moment on Python. Let's take away this C code and what was the function we can use to get user input. Yeah, it was called a little louder. It's just called input. So let's get rid of CS 50's get in already and use input instead. All right, so this program is much simpler already. So let's go ahead and reopen the terminal window, run Python of calculator.pi, do one again for X, 2 again for Y, and of course, 1 + 2 equals 12. So what's going on here? Cause clearly this is a step backwards, yeah. Yeah, so in the context of strings, plus represents concatenation, the joining of two arguments on the left and the right. Here that seems to be what's happening because it's not 12 per se, it's more literally +12 concatenated together, but why is that? Well, apparently the

input function indeed. Returns a string that is those are the key strokes that came back from the user might look like numbers in Arabic numerals to us 1 and 2, but it's being treated as a string. More technically like underneath the hood, there is some char star stuff going on there, even though we're not using that same terminology. So intuitively, what's going to be the solution without just reverting to using the training wheel that is the get in function from CS 50? Put another way, how did CS 50 probably implement get in, might you think? Yeah. Yeah, so we recall that in C. We could cast some data types to other data types, typically ends to chars or charges to ends. It's not quite as simple as casting in this case, because underneath the hood, thanks to our knowledge of C, there's a bunch of stuff going on. There's probably a 1 and there's a null character. There's a 2, and there's a null character, so Not quite as literal as a char to an end or an end to a char, so we're going to more properly convert the H string or the stirr to an end. We're not casting, but converting, and converting just implies that there's a little more work that has to be done. But thankfully Python can do this for us. In fact, let me go up to line 4 here and say, A pass the, well, actually let's do it in this a couple ways. Let's first convert the X value to an integer. Let's convert the Y value to an integer as well. So funny enough, it's very similar syntactically to casting, but in C, when you cast something, you actually wrote the data type in parentheses. Now the data type itself is a function that takes an argument which is the stir or string that you want to convert. So let me go back to my terminal, do Python of calculator. ie, enter, type in 1, type in 2, and now I get back my 3 answer. Now as you might imagine, just like in C, we can kind of play around with where we're performing some of these operations, and this looks, you know, arguably a little less obvious now as to what is being added. So I really like the simplicity of $X + Y$. It just does what it says. So I could convert these in other ways. I could say after line 4, you know what, recharge. X to be the in version of X, but generally speaking, that's kind of wasting a line of code by just doing something you could do on a single line. So let me delete that and instead just say that well if I know the return value of the input function is a stir, let's just pass that output as the input to the end function and it'd be a little more pythonic. So to speak, to just pass the input functions output as the input to in, which is really hard to say, but we've done this in C, just nesting function calls like this. All right, so if I run this one more time, Pythonocalculator.pi type in 1, type in 2, we're back now in business. Now what I won't trip over just yet is a subtlety that Whereby I'm deliberately typing in actual numbers like 1 and 2, but if you are following along at home or on your laptop, if you were to type in cat and dog, like bad things will happen, but we'll come back to that before long. Our questions though on any of this conversion of our strings to our. Integers in this case. All right, well, what more does Python offer to us? Well, in addition to these data types, there's actually going to be a bunch of others, a few of which we'll actually use today. In fact, we'll see ranges of numbers that's like that's a thing built into Python. We'll see lists of numbers which is going to be like a new and improved version of an array that solves like all of last week's problems when we talked about the downsides of using arrays. There's going to be tups for things like X Y. Coordinates or GPS coordinates or anything where you have collections of values. There's going to be dictionaries whereby you can have key value pairs provided to you without having to write a whole hash table yourself and you can have sets which you can use to just contain unique sets of values that you just want to check for membership and there's bunches of other data types as well. And this is where languages like Python start to get really powerful because all of the Data structures we talked about in C we really only got from the language itself in array. Everything

else we had to build, or at least talk about building in class, these now and more come with the language. Meanwhile, in the CS 50 library for Python, just so you know, there are a whole bunch of functions. These were the C versions in Python it stands to. And that we don't need as many because there's fewer data types in Python, but `get flow`, `get in` and `get string` do all exist in the CS 50 library for Python. You're welcome and encouraged to use it because indeed among the goals for problems set 6 are going to be to redo some of your C problem set problems in Python where you can look at your own C code and hopefully. Uh, you like that solution and figure out how to convert it line by line essentially to the corresponding Python version, but clearly we've seen ways of taking these training wheels off quite quickly as well. And in fact, if you wanted to import all three of those functions for a larger program, you could do this just following the approach that I took so already, but you can also just separate them by `com.` like this or it turns out you can also import the whole CS 50 library as you'll see in some code and then just access the functions within with slightly different syntax as well. All right, how about another construct from scratch and from C now in fact in Python. So in Scratch, if we wanted to do a comparison like is `X` less than `Y` where each of those are variables, then say. Much here in C it looked like this and nicely enough you can probably guess already which what's going to change here. Like the `F` is about to go away, the `backslash n` is about to go away, the semicolon is about to go away, but some other stuff's about to go away as well. Focus your attention on the syntax like parentheses and curly braces because in Python it's just that. So we got rid of the parentheses because they didn't really add all that much logically. We got rid of the curly braces, which technically we could do in C anytime there's a single line of code inside of a conditional, but for consistency stylistically we always use them as well. Python though does not have you use any of those curly braces at all, but Python requires that you indent your code. Properly. So if you've ever been among those who are writing out your program and like everything is just crazily like left aligned and just a big mess until Style 50 swoops in and cleans it up for you, you're not going to be able to write Python code like that anymore. That's been such a societal problem among programmers, newbies and professionals alike, that the language itself requires logically that if you want this line of code to execute if this boolean. Expression is true, you've got to indent this line by convention 4 spaces. You can't be lazy and leave it all left aligned and sort of fix it up later. This has made Python code arguably more readable because of these language-based requirements. Meanwhile, let's look at a `ifE` construct in Scratch, which looked a little something like this and see, it looked like this, which is kind of a lot of lines just to express a simple idea. All of those same things. Going to go away whereby in Python it looks like this instead and the only other difference worth calling out is that because you don't have the curly braces, you do have a colon which precedes the subsequent indentation as well. Meanwhile, if we've got an `if if els` in scratch in C, of course it looked like this. A lot of this is going to go away in the flash of a screen, but there's going to be a curiosity, which is not in fact a typo. Notice what happens with the `LSIF`. It's abbreviated `L if`, and honestly to this day, all these years later, I can never remember if it's `L if` or `LSI` because different languages use different shorthand spellings of this phrase. It's `LF` in Python, because that's maybe the most succinct you can make the two words themselves, but everything else is effectively the same, including the additional colon this time. OK, questions on. Any of those conditionals and syntax, yeah. What a good question. What language did they code Python in? The interpreter we are using within VS Code is itself written in C, AKA C Python. However, you can implement a Python interpreter

really in any language, including machine code like raw zeros and ones if you have that much free time in assembly. Language which we saw briefly weeks ago. You could write an interpreter for Python in Python if you really want to be meta about it or in C++ or in Java. This is the thing about programming languages you can use any language to create a compiler for or interpreter for another language. What's going to vary is just how easy or difficult it is and how much time it therefore takes you. Good question. Other questions on any of these here features. All right, well, let's do something a little bit different in Python via VC by opening up maybe a comparison program that we looked at some time ago. So let me go back to VS code here. I'm going to close my calculator and I'm going to open up now from my distribution code today a version of our comparison program from a while back which was essentially the Uh, version 30 index thereof. So this one has comments, which the very first one in week 1 did not, but notice as a refresher what this comparison program was doing. It was including CS 50.H and standard.O.H. It was prompting the user for two integers via get in X and Y. It was then doing a very simple comparison comparing X against Y to determine if it's less than, greater than, or the same as. X and uh the same or equal to the same. So, just so that we can go through the motions of converting one of these to the other, let's do that side by side. Let me code a program called Compare.i. Let me close my terminal, drag the Python version over to the right here. And without comments this time, let's just do from CS 50 import get in. Then below that let's do x equals get in and ask the user for what's X question mark. Then let's ask the user for Y using get in quote unquote what's why mark. Then below that let's do if x less than Y call in. Go ahead and print quote unquote X is less than Y L if X greater than Y, go ahead and print quote unquote X is greater than Y else coin, let's go ahead and print out quote unquote X is equal toy. So I dare say these are now equivalents. It's clearly fewer lines because a lot of the lines that left were admittedly comments, but also some curly braces and there's more syntax like parentheses that we got rid of too. Let me open my terminal window. Let me run Python of compare.i. We'll type in 1 and 21 is less than uh X is less than Y. Let's do it again using 2 and 1. X is greater than Y. Let's do it one last time, 1 and 1, and of course those two now are equal to. Each other. all right, but why go down this road again because that was kind of a simple exercise but recall that we introduced this comparison of ants because it was so sort of stupidly simple, even if the syntax of that week was completely new but we ran into an issue pretty fast when we started comparing. Strings and that was a problem we really only fixed in week 4 when we finally revealed what a string actually is. If we focus a bit more on Python strings, it turns out that we can solve that problem much more easily in the world of Python. In fact, let me go back to VS code here. Let me close these two versions of in comparison. Let me open up at left a version of my program. That I brought with me here that contains a version from week 2 wherein we finally revealed that a string is just a char star but recalled that the solution in week 4 as well as in week 1 when we first encountered this problem was to use stom, a function that whose purpose in life is to compare two strings character by character by character using a 4 loop or something like that. But they have knowledge, therefore, of how to navigate pointers, how to look for the null character, the backslash zero at the end, and all of that came from our friend string.h. Well, how can we go about implementing the same idea in Python? Well, let's open up VS codes, Terminal window, open up a new program called Compare.ie, but this time, let's get rid of the integer version there. Of let's get 2 ins from the user, and I won't even use any CS 50 training wheels. Let's just use the input function to get S and ask the user for a value of S. So S

colon close quote with a space, T equals input ask the user for a variable T. And then let's just ask the question if S equals equals t, then print out quote unquote same. Else, go ahead and print out quote unquote different. Let me move these side by side, just so you can see the difference. Notice how much code we have to write and how much we needed to understand in order to compare something as trivial as two strings and see. But in Python we're literally just using equals equals and let's see if it actually works. So Python of compare.pi enter. Let's type in maybe cat for S and dog for T, and those are in fact different, but we would have gotten the same answer in C. Let's rerun Pythonfcompaed.pi and type in cat, type in cat again. And now it's detecting them the same. So wonderfully, Python has solved that seemingly annoying problem of not taking us literally like don't compare the pointer against the pointer. Compare what a reasonable programmer probably really cares about the values of those strings. So the equal equals is doing all of the 4 loop or the Y loop, iterating over those things character by character and actually giving us the answer we want. So what else gets easier in Python? Well let's focus a bit more on these strings. Let me go back into VS code here. Let me close out our two comparison programs and clear my terminal, and let me go ahead and open up a prior program that we wrote, that one called Agree.c, and namely in the staff version of the code online, this was agree2.c, which is where we left it. Now recall in this C program that we did the following. We first, using CS50's GetHA function, prompted the user for a char, hopefully Y or N for yes or no, respectively. And then we used a Boolean expression and actually the combination of two using the two vertical bars to ask whether the inputted character is capital Y or the inputted character is lowercase y. And if so, we went ahead and printed out that the user agreed. Otherwise if they type in anything else for that character, we simply print it out, not agreed. Well, how can we go about implementing that same program in Python, for instance, in a file called agree.py? Well, let me go ahead. And open up my terminal window again. Let's create a file called agree.py. As before, let me go ahead and drag it over to the right so we can see these two things side by side, and let me go ahead and do this. I'm going to set a variable, say, called S equal to the return value of input, quote unquote. Do you agree there by asking the user the same question as before. No need to use the CS 50 library because the input function here suffices. And instead of using C, I Deliberately using S because it turns out in Python there is no way to get a single character per se, but you can get a string that has a single character. Indeed, char is not a data type in Python, but once we have this input from the user, let's now go ahead and implement a conditional using one or more Boolean expressions. Well, let's ask if S equals equals quote unquote Y or S equals equals. Or Casey, then let's go ahead and print out as before, quote unquote agreed. And now notice what's different this time. I'm literally using the word or instead of the two vertical bars because in the spirit of Python things tend to be a little more English like, a little more readable, top to bottom, left to right, and indeed or hits that nail on the head. Otherwise, if it is not in capital Y or a lowercase y, let's go ahead and print out, quote unquote not agreed. And that's it for converting this program from C here into Python. But of course this isn't the most robust version of the program because it would be nice if the user could type in something like yes, YES capitalize maybe in different ways. So how might we go about implementing that? Well, we could do this in a few ways. I could, of course, and let's go ahead and get rid of my C version now and focus just on the Python. I could do something like this and just start oring together more possibilities like or S equals. Quote unquote yes or S equals equals quote unquote yes very emphatically or and so forth, but you can imagine that

this doesn't scale very well. If I want to consider all the possible permutations, maybe of the caps lock key being up or down, that's quite a few possibilities to enumerate. So perhaps we could do this a little bit differently. And in fact we can by maybe storing all of the possibilities in a so-called list. So whereas C had of course arrays, Python has what are called lists which. Effectively underneath the hood are indeed linked lists as we explored in week 5. Now a linked list of course can dynamically grow and even shrink, and that's indeed what Python does for us. I can simply create a list of values from the get-go, or as we'll eventually see, I can add things to it, remove things from it, and all of the underlying memory gets managed for me. And in fact with lists we get a whole bunch of features that can make this possible. But for now, let's use them simply as statically initialized list with values I know from the get-go that I want and I'm gonna go. And do this in VS code. I'm going to delete most of this Boolean expression, the combination of all of those phrases, and I'm going to simply say if S is in using a Python keyword in, literally the following list of values unquote y, yes. And for now I'm going to use just those two, but let's see how it works. Let me open up my terminal window again. Let me run Pythonore.ie really for the first time, but let me claim that it would have worked even in the previous version. Enter. I'm going to go ahead and type in lowercase y, and I've agreed. I'm gonna go ahead and run it again and type in lowercase n, and I've not agreed. I'm gonna go ahead and run it again and I'm going to type in all caps, yes, because I really agree, and yet I don't because there is a bug still in this version. So even though up here. In my Python implementation, I do have a list of values that I'm looking for. Python's gonna look literally for those values, so lowercase y and lower cases. So how can I go about tolerating different capitalizations by the user? Well, I can do this in a few different ways. I could, for instance, after getting the user's input in a variable called S, I could updates to be S.lower, which is going to have the effect of lower casing the word for me and then updating the value itself of S. And now I think this will work. Even for an uppercase version, let me go ahead and run Pythonore.ie emphatically type in yes enter, and yet this time I've agreed because I forced the user's input to lowercase and then I have compared against the canonical forms I've written which are all lowercase. I could have done the opposite. I could have forced the user's input to uppercase and then enumerated in my Python list in between those square brackets, capital Y and YES, but either approach here is fine. Now technically I don't need this additional line here. I can go ahead and delete. that line wherein I lowercased it and in Python I can actually chain some of these function calls together by saying input.lower so that the return value of input ultimately gets forced to lowercase by using dot lower here. Alternatively still, I could just lowercase the very at the very moment I'm actually comparing it, and down here I could do s.lower and then compare the lower case version of what's going on toy or yes. Now what's really this all about? Well, this is actually an example. Of what's generally known as object oriented programming or OOP for short, whereby in Python and a lot of other languages now you can have variables and data types more generally that have not only values associated with them like why or yes, but also functionality built in. In other words, whereas in C we would have used a function from like the C type library called 2 upper or 2 lower, and we would have passed as an argument to those functions the very character that we wanted to force to uppercase. Or to lower case well in Python and indeed object oriented programming languages in general, the developers behind the language recognize that sometimes there's functionality that's inherently related to the values in question. Indeed, when we're dealing with strings, it's pretty reasonable to want to sometimes uppercase them or lowercase them,

capitalize them or do any number of other things and so built into the string type in Python is in fact the lower function itself, as well as a whole bunch of others. In fact, at this URL here, can you see the document. for all of the string functions built into Python and more technically when a function is built into a data type and you access it via this dot notation instead of by calling some global function and passing an argument into it, you are using what are called methods. So methods are simply functions that are inside of objects and in this case the object in question itself is a string. So what's really happening with this here example when I'm checking whether the user has agreed or not is I'm taking that value that's. Strings, which is technically now an object in memory, and inside of that object is not only the user's input but some built in functionality otherwise known now as methods, and those methods were written by the same people who invented the string data type itself. So this is just the first of these examples, but we'll see yet others. But notice the syntax is actually quite similar to C, just isn't C when you wanted to go inside of a structure. You can similarly go inside of an object in Python and access not just the values ultimately, but also these built in methods. All right, how about another comparison of C to Python again involving strings? Well, let me go ahead and reopen and clear my terminal and close out of agree.ie. Let me go ahead and open up a version of copying strings from a couple of weeks back whereby we finally started solving it correctly by doing some proper memory management. So here in the staff version of Copy 5.c we have not only a commented version of what we did a couple weeks back, but we also have a reminder of how what was involved in. Copying strings and C. Recall for instance that we prompted the user in this example using CS 50's get_string function for a string that they wanted to make a copy of, and then we did some error checking ultimately to make sure that there was enough memory and nothing went wrong. Then recall that the right solution to this problem in C was not to just use the assignment operator and assume that S can be copied into T, but rather to allocate using malloc enough memory for the copy plus one more byte for the null character again. Making sure that all is well by checking the return value of that and then actually copying character by character by character the characters from S into the chunk of memory now known as T or ultimately recall we used the built in strcpy function which does all of that looping for us. And then when it came time to capitalize just the copy, we did a quick sanity check. Is the length of T greater than 0, otherwise there's nothing to capitalize? And if so, go ahead and use the C type libraries to upper function. Passing as input that specific character T[0] and then updating T[0] itself. So here's an example of procedural programming in contrast with object oriented programming again, I'm passing the argument to be uh uppercase into the two upper function as opposed to simply going to that character and asking it via some dot operator to, for instance, uppercase itself. Now I went ahead in the C version and printed out the two strings. I freed up my copy of memory that I myself had allocated, and that was it for this program. So it was a decent amount of work we're calling C to actually go about just copying a string. Well, as with so many things in Python, it's going to be so much easier. Let me go ahead and do this. Let me open my terminal window. Let me create a file called copy.ie. Let me move it over to the right hand side so we can see them side by side, closing my terminal window, and let's do roughly the same. Let's create a variable called S, set it. Equal to on the right hand side, the return value of Python's own input function because we don't really need CS 50's own get_string function and ask the user for S. Then let's go ahead and create a second variable called T, set it equal to literally S.capitalize, whose purpose in life, if we read Python's documentation for string

methods, will be to uppercase the first letter of the word that the user has presumably just typed in. Then I'm going to go ahead and print out as before. The user's input, and I can do this in a couple of different ways, but I'm going to use one of our format strings and say S colon and then interpolate that variables by using my curly braces to say put the value of S here. Then I'm going to go ahead and print out T by saying T colon, interpolate its value here inside of quotes, close parenthesis. So let's see if this works. Let me go ahead now and run Python of copy.ie. I'm going to go ahead and type in, say cat and all. Case and hit enter and now notices remains in all lower case, but the copy indeed has been capitalized alone. All right, well let's take a look at one other example involving strings between C and Python equivalents. Uh, let me go ahead and remind us that a few weeks back too, we created this uppercase program whose purpose in life was to prompt the user using Get string for a string, saying, Here's the before string, then it prints out after. Because the purpose in life of this program was to uppercase all of the characters in the string, not just capitalize the first one. So as you might expect, we used a loop a few weeks back and we iterated from zero on up to the length of the string using plus plus to increment I in each iteration, and then each time we went ahead and printed out one character at a time. So strictly speaking, we didn't change the string from lower case perhaps to uppercase, we just changed each letter uppercase and print. It out right away. Well, how might we do something similar in Python? Well, here too, we have a couple of different approaches. Let me go ahead and open up my terminal now, run a code of, say, uppercase. i, close my terminal window, and let's drag this to the right so we can see them side by side, and let's do roughly the same. Let me create a variable this time called before, set that equal to the return value of input and just prompt the user for that before string. Then after that, let's go ahead and print out preemptively after colon space space just to align everything nicely, but let me not print a new line yet because I want to go ahead and see uh the following string on that same line. And then let's go ahead and do this analogously to the C version first, but then tighten things up. Here's how we can iterate in Python over every character in a string. I don't need to bother. I and indexing into the string or anything like that, I can using a Python 4 loop, simply say for each character C in that string called before, go ahead and print out the uppercase version of that character, but don't yet print out a new line. But at the very end of this loop, go ahead and print out nothing but a new line. Let me go ahead and open my terminal, run. Python of uppercase.ie enter type in cat in all lower case, cross my fingers, and after each and every one of the characters is uppercase. And what's nice about this, if nothing else, is that this 4 loop in Python there on line 3 is pretty elegant, whereby you implicitly get access to each character in the string because that's how Python knows how to iterate over a string object. But it turns out we don't have to do this quite as analogously in Python as we did in see we don't have to do it character by character in so far as Python is object oriented and these strings are objects and those objects have methods. Those methods will actually operate on the entire string at once, unlike the more pedantic work we had to do character by character in C. So in fact, let me go ahead and close the C version here. Uh, clear my terminal and hide it, and let's go ahead and make this quite simpler. Let's get rid of the 4 loop altogether and let's simply and let's get rid of that print statement altogether, leaving only the before variable and getting the user's input. And now let's create an after variable, set it equal to 4. upper, thereby upper casing the entire string called before and setting the return value. To the after variable and then let's go ahead and print using our old friend F string, uh, after colo uh space space, and then interpolate the value of that

after version. So now we're down to just 3 lines at that. Let me go ahead and reopen my terminal python of uppercase. ie enter, type in cat in all lower case, and voila, now I have capitalized the cat all at once. All right, before we take a break for some fruit by the foot, let's go ahead and take a look at Python's implementation of loops further. So in Scratch, recall that we implemented a loop with something like this. If I wanted to meow 3 times on the screen, I would literally use a repeat block. In C, it was a little clunkier to mimic that same idea. Like we could implement a variable called `i` and set it equal to 0. Then we could ask a Boolean expression is `i` less than 3? If so, print me out and then increment `i` using our old. plus friend, which in Python is now gone. In Python we can do this almost the same, except I don't think we need the data type. I don't think we need the semicolon. We don't need the parentheses while still exists. We don't need the curly braces, and we can't use the plus plus. We don't need the `for`. I mean, we're mostly just trimming clutter from this here implementation. So this is the C version. This now is the Python version, a little tighter, a little easier to read. It's pretty much the minimal syntax available to get the job done. So how can we actually have a cat meow in this case? Well, let me go into VS code and I'll stop doing everything side by side and just stipulate that we've done most of these examples previously in C and in my first cat, well, I could certainly do it the easy way and let me go ahead and create cat. ie and like we always started in the past with, I could just do meow and then our old friend copy paste, and this of course was bad for bunches of reasons, but it gets the job done. In Python, if I want to do this, well, I can just borrow. That same inspiration and I could say set `i` equal to 0, then do while `i` is less than 3, colon, then go ahead and print out meow, and then go ahead and do `i` equal or rather `i` plus equals 1 is maybe the most succinct way to express that same idea. All right, just to confirm that this works, python of cat. pi enter meow meow meow. All right, so how else can we do this and how can we do this more pythonically? This is perfectly correct. Many people might implement it this way, but it's not quite as succinct as we could alternatively do in Python, yeah. Yeah, so we could maybe use a `for` loop, and in fact let's let's go there because we don't quite have the same types of `for` loops in Python as we did in C. While loops are essentially the same, but `for` loops are actually a little bit different and actually a little bit better. So let me go into my code here, delete all 4 of these lines, and literally just say 4, `i` in this list of values 0 1 and 2 colo, print, meow. In other words, in `for` loops and Python, you don't have the parentheses, you don't have the two semicolons, you don't have the initialization and the Boolean expression and the update. You just say a little more English like for each `i` in the following list or for each value of `i` in the following list. And what Python will do for us is automatically on the first iteration set `i` equal to 0. On the second iteration set `i` to 1. On the 3rd iteration set `i` to 2, and then there's only 3 things in the list, so that's it. And so just as before with the Y and the yes example where I use square brackets similar to arrays and C, I was using a Python list of strings in that case. Here I'm using a Python list of integers 0 1 and 2, and they're integers in the sense that they have no quotes around them, so they're obviously not strings, and I'm printing out meow this many times. And indeed if I do Python of Cai again. I get meow meow meow. This is correct. This is arguably better, at least in the sense that it's two lines of code instead of 4, and it's arguably more readable as well, but what do you not like about this perhaps, even if you're only seeing it for the first time. Yeah, it's going to be a lot more difficult to do things more than 3 times because we're calling Python and Scratch at least, and in C we had the ability to either express ourselves literally or at least in C we could just change that 3 to any number we want 30, 300, no big deal.

It's a super simple change even though it was kind of annoying to type all of this out. Well, in Python. Yeah, I could do this and say for 1 and 01 and 2 just to mimic the numbers that we'd be setting `i` equal to in the C version. Frankly, this can be any list. It could be 123456, cat, dog, bird, or any three things whatsoever, but I'm just using 01 and 2 for consistency with the way C would have done it. But slightly better than this is to use one of those other data types that was briefly on the screen earlier. We have not just floats and ints and strings and lists and tuples. We also have what are called ranges, and range is not only a data type in Python but more literally a function that you can call to get a range of values from zero on up. So I can change this list of three values to a function call to a function called range, passing how many things I want, and by default. Per the documentation, I'll get back a list of numbers 01, and 2. And nicely, Python's pretty smart about this. It technically doesn't hand you back all of the numbers at once, whether it's 3 or 30 or 300 or 3 million. It sort of hands them back to you one at a time so you're not using more memory just because you're doing more iterations. So now if I do want to iterate 4 times, 5 times, 30 times, 300 times, I again can just change this single value. And if you want to be fancy too, you can skip numbers. You can go. Count all the way through odd numbers or even numbers. You can change the incrementation factor, but the default and the most canonical is indeed just to count up like that. So if I go back to VS code here and improve this, I can change that hard coded list to just range of 3, clear my terminal, run this cat one more time, and now I'm back in business as well. In fact, this is so common, let me throw up one alternative to this. You'll notice that in the previous example, both in VS code and on the screen, um. I am not actually using `i` in any way. In fact, if you look back at how we converted the scratch to Python code, I'm using `i` because when you use a `for` loop in Python, you have to give it a variable in some list or range of values. That's just the way it is. But I'm technically not using or printing `i` anywhere, and that's fine. And so it's arguably Pythonic too if you have a variable out of necessity, but you're not actually going to use it for anything useful, just. Call it an underscore instead. And even though this is weird looking, an underscore is a valid symbol for a variable name in Python, so it is Pythonic to just use this just to signal to yourself later and to colleagues that, yeah, I'm using a variable because I have to, but it's not one I'm actually going to use elsewhere. It's a minor subtlety and not strictly necessary but perhaps commonly done. All right, how about a couple final versions of cats then. So recall that if we wanted to do something in scratch forever, we had a forever block, which literally did that. Well, in C we couldn't quite translate that literally. So the closest approximation was probably this while true, whereby you have a boolean expression that by definition is always true, so the loop is never going to stop, thereby infinite if you wanted to print out. Meow meow meow on the screen ad nauseam. In Python you can do it almost the same, but the curly braces are about to go, the `for` is about to go, the backslash N, the semicolon, and the parentheses. But for whatever reason in C we lowercase true and false. In Python we capitalize true and false. So a minor subtlety, but it's now indeed capital T. But the indentation has to be the same and the colon has to be there as well. So with that we can of course induce intentionally or otherwise some infinite loops. As with C you can break out of them if need be with control C to interrupt the process. But let's just see lastly with this cat how we can make it a little more abstract like the final versions of our cat in scratch and see. So let me propose to open up here in a pro version of CAT that we looked at that we wrote in the past. It was version 12 at the time, which looked a little something like this. This was one of the final versions of our cat in C that simply allowed me in Maine to call meow

function that took an argument, which is the number of times I wanted to meow. This in C is how we implemented that helper function, so to speak, that returned nothing, so its return type was void, but take an integer called N as its input and then there was a 4 loop inside of there that printed meow that many times. So long story short, this was how both in Scratch and in C we invented our own functions. Well, how can we do this now in Python? Well, let me bring this version of CAT over to the right here, delete that previous version, and let me propose that. We do this for I in range of 3. Let's go ahead and assume for the moment that there is a meow function in Scratch whose purpose in life is to just meow on the screen. Well, that of course does not exist. So in Python I'm going to use a trick that allows me to define my own function, and the keyword for this is literally deaf for define. The name of the function and then parentheses if it takes no arguments. You don't need the void keyword even if it takes no inputs. So let's do a simpler version of the cat first that takes no arguments, and then we'll add back that argument. How do, how does a cat meow? It literally just says meow on the screen. So already we seem to be an improvement. I've got like 4 lines of actual code here versus like 20 or so on the left hand side. Let's go ahead and run Pythonoca.ie. Enter and we see the first of our errors, which is remarkable because usually I would have messed up by now. So here we have in Python the equivalent of like a compiler error message. The program has not run. It's tried to run. It's tried to be interpreted, but it encountered some error. These are generally called trace backs in the sense that you. See a trace back in time of everything the program was trying to do just before it failed. So if you call it the function, which called the function, which called the function, you'd see all of those function calls on the screen. I've just tried to call one function, so it's a relatively short error. This is clearly a problem, and here's the type of problem name error. The name meow is not defined. So intuitively, even if you're seeing Python for the first time, why is meow not defined even though it's literally defined. Right there, yeah. Yeah, as smart as Python is vis a visc, still kind of naive in that meow doesn't exist until line 4. So if you try to use it on line 2 too soon. All right, so in C we fix this problem by initially just kind of hacking things together by just, all right, well, let's just define it up here and then move that down there and that's totally reasonable. And in fact if I clear my terminal and rerun Python of Capi, we're Back in business, but I'd argue you can only do that so many times, especially once you've got a bunch of functions. You don't want to relegate like the main part of your program, which really this loop is to the very bottom of the screen, if only because like that's the first thing you care about. I want to see at the top of the screen that's the whole point of putting main at the very top. So what was the solution in C? The solution in C was to put the prototype for the function at the top of the file. That though is not a thing in Python. You don't just copy that first line of code, put it at the top of the file, add a semicolon, and then it works. Instead, the Pythonic way to solve this problem, for better or for worse is to actually put your code in a main function. Main and Python has no special significance in this sense. It's just convention to borrow the name that so many other languages use as the main function in those languages, but you just wrap your function in a function main so that you're defining main, then you're defining meow before you're actually using the meow function per se. But I have made a mistake. If I run Python of cattai now, cross my fingers for good measure, and Now the program does nothing. Why is that? Yeah, why is that? Oh, sorry, go ahead. Yeah, curiously, I never called the main function. So whereas in C and in Java and C++ and a bunch of other languages, Maine is special. Like Ma is the function by definition that is automatically called Python has no such special magic.

It's not going to call Maine for you just because you created it. In fact, I didn't even call that main function Main. It's just a convention, but the solution is exactly that. Well, if the problem is that Maine wasn't called at the bottom of this file, what I can do is just literally call Maine, which we would never have done in C, but this is conventional to do in Python, so that after you've defined Maine up here and then define Mao down here, now you can call Maine, which in turn will call meow, but at that point in the story, both of those functions exist. So if I go down here and run cat.ie again, now I see my meow, meow, meow. Now let me add one final flourish because this version of the code in C recall actually let me specify how many times I want to meow, whereas here I actually have my 4 loop in Maine at the right, and I'm calling meow that many times. Well, what if I want to get rid of this loop over here? And deindent meow here and pass in literally the number 3 here. Well, in Python you can just say inside of the definition of a function that it takes an argument like N. You don't have to specify the data type. Python's smart enough to figure it out. Then in your function you can use that as with 4 i in range of n. Go ahead and print meow. So now the right hand version of this program is pretty much equivalent to the left hand version of this program as always using fewer lines of code. Let me go ahead and run Python of cat up i meow meow meow. We're good. And then let me make one final change, if only because most every documentation you see online or website you. Orals on Python will actually have you not just literally call Maine at the bottom, but you'll do this crazy syntax that is solves a problem that we won't trip over in this class, but typically it's Pythonic to actually call Maine after asking the question if name equals equals quote unquote_ma__ colon Ma. This is a stupid mouthful of code that even I had to think about when I was typing it out if I got all the underscores correct, but long story short. This convention of using a conditional before you call main allows you to write more modular code in Python so that some of your files don't actually do anything other than define, define, define, defined functions that you can then import into other files you write. So in short, this is the right way to do it even though in CS 50 it is unlikely that we are to trip over this bug. Questions now on that last piece of how we define functions in Python, yeah. Ah, good question. Good eye. Why do I have two lines between my functions in Python? As you will see via Style 50, it is Pythonic, that is Python convention, to separate functions in your code by two lines, whereas there is no such convention in C. So I'm trying to be consistent with what the world does, yeah. If you want to count backwards in a loop, can you do that? Absolutely. You could use the range function in a different way. Start start with a much larger value and count down. How but you can alternatively do that with a while loop. I would say that yeah, you can make that work, but you shouldn't. It's just people don't do that unless it does actually solve a problem for you. Other questions on this. All right, well, when we looked at C, recall there was a bunch of things that ultimately like we couldn't do well. We ran into issues of like full loading point precision and integer overflow and truncation and like all of these world's problems. Um, there's still going to be some of those, but first, let's take a fruit by the foot brake and we'll be back in 10. Help yourself to seconds today. All right, so we're back, and let's use our remaining time together to focus not only on some of the problems that Python can solve more readily than C, but also some of the problems that remain. So here was a program early on in our discussion of C that had this weird bug whereby when we implemented a relatively simple calculator to divide two numbers x divide. By we experienced what we called truncation at the time whereby 1 divided by 3 was curiously 0 and like something like 4 divided by 3 was curiously 1 and we were losing everything after the decimal point and this was true

even if we tried using floats because with truncation recall everything. the decimal point with integer math is simply discarded. So if you do in divided by in, you're going to lose what is after the decimal point. So let's take a look in Python at whether this is still actually a problem. So let me go back into BS code here. We'll close out the C version thereof, and let's go ahead and create our own program called Calculator.i. And in this version, let's modify the original which just did some addition and instead have it do some. Division instead. I'll get rid of my outdated comments and perform now division instead of uh addition by doing X divided by Y. Python of calculator.pi, let's try 1 and let's try 3. And oh, our fractions are actually back. So it turns out in Python, even when you're manipulating integers, if you divide one by the other and the result logically should actually be a floating point value, that's what in fact you're going to get back and you don't have to jump through the same hoops that we did before to actually force things to float and then do floating point arithmetic and so forth. In fact, if you Want the old behavior, it's still actually there and you can use two slashes in Python to use the old integer division as opposed to what we're seeing here. But a typical programmer, I dare say nowadays, would want it to behave in exactly the same way. So truncation seems to be less, therefore of an issue for us. All right, well, what other problems did we encounter at the time? We'll recall we had issues of Floating point in precision whereby even when we divided something simple like 1 divided by 3 and in grade school we learned that was like 0.3333333 repeating infinitely many times, we started seeing weird numbers that were not 3 at the end of that value back in the day and see unfortunately that's a problem that's still with us. In fact, if I use this same program here. Let me go into VS code and instead of printing out just X divided by Y, let's go ahead and do this temporarily. Let me give myself a variable called Z and set it equal to x divided by Y only because it'll be a little easier to see the formatting trick I'm going to use. Let's go ahead and print out a format string that prints out Z, and for the moment, let me just claim that this is going to do the exact same thing. It's just completely gratuitous that I'm using an F string now as opposed to just printing out Z, but I do 1 divided by 3, we're still seeing 0.333, but we're only seeing just over 10 or so digits here. What if we want to see like 50 digits and really start poking around at what's being represented? Well, the syntax is a little weird, but in Python using an F string, you can do tricks similar to what we did with the percent F with print F and C. And if after my variables name in this set of curly braces, I do a colon. And then a dot because I want to see numbers after the decimal point and say something arbitrary like show me 50 digits after the decimal point and treat this as a float. This is a crazy incantation I do think of a format string even I am sort of cheating off of the paper in front of me, but this is how you format strings if you want to see them with a little more precision or so I think if I rerun Python of calculator.ie and do 1 divided by 3, darn it, we're still in the same mess that we were before. Now why is this? Well, it's still. The case that I'm running the code on the same kinds of computers that I did before, it's still the case that these computers only have a finite amount of memory. And so even though I'm manipulating clearly floating point values, Python is only allocating, say, 64 bits to those float variables, and so there's only so much precision that's possible. And so what we're seeing is essentially the closest representation to an infinite number of 3s that we can represent using binary, using a floating point representation therein. So still a problem. But I do think in Python you'll find that there's so many more libraries out there, third party software that comes not just with the language itself but from others, whereby you can use libraries for more precise scientific computing that essentially implement their own versions of floating point values so that

you can use not 64 but 128 or more bits than that when it really matters to some level of precision. Thankfully though, one problem is at least solved for us, namely integer overflow. So recall that this was another problem we ran into. Whereby if you try counting higher than say 4 billion or even higher than 2 billion if you're representing negative numbers, which halves the total range that you have available to you in the positive range, we ran into the situation where it somehow wrapped around, became negative, and then even ended up being zero as a result. Well, Python wonderfully nowadays just gives you more and more bits. As needed if your integers are getting larger and larger, so this is a wonderful feature and that we've at least addressed one fundamental limitation we ran into in C and this time the language itself provides us a solution. Python 2 has some pretty handy features as well. One of them is what are called exceptions, and so an exception in Python is. A way of handling error conditions without relying on return values alone. So recall that and see if you ever wanted to signify that something went wrong, you have to return like most recently like null, N U L L, which was a special sentinel value. Technically it's just the zero address and by checking for that you can make sure that you know if you're getting back a valid pointer or not. And in other functions, if something went wrong you might similarly have to check the return value, maybe checking for 0 or -1 or 1 or something like that, but return values were the only way in C that functions could communicate back to the programmer that something went wrong, and this is problematic because if you imagine implementing a function that's supposed to return maybe an integer, whether positive, negative, or zero. It's kind of unfortunate sometimes if you have to steal one of those values and say uh uh you can't use this value. It's fine in the world of pointers because the world decided years ago we're never going to use the actual address 0x0, the zero address, but that's still technically costing us one or more bytes of space. But in general it's a bit annoying if your function can't truly return all possible values. Think about a function like get_string. If something went wrong in get_string, what do you want to return? Well, we saw in the CS 50. Library we do in fact return null once we introduce that. But in general, wouldn't it be nice if functions could somehow signal out of band, so to speak, that something went wrong. So by that I mean this. Let's go into a new program that's inspired by one of our programs today, and in VS code I'm going to go ahead and close my calculator, open my terminal window, and create a new program called Integer.i. So in integer.i, let's just play around with some integers and see what we can break. So here I'll define a variable. And and set it equal to the input function which comes with Python, just asking the human for some input. Then I'm going to go ahead and ask a question. Is the user's input numeric? And it turns out if you read the documentation for strings in Python, they come with not just an upper function, a lower function, aka methods, but also it is numeric function or method that tells you whether or not the string itself happens to be numeric. That is, looks like a number. All right, so I think if I do that. I could then do something like this. If N is numeric, I'm gonna go ahead and claim that in fact it is an integer. Else if it's not numeric, I'm going to claim that it's not an integer. I have no idea what it is. Maybe it's cat, maybe it's dog, maybe it's a mix of numbers and letters, but it's definitely not an integer as defined by a sequence of decimal digits in this case. All right, so let's try this out. Python. Of integer.pi enter we'll type in 1. That's an integer. We'll type in 2. That's an integer. We'll type in 0. That's an integer type in cat, not an integer. So that seems to in fact work. But what if I wanted to immediately convert this to an end as we did in the past? And so let me modify this a bit here and say instead this N equals not just input sing the user for an integer or rather let's just ask

them more generally for input, but let's assume that we want to convert this input to an integer and actually we can go ahead and say integer here. All right, well, here I'm going to go ahead and just print out the claim that yep, this is an integer because if we get to line 2, well, clearly we've handled the user's input correctly. In other words, how can I get rid of constantly checking the return value, sorry, how can I get away from constantly checking the return values of functions to make sure it is what I expect? All right, well, let's go ahead and run Python of integer.py now. Enter, type in 1. It tells me it's an integer. Type in 2, tells me it's an integer 0 tells me it's an integer. Type in cat. Notice this time what goes wrong. Whereas last time we saw this kind of trace back error message, it was a name error because I was using the meow function name too early. Now I'm getting a value error, which is a different type of error that relates to invalid literal for int with base 10. Now that's a mouthful, so unfortunately Python's error messages aren't all that much better than. error messages, but clearly the interpreter does not like the fact that I'm passing something to int related to base 10, but that's quote unquote C and really the best you can do with this kind of error is realize like, OK, it's clearly the case that cat is not an integer, so it's having trouble converting cat to an integer. It makes no logical sense. All right, so what's the gist of the problem? Well, I'm just blindly converting the user's input to an integer. Even if it's not input, even if it's not an integer. Well, all right, well, I could rewind to the previous version of my function, use the is numeric function, and then conditionally convert it, but I'm trying to move away from constantly checking return values of error messages. And wouldn't it be nice if I could somehow catch this value error and just deal with it if it happens? And in fact you can with Python exceptions which exist in other languages as well, Java among them. You have the ability to sort of listen for errors happening inside of functions without having to rely on return values alone. So let me go back to VS code here, clear my terminal just to simplify things a bit, and let me literally say to the interpreter, please try to execute the following two lines of code, except if something goes wrong like a value error, in which case go ahead and print out something like not integer. So wouldn't it be nice if you could just wrap all of the code you've written in CS 50 thus far with try and sort of ask the computer politely like please try to execute this code, but that really is the semantics behind it. Try to execute these lines of code except if there's an error, then do this other thing instead and therefore you don't have to check any return values. You can just blindly pass the output of the input function. As the input to the int function knowing that if something goes wrong inside of there, Python is going to execute this code instead, except when something goes wrong. So let me go ahead and run Python of integer.py now. I'll type in one, and that works because it's trying to execute line 2 and succeeding. It's trying to execute line 3 and succeeding, so lines 4 and 4 never actually kick in. But if I try again here with cat, line 2 is going to fail. Line 3 is never going to get reached because Python is immediately going to jump to this exception handler, so to speak, thereby catching the error or the exception and printing not integer instead. So it's a little bit of a weird convention. It's different from what C offers, but a lot of newer languages nowadays do offer this because it's a better way of just writing code that you know should work 99% of the time. But if something does go wrong out of memory, the human types something wrong in or something like that, you can handle all of those exceptional cases, exceptional in a bad sense, using this except keyword instead. Questions on. Any of this here technique. Yeah. A really good question. In this case, I used a value error. Do I need to define every possible thing that can go wrong? Short answer yes. Now there aren't terribly many. There's some standard ones and they're all

capitalized in this way, capital letter, capital letter, something error typically you can even invent your own, and it's good practice to enumerate the kinds of things that you think can go wrong. Value error is pretty. Generic, but there could be memory related errors. There could be file not found related errors. There's a bunch of different exceptions that are all documented in Python that you can listen for. That said, as nice as Python's documentation is overall, it is not good at documenting four specific functions, what exceptions they can throw, and I've never understood this after all of these years that no human has gone. The documentation and painstakingly enumerated all of the possible things that can go wrong. What's too often the case in the real world with some of my own code included is if you encounter an exception that you didn't think was going to happen, you go in and improve your code and add to this list of accept clauses. What else might go wrong? It shouldn't be that way, and different libraries are better about documenting these things. All right, well, with that in mind, let me propose that in the CS 50 library for Python, get in and get float, they work just like the C library, whereby if you type in cat or dog or bird into those functions, they just reprompt you. They just reprompt you. And long story short, this is the kind of code we wrote in Python. Try to get input from the user, except if something goes wrong, prompt them again, prompt them again. So we too are using precisely these features even though it wasn't something that was available to us in C. All right, but something else that we didn't see was play around with Mario in a few different forms. And in Lecture recall a few weeks back, we experimented with like using some Asky arts, some very simple text to print out something like this pyramid of height 3. Well, how can we go about printing something like this? Well, I would propose that if I go back to BS code here, let's close out my integer examples, code up a new version of Mario and Mario.ie. This one's kind of simple. I can say something like 4 I in range of 3, go ahead and print out, quote unquote a hash down in my terminal window. Python of Mario.3 and I've got really the closest analog to 3 bricks stacked on top of each other in this way. But in C and eventually our implementation of Mario started to get a little fancy, and we started to prompt the user for the height of the of the wall and therefore we could have not just 3 but maybe 4 or even more bricks being printed. So let me actually open up that version from a few weeks back whereby from week 1 we had a version of Mario. That looked like this whereby we after including some header files, declared in main a variable called N. Then we saw a new construct at the time, a do while loop that just keeps using get in, get in, get in, so long as n is not 1 or greater. Equivalently, so long as n is less than 1 and kept prompting the user again and again, the reason for having N up here recall was issues of scope. This therefore it's accessible lower in the function as opposed to it being confined to those curly braces. And then down here we used a 4 loop to actually print out that many hashes. So in short, the do while loop solve the problem in C, whereby you want to get user input at least once and maybe again and again and again if they don't cooperate the first time. And that's where do wild loops really shine. Do something at least once and maybe again, again and again. Otherwise it's a little more annoying to do it with wild loops or 4 loops. Unfortunately, Python does not offer a do while loop. And so here too we have an opportunity to introduce you to what the world would call Pythonic. What is Python's solution there too? Well, on the right hand side here in Mario.ie, let's change this a little bit and let's do from, uh, let's go ahead and do. Uh, while, whoops, while true, capital T, go ahead and use a variable N, set it equal to in input. Height asking the human for the height of the wall, and I'm going to just cross my fingers that they're not going to type in cat or dog or something that's not an in in this case.

I'm going to say if N is greater than 0, that is a positive number that's useful, we can proceed. I'm going to now break out of this loop and then lower in the file I'm going to say for I in range of N, go ahead and print. Out the hashes. So we still have that same lesson as before like the Python version seems to be shorter, more concise, even if you ignore the comments on the left hand side, and I've completely avoided using a do while loop, but there are a few things that are different nonetheless that feel like versus C shouldn't even work. Like what's weird about this solution, even though I think it's actually correct. Yeah. I have 2. OK, so it's not correct. That's one of the first things to point out. So too many prepositions for this was supposed to say for I in range. OK, so now that this program is correct, what looks weird to you and probably could break it, yeah. Yeah, so the N variable should be, it seems to be scoped to the Y loop, at least insofar as it's indented inside the Y loop, which feels analogous to being inside of curly braces and C. And so it seems weird that I'm presuming to use N on line 6 even though it was only defined on line 2. It turns out this is possible in Python. The issue of scope that we encountered in C is not as rigorously enforced, we'll say for today, such that when you define N up here you can actually use it down here and you can think of this as being a little reasonable because if there's no more specific. of what data type N is and no more semicolon, just imagine it would look kind of stupid if you just put a blank end there and hit enter just so it kind of exists. There's no way to express the idea of create this variable in advance without actually assigning it a value, whereas in C we could do that. So this is in fact OK and correct. What else is going on here? Well, instead of a do while we're kind of just implementing the idea of it. I'm just blindly inducing deliberately in infinite loop like do the following forever. But then as soon as I have the answer I want, like a positive integer from the human, break out of this loop, and this is indeed the Pythonic way to say get user input because this will minimally ask the user for a height once and maybe more and more times. So no do while loops, only while loops and 4 loops, and only while loops are really the same as in C. Even 4 loops we've seen are a bit different. All right, well, how about instead of just that Mario example, recall this one where we wanted to print like 4 question marks in the sky side by side. We can do this in a few different ways. Let me go back to VS code, close the C version, and let's just completely change Mario.ie to implement this now. I want 4 question marks in the sky, so I think I can do something like 4 I in range of 4. Go ahead and just print out quote unquote question mark. Do you like this Python? Of Mario.ie, Should I run it? No. why? This is how I did it in C, yeah. Yeah, I've got to edit the end value, the named parameter for the print function, because otherwise if I hit enter they're all on different lines, which is not the effect I want when all four question marks are meant to be side by side. All right, well, that's an easy fix. I can pass the named parameter called end into the print function, set it equal to quote unquote with double quotes or with single quotes, as always stylistically I would be consistent. So I'm going to use double quotes even though the documentation is consistent with its single. Quotes. Now I'm going to rerun Mario of Python Mario.ie, and I'm so close now they're on the same line, but the stupid cursor didn't move to the next line. That's fine. How to fix this? Well, just logically, I can put a blank print statement below, and even though I'm not passing anything in, you get a new line for free when calling print. So even though I'm not passing in any arguments, I am getting the aesthetic effect that I want. So that is a perfectly reasonable way to do it. Now if you feel yourself becoming a bit of a geek though in learning about Python and previously see, you can even solve this problem even more pythonically by saying print, quote unquote question mark 4 using multiplications similar in

spirit to the plus operator for concatenation and now multiply the exclamation point by itself 4 times. So now if I go down here and run Python of Mara. i, I got a very elegant solution to exactly that same problem even more concisely than my previous version. What if I want to do something in two dimensions? We'll recall that we moved to the underground of Mario Brothers here and we had like a 3 by 3 grid of bricks. How can we do that? Well, see, we had nested 4 loops using I and J back in the day. And I can do the same thing in Python. Let me go back into VS code here and let me do one outer loop for I in range of 3. Then let me do an inner loop for J in range of 3. Then let me go ahead and print out a hash, but let me learn from my past mistakes. I don't want to print out a new line every time, so let's override that default. But after each row, let's print a new line so that down here I can go and Mario.ie, run it, and I've got my 3 by 3. of bricks. I could change this a little bit and call this row and column, even though here too, even more so I'm not literally using row and column anywhere explicitly, but semantically it kind of explains maybe a little clearer to the reader what's actually going on. So that might help, but we can tighten this up too, right, if I just want to print a 3 by 3. Well, I know that the top thing here will iterate 3 times, and I know how to very elegantly print things out with a one liner, so I could just print out a hash times 3 in this case. And then down here I can go to Pythonomaiodai and voila, I'm back in business too. So it's just sort of easier to do these kinds of things and express yourself all the more succinctly. Well, what else can we do? Well, it turns out in Python. That unlike arrays, you can ask lists how long they are, so you don't have to keep around a variable of how large an array is. You can just add stuff to a list and then ask Python how long is this list, how many elements are in it. Case in point, let me go back to BS code and clear out Mario.ie and let's reimplement from a few weeks back the notion of calculating like in the average quiz score that you might have in a class. So in score.ie, Let's go ahead and create a program that's got a list called scores of three scores that we've seen before 72, 73, and 33 and recall that we tried a few weeks back and see to average these together. And to do that, we had to add them all together. We had to divide by the total number of elements in the list. It wasn't that hard. It was sort of like grade school arithmetic to calculate an average. But Python has more functions available to us, not just length, but even summation. So let me go ahead and do this. Let me say that my average variable shall be the sum of those scores divided by the length of those scores. And indeed per the documentation, Python has a lang function, LEN for short, a sum function. In which takes the which adds together all of the elements in that list. And so down here now I can say something like print with an F string or format string that the average is whatever that value is and I don't have to do any loops or math myself. I can just call the function like I could in Excel or Google Sheets or Apple Numbers, Python of score.ie. Enter and my average is in fact 59.3333 and then some weird imprecision at the end there. And in fact just for consistency with our C code, let me rename this. I'm going to rename score to scores plural. That's going to close the window, but now at least you'll see online that we have a program indeed called Scores. Well, this is not that interesting because I've just hard coded my 72, my 73, and 33. What if we want the human to be able to type that in? Well, I think we can do that too. So let me actually open up. That version of the file now pluralized. Let me go ahead and not initialize the list for the human, but let me set it equal to an empty list just using an open square bracket and closed square bracket like an array that has nothing in it, but this one is literally a size 0 at the moment. And now let me do 4 I in range of let's just for now ask the user for 3 scores, even though we could certainly ask the user how many scores do they want to input and

then use that number instead. So in each of these iterations, let's ask the user for a score using something like `input score`. I'm going to set aside the reality that if the user types in `cat` or `dog`, the whole thing's going to break and therefore I should really add my `try` and my `except`, but I'm going to discard that error checking and focus only on the essence of this program for now. Now after line 3, if I have in a `score` variable, the user's quiz score, how do I put it into that array? Well, in that list, well, with an array I had to use the square bracket notation, keep track of how big it is, and use `len` or something like that. No longer in Python because a list is an object that has not only data but functions, aka methods associated with it. I can just call a method that comes with every Python list called `append`. And pass in that score using that same dot notation as before. The rest of my code can stay exactly the same. If I now run `Python of scores.py` and I type in 72, 73, 33 manually though, I still get that same average and notice I did not need to decide in advance how big that list of scores was going to be. Questions on what we've just done with lists. No. All right, even cooler for some definition of cool is that we can now implement hash tables or more generically dictionaries, sets of key value pairs by just using a data type that comes with Python. I claimed last week that like Python, the dictionaries are sort of hash tables in particular are sort of the Swiss Army knives of data structures and that they just let you associate some piece of data with others with Python, you do not need to jump through the hoops that you needed to with problem set 5 implementing your own spell. In your own hash table, you just create a dict object in Python, a dictionary that gives you the ability to associate keys with values. So case in point, let's do this. Let me go back into VS code and close `outscores.py`, and let's create a new and improved version of our phone book in `phonebook.py`. Let's go ahead and come up with a list of names just to demonstrate how we could store a bunch of names in the phonebook irrespective of numbers and set those equal to say Kelly's name. And my name and John Harvard's name just by putting 4 quoted strings or stirrers inside of this list. Now let's ask the human using the `input` function for the name that they want to search for in this list, and now let's implement linear search using Python. I can do this in a bunch of ways, but one way. is to say for each name we'll call it `N` in names go ahead and ask the question if the name I'm looking for equals the current name in the list that I'm iterating over, go ahead and print out just something generic like `found` and then break out of this loop. And let's see if we can find Kelly or David or John or someone else. Python of `phonebook.py` enter, searching for the name say David enter, and it was in fact found. Let me go ahead and search for someone else's name that's not in there, Brian. And now it's not in fact found, although it's not all that enlightening to just ignore the question altogether, it would be nice to say not found and here where is where and see it would be kind of non-obvious to do this and see if you wanted to print out found or if you get through the whole list and you still haven't found the user print not found, you'd have to like keep track with the variable of whether or not you found the person or you'd have to return from the code prematurely just to get out of it logically. Turns out somewhat weirdly but wonderfully usefully for loops in Python can have `else` clauses associated with them, whereby I can say down here print not found. If I run this version of the program and search for someone who's not in the phone book like Brian, now I actually see not found semantically it's a little weird, but essentially what's happening is if you get through this whole loop and you never call `break`. Then you've not actually broken out of the loop, so you're going to hit the `else` and in that case you're going to print out not found, and this is such a common thing to like do this kind of bookkeeping and keep track of whether or not

something has happened inside of a for loop and if so, do this, Els do that. ELTS literally handles that scenario in Python, and this is the most unlike thing that we've perhaps seen in terms of features with regard to at least loops. All right, well this is great that I've kind of implemented linear search, but like we did that in C and it's getting a little tedious. Can't we do better? We actually can. Let me clear my terminal and tighten this up instead of iterating over every name in names just like we keep iterating over in. In ranges and checking for each name if it equals the thing we're looking at, you can actually do something much more clever. You can just literally ask Python if the name you're looking for is in the names list, then go ahead and print out. A found else print not found. And so this is where Python 2 gets kind of cool. In line 5 you have just a simple if condition with a Boolean expression name and names. How does Python know if name is in names? It uses linear search presumably to search over the whole list of names looking for what you care about and then tells you true or false if it found it. You don't have to write the code to iterate over it with a Y loop or for loop or whatnot. You. Just say what you mean. And so here too it's a little more English like if name in names question mark then print found much more so than it would be pronounceable in C. So that's one other cool feature that we now have at our disposal. What's yet another? Well, when it comes to dictionary objects in C, or rather in Python, a dict object really just gives you a set of key value pairs, and we've seen this kind of chart before whereby we might have name and number and name and. Number and name and number. How do we translate this to code? Because in C, as with problem set 5, it was going to be quite an undertaking to be able to store a whole bunch of things in memory in the form of something like a hash table. Well, in Python we can actually define a dictionary ourselves, so these square brackets represent a list, but I can alternatively use curly braces for a very new purpose. I'm going to go ahead and hit enter just to move the second curly brace to a new line. And I am going to now enumerate a bunch of key value pairs, namely, quote unquote Kelly for the first key, Colin, then we'll do plus 1, 617495, 1000 as the number. Then I'm going to go ahead and do quote unquote David for the second key, and since we both work here, I'm going to go ahead and just use that same number as we've done it before. Then a third key for John Harvard Colin and for John we'll use + 1949. 4,682,750, which is fun to call or text. This now, even though it's syntactically a little different, gives me the equivalent of this chart here key value pairs where the keys are the staff names and the values are the staff numbers that implements all of that, a hash table, if you will, in Python's own syntax. So how do I now use this? Turns out I can actually use it in exactly the same way. I'm going to go ahead and generalize this now to people because it contains not just names but names and numbers. So I'm going to change this variable down here to people too. But notice the syntax now. I can still ask the human for a name they want to look up. I can now still say if the name is in the people dictionary, and by definition Python's going to interpret that preposition in as meaning. Is the following key in the dictionary and if so, it's going to return true. But what's cool about this is that besides just making this work as follows Python a phonebook. ie and let's type in David and there's my number. Oh, that's not my number. It just says found. Let's run it again and type in say Brian, not found. OK, that's as expected, but I'd like to know what my number is or Kelly's number or John's number. Well, that's an easy fix too. Inside of this conditional, I can say something like this number equals people bracket name. And we've not seen this before, but we have seen square brackets in C, when we had arrays, this square bracket notation is how you indexed into an array to get a specific value 0123, 4th. What's

amazing about dictionaries, not just in Python but in other languages as well, you can now index into a dictionary just as you can index into an array, but whereas in array you use numeric indices. In dictionaries you use string indices. You can use strings to look up their corresponding value. So to be clear, name at this point is given to us by the human's input. So if I typed in DAVID, name equals David. So this is like saying people square bracket, quote unquote David find David's number that stores the answer. From this two column chart in the variable called number and all that remains is for me to print it out, which I can do using an old F string now. Let me go down into my print statement, change this to an F string, add a colon, add the number variable to be interpolated, rerun this program as Pythonphonebook.ie, type in my name, and there's my number as found. And this is incredibly powerful and why again. hash tables and in turn more generally dictionaries are sort of the Swiss Army knife being able just to look up data with such simple syntax is wonderfully useful and powerful. And in fact we can even do more than this, for instance, let me propose that if you think about other incarnations of key value pairs, you see them all the time, for instance, in like spreadsheets like here's a. screenshot of Google Sheets whereby I've got the beginnings of a spreadsheet with names and numbers. But in this model I want to actually associate some metadata with my data. So the data I care about is the actual names and numbers. But you could imagine having a third column like email address and maybe home address or any number of other pieces of data associated with these 3 people. For now I've Got two columns or two attributes, names and numbers. Each of the rows in a spreadsheet, as most anyone knows who's used a spreadsheet before, represents different records or different pieces of data like this is Kelly, this is David, this is John, and so forth. We can implement this idea using dictionaries and lists together. So the syntax is going to be a little strange at first, but let me go back to VS code here and let me change my people dictionary to be a people list. Between square brackets and the elements of this list now are going to be uh dictionaries themselves. I'm going to use some curly braces inside of these square brackets and say that the name of one person is quote unquote Kelly, and the number for that person is quote unquote plus 1617495 1000. Close quote. Then comma on the outside of the curly braces. Then I'm gonna have another quote unquote name Colin D A V I D comma, then another number, Colin. I'm gonna borrow the same phone number because we both work here. Then lastly, comma and finally, quote unquote name Colin quote unquote John. And then, lastly, a quote unquote number for John Coen + 1 9,494,682,750. Whew. All right, so what's going on here now? Our people variable is now not just a simple dictionary with just individual key value pairs name, number, name, number, name, number. We now have a more generalized way of storing not just a name or a number, but an email address or a home address or any number of other values. How will the commas just separate the key value pairs now. So if I do have email addresses for us, I can put comma, quote unquote email. Colin, like male at Harvard.edu and I can just keep adding these key value pairs to each of the dictionaries because the dictionary is a collection of key value pairs. So it stands to reason that I can associate name with David, number with the number, email with mail at harvard.edu, and so forth, effectively implementing this idea now in the computer's memory and at the risk of significantly oversimplifying, this is what Google and Microsoft and Apple are doing with their spreadsheet soft. Where they have written code that presents to you a nice table with a graphical user interface on the screen, but underneath the hood what they effectively have is lists of dictionaries representing each of those rows, and we're going to come back to this when

we start experimenting before long with our own databases in the back rows of data from databases, we are going to store that data in lists of dictionaries for the same reason as well. So how can we use this? Well, let me hide my terminal for a second and tweak the program just a little bit. I'm still going to get the name of a person to look up their number. I'm still going to, uh, how about iterate over this because I've lost the ability, at least for now, to just ask a question like is this name in the structure because it's a list. I do now need to iterate a little bit differently, so I'm going to do for each person in the people list, go ahead and check is the current person's name. Equal to the name I'm looking for and if so, go ahead and create a variable called number set it equal to that person's number, and then go ahead and print out for instance found colon then in my curly braces that specific number and then after all that break out of this. So this is a mouthful, but. Recall that it's all the same syntax we've seen before in smaller parts. Square brackets and square brackets means here comes a list. What are the elements of this list? dict, dict, dict three dictionaries back to back to back, each of which has a key and a value and a key and a value called name and number respectively. The second one temporarily has name and number and email as keys plus 3 values and the third one has keys of name and number as well with their corresponding values. So when I iterate over each person in the people list, that means on each iteration person is going to be set to this dictionary, then this dictionary, then this dictionary. On each iteration I'm asking this question Is that current person's name key? Uh, is, rather, is the value of that person's name key equal to the name I'm looking for, and if so, grab a variable called number, set it equal to the value of that person's number key, and then just print it out. And if we wanted to email instead, I tweak the word uh number to email. If I wanna look up anything else you can tweak that code there, but being able to index into dictionaries using strings is sort of the fundamentally powerful new technique that we have here. Question now on any of this, yeah. Good question. If you wanted both name and number on the screen, do you concatenate? Sure, you could do that or print them out by passing a comma into the print function and printing one out each way. Absolutely, however you want to format it. And actually, just as an aside too, even though this becomes a little less readable, this is a little silly that on line 11 I'm declaring a variable called number only to use it one line later and then never again. Technically with those curly braces and format strings, I could just take this code on the right, plug it into those curly braces, and get rid of this variable altogether. Just at some point though, F strings start to get a little too hard to read with quotes inside of quotes, and so like I kind of prefer being a little more pedantic about it and explicitly putting it in a variable and then interpolating just that variable, but you could do it in different ways still. All right, a couple of final features of Python that will get us on our way with doing other things. Turns out there's a whole bunch of libraries that come with the language itself that you nonetheless have to import, even though they're not third party. You don't have to install them. You just need to add them to your code by importing them. One of them is SIS, and among the things that the SIS library has in Python is the ability to give you access to command line arguments. After all, we've lost access to command line arguments because there's no more main, at least by. There's no in main void. There's no in Main RVRC stuff going on in our code, but all of that functionality is still available in a library called SIS. So how do we use this? Well, let me go back to VS code here now. Let me create a relatively simple program called greet.ie, similar to a few weeks back, that's just going to greet the user using command line arguments instead of get string or the input function. I'm going to do this by saying from the SIS library

import RV. In this case, RV is essentially just the list. It is a list of the command line arguments that the human has typed. It's a list, which means you can just ask the length function LEN what its length is. So there's no need for RGC anymore. You can just literally ask RV how long it is, which is kind of nice. So I'm going to say this if the length of RGV. Uh, equals equals 2, which means the human type two words at the prompt. OK, let's go ahead and greet them, assuming that's their name and say hello, comma, and then whatever their name is. Let me make this a format string and to be pedantic, let me create a variable called name and set it equal to RV1. Which is going to be the second word that the human typed in, as has been our convention in the past, else if they didn't type exactly two command line arguments, let's just go ahead and print out something like hello world as generic. Let me run Pythonogreet.ie, E, and you see hello world, because I apparently did not type in exactly two words, and yet I did. So let's see where this is going. Let me rerun Pythonogreet.ie but type in my name David at the command line. Enter. And huh, I screwed up unintentionally. What did I do wrong? All right, print F is not a thing. So that's an easy fix. Let's delete it. Let me clear my terminal window, rerun Python of greetie David, enter, and now I get hello David. The only thing that's weird here is that I typed in 3 words at the prompt, and yet I'm checking for 2. And it's a bit subtle, but with Python and RV, it ignores the Python interpreter. It goes without saying that you're using the Python interpreter to run a Python program. So the only things that are being counted are the words after the Python interpreter itself. So when I type greet.ie and David, that's 2, when I only typed greet.ie, that's 1. Instead All right, so now that I've done that, I have access to my command line arguments again. What about my exit statuses? This was getting a little low level, but in recent C programs we've had you all returning 0 on success, returning 1 on error. Can we still do that? Well, yes, and in fact, the SIS library is used for that as well. So if I want to actually add some exit statuses to a program to facilitate Check 50 and automated tests in the real world, I can do that with a program called, let's call this exit.ie. And in exit up I, I'm similarly going to import uh CIS, but in a different way, I'm gonna give myself access to. Well, yes, let's go ahead and import the whole library just to demonstrate how you can access things inside of it without explicitly saying from SIS, import such and such. as before, if uh the length of sys.orgv. So this is a little bit different, but I'm asking the same kind of question, does not equal to. I want to go ahead and print out to the user missing command line argument, which is something we did a while back as well. And then I want to exit with code 1 CIs.exit 1. El if I don't run into that issue, I'm going to go ahead, actually, let's not even bother with the NELTS. Let's for parity with our C version, let's do this print Fo hello. Uh, sis.RV1 close quotes.exit. All right, that's a whole mouthful, but what's really going on? So I could have done from sys import RV, but I don't need to enumerate every single variable or every single function that I want from a library. I can also just more generally say import the whole library. Give me access to everything, and then I'll tell you what I want from it later. Therefore, on line 3, I can still access RV. I just have to scope it to the SIS library so that I say sl.RV means go inside of that library and find me RGV instead of elevating it to a variable unto itself in my own code. Why am I saying not equal to 2? Well, if they don't give me two words after the interpreter's name, I want to yell at them and say missing command line argument and then exit 1. I'm not going to give them a default hello world anymore. I want them to give me their name. Meanwhile, if I get this far and I haven't exited from the program, I can print out sl.RV1, which is going to be David in the example I typed before, and this means success, so sys.exit 0 signifies success. It's more syntax than before than it was in C, but we

have the exact same functionality available to us as we have in the past. How about one other example that we've had in the past? Let's convert it to Python as well. So you have a few more tools in your toolkit. How about implementing a version of this phone book that actually persists? So instead of hard coding into it, Kelly and David and John in this way, let's actually let the user type in a name and a number just like on your iPhone or Android phone and add it to a text file, like a CSV file as we did before using comma separated values. Well, it turns out that Python comes with a library to handle CSV files. We don't need. Hackishly implement our own CSV support by printing the commas ourselves. Instead, we can import the CSV library. We can then create, say, a variable called `file`, set it equal to `open` and open a file called `phonebook.csv` in a `pend` mode. So this is almost the same as C, except it's `open` instead of `F open`, which we saw a couple of weeks back. Now let's ask the user via the `input` function for the name they want to add to their contacts and the number that they want to add to their contacts, and then And so after that, let's go ahead and do this, which is a bit of muscle memory to remember, but I'm going to create a variable called `writer`, but I could call it anything I want. Set it equal to `csv.writer`, which means there's a function called `writer` in the CSV library that I'm simply accessing it because I didn't import it explicitly by name, and I'm going to pass it that `file`. This tells Python, turn that `file` into a CSV that can be written to. The next line of code I'm going to literally say `writer.writerow`. `Write row` is a method, aka function associated with this `writer` object, and I know that only because I did actually read the documentation for the CSV library. What do I want to write? Well, I want to write out a list of values, namely a name. And a number and I'm using square brackets to tell the `write row` function that here you go, here's a list of values, 2 of them a name and a number. After all that, I'm going to do `file.close` and just close the whole file. All right, so where does this actually get me? Well, let me go ahead and open up `phonebook.csv`, which is initially empty. I'll move this over to the right hand side. But when I now run this program with Python of `phonebook.py`, I'll type in say Kelly's name, E + 16,174,951,000 enter and voila, it ends up in the CSV using a little bit less code than we had to last time with C. Let's run it once more and I'll type in my name and I'll again use +1 617495 1000 enter. It's being appended to that file as well. And one last time for John + 19494. 682,750 enter voila. So it's pretty easy that is to say in Python to start creating files like this, but this isn't really Pythonic. Let me in fact close the CSV file, hide my terminal, and propose that we can tighten up this code a bit too. I don't need to open up the file way up here. I can go ahead and get my variables values this way first. And in fact, I could have done that code a little later anyway. But I can do this in Python. I can say with the following file opened `phonebook.csv` in a `pend` mode and refer to it as a variable called `file`, do this stuff and close the file yourself. So this program's suddenly significantly shorter because this one line has the effect of opening the file for me in a `pen` mode, assign it to a variable, do this stuff, and. As soon as the program's indentation ends and there's code over here or no code whatsoever, the file gets closed for me automatically. This just helps us avoid like memory leaks and like stupid mistakes we've made in C because you forget to close a file that you have to open and you don't necessarily notice unless you run Valgrind or something on it. Python tries to avoid this by giving you a new keyword with that doesn't really make sense semantically except with the following `file open` and it will close the file for you. So that's 2 among the features that you sort of get. With Python, the catch though is that the CSV is fairly simplistic. In particular, it's missing a header row that actually indicates what is in each of the columns. In fact, if I go ahead and run code of

phonebook. CSV, we'll see again that the file contains just one row for Kelly, for me, and for John, whereas ideally it would look a little something more like this Google Sheet version, which actually has at the very first row something saying name and number, which then describes the data therein, after which are the three actual rows. Now the simplest. here frankly would probably be to just start with name number at the top of the file and then assume that my phonebook.ie program is just going to append, append, append additional rows to the file containing the names and numbers respectively. I could have done that from the get-go, and in fact that would be better than putting some code inside of phonebook.ie that writes out that specific row because after all, if I'm running this program again and again, I don't want the header row to appear again and again and And unless I complicate the program a little bit to ensure that I only do that once. But assuming that I do go into phonebook.csv and from the get-go do have a file that contains name and number, we can actually start to improve upon the implementation of Phonebook.ie because we can take advantage of the fact that my dictionary can actually that my writer can actually read that same header. In fact, let me put these files side by side here and then in phonebook.ie, let's go ahead and transition away. From using a writer to using a so-called dictionary writer or dict writer for short, capital D W and then let me go ahead and specify one additional argument to this particular function, namely field names, which I know exists because I looked it up in the documentation, and the value of this argument is supposed to be a list of the fields that are presumed to exist in the CSV that we're about to write to. So I'm going to do quote unquote name, unquote number. The lines a bit long, so it's scrolling there, but if I scroll back to the left. We see that the line is otherwise unchanged, but when I go down now to right each respective row, notice that I don't have to rely on this list, which just assumes somewhat naively that name will always be in the first column or column 0 and number will always be in the second or column 1. After all, if someone were to move that data around, at least in the spreadsheet using Excel or Google Sheets or something else, my code would end up being fairly fragile because at the moment it's just assuming blindly that name goes first, followed by. Number, but once we have that header row in there and tell dict writer about it, we can actually now pass in not a list but an actual dictionary of key value pairs and let the dictionary writer figure out where in the file which column those values should go in. So inside of this dictionary, I'm going to have one key called name, the value of which is indeed the name the user typed in. The second key of which is going to be quote unquote number, the value of which is the number that the user typed in. And let me go back actually now and fix a typo from earlier. We're only asking the user for. Number. So all this time I should have just requested one number aesthetically with my input function there. Now notice I have the file ready to go. Indeed, name and number are there. That matches the field names I've provided to my code and it matches the key value pairs that I'm subsequently passing to right row. So let's go ahead and give this a try. Let me go ahead and run again with this otherwise empty CSV file, say for the header phonebook.ie with Python of phonebook.ie. Enter. I'm gonna now go ahead and type in, say the first name, which was Kelly before, plus 1617495 1000 and watch what happens at top right. Kelly and her number end up in the file, even though I didn't actually specify explicitly as with the list or numeric indices which value goes where. Let's run it once more and put in myself again plus 1617495 1000. Enter and there again I am. And lastly, just for good measure, let's go ahead and put John back in the file with. Plus 1949468-2750, which if you still haven't called or texted, do feel free. Enter and voila in phonebook. CSV we have all of

those same rows and code that's a little more resilient now against any changes we might subsequently make there too. All right, how about now some final flourishes using some other features of Python that we did see a glimpse of some time ago, namely the ability to install libraries of our own choice. So up until now in CS50.dev, we CS 50. have pre-installed most of what you need, including back in the earliest weeks of the class when we had that Calay program that I wrote that was using a third party library that I had installed into my codespace in advance. Well, you can use a program called PIP to install Python packages into your own code space and if you're using your own Mac and PC onto your own Macs and PCs as well, if those libraries are freely available as open source online and in the repository from which the Python. Uh, PI program actually draws. Let me go back to VS code and let me go ahead and create a new program called Cow.i. And with this program, I'm going to go ahead and import that library CalSay, and after that I'm going to call `coa.cal` quote unquote, say this is CS 50 to have a cute little cow on the screen, say exactly that. Now in a previous lecture I had pre-installed this library, but suppose I had forgotten to do so today. Let's see what other type of error we'll see on the screen. Well, let me go ahead and run Python of cow.ie. Enter and there's another one of those trace backs. This one's a little more straightforward than the name error and the value error we saw in the past. This is a literally module not found error, no module named CalSay. Well, this is where the PI command comes in. If something hasn't been preinstalled for you in CS50.dev or in the real world on whatever system you're using, you can use PI install CalSay, and assuming you've spelled it correctly and assuming the library is publicly available, hitting E will result in Pip automatically download. In the latest version installing it in this case into your code space, and solving hopefully that problem. Let me clear my terminal window. Run Python of cow.ie again, definitely cross my fingers, and there's the most adorable cow, and if we full screen the terminal, we'll see that he's indeed saying this is CS 50. Now that's just one of the things we can install with Coway. I could also install libraries onto my own Mac and PC. In fact, in just a moment I'm going to switch over to another computer here where I have a terminal window open on my. My own actual Mac and I'm doing this because I'd like to play around with some speech, some text to speech library functionality which you can't really do in CS50.dev because it's browser based and when you run code in the cloud, it's not going to pass the audio along to your speakers on your laptop or desktop. But if I'm running Python and my own code on my own computer, a Mac in this case or a PC in someone else's case, I can install that kind of library, speech to text, and have my own code on my own computer use my own. Speakers to verbalize some string quite like that. So how can I go about doing this? Well, I read some documentation, I'm going to go ahead and install with PI a library called Pi to Text to Speech version 3. Hitting Enter goes and spines and downloads as needed the library if it's not already installed and then brings me back to my terminal. And I'm going to use an older school program here called Vim or VI to actually implement a Cal program on this computer whereby I'm going to go. Head and write some code using this library without VS code but with just another text editor instead. To do this at the very top of my file, I'm going to import this library called Python Text to Speech, so `PY TT SX3` for version 3, and then I'm going to use only 3 lines of code to synthesize some voice. I'm going to say a variable called engine, set it equal to `pittsX3`. in it because the documentation taught me that I need to initialize the library the first time I use it. I can then use this variable called engine to actually say something quite like scratch, albeit verbally instead of pictorally, like this is CS 50, quote unquote, and then lastly I can use

engine.ru and wait similar to some scratch blocks so that the whole Expression is actually verbalized before my program actually quits. Now, the first time I run this, it might take a moment for the library indeed to initialize itself, but on my own Mac here I'm going to run Pythono cow.ie. If we could raise the volume just a little bit, hopefully we'll not see but hear this cow's greeting. This is CS 50. It was very much in a rush to say it, but after initializing for that long, and if we ran it again and again and added some optimizations, we could get it talking much more quickly than that. But we now have a version of the program that indeed verbalizes what string or stir it is that I've passed into it here CS 5. It's really in a rush to finish there. All right, let's try one final flourish of another library that's fun to play around with, if only because it'll motivate some of the things you can now do in Python yourself. Let me go into VS code in my code space because this one does not require my speakers. I'll close that first version of the cow, and I'm going to go ahead and create a QR code generator after installing with PI a library called QR code, which I read about online, and now it's Installed in my code space. I'm going to now go ahead and create a file called qr.ie. So let's go ahead and code up QR.ie, and I want to generate my own QR codes. Most of you in the are in the habit. If you've ever generated a QR code before, you probably just Google around for some generator online for which someone else wrote code to generate the QR code, but I can do that for myself and actually generate my own images. I'm going to go ahead and import the library that I just installed, import QR code. And then below that I'm going to create a variable called, for instance, image and set that equal to this library's QR code.make function, no relation to the make that we use for C, and I'm going to make a QR code containing a URL, maybe of one of the lecture videos. So let's do HTTPS call in slash Youtube.com, the short version, and then XVFZJO5PG. Uh, GG 0 if I got that just right. Then after that I'm going to go ahead and call image.save to save that URL as a file called qr.png quote unquote, and then PNG will be the format which is portable network graphic, which is akin to a JPEG or a GIF but with different features. I'm just going to double check my writing here so we go to the right lecture video and I think we are indeed good. And what that should do after running my code is leave me with today's final flur. a ping file in my code space that when opened is going to be QR code that you can scan with your phone. So if you'd like to get ready for this final flourish, I'm going to go ahead and run Pythonoqr.ie and hit enter. Thankfully it worked. I'm going to now open up QR.png and close my terminal window and for our final moments together this year in week 6, after which we'll ultimately transition to yet more languages and problems to be solved. Here is a final code for you to scan. Of today's here lecture. All right, that's it for today. We'll see you next time.

All right, this is CS 50, and this is already week 7 where wherein we introduce another programming language, this time known as structured query language or SQL or SQL for short. Now SQL, as we'll see, is a different sort of programming language that allows us to solve like a lot of the same kinds of problems that we've been dabbling with over the past several weeks, but arguably in a lot of contexts, it allows us to solve those problems more easily. Indeed, among the goals for. Today are to demonstrate that sometimes there's multiple tools that you can use to solve the same problem, whether it's C or Python or today's SQL but we'll also see that SQL allows us a different sort of approach to solving problems, whereas C very much so and Python to a large extent are very much procedural programming languages whereby you have to write these procedures, functions step by step that tell the computer what to do, including loops and conditionals and all of that. SQL is said to be a declarative programming

language, which is a different sort of paradigm whereby when you want to solve some problem you essentially declare what problem you want to solve or you declare what question you have, and it's up to the programming language to figure out using loops and conditionals and all of those lower level building blocks, how to get you the answer. So ultimately today is all about teaching you yet another language, mostly so that you can learn again to teach yourself new languages and to appreciate that when. You exit a class like CS 15 or out there in the real world really isn't all that big a deal to pick up new programming languages, especially when in advance you've seen different programming paradigms like procedural, like object oriented, like today declarative as well. But today ultimately is also about data and so to get us started, we thought we'd collect some real world data by asking all of you a couple of questions. So if on your laptop or phone you would like to pull up this URL here. It will also exist in just a moment in QR code form. So if you'd like to go to that URL there or simply scan this here QR code with your phone, that's going to lead you to a Google form. For those unfamiliar, Google has lots of tools, among which are, uh, is a tool via which you can ask people questions via forms. Microsoft has something similar as well, and at that URL. What you'll soon see is a form that looks a little something like this among whose questions are which is your favorite language, at least among those we've studied thus far. So go ahead and anonymously answer the questions you see on this form you'll see which is your favorite language and also which is your favorite problem in problem sets thus far. And meanwhile, as you might know, if you've used Google Forms yourself to collect data, we can move from questions here to actual responses. And as people start to buzz in, we'll see that the data set here is starting to update in real time, and Google gives us these nice graphical user interfaces or GUIs via which we can analyze the data. And so far, Python is easily the winner with 70% plus of you preferring it, 11% of you, wishing we were still in scratch, and 18% of you in C, and you'll see the responses are coming in here, but for our purposes. Today, what's more interesting than the actual answers to these questions is how we can get at the raw data. So among the things you can do in Google Sheets is quite literally click View in sheets, which is in Google Forms, is click on View in sheets, and what this is going to allow me to do is access the underlying raw data. Now because Google has forms and spreadsheets, they sort of tied these two products together. But what's especially nice about Google spreadsheets is that I can also download the raw data as a file. I can download it as an Excel file, a text file, a PDF, but for Today we're going to download it in a very common format known as CSV for separated values. And indeed, if I go to the file menu download separated values, this is perhaps the most straightforward, easiest way to get raw data out of any kind of tabular data like this to load it into code that we are about to write. So if you haven't buzzed in already, that's fine, but at this point in time, now that I've clicked the button, I now have a CSV file in my Mac downloads folder which if I go ahead and open up here, I can see that indeed. I've got this long named file favorite form responses1.csv. I'm going to shorten that file name to just favorites.csv. And what I'm going to go ahead and do is open up VS code and in my File Explorer, I'm going to literally just drag and drop favorites.csv from my Mac. That's going to have the effect of uploading the file as it was at that moment in time so that we can now begin to write some code using this file. And VS Code is automatically gone ahead and opened it up for me. And what you're looking at here is what we're Start to call a flat file database. It's a very lightweight database in the sense that it stores a lot of data, and it's a flat file in the sense that it's literally just a text file. And by convention, the way the data is stored in

this file is indeed by separating values with commas. There are other conventions as well, but CSV is probably the de facto standard. But TSV is a thing for tab separated values. PSV, which is pipe separated values where you might have a vertical bar. Essentially these file formats try to use a character that might not appear in the actual data so as to. Separate your rows and columns. So indeed if I switch back to VS code here and we take a look at the data, you'll see that from Google Sheets I've been given three columns time stamp, which was automatically generated for me, the language as well as the problem, and what I see here is that we had a few respondents buzz in a little early, very excited for today's data, but here's the rest of them from like 1:30 p.m. Eastern onward, and you'll see separating separated via commas are effectively three columns of data. So everything before the first column represents a time stamp. Every. Between the 1st and 2nd comma represents the choice of language that you all buzzed in with and then everything after the 2nd comma represents the problem. Now it's kind of jagged edges. It doesn't line up in nice rows and columns because some answers are longer, some answers are shorter, but the commas are sufficient to tell the code we write where one column ends and the next one begins. So how do we go about writing code like this if we'd now like to ask some questions about the data like what is the most popular language, what is the most popular problem, or. Conversely, the least of each of those, well, we could look at the original data in Google forms, and that's where we got the pie chart. But how is Google figuring out what the most popular answers are and what uh pie charts it wants to depict? Well, they probably wrote some code not unlike what we're about to do, although we'll start with just a command line environment as always. So within VS code, I'm going to go ahead and do this. I'm going to go ahead and open up a program called Favorites.ie and let's write a program whose purpose in life is to open the. CSV file. Read it top to bottom, left to right, and then crunch some numbers, figure out what the most popular answers are to those questions. So I'm going to go ahead and import a package that comes with Python, a library called the CSV library, and nicely enough, this is just code that someone else wrote years ago that figures out how to read data from a file separating it via comma so that you and I don't have to write all of that ourselves. Then I'm going to use this Pythonic convention with open quote unquote favorites.csv as file. If I want to be super explicit that I intend only to read this file, which is the default, I'm going to go ahead and explicitly say quote unquote R just like we did in C when using F open to open a file and read mode. And now I'm going to do this. I'm going to go ahead and say reader equals csv.reader file. So this is a Python convention whereby the CSV library comes with a function called Reader that takes as its sole argument here a file that has already been opened, and what that reader will do is Figure out where all of the commas are so that I can iterate over this reader in a loop and get back row after row after row without me having to write all of the code to figure out where those commas are. So what I'm going to do in this loop here in this block of code is for each row in that reader, let's go ahead and just print out maybe the second column which was the language column. So I'm going to go ahead and say print row 1 because what we'll see. Is that this reader, which again comes with Python, hands me a list, a list, a list for each of the rows wherein bracket 0 would represent the first column, 1 would represent the 2nd, 2 would represent the third because everything is 0 indexed in Python. All right, so let's see what the effect is here. Let me maximize my terminal window, run Pythonofas.ie, cross my finger that I got this right, and voila. There is every language that was selected by you all in the form from top to bottom by default chronologically. But there's a bit of a bug, I dare say. Let me scroll

up and up and up in this output through all of these answers until I get to the very top where I ran the program myself, which is here, Pythonofavos.ie. There's a minor bug here. What's the bug in the output? yeah. Yeah, it accidentally includes the header, which is a bug in the sense that I really just wanted to see the languages, but the code is doing what I told it to, which is just print out every row. So there's a few ways we could ignore this. Let me go ahead and minimize my terminal window and let me go ahead and say, well, you know what, after we create this reader, let's just skip to the next. Uh, let's just skip to the next row and ignore it effectively and then begin iterating over everything thereafter. And so what happens now is if I re-maximize my window, rerun Python of favorites.ie, enter, and now scroll up again to the beginning of this incarnation of the program, you'll see that the very first thing I see after my program was run was indeed Python, Python, Python, Python, and so forth. No more quote unquote language. So how is that? Well, this is a feature we haven't quite seen before or talked about in much detail, but this reader is is stateful in some sense, and this was actually true of all of the file IO we didn't see whereby when you were using Fread or some other function to read data from the file, something was remembering where it was in the file so that you didn't get the same bytes again and again and again. It was more like a cassette tape, an old school cassette tape if you will, or a scrubber. Along the bar along the bottom of like any streaming video whereby when you just read some data it grabs the next chunk, the next chunk, the next chunk, the next drunk, and something inside of the computer's memory remembers where it is. So this says skip to the next row and thus when you do 4 row in reader, you get everything but the first row because the reader is stateful. It remembers where it is in memory. All right, well thus far this isn't all that useful because all I'm doing is just printing out the data, but let's take a step toward making this program a little more useful. Particular, let's just be a little more pedantic and specify that what I'm really doing here inside of this loop is figuring out what the current row's favorite is. So I'm going to create a variable called favorite and set that equal to row 1. And then, even though this doesn't change the functionality, I'm going to print that favorite just because semantically, stylistically, it's nice to know what row 1 is as by defining a variable that tells me or anyone else who reads this code in the future what it's actually doing. All right, but readers are only so useful, and in fact, if I were to open up this CSV file maybe in Microsoft Excel or Apple Numbers or Google Sheets again, you could imagine someone kind of moving the data by just dragging one of the columns to the left or the right such that now it's no longer time stamp language problem, maybe it's time stamp problem language or maybe time stamp is all. way over to the right, you can imagine therefore that the indices we're using 01 and 2 could be a little fragile because if someone changes the data on me now my code is just going to break because I am blindly assuming that the second column, aka 1 is going to be the language column, but that might not be the case. But there's an alternative to this. And you might recall having seen this before, I'm going to go into favorites.ie and tweak my code a little bit, not just to use a reader, but a dictionary reader. So I'm going to change this to dict reader instead of just reader. And then the upside of using a dictionary reader is that every time I go through this loop reading row by row by row, each row that I'm handed by this reader is not going to be a list anymore that's numerically indexed with zeros and ones and twos. Each row is going to be, as you might guess, a A dictionary, which is a collection of key value pairs, which means now we can use words as our indices instead of just numbers, which is to say if I switch from reader, which gives me lists to dict reader, which gives me dictionaries, I can change this line 10 now. And say I

specifically want the language column wherever it is all the way to the left or the middle or the right so in general using a dictionary reader is probably just going to be more robust because it's resilient against changes in that actual numeric ordering. Alright, let me pause here to see first if there's any questions on this exercise whose purpose in life is just to demonstrate how we can download the CSV data then iterate over it line by line without actually analyzing it yet. No, OK, so let's ask maybe the most natural question, which is like how many people prefer Python? How many people prefer C or scratch in turn? In other words, how can we recreate in our own code what Google Forms is doing for us graphically with those pie charts? Well, I think what we could do is write some code logically that essentially relies on this mental model. What I have here is an opportunity to use a bunch of key value pairs because if I want to know how many instances of Python there are. and C and scratch. Well, those might as well be three keys, the values of which are hopefully going to be 3 numbers that represent the counts of the popularity of each of those languages. So in memory, I essentially want to construct something that looks like this and would if I were doing this on a chalkboard, but recall that this mental model maps perfectly to the notion of a Python dictionary, because a dictionary in Python is indeed key value pairs, and we've seen it already because that's how the dictionary reader works. But we could certainly use our own. Uh, dictionaries to solve the same problem ourselves. So the goal at hand is to count the number of people who said Python and C and scratch respectively. So how to do this? Well, I think what I could do is, I actually let me delete this line because we are using a dictionary reader, we no longer need to skip the first row. It is automatically consumed by the dictionary reader for us, so this now would be the better version of the dictionary reader. Let's go ahead and do this. Let me declare some variables first that will store for me the total number of people who said Python, scratch, and C respectively. So I could say scratch equals 0, C equals 0, Python equals 0, and I could just set 3 variables equal to 00, and 0. If you haven't seen it before, there are some pythonic tricks you can do here if you've got 3 variables that you want to initialize all at once because it's that simple. You could alternatively do scratch, C, Python equals 0.0.0. This too would have the intended effect and it looks a little better because it's all a simple one liner. But what do I want to do now? Well, down here, let's go ahead and do a simple conditional before we enhance this by using an actual dictionary. Let me go ahead and say if the current favorite in that reader equals equals scratch, well, Let's go ahead and increment the scratch variable by doing plus equals 1 as we saw last time. E if the favorite in the current row equals equals unquotedec, well, let's go ahead and then increment the C variable by 1. E if the favorite equals equals Python, then let's go ahead and increment plus equals Python by 1 instead. I could technically get away with saying else here, but I'm consciously this time not trying to over optimize this because if someone changes the form maybe next semester and whatnot, and we're asking about 1/4 language, I wouldn't want my code to assume that anything that isn't scratch or C must be Python. When there could be some future 4th language. So this is a little more robust and in this case we'll just ignore anything that isn't scratch or C or Python. All right, at the end of this, let's go ahead and not just print out the favorite, but outside of the 4 loop, let's go ahead and print out, for instance, the scratch count is this, then let's go ahead and print out the C count is this, and then let's print out the Python count is this, but of course there's a subtle bug here. Yeah. Uh, so I didn't format these things. It's F strings, so I need the little F over here to the left of each of these strings. All right, so let me go ahead and maximize my terminal window, run Python of this version of favorite stoppi and hopefully what we'll see is not

every row again and again and again, but 3 lines of output, giving me the total counts instead. All right, this seems to line up with the rough percentages that we saw coming in earlier on Google Forms, 109 of you like Python, followed by 58 of you and C and 24 of you preferring scratch instead. All right, but why does this perhaps rub you the wrong way? I already alluded to the fact that we're going to get rid of this, but why is this not the best design? Just using three variables like this, yeah. Yeah, exactly. If we were to add a bunch more languages, a 4th 1, a 5th 1, a 6th 1, a 10th 1, a 20th 1, like having that many variables, it's just certainly going to look unwieldy and it's just not gonna, it shouldn't rub you the right way. At that point we should really be graduating to some proper data structure whether it was an array in C or better still in Python, an actual dictionary. So let's do that instead. Let me go ahead and in a newer version of this file, let's get rid of these variables and let's just have a generic variable called counts for instance and set it equal to an empty dictionary and just using two curly braces will give me an empty dictionary or if you want to be more pedantic, you can actually call the dict function which will return to you an empty dictionary. I'd argue though that most people would probably just use the double curly braces like this to indicate that here comes a dictionary for me now. Do I use this? Well, I don't need to update three separate variables. I think I could just do something like this. I could say once I've determined what the current row's favorite value is for language, I could say counts, bracket favorite, so use the current string as an index into the dictionary, so it's going to be quote unquote scratch or C or Python, and then just increment that by 1. And then down here we don't have these variables anymore, so I'm going to go ahead and instead say. Uh, how about this? We'll use a loop for each favorite in those counts, let's go ahead and print out, uh, how about the favorite value and the counts thereof without any F string for now. OK, so the only thing that's different is I'm using a dictionary here which is essentially the code version of this two column chart whose keys are going to be the favorite strings, scratch or C or Python, the values of which are going to be the actual counts, and I'm just doing some simple math by plus plusing or incrementing the count each time I see a certain language.

Unfortunately this code is not quite going to work. Let me go ahead and run Pythonofavours.ie and Dang it, there's a key error. Let me minimize the terminal window so we can see both at once. Why is there a key error apparently on line 11, wherein I'm indexing into the counts array uh dictionary. What's going on? Yeah, yeah, it's a little subtle, but if this is like the very first time through the file, there is no key Python. There is no key C or scratch because no one has put them there and yet recall that plus equal means you're going to that location in the dictionary and just blindly incrementing it. But what is it? Well, it's effectively a garbage value, but it's not even that because there's no actual key there. So we need to do a little bit of logic here and we can solve this in a couple of ways. Well, I could say something very pedantically like this. I could just say, well, if this favorite is in the counts dictionary, this is the Pythonic way to ask that question is this key in this dictionary? If so, well then it's safe to go ahead and increment it just as I've done before. But if it's not, what I think I want to do is set counts favorites equal to. One instead because either I want to increment the current count by 1 or this is the first time logic like I've seen this favorite, so I want to set it equal to 1 instead. We could do this a different way logically just like we could in C solve problems differently. I could instead say something like this. I could get rid of all this code and just say if. Favorite not in counts, then I could say counts bracket favorite equals 0. So just always initialize it to 0 if it's not there. Now I can safely blindly update the count by 1 because now I know no matter what, once I get to line 13, that count is

actually there. All right, so let's see with this version of the code, let's go ahead and clear my terminal window, uh, rerun Pythonofavos.ie, cross my fingers, and There we go, Python and Scratch and see. Interestingly, the order switched around this time based on the order in which I was inserting things into the dictionary, but we'll see how we can exercise a bit more control over that. But let me propose that that key error recall we discussed briefly last week that whenever you have these kinds of trace backs that refer to certain exceptions like. Exceptionally bad situations that can happen, you can also change your code to just try to do something and then try to catch the exception instead. So an alternative way to do what we initially did would be this instead of just blindly saying go into the counts dictionary, index into it at the favorite key, and increment it by one, what we could do is. Try to do that please, except if there is a key error, in which case, you know what, go ahead and just initialize that value to one instead. So in short there's like 4 different ways already to solve the same problem whichever way you prefer is quite reasonable. This is just another way and arguably another pythonic way to do things by trying to do something but anticipating that something in fact can go wrong. A while ago what? Correct. A while ago I removed next reader because that was only necessary for CSV.reader because that was just reading every row again and again, but when you use a CSV dictionary reader, that automatically consumes the first row because that's how the dictionary reader knows what the columns will be called and so you don't have to skip over it instead, a nice enhancement. Other questions. On what we've just done. Here All right, so let me propose that like writing this amount of code is kind of annoying just to ask a relatively simple question like what's the most popular language in this file, right? It's nice. It's sort of a step backwards from Google spreadsheets and Apple Numbers and Microsoft Excel where you could really just like highlight the column and it would just tell you the answer usually in the bottom right hand corner or you could use a function in one of those spreadsheet tools to ask the same question. So it's starting to feel like with almost 20 lines of code like maybe. There's a better way, and I dare say there is rather than use a flat file database, let's graduate already to what the world calls a relational database, and a relational database is simply data in which you define relations among your data, which isn't so much relevant now except that that time stamp is associated with that language is associated with that prefer favorite problem as well. But we'll see that data sets can be much more, much larger and more complicated and it might be valuable. We can actually express relationships across multiple pieces of data. In particular, let's introduce already a programming language called Structured Query Language or SQL for short, AKA SQL and SQL essentially only has 4 fundamental operations. So even though we're transitioning into a new language, by the end of today we're gonna transition out of the new language because there's only so much you can do. Now, as with any language, it's going to take time and practice or to sort of get a hold the hang of it, but take comfort in knowing that SQL really just supports four fundamental. operations and the acronym that the world uses is indeed CRD, which stands for Create, read, update, and delete. That is to say, when using a relational database, you can create data, read data, update the data, or delete data, and that's pretty comprehensive as to what's possible. Now what is an actual database? Well, generally speaking, a database is just a piece of software that's running on a computer somewhere inside of which is stored a whole lot of data and that database therefore provides you with access to that data. At any time, whether it's on your local Mac or PC somewhere in the cloud or to a whole cluster of web servers which we'll talk about in the weeks to come as we transition from command line tools to the web. Now

technically in SQL, the commands you actually use to implement this idea of creating data, reading data, updating, and deleting data is almost the same, but for whatever reason, uh, the world chose the command select, which is equivalent to reading. Data so we'll soon see that there's a command in SQL that lets us select data which is equivalent to this idea of reading it, whereas the other three options refer of course to writing data that is changing data. Um, technically speaking, we'll be able to insert data into a database as we'll soon see, and we'll also be able to drop data altogether, not just delete individual rows but whole tables, so to speak, of rows instead. So what does this all mean? Well, let's go ahead and do, say, an example of using SQL to solve, to ask some relatively simple questions and begin to develop some muscle memory for using this new language. If I were to manually load a bunch of data into a proper database for SQL, I would actually use code like this. I would literally type Create table. I'd come up with the name of the table, aka sheet, and then I would specify every column that I want to put in that table. And here's where the vernacular changes. So whereas in the world of spreadsheets you have sheets, tabs that contain rows and columns, in the world of databases you have tables which are just rows and columns. It's different terminology, but it refers to conceptually the same thing. In CS 50 we're going to use a specific version of SQL known as SQL Light, which is like a lightweight version of SQL that's actually very commonly used in web applications, in mobile applications, but it doesn't have all of the bells and whistles or all of the scalability uh that your Oracle, SQL servers, Microsoft Access, Postgress, MySQL, those are just product. Names open source and commercial alike, which if you've ever heard of just represent bigger, faster versions of SQL databases, but we'll indeed use the lightweight version of it known as SQL Light, and the command we're going to start to run is quite literally SQL Light 3, which is version 3 of the same command which we've pre-installed into your code spaces for you. So let's go ahead and do this. Let me go ahead and run a command called SQL Light 3 which is going to let me create my very first SQL Light database, and I'm going to import into that database the CSV file that we downloaded from Google Forms. In other words, I'm going to load that same data set into a different program, an actual database, so that I can use a completely different programming language to ask questions about it instead of writing, as we just did some Python code. So let me go back into VS code here. Let me close my CSV file and my Python file. Let me reopen my terminal window and let me go ahead and run SQLite 3 space, and then the name I want to give to this database, which for instance will be favorites.db for database by convention. Enter. I'm going to be prompted to make sure I want to create this new file. Y for yes, enter, and now I'm inside of the data. Base running a command at a prompt that now says SQL light and then an angle bracket. I'm not going to be using any.SQL files for now, although you can actually write SQL code in separate text files. I'm actually going to use the database's interactive interpreter to just run all of the commands I want interactively by just typing them out. Semicolon enter. Type it out, semicolon, enter back and forth, but you can save all of these commands as you'll see in problems. 7 in files as well. Now how do I go about actually importing that CSV file into this lightweight database? Well, for this, I'm going to execute 3 commands, and any command in SQL Light that starts with a dot is specific to SQL Light. This lightweight version of SQL, anything that doesn't start with a dot is generalizable and will work on most any SQL database anywhere in the world, no matter the product you're using. So I'm going to go ahead and in my SQLite terminal I'm going to change my mode to CSV mode just to tell the database that I want to load some CSV data. I'm going to then literally import that data

from a file called favorites.csv, which is the file we downloaded earlier and then uploaded to my code space. And now I have to specify the name of a table, so I'm going to call this table, aka sheet favorites just to keep everything consistent. And that's it. In the absence of an error message, everything probably worked fine. I'm going to do quit. That quits out of SQL light, but what you'll now see if I type LS is that not only do I have favorites. CSV, which I uploaded, favorites.ie, which we wrote a few minutes ago, but I also now have favorites.db, which is a database version of that same file. Now I can't actually see what's inside of it because if I go ahead and run a code of favorites.db. I'm going to see this file is not displayed in the text editor because it is either binary or uses an unsupported text encoding. This is to be expected because this database is stored essentially in the form of zeros and ones that the SQLite 3 program knows how to read, but it's not something that VS Code can just show me everything they're in. And generally storing data in binary is going to be more efficient than storing things purely textually because we're going to be able to use various data structures and algorithms that we've been talking about. For weeks more easily on that binary data. All right, so let's go ahead now and see what this import command did. I'm going to again maximize my terminal window. I'm going to go ahead and run SQLite 3 again, passing in favorites.db. Enter this time it already exists, so it just opened it without prompting me. And now I'm going to go ahead and type another SQL Light specific command called schema. The schema of a database is just the design of the database. It look like? What are the rows and columns and tables therein? So if I type dot schema, what I'm going to see is this SQL command Create table, if not exists, quote unquote favorites, which is the name of the table. Then in parentheses there are going to be apparently 3 columns, one of which is called Timestamp, the next of which is called Language, the third of which is called Problem, and each of those columns is going to be raw text. Now we'll soon see that it doesn't have to just be text, but when I use the dot import command. This is the default table that SQLite created for me. Soon we'll see that I can exercise more control, especially over the types of data that I'm putting in this database. But what's really nice about the import command is it could not be easier to convert a CSV file to a SQLite database so that now as we're about to see, we can use SQL on it instead of Python or any other language instead. OK, so how do we go about getting data from this database? Well, the first of our commands that we'll explore is that one called Select. So select data means to read data from the database, and in this sense it's going to be a declarative language because I'm just going to declare what data I want to select from the database, and I'm not going to worry about opening the file anymore or iterating. It with a 4 loop or a while loop or defining variables of the like, I'm just going to select syntactically what I want. So let me go back to SQLite here. Let me clear my terminal just to get rid of the past commands and let's do the first of these. Select star from favorites, and I regret to say the semicolon is back for the SQL code we're now writing. Enter and we will see a sort of Asky art version now so even better than the raw CSV file of all of the data that was inputted into this table. So select star from favorites is apparently selecting everything so the star in this context is a wild card of sorts. That represents all of the columns in the table. The table itself is called favorites, so I'm selecting all of the columns from the table called favorites, and here you have it with sort of simple Asy art first column, second column, 3rd column, chronologically listed because that's exactly how it was loaded into the database. All right, so if star is wildcard, what more can we do? Well, if you don't care about all of the columns, you can actually be a little more specific. So I could say instead select just the

language column from the favorites table, semicolon, enter, and now I have just a single column of data that shows me one cell for every submission but not the time stamp or the favorite problem that that person put in. Or if I want to declare that I want a couple of columns so I can say select language and problem. but I don't care about the time stamp from favorites as such, and now you get two columns instead. So in short, rather than write the dozen or so lines of code that we earlier did with Python to open the file and then iterate over it with a reader, we just select what data we want from this here database. But even more powerfully, SQL comes with a whole bunch of functions built in quite like the spreadsheet software that you and I are already familiar with in the real world like Excel and Numbers and Google Sheets. SQLite comes with an average function, account function, distinct lower min, uh, max min, upper case, and so forth. There's a whole list of them. We'll play around with just a couple of these if we want to transform some of this data. Let me go back into VS code, clear my SQLite terminal, and suppose I just want to get the total number of rows in the favorites table, like how many people at the moment in time I downloaded the file, even if not everyone had quite buzzed in yet. Did I end up with in that file? Well, I could say select the count of all of the rows from the favorites table, semicolon, and now I'll get back a single cell, which gives me 272 submissions had come in the moment I downloaded that file. Suppose I want to see just to confirm that no one submitted bogus data which languages were actually among those typed in. Well I can select only the distinct. Languages that were typed in from the favorites table and now I get a unique list of languages that everyone buzzed in with irrespective of how many times. If I want to maybe get um how many distinct languages there are, if it's not as obvious as 3 here, I could select the count of distinct languages from the favorites table, and that would just tell me the answer. 3 is the total number of languages that are distinct. In that submission, so again it's easy to just eyeball this, but very quickly with single statements that are sort of English like left to right is enabling me to just select the answers I want to some of these problems. Well, what more can SQL do? Well, here is a bunch of other. A keywords that we can add to our SQL commands that allow us to control further what kind of data we're going to get back. We're going to be able to group data by similar values. We're going to check for not not just string equality but for fuzzy matching, checking if something is close to a string that we're looking for. We can limit the total number of rows coming back. We can order or sort the data by a certain column, and we can actually have predicates, so to speak, using a where which is similar in. It to an if condition, but a little more succinctly written instead. So for instance, let me go back to VS code here. Let me clear my terminal again and let me go ahead and select how many of you answered C is your favorite language without selecting all of the counts again. Let's just hit the nail on the head. So let's select the count of rows from the favorites table where the language. Selected equals quote unquote semicolon, and I get back a simple answer 58 of you buzzed in with the answer C. How many of you liked both C and very specifically the problem called hello world if you sort of that was the extent of your sort of um the passion for for code, let's go ahead and select the count of star from favorites where the language you typed in equals quote unquote C. Uh, and the problem you typed in equals quote unquote hello world semicolon, and it looks like 5 of you said your favorite language was C and your favorite program was hello world. Great. All right, so it's getting a little more interesting. What about the other version of Hello World where we called it hello it's me? Well, that one's interesting because I think it's gonna break my convention of using single quotes, which would be convention here in SQL. Whenever you're using a raw

string, single quotes here would be the norm, but let's type this. Out so select count of star from favorites where language equals quote unquote C and the problem this time equals quote unquote hello it's me. So at a glance this is probably going to confuse SQLite 3 because does that middle apostrophe belong to the first one or the second one. is ambiguous and this is weird. In C we would solve this problem by putting a backslash in front of it and a so-called escape character. Different languages have different conventions. This one's a little weird, but in SQL light what you instead do is doubly single quote it. So putting two single quotes is the convention for escaping a single quote just because you got to remember or Google these kinds of things in the real. If you forget. Enter now I get back that. So it was not the case that any of you liked both C and that problem specifically. Well, what if we want to be a little more inclusive of either hello problem? Well, I could do this in this way, just like in my code spaces terminal. I can go up and down to go back through my history. Same thing in SQLite, so I can go back to commands to get up here. And let me go ahead and write something longer where the problem is hello world or the problem equals quote unquote hello, it's double apostrophe me single apostrophe semicolon and parenthesis so it's wrapped onto two lines here so it's a little messy, but I'm just logically saying where you buzzed in with C as your language and a problem of. A world or a problem of hello it's me E. It should be the same answer as before because none of you liked hello it's me, but I chose this syntax because I can actually make this a little cleaner. I can go and delete this whole parenthetical and just say we're language equals C and the problem is like quote unquote hello percent sign single semicolon. So this is a little weird too. It's just how SQL does this instead, but whereas previously I was using an equal sign to check for literal string equality, like literally those problem names like allows me to use wildcards, and it's not a wildcard quite like the previous use of the asterisk that we saw when you are using a Wildcard in a string in sequel you say percent sign to represent zero or more characters there so hello comma space percent is going to hopefully match this or the other problem that started with hello comma. So let me go ahead now and hit enter. The answer is still gonna be the same, but indeed it's demonstrative that that is how you could express yourself a little more generally if you wanted a pattern match like that. Questions now on any of these techniques, yeah. they have Uh, good question. Does it have to be capitalized when doing string equality, yes, but not with like. Like will tolerate case insensitivity, so upper case or lower case. 00, I see. Good question. So the capitalization, so stylistically in SQL, I would argue, and this is a stylistic convention in SQL, certainly for CS 50 and also for a lot of companies and communities in the world to uppercase your SQL keywords just to make them stand out from words that you and I. Chose is like the name of the table or the name of the columns they're in. This is just a convention. I would propose like always to be consistent, but for CS 50 and for Style 50's sake, I would propose that you indeed capitalize like this and frankly it just makes it easier to read to my eye because the SQL stuff jumps out and then the lower case stuff is specific to your data set. A good question. All right. How about another uh set of keywords that we saw on the screen earlier, namely grouping by. Well, suppose we have a data set like this, whereby, oops, we suppose we have a data set like this, whereby How does this go? Happy Halloween. Whereby here's just an excerpt from that table. So for as languages go, uh, say one of you liked C, 2 of you like 3 of you liked Python, and then now that we're introducing SQL, let's imagine that 2 of you now like SQL even better. So that's the extent of the data set. Wouldn't it be nice to be able to figure out how many of you like C or Python or SQL? Well, I could write some Python code,

open the file, iterate over it using variables, using a dictionary, and those what 20 or. Lines of code we wrote earlier to answer this question. Wouldn't it be nice to just ask the SQL language to figure out how many of you like C, how many of you like Python, how many of you like SQL? We can do this by grouping these cells by common values. Let's group all of the Python rows together and all of the SQL rows together, and even though there's just one, all of the Crows as well. So how can we do this? Well, let me go back to VS code here and clear my terminal, and let's do this. Let's select every language, but it's respective count as well from the favorites table, but before you do any of that group everything by language. So this one takes a little more practice and getting used to, but this is simply saying select all of the uh it's saying look at the languages, essentially group all of the common languages together and then figure out what count that gives you for all. Of the grouped rows. If I hit enter here, we'll get an answer just like the Python code that took me 20 lines of code to write earlier. What's really happening though in the database is something a little bit like this. Notice, of course, that there's only one version of C. There's then 3 versions of Python and there's 2 examples of SQL, and the table I'm essentially building is to group all of those by identical values and then spit out the total counts here. Now on the screen it's just 13, and 2 in the data set. With some 200+ responses we have much larger answers, including Scratch instead of SQL right here. But this now sort of speaks to just how much more convenient it is to, if you want to ask a question like that, especially if the data set is more than a couple 100 rows, if your boss, for instance, in the real world has a CSV data set and wants you to analyze the data, well, you can literally download it, import it into SQLite, run one command, and boom, like you've got this analysis done if the extent of it is just to group the data and figure out. Uh, what kinds of, uh, accounts you have in the data set. All right, what else can we do? Well, we can play around with this a bit more. Let me go back here into VS code and propose that we could. Uh, order those results more than just the uh the default way. So let's go ahead and select the language and the count from the favorites table yet again. Let's group by language yet again, but this time let's order by the counts column in descending order. So it's a bit more of a mouthful and it takes some practice to memorize all of the syntax, but when I hit enter now, I get back the same answers, but Python. is at the very top of the list. Now Countar isn't necessarily all that self explanatory and indeed it's a little annoying that I have to write out Countar here at top right as well as in the beginning. So it turns out SQL also supports aliases. So if you want to change the temporary name of the column to be something else like N for number, well then I can actually define an alias with the keyword as order by N at the end. Of this statement and then hit enter and get back the same results too. And so if it's not sort of implicitly clear already, each of these SQL select commands is essentially giving me back a temporary table. This is not being saved anywhere like now it's gone from the computer's memory once I've actually gotten my answer, but it's essentially returning a subset of the tables that do exist in the computer's memory because that's what the import command did for. Me, it loaded the whole data set into memory, and now I have these temporary tables that are just containing the answers to questions I care about. And if you only care about the top one language, well, there's a limit keyword too. I can literally just say limit one at the end of that exact same statement enter, and now I've got the single answer to my question, a single row saying Python was the most popular with 190 people selecting that. All right, for now, I think that's enough on select. There's a few more keywords, but it really is just a matter of composing these building blocks, questions though on these capabilities.

Fundamentally. No. All right, well, how about maybe inserting data instead? So here might be the canonical way to insert a row into a table in SQL. You literally say insert into then the name of the table, then in parentheses the one or more columns for which you have data, and then literally the word values, and then in another set of parentheses a comma separated list of the one or more values that you want to insert into those that. columns. So for instance, let me go back into VS code here and of course at the time we circulated this form a few minutes ago, we had not yet assigned problem set 7. But in problem set 7 is a problem called 50ville which let's propose might very well be someone's favorite in a week. So let's go ahead and insert that row now preemptively. Let's insert into the favorites table. Two columns language and problem. Why? Well, I don't really care to figure out what the time stamp is and the format thereof, so I'm just going to omit the time stamp altogether, but the values I'm going to insert for this new row are going to be are going to be quote unquote SQL 50 ville close close parenthesis semicolon. E. Nothing bad seems to have happened. Let me go ahead and select star from favorites just to see what my data set looks like now. And indeed at the bottom of the file or the bottom of the table, indeed there is that new row. But what's sort of noteworthy is that this isn't just blank. There's our old friend null, which is not a null pointer, it's the same word, literally NULL, and it refers explicitly to the absence of data. And this is actually a nice feature because if any of you have ever used like Google. Spreadsheets, Apple numbers, Microsoft Excel and thought about uh or looked at cells that are blank. Like what does it mean if a spreadsheet cell is blank? Does it mean like there's literally no data there? Does it mean that you just don't have the data there or it's missing in some form? Well, how do you address that? Well, maybe you put like N/A in English for like not available or something like that, but that's kind of hackish and if you use NA, that might mean that no one can actually type NA at the. Answer. And so what's nice about SQL and data and database languages more generally is that null signifies the conscious omission of data. It's not just a missing value, it's consciously not there. It's not just the empty string, quote unquote, for instance. So we might see different examples of that, but what's nice now is that I can distinguish null from other values and in fact if that is not a good idea to have any data in my data set. That is null for whatever reason like it just looks like bogus data. It would be nice to know who inserted that when, no problem. We can also delete data from a table in SQL, and I can delete from the name of the table where some condition is true. So for instance, if I want to delete that, I can do this in a couple of ways, but perhaps the simplest is to delete from. Favorites, where, uh, time stamp is null, semicolon. So is 2 is another SQL keyword here, and that will go ahead and delete only those rows where the time stamp is null. Enter. Let's do the same select command as before. Enter and voila, that row is now gone. Be very, very, very careful with delete statements. If I had foolishly done this, wanna guess what the results would be. It would delete everything and like you can Google around and see actual articles of like interns at companies who had way too much access to a company database executing something like delete from favorites because they forgot the predicate they hit enter too soon and boom, all of the data is now gone. So these are very destructive commands and just like in the real world if you don't have backups or versions of these same tables, the data can indeed be lost. So don't do that. Always have your wear and make sure your wear is correct. All right, well, let's go ahead maybe and um suppose let's claim that maybe 50ville is going to be a really popular problem among students, so much so that it becomes overnight everyone's favorite problem. Well, we can update the table as is. Here is the general syntax for

updating rows in a table. You literally say update the name of the table. The word set and then a bunch of key value pairs. The column that you want to update setting it equal to the value that you want to update it to where some condition is true. So what does this mean concretely? Well, let's say that we want to change everyone's favorite to SQL and 50ville. I could do this update favorites, set language equal to SQL, problem equal to 50 ville. Close quote semicolon, and this is where again it can be dangerous, but in this case I'm gonna go ahead and hit enter without any predicate to filter this. Nothing bad seems to happen, but if I now do select star from favorite semicolon, all of you would seem to like 50ville, and there is no going back to the previous version of the table unless I quit out of this and I import the whole CSV again maybe after deleting the data entirely. All right, so how do I get rid of all of the data? Well, if you want to delete from favorites for real now, enter select star from. Favorites, we can confirm that that was a bad idea. There's literally no data in the database anymore, but we can certainly restore from our actual CSV. So in short, we've got select, we've got insert, we've got update, we've got delete, we've seen. 8 albeit automatically generated by SQL Light 3, maybe we'll see Drop and actually we can see Drop now. So recall that if I do. schema, I can see all of the tables in this database. If I do drop table favorites, semicolon, and now again do do schema, now there is nothing in this database at all. So that's an even worse command to run unless you know and intend what you're doing. Questions then on these crud operations creating, reading, updating, deleting yeah here first. Why do you not do quotation marks around null? So null is a special symbol, and if you put quotation marks around it, you would literally be looking for the value NULL that maybe was the name of a language or the name of a problem or something literally in the CSV. We are looking for the absence of that data altogether, yeah. Really good question, because it's so easy to destroy data like this, are people actively backing up their data? Short answer yes, absolutely. Like all of CS 50's web apps and the like are automatically backed up on some schedule. Even then we have to decide what that schedule is, and if it's daily, for instance, nightly, we could lose up to like 23 hours, 59 minutes of data in some case, maybe companies would therefore version their data more tightly, like every 5 minutes, every minute, although that's going to consume a lot more space, but there already is this theme of trade off certainly in computer. You can also implement forms of access control. So SQL Light is lightweight. It has no notion of usernames or passwords. If you have access to the data, you can touch everything, but in the real world with commercial and open source software like Oracle and SQL Server and Postgrass and MySQL, you actually have usernames and passwords and specific permissions, so you can give users interns the ability to select data but not update or delete or insert data or any combination thereof. So there are defenses. Other questions on these here crud. Commands OK, let's go ahead and play with some real world data. So many of you might be familiar with IMDB, the Internet Movie Database, which is a great repository of data for movies and also TV shows and actors and the like. And within IMDB's website you can actually download uh TSV files, tab separated values of files that contain a lot of the data from that website. So we went ahead and did this. We then converted that TSV. Data into a whole bunch of SQL tables so that we can begin to play with it in the context of TV shows. However, let's start first with a question about how you could go about modeling data for TV shows themselves. So for instance, in advance, I also created a few different spreadsheets that just allowed me to play with how I might model data, real world data at that. So the Office is very popular. Uh, TV show, the US version here is uh the US version here starred Steve Carell and

others. So if I think about how IMDB or maybe just even little old me with a spreadsheet, might keep track of who starred in what TV show, well, I might just use a Google sheet like this, and in the first column have a title column where this is the title of the show like The Office, and then if it stars one person, I would put Steve Carell in the next column, but if. As a second star I might put Rainn Wilson or John or Jenna or BJ Novak here column by column by column, and I could just keep adding show after show after show after show one row per show and then however many stars that are in there. What might you not like about the design of this data though or what might start to look odd? Yeah, it's a little weird that we have star star star star star. Just this repetition has tended to be bad any time we're copying and pasting should rub you the wrong way. Other observations about it too? Yeah. Yeah, at the moment I've got 12345 stars, and there's certainly TV shows with fewer TV stars and more, and so, OK, I can add more columns. I can just keep saying star, star, star, but then it's going to be a very ragged data set, very sparse data set where there's gonna be a lot of blank cells for shows that have small casts, but then a lot of columns for shows that have large casts, so it just feels like mm, this should be rubbing you the wrong way. It just feels like it's gonna get messy, especially as the number of stars, let alone shows, gets. Larger. All right, well, another version of this, uh, data set that I put together is this instead. So I didn't like the fact that I was going to have an arbitrary number of columns based on the specific show in question. So here I scaled back and I just have a single column for title as before, but now a single column for star, and I decided that if a TV show has multiple stars, well I just put each of the stars' names and then to the left of them specify the show that they're in. Seems to be a little better and that I've solved some of the redundancy problem, but I've kind of just kind of like covered up the hole in a leaky hose and now another leak sprung up here, which is to say there's still a bad design. What's bad here? Yeah. Yeah, now I've got the office, the office, the office, the office, the office, and that too feels like I'm wasting space. If I manually type this in, odds are eventually going to screw up and one of these is going to be misspelled, which is going to break something somehow. So this too doesn't feel quite ideal. So the third and final version I whipped up to model this data, which is going to lead us to a similar design in an actual database looks a little more arcane but is the right way, at least academically, to do things, and we'll see technologically too this is going to be a big game. So here I now have a spreadsheet with 3 separate sheets. One is called shows, which is selected at the moment. Another is called People, which is not selected yet, and the third of which is called Stars. What am I doing here? Well, notice that in the show sheet I've still got the title column, but I've decided to give the office a unique ID, much like a Harvard student has a unique ID number, much like an employee in a company, probably. A unique employee ID. Similarly, have I given the office a unique identifier that happens to be the same as it is in IMDB. Meanwhile, for all of the people that exist in the world of TV shows, for instance, these 5 folks, I have their names as well as unique IDs for them, and those integers are unique to the people and no connection per se to the show IDs just yet. But the third and final sheet I've whipped up is going to be a sort of cross referencing sheet that allows me to associate shows with people, and at a glance this looks the most arcane of the three because it's just numbers, it's just integers, but if you recall from a moment ago that the office's unique ID was 386-676, well, that's how we associated that show with this person, which happens to be Steve Carell, and so forth. Now at a glance, not very useful. To me, the human, unless I do some fancy spreadsheet stuff like the lookups if familiar or the like, but this is a stepping stone to how proper databases do

actually store data. What I have done here is normalize the data by eliminating all redundancies except for maximally some redundant integers. And why is that? Well, integers, at least we know from our days in C, are going to be a finite length. It's going to be 32 bits, maybe 64 bits, but it's always going to be the same number of bits, and that's nice because any time you have Fixed number of bits, it lends itself to storing things nicely in an array or doing binary search because everything is a predictable distance apart as opposed to strings like Steve Carell or John Krasinski or the names might vary in length. These IDs for the title of the show and these IDs for the persons are not going to vary in length because they're all just integers. But of course this spreadsheet now much less useful because if I want to figure out who is in the office, well first I have to figure out what show this is, then I have to figure out what. Uh, person this is and this is and this is, but that's where SQL is again going to swoop in and allow us to solve this problem. And indeed SQL is one of the most common ways that web applications today, mobile applications today store any amount of data at scale. They are most likely not using simple CSV files. They are using SQL Lite or MySQL or Postgress or Oracle or other commercial and open source incarnations of SQL databases, and odds are IMDb might be using. The same as well. All right, so let's go ahead and do this. I have created in advance a file called shows.db that contains hundreds of thousands of rows from TV shows and TV stars and other data from IMDb itself, and in a moment we'll see a database that if drone as a picture looks a little something like this. There is going to be a people table, there's going to be a shows table. Going to be a star's table that somehow links the two. There's also going to be a writer's table and a ratings table and a genres table. So overnight this sort of escalated quickly from just favorites, which was a single table, to now a real world data set that has 6 tables. But here is the relational in relational databases as these arrows are meant to imply right now, there are relationships across these several tables. Case in point, here's people. Here and we'll see in a moment that a person in the IMDb world has an ID number, a name, and a year of birth. A show in the IMDb world has a unique ID, a title, the year it debuted, and a total number of episodes. But there's no mention of people in shows. There's no mention of shows and people. But for the arrows, there's going to be this third table here, stars that somehow links show IDs with person IDs, and this is. Where relational databases get really powerful because you can solve all of those redundancy concerns and actually enable yourself to select data much more quickly instead. But let's focus on something simple first. Let's focus just on the show's table, which pictorial might look a little something like this. So in just a moment I'm going to go ahead and reopen VS code, and I'm going to open up instead of favorites.db. I'm going to go ahead and open up. Uh, a file called shows.db which again I arrived with in advance. So if I open up with SQLite 3 shows.db and hit enter, I'm back at a SQL prompt. Let me go ahead and type schema shows just to show you what command created this here table, and it got a little more interesting already. Notice that the table is called shows and it's got 123. Four columns the ID for each show, a title for each show, the year it debuted for each show, and the number of episodes. There's also clearly some mention of types and some other keywords that we haven't yet talked about, but let's focus now first on just what the data is. The best way to wrap your mind around a new data set if someone hands you a SQL database or you've imported a CSV into a SQL database is just select some data. So select star from shows, semicolon. That's a lot of data flying across the screen. It's not very easy to see because some of the show names are apparently crazy long and so it's wrapping, but it's still going and going and going. I'm gonna hit

control C to interrupt it. Control C, as with our terminals in general as your friend, let's run that same command but just limit it to the 1st 10 shows. So there are the 1st 10 shows in the IMDB database of TV shows. So we've got 10 rows in this data set going back to it looks like the 1970s is roughly. Where their data set starts. All right, so here's the data we have in here. Well, how much is there? Well, let's go ahead and check. So select count star from shows semicolon, and now we're talking. There's 250,000 87 shows in this database. And if I do the same for people, select count star from people semicolon looks like there are 704,315 TV. Stars associated with this year data sets. So here too, the data is much more interesting and much more representative of real world data. How about the ratings? IMDb, if unfamiliar, is also a place where you can go to check the ratings from users as to whether something is good, a good show, a bad show, or anything in between. So let's dotschema ratings and I'll see that, yeah, there's this table called ratings that as we saw briefly on the screen, there's a show ID. And then a rating and then the total number of votes that contributed there too and again some data types and other syntax that we'll get to before long. But let me go ahead and just do select star from ratings limit 10 just to get a sense of what the data is. That's now what the data looks like in that table. So to a human at a glance, not that useful because you don't know what those show IDs are, but in a moment we're going to see how we can reconstitute this data by linking these tables together. By way of those IDs and actually get answers to questions. So among other things, a SQL database or a relational database more generally supports 1 to 1 relationships whereby a row in one table can map to a one row in another table. So it's this in contrast to one to many, for instance. So 1 to 1 means one row over here somehow relates to one row over here. Again, the relational in relational database. Uh, how might we go about. Uh, seeing this, well, first, here's a tour of the data types that SQL Light supports, whereas in C we had a somewhat similar list and in Python that list went away, at least with regard to explicit types. In SQL we're back to when creating our tables explicitly stating what the types of those columns are. So you have integers, you have numeric, which is more of a catch all for things like times and dates and other useful real world data. You have real numbers which are like floats with. points you have text, which we've seen already, and then you have blobs, which is a great name which stands for binary large objects. You can actually store raw zeros and ones like files in the database generally that's frowned upon to store files, but there's certain times where you do want to store binary data and not pure text. That's it for SQLite. There are only these 5 types in other commercial and open source SQL databases like Oracle and MySQL and Postress and the same names I keep rattling off, you have even more data types than these. So that's a the additional features you get by using other databases as well. There's a few keywords though that are worth noting in SQL. You can specifically say when creating a table that this column cannot be null. If you don't want time stamp, for instance, to ever allow for null values, you can literally specify when creating that table this column cannot be null. And if I try to insert data into that table with a null value as by not providing a time stamp, the insertion will fail. And so Here's where things are different from just writing Python code or certainly using a spreadsheet you can actually have built-in defenses so that you and no one else messes up your data by inserting bogus or blank data accidentally. You can further say that things must be unique, so every element, every cell in a column must be unique to ensure that you can't accidentally put two things with the same ID to Harvard IDs, to employee IDs that are duplicates. You can avoid that altogether. But more importantly. Relational databases support

these two concepts primary keys and foreign keys, and this is where the magic really starts to happen. A primary key is the unique identifier for a table. It is the column of values that uniquely identify every row, so it's probably going to be the show ID, the person ID, the Harvard ID, the employee ID, any time you have a value, often numeric, often integral. All that uniquely identifies rows, you simply call that a primary key. When that same ID appears in another table for cross referencing purposes, you refer to it instead as a foreign key because that same key is over there in another table, thus foreign but they refer to one and the same things in the context of the table in which it's defined, it's primary if it appears in some other table, it is now considered foreign. All right, so how can we make use of this? Well, let me go ahead and propose that we execute a few SQL commands as follows. If I wanted to start asking questions about ratings, I could do something like this `select star from ratings`, where the rating is maybe a good show, so let's call it 6.0 or higher, but let's just limit this to the top 10 shows that meet that threshold. Enter. So here I now have a temporary table that gives me 3 columns from the ratings table show ID, which is a for the moment useless identifier because I don't know what show it corresponds to, but the rating value and the number of votes that contributed there too. Well, how might I actually get to the shows that are actually highly rated at 6.0 or higher? Well, I don't need to select star. If all I care about is these top 10, I can whittle this same command down to just selecting the ratings. And now, sorry, sorry. Not the ratings. I can whittle this uh this table down to just selecting the show IDs. So this is the answer to the question What are the top 10 TV shows whose ratings are 6.0 or higher? Well, from the table, these are the 1st 10 that come back, how do I now select. The shows that correspond to these values. Here's where things can be done a few different ways. I could select everything I know from the show's table where the idea of the show is in the following set. I'm going to do a parenthesis and then just for readability I'm going to hit enter. The and angle bracket just means. Continuing my thought, it's not executing the command yet. What is the query I now want to run? Well, it's going to be a nested query. I can now do the same thing as before select the show ID from the ratings table where the rating is really good, greater than or equal to 6.0, but let's then limit the total number of queries to just 10. So here, just like in sort of grade school math, we have parentheses. So the first thing that's going to be executed is the thing inside parentheses. So this is going to get me every show ID from the ratings table that has a really good rating of 6.0 or higher. That's going to return to me a column of values. I'm. Then gonna say `select star from the show's table where the ID of the show is in that list of values but only show me 10 of those` is what I'm asking here. So what I should now see is much more useful data, namely the 10 shows that are highly rated enter. And indeed I get back these 10 shows, all of whose ratings are indeed quite a bit higher. If I want to only care about the title, that too I can do. So let's do this again. Instead of selecting star, let's select title from shows where the ID of the show is in the following parenthetical `select show ID from ratings where the rating is greater than or equal to 6.0`. Close my parenthesis, limit to 10. Enter. And I see the exact same thing, but just the nail being hit on the head. Just give me the titles of those top several shows. Of course I might want to, might be able to do this differently. In other words, here's the top 10 titles. Well, what are the ratings? Like that's why you go to IMDB or Rotten Tomatoes or the like. You want to see the actual ratings, not the titles or the ratings. Well, it turns out we're gonna need another technique to do that, namely an ability to join two tables and in fact just as a teaser for this. If we want to start playing around with some real data, here might be, for instance, excerpts from two tables. Here's the show's table at left.

Here's the ratings table at right, or a subset thereof. If I want to figure out what the rating is for a given show, wouldn't it be nice if I could somehow like line these two tables up together such that just like the tips of my finger, I line up this value with its corresponding value over here, a cross reference of sorts? Well, just for the sake of. And let me just kind of visually flip this around, though that does nothing technically underneath the hood. Let me just scooch them together now after highlighting the common values, demonstrate that, well, wouldn't it be nice to take the show's table and join it with the ratings table in such a way that those IDs all line up and we're going to have the ability to do just this. This is a lot already and this isn't the sort of cliffhanger I'd wanted to end on because who cares about joints, but it's going to be cool. But let's take our 10 minute Halloween candy break and come back in 10 for the next. All right, we're back. So recall where we left off was essentially here. We had these two tables, the show's table at left and the ratings table at right, and the motivation here was like how do we actually associate shows with their respective ratings because the ratings of course are not in the show's table. As an aside, they could be. And in fact, because this is meant to demonstrate a 1 to 1 relationship whereby every show has one rating, we could have just put the rating and the number of votes into the show's table, but we chose not to because uh you know I am. actually stores their ratings as a separate TSV file, and so what we tried to do for parity with that is only import into a ratings table the very TSV file that we had downloaded from them. But that too would be a solution there too. So at this point in the story we've got the shows table here. We've got the ratings table over here. We've noticed that there are commonalities. There are show IDs that appear in both tables and in fact, to use some of the new vernacular, this is the primary key, the ID column here. This is that same value. But in this context it's known as a foreign key because it's in some other table, but that's gonna be how we link these two things together. So how do we select for not just the Office but maybe every TV show, its respective rating? Well, let's go back to VS Code and at my SQL light prompt, let me go ahead and do this select star from the show's table, but let's go ahead and join the show's table with the ratings table. How do I want to join these two tables together? We'll do so on. The shows tables ID column being equal to the ratings tables show ID column and then go ahead and filter the results in the following way where the rating we care about should still be greater than or equal to 6.0, and let's only limit this to the top 10 results. So it's a bit more of a mouthful, but what I'm doing is selecting everything from the result of joining shows and ratings on. This column with this column and the rest of the predicate is as before. So join is going to do literally that. Join these two tables as I have prescribed. When I go ahead here and hit enter now that I have my semicolon, I get back a complete table containing everything from the show's table, everything from the ratings table with those unique identifiers lined up. Indeed, if you look at the primary key over here, the ID column 62614. Over here you have show ID which came from the ratings table 62614. So we've taken two tables and really joined them together, but we're only seeing a subset because I limited it to 10 such rows. Now of course most of this data doesn't seem very interesting if my whole goal is just to tell me what the ratings are for these shows. Well, let's go ahead and in code achieve this sort of result. Let's literally join these tables together. Let's get rid of the redundant. See altogether and then really let's whittle it down to just a title column and a rating column. So how do we do that? Well, in code I'm going to go ahead and select more specifically the title of every show and the rating of every show from the show's table, but I'm going to join it with the ratings table on shows. ID equaling ratings.show ID and as before, I'm

going to limit it to where rating is greater than or equal to 6.0. And 10 such results. Enter and now I have a nice simple temporary table that in one column has the titles of these shows and in the right hand side has the ratings of the shows, even though those two data sets were completely separate in two separate tables. Indeed, if we think back to where this data came from, what we've been focusing on is the show's table, and we've joined it with the ratings table. Here's the primary key for shows. Here's the foreign key for ratings, and by convention notice that we've adopted a certain. Uh, a certain approach. Anything that's called ID here implies that it's a primary key. Anything that's something underscore ID implies that it's a foreign key, and the convention we adopted, which is actually quite common, is if the table is called shows plural, we call the foreign key show singularco ID. Different companies, different communities will have different practices, but we've been consistent across all of these tables with our underscore and lower case conventions. Yeah. Is all generate and relate to each other. Really good question. How do all these IDs generate and relate to each other properly? Well, in our case, I have no idea. The Internet Movie Database people came up with these unique identifiers somehow, and we simply incorporating them into our data set. In practice, what they probably did and what you will do, for instance, in future problem sets when generating data is you just assign an arbitrary integer starting at 1, then 2, then 3, then 4, then 5, and you just let it auto increment all the way up and you let the database ensure that you never have duplicate values, yeah. Just to clarify for the dot dot dot and er symbols, that's only to like make it look better, right? Like there's no like. Correct. The angled bracket that you keep seeing is just the continuation prompt, which means I have prematurely hit enter deliberately because I want to move everything onto the next line so it doesn't wrap ugly on to multiple lines. It is not SQL syntax. It's specific to SQL Light 3, and it's just a continuation of the thought. That's all. Good, good observation. Yeah. Um, when you limit it to 10, is it showing how does it earliest ones. Good question. When you limit something to 10, for instance, which ones do you get, you just get literally the 1st 10 rows from the table. And so it will typically be ordered if you don't use the order by uh keywords, uh, in the same order from which it came from those tables. And so you're just seeing arbitrarily the 1st 10 that match that predicate, which is rating greater than or equal to 6. We have not ordered it by rating, so I'm not getting like the 10 shows necessarily. I'm just getting the 1st 10 shows that are greater than 6, and the point for that is just I want it to fit on the screen rather than see hundreds of thousands of answers. OK, so you might recall now that there were certainly other tables besides these. So let's see in the broader scheme, not just shows and ratings, but let's focus on genres, if only because genres is interesting because it's no longer a 1 to 1 relationship because of course why would a show have multiple ratings that sort of has its own rating, but a show could certainly belong to multiple genres. You could imagine a show being a. and a drama or a musical and a comedy or any other number of combinations of one or more genres and so the way we've chosen to implement that here too is with a separate table called genres, which is not perfect. There's going to be some redundancies here that we have not yet eliminated, but it does indicate that we can go ahead and have multiple such values. Associated with each and every show. So how do we get there? Let's focus just on this. Let's go back in just a moment to VS code and let's take a look at the schema for now genres. In genres we have the following a table called genres, which has two columns a show ID, which is an integer that cannot be null, and a genre which is text, which is also not be null. And now for the first time, let's actually use some of the vernacular we've introduced. Here we have an example explicitly in SQL that

specifies when creating this table that it shall. The show ID column shall be a foreign key that references the show's tables ID column, and admittedly I think the syntax for creating tables is a bit of a mouthful. Even I often have to read to look it up to remember the order of everything, but here we have the columns listed first and then these key constraints, foreign key referencing this primary key over here. And in fact, let's rewind to look at the show's table now to see from which from whence we came. So if I do dot schema. Of shows, which we've done before but waved our hand at it then, we'll indeed see that shows has a primary key called ID, which is an integer. How do I know that? Because the very last thing in the parenthesis says that the ID column in this table is a primary key. Then we see that the title is text can't be null. The year is numeric, which again I described as sort of a catch all for other real world numeric types that aren't purely integers or real numbers per se. Episodes is an integer. Both of those are. Apparently can be null because maybe IMDB just doesn't have that data for some older shows, but primary key is indeed specified here. And just for thoroughness, let me distinguish now genres from ratings. If I do dot schema ratings again, which we waved our hand at earlier, very similar in spirit to genres in that there's an ID column that somehow references the show's table, and then some other column here genre. In this case we had ratings and votes which were reals and integers respectively. But notice this one additional constraint here. I deliberately specified that show ID in the ratings table must be unique. That is to say, you cannot have the same show ID more than once in the ratings table. Why? Because I indeed wanted a 1 to 1 relationship, and it would not be 1 to 1. If there were multiple show IDs that correspond to one ID in the show's table itself, but genres we're going to allow that it can be duplicates and so we don't have mention of unique there. All right, so where does this get us? Well, let me go back into my terminal here after clearing all of that, and let's go ahead and just see the data to wrap our mind around it a little more. Uh, real. So select star from genres, limit 10 just to see the the 1st 10. All right, so it looks like there's some comedies, avengers, comedies, family action, sci-fi, and so forth. Well, let's go ahead and look up just one show's information. In fact, I saw this number this ID before. How about let's just look up this show? What is this adventure show? 63881. So select star from shows where ID equals. 63881 semicolon. OK, so this is the show called Cat Weasel from 1970, which had 26 episodes in total, and that was indeed its unique identifier. So that's all fine and good if I want to see something about that specific show. But as before, how do I associate Cat Weasel in this case with all of its genres? Well, instead of it being a 1 to 1 relationship necessarily, maybe Cat Weasel is not just an adventure, maybe it's also a comedy and a family. Show and indeed if I go back to the results just now, you'll see that 68111 indeed lines up with adventure, comedy, and family and then the ID changes to be about some other show. So how do I select these three answers to the question What genre is cat weasel? Well, for this we need to talk about one to many relationships and how we can get those back. Well, let's go ahead and do this now in my terminal. Let me go ahead and say, Uh, the following select genre from the genres table where the show ID equals just that 63881, which I'm now starting to memorize adventure, comedy, and family. So that's the answer to the question, but this certainly isn't the best way to do this where you have to like look up the unique ID for the show you care about, then copy paste it or. and type it out into this query just to get the genres. It would be nice to just ask all of this in one breath. Well, we can do this even though it's a bit more verbose. I'm going to instead this time say select genre from genres where the show ID I care about equals, and now I'm just going to hit enter so as to move this nested query inside of

parentheses, and I'm going to. Well, I don't know off the top of my head what the unique ID is for cat weasel, but I can ask the database select the ID from the show's table where the title of the show equals cat weasel, and this now obviates the need for me to memorize or copy paste that unique ID. I'll hit enter and close my parenthesis. Uh, I'm gonna go ahead then and say. Uh, semicolon, enter, and now I get back the exact same answers but without having to know or care about these numeric values and that's kind of the point here, even though the database itself, the actual IMDB website, needs to use these unique identifiers to store everything in the database we. Humans generally speaking should not know or care what these identifiers are. They're just meant to implement this notion of relationships, these cross references, and so here we see an example where you can ask the question you care about without worrying about any of the underlying numbers or even seeing them as a result. All right, well, what's really, how else might we go about doing this? Well, let me propose that we join these two tables and ask the question in a slightly different way. So here's an excerpt from the show's table. Here's an excerpt from the genre's table, and clearly we could do something like we did before for ratings where we could line these two up and kind of join them together just for the sake of discussion, let me flip these columns around though that has no technical significance, and now we can clearly see 63881. There and here. The difference though, because now this is a one to many relationship, is that it's not quite as simple as just joining the rows together. I need to kind of join it here and here and here, and the database can do this for you, albeit at some cost in redundancy. So what I'm going to observe is that these IDs are all the same primary key in this context, foreign key in this context. Well, I'm going to start to join them together here. But it's not possible to return a temporary table that's just outright missing data. You have to get the same number of rows and columns everywhere in a grid. So what the database is going to do if I do join these two tables together and they are participating in a one to many relationship with each other, it's going to duplicate the data that's necessary to sort of make every row look the same. Downside is it might indeed be taking up some additional space unless the database is smart and somehow using pointers or something. That underneath the hood to avoid the redundancy, but for my purposes this is actually quite nice because if I iterate over these rows as I could in Python, as we'll eventually see, it's just nice to have all the data you care about in each and every row even though it's clearly redundant, but the data is not being stored redundantly in the data, it's just temporarily being presented to me with this here redundancy. So what do I really want to have happen? Well, I really care about actually joining these two tables together and ultimately just getting back the title and the genre respectively, so let me go ahead and my VS code here and do select title. And genre from the show's table, but let's join it this time on the genres table on shows. ID equaling genres.ho ID. So that's quite the same as with ratings where uh the ID equals just for time's sake, 63881, which I know is cat weasel, but I could certainly use a nested query if I wanted to do this as before. Enter. And I get back cat weasel's 3 genres, and if I were to loop over this data in some kind of like Python code, I would have access to the title and genre with each iteration, which I claim is. Useful, but if I don't care about that and I just really want to select the genres, I can do this with joins too. Let me just select the genre from shows joining it on genres on shows. ID equaling genres.ho ID where the ID is cat weasel 63881. And now I get back just that answer. So in short, what have we just seen? One, you can join two tables together and whittle down the temporary table to just the data you care about. Or if you prefer, and if I scroll back up in my history here, you could take a fundamentally different

approach but still get the same answer of simply using a nested query. I would say as you learn SQL for the first time, I think it's quite often easier to just do multiple. Nested queries because you sort of work your way from the inside out taking sort of baby steps to the problem. If the problem in question is give me all of the genres for a specific TV show. Well, first I need to know because I know how the data is laid out in the database. I need to know the unique ID of the show I care about. fine, that's pretty straightforward and hence this inner query. Once you have that you can parenthesize it and on the outside now you can select. The question to which you really want the answer, which is what is the genre that lines up with that show ID one or more times. So in short, nested queries probably easier and certainly when learning for the first time, but quite powerful are these joined queries where this achieves the exact same result, especially if I were to generalize away the 63881 and do a nested query here. Sometimes you want joins, sometimes nested queries suffice. What Oh my goodness, how does SQL do all of these searches? What's its time complexity? We'll talk about that to the end of today. In the most naive implementation, SQL is essentially just doing linear search from the top of the table all the way to the bottom. However, we as the programmers are going to have the ability to optimize those queries so that the database can actually do something closer to binary search and in general we'll be able to achieve much better performance as a result. A really good question. All right, let's go back to the big. Uh, flow chart of this data set we've looked now at shows and ratings. We've looked at shows and genres. Let's now focus on the juiciest part, like the part that associates shows with people, that is who stars in what, thinking back now to what I was mocking up in the Google sheet at the very start, whereby I wanted to somehow be able to associate the Office with Steve Carell and John Krasinski and Jenna Fischer and so forth, the right way. And the right way I claim is going to be like this. Here's my people table, which has a primary key of ID and then the name of each person and their birth year if known. Then we have the show's table, which we keep talking about, which again has a primary key, a title and year and episodes thereof. And then the stars table is somewhat new now because now when it comes to people starring in TV shows, we have a third and final type of relationship. A many to many relationship. Why? Because it's certainly the case that one person can be in multiple shows, and it's certainly the case that some shows have multiple people, hence many to many. So this is the 3rd and final relationship where just to recap, ratings was 1 to 1, genres was one to many, and now stars is going to be many to many. All right, let's dive in. So these queries will be a bit more verbose, but again they're gonna follow this principle of sort of taking baby steps to the answer we care about. Let me go back into VS code here and suppose I want to find out everything about the office that we know. So select star from shows where title equals quote unquote the office semicolon. Well, that's interesting. There's a whole bunch of offices. There was the UK versions. A few other variants, but the one we're probably talking about with these stars is the one that started in 2005 with 188 episodes. That's the US version, in fact. So let me be a little more precise. Let me select everything I know from the stars from the show's table where the title equals office and year equals 2005, so we don't confuse our answers with the other versions of The Office. Now how do I go about selecting all of the people who starred in that. version of the office. Well, I already have an answer to the question of what is the ID of that version of the office because it's right there in front of me. And in fact, I can narrow my query more precisely. Let's just select the ID from the show's table where the title is The Office and the year is 2005 386676. Now I could lazily just copy paste that or memorize it, but we're going to

do this query more dynamically. I want to next though figure out who is in. That show. So if I have a show ID, I want to figure out who's in it, but how do I get to the people and the names of those people? I have to logically go through this cross referencing of the stars table. So here's where this query is going to be a bit meatier than the past ones and that we need to do a bit more work than before. All right, well, what's the work I need to do? Let me go ahead now. And do the following select all of the person IDs that are associated with this show ID. So how do I do that? Select person ID from the stars table where the show ID equals, and I could lazily copy paste this, but let's avoid that, where the show ID equals, let me now in parentheses do this, select ID from shows where title equals quote unquote the office and. And year equals 2005 and then close my parenthesis semicolon. So what am I doing? I'm taking a second baby step, if you will. The innermost query inside the parentheses is just again dynamically figuring out the unique ID of the office I care about. The outer query is now figuring out all of the person IDs associated with that show as per the stars table, and the stars table has only two columns show ID and person ID. That's how the linkage is done just with those integers. Enter. I now have a column of person IDs that are starring in that version of The Office, so how do I take this one final step if I really want to care about their names and not their random person IDs? Well, I could go ahead and select the name from the people. table where that person's ID is in the following set. So when I'm dealing with a single value, I just use equals for equality, but when I'm dealing with a whole result set, a whole column of answers, I use the preposition in in SQL instead. So where the person's ID is in the following data set. Well, let's do the same queries as before. Select all the person IDs from the stars table where the show ID I care about equals, because there's only one show I care about. I'm going to further parenthesize this select ID from shows where title equals quote unquote the Office and year equals 2005. Uh, I'll close my parenthesis. Enter. I'll close my parenthesis semicolon, and now from the outside in I've taken three baby steps. The innermost one just gets me the show ID. The second one in the middle gets me all of the related person IDs, and the last one is really the final flourish. Get me all of the names of these people based on those IDs. Enter and now we see all of the stars in this show beyond even the subset that we've been playing with visually on the screen. OK, that's a lot. Let me pause here and see if there's any questions. Yeah. This outermost query is what gives me the names, but that query needs to know the ID of the person whose name you want. So the middle query actually gets all of those person IDs, but to get those person IDs, I need to know the show ID. So the innermost query, this one, gets me the show ID of the office itself. All right, so at the risk of overwhelming, here are other ways you can solve the same problem, but I do claim that the nested selects is probably conceptually and pragmatically the easiest way. But let's also solve this problem by doing a few joins just so you've seen it. Actually, before we do a join, let's let's flip the question around first. How about all of the shows that Steve Carell has starred in besides The Office? So let me select everything I know from the people table where the name of the person. Equals quote unquote Steve Carell semicolon. All right, there seems to be only one Steve Carell in IMDB born in 1962. That's all nice and good. What I really care about is his ID, so I'm going to narrow this down to selecting just his ID. Now I could memorize or copy paste 136797, but don't need to do that. Let's just use this as part of a nested query. Let's now select all of the show IDs from the stars table that are somehow related. Need to Steve Carell's person ID. So where person ID equals, and I could copy paste this, but that's generally frowned upon. So let's not do that. Let's just set it equal to a nested query where

I do the same thing as before. Select ID from people where name equals Steve Carell, then close my parenthesis semicolon. All right, he's been in a lot of TV shows, but this is not useful because I have no idea what all of these integers are. So the final flourish, select the title from the show's table where the ID of the shows I care about is somehow in this parenthetical list. Well, what's that parenthetical list? Well, select the show ID from stars where the person ID equals Steve Carell's. What is his ID? Well, I didn't memorize it, so I'm gonna select ID from people where the name of the person I care about is Steve Carell. I quote unquote close these this parenthesis, close this parenthesis, semicolon, enter, and now I see all of Steve Carell's shows. And even though we're doing this in a black and white command line environment, think about what the actual IMDB is doing with both of these queries. If you go to IMDb.com and search for Steve Carell, even though there's going to be a lot of colors and pretty pictures and whatnot, you'll probably get in some form a list of all of Steve Carell's shows. Or if you search for the Office, you'll get a list in some. Of all of the stars they're in, I could claim then that if IMDb.com is using SQL, which it very likely is, but not necessarily, they are executing queries just like we did. And when you type into the search box something like The Office or Steve Carell, they're essentially just copy pasting your user input into a prefabbed SQL query that they wrote in advance so as to get you the answers that you actually care about. So this is how a lot of today's websites and mobile apps are actually working. The programmer comes up with sort of the template for the queries you might ask, and then you supply the actual data you're searching for. All right, how about now, as promised, a couple of other ways to implement these many to many relationships, uh, based queries, but by using joints. If I know I need to involve the show's table, the people table, and the stars table, I can actually do this all in. One breath without any nested queries. Select for me the title from the show's table, but let's join that on the stars table on shows. ID equaling stars.sh show ID. Uh, But let's additionally join the show's table on the following. Let's join it on people on stars.person ID equaling people. ID. In other words, if you know conceptually that you've got these 3 tables, you want to somehow combine them without using nested selects, just figure out how to line them all up. So again, I'm selecting from the show's table, but I'm joining it with the stars table. By lining up the shows, tables, primary key with the stars tables foreign key, and I'm lining it up with the people table by lining up the stars tables foreign key with the people table's primary key. I'm just kind of logically connecting all of the things I know to be related and lastly, let's just say where the name I care about equals quote unquote Steve Carell semicolon. It's a little slower for now and this speaks to the question that was asked earlier, how is the database doing this? Well, slowly, apparently by default unless we optimize it, I got back essentially the same results although there is some duplication as a result, uh, which alludes to the. Um, filling in blank of blanks that I alluded to earlier. But let me show you one other technique too. But again, I would encourage you certainly for problems at 7 to focus on nested queries when you can because they're a little conceptually simpler. If I care about the titles of those shows, I could select title from the show's table and the stars table and the people table all at once in one breath. But I want to do so where the show's table's primary key equals the stars tables foreign key. Uh, and the people table's primary key equals the stars tables foreign key, and the name I care about is Steve Carell. In other words, this is just a third way to express the exact same idea by doing implicit. Joins by selecting data clearly from all three tables as per this comma separated list of table names but telling the database with your predicate, the where clause, how you want to line all of those tables up. If I

hit enter here, cross my fingers, I should get back the same results as well, albeit with duplication, which I didn't see in the nested queries. OK, that too was a mouthful. Let me pause here for questions. Yeah. do that No the internal structure. Correct. In order to do this, you, as the programmer must know the internal structure of the database, which is quite often the case whether you created the database yourself or you work with a colleague who designed the schema for the database. That said, I think your question's hinting at sort of the challenge like I really need to know the underlying implementation details when really all I care about is the answers to my questions in code quite oftenly nowadays, um, there are object relations. Mappings whereby you can use ORMs for sure, whereby you can use libraries that they understand the underlying database schema you as the programmer do not need to because it figures out how to do all of the joints for you. So for CS 50 we're introducing everyone to the bottom up understanding of how these joints work, but that too can be easily automated because of those schemas. Yeah. That is Good question. Is indentation in SQL important? Technically no, but like with any of the languages we've talked about thus far, it is good for the humans and certainly good for the students in a context like this. Python of the languages we looked at is the most rigorous, whereby indentation very much matters and the consistency thereof SQL, I'm just trying to pretty print things to make it easy to grok visually. All right, so those last two queries were arguably kind of slow, whereas with my nested queries, I actually got lucky and just boom, I got the answer quite quickly. Those joins seemed to be a step backwards and that it was taking more time to get back the same data that I actually cared about, but that's something we can actually chip away at. It turns out that one of the other values of a relational database vis a vis something like a spreadsheet is that you can actually tell the database in advance how to optimize for certain queries. This is not the case for spreadsheets if you. Have a lot of data in Google spreadsheets or Microsoft Excel or Apple Numbers, tens of thousands of rows, hundreds of thousands of rows, millions of rows. Your computer's going to slow to a crawl, and at some point those software packages are just going to say sorry file is too big, and they're certainly not going to be terribly fast at searching the data. But with a SQL database and relational databases more generally, you are as much the architect of it as you are the user of it in this case. And so you can tell the database in advance if you want to. Optimize for certain queries like select statements. So for instance, let me go back to VS code here and just for the sake of discussion, let's time how long it takes to find all of the shows whose name is the Office. I'm gonna use a SQL like command called Timer and I'm gonna set it to on, and this is just now going to tell me for every command I run how long it took. I'm gonna now select everything from the show's table where the title of the show equals quote unquote the office. Semicolon Enter and that query took, let's say in real terms 0.042 seconds. That's crazy fast. Like it's less than a second. I mean it's truly a split second, so no big deal, but it's a fairly simple query. But I bet we could optimize even this. Now why would you want to optimize even queries that are already pretty fast? Well, if they're very commonly being executed, and I dare say someone going to IMDb.com and searching for The Office or any TV show, like that's the common case. People are looking for TV. Shows, movies, actors, and so forth, it'd be nice to use as little amount of time to answer those questions as possible. Why? One, it makes for happier customers and users because you're getting them the answer faster. Two, it saves you money because presumably if you've spent \$1000 for a server and that server has a certain amount of RAM, a certain speed CPU or brain, it can only do so many searches

per unit of time per second, per minute or the like. So wouldn't it be nice if all of those searches is faster. Using less time so you can handle not 1000 users at once but 2000 users or 5000 users all with the same hardware. So there's certainly upsides there. Well, how can I go about optimizing a query? Well, I can create my own index, another use of the create keyword in SQL, where I can tell the database to optimize for searches on a specific table and specific columns therein. I say create index, and then I come up with a name for the index, whatever I want. the name of the table that I want to index, and then in parentheses the columns that I want to optimize for. So what does this mean in real terms? Well, let's go back to BS code here and let me create an index called, for instance, title index, though the name doesn't matter, on the shows table using the title column. In other words, tell the database, please expedite searches on the shows tables title column. After all, that's what I just searched on. Enter. Now that took a moment, almost half a second, but that's a tab that's an index that only has to be created once. If I do a lot of updates and deletes, it might actually take a little bit of time over over the course of using the database to maintain that index, but for now that's a one-time operation creating the index. But watch what happens now. If I scroll up in my history and go to the exact same query as before, which previously took 0.042 seconds, which, yes, is fast but not nearly as fast as the new version. Which is 0.001 seconds instead. orders of magnitude faster so I can handle 42 times as many users on the same database, so to speak, than I could have previously just by building this index. So what actually is an index? Well, we come full circle to discussions in like week 5 of the class. So an index in a database is very often created using what's called a B-tree. This is not a binary tree. B. The tree is its own distinct structure that's very similar in spirit in that it's fairly shallow because most of the nodes have children, but it doesn't necessarily have two children. It might have more children and in fact, the more children the nodes have, the sort of higher up you can pull all of the leaf nodes and the shorter you can make the height of the tree. So this is just a generic representation of a B-tree, but what this implies is that when I am now searching for titles like The Office, the database doesn't have to do the. Fault behavior which is start at the top and use linear search all the way to the bottom. If it has proactively built up an index in memory thanks to my command, it now has a tree-like structure storing those titles that allows it to find in some logarithmic time, whether it's logbase 2 or some other base, the same data much more quickly, and that's how we went from 0.042 to 0.001 2nd instead in this case here. Questions then on these here indexes. No. All right, well, let's propose that we can combine some of today's ideas. It turns out that now we're getting to the point in the course where you're not just choosing between this language and another, you're generally using a suite of languages to solve problems. And indeed in the coming weeks of the class when we transition to web-based applications, you're going to use a bit of Python. You're going to use a bit of SQL, you're going to use a bit of JavaScript and two other languages called HTML and CS. You might be using like 5 different languages at a time just to build one application. Why? Because some of them are better for the job than others, and indeed that's the ecosystem in which real world software development is done. Well, to make this bridge, we have a version of the CS 50 library we're called for Python which has functions like Get string, even though it's not that useful because it's just like the input function, but get in and get float, but also in the CS 50 library. For Python, we have a module that specifically makes it easier to use SQL from Python code. After all, wouldn't it be nice if I could get the best of both worlds and implement like an interactive program in Python but that uses SQL to actually get back data? Or I can build a

website that allows people to search for TV shows or TV stars and actually get that data from. A database but use Python to generate the web pages themselves. Well, we have some documentation for this library here, but I'm going to go ahead and use it in real time to show you how much more easily you can solve certain problems by using each tool for what it's good at. So let's go back to VS code here. Let me exit out of SQL light and get back to my normal terminal, and let me go ahead and let's say, minimize. My terminal here. Uh, actually, let's go ahead and open up favorites.ie, which is where we left off before, and recall that in the last version of favorites.ie we had simply used a dictionary to go about keeping track of how many of you said Python or C or scratch. And when I last ran this program with Pythonofavos.ie, the answer looked like this. Now notice that it's not sorted alphabetically, otherwise C would be first, and it's also. Sorted numerically, otherwise C would be second. So it would be nice in Python to maybe exercise some control over this, but I stopped short of doing that before because it gets very annoying quickly. And by this I mean the following. Let me go back into VS code here and into favorites.ie. And if I wanted to sort by the counts here, I could do this. I could change my Loop from iterating for favorite in counts to favorite in sorted counts. So this is actually not too bad thus far. I can actually sort dictionaries pretty readily. So now if I run this, and let me make my terminal a little bit taller so we can see both the results, if I run the program now, you'll see that it's sorted alphabetically by key. So apparently when you use the sorted function in Python and pass it a dictionary. Can still iterate over all of the key value pairs in that dictionary, but it's been sorted now by key. So that's nice if that's to be my goal, but maybe that's not really my goal. And here's how alternatively I could sort by value the 190, the 58, and the 24. I can still use the sorted function, but I need to tell Python to use a key, a sorting key of the counts dictionary's gets function, and then if I run it again. I now see it's sorted by value, but darn it, it's now sorted in the opposite order. I see scratch at 24, then 58, then 190. If I want to reverse it, well then I have to go up here and add another named parameter. Reverse equals true. I can run it another time and now I get the result. I. Long story short, this is just very annoying to have to use that amount of code to actually answer relatively simple questions and this is why we did transition for much of today to a declarative language like SQL that just let me select what I care about in that data. So if I again I go back into my database version with SQLite 3 of favorites. DB, I'll maximize my terminal window. What did we do before? Well, we can select uh from the database, uh, select. Uh, let's see, favorite com count star from favorites group by, uh, favorite semicolon, whoops oh it's fine. Oh, sorry. What do we do? We do select language com count star from favorites group by favorite. Oh damn it, what happened? Oh, we deleted it. See this is why you don't use the delete or drop commands. So I'm not going to demonstrate this again, but recall before break that when we last selected this information, we used the group by command to actually group by the language in question, and we got back all the counts, but then we were very easily able to reorder things by actually just using order by and then doing something in ascending order or for instance descending order. Well, now let's actually combine these worlds of Python and SQL together to write first a program that does just that. But to do this, we're going to need to restore that database. So let's go ahead and do this. Let's remove favorites.db, which is just a file in my account. Let's go ahead and run SQLite 3 of favorites.db. Create a new version thereof. Let's now go ahead and change my mode as we did earlier in class to CSV. Let's now do import of favorites.csv into a table called favorites, and now let's do do quit. And when I do LS, OK, now it's back, favorites.db in addition to today's other files. Now let me go

ahead and run SQLite 3. Of favorites.db and just as a sanity check select star from favorites semicolon, there's all of the data back minus the addition and subtraction that we ourselves made earlier manually and let's go ahead and in SQL go ahead and do select language count star from favorites. And group by language, but let's order by count in descending order, and that's one of the last commands we ran with this file and there's the answer in a single line of code instead of some 17 lines of code plus or minus some white space here. Can we merge now these two ideas? Well, let's see how to do this. Let's go back into favorites. here and make a new and improved version of it that actually uses SQL and no dictionary, no for loop, no try accept or any of this. Instead, let's go ahead and from CS 50's own library, import a SQL function which will give me access to this functionality. Let's create a variable called DB by convention, but I could call it anything I want and set it equal to CS50 SQL function and pass to CS50 SQL function. The path to the database file I want to open. This is a little weird, but the syntax here is SQL Light without the 3, colon slash slash slash favorites. This syntax, otherwise known as URI, is going to allow us to use the SQL light uh protocol in order to open up favorites.db, which is the very file I was just experimenting with manually in my terminal. Here now is how I can execute a SQL query in Python using CS 50's library. Now, as an aside, even though this is indeed meant to be a training wheel, CS 50's library is just easier to use than a lot of the. World libraries that makes this possible. So because we spend so relatively little time on this, we're still using this training wheel for this. Give me a variable called rows because I want to get back all of the rows from this table that contain those languages and db. execute. The only function that's useful in the CS 50 library for SQL is this execute function which allows me to write literally a line of SQL like select language, count, star. Uh, from favorites, group by language, order by count, star. Uh, descending order just to make my life easier, I'm going to add that alias trick that we saw before. So as N to change the count to the variable n, and then here I can just do order by N instead. It's a little long, but notice that now I'm using SQL as a string that I'm passing as an argument to this DB execute function. So at the very end of this I've got to close my quote, close my parenthesis so as to use. One language in effect inside of another. Now assuming I do get back at temporary tables, rows with that line of code on line 5, let's do this for each row in rows, go ahead and do the following create a variable called language and set it equal to row, quote unquote language. Then create another variable called N for instance and set it equal to row, unquote N. and then let's just go ahead and print out language and N respectively. So what does CS 50's library do? It returns by design a list of rows. Each of those rows is a dictionary of key value pairs. So when I do 4 row and rows, this is just iterating over a list of values, and we've done that over the past couple of weeks. Inside of this loop, I'm just creating temporarily two variables, language and N, to show you that. Each row is indeed a dictionary, which means I can index into it using strings like quote unquote language and quote unquote n because those are the columns that I selected using this query up above. Strictly speaking, I don't even need these variables. I can just get rid of that and a little more succinctly just pass in row bracket language and then row bracket N instead. So let me go down to my terminal window here, exit out of SQL light, run Python of favorites.pi in this form, enter, and I get back, it would seem. The same exact answer, 190, 58 and 24 in this case. Questions now on this co mingling. Of languages All right, how about one final thing. Once we have the ability to like use Python, now we can in fact make things interactive. So for instance, let me close my terminal temporarily. Let me go ahead and now ask for some user input. So

after opening the database, let's do this. Let's ask the human using Python's input function, or equivalently CS 50s get_string function for their favorite TV show and store it in that same variable. Then let's do a SQL query that selects that data row equals db.execute. Select and let's see how many people selected this favorite problem rather than TV show. How about favorite problem from our favorites data set. So select count star as N from the favorites database where the problem in question equals. Well, now I need to put the user's inputs. I don't know what that is yet because they haven't typed it in yet, so what I'm going to go ahead and do is a placeholder and say favorite, close quote, and make this whole thing an F string. Then I'm going to go down here and I don't need to iterate because ideally I'm just getting back a single answer. How many people chose this problem as their favorite? So I'm going to say. That the row I care about is simply the first row. So rows is a list, so rows[0] is the first and only row in that list, and then let's go ahead and print out row N. Let's see the results here and then see what happens. Let me put some single quotes here and single quotes here. Let me open my terminal. Let me do Python of favorites.py. And I'll say hello world enter and as before at the start of class, 42 of you like that. However, this is not, not, not how you should ever write SQL code in Python. What could go wrong with this code? Nothing went wrong a moment ago, but what could go wrong? Yeah. The user input, how so? Injection True, I don't know what those are yet, but we're about to go there. What even more simplistically could go wrong by plugging in the user's input here? Yeah. Exactly. If I inputted the other problem we played with hello, it's me, where it was IT apostrophe S, that if interpolated right here is clearly going to confuse the single quotes such that who knows what's going to come back. Now in the best case, the code might just not work and I'll get some kind of error on the screen, which is not great for the user because the program's not going to be useful. There's no user friendly error message, but in the worst case, the user could do something incredibly malicious if you are simply blind. Blindly trusting user input and plugging their input into a SQL query that you yourself constructed. Why? What if the user types something crazy like the word delete or drop or update or any of those destructive commands that we saw earlier and somehow tricks your code into executing maybe the select, but then eventually an additional query like a delete. Maybe they type in a semicolon and then delete or a semicolon and then drop or something like that. This is. The biggest threat to taking user input and trusting it in the context of databases, and it's called, as one of your classmates knows already, what's known as a SQL injection attack. A SQL injection attack is the ability for an adversary or an unknowing user to somehow inject code into your database. A SQL injection attack then might look something like this in the real world. Here for instance is like the login screen. To GitHub.com, they do actually use SQL among other languages underneath the hood, I believe, not necessarily for this, but suppose they did. And when logging into GitHub.com, you're prompted for your username or email address, and then of course your password. Well, what if I know a little something about SQL and suppose for the sake of discussion GitHub is using SQL Lite, which they're not using because it's not meant for massive, large massive data sets like this, but suppose they are, and just to be malicious, I type in my username maillin@harbor.edu. But then I use a single quote and then da da. Well, the single quote is there, me being an adversary in the story, because maybe I can confuse their code by closing their quotes sooner than they intended, and we haven't talked about this yet, but it turns out that in SQL is the comment character. So it's like hash in Python or slash slash C. This in SQL means ignore everything to the right. That alone can be used fairly maliciously as follows. Here, for instance,

could be the code that GitHub is using underneath the hood, whereby they might have some Python code, and heck, maybe they're using the CS 50 library that executes this pre-made query. Select star from the user's table where the username equals this question mark and the password equals this question mark, passing in username and password, for instance. Uh, but if they are trusting the username and password I typed in and just plugging it right there, they could be vulnerable to indeed a SQL injection attack. For instance, this code we'll soon see is actually the right way to do it. But suppose they were doing it with F strings like I started to in my version of favorites. ie. Same thing, select star from users where username equals this username and password equals this password, and the little F here means here's a format string. What could go wrong? Well, let me actually. Paste in the mail at harvard.edu single text here. Notice that this single quote and this single quote are meant to surround the username and same thing for the password there. But watch what happens when I type in my data maillin at harvard.edu single quote. So this would seem to finish the thought prematurely, and then it says, and so that just means ignore everything else. And so the effect here is essentially to gray out all of that stuff because it's effectively been commented out. So what GitHub ends up doing accidentally in this case is selecting star from users where username is mailen@harvard.edu irrespective of what his password actually is. And if you assume that down here they've got some conditional logic like, well, if we get back some rows, that means that Malin is in fact a registered user, go ahead and log him in. We don't know what the code looks like, so it's you've just enabled anyone on the internet to log in as me or anyone else just by suffixing their input with a single quote. And and that's the least of our concerns. If we additionally went in there and maybe instead of we put a semicolon and then delete from users or drop users, we could cause massive havoc on their database. This happens all the time, even now in the current year, you can Google around and see examples of companies that have not used proper sanitization of user input, and it's not just the intern, it's like random people on the internet are accessing or destroying their data maliciously. So what is the solution to a problem like this? Well, one, do not use format strings in Python to simply plug in user input. But the more important lesson is never trust users' input. Either they're going to do something accidentally or they're going to do something maliciously, and you do not want that to happen. So the solution then is to use a library, almost always use a library. This is not a wheel you should reinvent yourself, and by library I mean something like this. If you instead. Use a library like CS 50s and you don't just use F strings, you'll see in a moment you use question marks. What will happen is this when the user goes and types in mail in at Harvard.edu single quote that's fine. Let them put weird scary characters like single quotes in their input. The library will take charge of escaping user input, so anything dangerous in their input will be changed from one single quote to two because we saw earlier today that that's how you escape a character, and that means that now what you have is an effect my username is apparently Malin@harvard.edu apostrophe, and that's my username. Well, that's obviously not a real email address. It's not a real username. This is just going to return false. No rows are actually going to come back. And the way to do this now in our favorites example analogously is in VS code here to actually go up into this execute line, don't use an F string, change the value of problem to be a placeholder instead, and then pass into this execute function. One or more arguments that will be substituted in for that question mark and this is not a CS 50 thing, this is a uh industry convention whereby you quite often use literally a question mark and that means that whatever this variable's value is will get plugged.

Into that question mark for you, but the single quotes will be added. Any dangerous characters will be escaped for you, and at that point you can trust that the user can type in anything they want. Your code is not going to break. You can see hints of this actually in the real world if you've ever gone to a website. And they tell you like oh you can't like for passwords for instance, like all of us probably intuitively know that you should have pretty long uh hard to guess passwords with letters and numbers and punctuation symbols sometimes websites very stupidly prohibit you from using certain punctuation symbols, which should drive you nuts because there's no computational reason that you have to put the onus on the user to sanitize their own input, but quite likely those websites have kind of learned. Part of this lesson, and they know some characters can be dangerous in sequel like semicolons or single quotes or the like, and they just don't want you to ever type those in even though there are solutions to this problem. Use a library that someone else smarter than you with more history of writing code than you has used that's open source so that many people have seen it and banged on it over the years so that this problem is not something you're vulnerable to. Questions then on what these here SQL injection attacks. Are all about, yeah. also telling them what you're using and so maybe that. Good point. So if by also telling people what characters they shouldn't use, you're leaking information because a smart adversary might know, oh well, if they don't want me using that symbol, they're probably using this language or this technology. Yes, no good comes from telling the world more information than they need to know. So that's another good paranoia to have. How about one other issue before we come full circle to the SQL injection attacks. There's another challenge with relational databases and with SQL uh itself, namely race conditions. This isn't so much a problem when I'm writing a, a little program here on my own computer. Uh, but when you're running SQL code on a database in the real world in the cloud where you have many different servers talking to that database and many different users, uh, talking to those web servers, as is going to be the case at Meta and Google and Microsoft and any number of popular companies nowadays, and even some of CS 50 zone apps uses centralized SQL databases where if multiple people are trying to do the same thing on them at the same time, submit their homework, run Check 50, we too are vulnerable to what are called. Race conditions. So what is a race condition? Well, the way I learned this back in the day when taking a course on databases and operating systems more generally was to think of a scenario like this. Maybe in your dorm you and your roommates have a little dorm fridge and you're both in the habit of really liking to drink milk, as the story was told to us. And so maybe one of you comes home from class one day and you get to your room, look in the fridge, there's no milk in there, and so you decide to walk across the street to CVS or some other store to get milk. Meanwhile, your roommate comes home from there. Class and opens the fridge and it's like, oh, we're out of milk. Let me go to the store too. And for the sake of the story they go to a different store altogether so that you don't run into each other and the problem solves itself. So now both of you are on your way to a store to get milk. Time passes, you both come home, one of you puts a jug of milk in the fridge, the other one gets home and it's like, Oh damn it, like we already got milk. I can't fit this milk in the fridge, or now it's too much milk. We don't really like milk this much. It's gonna go bad. Like a very bad outcome here, having too much milk is the moral of the story. But what's the stupid story. What's the, what's the real takeaway? Why did we find ourselves in a situation where we ended up with too milk, too much milk? You know what the other We didn't know what the other person was doing, and to really geek out on this, we

inspected the state of a variable that was in the process of being updated by someone else. And this is a thing in computing as far back as scratch. Recall with scratch, you could have multiple scripts running at the same time for a single sprite because Scratch in effect is multi-threaded. You can have a single sprite doing multiple. Things in parallel by having those multiple scripts. Similarly here, your room is sort of multi-threaded because you have two independent beings who can both go to the store to solve the same problem in parallel. The problem though is that if one is not aware that the other is doing that work already, you might make poor decisions. So in the real world, what should the first roommate have done after inspecting the state of the refrigerator and realizing, oh, we're out of milk. OK, call the other roommate or maybe more simply like put a note on the door or like maybe dramatically lock the refrigerator somehow and in fact that's a term of art in databases is to actually use a database lock so that if you are in the process of updating the value in the database, lock it so that no one else can inspect the value of that database and potentially make a poor decision. So when might this actually happen in the real world rather than the contrived milk example. So there are a lot of social media posts nowadays that are quite popular to this day, as of today, this is still the most popular Instagram post, for instance. And imagine when this was first posted hundreds, thousands, hundreds of thousands of. People might have all been clicking the heart icon essentially at the same time. Now Mehta, the company behind Instagram, presumably has lots and lots of different servers, but let's suppose for the sake of discussion they have a single database, which is not true, but the danger is still there even with multiple databases. All of these different web servers are talking to the same database, and suppose those those servers are using Python code and hey, the CS 50 library that might look a little something like this in order to decide how to update the total number of likes for an Instagram post. The first line of code running on meta servers. Might say this get these rows as follows. Execute a query like select the current number of likes from the posts table where the ID of the post is whatever it is 123456, whatever. Notice no SQL injection attacks possible here because I'm using the placeholder, not an F string. Then the next line of code running on MetaServer maybe just stores in a variable just to make the code more readable. The first rows likes column. So it's again it's the CS 50 library in the story. Rose is a list of dictionaries, so this is the first such element in the list, and this is the likes column in the column we just selected, the temporary. Lastly, what do we want to do? Well, we want a plus plus essentially that total, so we update the post table setting the number of likes equal to this question mark where the ID equals this question mark and we didn't see this already, but the CS 50 library supports indeed multiple arguments after the SQL string. I'm going to update the number of likes to be likes plus one, plugging in the same ID of that post. So in short, take on faith that it's quite common that in order to achieve one small goal like updating the number of likes stands to reason you might need to do 2 database queries or 3 lines of code. Now if these lines of code are executing on multiple web servers, you could certainly imagine that if people are hitting the like button pretty much at the same time, maybe one server is going to execute this first line of code and it's going to get its answer. Maybe there's 100 likes at this point in the story. And then just by chance. Another server, this line of code is also executed, but it too gets the same answer. There's currently 100 likes. Meanwhile, the first server in the story continues to do its execution of code such that it updates the number of likes from 100 to 101. But because the other server was essentially running the same code in parallel, it's going to make the same mathematical decision and update the number of posts, the number of likes from 100

to 1001. But at this point in the story, the number of likes should obviously be 10. 2, so we've lost data, and that's one of the dangers of a race condition is that you'll end up with an inaccurate result and for a company like Meta they don't want to go losing data like likes like this like that actually drives engagement and so forth and so like that's genuinely a technical, if not a business problem as well. So it's analogous to sort of the milk problem but actually at scale. So what's the solution? There's a bunch of different ways, but conceptually we just want to lock the database when this logic is being executed. such that when one server is updating the number of likes, no one else should be allowed to update the like count at the same time. Now that's a little crazy for someone as big as meta because you're really just serializing all of these likes and slowing things down. So there's more fine grain control nowadays, namely called transactions where you can essentially lock not the whole table and then certainly not the whole database, but just the row in question, for instance. And so you would use commands in SQL like begin transaction. And then execute the lines of code that you want, and then when you're ready to commit it, that is save it, you use a commit command, but if something goes wrong or you get interrupted, you can actually roll back the whole thing. And what this kind of code does in effect by using more verbose CS 50 and Python code like this is you can ensure that those three lines of code inside or technically the two database queries inside, will either both be executed or not at all. They will not be interrupted, and that's the fundamental solution to this problem analogous to putting a lock on the fridge or by leaving a note or calling your roommate, preventing them from making the same decision themselves. Questions then on these race conditions, the solutions again, even though this won't be germane for CS 50, simply using techniques like locks and what we called transactions. No. All right, then a final moment to end on. Uh, we would not be a computer science course if we didn't introduce you to a few pieces of CS Canon. Uh, here is a sort of meme that's circulated for years when it comes to like optical character recognition, OCR of like toll booths trying to detect your license plate automatically. This is someone trying to have a funny old time tricking the city into deleting their database altogether because if you're just scanning this off of someone's license plate in front of the car and just blindly plugging it in without sanitizing their input. Escaping their input with something like a good library, you might very well still drop the entire database. As an aside, something one did something similar to where I think they made their license plate null, NULL, which just confused the heck out of the system too because the programmers didn't understand why null was all over the place when lights were being run and whatnot. And lastly, a very famed character in the world of XKCD as computer science circles goes is this, so we'll end as we've done before on an awkward silence as you process this here canonical CS joke. Oh Now you two know who little Bobby Tables is. All right, that's it for week 7. We'll see you next time. All right, this is CS 50, and this is our lecture on artificial intelligence or AI, particularly for all of those family members who are here in the audience with us for the first time. In fact, uh, for those students among us, maybe a round of applause for all of the family members who have come here today to join you. Nice. So nice to see everyone. And as CS 50 students already know, it's sort of a thing in programming circles to have a rubber duck on your desk. Indeed, a few weeks back we gave one to all CS 50 students, and the motivation is to have someone, something to talk to in the presence of a bug or mistake in your code or confusion you're having when it comes to solving some problem. And the idea is that in the absence of having a friend, family member, TA of whom you can ask questions is to literally verbalize your confusion, your

question to this. object on your desk and in that process of verbalizing your own confusion and explaining yourself, quite often does that proverbial light bulb go off over your head and voila, problem is solved. Now as CS50 students also know, we sort of virtualized that rubber duck over the past few years and most recently in a form of this guy here. So in student's programming environment within CS 50, a tool called Visual Studio Code at a URL of CS50.dv, they have a virtual rubber duck available at available to them at all times. And early on in the very first version of this rubber duck, it was a chat window that looked like this and if students had a question, they could simply type into the chat window something like, I'm hoping you can help me solve a problem. And for multiple years, all the CS 50 duck did was respond with 12 or 3 quacks. We have anecdotal evidence to suggest that that alone was enough for answering students' questions because it was in that process of like actually typing out the confusion that you realize, oh, I'm. Doing something silly and you figure it out on your own. But of course now that we live in an age of chat GPT and Claude and Gemini and all of these other AI based tools came as no surprise perhaps when in 2023 the same duck started responding to students in English and that now is the tool that they have available, which is in effect meant to be a less helpful version of Chat GPT, one that doesn't just spoil answers outright, but tries to guide them to solutions akin to any good teacher or tutor. And so today's is indeed on just that and the underlying building blocks that make possible that they're rubber duck in all of the AI with which we're all increasingly familiar, namely generative artificial intelligence using this technology known as AI to generate something, whether that's images or sounds or video or text. And in fact, what we thought we'd do to get everyone involved early on is if you have a phone by your side, if you'd like to go ahead and scan this QR QR code here, and that's going to lead you. To a polling station where you can buzz in with some answers. CS 50's preceptor Kelly is going to kindly join me here on stage to help run the keyboard, and what we're about to do is play a little game and see just how good we humans are right now at distinguishing AI from reality. And so we'll borrow some data from The New York Times, which a couple of years back actually published some examples of AI and not AI, and we'll see just how good this this technology has gotten. So here we have two photographs on the screen. In a moment, you'll be asked on your phone if you were successful in scanning that code, which one of these is AI? Left or right? So hopefully on your phone here, if you want to go ahead and swipe to the next screen, we'll activate the poll here. In a moment you should see on your phone a prompt inviting you to select left or right. And Feel free to raise your hand if you're not seeing that, but it looks like the responses are coming in, and at the risk of spoiling, it looks like 70%+ of you think it is the answer on the right. And if Kelly, maybe we could swipe back to the two photographs in this particular case, yes, it was in fact the one on the right, maybe. Looked a little too good or maybe a little too unreal, maybe let's see, maybe a couple of other examples. So same QR code, no need to re-scan. Let's go ahead and pull up these two examples now 2 photographs, same question. Which of these is AI? Left or right? Left Or right Alright, I want to take a look at the chart, see what the responses are coming in a little closer in this case, but a majority of you think the answer is in fact left here, though 5% of you are truthfully admitting that you're unsure. But Kelly, if you want to swipe back to the photos, the answer this time was in fact a trick question. They were both in fact AI, which perhaps speaks to just how good this technology is already getting. Neither of these faces exists in the real world. It was synthesized based on lots of training data, so two photographs that look like humans but do not in fact exist. How about

one more this time focusing on text, which will be the focus of course underlying our duck. Did a 4th grader write this or the new chatbot? Here are two final examples same code as before, so no need to re-scan. And here are the texts. SA 1 I'd like to bring a yummy sandwich and a cold juice box for lunch. Sometimes I'll even pack a tasty piece of fruit or a bag of crunchy chips. As we eat, we chat and laugh and catch up on each other's day. SA 2. My mother packs me a sandwich, a drink, fruit, and a treat. When I get into a lunchroom, I find an empty table and sit there and eat my lunch. My friends come and sit down with me. The question now lastly is which of these is AI? 1 or 2? SA 1 or 2. The bars here are duking themselves out. Looks like a majority of you say SA 1. Let's go back to the text and some of you who, one of you who says SA 1 Y if you want to raise a quick hand, why SA 1, yeah. OK, and uh it's so SA2 looks more like you would write and kind of ask what grade you are in? A 5th grader. So is this a new 5th grader or not? The answer here, in fact, is that SA 1 is the AI because indeed SA 2 is more akin to what a 4th, or if I may, a 5th grader would write. And I dare say there are maybe some telltale signs. I'm not sure a typical 4th grader or 5th grader would catch up on each other's day in the vernacular that we see in SA 1. But suffice it to say this game is not something we can play for in the years to come because it's just going to get too hard to discern something that's a. I generated or not. And so among our goals for today is really to give you a better sense of not just how technologies like this duck and these games that we've played here with images and text work, but really what are the underlying principles of artificial intelligence that frankly have been with us and have been developing for decades and have really now come to a head in recent years thanks to advances in research, thanks to all the more cloud computing, thanks to all the more memory and disk space and information, sheer. The volume thereof that we have at our disposal that can be used to train all of these here technologies. So that their duck is built on a fairly complicated architecture that looks a little something like this where here's a student using one of CS 50's tools. Here's a website with which CS 50 students are familiar called CS50.AI where we, the staff, wrote a bunch of code that actually talks to what are called APIs, application programming interfaces, third party services by companies like Microsoft and OpenAI. Really have been doing the hard work of developing these models as well as some local sweets, some local sauce that we CS 50 add into the mix to make it specific, the duck's answers to CS50 itself. But what we've essentially been doing is something that with which you might be familiar in part, prompt engineering, which has started popping up for better or for worse on LinkedIn profiles everywhere, and prompt engineering really, it's not so much a form of engineering as it is a form of asking good questions. And being detailed in your question giving context to the underlying AI so that the answer with high probability is what you want back. And so there's two terms in this world of prompt engineering that are worth knowing about. So in CS 50 has leveraged both of these to implement that duck. We for instance wrote what's called a system prompt, which are instructions written by us humans, often in English, that sort of nudge the underlying AI technology to have a certain personality or a specific domain. Of expertise. For instance, we CS 50 have written a system prompt essentially that looks like this. In reality it's like a lot of lines long nowadays, but the essence of it is this. You are a friendly and supportive teaching assistant for CS 50. You are also a rubber duck, and that is sufficient to turn an AI into a rubber duck. It turns out. Answer student questions only about CS 50 in the field of computer science. Do not answer questions about unrelated topics. Do not provide full answers to problem sets as this would violate academic honesty. Answer this question colon. And after that

preamble, if you will, aka system prompt, we effectively copy paste whatever question a student has typed in, otherwise known as a user prompt, and that is why the duck behaves like a duck in our case and not a cat or a dog. Or a PhD, but rather something that's been attenuated to the particular goals we have pedagogically in the course. And in fact, those of you who are CS 50 students might recall from quite some weeks ago in week zero when we first introduced the course to the class, we had code that we whipped up that day that ultimately looked a little something like this, and I'll walk through it briefly line by line, but now on the heels of having studied some Python in CS 50, this year code that I whipped up in the first lecture might make now a bit more sense. In that first lecture, we imported OpenAI's own library code that a third party company wrote to make it possible for us to implement code on top of theirs. We created a variable called client in week zero, and this gave us access to the OpenAI client, that is software that they wrote for us. We then defined in week zero a user prompt which came from the user using the input function with which CS 50 students are now familiar, and then we defined this system prompt that day where I said limit your answer to one sentence, pretend you're a cat, I think was the persona of the day. And then we used some bit more arcane code here, but in essence we created a variable called response which was meant to represent the response from OpenAI Server. We used client.responses.create, which is a function or method that OpenAI gives us that allows us to pass in three arguments the input from the user, that is the user prompt, the instructions from us, that is the system prompt, and then the specific model or version of AI that we wanted to use. And the last thing we did that day was print out response.output_text. That's how we were able to answer questions like what is CS 50 or the like. So we've seen all of that before, but we didn't talk about that week exactly how it was working or what more we could actually do with it. And so in fact what I thought we'd do today is peel back a layer that we've not allowed into the course up until now and indeed you still cannot use this feature until the very end of the class in CS 50 when you get to your final projects, at which point you are welcome and encouraged to use VS code in this particular way. So here again is. Code for those unfamiliar, this is the programming environment we use here with students. And let me open up some code that was assigned to students a couple of weeks back, namely a spell checker that they had to implement in C. So I came in advance with a folder called speller, and inside of this folder I had code that day and all students had that week called dictionary.c. And in this file, which will not look familiar to many of you if you've not taken week 0 through 7 up until now, we did have some placeholders for students. So long story short, had to answer a few questions, that is, write code to do this to do this to do this to do, and one more. There were 4 functions or blanks that students needed to fill in with code, and I dare say it took most students 5 hours, 10 hours, 15 hours, something in that very broad range. Let me show you now how using AI you soon the aspiring programmers can start to write code all the more quickly, not by just choosing a different language, but by using these AI based technologies. beyond the duck itself. So what I've done here on the right hand side of VS Code is enabled the feature that CS50 disables for all students from the start of the course called copilot. This is very similar in spirit to products from Google and Anthropic and other companies as well, but this is the one that comes from Microsoft and in turned GitHub here, and it too gives us me sort of a chat window here and this is just one of its features. For instance, if I wanted to implement to get started the check function, I could just ask it to do that. Implement the check function. And uh how about using a hash table in C? I'm going to go ahead and click Enter. Now it's going to work. It's

using as reference, that is context, the very file that I've opened, which is dictionary.c here. Co-pilot in general, as well as a lot of AI tools are familiar with CS50 itself because it's been freely available as open courseware for years. What you see here doing is essentially thinking, though that's a bit of an overstatement. It's not really thinking, it's trying to find patterns in what the problem is I want to solve among all of its training data that it's. before and come up with a pretty good answer. So for today's purposes, I'm going to wave my hand at the chat GPT-like explanation of what to do that was appeared it right, but what's juiciest to look at here is on the left, if I now scroll down, is highlighted in green is all of the suggested code for implementing this here check function. Now it might not be the way you implemented it yourself, but I do dare say this has hints of exactly what You probably did when it came to implementing a hash hash table. And in fact, I can go ahead and keep all of this code if I like how it looks. Let's assume that's all correct there. It might be the case that I want to now implement the load function. So how about now implement load function, enter, as simple as that, and what data is being used? Well, a few different things. It says one reference, so it's indeed using this one file, but there's also what are called comments in the code with which All students are now familiar, these slash slash commands in gray that are giving English hints as to what this function is supposed to do. There's implicit information as to what the inputs to these functions, otherwise known as arguments, are meant to be, what the outputs are meant to be. So the underlying AI called copilot here kind of has a decent number of hits, hints, and much like a good TA or a good software engineer, that's enough context to figure out how to fill in those blanks. And so here too, if I scroll down now, we'll see in green. Some suggested code via which it could solve that same problem as well, the load function, and I dare say I've been talking for far fewer minutes than CS 50 students spent actually coding the solution from scratch to this here problem. So I'll go ahead and click keep. I'll assume that it's correct, but that's actually quite a big assumption. And those of you wondering like why have we been learning off all this if I could just ask in English it to do my homework for me, I mean there's a lot to be said for the muscle memory that hopefully you feel you've been developing. For the past several weeks, the reality is if you don't have an eye for what you're looking at, there's no way you're going to be able to troubleshoot an issue in here, explain it to someone else, make marginal changes or the like, and yet what's incredibly exciting even to someone like me, all of the staff, friends of mine in the industry is that this kind of functionality in AI amplifies your capabilities as a programmer sort of overnight. Once you have that vocabulary, that muscle memory for doing it yourself, the AI can just take it from there and get rid of all of the tedium, allow you to. Focus at the whiteboard with the other humans on sort of the overarching problems that you want to solve and leave it to this AI to actually solve problems for you. A fun exercise too might be to go back at term's end and try solving any number of the courses assignments. For instance, let me go ahead and do this. In my terminal window here. I'm going to go back to my main directory. I'm going to create an empty file called Mario.c that has nothing in it, and I'm going to go ahead in my chat window here and say please implement a program in C. That prints a left aligned pyramid. Of bricks using hash symbols for bricks and use the CS 50 library to ask the user for a non-negative height as an integer period. I dare say that's essentially the English description of what was for CS 50 this year problem set one to implement a program called Mario.C. This too is sort of doing its thing. It's using one reference. It's working, it knows. As a hint that this file is called Mario.c and it's seen a lot of those in its training data over time. There's an English explanation

of what I should do and those CS 50 students in the room probably recognize the sort of basic structure here of using a do-while loop to prompt the user for a height using the CS 50 library, which has been included, print a left of line pyramid using some kind of loop, and boom, we are done. And these are fairly bite-sized problems as you'll see as you get to term's end with your final project, which is a fairly open ended. To apply your newfound knowledge and savvy with programming itself to a problem of interest, it will allow you to implement far grander projects, far greater projects than has been possible to date, certainly in just a few weeks we have to do it because of this amplification of your own abilities. So with that promise, let's talk about how in the heck any of this is actually working. I clearly just generated a whole lot of stuff and that's how we began the story with the generation. Those images and those two essays by kids, but what is generative artificial intelligence or really what is AI itself? And these are some of the underlying building blocks that aren't going anywhere anytime soon and indeed have led us as a progression to the capabilities you just saw. So spam, we sort of take for granted now that in our Gmail inboxes, our Outlook inboxes, most of the spam just ends up in a folder. Well, there's not some human at Microsoft or Google sort of manually labeling the messages as they come in deciding spam or not spam. They're figuring out using code and nowadays using AI that looks like spam and therefore I'm going to put it in the spam folder, which is probably correct 99% of the time, but indeed there's potentially a failure rate. Other applications might include handwriting recognition. Certainly Microsoft and Google doesn't know the handwriting style of all of us here in this room, but it's been trained on enough other humans' handwriting styles that odds are your handwriting and mine looks similar to someone else's. And so with very high probability they could recognize something like Hello world here as indeed that same digital text. All of us are into streaming services nowadays, Netflix and the like. Well, they're getting pretty darn good at knowing if I watched X, I might also like Y. Why? Well, because of other things I've watched before and maybe upvoted and downvoted, maybe because of other things people have watched who like similar movies or TV shows to me, so that too. AI. There's no if, if, if construct for every movie or TV show in their database. It's sort of figuring out much more organically, dynamically what you and I might like. And then all these voice assistants today Siri, Alexa, Google Assistant, and the like, those two don't recognize your voice or necessarily know what questions you're going to ask it. There's no massive if else if that has all possible questions in the world just waiting for you or me to ask it. That too, of course, is dynamically generated. But that's getting a bit ahead of ourselves. Let's like rewind in time and some of the parents in the audience might remember this year's game, among the first arcade games in the world, namely Pong. And so this was a black and white game whereby there's two players, a paddle on the left, a paddle on the right, and then using some kind of joystick or trackball, they can move their paddles up and down, and the goal is to bounce the ball back and forth and ideally catch it every time, otherwise you lose a point. This is just an animated GIF, so there's nothing really dramatic to watch. It's going to stay at 15 against 12, just looping again and again. Nothing interesting is going to happen, but this is a nice example of a game that lends itself to solving it with code. And indeed it's been in our vernacular for years to play against not just the computer, but the CPU, the central processing unit, or really the AI. And yet AI does not need to be nearly as sophisticated as the tools we now see. For instance, here's a successor to Pong known as Breakout, similar in spirit, but there's just one paddle and one ball, and the goal is to bounce the ball off of these colorful bricks and you get more and more points depending on how

high up you can get the ball. All of us as humans, even if you've never played this old school game, probably have an instinct as to where we should move the paddle if the ball just left it going this way, which direction should I move the paddle. I mean probably to the left and indeed that'll catch it on the way down. So you and I just made a decision that's fairly instinctive, but it's been ingrained in us, but we could sort of take all the fun out of the game and start to quantify it or describe it a little more algorithmically step by step. In fact, decision trees are a concept from economics, strategic thinking, computer science as well. That's one way of solving this problem in such a way that you will always play this game well if you just follow this algorithm. So for instance, how might we implement code or decision making process for something like breakout? Well, you ask yourself first is the ball to the left of the paddle? If so, you know where we're going, then go ahead and move the paddle left. But what if the answer were no in fact? Well, you don't just blindly move the paddle to the right, probably, what should you then ask? Are you right below the ball. If the ball's coming right at you, you don't want to just naively go to the right and then risk missing it. So there's another question to ask. Is the ball to the right of the paddle? And that's a yes no question. If yes, well then, OK, move it to the right. But if not, you should probably stay exactly where you are and don't move the paddle. All right, so that's fairly deterministic, if you will, um, and we can map it to code using pseudo code in, uh, say a class like CS 50. We can say in a loop, well, while the game is ongoing, if the balls to the left of the paddle. Then move the paddle left. ET if the balls to the right of the paddle, sorry for the typo there, move the paddle right. Es just don't move the paddle. And so these decision trees as we drew it, have a perfect mapping to code or really pseudo code in this particular case, which is to say that's how people who implemented the breakout game or the pong game who implemented a computer player surely coded it up. It was as straightforward as that. But how about something like tic tac toe, which some of you might have played on the way in for just a moment on the scraps of paper, um. That you might have had, uh, here we have a tic tac toe board with two O's and two X's. For those unfamiliar, this game tic tac toe, otherwise known as knights and Crosses, is a matter of going back and forth Xs and O's between two people, and the goal is to get 3 O's in a row or 3 X's in a row, either vertically, horizontally, or diagonally. So this is a game. in mid progress. Well, let's consider how you could solve the game of tic tac toe like a computer, like an AI might. Well, you could ask yourself, can I get 3 in a row on this turn? Well, if yes, we'll play in the square to get 3 in a row. It's as straightforward as that. If you can't though, what should you ask? Well, can my opponent get 3 in a row on their next turn? Because if so, you should probably at least block their move next so at least you don't lose now. But this game tic tac toe is relatively simple as it is, gets a little harder to play when it's not obvious where you should go. Now all of us as humans, if you grew up playing this game, probably had heuristics you use, like you really like the middle or you like the top corner or something like that, so we probably can uh make our next move quickly. But is it optimal? And I dare say if back in childhood or more recently you've ever lost a game of tic tac toe, like you're just bad at tic tac toe because logically there's no reason you should ever lose a game of tic tac toe if you're playing optimally. At worst you should force a tie, but at best you should win the game. So think of that the next time you play tic tac toe and lose, like you're doing something wrong, but in your defense it's because. Question mark is sort of not obvious. Like how do I answer it when the answer is not right in front of me to move for the win or move for the block? Well, one algorithm you could have been using all of these years is called Mini Max, and as the name suggests, it's

all about minimizing something and or maximizing something else. So here too, let's take a bit of fun out of the game and turn it into some math, but relatively simple math. So here we have 3 representative boards. O has won here. X has one here, and the middle is a tie. It doesn't matter how we score these boards, but we need a consistent system. So I'm going to propose that anytime O wins, the score of the game is -1. Anytime X wins, the score of the game is a positive one, and anytime nobody wins, the score is 0. So at this point, each of these boards have these values -1, 0, and 1. So the goal therefore, in this game of tic tac toe now is for X to maximize its score because one is the biggest value available and O's goal in life is to minimize its score. So that's how we take the fun out of the game. We turn it into math where one player just wants to maximize, one player just wants to minimize their score. All right, so a quick, uh, sanity check here. Here's a board. It's not color coded. What is the value of this board? One, because X has in fact 1 straight there down the middle. So X is 1000 is -1, otherwise a tie. So now let's see how we go about with those principles in place, figuring out where we should play in tic tac toe. Now here's a fairly easy configuration. There's only 2 moves left. It's not hard to figure out how to win or tie this game, but let's use it for simplicity. O's turn for instance. So where can O go? Well, that invites the question, Well, what is the value of the board or how do we minimize the value of the board for O to win? Well, O can go in one of two places top left or bottom middle. Which way should O go? Well, if O goes in top left, we should consider what's the value of this board. Is it minimal? Well, let's see. If O goes here, X is obviously going to win, so the value of this board is going to be a 1. Now since there's only one way logically to get from this configuration to this one, we might as well call the value of this board by transitivity 1. And so O probably doesn't want to go there because that's a pretty maximal score and O wants to minimize. Over here though, if O goes bottom middle, well then X is going to go top left and now no one has one, so the value of this board is thus 0, we might as well treat this as 0 because that's the only way to get there logically. So now O more mathematically and logically can decide do I want an endpoint of 1 or an endpoint of 0? Well, 0 is probably the better option because that's less than 1, and thus it's the minimal possibility. So O is going to go ahead in the bottom middle and at least force a tie. And so that's where you. The evidence where if you humans are ever losing the game of tic tac toe, you have not followed that their logic, but you could probably do it if there's just 2 moves left. But the catch is, let's go ahead and sort of rewind to 3 moves left. Here there are 3 blanks, and I've kind of zoomed out. The catch is that the decision tree gets a lot bigger the more and more moves that are left. It gets sort of bigger and bushier in that it's essentially doubling in size. And with, and that's great if you have the luxury of writing it down on a piece of paper, but if you're doing this on your head while playing against a 1/5 grader, if I may, you're probably not drawing out all of the various boards and configurations trying to play it optimally. You're going with some instinct, and your instincts might not be aligned with an algorithm that is tried and true mini Max that will ideally get you to win the game, but at least will get you to force a tie if you can't win. But Tic Tac Toe is not that hard. I mean, how many different ways are there to play Tic Tac Toe? We could write a computer program to pretty much play Tic Tac Toe optimally. Um, we could use code like this if the player is X for each possible move, calculate the score for the board at that point in time, and then choose the move with the highest score. So you just try all possibilities mathematically and then you make the decision. Most of us in our heads are not doing that, but we could, else if the player is O, essentially do the same thing but choose the minimal possible

score. So that's the code for implementing tic tac toe. How many ways are there to play tic tac toe though? Well, 255,168, which means if we were to draw that tree, it would be pretty darn big and it takes you quite a bit of time to sort of think through all those possibilities. So in your defense, you're maybe not that bad at tic tac toe, it's just harder than you thought as a game. But what about games with which we might as adults be more familiar? Well, what about the game of chess, which is often used as a measure of like how smart a computer is, whether it's Watson back in the day playing against it or something else? Well, if we consider even just the first 4 moves of tic tac toe, whereby I mean black goes and white goes, and then they each go 3 more times, so 4 pair wise moves, how many different ways are there to play chess? Well, it turns out 85 billion just to get the game started, and that's a lot of decisions to consider and then make. How about the game of Go familiar? Consider the first four moves 266 quintillion possibilities. And this is where we sort of as humans and even with our modern PCs and Macs and phones kind of have to throw up our hands because I don't have this many bits of memory in my computer. I don't have this many hours in my life left to actually crunch all of those numbers and figure out the solution and so. So where AI comes in is where it's no longer as simple as just writing if, else's, and loops and no longer as simple as just trying all possibilities, you instead need to write code that doesn't solve the problem directly but in some sense indirectly. You write code so that the computer figures out how to win, perhaps by showing it configurations of the board that are a good place to be in that is promising and maybe showing it boards that it doesn't want to find itself in the configuration of because that's going to lead it to lose. In other words, you train. It but not necessarily as exhaustive, and this is what we mean nowadays by machine learning, writing code via which machines learn how to solve problems generally by being trained on massive amounts of data and then in new problems looking for patterns via which they can apply those past training data to the problem at hand. And reinforcement learning is one way to think about this. In fact, in fact, we as humans use reinforcement learning, which is a type of machine learning sort of all of the time in fact. A fun demonstration to watch here involves these pancakes. So in fact, let me go ahead and pull up a short recording here of an actual researcher in a lab who's trying to teach a robot how to make, how to flip pancakes. So we'll see here in this video that there's a robot that has an arm that can go up, down, left, right. This of course is the human, the researcher, and he's just going to show the robot one or more times like how to flip a pancake. And crosses his fingers and OK, seems to have done it well, does it again, not quite the same, but pretty good. And now he's going to let the robot just try to figure out how to flip that pancake after having just trained it a few different times. The first few times, odds are the robot's not going to do super well because it really doesn't understand what the human just did or what the whole purpose of. But, and here's the key detail with reinforcement learning behind the scenes, the human is probably rewarding the robot when it does a good job, like better and better it flips the more. It's rewarded as by hitting a key and giving it a point, for instance, or giving it the digital equivalent of a cookie or conversely, every time the robot screws up and drops the pancake on the floor, sort of a proverbial slap on the wrist, a punishment so that it does less of that behavior the next time. And any of you who are parents, which by definition today many of you are, odds are, whether it's not this or maybe just verbal approval or reprimands, have you probably trained children at some point to do more of one thing. And less of another. And what you're seeing in the backdrop there is now just a quantization of the movements X, Y, and Z coordinates so that it can do more of the X's and the

Y's and the Z that led it to some kind of reward. And now after you're up to some 50 trials, the robot seems to be getting better and better such that like a good human, we'll see if I can do this without embarrassing myself, can flip the thing. That's pretty good. That's pretty, I've been doing this a long time. OK. So we've seen then how you might reinforce learning through that kind of domain. Let's take an example that's familiar to those of you who are gamers. Any time you've played a game where there's some kind of map or a world that you need to explore up, down, left, right, maybe you're trying to get to the exit. So here simplistically is the player at the yellow dot. Here, for instance, in green is the exit of the map, and you want to get to that point and maybe somewhere else in this world there's a lot of like lava pits. You don't want to fall into the lava pit because you lose a life or you lose a point or there's some penalty or punishment associated with that. Well, we, with this bird's eye view can obviously see how to get to the green dot, but if you're playing a game like Zelda or something like that, all you can do is move up, down, left, right, and sort of hope for the best. So let's do just that. Suppose the yellow dot just randomly chooses a direction and goes to the right. Well, now we can sort of take away a life, take away a point, or effectively punish it so that it knows don't do that. And so long as the player has a bit of memory, either the human player or the code that's implementing this, just with a dark red line, that means don't do that again because that didn't lead to a good outcome. So maybe the next time the yellow dot goes this way and this way, and then, ah, I didn't realize that that's actually the same lava pit, but that's fine. Use a little bit more memory and remind me, don't do that because I just lost a second life in this story. And maybe it goes this way next time, ah, now I need to remember, don't do that. But effectively I'm either being punished for. Doing the wrong thing, uh, or as we'll soon see, being rewarded for doing more of the successful thing and just by chance maybe I finally make my way to the exit in this way and so I can be rewarded for that. Now I got 100 points or whatever it is, the high score. So now as for these green lines, I can just follow that path again and again and I can always win this game kind of like me nowadays, like 30 years later playing Super Mario Brothers because they can get through all the warp levels because I know where everything is because for some reason that's still stored in my brain. Is this the best way to play? Am I as good at Super Mario Brothers as I might think? What's bad about this solution? Yeah. Exactly, yeah, I've moved many more times than I need to and just for fun today, what grade are you in? 7th grade. Wonderful. So now 7th grade observation is like exactly that, that we could have taken a shorter path, which is essentially that way, albeit uh making some straight moves. And so we're never gonna find that shorter path. We're never gonna get the highest score possible if I just keep naively following my well trodden path. And so how do we break out of that mold? And you can see this even in the real world, another sort of. No example is I'm the type of person for some reason where if I go to a restaurant for the first time, I choose a dish off the menu and I really like it, I will never again order anything else off that menu other than that dish because I know it is good, but there could be something even better on the menu, but I'm never going to explore that because I'm sort of fixed in my ways, as some of you from the smiles might be too. But what if we took advantage of exploring just a little bit? And there's this principle of exploring versus exploiting when it comes to using artificial intelligence to solve problems. Up until now I've just been exploiting knowledge I already have. Don't go through the red walls, do go through the green walls. Exploit, exploit, exploit, and I will get to a final solution. But what if I just sprinkle in a little bit of randomness along the way and maybe 10% of the time as represented by this epsilon variable. I, as the

computer in the story, generate a random number between 0 and 1, and if it's less than that, which is going to happen 10% of the time, I'm going to make a random move instead of one that I know will get me closer to the exit. Otherwise I'll indeed make the move with the highest value. Now this isn't going to necessarily win me the game that first time, but if I play it enough and enough and enough and insert some of this randomness, I might very well find a better solution. And therefore be a better player, a better winner overall. If I just 10% of the time ordered something else off the menu, I might find that there's an amazing dish out there that otherwise I wouldn't have discovered. And so indeed using that approach, can we finally find a more optimal path through the maze, as was shorter there, presumably therefore maximizing our score and doing even better than we might have by just. Exploiting the same knowledge, so you can see this even in the game of Breakout, especially if you write a solution in code to play this game for you. Let me go ahead and pull up another video recording of an AI playing Breakout, and what this AI is doing is essentially figuring out, maybe more intelligently than you or I could, how to play this game optimally. And what we'll see here is that. Just like the pancake flipping robot, there's some notion of scoring and rewards and penalties here. So like right now the paddle's just doing random stuff. It doesn't really know how to play the game yet, but it realizes after 200 episodes that oh my score goes up if I hit the ball and it goes down equivalently if I miss it, and it's still a little twitchy, it doesn't quite understand what it's supposed. To do and why, but if you do it again and again and again and it's rewarded and or punished enough, you'll see that it starts to get pretty good and closer to what a good human might do. But here's where the algorithm gets a little creepy. If you let it play long enough or if you and I, the humans, play long enough, you might find a certain trick to the game. I dare say the AI becomes a bit scarily sentient. In that turns out if you're smart enough to break through that top row, you can let the game just play itself for you and maximize your score without even touching the ball, something that I do find a little creepy that I just figured out how to do that without being told, but it's just a logical continuation of rewarding it for good behavior and punishing it for bad behavior. So that next time you have an occasion to play Breakout, consider that kind of strategy as opposed to doing more of the work yourself, let the computer do it for you instead. Well, what else is there to consider in this world of AI in the context of machine learning? Well, there's specifically a category of learning that's supervised, and we've been using this for years. And in fact, our first example of spam early on was certainly supervised. Why? Because it was you and I who was like putting the email into the spam folder and to this day, maybe once a day I hit the keyboard shortcut in Gmail to say, Ah, this is spam. You should have caught this, and that is training Google's algorithm further, assuming it's not just little old me, but maybe thousands of people tagging that same kind of. Email as spam, that's supervised learning and that there's a human in the loop doing at least something so spam detection might be one of those. But the catch is that labeling data in that way manually just doesn't scale very well. That would be akin to having someone at Google or Microsoft labeling every email or someone at Netflix doing the same for all of the videos out there. It's expensive in terms of human power, and there's certainly problems out there with so much data. It's just not realistic for humans to label millions of pieces of data, billions of pieces of data. We've got to move to an Supervised model. And so this is where the world starts to consider deep learning, solving problems using code whereby you don't even have humans in the loop in quite the same way. And neural networks inspired by the world of biology are sort of the inspiration for what is the state of the art, even underlying today's

rubber duck and more generally these things called large language models like chat GPT, and the like. So here pictured somewhat abstractly is a neuron, and it's something in the human body that transmits a signal, say, from left to right. Electrically and if you have multiple neurons, you can intercommunicate among them so that if I think a thought, then I know how to raise my hand because some kind of message electrically has gone from my head to this extremity here. So that's in essence what I remember from 9th grade biology. But as the computer scientists, we sort of abstract all of this away. So instead of calling these two neurons drawing them as neurons, let's just start drawing neurons as these little circles, and if they have connective tissue between them of sorts, we'll just draw a straight line, an edge between them. So this is what a computer scientist would call. A graph. If you have two such neurons over here leading to 11 neuron here, you can think of this as being like maybe 2 inputs to a problem and now one output there too. We can represent the notion of problem solving, which is what CS 50 and intercourses more generally are all about. So let's solve a problem with a neural network without necessarily training it in advance, just letting it figure out how to answer this question. Here's a very simple two dimensional world, XY grid, and here are two dots, and the dots in this world are either blue or they are. Red, but I have no idea yet what makes a dot blue or red. However, if you train me on those two dots, I bet I could come up with predictions, especially if you let me label this world in terms of X coordinates on the horizontal, Y coordinates on the vertical, and then you know what we can think of this neural network very simply as representing the X coordinate here, the Y coordinate here, and the answer I want to get is quote unquote red or blue or 0 or 1 or true or false, however you want to think of the representation. So how do I get from a specific. XY coordinate to a prediction of color if I only know the coordinates. Well, up from the get-go, maybe the best I can do is just divide the world into blue dots on the left and red dots on the right, a best fit line, if you will, based on very minimal data. Of course, if you give me a third dot, it's going to be pretty easy to dize that I was a little too hasty. That line is not vertical, so maybe we pivot the line this way and now I'm back in business. Now I can predict with higher probability based on XY what color the next dot will be. You give me enough of these dots. I can come up with a pretty good best fit line. It's not perfect, but here's a hint at why AI is not perfect. But 99% of the time maybe I'll be able to predict correctly, and I can do even better if you let me squiggle the line a little bit and maybe make it more than just a simple slope. So what is it we're really doing with implementing this neural network, albeit simplistically with just 3 neurons? Well, essentially we're trying to come up with 3 values, 3 parameters an A, a B, and a C. And what do those represent? Well, really just a solution to this formula that there line we drew can be represented if you think back to like high school math with a formula along these lines whereby it's $ax + B \text{ times } Y \text{ plus some constant } C$ and we can just arbitrarily conclude that if that value mathematically gives me a number greater than 0, predict it's going to be blue. Otherwise predict it's going to be red. We can sort of map our mathematics just like with tic tac toe to the actual problem we care about by defining the world in this way. And so if you give me enough. Data points and enough data points I can come up with answers for that A, B, C, the so-called parameters in neural networks. Now in reality, neural networks are not composed of like 3 neurons and a couple of edges. They look a little something more like this, and in practice they've got billions of these things here on the screen, in which case pretty much every one of these edges represents some mathematical value that was contrived based on lots and lots of training data. And whereas I, the computer scientist, might know what these neurons over here

represent because those are my. Inputs 3 in this case, and I, the computer scientist, know what this one represents at the end. If you sort of took the hood off of this thing and looked inside the neural network, even though there'd be millions, billions of numbers going on there, I can't tell you what this neuron represents or why this edge has this weight. It's because of the massive amount of training data that that's just how the math works out. And if you feed me more data, I might change some of those parameters more. So the graph ultimately might look quite different, but my inputs and my outputs are going to be what I use to solve that problem. So if you want to predict like rainfall from humidity or pressure, you can have two inputs giving that one output an advertising dollar spent in a given month that might predict sales by just having trained again on such volumes of data. And when we get now full circle to something like CS 50's rubber duck and large language models like Claude and Gemini and Chat GPT, what's really happening, and this is all hot off the press in recent years, screenshotted here are some of the recent research papers that have driven a lot of this advancement in recent years. You have. From OpenAI say a generative pre-trained transformer, which is a lot to say, but there's the GPT in chat GPT, and essentially this is a neural network that's been trained on large volumes of textual information that gives us the interactive chat feature that we have in the class and we all have more generally in chatBT itself. So an example of what is actually happening underneath the hood of these GPTs. Well, here's a paragraph. That up until recent years was kind of a hard paragraph to end with the Massachusetts is a state in the New England region of the northeastern United States. It borders on the Atlantic Ocean to the east. The state's capital is. Almost anyone living in Massachusetts probably knows that answer, but if this AI has just been trained on lots and lots of data, there's probably a lot of people who say Massachusetts in part of a sentence, and then the answer, which I won't say yet, is in the other part of the. Sentence. But in this example, given that the question we're asking is sort of so far from some of the useful key words up until recently this was a hard problem to solve because there was so much distance. Moreover, there's these nouns that are being used to substitute for the proper noun like we suddenly start calling it a state, we call it a state down here, and it wasn't necessarily obvious to AIs that we're talking about the same thing as if it were just city comma state where you'd have much more proximity so. In a nutshell, what we now do, especially to solve problems like these, is we first break down a sentence or the training data or input a like into like an array or a list of the words themselves. We come up with a representation of each of these words. For instance, the word Massachusetts, if you encode it in a certain way, is going to be represented with an array or vector of numbers, floating point values, so many so that the word Massachusetts in one model would use these 1,536. Floating point numbers to represent Massachusetts essentially in an N-dimensional space, so not just an XY plane, but somewhere sort of virtually out there. And then, and this has been the key to these GPTs, attention is calculated based on all of that data whereby in this picture the thicker lines imply more of a relationship between those two words. So Massachusetts and state is inferred as having a thicker line, a higher attention from one word to the other, whereas our and our is and our thes have thinner lines because they're just not as much signal to the AI as to what the answer to this question is. Meanwhile, when you then feed that sentence like the state's capital is one word per neuron here, the goal is to get the answer to that question. And even here this is way smaller of a representation than the actual neural network would be. But in effect, all these LLMs, large language models are just statistic. Models like what is the highest probability word that it

should spit out at the end of this paragraph based on all of the Reddit posts and Google search results and encyclopedias and Wikipedias that it's found and trained on online? Well, the answer hopefully will be Boston. But of course 1% of the time, maybe less than that, the answer might not be correct and even CS 50's own duck is fallible, even though we've written lots of code to try to put downward pressure on those mistakes, and those mistakes are what we'll call. Lastly, hallucinations where the AI just makes something up perhaps because some crazy human on the internet made something up and it was interpreted as authoritative or just by bad luck because of a bit of that exploration 10% of the time, 1% of the time, the AI sort of veered this way in the large language model in the neural network and spit out an answer that just in fact is not correct. And so I thought I'd end for today on this final note. A poem with which many of us might have grown up from Shel Silverstein here about the homework machine which years ago somehow sort of predicted the state we would be in with these AI machines. He said, the homework machine, oh, the homework machine, most perfect contraption that's ever been seen. Just put in your homework, then drop in a dime, snap on the switch, and in 10 seconds' time, your homework comes out quick and clean as can be. Here it is, $9 + 4$, and the answer is 33? Oh me, I guess it's not as perfect. As I thought it would be This then was CS 50. See you next time.

All right, this is CS 50, and this is already week 8 and up until now, of course, in so many of our problem sets like we've been writing command line code like a black and white terminal window and everything is very keyboard based, very textual, but of course like the apps that you and I are using like every day are in the form of a web browser and on our phone. And so today and really for the rest of the semester we now transition to using all of the building blocks that we've been accumulating over the past few weeks, but to redeploy. Them in the context of web apps and for your final project, for instance, if you so choose, even mobile apps as well. So today we're going to understand how the internet that we use every day actually works. We're going to introduce you to a language called HTML, which is the language in which web pages are written, a language called CSS, which is the language with which web pages are stylized, and then lastly JavaScript, which of those is the only actual programming language. But even though we'll spend quite a little time on it, you'll see syntactically and Functionally it's very similar to C to Python and languages indeed that have come before. All right, so we use the internet every day. So what exactly is it? Well, in the simplest form, like we've got networks in the world, and networks are interconnections of computers, whether with wires or wirelessly. You have a network at home nowadays for the most part. You certainly have a network on a campus like this. In corporations you have networks, so interconnections of computers. As soon as you start networking the networks, if not networking the networks. Of networks you have in effect the internet. So this global interconnection of computers, servers, devices, and so many other things literally nowadays that we take for granted every day. But how does it actually work and where did it come from? Well, if we rewind to like 1969, the internet in its original form, really something known as ARPANET for the Advanced Research Projects Agency, a project from the Department of Defense that was really designed to interconnect what limited supercomputers we had. Back then that were otherwise geographically inaccessible to so many researchers and others, the internet or ARPANET really just looked like this with UCLA and just a few other nodes, so to speak, interconnected somehow. Just a year or so later, did we have Harvard and

MIT and others on the east coast, and if we fast forward now to today, of course we can find and route data most anywhere in the world. And in fact, the world is now filled with these things that are called routers, a router's just a computer, a. that routes data up, down, left, right, geographically and of course in the real world it might go out this wire here, out this wire here out this wire or out this wire. And in fact, just to make more real what we're about to be talking about when we talk about networks of computers and eventually the internet, we engaged some of our teaching fellows over the past few years to perform a little skit of sorts for us using Zoom, if you will, whereby each of the teaching fellows or humans you're about to see consider them as representing a router, a device on the. whose purpose in life is to route data, and what they're routing is what we're going to start calling packets, packets of information which metaphorically you can think of as just like a little white envelope like this that we use to send things via snail mail via the US Postal Service or beyond that internationally. So I give you in just 60 seconds or so what it means to send a packet on the internet, for instance, from Phyllis in the bottom right hand corner to a familiar face, Brian at top left. If we could dim the lights, if only to be dramatic. Thank you. Sure, we can clap for that. And we actually should clap for that because you're seeing the sort of final version which looks kind of perfect, but they were all smiling and clapping because it took us so many damn takes to like actually get the coordination of that correct. But for now assume that it was in fact correct, but notice what's among the takeaways from even that little skit is that the packet, the envelope from Phyllis to Brian could have taken any number of pads. It could have gone up and then to the left. It could have gone left and then up. It could have zigzagged and the like, and that's actually representative of how the world now looks because of so. wires and so many wireless connections, there's actually a lot of ways that data can travel from point A to point B, and it turns out it's not even necessarily going to be the shortest difference. It might be the least expensive distance, or perhaps just the result of how some humans or some how some servers have automatically configured the routes to get from point A to point B. So let's consider how the data is actually getting there. So long story short, all of those routers and indeed all the device. is on the internet, including the ones in your pocket or on your laps, speak a language, more technically a protocol nowadays known as TCP IP, and this is actually a pair of protocols which is a set of conventions that governs how computers behave on the internet. In the human world we have protocols as well. For instance, when I meet someone for the first time, I very often instinctively sort of extend my hand just sort of hoping that they too will extend their hand and shake, and that's a human protocol in that it governs how to people in that case. Communicate. Well, servers have the same kinds of protocols, but it's all text-based or bit-based instead of, of course, physical. But TCP and IP are two different protocols that solve two different problems. And let's focus on the last of them first. So IP, short for Internet Protocol, is simply a protocol that decides to give all of us a unique address in the world. In other words, there are these things called IP addresses. It's a numeric address that literally every computer. In the world has in order to uniquely identify it. Case in point in the real world, we have addresses too. For instance, in this building here, Memorial Hall, we're at 45 Quincy Street, Cambridge, Massachusetts 02138 USA and theoretically that unique identifier should get an envelope in the physical world to this location from any other in the real world. IP, as applied to the internet, just means that similarly do devices, Macs, PCs, phones, and everything else on the internet. Have a unique identifier as well known as an IP address. It's a number, but it's typically formatted in dotted decimal

notation, so to speak, so it's something.somethingsomethingsomething. And just as a bit of trivia, each of these number signs represents a value from 0 to 255. So there are 4 such values apparently, and just doing some quick we zero math, if each of those values can be 0 to 255, how many bits is an IP address, presumably. So 8 bits per number and how many was this? So 32 bits, because if you're counting from 0 to 255, well, it's 256 total possibilities, that's 28, which means 8 bits, 8 bits, 8 bits, 8 bits. So IP addresses are 32 bits, little trivia that's germane only insofar as it does kind of limit how many total devices we could seem to have in the world. If you've got only 32 bits, how high can you count? Roughly 2^{32} power, which we generally ballpark as 4 billion, which is to say you can have 4 billion devices total, it would seem on the internet, which is a big number, but there's also a lot of humans nowadays, and odds are most everyone in this room has at least 2 devices to their name, maybe a phone and a laptop with which you're taking the course, maybe even more devices thanks to the internet of things like smart home devices. We have so many IP addresses being assigned to things. So long story short, the world is gradually transitioning from this version here, IPv4 to IPv6 which instead of using 32 bits is actually using 128 bits, which is crazy large and gives us more than enough IP addresses for the foreseeable future. To be fair, we've been talking about this for like 20-30 years, transitioning from V4 to V6, and it's still gradually in motion. But for simplicity in the class and in general, we'll still use IPv4 if only because it's a little easier to wrap your mind around. Now this is admittedly a pretty arcane diagram, but this is the diagram Ay art if you will, that's in the Of official specification of what we mean by an IP datagram. More colloquially, this is what a packet actually looks like. Now what are we looking at? Well, you're just looking at like a grid of bits. So this here represents 32 bits total, where this is bit zero, and that's bit 31, 0 indexed all the way over there, and then each row represents 32 more bits, 32 more bits, 32 more bits, which is to say anytime a computer like Phyllis sends an envelope of information on the internet. It contains at least this information, a whole bunch of bits broken down into bytes. Now the only ones we'll really care about today are this one here, source address, which is to say when Phyllis sends that packet, she writes her source address, her IP address, something.somethingsomething on the outside of the envelope, so to speak, and she also puts Brian's IP address, whatever that is, something else, something else that's something else, do something else on the outside of the envelope as well. There's a whole bunch of other bits involved which are useful, but we'll wave our. at those for today, but that really speaks to what's actually happening. And if we do this metaphorically in the real world, it's kind of like taking out that envelope. And for instance, if Brian's IP address is 1.2.3.4 for the sake of discussion, Phyllis in advance of our filming that bit would have written something like 1.2.3.4 in the middle of the envelope just like we would in the real world. But presumably she wants Brian to be able to reply to acknowledge receipt or send his. own message. So she's also going to put her IP address, for instance, in the top left corner of the envelope 5.6.7.8 for the sake of discussion so that Brian knows when he writes out his own packet of information how to actually or to whom to reply. But at the end of the day it's all just bits being sent in a specific pattern and there is formal documentation. It's the order in which all of those bits will actually be sent out on the wire or wirelessly. So in short, IP ensures that all of us have unique IP addresses via which data can go from us or to us. That's only one problem nowadays, of course, servers can do so many other things. They can do email and chat and video conferencing, game servers, and who knows what, and it would be nice if a single server certainly could do multiple things. And in fact, that's

very much the case. Single servers nowadays, and a server is just a term of art for a computer used to serve information to other people. By contrast, our Laptops or desktops are generally clients because they only serve one of us, not multiple people, but these are just terms of art we're describing at the end of the day still computers. IP only ensures that we can uniquely address computers on the internet, but there's another protocol in TCP IP, namely the TCP portion that allows computers to uniquely identify services that they're offering to the rest of the world. So for instance, TCP allows us to Allow a computer to distinguish whether it has received a packet that's an email or receive a packet that's a chat message or a piece of a video conference or the like, which is to say there's more than just IP addresses on the outside of these envelopes. There are also what are called port numbers as well. Similarly, numeric values that are usually in the range of like 0 to 10 on up and the low thousands, and they're standardized. For instance, if you are requesting a webpage using HTTP and. Slash with which all of us are presumably familiar, unbeknownst to you on the outside of the virtual envelope that your computer subsequently sends is the port number 80 because when the server receives that it knows, oh, this human is requesting a web page and not for instance their email or something else. Or nowadays if you're using HTTPS where the S denotes secure in the URL, you're actually using port 443, which is just an arbitrary number that a bunch of humans in a room decided on years ago to standardize what goes on. The outside of an envelope. So just to be more clear then, when Phyllis is sending a request to Brian, and if Phyllis, for instance, is the client, just a human using a computer, and Brian in this story is now a web server, better yet, a secure web server that's somehow encrypting or scrambling the information to keep it secure, well, on the outside of this envelope after Brian's IP address, which was 1.2.3.4, Phyllis is also going to write the number 443, so that when Brian receives an open This envelope, he knows what he's looking at a request for a web page and not an email or a chat message or something else. Moreover, we can continue the story just a little bit further. Phyllis also writes on the envelope not only her IP address 5.6.7.8, but some number as well in that top left hand corner, whatever it happens to be, which is a port number via which Brian can reply to her. In this way, Phyllis can in effect have multiple tabs open, be using Zoom and uh. Some chat software or something else running multiple programs on her computer and the internet packets are all coming in, but her computer knows to which tabs or applications those packets belong. So if you really want to geek out, here's what this thing looks like. This is just the sequencing of bits for TCP as well, which is to say in addition to the dozens of bits we looked at a moment ago that standardize what IP is putting on the outside of the envelope, TCP is adding 16. bits that specify a port number, which means you can indeed have tens of thousands of possible port numbers, a destination port number, and a bunch of other stuff including this so-called sequence number, which happens to be a 32 bit value which is actually pretty important because quite often when sending messages on the internet they're pretty large and it would be nice if one person downloading a big image or one person downloading a movie or streaming a movie doesn't mean that no one else on the internet can do something else at that moment. Time. So for the sake of discussion, suppose that this very happy cat here is a very large JPEG, for instance, a very large graphical file. It would be nice, let's say that if Phyllis is trying to send or receive an image as large as this, it's not just in one massive envelope that's going to prevent a whole bunch of other users from similarly using the internet at that moment in time. So at the risk of a bit of heresy, we can actually tear this cat in half and fragment it really and then inside of fill.

This envelope or equivalently Brian's reply, depending on where this cat is coming from or going to part of that cat can go in this envelope and now say in the bottom left hand corner of this envelope, Phyllis or Brian could write the sequence number in question 1 out of 42 out of 43 out of 44 out of 4, so that when this and hopefully the other packets arrive at their destination, the recipient's computer can check, OK, this was a really big file in this case, do I have all of the Parts, yes, it can be inferred from the so-called sequence number, which we've represented there in that memo field of the envelope. There's a bunch of other stuff that can go on here too, including prioritization of data as well, but ultimately TCP just allows servers to handle multiple types of services and also allows it to receive data reliably, because if, for instance, a recipient only gets 2 out of the 4 packets or 3 out of the 4 packets, the fact. That there's a sequence number involved is enough information for that recipient to say to the sender, Hey, I'm missing 1 or 2 or 3 or more packets. Please resend them. So in short, TCP guarantees delivery by just doing some bookkeeping on the outside of these envelopes. So in short, IP allows us to uniquely identify computers, and TCP guarantees delivery and allows us to multiplex, so to speak, among multiple services on the same device. Questions on the this jargon. Thus far, because it is filled with acronyms, unfortunately. Questions IP TCP. Or anything else, OK, so seeing none, uh, as promised, let's do yet another acronym. So it would be pretty tedious if Phyllis and Brian and all of us humans had to write actually IP addresses into our browsers when visiting websites. And in fact, when most of us never do that, instead we go to Google.com or Harvard.edu or actual domain names, so to speak, which were so much easier for us humans to remember than these ar. IP addresses that are either automatically assigned to computers or manually configured by humans configuring servers, but there's another acronym in the world and there's another technology used on the internet, namely DNS for domain name system, and this is just a certain type of server that every home has, even if you didn't know it. Every campus has, every company has. There are so many DNS servers around the world, but their purpose in life quite simply is to translate what you and I know as domain names like Google. Harvard.edu, and the like into their corresponding IP addresses. And so in short, inside of these DNS servers are essentially like a two column table or spreadsheet, however you want to think about it, whereby here is all of the domain names in the world. Here are all of the corresponding IP addresses in the world. And so when your Mac or PC or phone being used by you is trying to access Google.com or Harvard.edu, that device, certainly when it's first booted up, has no idea what IP address. The IP address is for that server. It's not the case that Apple or Google are pre-installing billions of IP addresses inside of our devices, but your device is smart enough to ask the local network at home, on campus, or at work, well, what is the IP address of Google.com? What is the IP address of Harvard.edu? and then what your Mac, PC, or phone actually do upon getting that answer from one of these local DNS servers is it writes the corresponding IP address on the outside of that envelope. So it's a wonderful. A useful service that just makes the internet more useful for you and I to use because we can use names instead of IP addresses as well. Technically these things are called fully qualified domain names. Where do they come from? Well, some of you might actually have your own personal website. You might have gone through this process. It's actually not that hard to get your own domain name. You can go to any number of what are called internet registrars and pay them some money, and it's essentially on a rental basis, so you rent a domain name for a year or maybe 3 or 5 years at a time. can automatically bill you the domain name might be as little as

\$1 per year or thousands of dollars per year depending on whether someone has scooped it up and is maybe squatting or the like, but all you do ultimately is pay someone money and they give you the rights to use that domain name. And then what you do technically is you configure some DNS server somewhere in the world to know what the eventual IP address is for your server that's going to serve up your domain names web pages. And long story short, with DNS. I say that you have one in your home and in your work and in your campus because it's a very hierarchical kind of structure. Like there is out there somewhere these so-called root servers that essentially know what the IP addresses are of all of the dot-coms, for instance, or all of the dot EDUs or the like, but my Mac doesn't know that. And so my Mac might actually ask that root server what. That IP address, but in more efficiently, my Mac is better still going to ask the local network first. When I'm at home, it asks my home DNS server, which is built into the little home router that you've got somewhere in there, or if you're on campus, it asks Harvard's DNS server, and this whole design is recursive, to borrow a term from a few weeks ago, in that if my computer doesn't know. Answer. What's the IP address for this domain? If Harvard doesn't know the answer, it eventually gets escalated to those so-called root servers, but then cached that is remembered by all of these other DNS servers along the way. So it's a very elegant hierarchical design, but at the end of the day it's just doing this. It's a big cheat sheet of domain names to IP addresses and the server is responding for us. All right, one more acronym. So how do I know what my Mac IP address should be? How do I know what my phone's IP address should be? How do I know what the IP address is of the DNS server of whom I should be asking any of these questions? How do I know the IP address of the router to whom to hand my data off to? Like there's a lot of assumptions built into the story we've been telling, and the answer is unfortunately yet another acronym DHCP is the solution to all of those problems, and it wasn't always back in my day we used to have to manual. We type in what our computer's IP address was based on what some human told us it would be. We had to type in our DNS server, type in our router address, but now DHCP is just yet another server running in your home network, running on campus, running in your corporate network whose purpose in life is to answer questions of the form. What is my IP address, which is to say when you boot up your Mac, your PC, your phone for the first time, it essentially broadcasts a message like Hello world, what's my IP address? And hopefully there's one such. DHCP server on that local network wired or wirelessly that will respond based on how Harvard or Comcast or Verizon or someone at home has configured it to tell you what your device's IP address is, what the IP is of your local router, what the IP address is or are of your DNS servers, and the like. And so this is why things just work nowadays once you've connected to like a Wi Fi network or physically plugged in. Dynamic host configuration protocol didn't always exist. Wonderful that it now does. All right, enough sort of outside of the envelope stuff. Everything else today will be a deeper dive inside the inside of this envelope to look at what actually are the messages that we are sending, receiving, how are you structuring the web pages and designing everything that comes back from the server to the client. And let's dive in then to this acronym HTTP, which you've been typing for years or seeing for years, even though you don't really have to type it anymore because browsers just assume that this. What you want, but HTTP is another protocol, hypertext Transfer Protocol, whose purpose in life is to request web pages and receive web pages. As a protocol, it just standardizes like what goes inside of that envelope when you're trying to use the web. There are different protocols for email, different protocols for Zoom, different protocols for Discord, and

any number of other internet services. We'll focus predominantly today on HTTP, which happens to use ports 80 and 443. For 3 among others as we saw. So let's see what HTTP it is all about, or HTTPS, the corresponding secure version thereof. So here is a URL, canonical URL, in that it has a whole bunch of components. Let's consider what some of the jargon is that we're going to start taking for granted. So if you go to HTTPS www.example.com/, you are implicitly requesting the root of that website. Root just means the default directory, the default folder, if you will, and that's what the yellow highlighted slash here just means like give me the default webpage. Technically speaking, what you're going to receive in your browser, unbeknownst to you, is an actual file. By convention it's a file called index.html, maybe index.htm or any number of other files, but it would be pretty stupid if we as humans all had to type out the actual file name that we want. So the server by default is just going to return you the root. Of the website. If though you're inside of a folder or you do actually click on a link that leads you to a file, you might very well have at the end of this domain name a full path as well, which might contain zero or more folder names and 0 or more file 0 or 1 file names as well. In fact, it could be explicitly file.html or folder or folder file.html. You've probably seen thousands of these over time, even if you haven't really given it much thought. So we today onward will be creating all of this stuff here, but we need to understand what's going on to the left too. So here's the so-called domain name, or more properly the fully qualified domain name, and it has a few different parts too. So this is technically the domain name as we all refer to it, something.something.com means commercial and that.com is more specifically known as a top level domain or TLD. Back in the day there were only a few of these.gov.com, .net.org.edu, and a bunch of others. Now there's hundreds if not thousands of them. Many of them aren't really used prominently in the wild, but there are some not on that original list like CS50 uses.io a lot, which doesn't mean input output. It's actually a two letter country code that has been essentially rented to. Us and anyone else using that same TLD because in the English speaking world. I actually sounds kind of cool. It's kind of connotes indeed input and output.tv is another one that actually belongs to a country, but in fact also sounds like in English television, and so that too has been used as well. But in general there are top level domains like these. Some of them now are full words, some of them are two characters denoting they belong to a country. They are. The sort of top level indeed categorization of all of these websites. Meanwhile, many URLs, but not all, also have something to the left of the domain name known as a host name which technically speaking refers to the name of the server that you're requesting specifically. It doesn't have to be literally one server. www can refer to dozens of hundreds, thousands of servers. Indeed, if you go to any popular website like gmail.com or the like. Even though you only have one domain name, somehow or other technologically, it is referring to clusters of hundreds or thousands of servers that ensure that they can handle all of the customers that might visit that site. And then lastly there's this the scheme or the protocol in use specifically and for our discussion today it's always going to be HTTPS, which is ideal because it's secure and encrypted somehow, but it can also be indeed HTTP colon slash. So that's it. Like that's just the jargon with which you should be familiar when it comes to URLs like these, and what we'll be doing today is actually creating content that lives at URLs like that and serving it up to us, but What do the messages ultimately look like that are going inside of these envelopes? What the URLs are doing are just getting us to the right place, but how do we express in some form of code that we want this file.html from this server using encryption in this way? Well, inside of the virtual envelopes that Phyllis was sending to Brian

and he would have ultimately sent back to her are messages that look like this get post. And a bunch of other verbs, if you will. So HTTP supports a bunch of operations or verbs, namely get Post and a few others, and it was indeed the first of these that Phyllis would have put inside of her envelope initially in order to get a web page like a cat from Ryan. Specifically inside of the envelope she would have had a textual message. It's not code per se. There's no functions or loops or variables or anything like that. It's a protocol just in the sense that humans years ago standardized what messages should appear inside of those envelopes if you want to get a webpage from a server. So for instance, if Brian in this story is now suddenly Harvard.edu, specifically www.harvard.edu, Phyllis's envelope would have contained a message saying get in all caps slash if she just wants the root or the default page from Brian's server, the version of HTTP that she's using, for instance, version 2, and she would also specify just in case Brian is multitasking and serving up websites for different. Domain names on the same physical box which actual host that she wants, and maybe a bunch of other lines as well. And hopefully if all goes well, Brian would have responded with an envelope of his own containing an HTTP response in answer to her HTTP request and Brian's envelope would have contained a textual message that just confirms what version of HTTP he's using a status code, which is an arcane number that just indicates in this case that everything is OK. All is well, and he would specify the type of content he's sending back to her in his own envelope because it could be HTML. More on that later today. It could be a JPEG, it could be a GIF. It could be any number of other file formats, and this is just a hint to Phyllis's browser as to what's going to be inside of that envelope she is getting back within her browser and then maybe a bunch of other stuff as well. So even though some of these details, like these underlying implementation details might visually be. New to you if you've never really thought about it. Turns out we as aspiring programmers can actually see and and poke around with these building blocks and ultimately today take advantage of them. So you're about to see a program that's called Curl, which stands for Connect URL. It's installed in Linux systems like CS50.dev. It also comes with Macs and PCs quite frequently or you can easily install it and essentially it's a headless browser that allows you to pretend to be a browser and grab the response. from a server by pretending to send by actually sending the contents of an envelope like this. So for instance, if I want to pretend to be a browser and request harvard.edu, I can type this in my CS50.d terminal window and let me go ahead and maximize that size and do the following Curl I, which specifically is only going to show me the headers, the text that we were just talking about, and it's not going to send any of the contents of Harvard's website. Curl I HTTPs www. Harvard.edu. So if I were typing this into a browser, I would actually see Harvard's homepage. In this case, I'm just going to see the contents of the envelope as black and white text on the screen, specifically only the first few lines, the so-called headers that the server is responding with just as I claimed Brian would to fill us. I hit enter. And there's indeed more lines than I had in my slide, but you can see that everything is in fact 200 OK. This is a convention. 200 means all is indeed OK. There's a bunch of other information here, including the date and time in which this response came back. Here's that content type line text HTML and then some other details and a whole bunch of other information as well. So that's one way of seeing what's going on underneath the hood. Well, what other responses might come back? Well, it turns out that 200 OK is the best possible outcome, but there's another, a bunch of other outcomes that are possible as well. For instance, sometimes you'll get not 200, but 301, which means moved permanently colloquially speaking.

And what does this mean? Well, if a server responds to a browser with a numeric code of 301. That means that the browser is supposed to go to this location instead. It's sort of like putting a detour sign on the server that says there's nothing for you here. Go over here to this location instead. And now notice in this example it's telling the user to go to HTTPs www.harvard.edu/. That's actually. What I typed before, so I would not have seen that myself. But if I go back to VS code here and let's run the exact same command, but let's try to visit the insecure version of Harvard's website HTTP and Slash, which just means that anyone else on the internet can technically see what it is I am now doing with my browser, which might not be desirable, enter. This time Harvard Server does not just tell me 200, OK, it actually says 301 move permanently, and if I read lower in these lines, there indeed is the location to which I should actually go and it's a subtle difference. It's forcing me to go to HTTPS instead without actually showing me the contents of Harvard's. So nowadays you and I don't even have to think about this. You and I are not even in the habit surely of typing HTTP or HTTPS, but the browser is ensuring in this case that you are redirected, so to speak, automatically to the secure version of that site instead. Now there's other status codes and in fact, even if you never realized it before now, what numeric code do you essentially you sometimes see on the internet when something goes wrong? 404. So 404 is a weirdly public arcane error number error number or status code that just means file not found, and we can simulate this as follows. For instance, if I ink my terminal window, do curl I HDPs www.harvard.edu. I'll suppose that Harvard has a whole department dedicated to cats, which it does not, but if I Enter here, you'll see that I get an HTTP 2404 status code, which just means the website does not in fact exist. And if I visited HDPswww.harvard.edu/cats in my browser, I would presumably see some error page that may or may not show me visually 404, but many websites, most websites for better or for worse, reveal this number so much so that Most everyone in this room is probably familiar with 404, even though its origin is this very low level arcane status code buried in the HDB headers inside of envelopes like these. There's a whole bunch of others if you'd like some fun facts. 200 is indeed OK. 301 is moved permanently. There's a bunch of other 300 ones that all relate to go elsewhere. 400 generally means. that you as the user have somehow done something wrong or next week as we start writing code that talks to web servers, maybe your code has done something wrong when requesting a website. 500s are really bad. It means the server is messed up somehow, either it's not available or the programmer made some bug in their code such that it's crashing with, for instance, something like an internal server error. We included 418, which is not actually a thing, but it was a fun. A sort of April Fool's joke years ago where a bunch of humans thought it would be funny to write up a whole specification for what it means for a server to respond with a number of 418 inside joke. Not funny at the moment, but it is sort of part of internet lore nowadays. We can have a little bit of fun with this, maybe with the at the expense of our dear friends down the road. For years now, someone has been paying for. Uh, the following behavior. Let me go back to VS code here in my terminal window. Let me do curl-I HTTP cosafetyschool.org. Have you ever been? Ever apply perhaps? Well, let me actually go to HDP/safetyschool.org, and just for fun, hit enter. Oh my goodness, look at where we are. So how is this implemented? Well, if I finish what I began over here by just looking at the HDB headers inside of the envelope my actual browser just sent to safetyschool.org for like 20 years, presumably some Harvard alum has been paying the bill to rent this domain name just to have this trick implemented such that 301 moved permanently. is directing people ever since to

yale.edu. There's a bunch of others if you go down the rabbit hole of looking on Reddit and the like Stanford, Berkeley, there's a healthy competition on East Coast and West Coast, but it all boils down to very arcane understanding of how HTTP works, the protocol that governs how data is sent from web browsers to web servers. Now you can of course use Curl for connecting to URLs in the context of something like CS50.d. You could have been doing stuff like this all the time though with your actual browser. So I'm using Chrome here, but most any browser nowadays has the ability to give you developer tools natively, which is to say somewhere there should be a menu option that lets you use developer tools that are conducive to someone who knows a bit of programming to poking around underneath the hood of the browser and see what's going on. For instance, I'm going to go ahead and open up a new window here and I'm going to right click on the background or I can go to the appropriate menu in Chrome's menu and I'm going to go to inspect, which pulls up what we're going to call developer tools. I'm doing it incognito mode for reasons we'll see next week. This has the effect of clearing automatically any of my cookies, my browser history, because most any time I do something with the web browser today, I want to pretend like I'm doing it for the very first time so that the behavior is exactly as we suspect. expect. So down here now that I've opened up the so-called developer tools in Chrome and they look almost the same in Safari and Edge and a bunch of other browsers as well, I will see a tab called Elements which shows me all of the elements of this webpage once it appears, including the so-called HTML code we're about to write. I can see a console where error message might sometimes appear similar in spirit to the terminal window in VS code. I can also see the network connections. The browser is making to the server and that's where I thought we'd start our attention here. Here I have a brand new browser window. I'm clicking on network over here just to make sure we can see everything without it getting automatically deleted. I've clicked on preserve log and disable cache just so that it behaves exactly as expected. And now let's go up here for the first time in this Incognito window and go to [HTTP/safetyschool.org](http://safetyschool.org). Enter and you'll see a whole bunch of outputs including this warning in this particular mode. This is increasingly common nowadays for websites that do not support HTTPS, which this alum hasn't been paying for. You'll get a warning typically that specifies you might not want to do this because the whole world, at least the whole world between you and point B, might know what it is you're accessing on the web. I can go ahead and pass through this. In fact, once I do that. And click on Connect to site we'll see even more output at the bottom and a whole bunch of output that's kind of overwhelming. Notice that bottom left here, just going to safetyschool.org resulted in 61 HTTP requests, in effect, 61 envelopes going back and forth. I'm going to focus though on the ones at the very top here whereby when we finally click through that warning and I got back a response from the server having visited safetyschool.org. Here is Chrome's presentation of the same information that Curl was showing me in my terminal window. The message that came back was 301 moved permanently. The protocol or the verb being used was get. There's some mentions of the IP address in question here and a whole bunch of other stuff that we'll wave our hands at for today. So all of this time you can see the same, and let's try this with some cats. Let me click on the little Ghostbuster symbols to clear. Everything down in the developer tools. Let me zoom out and this time let me go to [HTTPS www.harvard.edu/cats](https://www.harvard.edu/cats), which were called did not exist according to Curl. If I hit enter, I do see a web page. It's interesting that Harvard has chosen to fairly arcanelly reveal to all visitors 404, which means nothing except insofar as the status code. But if I scrolled through

all of the 59 requests that were involved and just displaying this very graphical page and go back to the top, you'll see by clicking on the first row for Cats itself that I used get to get it that URLs cats in the end, and it was indeed 404 not found. So you can sort of have all this fun on your own by just poking underneath the hood of what your browser has been hiding from you all of this time. All right. Any questions now before we dive in? No. All right, well, that's the network tab. Let's look at some of the others and see how we can start writing this stuff ourselves. Let me go to stanford.edu enter. A whole bunch of things will fly across the screen, but this time I'm going to go to the elements tab. And what we're about to dive into is an actual language, not a programming language, a markup language called HTML, hypertext markup language, whose purpose in life is just to tell browsers what to display on the screen. So here is all of these so-called HTML that some humans or humans or soft. Where at Stanford wrote in order to create Stanford's homepage, which as of today looks lovely like this. The interesting thing though about the code that Stanford has written to generate this website is that it's being sent to me as a copy, and this is quite unlike the code we've been writing thus far. When you wrote code in Scratch, it was sort of there in the browser and stored on MIT's server. When you wrote C code and ran it, it was inside of the code space and not given to any user who might access it. The way the web works though is a little bit different. Inside of those envelopes are literally copies of what's on the server being sent to the browser, and so it's your browser, the so-called client, that's actually reading that code, HTML in this case, top to bottom, left to right, and figuring out how to display it. It's not executed on the server per se. Now that story is going to change a bit next week when we start using Python to dynamically generate HTML so that we're not writing all of this code by hand after this week. But for now everything you see was the result. Of the browser executing code that Stanford wrote. The implication of that is that we can have a bit of fun with these same developer tools. For instance, if I control click or right click on something like the word Stanford in the middle middle of their homepage, choose that same inspect option. What's nice about these developer tools is it's going to jump to the very line of code that created that Stanford brand name in the middle of the web page, and this is a wonderful teaching and learning tool because in the days to come when you're trying to learn more and more HTML. You can literally do this for any website on the internet and understand how it is someone implemented a design, for instance, that you really like and you can learn from other websites how they've constructed the same. So over here you'll see that the word Stanford is just in the source code of this page in the so-called HTML, and you know just for fun I can change it to Harvard, hit enter, and now Stanford's website looks like we've been there and rather hacked it. Of course it's not that easy to hack Stanford's website. What have I presumably only done just now? I've changed my local copy of that particular website, so if I just click on the reload icon, I'll actually see that Wes Stanford's website, for better or for worse, still looks like that. But this speaks to now the control that we have within our browser to actually manipulate and learn from what it is that's going on underneath the hood. So let's dive into this language called HTML, hypertext Markup Language. It's not a programming language, which means we're going to fly through it even quicker than usual because it really just contains some basic building blocks. They do have some interesting intellectual design under them, but for the most part it becomes an exercise ultimately and just like looking up other tags that exist, read the documentation and figure out how you can use them to do other features and websites. So let's take a look at perhaps the simplest of webpage and specifically glean from them what tags are

and what attributes are really the only two terms of art that are going to be germane for this particular language. No loops, no conditionals, no. Variables no complexity really other than basic building blocks like these. So here is HTML for the simplest of websites. This is like a mini version of what Stanford's team presumably wrote on their server, but it's only like a dozen lines of code instead of hundreds or thousands, however long that website was. Any webpage written today, assuming it's using the latest version of HTML, which happens to be version 5, as of today, begins with code that looks like this. This kind of code will presumably be stored in a file called file. HTML, index.html, Stanford.html, whatever the file is actually named, this is simply what's going to be inside of the contents. You could save this file on your own Mac, open it up, and your browser would open it, but you're going to be the only one in the world that can actually see the contents of that webpage if it's just on your Mac or just on your PC. So we of course are going to be writing HTML on a server so that not just you but in theory, especially for your final project, anyone in the world with an internet connection can access the same. So we within the context of CS50.dev are going to start using this new command HTTP Server, whose purpose in life is just to serve up files via HTTP. Now there's kind of an interesting design going on here because if we use HT, if we use CS50.dev, otherwise known as GitHub code spaces, there's already a web server running on that website because when you go to CS50.dev and log in and get redirected. It some longer URL, you're using a web application, aka VS code that allows you to write code in the cloud. Now that application by default is running on Port 80 and 443, so it doesn't matter if you start at HTP or HDPS. Both will work, but that means that your code that we write today and you write for the next problem set or for your final project can't live at Port 80 or Port 443 because GitHub, the company that hosts this. is already using those default standard ports, but we can use any number of other port numbers. I claimed earlier. There's 10s of thousands of numbers that we could use. So that's what we're actually going to do. So let me go back to VS code here. Let me shrink down my terminal window. Let me create a first file today called, for instance, hello.html. Enter, and now I've got an empty tab as usual. I'm going to very quickly whip up the exact same contents that we just saw, so Angled bracket and exclamation point doc type HTML, then open bracket HTML close bracket, and notice the autocomplete kicked in for this particular language so I don't have to type everything myself inside of this tag, so to speak. I'm now going to put a head tag inside of which is going to be a title tag. I'm going to say something like hello title, just to be quick, and then down here below those lines, I'm gonna put a so-called body tag inside of which is hello body just for some quick text, and that's it. This is now a file inside of my code space, and there's no command to just compile or run this in the terminal because the goal is going to be to open this HTML file with a browser. If I want to do that in another browser tab, I need to tell codes my code space to serve that file via HTTP. So the simplest way to do this is as follows HTTP server enter. You're going to see a whole bunch of text on the screen. You're going to see a green button. Pop up that says open and browser, which is going to allow you to open up and I'll zoom in the contents of the current folder with a web browser. My URL has changed to be different from what it was a moment ago. I came in advance today with my own folder of code like we usually do, Source 8, which contains all of today's pre-made examples. But here is the file I just created a moment ago. And if I click on that, hello. HTML, what we're looking at at the moment is just a directory listing, a directory index of all. The files in my code space right now. I see the simplest of web pages. It's a little underwhelming, but clearly here's Hello Body, which takes up like 95% of the

screen, the so-called viewport, which is just a big rectangular region of the screen, but there's the title in the tab up there. So if you've ever wondered or cared like where does the content in the webpage come from, well, here's the body content, here's the head or the title content, and then everything else is just sort of icing on the cake. So I've written at Point a file called hello. HTML. It has yielded this effect of having something in the head in the head of the page in the body, but let's actually tease apart what just happened. So at the start of any file written in this language called HTML, the latest version thereof, 5, it literally just starts with this, and this is just the kind of thing you memorize or copy paste. O bracket doc type HTML, close bracket over there. It looks a little bit different because we're not going to use. For the most part, the exclamation point syntax anywhere else unless we're using an HTML comment. So HTML has comments just like Python, C, and other languages. But let's focus really on this juicier part. Here we have what's known as an element in HTML. An element includes a start tag and an end tag, or equivalently an open tag and a closed tag. So here for instance is syntax that essentially is going to tell the browser when my browser reads this file top to bottom left. Right, hey browser, here comes the HTML of my page, and the language in which the contents of this page are written are in English. So HTML, all lowercase, is the name of the tag, so to speak, and equivalently the name of the element. Lang is what's going to be called an attribute, which just modifies the default behavior of the element, and quote unquote is the value thereof, which is the shorthand notation for English, and there's shorthand notations for most every human language as well. So you have a tag name and an attribute with a value, and we've seen these things so many times, these key value pairs in the context of dictionaries or hash tables or any number of other contexts. Key value pairs in HTML are separated by an equal sign with the value typically quoted in this way. Double quotes or single quotes, but being consistent. Then notice at the end of this file, as per the indentation. There's something symmetrically down here that has the effect of closing the tag or ending the tag, and this effectively tells the browser, he browser, that's it for my HTML. Meanwhile, everything else follows the similar paradigm inside of those two tags. Here is a head tag that says, Hey browser, here comes the head of my page. Hey browser, that's it for the head of the page. Hey browser, inside of the head, here comes the title. That's it for the title. Well, what is the title? Hello title, just as I wrote in my code space. Same story for body. Hey browser, here comes the body of the page, the 95% of the screen. That's it for the body. But what's in the body is exactly that. The indentation is nice and pretty printed. I've used 4 spaces as we commonly do, not strictly necessary. In fact, in my own code space, I didn't even bother putting these on 3 separate lines. I just did one line. That's fine because as we'll see, browsers typically ignore white. Space, but I've done it there as we often do just to ensure that things are pretty printed and therefore readable by us humans. Let me call your attention to one other thing on the screen. Up until now, before every lecture, I've been hiding a whole bunch of tabs in my terminal window, but today I left enabled one that you've probably seen but not cared about before, namely ports, and it's under this ports tab that you can actually see a real incarnation of a TCP port. By default, when you On the command HTTP server, it serves up my current folder's content on its own web server, its own HTTP server, but not using the default port 80 or 443 because GitHub is already using those on CS50.dev and their product, but by default we've chosen another common developer port number, 8080, which is interesting only insofar as it's 80 twice, but it's a human convention, but it could have been any number of thousands of other possibilities. But this line here is just telling me. That I am

some apparently running a server on Port 8080, and if I click on there too, I can manually open the same tab, but that's what the green button was doing for me. It was informing me, hey, you've just started a web server on this port. Do you want to open a new tab with the contents thereof. So this is the picture we're now painting. Let me pull back up the code that we just wrote and let me propose that. What we've really done is build a tree in the browser's memory. So we kind of have come full circle with week 5 when we talk about trees and other hierarchical structures. If we assume that the document can be represented with a node that looks a bit like an oval up here that just represents the whole content. The file. Well, it starts with a single root element by convention, the HTML element, and your page can have only one of those elements, but the HTML tag inside of it can be a head tag and a body tag. And in this case, the head tag recall had a title tag as well as the actual text thereof, which was hello title. Meanwhile, the body had just the text thereof as well. And so when I keep saying That the browser is downloading the file, for instance, hello.html reading it top to bottom, left to right, it's doing literally that, but somehow or other it's using Mao or whatever language it's written in to allocate node, node, node, node, node, and populating that tree in your browser's memory or RAM, a data structure quite like that. So it's all sort of germane to where we've been before. Before now we take I think a snack. Are there any questions? About what we've just seen Anything at all shouldn't have prefaced this with the only thing between us is uh these questions and snacks. No, right, snack time. All right, see you in 10, snacks. All right, so we are back and pretty much everything we do here on out will look structurally like this, and we're just going to introduce a few more tags and a few more attributes to give you a sense of some of the basic building blocks of most any website out there, and you'll find pretty quickly that it starts to get kind of tedious writing it out. In fact, I will resort to some copy paste today just to kind of speed things up, but this is going to motivate indeed next week when we reintroduce Python as well. SQL to actually automate generation of HTML as well. So all of today's websites and many of today's mobile apps are written in HTML, but people are in decreasingly writing this kind of stuff by hand. Rather, they are writing code that generates precisely what we're going to learn. So understanding the fundamentals will still be useful so we know what code to write next week and beyond. So let me go back into VS code here and what Going to go ahead and do is open up another terminal window so that I can leave HTP server running in this first terminal window. And what I'm going to go ahead and propose that we do is implement a web page that has not just a single line of text, but maybe some paragraphs. So I'm going to call this paragraphs. HTML. That's going to open up a new tab. And here's where I'm going to save some time. I'm going to go back to hello. HTML and just highlight all and copy paste. This is the beginning of this file, but what I'll start doing is just changing the title of each page to match the file name. So this is going to be my paragraph's example. And instead of saying just hello body, let's actually have a few paragraphs of text. I'd rather not waste time writing even full paragraphs of text. So let's actually open up the dock and let's log in and for instance just ask it for a quick helping hand here, write three paragraphs about. Uh, computer science, don't really care what the output is. All I want is some dynamically generated text to save me some keystrokes, and here we have an educational answer there too, even though all we really care about today is the fact that this is 3 chunks of text. Hopefully that's all quite accurate. All right, I'm going to go ahead and highlight all of that and go back into my paragraphs. HTML tab, paste it inside of the body. It's so long, the paragraphs that the text scrolls. I can at least clean this up slightly. I'm going to

go ahead and just indent it twice just so that at least it's pretty printed. Inside of the body and now I'm going to go back to my other tab which represents the contents of hello. HTML. I'm going to click back which is going to show me that same directory listing again which now has a new file, paragraphs. HTML, and I'm going to click it so as to see these three paragraphs of text. What looks wrong? Yeah. There's no paragraphs. It's just one big blob of text. It's the same text, but buried in there is the end of the first paragraph and the start of the next and same for the third. So what's going on? Well, apropos of my comment earlier about browsers not really caring about whitespace, you can put all the white space you want there. It's just going to ignore it in this particular case. All it's going to give me minimally is a single space between each of these paragraphs of text. So, is very pedantic. Like if you want there to be more paragraphs, you need to tell the browser, Put a paragraph here, put a paragraph there. And the way to do this thankfully isn't all that hard. I'm going to go inside of the body here and I'm going to simply open a tag called O P for paragraph for short. Notice that VS code in this particular case is a little annoying because it's trying to finish my thought, but it doesn't know that I already wrote this text, so I'm just going to delete what it automatically. and then I'm going to manually indent this and I'm going to do the same thing again for the other paragraphs up here I'm going to open the paragraph tag. I'm going to delete temporarily the close tag so that I can actually put it below that chunk of text here, indent this, and then down here, and this would have been easier if I just did it right the first time, I'm going to do the same thing with the 3rd and final paragraph. So now what we in effect have 3 times in a. Hey browser, here comes a paragraph. Then the first paragraph, Hey browser, that's it for the paragraph. Hey browser, here comes a paragraph. That's it for the paragraph. Hey browser comes a paragraph. So 3 times in total with open, close, open, close, open, close. Now if I go back to the browser, nothing appears to have changed yet, but that's because I'm looking at a copy that was downloaded a moment ago in that virtual envelope. So this is why, among other reasons, we hit reload on web pages to get the latest version. And voila, now we have 3 actual paragraphs. The white space is inserted automatically by the browser, but it's at least prettier to the eye now. So that then is the paragraph tag. So useful, of course, if we have paragraphs of text, what are some other tags we might introduce? Well, maybe you're writing a paper or a blog post or the like. It's pretty typical to want headings of sections of the page, maybe chapters and then sections and then subsections or the like. HTML can help with this too. So let me go into my terminal window again. Create a file called how about uh let's call it headings. HTML and then in this file let me similarly go back to hello. HTML, copy paste it into headings. I'm gonna close paragraphs because we're done with that, and I'm just gonna change the title now to headings and inside of the body here what I'm gonna go ahead and do is, uh, you know, it would have been nice to have some of that same text. Let's. Let me go back one step. Let me grab the paragraphs and paste that into this new file. Let me rename it to headings to make clear which file we're in. And now let me go ahead and propose that wouldn't it be nice if I made clear that this is the first paragraph? So I'm going to use the H1 tag, which is the heading 1 tag, and I'm just going to say 1 for the sake of discussion. Down here I'm gonna say H2 and say 2 for the sake. Discussion and down here H33 because I don't really care what these things are called. I just want to demonstrate the functionality. If I go back to my other tab now back to the directory listing, there's my brand new file headings. HTML, and it's the same paragraphs, but now you have some big bold text that looks reminiscent of the chapter heading, the section heading, the

subsection heading, and the like, or that you might see on a news site or a blog site or the like. So you've got H1 through H6 from biggest and boldest to smaller but still bold, and the browser decides on all of those settings for us, but it also makes some semantic clarity to me that probably the most important thing on the page, at least to begin with, is that H1 tag, and then everything else is like supporting paragraphs or arguments or whatever the case might be. There's a hierarchy implicit there. All right, what are some other things we can do with web pages? Well, let me open my terminal window again, and why don't we code up how about a list of values because lists are everywhere on the internet. So let me open up list. HTML and then close my terminal. Uh, I'll go ahead and start with that same file headings. HTML, paste it into lists, change the name here. Let's delete everything I did. And again, the only reason I'm copying and pasting is just to avoid writing out the same boilerplate code again and again with the HTML tag, head tag, body tag, and so forth. Let's focus on the new stuff. The new stuff in this example will be a list of values like the words food, bar, and baz, which, much like a mathematician, might go with X, Y, Z as. Holders, computer scientists would typically reach for words like food, bar, and baz when nonsensical placeholders, and this looks like a list of three values one after the other. Of course, if I go back into my directory index, click on list, how many list items am I going to see per line? Yeah, well, it's going to be just one big blob of text here too. It doesn't matter if it looks like a list. It is just going to be text after text after text separated by a single space, not the multiple lines I had. So here too, we've got to be pretty pedantic. If I want a list of values, I need to use a tag that conveys that, and the tag I'll use first is going to be UL for unordered list, which gives me a bulleted list. And then inside of this unordered list I claim. We can have a whole bunch of list items or LI for short, uh, like food, like bar, like baz, or any other things that you want to put in your list. If I now go back to my other tab, reload, now you get the familiar bulleted list that you might see in any number of websites, Google Docs or the like. How does Google Docs do it. Neath the hood while they're just using a UL tag and some LI tags inside of that to give you the bulleted list that's just happening automatically when you click the appropriate button in something like Google Docs, which at the end of the day is just a website. Well, what if I want to number these things? Well, if I go back to VS code, I could certainly just start numbering them like 123, which is fine. And but honestly, like computers can count and with loops pretty quickly. Also it's a little annoying if I want to go back in later and insert something between some of those elements. I then have to renumber everything manually. I mean this is one of the things computers are good at. So take a guess. If I want not an unordered list but an ordered list that is numbered, what might you change? Yes, OL is a good bet. Let's change both the open tag and the close tag. Let me go back to this my second tab, reload, and now we have it 12, and 3, and you can actually use a whole table of contents. You can use sub bullets or sub numbering, anything you can do in like a table of contents HTML can do for you automatically here. Well, what about tabular data laying out data in kind of rows and columns? Well, we can do that too. Let me go ahead and open up a new file. How about Table. HTML. Let me go ahead then in this file copy paste as before just so I have some boilerplate. Let's get rid of everything in the body and then let's just manually whip up a little table like this. Open bracket table inside of the table tags I'm going to have a TR tag for table row. Inside of this table row I'm going to have a table data tag which is going to have the number 1. I'm going to give myself another 2, another 3. Outside of the table row I'm gonna have another table row and I'm gonna create maybe 4, and now I'm gonna do 5 and now I'm gonna do 6 and you can

perhaps see where this is going. After this, I'm gonna do 1 more table row. How about a little tediously 7, how about 8? How about 9? And then lastly, just to make it look a little familiar, final table row, how about with a TD of an asterisk, and then how about a 0? And lastly, how about a pound symbol? Maybe, any guesses as to what we're making in HTML here? Like a telephone keypad. Yeah, so let's go back over to, let me close the old file back over to the browser, click back. There's my new file table. HTML, and it's not going to be very pretty, but I dare say that's exactly what you see when you pull up the phone app and you start dialing a number. It's sort of a numeric keypad laid out automatically for me in rows and columns. Now this one's a little underwhelming. Let me open up a file that I made in advance of. Class today in my favorites file here I'm going to go ahead and copy a premade example. I'm going to open up this file called favoritezero.html and what you'll see here is a slightly more complicated table still with a table tag, but this time with a Thead tag for tablehead and then a T body tag inside of which are all of those rows, and I know this just by having read the documentation and then notice this. Inside of the first TR in thethead there are 3 TH's table headings, time stamp, language, and problem, which might sound a little familiar when we last collected data from everyone via that Google form. Well, let's go ahead and spoil what this is. Let me go back to the directory index. There is this pre-made file favorites.html, and arguably a more compelling use of a table. Now we have an HTML table. all of the form submissions that you all clicked in with the other day when we were asking you your favorite language and your favorite problem. It's not super pretty, but indeed it's in rows and columns and so it's reminiscent of the HTML that Google is using in the actual Google Sheet software to lay out a sheet of data for you in those same rows and columns. All right, well, let's do something that's a little more visually interesting. Let me go back to VS code here. Uh, close out those first, uh, those last two, and how about let's do something with images. Well, I brought again. Uh, inside of today's code, how about our same bridge that we keep opening up in class, and this is the week's bridge, looks a little something, whoops, looks a little something like this. Here though is just the raw image. How could I include an image in a web page that I serve up on the internet? Well, let's go ahead and try this. Let me close the ping itself. Let me copy this and create a new file called how about image.html. Hide my terminal, copy paste that. just quickly change the title to image so we know where we are and inside of the body of this page, let's go and embed that image so that we can include not just the image, but if we want paragraphs of text around it, headings as well, heck, maybe a table, any other features that we've seen already. I'm gonna say IMG, which is image for short, source SRC for short, equals quote unquote bridge. PNG, and then I'm going to close the tag here. Now I'm going to go back to my other tab, go back into my directory index. Here's my brand new file, image. HTML, and this isn't going to look all that different from the actual image because I have no other content. But when I click on this, you'll see that there is the full screen image, and it's even a little too big to fit in my viewport in the body of the page, but we can fix something like that later. I've embedded in this website precisely that image, but I should do a little bit better here. In fact, if the image is slow to load or if someone is visually impaired and doesn't know what they're looking at, it would be nice to have some alternative text that something like screen reader software could recite. So there's another attribute for this tag specifically called alt for alternative, and I can put something like Harvard University to at least give the user a textual description of what kind of photo they're looking at. You'll also see that text if indeed the image is slow to load or if it's broken, like missing altogether, you won't see 4.

4 you'll see like a broken image icon, but at least with some explanatory text as to what the developer intended you to see at that point. It's not going to change at all if I reload here by going back to image.html, but again, a screen reader or an astute viewer would see that ultimately in the browser. But there's something different, and this isn't a mistake for once. What have I done differently but apparently not wrong, I claim. Something new or noteworthy about this particular image tag, yeah. Yeah, there's no like closed tag. There's no like open brackets slash IMG, which is the pattern we followed for every other tag like closing the HTML tag, the head tag, the body tag, and so forth. I just don't see any end tag here and it's just not necessary. Turns out there are certain HTML tags that can be empty elements, which is to say it doesn't make semantic sense to start and end an image like it's either there or it's not. And so some tags just don't require an end tag if it's sort of obvious to the browser that the image should go there. So image is one such of those tags. And then I noticed I'm missing the lay here, which isn't strictly necessary because I've got no textual content, but just for consistency, let me go back and put that in as before. Meanwhile, the image is exactly as it would appear in the screen, but it doesn't have to be just an image we embed. We can do something with like video. So let me go ahead and open up a file called video.html. Let me copy paste some of that. code change this to video and instead of the image tag, as you might imagine, there's also a video tag. It's a little more involved, but per the documentation, I know I can do this video and then inside of the video tag I can actually have multiple sources just in case the browser might want different versions or different resolutions, sort of qualities thereof, and this somewhat confusingly is an actual tag called. Source not shortened, but stupidly this tag has an attribute called source which is shortened that equals the name of the file you want to embed and I came with today's examples a video file called video.mp4, which is a small video that you can embed, and I can tell the browser what type of video it is, to be clear, and the convention here or content type is to say the type of this video is an MPEG-4 video. There are other features though for the video tag. In fact, when you see a video on a page, you can very often see like a play icon, a pause icon, maybe some other controls. Well, it turns out you can put an HTML attribute on the video tag literally called controls that will enable those. If you don't turn them on, there's no way to like start and stop the video or see rather those controls visually. This way the user actually sees them, but this attribute is a little bit different from others. It doesn't actually need a value. It just has to be present, and the browser will know when it sees the word controls, oh, I should turn on the controls feature, and for good measure, especially in today's world of advertisements everywhere, if you want the video to play automatically, potentially, or at least not annoy the user, you might want to mute it by default as well. So another attribute per the documentation for the video tag is that you can start the video muted as well, and only when the user clicks on it might you actually start to hear something. But of course these are fairly basic examples of media inside of pages. Let's actually do what the H is meant to imply in HTML, the hypertext, the ability to link from one page to another. That is a feature we haven't yet seen. So let me go ahead and do this. And let me just for completing this, let me go back into hello.html because I completely forgot the language attribute, even though that's really just there for SEO, search engine optimization or for tools like Google Translate or the like that know therefore what language they're translating from. Um, let me go into my terminal window here. And let's create another file called link.html, which demonstrates exactly that, the ability to link from one webpage to another. Let's go ahead here and change the title to Link so I know where I am. And

in the body of this page, let's go ahead and create what's called a hyperreference or hyperlink. I'll encourage people in this page to visit the actual. Harvard website. So let's do visit. How about Harvard period just to demonstrate where we're beginning. If I go back into this directory index, click on link. HTML. This of course is not yet a link, so I should probably make it one. Well, instead of just saying visit Harvard, maybe I should say harvard.edu. Back to the other tab reload, and it's harvard.edu button I can click and highlight it, but it's not click a bull. It's not underlined like a link. All right, well, maybe I need to do like www.harvard.edu. Reload. Still nothing happening. All right, well, maybe I need the full URL in the scheme HTTPS call in slash and maybe the slash at the end. Reload again. And nothing's happening. So here too, HTML is pedantic. Like it will not create a link for you unless you tell it to create a link. And the fact that when you post on social media nowadays or on Google Docs, things are automatically hyperlinked for you. Like that's a feature implemented in code very often Python or JavaScript or something else where some human wrote code that looks for patterns in the input you've typed in, and if it looks like you've typed a URL, it will. Automatically link it for you, but what are those websites doing for you automatically? Well, they're doing this. If you want to have a tag, a link here to Harvard's website, you use open bracket A for anchor, H for hyperreference, set that equal to the URL to which you want to link, close the tag, and then in between the open tag and the close tag, put the actual word you want to link to. So now if I go back to this page and reload, now I have what looked like my original attempt just visit Harvard, but it's a hyperlink, and this is super subtle, but if I hover over that underlined word which is blue by default, you'll actually see in the browser's bottom left hand corner where you're going to be whisked away to, even though that's all too subtle. But this now looks like I intended an actual hyperlink to Harvard. In fact, I could link it to the full URL, but it would be a little redundant and even though this looks like you shouldn't have to do this, this is indeed how HTML works. The HRF attribute is where you're going to go. The text inside of the open and closed tag is what the user will see. So if you want them to see the full URL, you've got to put it there, and now I can see the full URL to where I'm being led. But here's where you can actually introduce discussions of like cybersecurity. How could this feature be abused, might you think? This stupid simple feature, yeah. Yeah, you could have a display one thing but lead to somewhere else, and it wouldn't be that hard for the adversary who's maybe tricked you into visiting their web page to say, Oh, you're actually going to go to yale.edu instead of Harvard, but if I reload the page, it doesn't look any different unless the viewer is astute enough to look at this tiny little text in the bottom of the screen or just click on the link and be whisked. Way to the wrong destination that can be problematic. Like this is a nice ha ha sort of prank, but you could certainly imagine doing this with like PayPal.com addresses or any number of banks or anything where you're trying to collect personal information from someone. And if the resulting website looks quite like the one you're actually creating, it looks quite Like the website they're expecting, but is actually your copy thereof, it's all too easy to wage what are called phishing attacks, P H I S H I N G, which means to lead someone to what looks like the real site but is not typically to get their username, their password, their credit card information, or something else. But it boils down to just these basic building blocks like this. Questions then on any of these building blocks that we've seen thus far, yeah. Sure. Um, like To open up like did you Oh, good question. How did I get it to open up? So let me rewind. So the very first thing we did after creating Hello. HTML was open a terminal window and specifically I ran a command which was HTTP server, HTTP Server, which

starts my own web server in my code space but not on the default port 80 and 443 because that's what CS50. Dev is already using. Instead it chose by art design 8080, which is commonly used by developers when making websites. Then I just kind of hid my terminal because it's. Interesting to see constantly then, but that web server is still running in my code space and any time I'm saying let's go back to this tab, I am now visiting a different URL that was the result of my clicking on that green button which led me to my own website. If you ever get lost or close that tab by accident, no big deal, if you go to the ports tab of your terminal, you can actually hover over this and click on that same URL and open up the contents of your own site instead. Fluffy meme? Yes, these are randomly generated names by GitHub, which is the company that hosts VS Code in this way, and they do this to ensure uniqueness without it being some arcane sequence of random letters and numbers. They concatenate random English words together. A good question. All right, so what else can we do here? Well, let me propose that there's a bit more you can do with even these URLs. Here, of course, is the scheme and the host name and the domain and the TLD, but after the URL, things can get a little more interesting than just folder names and file names. In fact, it's quite common to see URLs that have somewhere in them a question mark and then a bunch of other key value pairs, which is this omnipresent computer science thing, it seems, including in the context of URLs, whereby if you want to pass. A input to a web server, one means by which you can do that is literally in the URL itself. So for instance, if you visit google.com and you want to search for something, you and I are all in the habit, of course, of just typing into a search box. But how is that search box actually getting the data into Google's servers? Well, it's via these URLs, and if there's not one input but 2 inputs, the URL might be a bit longer and there might be one or more. Persands in the URL that just separate more key value pairs and it turns out we can see this in the real world as follows. Let me go back to VS code here. Let me open up a new tab and let me open up Google.com and I'm just going to hit enter on the shortest way of saying it, so I get to Google's home homepage here even though notice I ended up at some longer form of the URL. In fact, I'm going to delete everything else from the URL that's not relevant to us today. It's still forcibly coming back, so Google is somehow trying to track me by putting that in there. That's fine. All I'm gonna do is search for cats. Now there's a whole bunch of other functionality that's clearly happening like autocomplete and it's trying to figure out what results or words I might want. I'm just gonna go ahead and hit enter. And this is all to say that notice if I zoom in on the URL at the top of my screen, it's a crazy long URL because Google probably is doing a bunch of tracking and advertising and analytics, technologically none of which is relevant to us today, but notice after www.google.com there's slashes, which is the path on their server, the search program that someone there has written. There's a question mark, and then there is an HTTP parameter, as these things are called. The more precise name for key value pairers and URLs, this is an HTTP parameter. Its value after the equal sign is in fact cats. All this other stuff, I have no idea what it is. I'm going to just delete it and hit enter and it stays gone, but I still get cats in my search results. So this, I would argue is sort of the canonically. Shortest form of a Google URL that's useful. In fact, if I want to search for dogs instead, I don't have to use the search box. I can literally manually make my own URL, hit enter, and if I zoom out, there are Google search results about dogs. So this URL too is sort of the essence then of how URLs work and specifically the get verb, which was that keyword in all caps that I claimed was inside of the envelope. And it's what Phyllis's browser was sending, and it's what my browser has been

sending through all of these examples. But here's where things now can get interesting. If I know how Google's server works, it's back end, the part that knows all about cats and dogs on the internet, I can implement my own front end by just knowing a bit of HTML. So let me actually go back into VS code here. Let me go into My second terminal, which is blank, and let me go ahead and create something called search. HTML. I'm going to go ahead and copy my original code, close the link, and paste it here, hide my terminal, call this thing search, and then inside of the body of this page I'm going to make my own version of Google here. I'm going to use a form tag and I'm going to in that form specify an input tag whose name is going to be exactly equal to what I saw Google uses Q, which happens to stand for query. Uh, I am then going to add another one, inputs, uh, the type of this button, actually, let's say the type of this box, this input is going to be text, the type of this next one is going to be a submit button, uh, and then that's it. Let me go back into my other tab, go back into my directory listing, click on search. HTML, and this is not pretty, but it is the beginning of my very own search engine. Unfortunately, if I type in cats, notice what happens my URL changes such that it's search. HTML mark Q equals cats. I know nothing about cats. I don't have a database of cats. I haven't done any back end. Work just the front end. The front end is what the user sees. The back end is what provides data to the front end. But why don't I tell this form not to submit to me, but let's say that it's action should actually be go to go to HGPS www.google.com/search, which is the URL that I saw in my browser. I'm just inferring how Google works. I'm going to be pedantic even though this is the default. I'm going to say the method I want my form to use is get confusingly, it should be lowercase here even though inside of the envelope it will be all caps. And then I'm going to go back to this page, reload after going back and you'll see the same exact box, but when I search now for cats, submit, notice my URL changes to Google's own. It's like voila, like I just implemented my own Google without doing the actual hard part. I've actually just done the more simple front end, and there's a few other things I can do here that are sort of nice. I can change the type to be a search box. I can change the value of my button, not to be the default, which notice was submit. I can say Google search and I can keep tweaking this to make it even prettier and prettier here now. In my version is now a box that has uh cats. Notice that it's trying to complete my thought. I can actually go back into the form. I can say auto complete equals off to turn off that feature. So now if I click in this box and um. Mm. Autocomplete equals off. Why is it still there? Did I forget to refresh? Oh, thank you. I forgot to refresh, hence my point. So you always have to reload after making a change, and now the autocomplete feature is off and this other little thing, it's subtle, but this little X that will just clear the whole thing, that is simply the result of having changed text to search for the type of that box. There's other things you can do too for accessibility or user friendliness. I can do auto focus here, for instance, without any attribute or without any value. If I now reload this page, notice. That the cursor is automatically blinking in the text box, which is a marginal change but much easier for me to now typeAs without having to stupidly click in the box in order to actually foreground it so I can type input. So suffice it to say this is not really the business that Google is in. They do much more on the back end than they do on the front end. But with just these basic building blocks, can I implement the beginnings of the same website? In fact, let me do one other flourish. You'll see that text box is blank, not clear what I might want. Do well, there's another attribute I can use placeholder equals something like query. I can at least tell the user what to search for. If I reload again now I see in gray text query instructions so that I roughly know what now to type. So all of these things

that you see every day on websites are really as easy as just coding up some HTML like that. But what else can we do with HTML? Well, it turns out this is a topic for another longer day too. There exists in computing what are called regular expressions, which is a fancy way of describing patterns which are quite useful when you want to validate input. For instance, if you want the user to have to type in an email address with the at sign with the TLD, and so forth, it would be nice to make sure that they get a warning if they try to skip that field or they mistype something in it as well. With the world of regular expressions known in short as reg X's, you have a whole bunch of documentation here that in a nutshell will introduce you to some pretty powerful syntax. We won't spend much time on it all today, but it's syntax that exists not only in the world of the web, but in Python and so many other languages as well. So consider this just a quick crash course. If you want to define a pattern in say a website that ensures that the user types in an email address, you can use these textual building blocks whereby in the world of regular expressions, a single dot represents any character. If you don't care what the character is, dot confusingly doesn't represent a period, it represents. Any character star represents 0 or more times. plus means 1 or more times. Question mark means 0 or 1 time if you want something to be there or not. Curly braces with a number means this many times and, and you can even have a range of values instead. And then you can use square brackets and some other syntax to say I want the user to type in any of these characters or digits in this case, or you can do ranges like this. I want them to type in any decimal digit between 0. 9 or backslashd represents any digit. Backslash capital D means anything that's not a digit. Long story short, humans over the years have come up with shorthand notation known as regular expressions via which you can define patterns. This is useful because if I wanted to make a web page that does in fact require that someone type in, say, an email address, I can enforce that to some extent. If I go back to my browser here and into VS code, let me go ahead and create a new file. Called register. HTML to be representative of registering for some website. I'll change the title here real quick. I'm going to keep the form, but in this case I'm not going to bother with Google anymore, so let's make it a bit simpler than before. And let's go ahead and do this inside of the form. I'm gonna have an input, uh, I'm gonna have the name of this input be email because that's what I'm collecting. I'm gonna have a placeholder be quote unquote email so the user know what to type in. Um, and I'm gonna go ahead here and have something like, how about Uh, this a pattern as well. So actually let's say, uh, let's say type equals text, but I'm going to specify additionally a pattern. So the pattern I want the user to type in in between these quotes is going to be any character one or more times, that is to say their username, then an at sign, then any character 11 or more times, uh, then literally. Period and we didn't see this on the screen, but just like can see when you want to escape special characters, if you want literally a period in their input as the like the dot in Harvard.edu, you can say backslash period to mean a literal period and then the word or the TLD EDU. So I think now what this means, and let me go ahead and give myself a button, and just so you've seen it, there's also a button element in HTML which is similar in spirit to the submit button we saw a moment ago. Let me go back to my directory listing, go into register. HTML and let me go ahead and just type in like Mailen as my name, register. And you'll see please match the requested format. So I've not satisfied it properly until I actually type in something like Main at Harvard.edu and now it's happy. Alternatively, it's a little tedious to actually type in these patterns, so there are some shorthands for them. I can actually get rid of this pattern and if I read the documentation for HTML. There is

actually an input of type email which just does all of that pattern matching for you, but the scary thing is that it's actually pretty involved to validate email addresses. I did a very simplified version of username at domain.tld. This is the regular expression that some browsers use to validate email addresses because even though mine is relatively simple, mailland at Harvard.edu turns out there's a crazy amount of syntax that is valid in email addresses. And this is where regular expressions get scary, but for our purposes today, they're a thing that exists. You might find them useful in HTML. You might find them useful in Python. They're incredibly useful when it comes to extracting information from web pages. If you're analytically minded, you like the world of data science, you like to Gather and analyze data. You can use regular expressions not just to validate data, but to find patterns of data in actual websites or documents and extract that data so as to perform operations or analysis on them. It's a wonderfully useful if complicated tool. The catch though is this Notice that here I'm still required to type in a valid email address, register, and I'm getting even more explicit information this time because I use the type equals email. The catch though with web pages is that they're not to be trusted insofar as this HTML came from the server and is downloaded onto the user's Mac or PC or phone where they have a copy thereof, I can open up developer tools as I did before by right clicking or control clicking and choosing Inspect or whatever the menu. might be I can go into the elements of this page, literally the HTML, and if I don't want to type in email, I want to just type in any old text and see if I can break your site. I can just change it. And now there is no such warning, which is to say, even though you will encounter not just today but over the coming weeks as you play with HTML, certain features, they are not to be trusted in general when it comes to security. And just like our discussion in the world of SQL and SQL injection attacks, this is one of the attacks. Vectors if two people are working on a website, one person's implementing the database stuff, one person's implementing the HTML, and the database person's like, Oh, I don't need to worry about escaping characters because we're doing, we're using the pattern attribute in the HTML. Bad idea because it's this easy to hack a website, disable features that have been written for the site by just literally deleting them in your own copy. So we'll see next week how we can defend against this on the server side, but the point now is just not to trust the user's input at all. All right, how can we be sure our HTML is right? Well, there's a bunch of ways, but one tool that's worth knowing about is this one here at validator.w3.org is a website by the group that essentially standardizes this and other languages. If I click on there, validate my direct input tab and I quickly go back into VS code and let me grab the simplest of my examples. Hello. HTML. I can just copy paste that. Into their website, click check, and they have written code to validate that the HTML I have written is in fact correct. Anything I've opened that needs to be closed has been closed. I don't have any stupid typos or missing brackets or quote marks. This is a wonderfully useful tool just to validate that your code is syntactically correct, even though it might still look like a mess visually on the screen. This will at least check for you the underlying HTML. All right, so up until now everything I've done has been pretty boring. It's black and white. The pages are fairly simplistic. Turns out we can take things the final mile using another language altogether, namely something called CSS, which is the second of our three languages today. This too is not a programming language, although curiously they keep adding more and more features that are making it more and more like a programming language, but more on that another time. This stands for cascading style sheets, and whereas HTML is About the skeleton of a website, the structure

thereof, CSS is like the skin, the aesthetics thereof, the final mile that actually allows you to control the positioning of things more precisely, the colors, the font sizes, all of the aesthetics. It lets you do the finer touches on the website. And with CSS we have slightly different syntax, but frankly it just boils down to even more key value pairs. And as with HTML, we'll give you a taste of the basic structure. Principles underlying CSS. There are so many key value pairs that are possible that we certainly won't do them justice today, but it's the kind of thing where you ultimately look it up in a reference, a book, a website, or the like to pick up even more than these techniques. Well, let's do this. Let me propose that in a moment we're going to see what are called properties. This is CSS's jargon for key value pairs. Why do we have yet another words? Because a different group of humans in a different room came up with this language versus the other people, but it's just Key value pairs known as now as properties instead of as attributes in HTML itself. There's going to be different ways we can define properties, and this is kind of a laundry list of some of them, and we'll see them in context, but in short, CSS is just going to allow us to slap a whole bunch of key value pairs on our HTML elements to make them hopefully look prettier or be more precisely controlled aesthetically. So in my HTML thus far we've generally had something that looks like this. Turns out if I want to start using some CSS. I can introduce, as we'll see, a so-called Style tag in the head of my page, and inside of that style tag I can put these so-called key value pairs or as we'll soon see too, if I want to factor them out and put them into a separate file, I can actually use a link tag which confusingly has nothing to do with hyperlinks or clickable text but just links in another file in this case styles.css, the relationship of which Be that of style sheet. This is the sort of copy paste stuff that you do where the only thing you really care about is the developer is the name of the file in which you're putting your styles. All right, let's do this. Let me go back over to VS code, close out register. HTML, open up a new file this time called Home. HTML, and let me purport to make a simple homepage for someone like John Harvard. I'll copy paste my boilerplate. I'll Change the title here just to be uh let's say uh home and then inside of the body of this page, let's do the simplest web page possible for someone called John Harvard. I'm gonna say here's a paragraph of text when John Harvard is going to be the person's name. Here's another paragraph of text. Welcome to my home page we'll be in the middle of this page. Then a final paragraph of text. Inside of which is like copyright, see about uh John Harvard down here. So it's a basic website. It's just three paragraphs. It's not going to be pretty, but let's make sure I haven't done anything wrong. Let me close my developer tools, click back, click home, and there we have it, the simplest of pages for John Harvard. Welcome to my home page, copyright John Harvard. Let's at least start to exercise some control over this. Let's change the font size and the alignment of the text. So back in VS code, let's go ahead and add uh For now, actually not even a style tag, but a style attribute. I'm gonna go ahead here and type in style, equals quote unquote font size large, and then text align colon center semicolon, and I apologize, but semicolons are back in CSS. Then in my next paragraphs open tag, let's do something similar but different font size colon medium for medium text, text align colon center, semicolon. Uh, and then lastly down here, let's do style equals quote unquote font size colon small because it's the footer, so who cares? Text align, colon center, semicolon, strictly speaking, at the last key value pair, otherwise known as a property, you don't need the semicolons, but just for consistency. I'll keep them uh for for that. All right, let's go back to this page, reload and watch all of the text a moment ago was left aligned and the same size. Now it's a little subtle, but it's clearly centered,

but it's large, medium, and small, respectively. Even if you've never seen CSS before, what rubs you wrong about this design though based on all weeks past. Yeah. Yeah, for every line I've been repeating myself with text line center, text the line center, text the line center, and if we really want to nitpick, these aren't really paragraphs, right? There's like no phrases or full sentences, let alone paragraphs. So it turns out there's a whole bunch of tags we can use to lay out a page, and in fact I'm going to transition to one that's a little more generic than paragraphs, namely div, which is just going to create a division in the page for me. And this doesn't have any functional impact, but semantically it's a little nicer because it means I've got the division here for. Header, the division here for the main part, and the division down here for the footer, it's just a different way of thinking about it. It's just different rectangular swaths of the page, but I like your point that text aligned center is kind of stupidly duplicated all of these times. Let me actually go ahead and first reload this change because there is one side effect that we might want to get back when I reload now using dubs instead of paragraphs. Well, there goes the nice white space in between my text. dubs just give me rectangle after rectangle, and as an aside, Let me control click or right click, open up developer tools yet again and notice this other trick with your elements tab. Whatever you hover over at the bottom of your screen will be color coded at the top of the screen. So if I dive into the body by clicking this little triangle, let me zoom in at bottom left. I can now see my own HTML much more pretty printed and colorful down here. If I click on this one or hover over it, you'll see that the first div, the rectangular region, is highlighted. Now the second, now the 3rd, that's all we mean by divisions of the page. This allows me to see my copy of it in the browser as opposed to in the original file. So just another technique for developer tools. All right, but I don't like this duplication, but here is now the C. In CSS cascading style sheets means that if you want one property or key value pair to sort of cascade down on all of the other tags inside of that one, you can do that. For instance, in the body tag, I can add my own style attribute here and put all of that text aligned center there. Why? Because div div div are the three children of the body tag to borrow a vernacular from family trees and from trees more generally. So this too should work because Text aligned center should cascade down now on all three of those children, and indeed if I reload the page, nothing visually changes, but it's arguably now better designed. All right, what more could we do here? Well, how about this? It would be nice to make clear to servers out there like search engines, like what's going on in the page semantically, and the term of art out there nowadays is the semantic web, which essentially is about putting more hints in your HTML so that servers like search engines kind of know more so what they're looking at. This is pretty generic right now div div div, but presumably the top of the page. Among the most important things because that's effectively like the header of the page, then the middle div is kind of the second most important because it's like the main part of the page and the footer is like the least important. So it turns out there are other tags in HTML besides paragraphs and dubs. There are literally tags like header which allows me to define the header of the page, main, which allows me to define the main part of the page, and then even footer, which allows me to define that too. So now if Google and Bing and other search engines are sort of crawling my website once it's public. They know that John Harvard's important because it's in the header. Welcome to my home page is important because it's in the main page. They're probably not going to care as much about the copyright because it's in the footer, so it's just providing more hints to these kinds of services. Moreover, we can do some other things here. This is kind of a hackish way to

implement a copyright symbol. HTML also has what are called entities. Where if I can do this magical incantation here, Ampersan hash symbol 169 semicolon, notice that VS code recognizes this as an HTML entity. If I go back to this page and notice my first approach was just parenthesis C parenthesis. If I reload now, having used that HTML entity, which I only know by having looked it up, now I get the copyright symbol that actually comes in the font that's being used here. All right, so let's transition now to this approach whereby I claimed before that you can actually use a style tag and why might we want to do this? Well, looking back at my code here, this is sort of hinting at potentially bad design even though there are different arguments for and against this right now I'm sort of combing. my data with my presentation thereof, like John Harvard. Welcome to my homepage and copyright such and such is sort of the data I care about, but I'm sort of mixing in the stylization of all of this stuff by putting CSS and HTML in the same place. So to be clear, all of the green stuff and well. Everything we've seen thus far, the tags and the attributes, that's all HTML syntax. Everything between the quotes is now CSS, and this is the first we've seen this before only in the sense that we've used SQL inside of Python code. Here we're using CSS inside of HTML code, but the CSS syntax is everything thus far inside of those quote marks. Wouldn't it be nice to kind of factor that out so that I can see it all in one place and better still factor it out ultimately to another file and I Do this as follows. Let me in my home. HTML, get rid of all of these style attributes and really go whittle the page down to its essence whereby I just have the header, main, and footer tags inside of which is that content. It's already easier to read, at least for me, the human. Inside of my head tag now though, let me go up and say style, and inside of this new style tag, let me show you another approach for stylizing the page. Up here. is where we can actually select elements to operate on using what are called selectors. So if I want to modify the style of my page's body, I can do that by typing body, and then I'm afraid curly braces are back in CSS 2. I can put text aligned center up here and the fact that I've put the word body before those curly braces just means all of these key value pairs, one in this case will operate on the body. Meanwhile down here I can say the header is going to have font. Size colon large, uh, the main part of the page is gonna have font size colon medium and then lastly the footer of the page is going to have font size colon small, you know, definitely more lines now, which isn't the best, but the effects now if I go back to my browser and reload visually is pretty much the same. I've just relocated all of those key value pairs elsewhere, but as a stepping stone now for doing something a little smarter whereby I. Now can lay the foundation for putting this in another file altogether. But first, let me note this too. The fact that I've put all of these key value pairs associated with specific HTML tags doesn't really make them very usable or rather reusable. And so when I alluded to earlier that these properties can be applied to different selections of HTML type selectors, class selectors, ID selector, attribute selector, let's just give you a little taste of this. What do we mean? Well, suppose that I want to generically be able to use text aligned center without associating it only with the body. Maybe I want to use this for a larger project where I want to center many things on the page. I can define my own keyword like the word centered, which doesn't exist per se, but if I prefix it with a dot, what I've just created is what's called the CSS class, and a class is just a set of key value pairs, properties. That you can associate with any HTML tags. Meanwhile, if I want this key value pair to be associated with the notion of large, I can define large. I can define medium, and I can define small down here. The motivation for which is that now in my page, if I want to center the body, oops, let me fix my own typo, if I want

to center the body, I can say please use the class known as centered on this tag. And then on the header I can say please use the class known as large on this tag, and then please use the class called medium here and then lastly, use the class called small here. So now in the spirit of a lot of the modularization we did in Scratch and in seeing Python of making your own fun. Classes aren't functions, but they are a way to encapsulate one or more properties and use or reuse them anywhere you want in a web page. It's not that it's not that impressive here in the short one, but it lays the foundation for doing much more interesting things soon down the road. In fact, let's take a step in that same direction. Let me go ahead and now highlight everything I've put inside of the Style tag. Um, and cut it onto my clipboard. I'm going to get rid of the style tag altogether. I'm going to create quickly a new file, comb.css, and I'm just going to paste all of that stuff in there and just to be nitpicky, I'm going to de-indent it so it's all left aligned. So all I've done is just move everything I just wrote into a new file called home.css. I'll close that out of sight, out of mind, but what I'm going to do now in the head instead of a style tag which contained all of that clutter, I'm To say link Href equals home.css and then this rel tag, which just means the relationship of this file to this one shall be that of a style sheet, and this tag 2 does not need to be closed, it just is. And now if I go back here and reload, still no changes other than the tweak the font a moment ago, still no changes, but now it's better designed with that file completely separate. So where are we going with this? Well, just to kind of circle back to something we did earlier, let me open up my terminal window and recall earlier we had this file like favorites zero. HTML, and this contained all of the data from a couple of weeks back that we solicited via that Google form and recall a bit ago when we went into favorite zero. HTML. I mean it was just kind of an ugly table structure, but it turns out in the world of In the world of HTML and CSS, there are also what we're going to call frameworks, which is a fancy word for library, but a framework is sort of a way of doing something by using someone else's library. And to do it their way, you just read their documentation and then you adopt their functions in the case of code or you adopt their CSS classes in the case of this example. So one of the most popular frameworks out there nowadays and among the simplest and best documented is one called Bootstrap, which is a set of CSS classes and other features that you can use because it's open source in your own code. In fact, all of the documentation is at this URL here. I read the documentation before class and I copied really the one line of code that I need to make favorites. HTML even prettier. So let me go back into VS code and let me copy my pre-made example from earlier. And you'll see that in favorites, whoops favorites 1. HTML, I have all of the same code, all of those lines of everyone's submissions, but notice I've added now this link tag, and it's a little longer than the one I wrote. It's referencing a third party website, JSD Deliver, which is a CDN content delivery network, which is to say a server that just serves up content for other people to use, but I copied that from Bootstrap's own documentation. And what I did here is the following I added. A class to my table tag specifically with a value of table and followed by a space table striped. Why? Well, I read bootstraps documentation at that previous URL and I liked the look of their tables because it lays it out with nice stripes like white and gray and white and gray and it sort of formats everything quite a bit nicer. So if I go into this version in my second tab by going back first and now opening up favorites1. HTML, same exact data. Two lines of change and voila, now we're talking. This looks much more like a table that you would see on any pretty website like your Gmail inbox or the like, all by simply changing the CSS and not really the HTML at all. So the motivation for introducing those classes a moment ago was so

that we can have reusability of code and better still, we can start to stand on the shoulders of others by using code that other people have written in order to improve the aesthetics of our own websites as well. All right, how about a couple of final flourishes with some style? Let me close out these examples here and let me propose to go into how about that same link example from earlier. So let me reopen Link. HTML, which recall had this phishing attack at the time. I'm going to revert this to the safe version and just say visit Harvard at Harvard's actual URL. Suppose I wanted to stylize this link beyond the defaults. Well, let's see what it looks like by default. If I go back into Li.H. Now, this is what it looked like before blue and underlined by default for the browser's decision, but I can override that in any number of ways to keep things simple, I'm just going to stay in my same file now rather than be pedantic about moving it to another file. And if I want to stylize the anchor tag just as before, I can say A, and then in some curly braces here I can do something like this color, colon, red if I want to make it crimson-like instead. Let me go back to VS code or my tab, click reload, and now we have a red tab. I can really geek out and if you remember your hexadecimal codes from our discussion of images a few weeks back, I can do has FF0000, which is a lot of red, no green, no blue. And if I go back to my other tab, click reload, same exact thing. You have that much control over even the color codes that you might use. Maybe you don't like the underlining in this particular case. Well, that's fine. I can do something like text decoration, none. Per the documentation I can reload and gone is that underline. Maybe it'd be nice to hover over the word and then see the underline. Well, I can do that too. Turns out I can have these pseudo selectors whereby I say the name of the tag, then a keyword like hover, which browsers know to recognize, and when I hover over an anchor, what I want to do is change the text decoration to underline temporarily. If I go back to this tab now. Reload looks the same, but as I move my cursor over, notice that it's underlining it for a visual effect. Let's see what's going on with my developer tools. If I right click anywhere and choose inspect, notice a detail I haven't showed us before is not uh is under the elements tab here. Notice if I go down to my link here and let me just make the right hand pane here a bit bigger. All this time but ignored up until now has been. This part of developer tools whereby I can actually see all of the CSS that applies to the element I have just selected, namely this link, and I see here in nice pretty printed fashion that I'm using this color FF 0000 text decoration none. Why is this useful? Well, one, if you want to learn from another website how it's doing its thing, you can just look at the CSS, but also if you want to be able to iterate more quickly and just kind of tinker with things. I can actually turn the color on and off by just hovering over the inspector here and just turn it on and off by clicking and unclicking, and if I want to just play around with, oh maybe maybe Harvard should be 00 FF 00 E, I can make a green instead, so you can temporarily change the browser's copy of your own HTML or CSS just to tinker and iterate quickly, just like I tinkered with Stanford's own website or at least my own copy thereof. Lastly, how about in terms of the selectors? These are using type selectors, that is selecting the name of the tag. If I want to actually affect one tag specifically, a very common convention is to give an HTML element a unique ID. For instance, I'm going to call this Harvard, and by honor system I should not give any other element in this. Page in ID of Harvard. The motivation is that I can now uniquely identify this tag by for instance changing this to hash Harvard, which is just the convention for specifying that it's not a class now. It's instead an ID. You do not put the hash though in the actual value down here and what I can even do down here is something like um. Uh, Hash harbor to scope that as well. If I now reload, we're

back to the red version and the same functionality as before, and it's just a more precise way now to target your CSS properties to a very specific element instead. OK, that was a lot. Any questions on any of this thus far? Now that clear. All right, well, one last language for the day, and, and we do mean what we say like that is the extent to which you will learn formally HTML and CSS. Like everything else just follows those exact same patterns. It's different classes, it's different attributes, it's different tag names, all of which can be picked up through practice, through osmosis, through references, but that's really it for the fundamentals. And so our last focus today is on an. Actual programming language that we'll just scratch the surface of if only because it's so darn omnipresent nowadays. Most every website you use is made from not only HTML and CSS, but if it's in any way interactive, odds are it's using JavaScript, a programming language that is very commonly used client side, whereby humans write the code on the server, but then your browser, as before, downloads it to the client and then it runs in your own Mac, your PC or your phone. That said, JavaScript. Also very popular on the server nowadays. It's not just a browser-based language. In JavaScript. What you have most powerfully though is the ability in memory to mutate this tree in real time. In other words, think about even your Gmail inbox or your Outlook inbox. Typically you see email after email after email after email. Odds are per today what HTML tag is creating that UI of row after row after row. Which tag? Like table tag, like the table tag, probably right table row, table row table row, but it wouldn't make, well, actually this is the way things used to work in my day back in the day when you visited, not even Gmail before it existed, but your email inbox, you would download from the server a web page containing a table tag with table rows and table data elements, and that was your inbox. If you wanted to see if you got new mail, you'd just reload the whole page and it would download new contents from the server and show you the new HTML with just. JavaScript though, which has come onto the scene over the past 20 plus years, you have the ability to download the data once initially, then use code to just grab some more data every 30 seconds or some more data pretty much any time an email arrives. And if this picture here represents not our super simple hello title, hello body page, but a whole bunch of table rows for your existing email, the moment you get more email, you can use JavaScript code to add another node to this tree, another node to this tree representing the table row tag. The table row. Tag again and again. So in short, with JavaScript you have the ability to change the tree, otherwise known as the Document object model, or DOM for short, dynamically in order to evolve the web page. So let's take a quick tour of what JavaScript does have syntactically and then I'll just demonstrate some of the capabilities thereof without dwelling today on syntax beyond this. So in Scratch, which is looking pretty good now, you had conditionals which look like this. In JavaScript it's pretty much the same as C, the curly brace. are back at least for two or more lines, but indentation doesn't matter except for the style thereof as in contrast with Python. If you have an if L, it's going to look the exact same in C. If you have if if L, you have the exact same thing in C, different from Python because this was L if in Python. Now we're back more verbosely to Ls if as in C. Variables in JavaScript. Well, here in Scratch is how you set a variable counter to zero. In JavaScript, there's a few ways to do this, but the most A reasonable for now is to let counter equal 0, so you don't specify the type. This is more of a polite way of asking the browser. Please let a variable called counter exist and set it equal to 0 by default. Semicolons are back, however, that's not strictly true. Browsers are smart enough to know where semicolons actually matter, but for our purposes assume that they're always there. How do you change counter by 1? Well, you can do it the pedantic way, which is a

little verbose. You can do the plus equals trick or nicely back in play is the plus plus in JavaScript. Just like in C, but not in Python. Loops in JavaScript. Well, in Scratch, if you want to do these 3 times, here's how you would do it in JavaScript. It's pretty much the same as C, except for not mentioning the data type. Instead you use the keyword `let` here, but otherwise this is exactly the same as in C. If you want to do something forever, for whatever reason in JavaScript, you can say `while true`, which is exactly how we did it in C. If you have a web page like this, meanwhile, And you want to insert some JavaScript to it, you can do it in a couple, a few different ways. You can put a script tag just like the Style tag in the head of the web page. This can get you into trouble though for reasons you might encounter whereby if you put your JavaScript code up here and you try to use it to modify the web page, but the web page isn't defined until down here, you can get into some a race condition really where the data does not yet exist. So instead of putting it there or even in another file, it's actually. Common to to avoid that altogether by putting your script code or your script tag at the end of the page just before the end of the body to ensure that all of the webpage exists already. This is similar in spirit to the deaf issues we saw in Python or the prototype issues we saw in C. There's bunches of solutions though to this here problem. But let's now take some JavaScript for an actual spin and use VS code to write some of it as follows. In VS code, let me go ahead and close link. HTML. Open up my terminal temporarily and let's improve my, actually, let's just improve the very file hello. HTML that I have here in front of me and actually have it be more interactive and give me sort of a pop up on the screen when I type in my name. So let's start as follows. First, let's go ahead and change this just to uh hello, just for short. And in the body of this page, let's give myself a form, and in this form, let's give myself an input. Uh, we'll turn off autocomplete just to avoid distractions. We'll turn. On autofocus to save me a click, I'm going to give this HTML element an ID uniquely of name, a placeholder also of name just so the human knows what to do, and the type of this field shall be text. In other words, I want to create a program a week one and week 0 where I type in my name and say hello such and such. I'm going to give myself an input, a submit button with input type equals submit. Don't really care what the button says, but I do care now when I go back to my other tab, close my. Developer tools, go back into hello. HTML. I now have something that looks like this. It looks similar to our search example for cats, but now I'm asking the user for their name along with the submit button. But what I want to have happen is when I type in David and click submit, I want to see Hello David somewhere on the screen. Well, how can I do this? Well, a few different ways, but JavaScript allows me to do things like this, and for upcoming problem sets, you won't necessarily have to write JavaScript like this. So consider this a whirlwind tour, not so much something to ingrain. Here I can add a new attribute to the form tag called `onsubmit`, which, as the name suggests, means call the following function when this form is submitted. Well, what function do I want to call? I'm going to call it a greet function, and that's it for now. How do I define a greet function? Well, I could, among other places, put this inside of the head of. Page in a script tag. I can define a function in JavaScript by literally saying `function` and then the name of the function and then in parentheses any arguments there too. I'm not going to have any. And then in Curly braces I can actually define the meat of that function and for instance I can do this. Let name equal the following document. query selector. And now what I want to do is this Document is a global variable that just comes with JavaScript in the browser that allows me to write code involving the whole document, the webpage itself. Query selector is a fancy name for a function. That lets me select specific elements of the page

using CSS selector. So the very same syntax we saw with names and with dots and with hash symbols a moment ago are back in play for JavaScript here. So if I want to create a variable that stores the name that the human typed in, what I can do is pass to query selector a selector for that element which is quote unquote hash name or hash. Just means ID, but the reason I'm using name is because the unique identifier I put here is name. If I change this to Foo nonsensically, that's fine. I just have to change this to Foo up here, so I'm in full control over what is called what. But if I want to get the value that the user typed into that box, I now do value, and we've seen these dots before. In C they were for accessing structs. In Python they were for accessing contents of objects. So this just means use the document global variable. Use the query selector function or method inside of it. Get the element whose unique ID is name, and then go inside of that text box and give me its value. So it's a very long winded way of saying store the user's input in a variable called name. But what's nice now, even though this is going to be a bit ugly. I can then use a built in JavaScript function called alert, and I can say something like hello, comma, close quote, then plus, which we've seen before in Python and concatenate with it that name's value. Now this isn't quite complete and for reasons I'm going to wave my hand at, I also need to add annoyingly returned false down here because otherwise if I click submit, yes, the great function will get called, but the browser will still try to submit the form to a server, which is going to interrupt my own code. So long story short, this is a bit of a hackish approach for now. So just making sure that the only thing that happens when I submit this form is that my function is called. Now if I didn't screw anything up, I should now see after reloading this page. A prompt for my name. I'll type it in and when I click submit, I should see an ugly but functional alert box pop up with dynamically generated text, namely hello David. I say it's ugly because by convention, Chrome shows you the full URL or the domain name of the website in question, which is my randomly generated one, which does look stupid. So we can do better than this. But the point is now that I have written code in JavaScript to listen for the submission of this form and when that happens call that their function. And this is generally the paradigm of JavaScript. There exists in the context of websites a whole bunch of events that can happen and this is a word we haven't used since week zero in Scratch. Recall that in Scratch you have events like when green flag clicked and when the green flag is clicked, you can do something in response. Same thing in the world of web programming. Here are just some of the events that can happen in a web page like. The user can change something, click on something, drag something, key up, put the keyboard up, put the mouse down, or other things. What I'm listening for is the submission of a form, which is cool because in JavaScript then you can essentially write code that listens for any number of these events and then does something when it happens. Consider after all in Gmail, if you click the little refresh icon within Gmail itself to get new mail, it runs some JavaScript code, it turns out to talk to Google's server. Get more email and update your site. If you click and drag on Google Maps to see like higher up geographically, well, what's happening? Some JavaScript code is listening for your mouse going down and dragging so as to go fetch more tiles, more rectangular pictures of the map wherever you're trying to drag. So anything that's interactive in websites nowadays like that is using JavaScript by just listening for things that you or someone else might actually do. Well, let me go ahead and start opening some pre-made examples just to give you a sense of the other syntax that is in use today with JavaScript. I'm going to go ahead and open up a version of this Hello program called Helloo2. HTML, which is different in that I'm practicing what I preached earlier by putting

the script tag at the bottom of the page just to ensure that the form and everything inside of it already exists for sure by the time this code executes. Moreover, what I'm getting out of is the business of using the on-submit attribute. So just as I tried to get my CSS out of my HTML and put it elsewhere, similarly, I'm trying to get my JavaScript code like the greet function out of the HTML and putting it down here. Now why is this useful? This is a big mouthful, but it just follows a general pattern as follows. `document.querySelector('form')` is just getting a reference to the actual form element in the page. So if you imagine in your mind's eye that this is drawn out as a tree in the computer's memory, this is just getting me a point or two, the form node in that tree. Haven't seen this before, but it kind of does what it says. `addEventListener` is a function or method that you can call on any element that just tells it to listen subsequently for this event, and when that event is heard, submit in this case, call the following anonymous function, otherwise known as a lambda function. But long story short, this syntax just means when submit happens on that element, execute the code between these curly braces. What happens? `alert('hello');` `document.querySelector('p').value`. I didn't bother with the variable this time. This does exactly the same thing but is a purely JavaScript solution without using the `onsubmit` attribute, and we show you this only because, especially for final projects, you might want to do something like add event listener to make like maybe a drop down menu or some interactive clickable thing in your website that just listens for one of these events to happen before actually executing some code. Notice I've been conventionally using single quotes in JavaScript because that's just a thing in the JavaScript community to generally prefer single quotes over double quotes. Why? Well, it means people in JavaScript are hitting the shift symbol like much less than the rest of the world to get double quotes. It's just a convention so long as you're consistent, either is fine, conditional on not having actual apostrophes and text and such. Let me show you one other convention. Instead of putting my code at the bottom of my page just before the body ends, it is also alternatively conventional, as in hello 3. HTML to do this, to still maybe put the script tag at the top of the page, but to additionally have this magical line whereby you add an event listener before you do anything else that listens for this crazy weirdly named event called `DOMContentLoaded`. But now that you've heard DOM briefly, DOM is document object model. It just means the tree in memory. This is just the fancy way of saying when that tree is loaded, go ahead and do the following, and this ensures that when a browser reads all of this code top to bottom, left to right, this code won't actually be executed until the whole DOM is loaded into the computer's memory. That whole tree is built so that's all that's being referred to there. The rest of the code is actually exactly the same. Um, what more can we do? Well, just so you've seen it, I can delete all of that code, move it to a file called like `hello.js`, and in the fourth version of this example, I'm back to just HTML because I can put all of the fancy complexity inside of my script tag here, factoring that code out into `hello4.js`, but the code is otherwise I claim unchanged. All right, there's a lot. I know it's quick, but do the general principles make sense, like just listening for events and running some code in response. That's really all we're talking about a week zero. With scratch. All right, well, let me let things escalate just a little bit, and this time I'll open the demo first. Let me go ahead and open up hello 5. HTML, which I wrote in advance, which, OK, is definitely starting to look like a mouthful, but in a moment it'll make a bit more sense. Let me go ahead into my other tab here, click back, go into Source 8, which is all of my pre-made examples, and I said that we're in Hello 5 now. And in Hello 5 there's no submit button because watch this

fanciness when I search for something like C. Uh, or David as a full fledged word there. Notice it's just happening inside of the web page. Moreover, if you poke around, let me right click on the page. Let me inspect to open my developer tools. Let me expand the body down here and actually let me reload the page. So notice by default this is what my web page looks like. It's just got an empty paragraph tag for some reason, but watch what happens at the bottom of the screen, and I'll zoom in a bit more when I start typing my name like D. And then let me expand this triangle. You see at the beginning, A, V, I, when I say that JavaScript can mutate the DOM, the actual tree in memory, like that's what you're seeing. You're seeing the HTML pretty parented color coded version of that tree in memory and how is it working? Well, if we go back to the code here, well, let me wave my hands at this first line. This just means don't do this until the whole DOM is loaded. Let's look at this line, which means give me a variable called input and set that equal to. OK, the input tag on the page, the text box, and then do what? We'll take that input, add an event listener that's forever listening for key up, like my finger going up off the keyboard, and when that happens, call the following function, which has no name, but that just means call these lines inside of the curly braces. Well, what happens inside of those curly braces? Well, here's a variable called name, and this is just pointing at the paragraph tag apparently. I'm checking this question, if there's hm if input. value, so this is like saying if input value does not equal quote unquote, just implicitly, go ahead and set the inner HTML of that name. Variable equal to hello input. value. Now this is crazy syntax and I'm showing it just because you'll see it on documentation online. This is similar in spirit to Python's F strings. It's ugly syntax with dollar signs and curly braces and worse yet back ticks. However, this is a manifestation of really the JavaScript community presumably deciding that if you want the language to evolve. So you have to make sure you're backwards compatible with old versions of the language. So they chose characters and syntax that probably do not appear already in the wild. That's why sometimes things look uglier, I would surmise than otherwise. But long story short, this just means if there's input there, go ahead and say hello, input. Otherwise it says by default hello whoever you are. And in fact, if I go back here and delete my name, watch. What happens, it goes back to that default. So here's just an example of listening for keystrokes going up and down and making sure that the page responds accordingly. How about something else? Let me go back into my directory listing. Let me open up Background. HTML, which I wrote in advance. It's super simple, but this is the first of like an interactive website that has 3 buttons labeled R, G, and B, as you might imagine, clicking on R does that. G does this. B does that. Well, how is this working? This is the first example now where you can use JavaScript code to alter CSS dynamically. So let me reload the page so it's back to white. Let me open developer tools and watch what's happening now on the body tag specifically. Initially there's no stylization on the body other than the browser's default margins and whatnot over here, but watch what happens at bottom right when I click on the R button. You see that all of a sudden background color coal in red was dynamically added. Now it's green, now it's blue, and notice the HTML at bottom left is changing too. So somehow I am listening for clicks and then changing CSS in response. So if I go back to VS code, let's close hello. Let's open up Source 8's version of Background. HTML and in here it's a bit of a mouthful, but the HTML is simple. Here's 3 buttons, and because I wanted them to be uniquely identifiable, I gave them all IDs of red, green, and blue, respectively. And then this code is a bit of copy paste, and frankly I could probably avoid that if I were more elegant, but just to be pedantic, here's what's happening. Here's a variable

called body that's just getting the body element, the node in the tree at that moment in time. And then these three lines of code, their purpose in life is to handle the red clicks. How? Well, we're telling the document to select the element whose ID is red, listen for the click event, and whenever that happens, do this body, which is the same variable as before, style, which we haven't seen before, but any element can have a style property associated with it in JavaScript. Background color equals unquote red, and the other blocks of code are the exact same thing for green and for blue. The whole. here is we're now listening for clicks on buttons and changing not the contents of the button, but rather the style thereof of the whole page. As an aside, this is curio curiosity. This is what's known as camel case whereby like a camel has a hump in the middle. This word has a hump in the middle like capital C all of a sudden to separate the two words. In CSS recall it was a moment ago background color. Anyone want to guess why this is not how you write it in JavaScript? Anything with hyphens in CSS is changed to Camelcase in JavaScript. Uh, it's not related to comments. It's simpler than that, yeah. Yeah, right, and like left hand wasn't talking to right hand, and people were like, oh damn it, like this now means background minus color, which is not a thing because minus is indeed just like in CNN Python, a mathematical operator. So the world decides to reconcile this problem by just capitalizing. Uh, the character that would otherwise be where the hyphen is. Well, little CSS trivia. All right, what else can we do? How about a couple of final examples here? So what more can we do with CSS? So back in my day too, we had a tag called the HTML blink tag, which is among the few tags in the world of HTML. That has actually been deprecated that is removed from language. Like no one removes things from languages generally, but the blink tag was so hideous followed only by the marquee tag where by my own home page just like a freshman had like welcome to my home page just moving across the screen like this from left to right for no good reason, like an ugly marquee and like. on like a digital signage nowadays, but we can bring it back as follows. So if I close out my developer tools, go back into my source directory, and open up blank, this is what the blink tag used to do back in the day. Now this version is implemented instead in JavaScript code as follows. I have a function here called Blink, which I'm apparently calling every once in a while. How is that happening? Well, let's scroll down. Here's my HTML. Super simple, literally just says hello world, but notice this there's another global variable we haven't seen in JavaScript called Window that refers to like the general window, not necessarily the contents of the page, where you can call a method called set interval and you can tell that method set interval to call a specific. Function every number of milliseconds. So if I want to call blank every 500 milliseconds, that's the line of code that I use. If I scroll up to now this function, let's see how blink is implemented both now and perhaps back in the day. Well, body is a variable here that's just pointing to the body node in the DOM, and this is a big mouthful, but if that body's styles visibility property in CSS is quote unquote hidden. Then change that body style's visibility property to be visible. Otherwise change it to be hidden instead. Here too, don't understand why left hand and right hand weren't talking to one another. You would think that the opposite of visible would be invisible, but in CSS the opposite of visible is hidden. Just have to memorize stupid things like that, but what's this really doing? It's just changing the CSS from hidden to visible, hidden to visible every 500 milliseconds. So in fact what you're seeing here in the blink is if I inspect this page too and now notice it's kind of fun just to watch it, you can see the HTML at bottom left and the CSS at bottom right just automatically changing because I'm doing that every 500 milliseconds. All right, how about one other? Well, autocomplete. Well, we

saw. a step toward this with my he David example a moment ago. Super common though in Google and like every website now to automatically try to finish your thought. How is that happening? Well, that's not just HTML and CSS. That is also some JavaScript thrown into the mix. So for instance, let me go into my terminal and open up Source 8's example called Autocomplete. HTML, and here I am going to borrow a file called Lage.js, which is just a massive version. I'll Open that too if you're curious. Large.js is just a huge JavaScript array, a razorback containing all of the words from Problemset 5, the spell checking problem set, where you had 100,000 plus words in in a file given to you. Now we've converted that to JavaScript by using a global variable like this. In the code here, what's happening? Well, apparently there's going to be a text box at the at the top of the page that we see. Then there's an empty unordered list, so an empty bulleted list, and then there's this code down here. I'm apparently creating a variable called input that's referencing that text box. I'm then listening for key up just as like we've done before, and then I'm doing this. I'm setting a variable called HTML equal to quote unquote nothing, so an empty string, and then I'm checking, does the input text box have any value implicitly. If so, what am I doing? This is kind of cool. It's a bit of Python and C together syntactically. For each word in the words array, JavaScript uses the keyword of instead of in like Python, but so be it. What I'm doing now is in JavaScript I'm saying if that current word in that big file of 100,000 words starts with whatever the user typed in. Go ahead and add to that HTML string using plus equals, which is just concatenation, we've seen plus before, the following an LI tag inside of which is that specific word. And so in effect what you're seeing now is what every, almost every website nowadays does. They're not manually writing HTML like we've been doing much of today. They're writing code that dynamically generates HTML because the programmers understand what HTML is. They understand that unordered lists have LI children and so using this string that I've highlighted, they're creating LI elements after LI element for the purpose of changing the inner HTML of the. UL element to be the value of that variable and this is a very long way of saying how is autocomplete implemented in general. Well, just like this if I search for cats by typing in C, there's every word in that 100,000 dictionary that starts with C, A, T, S, and there's every word that starts with C A T S. Meanwhile, watch what happens underneath the hood if I open up my inspect tab again and I go to my body. Inside of this is the empty UL, but watch as soon as I start typing something like C, now I can expand the triangle because there is an LI element that's been created for every one of the words that match as I do ATS. Now I've got just 4 of them, and there is cats, there is catski. And so forth. So any time you go to google.com like we did earlier, we went to Google.com and started searching for CATS. Where are all of those search results coming from? Someone wrote JavaScript that's listening for keyU or the like, and then dynamically populating an unordered list or in this case a much prettier list of the matching results. And the final example that we thought we'd leave you with, and again, the whole purpose of introducing JavaScript is to give you a taste of its syntax and its relative familiarity, but with the. that you can the power with which you can leverage it to make websites so much more interactive. And in fact with Bootstrap you don't just get CSS you can use, you have a whole set of JavaScript functionalities so you can have drop down menus and the like. For instance, for instance, among the things you'll use for an upcoming problem set and perhaps your final project, something that looks a little like this in bootstrap. HTML. Here's a whole bunch of code that I literally copied and pasted from Bootstraps. Documentation and it's just like boilerplate code for a corporate website that has

features with pricing and disabled menu options as well just for the sake of discussion. And then here if I go back into this example you'll see fairly simple website that looks like this, a so called Nav bar with all of the main menu options of like a corporate website and notice if you start to resize the window, which I'll do here and put it into sort of mobile mode because it's so narrow now thanks to JavaScript. It's listening for clicks on this hamburger menu and revealing the menu options that way. This is quite like how CS 50's own website works and so many other websites out there. But the last one we thought we'd use is you're so in the habit of using Google Maps or Uber Eats or any number of apps that need to know your location, that too is exposed through JavaScript quite simply. Let me go ahead and in geolocation. HTML, open up the following code whereby Super simple, even though some new functions, there exists another global variable in JavaScript in browsers called Navigator, which has a property called an object called geolocation which has a function called get current position that takes an argument which is just an anonymous function, which means call this code when you're ready to know the coordinates because it might take a while to figure out your GPS coordinates. And once you do, this simple example is just going to write to the document that is the rectangular page. The positions latitude that comes back and the position's longitude that comes back. So to see this in action, let me go ahead and uh open up that second tab, go back into geolocation. It's notice for privacy sake, it's asking me to approve this, so I'm gonna say allow this time. There are apparently my laptop's GPS coordinates, and if I go to Google Maps.com, I can actually paste this in here, enter, and. Looks like if we zoom in, in, in, OK, I'm not technically outside, so it's only close to a degree of precision, but it's probably mapping to one of the Wi Fi access points that's on that corner of the building, so we're pretty darn close, pretty much close enough to get me my my food or my, my ride here. And a final note, now that you've seen a little bit of JavaScript, let me go ahead and open up just 60 final seconds of just how, how much effort it took us to put not only this lecture together but particularly that example of the teaching fellows passing packets. Everything we like to think is very finely flourished here, but here's a little bit of behind the scenes in these final 60 seconds together if we could dim the lights before we adjourn. OK. Buffering, OK. Gosh That's Moy got, no, oh wait. That was amazing, Josh. But Um, Sophie. Amazing. That was perfect. For me? I think I Thanks. That was amazing. Thank you all. All right. That's it for CS 50. We'll see you next time.

All right, this is CS 50. This is already week 9, and I dare say this week is the most representative of what you'll be doing after the class if you so choose to program in the future and tackle some project that's new to you. In fact, the closest to this week was perhaps week 6, wherein we didn't really introduce all that many new concepts but really translated them from C into Python. And so this week, in particular, the goal is to really synthesize the past 10 weeks of class, drawing upon a lot of the building blocks that are. Hopefully now, metaphorically in your toolbox and gives you an opportunity now to apply those ideas to new problems, in particular web programming. So every day you and I are using the web in some form, every day you and I are using mobile apps in some form, and we said last week that the languages underlying a lot of those applications are HTML and JavaScript for the layout and aesthetics, and then also in part JavaScript for a lot of the client side interactivity that you might experience nowadays. Well today we come full circle and bring back a server. Side component whereby we'll again write some Python, we'll again write some SQL code and use it to make our full fledged own web applications and in turn if you so choose mobile applications as for your final project as well. So

up until now when we did anything with the web, you ran this command last week, HTTP server, which literally did just that. It spawned a so-called HTTP server that is a web server whose purpose in life is just to serve up content from like your current folder, any files they're in, any folders therein. And so all of the U. URLs generally followed a certain format. So if your URL were example.com recall just denotes the root of the web server. And so in there typically by default you would see a directory index. We'll see today that that goes away because generally when you visit something.com/, you want to see the actual website, not the contents of everything in the server. So we'll see how to address that. But the URLs up until now have been a form like file. HTML, literally referencing a file in that folder or. folder slash, which just means whatever is inside of that folder or folder file. HTML or you can nest these things however long that you want and recall that more generally we said that you're referring to some kind of path on the server where path is a step of folders ending in perhaps a file name. So today we're going to generalize that at least in terms of nomenclature and start talking more about routes because essentially in web programming we are going to exercise a lot more control over what is in the URL. So back in the day. It referred to literally a file on the server, and as recently as last week, the URLs referred to literally a file on the server. However, we'll see in code that we can actually just parse this, that is analyze what is after the domain name in a URL and just use this as generic input to the server to figure out what kind of output to produce. We're going to see the same convention though if you want to pass in specific parameters, key value pairers, we'll use a question mark after our so-called route, key equals. And then if there's another one or more, we'll just separate them by ampersands. And to do all of this we're going to recall the inside of those virtual envelopes recall that if we did something like on Google.com to search for cats, what was really being sent to the server was a request for slash search, which notice is not search. HTML. There's no folder per se there. This is just the name of a program really running on Google servers, and that's going to be the so-called route that we ourselves start programming today. Question mark. cats just meant that the query parameter, the input from the web form, is going to contain in this particular example the word cats. So how are we going to do I'll do this? So we could implement our own web server in C. It would be a nightmare to like use a language as low level as C and actually deal with something as high level as writing code for the web. We're instead going to use Python for the most part, if only because it's much higher level, but even then we would probably, if we wanted to do this thing from scratch, we would have to write a A lot of Python code to like analyze the insides of these envelopes, figure out what inputs are being passed to the server, and then figure out how to access that in Python code. It's just a lot of work to just get a web application up and working. And so what the world generally does is they don't reinvent the wheel of writing their own web server. rather they use an off the shelf fairly generic web server or application server as it might be called, and we, for instance, are going to use something called Flask. Now Flask is a framework as the world would say, or More specifically, a micro framework, which just means it's a library of code that other people wrote to make it easier for us to implement web applications. So they took the time to figure out how to handle get requests on a server, post requests on a server, figure out how to extract key value pairs from URLs, the sort of commodity stuff that like literally every web application on the internet has to do anyway, so we don't have to retrace those steps ourselves. What this will allow us to do is only implement the problems that we care about by using. This framework and to be clear, a framework much like bootstrap is not only a library that someone

else has written for you, but it's like a set of conventions that you follow in order to use the library in their recommended way. So it's more of a generic term that includes library and a set of conventions. And how do you know how to use either? You just read the documentation or take a class in which we're about to give you an introduction to some of this right here. So instead of running today HTTP server to start a web server that just serves up static content. Files and folders in our account. We're instead going to run the command moving forward flask space, run, and this is going to look for code that we've written in our current directory. And if it is in accordance with the conventions to which I'm alluding by using this so-called framework, then it's going to start our web application on some TCP port, for instance, 8080, as we discussed last week. To do this, all we have to have in our current folder is minimally a file called app.py by default. This is hinting at an. Application written in the language called Python and what code we put in there we'll soon see. And then ideally we would have another text file called requirements.txt by convention inside of which is just one per line, the name of all of the libraries that we want this web application to include. In other words, if I go over here to VS code, if I don't have such a file, that's fine, but I want to use a framework like Flask, recall our PIP command for installing Python packages. I could just say PIP, install, flask, E. And that would go ahead and install the flask framework or library for me just like we did a few weeks ago with installing the silly little cow say library as well. I've already done that in advance and better still, I've installed, I've come with uh my code today, both of these files app.py and requirements.txt. And in fact, if I go ahead and create one just for fun here, all you need to do in a requirements.txt file is literally put the name of the library that you want to include, and then you run PIP in a slightly different. Way to install that library or any other libraries that are in that file as well. So let me wave my hands at the requirements.txt for moving forward. It just means what libraries do you want to use with this web application so you don't have to remember or memorize them and type them all out manually. All right, so what's going to go inside of app.py? Well, the minimal amount of code that we can write to make our own web application that does something like print out Hello world to my browser could look like this. Now there's a bit of new syntax here, but All that much today moving forward, the very first line just says from Flask import flask, which is a weird way of just saying give me access to the flask library. Capitalization no matters, and so the package that we're using is called Flask lowercase, but we want to have access to a special function in there called Flask F. So this is sort of a copy paste line. The next one's a little weird looking, but it essentially says Give me a variable called app and turn this file into a flask application. We haven't seen. In a few weeks, but there was that weird if conditional that we put at the bottom of some of our Python code a few weeks back that just said if and it mentioned in there name if name equals equals __name__. So we've seen an illusion to name for our purposes. Name just refers to whatever the name of this file here is, no matter what I call it, you can sort of access the current file by way of this special global variable. So this line collectively just means turn this file. into a flask application and store the result in a variable called app so I can now do stuff with flask. And what am I going to do? Well, down here, let me first point out a familiar syntax. I'm defining a function that I called index by convention, but I could have called it anything I want, whose sole purpose in life is just to return quote unquote hello world, which is the super simple output this web app is going to display. But, and this is the new syntax I'm using here what's generally called a Python decorator. Which is a type of function that essentially affects the behavior of the function right after it. So by saying @app.route, this is

telling the flask framework, associate this index function with this route, the single forward slash, and that's how we're going to take over the default behavior of the slash portion of the URL by telling it to return whatever this function returns, and we'll see this in action. Now, so let me go over here, say to VS code, and within VS code, I'm going to whip up exactly that application in a file called app.py just so as to combine this and some subsequent examples maybe the same folder. I'm going to first create a directory or folder called hello. I'm going to go into that hello folder. I'm going to go ahead and recreate that same requirements file just for good measure to tell the world that I want to use the flask library here and then I'm additionally gonna create now app.py, and I'll type this fairly quickly, but I'm just reciting what we saw a moment ago from the flask package, import the flask function, lowercase F F respectively, then give me a variable called app, set it equal to that function call passing in the name of this file, whatever it actually is. And then lastly, let's go ahead and call at app.route '/' which says hey Python, whatever the next function is associate it with this slash route and so I'm going to define that function. I can call it anything I want foo or bar or baz but insofar as slash represents the index of the website like the default page, I'm just gonna go ahead and call it by convention index and then return for now hello world. And that's it. So whereas last week when I was writing code in HTML files I was making web pages. Now I've created what we'll call a web application, and it's an application in the sense that there's actually some logic going on there. There's some functions, there could be some conditionals, there's clearly a variable. There could be loops and all of the sort of stuff we've seen in Scratch and C and Python as well, we'll now see back in this Python. File. So how do we now run this? Well, let me go back into my terminal window here and I'll clear it just for good measure. I'm going to go ahead and run flask, run, enter. I'm going to see some cryptic looking output, but there's that familiar pop up with the green button that wants to open up this application, whereas HTTP Server uses 8080 by default. Flask uses Port 5000 by default, and here we have it. I've just opened up my Second tab and we spent a lot of time there last week. This is the server I'm running, not on Port 8080, but on Port 5000 today, and there is the contents of what was spit out by my very first application. Now even though the browser is rendering this like it is a webpage, notice this if I inspect, if I right click or control click anywhere on the screen and go to View page source, you'll see. There's no actual HTML on this page. It's literally a single line of text, Hello, world. If I close that and right click or control click again and go to inspect like we did last week to open up developer tools, you'll see that the browser has actually filled in some blanks here for me by just rendering as it should the minimal possible web page, but the content I actually sent to the web browser is only literally hello world. So how can I actually send a web page of my own rather than letting the browser do something like this? Well, I could go ahead and. Close that and go back to my application. I'm gonna go ahead now and hide the terminal just because the server is still running, and what I'm gonna go ahead and do here is, well, nothing's really stopping me from returning not just a string of texts, but a string of HTML, and this might not look pretty, but let me go ahead and do open bracket doc type, HTML close bracket, then HTML, then head, then title, and I'll just title this for instance hello to keep it simple. backslash title, backslash head open bracket bod. hello world backslash backslash HTML close quotes and I used single quotes in this case, but I could have just as easily used double quotes, but that's a full fledged web page. Like that's the minimal amount of content we saw last week. Actually, you know what, for good measure, let's actually add lang equals. So it's actually fortuitous that you use single quotes because now I have some double

quotes inside. And even though this is not pretty printed, it's just one massive mouthful of HTML all along one line. When I now go back to the browser, reload the page. As by clicking here and then view page source again, here's what my browser received this time. Indeed it's the full fledged HTML and in fact if I close that tab and reopen developer tools by inspect, now we'll see in the tab absolutely everything that I sent over, including a title, including the lang equals the N, and had I typed even more, we would have seen that too. All right, so what was the point of this exercise? It feels as though that I've really just taken more time, added more complexity to achieve literally what I could have done last week by just creating Index. HTML myself without any Python code. But I dare say what we're trying to do is lay the foundation for a full-fledged interactive website that maybe has forms that we can submit to the application that allows us to generate not just one page, but maybe 2 or 3 or any number. So what you're seeing here is sort of the beginning. Of Google.com's search application or gmail.com itself or Facebook.com or any web application you can think of begins with a little code that theoretically looks a little something like this. But this is kind of stupid to put HTML hard coded no less in one long string here inside of my application. Let's try to factor this out. That was a lesson we preached last week about sort of factoring out our JavaScript, factoring out our CSS. We can do the same thing with our actual. HTML here and so what I'm actually going to do is import not only the flask function but also another function that per its documentation comes with Flask called render template with an underscore in between. This is a function whose purpose in life is to render a template, so to speak, of HTML. We'll see what we mean by template in just a bit. But down here what I'm going to do is now delete all of that code and let me just assume that I'm going to put that same code in a file called index. HTML just. I did last week. So let's instead return the return value of render template of quote unquote index.html. Now that file does not yet exist. Indeed, if I go into my terminal window, create a second terminal just so I can leave the server running but still see what's going on, I'm going to CD into that same Hello directory type LS to list my files, and I only see app.py requirements.txt. But it turns out per flask's documentation. If you want to create your own HTML files, you simply have to add a directory that by convention is called templates, and that's it. So in addition to app.py requirements.txt, I need a folder called Templates. So let's go back into VS code, make templates capitalization matters, all lowercase. Now let me go ahead and CD into templates and run the code command and create a file called index.html in the templates folder and then super. Quickly, let me hide this. Let me whip up that same page again. Doc type HTML, HTML lang equals EN close bracket head close bracket, title close bracket, hello, and then down here body close bracket, hello, comma world. So autocomplete is helping me type quickly, but now I have a file with my HTML that this application I claim is going to spit out automatically for me. So let's see the effect. Let me go back into my other browser tab. Let me close the developer tools and let me quite simply just click reload. And no apparent change. It's working exactly as it did before, but I've laid the foundation for making a much more useful layout of my files so that I can actually keep my logic, my Python code, and my HTML a bit separate from that. All right, well, how can we make this into something even more interesting? Well, let's start to take some actual user input, for instance. So wouldn't it be nice if I could pass in via the URL something like Q equals cats, but maybe something like name equals David or name equals Kelly and actually see the name that's being outputted. In other words, let me zoom in up here and let me pretend like this happened automatically. Let me do question mark, uh, name equals David E. Well, it would be nice if I saw

hello David. I'll I'll propose rather than just hello comma world. So how do I actually get access to everything after the question? Well, here is where a framework like Flask and any number of alternatives starts to shine. It gives me that answer for automatically. And so it turns out in Flask, once you've used it, you have access to a special global variable, as we'll call it called `request.args`, where `args` just means the arguments or the parameters that were passed in to this HTTP request. So how do we use this? Well, let me go back to VS code here. And at the very top line, in addition to importing Flask F render template, let's also import `request`, which is a global variable that comes with the flask framework, and then I'm gonna use it as follows. I'm gonna go ahead and say um. A second argument to the render template function where I'm going to say `placeholder equals request` actually let me not do that yet. Let me first create a variable name equals `request.args` and then let me go ahead and get the name key from the arguments. And then down here, let's go ahead and pass in `placeholder equals name`. So what am I doing here on line 8? I'm creating a variable called `name`. I'm storing in that the value that's in the request global variable in what's apparently a dictionary called `AGs`, specifically the name key therein. So if the thing after the question mark `name equals` is David, this should give me David. If it's Kelly, it should give me Kelly instead. Then what I'm doing is rendering this template called `index.html`, but I'm additionally passing in some named parameters. We talked. Briefly about that in week 6 when we introduced the idea that Python can take not only a comma separated list of arguments but some of which can have names. So I'm proposing that one such name of an argument to this render template function can be `placeholder`, for instance. Now at the moment this code isn't going to do anything useful if I go back indeed to the other tab, click reload after zooming in, even with my name in the URL, you'll see that we still see hello David. But here's where things now get interesting and here too is what we mean by template. If I go back into VS code, open up `index.html` again, and instead of putting the word world there, what I'd like to see is not hello world, but hello placeholder, but of course if I literally type that I'm gonna see literally placeholder unless I surround placeholder with pairs of curly braces like this. By using these pairs of curly braces, I'm telling Flask that I want to interpolate, so to speak, that variable. I want to substitute in its value. So this is yet another syntax in Python. We saw F strings. In C we saw percent S when using something like `print F`. In an HTML file when using flask specifically, we use these pair of curly braces to denote this is indeed a placeholder whose value should be plugged in. So now let's go back over to the second tab, recall if I zoom in that passed in already to this URL is question mark `name equals David`, and this time when I click reload, voila, now I see my actual name. And unlike the JavaScript examples last week which were doing everything client side, notice here if I. Go to right click or control click and view page source. What's noteworthy today is that David in this case literally came from the server. This was not rendered client side. The server sent this HTML and specifically this text. So if I go back to the same tab here, zoom in. And change David for instance to Kelly. What I should see instead when I hit enter is hello Kelly, and indeed if I go back to the source code and reload the page there, I should see in the view page source that the server sent indeed hello Kelly. So it's in this sense that it's an application. The URL is providing input to the application by way of this URL format, the so called get the get string that's being passed in. And if I look at the. that I'm running `app.ie` is the code that's running. It is grabbing that name from the URL. I am then passing it into my `index.html` file, and then my HTML file is plugging the actual value in for me. And so what's going on with, for instance, these curly braces? Well, here too is where we're

actually using a library and included in Flask is another library called Jinja and Jinja is what's called a templating library, and there's so many templating libraries in the world. Jinja is actually fairly simple, which is nice, and which is why Flask uses it. And for now you can just think of Jinja as being the library that knows how to interpolate variables inside of pairs of curly braces. So why are we introducing it another framework, another library? Well, the folks who implemented Flask decided that it was not worth their time reinventing the wheel of a templating language, a language via which you can figure out what values to plug in where. So they just lean on another library that someone else wrote years prior so as to not reinvent that wheel themselves, and that's all that's going on with the framework in this case. It's using perhaps multiple libraries instead. All right, so what then is a template? So this then is a template. What you're looking at here, `hellocom` placeholder is a template in the sense that it's kind of the blueprint for the web page I want the user to see, but it's going to be dynamically generated using indeed this blueprint by plugging in the value. Placeholder inside of those pairs of curly braces. And so that's why `index.html` starting today is in a folder called `Templates` because this is not just static HTML like the stuff we wrote last week. This is the blueprint for the actual HTML that we want the browser to spit out. But there's a bug here. Notice what's going to happen here. If I go up to this URL and I get rid of the name altogether. For instance, I just visit the slash route without any key value pairs and hit enter. This is sort of bad, bad request. It's an HTTP 400. In fact, if you look at the tab, here's another HTTP status code that we probably haven't seen before, but 400 just means the user did something wrong by not passing in the parameter that was expected. Well, that's a little bad design if like the user has to manually type in things to the URLs. Like no human actually does that. That's not good for business or customers in general. So I can go back into `app.py` and just make a little bit of conditional code here, and here's to where we see what makes this an application and not just a static page. Instead of just blindly getting the name here, I could instead do something like this. Well, if the name parameter is in `request.args`, and this is just Python syntax for asking if this key is in this dictionary, then I'm going to go ahead and define name and set it equal to `request.args.get('name')`. Else if there is no name in the request, well then I might as well give some default value like `name = 'world'`, and that alone logically makes sure that I only try to access `request.args['name']` if the key is actually there. So if I go back to the browser now, reload without any. Thing else in the URL now we're back in business and it's saying hello world, but if I go up to the URL bar and add `name=David` enter that too now works. So it's a web application in the sense that not only does it have function calls as well as a variable, but now we've got some conditional logic with Boolean expressions as well. All right, questions on anything we've done thus far, because it was a lot, all. At once, questions thus far, yeah. Good question. Let's try that. What if I just did `name=None`? Well, let me go back to that other tab, uh, delete the name David and hit enter, and I indeed see hello, nothing. Why? Because the name key is provided now. It just doesn't have a value. And so the conditional has the same answer. Well, yes, name is in `request.args`, but there's just no value associated with it. And here again is the value or a hint at the value of using a framework like Flask. The fact that I can just import the request global variable and then ask questions like `if 'name' in request.args` means I don't have to write any of the code that like figures out what the URL looks like, break it apart between the question mark and the equal signs and any ampersands therein. Like that's all sort of generic logic that every web application has to do.

So again, Flask is sort of doing that lift for me and I can just focus on focus on the logic that I actually care about. All right, well, a quick convention here. It's I've used the word placeholder here just to kind of hit the nail on the head and make clear this is a placeholder, but frankly it's a little more readable stylistically to not just put hello generic placeholder, but to say something like hello common name so that a colleague or even myself looking at this file down the line knows that OK, we're trying to print out the user's name here. That's fine. You can change the name of these variables to be anything you want, and even though it looks weird, it's conventional in Flask to do something like this name equals name. But each of these names means something different. This is the name of the placeholder that I'm going to put in my actual template. This is the value that I actually want to give it, and it just keeps me a little saner by just reusing the same name instead of calling it placeholder or placeholder 1, placeholder 2, placeholder 3, or something generic like that. Now it's just a little clear, even though it looks weird to say name equals name. Again, that just allows me to do this in my template. All right, well, what more can I do after that? Well, let me propose that we can actually go in. And simplify this code a little bit. It turns out this is so common to just ask a question as to whether the parameter is there and then do something with it or not that flask comes with some logic to do this. And in fact, I can get rid of all four of these lines, just go ahead and with confidence declare a variable called name, set it equal to request.args.get('name'), but in the so-called dictionary use a function called get that comes with it. Technically doesn't relate to the verb that was used by HTTP. This just means literally get me the following. And if you want to get the parameter called name, you literally just say quote unquote name. However, in case there is no name parameter, you can also give this function a default value like world. And so now we've collapsed into 4 lines from 4 lines into 1, that exact same. Logic. So this gets me the HTTP parameter called name, but if it's not there, it gives me a default value of world so that no matter what, this name variable has what I care about. Indeed, if I go back over here, let's type in how about name equals David again E that's there. If I type in uh no name enter that too is now working as well. All right, well, let's see if we can refine this a bit more. Let me propose that in our next version of this, let's introduce a second route. So two URLs, much like what Google has many different URLs, as does most any web application. At the moment I'm doing everything in my slash route. So how might I move away from this? Well, let me go ahead and not only add a Second route, but in actual form via which the user can type in their name. So to do this, let me propose that in index.html, instead of just printing out the user's name and trusting that they're going to have typed their name in manually to the URL, which again is not normal behavior, let's actually show the user a form via which they can do exactly that. So here's my form tag. Uh, let's say the method I'm going to use is get so that I see everything in the URL. Let's give myself an input, uh, that whose name is name because this is the human's name, and notice somewhat confusingly this name on the left is the HTTP sorry, this name on the left is the HTML attribute that we saw last week, so it's different from what we just did in Python, even though they're all called the same thing. The type of this input is going to be text. And let's go ahead and make this a little more user friendly. Let's put some placeholder text called name so the human knows what what to type in. Let's go ahead and disable autocomplete just so we don't see previous input into this text box, and let's autofocus it so that the cursor is blinking in the text box by default. Then lastly, let's go ahead and have a button, the type of which is submit, so that clicking this button actually submits the form, and I'm just gonna call this button like greet

because I want the user to be able to greet themselves by clicking this button. Now I should specify action. The only other time we used action is when we actually went to HDPS `www.google.com/search`. That's not relevant today because I'm trying to print hello world, not search for cats and such, but this is where I too have control. If I want to submit this form to a specific location on in my web application, action is where I can specify it. So why don't I pretend that they're. exists a route in my application called slash greet, and if you go to `example.com/greet`, question mark name equals David, this now will greet the user with hello David, for instance. But slash greet does not exist. If we go back to `app.ie`, literally the only route that currently exists is single slash, but I can change that. I can go into my `app.ie` as I have here. And below this function I can go ahead and define `app.route` quote unquote slash greet and just invent any route that I want. I can then define a function that will be called whenever that route is visited by convention to keep myself sane. I'm going to call the function the same thing as the route, but you don't have to do this. It's just to minimize decisions I have to make. And then in this function, what I'm going to do is this return render template. Greet. HTML, which doesn't exist yet, but there's a problem to be solved, and then I can pass in the name of the user. I'm gonna go ahead and save myself a line of code and just say `request.args.get('name', 'world')`. In other words, strictly speaking, I don't need that variable on its own line. This has the effect of what we already did in index, but I'm doing it all in one elegant one liner. And now in index, insofar as I want the index of the site to just show the user the form via which they can type in their name, this one's easy now render template quote unquote index. HTML and return that template. So to recap, here's index. HTML, which is now a form instead of a template for hello, such and such. `App.ie` is going to return that template whenever I visit the index or slash of the page, and then this greet route is going to handle the case of printing out. Greet. HTML passing in the user's name. All right, I think I'm not quite good to go yet, but let's try this out. Let me go back to my browser tab, reload, and there we have it. I have a web form now instead of the uh the hello so and so. I'm going to go ahead and type in my name and notice the URL at the moment, even though Chrome is hiding it, technically it's there slash, but Chrome and most browsers today sort of hide as much stuff as they can if it's not all that intellectually interesting, but watch what happens when I click greet to the URL. It automatically sends me to slash greet question mark name equals David, and this is just like the way the forms worked last week when we recreated our own version of Google in search. HTML because the action there was `google.com/search`. The user was whisked away to Google Server. Today I stay on the same server because the action I used was quite simply slash greet, which is assumed to be on my own server, but clearly I screwed something up because I have a big internal server error in front of me as you soon will too. Odds are as you dive into this 500 is the status code that means your fault somehow. Now why is that? Well, it's unclear from this generic black and white message. However, because I'm the developer, I can go back to VS code, open my terminal window, and recall that I have two terminals open now, one that I can type stuff in, the other of which is still running from before. Let me open up that one and you'll see if I maximize my terminal window, a whole bunch of scary error messages here, but the relevant one is probably going to be, let's see, down here, raise template not found error, jinja exceptions, template not found, greet. HTML. So there's a lot of esoteric error messages here, more so than usual, but the simple fact is that I just. Screwed up and I did not create re. HTML, so file not found by the server. So the user doesn't see all that complexity. That's deliberate by design. It's generally not

good for cybersecurity if you're revealing to the user all of the error messages that are happening on your server because maybe that suggests they can hack in some some way by taking advantage of those error messages and the information implicit in them, but they are there in your terminal window to actually see and die. No. So how do I fix this? Well, not a problem. Let me shrink my terminal window back down. Let me code a file called greet. HTML. And in greet. HTML, let's create the template via which I'm going to greet the user, which ironically is the exact same as index. HTML used to be. So let me recreate that real quick. Doc type HTML. Let me close my terminal. HTML lang equals EN head. Uh, title, hello, hello, comma, and there's my, uh, here's my placeholder hellocom name. So to be clear, the index. HTML template doesn't have any curly braces or anything dynamic. It just spits out the HTML for the form. Greet. HTML spits out HTML and the actual greeting, and it's app.ie that decides which of these to show the user, either index.html if they visit. Slash route or greet. HTML if they somehow find their way to the slash greet route, which they will automatically by simply submitting that form. All right, so let's go back into this internal server error and go back to the form. Nothing has changed with the form, but now when I type in David, click greet, not only will the URL change to be slash greet question mark name equals David, I actually now see the content that I expected a moment. Ago. All right, well, now it's an opportunity to critique. I have these two templates open index. HTML and greet. HTML and even if you've never done web programming before and even if you never did HTML before last week, what is bad about this design intuitively? Say again? Abstraction in what sense? Yes, so that's exactly the hang up I have here. There's a lot of duplication and technically I didn't copy paste, so I might as well have because notice as I very hintingly go back and forth, almost every line of code in these files is the same except for the form, which is there or not there, or the hello comma, like all of the boilerplate HTML, namely everything I just highlighted here. 1 through 7 in greet. HTML and this and this is what we really start to mean about a template. Like wouldn't it be nice if we could factor out all of that HTML that's common to both files, put it in literally a template that both routes can use so that I can write that boilerplate code once instead of again and again, because imagine in your mind's eye, well, if I have 3 routes. Or 4 routes or 5 routes. I'm going to be like typing the same darn HTML 345 times. That's got to be dumb and that's got to be solvable as we've seen in other languages as well. So let me indeed go ahead and try to improve this and the syntax is a little weird, but it's the kind of thing you get used to quite quickly. I'm going to go ahead and create a third HTML file now by going back to my terminal window. inside still my templates directory and by convention this file is going to be called layout. HTML. Why this? That's what the flask documentation tells you to do. So in layout. HTML, I can put all of my boilerplate HTML, the stuff that is invariant and doesn't change. So here we go, doc type HTML, uh, HTML tag lang equals EN, close bracket, uh, open bracket head, open bracket title, we'll call it hello for all of the pages, open bracket body, and here's where it gets interest. The body is the only thing that has been changing in these two examples. In Index. HTML, it was a web form. In Greek. HTML, it was just a simple string of hello so and so. So what I want to tell Flask is that everything in the body will just be a dynamic block of code and the syntax for that, which takes a little bit getting used to, but it's also sort of copy pasteable, block body using percent signs this time and because I don't want any. Such body in the template. I'm going to literally close this block as follows. And here you see another example of sort of HTML like syntax, but instead of using angled brackets, Gina, the templating library

that Flask uses, uses curly brace and percent sign to open the tag, and then the opposite to close it. So what you really have here are two ginja tags, as we'll call them. This one is called Block, and I'm defining an arbitrary name here. I could have called it Foo bar or baz, but because I want this block to refer to the. Body of the page by convention I'm going to call it body and then this weird syntax, which is used in some other languages too, just means end whatever block you just began. And so again you just see reasonable people disagreeing. The people who invented HTML used nice angled brackets and words like these. The people who came up with ginja used curly braces and percent signs. Why? Well, odds are these are not normal symbols that a human would type when writing code, at least in HTML, so they just chose something that probably wouldn't collide with. Actual syntax the human wants to use. So that's it for the template. This is now a, this is essentially a blueprint that doesn't have just a placeholder for a single word or value like name. I can put a whole chunk of code here now instead. And how do I do that? Well, let me go into index.html with the moment, which at the moment is a little duplicative in that it's got all of this boilerplate. So you know what? I'm going to go ahead and delete everything that is already in my layout, both above and below that web form. And now I'm going to use a bit more gina syntax. This tube takes a little while to memorize or copy paste, but if I want index. HTML to use the layout. HTML blueprint, I can simply say extends layout. HTML and then close tag using percent sign close bracket here. And then if what I want to plug into that layout is the following code, I can say as before block body. And then down here, I can say. End block and that's it. And just to be a little nitpicky, I'm gonna deinvent that slightly and now even though it looks like web pages suddenly look a lot uglier, well they do because like this is weird looking syntax, but I have now distilled index. HTML into its essence. This is the only thing that changes vis a vis the greeting page and so I've put my HTML here that I care about. I've said to Flask, this is what index. HTML. Body block shall be where to put it, well, put it into that particular layout. HTML file. And so the logic for greet. HTML is the same thing. It's going to look just as weird, but again, you get used to it. Let's go ahead and delete everything that's boilerplate in greet. HTML, both above and below. Up at the top, let's tell Flask that greet. HTML2 extends layout. HTML. And let's go ahead and say to Flask that the block uh called Body shall be this for greet. HTML. And the end of this block is now down here and just to be nitpicky, I'll deinvent that too. So again, the pages look a little weirder now, but it's going to follow a paradigm that we just see again and again such that the only juicy stuff is what's inside of that body block. So now if I go back to my layout, it looks exactly like this. This indeed is a placeholder, not just for a single variable like name. The placeholder we did before, this is the placeholder for a whole block of code that came from a file, not from a variable. And so if I go back into my other tab here, go click back to go back to the web form and reload, notice that I have the familiar looking form. But if I now look at my developer or if I look at view page source, notice everything that came. from the web page from the server. Here's that boilerplate up here. Here's that boilerplate down here, and here's the stuff that's unique to this page. And recall too, aesthetically I de indented it, which is why it's now no longer pretty printed and what the browser sees. Like that's OK. There's no reason to obsess over the indentation and the pretty printing of what the browser sees ultimately the reason I Did this indentation is because arguably when I'm in VS code here and I look at index. HTML, this is clearly indented inside of the body block just so I know what's part of that block. The browser does not care about superfluous white space or less thereof. All right, questions on what we've

just done here, which is to truly take this template out for a spin and now remove what redundancies I had accidentally introduced. Questions No, OK, amazing. All right, well, let's go ahead and look at this URL again. I'm not liking the fact that every example we've done thus far involves putting my name or Kelly's name right there in the URL bar. Well, why is that? Well, if I have like a nosy sibling and they sit down in my browser, they're gonna see like every URL I visited, including whose name was greeted. Now that's not all that big a deal, but now imagine it's a username and a password that the form. Submitting or a credit card number that the form is submitting or just search terms that you don't want the world knowing you're searching for, they're going to end up in the URL bar. Why if you are using method equals get for the form. That's how get works. It literally puts all of the HTTP parameters in the URL, which is wonderfully useful if it's sort of low stakes stuff like the Google search box or if it is. Or potentially low stakes stuff like the Google search box or if you just want to be able to hyperlink directly to a URL like this. In other words, if I put this into an anchor tag, open bracket A H ref, and a URL like this, I could deep link a user to a web page that just always says hello, David. So get strings contain all of the requisite information to render a page for the user, but this isn't really good for privacy. So recall that there's not only gets, but there's also something called post, and post is just a different HTTP verb that essentially with respect to those virtual envelopes next last week sort of puts the information more deeply inside of the envelopes such that it's not written right there in the URL bar, but it's still accessible by the server. So if I do this, watch what happens. Let me go back in. To VS code, let me go back into index. HTML, which has the form, and let me quite simply change the method from get to post. And now let me go back to my other browser tab, back to the form and reload so that the form knows that the method has changed. Now type in David and click greet and before I do that, let me zoom in on the URL bar. Notice that. The URL does change. I'm at slash greet, but I haven't revealed to the world or to anyone with physical access to my browser what URL I just searched for. All they know is that I went to slash greet, but not the key value pair or pairs that were passed in. Of course this clearly hasn't worked. I've got an HTTP status code of 405, which means method, not. That's because flask by default, when defining routes, simply assumes that you want get instead of post. Now get is good for the default page. In fact, when I go back here, this is equivalent to me visiting the Slash route just in the browser, so I want my index to generally support get. But the greet route should support post, and the simplest way to do this is to pass in another argument to the route function which we haven't needed before because the default is get, and I can instead tell Flask a comma separated list of the HTTP methods that I want this route to support. So if I wanted to support just post, I can pass. In a list containing just post and recall Python uses square brackets for lists, which are their version of arrays in C. Now by default this argument is this methods equals get and that's why the only thing supported a moment ago was get. That's why I'm now changing it to be post instead. I have to make one other change though. It turns out if you read the documentation. When accessing HTTP parameters via post instead of get, you move from using request.args to request.form. This is completely unintuitive that request.args is get and request.form is post. because they all come from forms, so it's bad naming admittedly. So you just kind of have to remember request.args is used for get requests. form is used for post. So all I need to do further is change this to be request.form. And that's it. Now my web application will support web forms submitting to it via post instead of get. Let me go ahead and type in my name now. I'll zoom in. Not that the URL

will again change to slash greet with no parameters evident, but I will be greeted this time because the server knew to look deeper into that envelope. For those key value pairs instead and just to be now uh sort of diagnostic about this, let me go back once more, let me right click or control click on my desktop and go to inspect. Here's where developer tools can be super useful as well. I'm gonna go in here. And I'm going to go ahead and clear this, and now I'm going to type in David again and I'm going to click agree. But because I have the network tab open like we played with last week, it's going to show me all of the requests going from my browser to server, which is going to be useful here because not only do I see, OK, it obviously worked because I got back at 200, but if I click on this diagnostic output, I can actually go to The payload tab here and I'll see that the form data that was submitted was name, the value of which was David. So you can see what you're submitting. So you can do this today. Like if you want to log into some website, Gmail, or otherwise, you can actually see all of the data that your own keyboard is submitting to the server, even if it's using posts because the browser that you control, of course, can see the same there. All right, any questions now on this transition from get to post? Well, kind of on a roll or not going so well. We'll see. All right, so what more can we do with this? Well, let's give ourselves a couple more building blocks before we transition to actually implementing some real world problems as I did years ago with one such example. Suppose that I don't like this direction I'm going in insofar as every time I have a page with a form, it submits to another route altogether because in your mind's eye I just kind of extrapolate. Well, if I have two forms on my page. I now need 4 routes. If I have 3 forms, I need 6 routes. It seems a little annoying that you use one route just to show the form and another route to process the form. This is going to get annoying over time because it's like twice as many routes as might be ideal. So is there a way to get kind of the best of both worlds and combine these two routes into one so that everything related to greeting the user all happens in one place? Well, you can as follows. What I'm going to go ahead and do. I delete my greet route altogether and most of my index route, but I'm going to ask a question. I'm going to first say that the methods that the index route support now shall be both get and post as a comma separated list there, and then inside of my index route I can simply ask a question of the form if the request that is submitted to the server has a method. Of post then assume that form was submitted. This is just a Python comment note to self that I'm gonna go back to in a moment. Else if the request method is not post, so I could technically say if if uh if request.dome equals equals get, then, but this is kind of dumb because I only support two verbs, so I might as well just assume for efficiency. Else handles the get implicitly then go ahead and assume that no form was submitted so show form. So just notes to self as to what I want to do. So how do I show the form? Well this line was easy return render template of index. HTML. If though the form was submitted, what do I want to do? Well, just as before, let. Return render template greet. HTML passing in a name value of request. form.ge quote unquote name else a default value of world. So the exact same logic from each of the two. Functions a moment ago, but I've now combined them into one by just using some conditional logic and just asking the server if the user got here via post. Well, the only way they could have gotten here via post is by having clicked that button and submitted the form. So let's just go ahead and greet them, else if they got here via get by just typing in example.com or whatever the actual URL is, let's. Go ahead and show them the template. So it's still good design in that I have a separate template for each of these pieces of functionality that is only minimally different, but I'm sort of deciding which of those to show

based on the actual logic in this here app. All right, so this is almost perfect except for one bug. What else needs to change if I've just combined my greet route and this default slash route as well? Yeah Yeah, so in the form that has index. HTML, recall that there's an action line that specifies like to what URL do you want to submit this. Well, let me go back to index.html. It can't be slash greet anymore because that doesn't exist, so I'm just going to delete the word greet and submit it to slash instead. Which will have the effect of also just omitting it entirely if you don't specify an action, it submits to the very location that it came from, but if you want to be pedantic and even more clear, just specifying that the action now of this form is just this, then that will work here too. All right, so let's test it. Let's go back to the other tab, back to the form, reload. It's blank now. Type in David, click greet, and this too is working. But again, if I go back and reload, get is working as well. But there's nothing ending up in the URL because I'm now using posts, which again tends to be a good thing for privacy reasons as well. Let me show one final flourish before we transition. Something real world motivated. If I go into app.ie for a while now, I've been passing in this default value of world, which is fine, especially if it's something short and sweet that's the default value, but I can actually put a bit of conditional logic in my template as well. So in fact, let me go into greet. HTML and trust that I will now be passed in a name variant. but I can decide for myself in the template whether I want to say hello name or if it's blank hello world instead. And how might I do this? Well, I can always say hello, but then I'm going to use some ginja syntax that we haven't seen yet. But it turns out in Jinja, the temping language that Flask uses, you can use Python-like syntax too, and you can ask questions like, well, if uh the name variable has a value, well then go ahead and output the value of that name. Else if the name variable does not have a value, go ahead and output a literal value like world and then down here and if. So ginja again is a little weird and then it says end block and if, but that's the way it is. But even though this looks a little weird, it's just a nice clever way of. A bit of logic into my template and if the name has a value, so it's not empty or none, go ahead and display it. Hence the curly braces else go ahead and literally say world. Why is it not problematic and you can see the dots here that there's all of this white space after the word hello, comma. Like otherwise this would seem to create quite a messy paragraph or phrase of text in terms of whitespace, but. HTML ignores superfluous white space, so anything more than a single space just gets canonicalized or collapsed into a single space. And we saw that recall last week accidentally when I had those three paragraphs of text from from the duck, but I wanted them deliberately to be separate paragraphs and they weren't because all of that white space was ignored until I actually. Introduced the paragraph tag instead. So this just moves some of that logic now to the templates. So for all this logic and more, here's the official documentation for Flask and specifically Jinja's own documentation, but for the most part we've seen what's possible already, and I promised a real world example. So here now it is. So. Um, back when I took CS 50 as a sophomore, there was no web programming in the class, and frankly there was barely any web actually in the world because it was all so new, HTML and the like. But it was my sophomore spring maybe or junior fall that I also got involved in the freshman intramural sports program or Frosh IMS for short, and back in the day we would walk from say Matthew's Hall to Wigglesworth freshman year at least, to register for sports by filling out what was called the sheet. Paper and then you would go to the Proctor's dorm room and slide it like under their door or through the mail slot and that's how we registered for sports. It was sort of ripe for disruption before that was even a phrase. And so one of the very first projects I took on myself personally

after taking CS 50 was to figure out how web programming worked and Python wasn't really wasn't a thing yet, nor was half of the topics we've been talking about thus far. But at the time I learned a programming language called Perl. I learned a little something about CSV files, which we did a couple of weeks back too. And I built this the Freshman intramural sports website via which you could click on a bunch of links and get some information, but most importantly, you could register for sports as by typing in your name, selecting the sport for which you want to register, click submit, and no longer walk across Harvard Yard with a piece of paper to actually register for sports. So we thought we'd use this as sort of the beginning of a motivation for how we can now solve problems using web-based interfaces using code and also what not to do, like background images that repeat like this are not really. In fashion anymore, nor arguably in 1997, but let's leave that as a cliffhanger and come back in 10 minutes after a snack with reimplementing the Frosh IM's website. All right, we are back. So among the goals now are to recreate the beginnings of a site like this for Frosh IMs whereby we want to enable students to visit a form, fill out that form, and submit it to a server, and then register, and we'll dispense with all of the amazing graphics and such, and keep it fairly simplistic and. Core HTML. So let's go ahead and do this. Back here in BS code, I've gotten ready now for this next set of examples and in particular I've created in advance a directory called Frosh IMs inside of which are App.ie, requirements.txt, and templates, which are essentially the same as the ones we just created, but I stripped out the hello and greeting specific stuff. I'm going to go ahead in this terminal and do flask runs, so I get the server up and running again on port 5000, and then I'm going to go ahead and open up another terminal here as I did before. CD and Frosh IMs in that terminal where I'll see the exact same files and I'll give you a quick tour of what I created in advance. So here in A.ie is quite simply the simplest of applications that just renders the index. HTML template with an expectation in a moment that we're going to make it more interesting than that. Meanwhile, if I open my, my terminal again and open up Requirements.txt, it just mentions flask, but it's already installed, so no more to say about that for now. Now let me go ahead lastly and open up template. It's the templates folder, two files they're in, the first of which is layout. HTML, which looks almost the same except I did add a slightly more user friendly tag to the head of the page, which you might not have seen before, but this is a tag that essentially you can copy and paste into templates of your own that help the content of a page resize to be mobile friendly. In fact, without this line, if you were to develop Problems at 9 or your final project for the web and then try to access the site on a phone, everything might look quite a bit. Small font sizes and more. This line tends to help the browsers resize dynamically so that it actually matches the width of the device's own with, for instance, a phone versus a laptop or desktop. But otherwise everything else is the same there, including the placeholder for the body block that I've find here on line 9. Lastly, there's one more file that at the moment doesn't do anything all that interesting except is ready to contain the contents of the registration form for Frosh. I am. So let's go ahead and start with actually that. Let me quickly whip up a form that minimally gives the user something that they can submit to the server to register for sports, and then we'll improve upon it a bit iteratively. So here inside of the body of index. HTML, which is going to extend the actual layout, the blueprint we already created, I'm gonna have a quick title for the page like register just to make clear to the student what they need to do using the H1, which is the big and bold tag. Then I'm gonna go ahead and have a form tag. Uh, whose action is going to be anything I want, but since I want the user to register, I'm gonna have it go to slash register, which makes

more sense romantically than greet now because we're doing something else. The method I'm gonna have the student use is post, if only because they don't want their roommates knowing what they visited in their browser, so this way it will tuck the HTTP parameters deeper in that virtual envelope so it's not stored in the browser's history. Inside of this form. I'm going to have minimally an input box for the student's name, so I'll call that aptly name and set name equal to name in my HTML. The type of this text box will be exactly that text. And then just to make it a little more user friendly, I'm going to add a placeholder of names so they know what to do. I'm going to go ahead and turn off autocomplete in case multiple roommates want to sign in from the same computer, register from the same computer, and then we'll turn on Autofocus to put the cursor in that name box. And then, and you didn't see this last week. But if you've ever wondered how drop down menus are implemented in HTML, if you've never done this yourself, those drop down menus on web pages are called select menus. And if I want the user to select a sport to register for, I'm going to call this input a sports, and this is an alternative to just having a generic text box where we have the students type in the sport they want to register for, which would be fraught with typographical errors and changes in capitalization. A drop down menu, of course, standardizes what the human can select. So inside. This drop down, I'm going to have a few options. The first of which will be, uh, basketball, for instance, the second of which will be soccer, and the third of which, which I think was the 1st 3 with which we debuted back in the day, was ultimate frisbee. Now these option tags can take some attributes by default. They will take on the value of whatever words are typed in between the open and closed tags. But just to be pedantic, I'm going to make clear that the value of selecting this. shall be basketball, but I could change it to be something else if I so chose. The value of this selection will be soccer, and the value of this last option will be ultimate frisbee just in case I want to store something else in my database ultimately. Now that is a complete index. HTML, I think. So if I go back to my browser tab, which previously was showing me the Hello program because I stopped and restarted Flask, and you can stop Flask by just hitting control C. Uh, for interrupting it, I'm going to reload the page and I should now see, OK, a slightly more interesting form with the name box with the cursor is blinking there, and then a select menu, a drop down with three options. Now it's a little presumptuous of me to select basketball by default, and in fact this is kind of inviting user error if they type in their name, don't really think about it, and now register for basketball accidentally. So I'm going to make a couple of improvements here. I'm actually going to have essentially a blank option at the top whose value is nothing. And I'm going to have it just labeled sport. And just to be super clear, I'm going to select this value by default. So the option tag in HTML supports not only a value attribute but it turns out a selected attribute which if present means that's the option that will be selected by default. So if we go back now to this page and reload to get a new copy of the HTML. looks a little better. I still have the name at left, but the sport now menu looks like this, so it's a little more clear what I want them to do from this drop down, and sport deliberately on the back end won't have a value, and theoretically this will help me determine if they actually selected a sport or just clicked register and ignored the drop down still. But I do need. Way for them to register ideally by clicking a button, so I'm going to add a button, the type of which is submit, and then I'm going to have this button's label be register. So now if I go back to the form once more, reload, I now have, I think, a complete form, albeit not very pretty, via which David can register, for instance, for basketball by clicking register and darn it, I have a 404 not found. But why is that? Why is nothing yet found. Why is slash register not

found, yeah. What's that? Yeah, I haven't, well, I haven't linked the option to anything. I think the form has been linked, whoops, the form is telling the browser to go to slash register, so this is correct behavior. But if we go to app.ie, like there's no route defined for slash register. So of course it's not found because there's an infinite number of routes that don't exist and register is currently among those. So I can define that myself. I can say app.route.register. I do want to use post, so I need to proactively say that the methods this function will support will be indeed post instead of the default of get. I'm going to define an actual function to call when this route is used, and by convention I'm going to call it just register, even though I could call it anything I want. And inside of my register function, well, for now I'm going to cheat a little bit. I'm going to at least just say. Uh, I'm going to at least check that the user has given me a name and a sport. So how can I express this? Well, because I've already imported the request global variable that comes with flask, I can ask questions of it and I can say something like if it is not the case that request.form.get('name') has a value, or if it's the case that or if it's not the case that request.form.get('sport') has a value, then let's go ahead and give the user uh a warning of sorts or return render_template of a file called failure.html. This doesn't exist yet, but no big deal. Let me go back into my terminal. Let me uh go into templates and create a file called failure.html, and in this file I'm gonna say that it extends layout.html and then it has a block body inside of which is gonna be something like super trivial for now just to get us going and this failure page is simply going to say you are not registered and then and blocked. So that's it just sort of an error page that now exists. I'm gonna close it out of sight, out of mind, but I think this now will work. If it is not the case that the user gave us a name or it's not the case that the user gave us a sport, we will show this error message otherwise. If all seems to be well for now we're not going to do anything useful with the information, but I'm going to go ahead and return render_template of success.html, which is simply going to assume that the user was successfully registered. So let's whip that up quickly. I'm going to go ahead and code up success.html. Inside of this file, which will similarly extend layout.html inside of which there's a body block that quite simply says how about you are registered and we'll just pretend that it is so and block. So that's it. In short, I wanted two templates that show failure or success respectively. So I think now in app.ie we're in better shape. I now have a register route that will get called if post is used to visit it, and I'm going to check request.form, which is where you get the post variables from, check whether name or sport is provided, and I'm going to render a template accordingly. So let's try this. Let me go back to my other tab and go back to the form. Let me type in my name, David, but no sport. Click register. And I have an internal server error which was not intended, so let's figure out how to diagnose this. So it seems to be the case that I'm at slash register that was intended, but something clearly went wrong, so let's go back. Now I could just kind of stare at my code endlessly, but recall that there should be some hints in my terminal window that's running flat. So let me go back to my other terminal, and there it is, unexpected char double quote at line 11. Well, so sounds like user error, so that is in failure.html and you can kind of see it because Flask is like underlining it literally for me. What did I do that was stupid? Yeah, I just didn't close my quote. So amateur hour here, so let me go into, I do need to open it after all, ironically, so let's go ahead in my other terminal, open up failure.html and there it is, one stupid character away from correctness. All right, let's close this again. Go back to the other tab. Let's try this again. David is my name, but no sport register. OK, you are not registered. I don't know why, but I know I'm not registered. Let's try it again with a uh with no name, but yes, a

sport, click register, you are not registered. All right, just for good measure, let's give no name and no sport. You are not registered. So that seems to be working. Let's now cooperate. Let's go ahead and register as David for basketball. Cross my fingers, damn it. And internal server error. Let's try to learn from my past mistakes. Let's open up this eyeball it. I did it twice, even though that was not copy paste, so 0 for 2. All right, let's go back here. Notice now I can actually just click reload because the browser's smart enough to remember what I just posted to the server. So if I click reload, you'll be prompted to confirm the form submission lest you be doing this on a website with your credit card or something where you don't want to send it twice. But in this case I'm fine with sending my name in basketball twice, so I'm going to click continue. And this time it worked, telling me that I'm actually registered so I'm not doing anything with the students' data, but at least I am validating that they gave me some inputs. Now there's a catch here. The catch, of course, with HTML is that it's all executed client side. And so for instance, suppose that a student is really upset that we only offer basketball, soccer, and ultimate frisbee, and maybe they really want to register for volleyball even though we're not offering volleyball. Well, there's arguably like a security vulnerability here where technically my code right now will tolerate any user input even if it's not in that drop down because after all, let me go ahead and right click. Or control click on my web page and open up the developer tools. Let me go into the form as sort of a hacker type student. Let me go into the select menu and OK, no big deal. If I want ultimate Frisbee to exist, well, I just need to know a little HTML. I'm going to right click on that element and click edit as HTML. This literally lets me start editing the HTML. of the page I'm going to give myself my own option. Option value equals volleyball close brackets volleyball enter and now when I close developer tools, woo hoo, I can register for volleyball if I want. So let's select volleyball, type in maybe Kelly is hacking the site register, and she is registered for volleyball. Apparently, all right, so the short answer is the short, the takeaway here is do not trust user input ever for reasons we've already seen when we discuss SQL ever more so now that we're dealing with the web, because who knows what users are going to do accidentally, foolishly, or even in Kelly's case here maliciously trying to pass data that we did not expect. So what would be the defense against this? Like this is just how HTML works. And assume that I'm actually registering Kelly for sports now and somehow she's now signed up for volleyball on our database. What would a solution be logically here? Yeah. Yeah, so maybe do some server side validation. So don't just blindly check that we have a value from the user. Actually check that it's one of those sports. So if I go back to app.ie, I could do this in a few ways, and maybe my first instinct would be this. Let's check for the name and do this, but let's also do this. Like if request. Dot form do get quote unquote uh sport and actually let's put this in a variable just to make it even easier to type so sport equals this if sport, uh, how about does not equal, uh, what was it basket ball and. Sport does not equal uh soccer and sport does not equal quote unquote ultimate frisbee, then render an error. So, uh, return, render template quote unquote failure. HTML. So now if I go back to this form and try to register as Kelly again. You are not registered, so I somehow caught her because volleyball, of course, is not in the list of sports that I put there. But what might you not like about this approach? Even if you've never done web stuff before, what's bad about this? Yeah, I have to hard code every single sport now in not only A.ie to check for the validity on the server of what the human is typed in, but recall that the drop down itself came from index. HTML. So I now in duplicate have to put like all of the sports there too. So like this just seems bad to have duplication and so better might be to do something

more like this at the top of my file. Here, why don't I go ahead and just give myself a global variable which in the context of this web app is perfectly reasonable so I can access it anywhere. Let's call it `SPORTS` in all caps just to connote that this is a global variable and constant, even though Python does not have `consts` in the sense that C does, but this is sort of on the honor system. If you see a variable in all caps like this, just don't mess with it, use it, but don't mess with it. So, uh, inside of the square brackets this is gonna be a list of the sports that I do want to support, so basketball, comma, uh, soccer, comma, ultimate frisbee, and that's it. Now instead of doing all of this, what I can instead ask is a. Simpler question like this if sport not in sports, then go ahead and return render template quote unquote failure. HTML and I can actually tighten this up a little bit. I don't need two calls to failure. HTML. Why don't I just borrow this code and say. Or, uh, sport, not in sports render a failure, and now I've tightened this up quite a bit more, but I'm essentially using Python to just ask, is the sport that oops sorry I deleted too much sport equals actually let's just tighten it up further. Sport does not exist, so let's do `request.form.get('sport')`. So if the sport that the human typed in or selected from the drop down somehow is not in this global list of possible sports, well then it's a failure. Don't let Kelly or whoever register instead, but if I now have this global variable, I can be a bit smarter in my template. I don't need to manually write out all three of these sports here. Instead, I think I can be smart about this, and when I render index. HTML itself, why don't I just pass in a variable called `SPORTS`, for instance, set it equal to the value of that global array, and then in my template, and here's where templating again gets interesting and starts to save you time, let me go into index. HTML. Delete all but the default value, the blank one, and do something like this. Jinja, it turns out, also supports loops like Python for sports in sports using the curly braces and the percent signs, I can now dynamically generate options as many as I want. So option value equals quote unquote the current sport close uh there, close brackets sport. So it's a little redundant, but again this is just how HTML is. This is what the human sees. This is the value that gets submitted to the server in case you want one to differ from the other, and then below that option line I can say `end=4`, which is a bit weird, but that's how it works in Jinja to stop that loop. So this is kind of powerful. Now if I have 3 sports, 30 sports, all of the options will be dynamically generated by this template. And so now we're starting to save ourselves time and I can centrally manage all the sports by just updating this global list here in `App.py`. So let's go back to the browser. Uh, back to the form reload, and you'll see that the drop down thankfully still works the same way, but all of those options were dynamically generated. Indeed, if I view page source from my browser, you'll see and there's some extra white space there because the loop was adding some white space on each iteration. I still have the 3 sports but not volleyball, as was my intention. So now if uh if Kelly even tries hacking this version of the site by going in here and select and typing in volleyball manually registering, the logic will still catch it because only those three sports are in that array. So it's perfectly fine for me now to register for basketball because it's among the sports, sorry, in that list, not array. Questions on any of these here. Techniques. All right, how about another type of form? So select menus are nice, but you also might see radio buttons on websites which are the mutually exclusive little circles that you can select to choose one or another option. Uh, let me go back to index. HTML and just show you how those can be created as well. Instead of using a select menu, turns out we can create a whole bunch of inputs of radio type type as follows, of radio button type as follows for each sport, so for sport in sports, let's go ahead and output. In between this tag and the end for the

following input type equals radio. Uh, and let's give it a name. The name of this radio box is going to be sports, and the value of the current input is going to be quote sports, and the word that the human's going to see is as before sport. So notice it's just another type of input. Previously we've seen text, for instance, two lines above. We also saw last time search. We saw email. There's a bunch of text input types. This one though is going to display as a. Radio button instead and the human is going to see this label here. If I now go back to my other browser tab and click back, click reload on the form, I should see it's not pretty, but it's a radio button in the sense that these are mutually exclusive. How does the browser know that I should only be allowed to select one of them? Well, because I use the same name for each of those radio buttons. It knows that means mutual exclusivity. In fact, if I view page source in the browser, you'll see that all three of the inputs that were dynamically. type equals radio. Type equals radio, type equals radio also have identical names. And so that's just how that works. And that's the only change necessary. If I now go ahead and type in my name, David basketball, click register, we're still up and running because what the server gets is still exactly the same inside of request. form they can access, you can still access name or sport no matter what type it was in the user's own browser. Questions on these techniques. All right, well, it's kind of obnoxious that when you don't do something right in this website like forget your name but you select a sport, all you are told is generically you are not registered. Like it'd be nice and much more user friendly, better UX user experience, so to speak, to actually tell the user what's wrong so they can actually fix the problem. Now there's a bunch of. We can do this, but I'm going to propose that we go ahead and do this. Let's create a template called error. HTML, whose purpose in life is just to tell the user a little something more about what they did wrong. So I'm going to go back into my terminal window here. I'm going to code up a file called error. HTML. Enter and I'm going to go ahead and before, as before, extend layout. HTML, learning from my past mistakes and closing that quote. Then I'm going to go ahead and do body block down here and then inside of this block body I'm going to go ahead and have just some simple text like an H1 tag that just says error to the user. Then a paragraph tag that's going to contain some error message to be determined and then uh that's it for now. So I've got the template for an error message screen. Let me go back into app.py now and let me add some logic because A.py does know what's wrong. It's just at the moment we're very generically returning a failure template instead of something more precise. But if I know that the user hasn't given me their name, well, let me say that error message. So let's actually get rid of these two lines and be a little more specific like this. So if, or how about let's do it like this, how about validate the user's name first? So name equals request. form.get quote unquote name. That just gives me a variable containing the user's name. If they didn't give me a name, which I can express with just if not name like if name is blank or none, then let me go ahead and return render template of that error template, but let's pass in a specific message like missing name. And so by passing in another argument to this template called messages, I can trust that flask will dynamically output that message where I tell it to using the old curly braces. Meanwhile, let's go ahead and validate not just the name but validate. Uh, sport. I can do this in a couple of ways. Let's do this. So sport equals request. form.get quote unquote sport. Then in here, let's say if there's no sport, go ahead and return render template quote unquote error. HTML message equals missing sport, so quite like name but we can be more specific now too. If the sport they did give me is not in the global sports list, well then it's Kelly trying to register for. Volleyball again, so let's return render

template of error. HTML, but this time the message shall be invalid sport or something like that. So we're being ever more clear, otherwise they are presumably confirmed because we got this far logically. So if I go back to the other browser tab, go back to the form, and let's go ahead and type in no name and just click register. OK, what did I do wrong accidentally? So let's go back to VS code. Open my terminal, open the first terminal window where flaskun is running, encountered unknown tag body. So I did something stupid in error. HTML. So let's go into error. HTML and uh block. Oh, it's subtle. I just transposed the words. It's supposed to be block body. That was dumb. All right, block body. I think that's correct. So let's go back to the browser. Let's reload. It's prompting me to reconfirm that I want to submit the exact same form, which recall had no name and no sport, but now I see an error in a good way. This is not a server error. This is my error missing name. Now it's not super user friendly, but it's at least more explained. then you are not registered. All right, let's go back. Let's give it a name, but no sport register ah missing sport. Let's go back. Uh, let's go ahead and give it a sport but uh a sport but no name missing name as before. And if I took the time to actually hack the HTML and do what Kelly did before and add volleyball, it would similarly say invalid sports in this case too because it's not in that same list. All right. Questions on this technique. All right, well, it's all fun and good to have a registration site that does this, but it's literally just throwing out the information. And what I did like years ago was actually even cut a corner initially where I think I wrote code that just sent an automatic email to the proctor running from IMs containing the person's name and the sport for which they registered, but that was very quickly replaced by a better feature which is actually store the data in the server itself and keep track of it rather than just send it off via email. So let's do a first pass at actually storing information on everyone who is registered for sports. Well, let me go up here and let me create another global variable to make my life easier here called registrants and set this equal to curly brace close curly brace. What do these two characters represent if empty, especially? What data type is this? It's a dictionary, so it's a Python dict, so you could similarly say dict explicitly open closed, but it's more pythonic generally to just use two curly braces. This is just giving me an empty dictionary. Why? Well, I want to store the two things I'm still collecting about all of the students their name and the sport for which they registered. So. value name sport. So how can I go about doing this? Well, it's pretty trivial down here in my register function, recall that I'm just kind of naively saying you're registered even though I'm not doing anything with their name or sport, but that's easy. Let's remember the student for real now. So in that registrants uh uh dictionary, let's go ahead and index into it using the student's name David or Kelly or whoever and set that equal to the sport for which they registered. And now notice the name is coming as before from request.form.get. For is similarly coming from that function and so this is just remembering that key value pair so that's all fine and good it's in the computer's memory how do we actually see it? Well, wouldn't it be nice after you register if you could see the actual registrants of the website, um. Uh, certainly if you're the proctor trying to run the sports, well, yes, so let's go down here and let's create another route like slash registrants, which is just gonna give me a list of everyone who's registered. Let's define a function called registrants, so I could call it anything I want, and this one's gonna be relatively simple. Render a template called registrants, which will soon exist and pass in all of the registrants that are in that global dictionary and again I can call this placeholder anything I want, but insofar as it contains the registrants, I'm setting registrants equal to. The registrants Global dictionary. So let's go now into my terminal window and create registrants. HTML and

create really the beginnings of an actual Frosh IM's website that's gonna show the proctor who has now registered. So let me go. Into this terminal and do code of registrants. HTML and close the terminal. Let's try to get this right finally extends layout. HTML close quote, uh, close bracket there. Then let's do block body in the right order, then end block. Down here and then inside of the block here this is gonna be a bit more of a mouthful, but let's use some of our HTML from last week. We'll give an H1 tag that says registrants so the proctor knows what they're looking at. Then let's put this in a table, for instance, with two columns, names and sports. So tabletag followed by a T head tag for the table heading, uh, then that heading is gonna contain just a single row for TR and each of those has a TH table heading. Uh, one of which, and actually I'll make it tighter, is name, the other of which is going to be sport. So these are the column headings, the table headings, TH tags for short. After the head of the table, let's go ahead and do a T body for table body and inside of here this is where jinja comes in use. I can say for each name in the registrant's placeholder that was plugged in. And for proactively, what do I want to do on each iteration? Well, I think I want to output table row, table row table row, and in here I can do TR and then inside of that a table data for the cell on the left, putting in the student's name which is coming from this for loop just like in Python, and then one more table data, namely the registrants, uh, placeholder indexed into at that name which because it's a dictionary will give me the sport for that student's name. And then I think we're good to go. And in fact, just to hark back to something I said last week when we were imagining, actually this isn't week 5 when we were talking about stacks and like your Gmail or Outlook inbox is essentially a stack with the newest emails on top and I hypothesized at the time that it's just row after row after row after row when we started talking last week about HTML. Here is what Google and Microsoft and others are probably doing anytime. You have tabular information in a page. They've got some data in memory like the registrants, and they're just using code like this in Jinja to output table row, table row, table row. Imagine this is your email instead, same exact idea. And now we have the ability to express that kind of logic. So let's go back now into the browser, click reload on the Let's register for instance David for basketball. Click register. It claims I'm registered, but hopefully now I'm legitimately registered because that variable is storing it in memory. And in fact, let's go ahead and go now to not slash register, but I'll zoom in at the top registrants and hit enter, and we will see a very. Ugly but functional HTML table containing two columns name and sport, the so-called T head in which David and basketball are present. Moreover, if we now go back to that form and let's try registering Kelly, for instance, for soccer, click register. Now let's manually go to registrants again. Now Kelly and David are in the server's memory as well. Questions then on what this example is now doing or how it's achieving these results, yeah. Really good question. If you wanted to restrict the registrants page to only certain people, ideally you would have a password on it. And in fact one of the next examples we'll do in a few minutes is a login page for exactly that reason. Right now just sort of on the honor system that only the proctor in question goes to this URL. But just for the sake of discussion actually, suppose that you did want the registration list to be public, if only to like hype up who has already registered. Well, it's not good to just tell people go to the slash reg. URL, we can actually link them to that in a few different ways. So for instance, I can go down to uh how about uh let's say success. HTML. So let me open up success. HTML. It just says you are registered. I can do something like this um A HF equals slash registrants. So I have control now over my HTML and the routes, so slash registrants will exist. uh see who. ET registered,

period. So this will create a nice little HTML link that links me to that route. So let's try this. So let's go back to the form over here. Uh, let's go ahead and register John for Ultimate Frisbee. And register. All right. And now we see you are registered. See who else registered. And if I hover over this, it's super small, but it would have showed me in the bottom left corner at the link. And indeed here now is John at the bottom of this table. And just to be clear, if I view page source on the browser, you see all of the TRs that we dynamically generated on the server side before they were sent as such to the browser. All right, what if we wanted to do something slightly more elegant here? Well, I don't have to just use this HTML hack. Like, why don't I just show the user who has registered automatically, and this is kind of a cool feature of web apps as well. In addition to importing Flask, render template, and request, I'm going to also import a function called Redirect that comes with Flask and indeed. Rather than just show success. HTML, I'm going to go ahead and return the result of redirecting the user to slash registrants. So to be clear, I'm in my register route and instead of showing them the success page anymore, which I might as well delete at this point, I'm just going to redirect them to this list of everyone who is registered, including themselves. So if I go back over. Here and type in someone like Doug who maybe will play basketball with me and click register. Watch what happens to the URL at the very top of the screen. I'm automatically whisked away to registrants in this case. I made a change to the code though, and so the server actually was smart enough to reload. So Doug is now the only one in the database, and this actually hints at. A problem we should really solve. Like, in fact, let's do this real fast. Let me go ahead and register myself again for basketball register now it's Doug and David. The catch though is if this server ever goes offline, maybe because it needs to be updated or it crashes or. It reboots when you hit control C and get back to your terminal. Flask server is no longer running, which means that global variable called registrants in all caps is gone. It's like free the memory has been freed. So if I were to rerun flask now, as would happen automatically if the server itself rebooted. Well, this is not great because if I go back to the registrant's page and click reload, no one has registered and in fact that's what happened with Doug a moment ago because I changed my actual app. ie. Flask was smart enough to realize, oh wait, the code has changed. I better reload the program which gave me a brand new version of that global. Dictionary. So what would be better clearly than storing registrants in memory in RAM in a variable in the server. Yeah. Yeah, so in an actual database and so here's to where everything kind of comes full circle and connects again. So let me go back into A. ie here. And I like generally the logic of what I've done. I don't like the fact that I'm just storing my registrants inside of this global variable, which is again just in the computer's volatile memory. Let's actually put this in a database instead. So let me go up here and get rid of this global dictionary and let me do something a little smarter up here. Let me import from CS 50's own library the SQL. Function that we've used before and again, even though we've been taking off all almost all of CS 50's training wheels, the reality is using CS50's SQL library even through final projects just makes using SQL and Python so much easier. But there's certainly third party libraries you can use. Um, let me go down now and in addition to creating my app, let's create a database, DB for short, setting that equal to SQL light and then SQL Light. And slash, which is not a typo, and let's assume that the database shall be called froshimMs.db. More on that in a moment. And then down here, now that I have a database variable, let's not remember the student by storing them in this dictionary. Let's actually execute a line of SQL. So dB.execute, insert into. Well, wait a minute, what am I going to insert them

into? Not to worry, I came prepared for this, so let me go ahead and maximize my terminal window and then run SQLite 3 of a file called FroshIMs.db. And this is a file I made in advance, but it's super simple. In fact, if I type schema just to see the design of this database, you'll see that in advance I created a table in this database called registrants. It has a column called ID. A column called name and a column called sport and the primary key of this table is to use the ID value, which is just an integer. And now notice I have some constraints here. I want the user to give me a name and a sport, so I've specified that it's not just text, it's not null. That is null value should not be possible to put in here. All right, so let me go ahead and exit out of SQLite 3. Let me go back into. Uh, my code editor here and now I know what to insert into. Insert into the table called registrants. What? Well, I want to insert how about a name of the student and the sport for which they registered, and the values therefore that I want to insert are going to be whatever they came from the post request. Here's where you do not want to make yourself vulnerable to SQL injection attacks. No F strings in here, no just plugging. Students input in blindly. This is where and why we use these placeholders in both CS 50's library and in many libraries in the real world to specify that I want the library to properly sanitize the user's input and get rid of any scary characters like apostrophes or semicolons or the like. So I'm going to pass in name and sport, and this one line has the effect of, as you recommend. It's storing the registration in an actual database on the server, not just in volatile temporary memory, but we do have to change one thing. This line here is no longer valid because there's no global variable there via which we can get all of the registrants, but that's no big deal. Here's how most web apps would do this. I'm going to define a variable called registrants and set it equal to DB.execute. Of select star from registrants. It's as easy as that to just get all of the registrants from my database and down here there's no longer an all capitalized variable, but there is a lowercase one registrants. So to be clear, in my register route, I am inserting the user into the database and in my registrants route I am selecting the users from the database and then the rest of the code I think can stay the same. So let's go back to Fresh AMs here. Go back to the form. Let's register David for basketball, register. Ah, I did screw up. You're seeing some weirdness here. What are you actually seeing? There's one user registered. Not intentional, but what does this syntax suggest we're looking at? This is a dictionary. Recall that the dB.execute method that comes with CS 50 SQL library gives you a list of dictionary objects. And so because there's only one registrant at the moment, you're seeing my dictionary for my registration, which is not what I want to show here and I forgot. I need to also go back into the registrants templates to tweak my syntax as follows. Let me go back into VS code here. Let me go into registrants. HTML and because I am passing in now not a dictionary but a list of dictionaries, I just need to think about the problem a little bit differently. So my syntax here is gonna be for each, uh, let's do this as follows. For each registrant. In that registrant's list of dictionaries, go ahead and display the current registrant's name and go ahead and display the current registrant's sports. In other words, I'm using Python syntax, which works as well in Gina here. This iterates over the list of registrants, each of which is a dictionary, so I'm using dictionary syndex now to index into the name key of the registrant dict object and the sport key of the same. So now let me go back to my browser and I'm just gonna go ahead and reload the registrants page without resubmitting the form. Now there it is, David and basketball. And now let's go back to the form and register a couple more people. Kelly for soccer register. Notice we're at the registrants link. Kelly is indeed still registered. Let me go back to this and let's

register John Ultimate Frisbee, register. Let's go ahead and kill the flask server by going to my first terminal window, uh, control C. And now let me go ahead and rerun Flask, which was bad before. That's how Doug ended up the only registrant last time. But this time if I go back to the registrants page and immediately click reload, even though the servers running anew in memory, the database is persistent, which was the whole point of using SQL from week 7 onward. And let's do one more for good measure. If I go back to the form, we'll register Doug so he can play basketball with me too, and we even have Doug now in the database. It's an ugly looking table, but the data is in fact all there. All right, questions now on this improvement, which is getting closer and closer to what the actual Frosh IM's database did, uh, website did so many years ago. All right, well, let me propose this now. We have this table of registrants. Suppose that um maybe uh Kelly was not a very sportsman like when she played soccer last time, so we want to deregister Kelly from soccer. That is, no, we're going to reject your registration. Let's. For a moment about the design here, like here's an HTML table containing names and sports, and wouldn't it be nice if we could add a button that would let me deregister Kelly or anyone for that matter. When I click on that button, what information should ideally be sent from the browser to the server to remove someone like Kelly from the database? Yeah, the ID of the person and you're proposing ID instead of name. Why? I need to be identified. Exactly, the ID uniquely identifies the user in the SQL table. So in fact, let's see this real quick. If I go back to VS code and we'll revisit essentially a week 7 issue here, let me go back into my second terminal where I can again run SQLite 3 after maximizing my terminal. And before I just wrote. schema to see what the table is, now I'm going to literally run select * from registrants in SQLite 3 and we'll see a little Askia table of all 4 of us who registered, but we also see. Unique ID and the value of the unique ID recall from week 7 is that it's the so-called primary key. It is the value that uniquely identifies users as minimally as possible, and that's a good thing because if we have another Kelly registering for Frosh IMs, we don't want to deregister the wrong Kelly or both Kelly's. We want only though Kelly with ID of 2. So somehow the button we add to the registrant's page should contain in it the ID of the person we want to delete, because if you do pass the ID. ID of the person that you want to delete to the server, the server can do some kind of select looking or some kind of delete statement using that ID number and delete just that row. So there's a few ways we can do this, but let me propose that we proceed as follows. In our registrants route, which is where we can currently see all of these users, let's go ahead and output an ugly but functional form for each of those users. So let me go ahead and minimize this. On hide my terminal window and in registrants, let's go ahead and just do this. In addition to outputting every registrant's name in sport, let's also output a third column whose purpose in life is to contain an HTML form. The action of that form will be a route like de-register, and the method we're gonna use is going to be post just so that we don't. Accidentally store personally identifying information in a URL or such. This form is going to have a button, the type of which is submit, and the button is going to say deregister, and I could now implement the ID in a couple of ways. I could do input, name equals ID, type equals text, and now if I go back to my other browser tab and reload, I should see a button for every one of these registrants. And I do, but this is kind of like the honor system where I just let the user type in the ID of who they want to delete, and it's sort of weird that I have multiple forms in that case, but here is where dynamically generating HTML can get pretty. Uh, useful. Let's change the type of this input to hidden and set the value of this uh input to be whatever the current registrant's ID actually is,

uh, storing this in here and let's go ahead and not confuse this, so we'll use single quotes on the outside instead. So inside of this value, I'm putting the current user's ID. So if I go back now, notice that the text boxes are going to disappear, but the buttons will not. But all of that information is still there. If I right click or control click and open up my developer, uh, let's open up View page source. It's just a bit bigger. Notice that David and Kelly and John and everyone else here has the same HTML as before, plus another column containing a form that contains, oh, I somehow messed up still. Why is this blank? So this is still not good. Ah, thank you. I accidentally pluralized this, but it should be registrant because I'm inside of this for loop and each iteration gives me a variable called registrant. So user error on my part. So let's go ahead and dramatically do this again. Let me view page source of the same page, scroll down a bit. Thankfully there is now for every one of these registrants a hidden ID. 1 for me, 2 for Kelly, and I bet if we keep scrolling, we'll see 3 for John and 4 for Doug. So now this form has enough information even though there's no user input other than the clicking of the button to tell the server whom to delete. So how do we delete the user from that particular registration table? Well, I think we just need to add a route. So let me go back into VS code here into app.js and let's go ahead and create another route, for instance, uh, in here, say, uh, we'll put it up here below uh up here below index, so app.route slash deregister, whoops, deregister. And now de register, but I could call it anything I want and how do I do this? Well, let's first get the ID from the form ID equals request.form.get ID. Let's do a bit of a sanity check here. So if there is an ID, and it's not blank for some reason, go ahead and do db.execute, delete from registrants where ID equals. Uh, question mark and now let's pass in the user's actual ID and then no matter what, let's go ahead and redirect the user back to the registrant's page so that we can hopefully see the result of that change. So again, I'm just using a bit of SQL per week 7. I'm using a placeholder by using the question mark passing in the actual ID from the form, and I'm only doing this if there is an ID that was passed in and I'm letting the database actually do the deletion. All right, so let's try to do this. Let's go back to the browser here, reload the slash registrants page for good measure. Let's decree that Kelly is now deregistered by clicking this button, and oh, so close. Method not allowed at the deregister route. What did I do wrong? Let me go back to the code. What's wrong with my deregister route? Well, what method is the form using? If I go back to registrants. HTML, the the form is using post. Yeah, so I need to override the default, which is get, so I need to go up here again and just change an argument to be methods equals and then in a list containing only post now instead of get. All right, let's go back to the form and go back and now let's try to deregister Kelly. She's gone. Let's get rid of me now. I'm gone. And indeed if I go back to VS code, open my terminal, maximize it, and select star from registrants again, you'll see that the two of us are indeed. Gone in this case. Questions now On this technique because now we have most of the plumbing in place for adding people to a database, deleting people from a database it's very similar in spirit now to most any website that has this kind of interactivity. All right, subtle question. I deliberately in my registrants. HTML file uh used post, as we just discovered instead of get why though? Because it wasn't that strong an argument that I hinted at earlier of like oh I don't want like Kelly's ID to end up in my URL bar or mine like IDs are not really personally identifiable they're just opaque integers at the moment. But why would it be bad if you could delete people by using the get method? So this is kind of subtle, but the catch with using get is that by definition you can visit that resource, that route by just typing in a URL or following a hyperlink. So for instance, if an adversary were to

type a URL like slash registrants question mark ID equals oh, I don't know. Uh, 4 and then send me this URL in an email or send this URL in an email to the proctor who's running the Frosh IMs program. If that Proctor simply clicks naively on this link as my code is implemented now and I've used get instead of post, what's going to happen? Doug gets deregistered just because the proctor followed a link in their email, and this is hinting at the kinds of phishing attacks that are possible too. Bad design, like generally when you are using get requests, that is just simple URLs that are clickable or typable, they should not have the effect of changing data on the server. Post is much better if only because you can't just click a link and post happens to induce a post request, you almost always have to click a button. So at least this case the proctor would receive an email. They would have to receive an email, click on a link, and then they would see a web page like this that clearly has a button label deregister or the like, which is an additional layer of protection. And there's even more tax that you can wage by supporting get. So in general post requests are preferred any time there's anything remotely personally identifiable or remotely destructive, like actually changing data on the database like this. Alright, well, what more can or should we do with Frosh IMs perhaps? Well, let's see, maybe one or so final flourishes here. Um, if I want to go ahead and maybe make those error messages a little more interesting, let's do that for just a second. Let me go back to. Uh, my other browser tab here, let's go back to the registration page where the form is, and let's deliberately not cooperate and just click register so that I get an error about missing name. Well, wouldn't it be nice if we made this a little more user friendly by including like an image on the page, as is commonly the case? Well, we can certainly include images and websites using the image tag, but the catch is we actually have to be a little more clever about how we store the image on the server in order for this to work. So for instance, let me go into that error page. We don't need success open anymore. We don't need layouts anymore or this index anymore. Let's focus on error and suppose that I did want to include an error message containing like a grumpy cat on the screen. Well, ideally I would just do alt or I would do open bracket image, uh, source equals, and then something like cat.jpeg where cat.jpeg is the name of a cat in this current folder. And just to be clear, let's have an alternative text of grumpy cat for screen readers or slow connections. OK, this unfortunately is not going to work. Let's go over here and induce the same error by just reloading and submitting the same form, and you'll see indeed a broken image because that image that cat. JPEG does not exist, but we do at least see the alternative text. Well, I did come prepared with a cat already and so let me go. and grab this cat from another folder, and this cat is going to contain, uh, is going to exist in a file called cat.jpeg and indeed if I type LS now after having grabbed a copy of that cat, it exists alongside A.ie seems good. Let's go back to the browser here. Let's reload and we should see. Oh, still no cat. Well, why is this? Well, this is a side effect of using the framework as well. It turns out for organizational sake, any images you want to display on a page or any CSS files or JavaScript files that you want to embed in a page, if they're static assets should actually be in a folder called static, and by static that just means unchanging. You or someone else wrote them once and they're not dynamic in the way that app.ie. So I'm actually going to use my MV command and move cat.jpeg into the static folder. Indeed, if I type LS now, CA is gone, but it is in the static folder. And now if I go back over here, I think we'll be good, except that I do need to go into error. HTML and say that the source of this image is actually in static scat.jpeg to make clear it's in that folder. And so indeed when I now reload the page once more, now I see a very grumpy cat at least guiding my

error message. Oh. But there is a difference here, even though when accessing the static directory I have to be explicit, notice that this whole time we have never once mentioned the templates directory. The render template function to be clear, knows automatically to look in the Templates folder for your template you do not and you should not say something like templates here you simply specify the name of the file, but in the in the uh HTML template you do actually have to include, as I did slash static in the HTML. All right, let's do one final flourish with the actual code. Suppose that it's time to modernize and let people register, not just for one sport as per the radio buttons, but multiple sports. It's a little obnoxious to make me go back and fill out my name again and again and again if I want to register once, twice, 3 times for sports. So why don't we go ahead and in terms of you. I changed those radio buttons to checkboxes. That's a very easy fix. Let me go into my templates folder and into index. HTML where this form is. And if I want to change radio buttons to checkboxes, literally just change radio to checkbox. If I go back to the browser here and reload, you'll see the familiar checkboxes now, which are not mutually exclusive. Lets me check multiple ones, thereby registering for multiple sports at once, but my logic has to change a tiny little bit here whereby if I want to go ahead and get all of the sports for which the user is registered, well, that logic has to change in app.ie. So where is my register route? Down here, and we haven't touched this in a while, but recall that the register route here. Has a validate name chunk of code validate sport chunk of code, and we most recently did the insert into chunk of code as well. But if the user's registering for multiple sports, I'm OK with having one row per sport. Though I'm sure we could do better than that, but how do I iterate over all of the sports that the user gave me? Well, I need to change my validation code here a little bit. If you know the user can select multiple values, as with checkboxes, you're going to use request.form.getlist. And then the name of the uh parameter that you want to get the value of and then this is going to give me back a list of values. So I'm gonna go ahead and change semantically my code to say sports because I'm expecting 0 or more sports now instead of 1. So if there are no sports, we're gonna just say missing sport heck missing sports. Um, but then I can't simply do this. I can't just say, is the sport for which the user registered in that array or not because they might have given me 2 sports or 3. So logically I should really check all of the sports that the human typed in for me and I should probably do something like this instead. So for each sport. In the sports that the user typed in, go ahead and ask the question if that sport is not in sports, then go ahead and output invalid sport. So it's just a bit of tedium here. We're just adding a bit of logic, but this way I'm iterating over every check box that the user checked and making sure they didn't do what Kelly did earlier and sort of make up her own sport and submit that to me. Among all of the others, but this now should let me let's try, let's reload. oh, and then actually one other line here we also need to do it down here, uh, for each sport in sports we better execute that line of code multiple times. So let's see what happens. Let's go ahead and register David for actually let's see what who's in the database. So registrants, so we've got John and Doug, no David or Kelly. So let's reregister David for basketball and soccer click register and now I'm indeed registered for both. And I observed that it's kind of bad design that I'm just inserting myself twice into the database. So let me go ahead and open up the Fros IMS database one last time. Uh, let me do a select, uh, let me do a select star from registrants. You'll see too that David and David are both there. What would be a better design here to get rid of the redundancy? And to know that I'm the same person ideally. Yeah. Yeah, I should probably have an ID for the the person as well, so this is going to complicate it more than we want to play

with today, but instead of just a registrant's table, I should probably have like a student's table that has an ID for every student and the name of every student, and then change this table as we've seen with the IMDB database and others, I should really be storing the IDs of the students, the Harvard IDs if you will, and not just their names like this. So there's room for improvement, but the point here is just how we can actually use check boxes and get back. Multiple items from folks. All right. That was a lot. Questions on where we're now at. All right, to make the coding a little less tedious, what we're going to do is look at a few final examples that have sort of come pre-made, and we'll walk through the code pointing out only what's different as opposed to some of the boilerplate that we keep seeing. Um, where we left off now recalls that we have App.ie, which is all of our logic requirements.text, which just enumerates the libraries that we want to use in the project, Static, which now contains any static files like CATs or JavaScript or CSS, and templates which contains our actual templates. It's worth noting that we're actually following a fairly common paradigm. This is not. Specific to flask, the model that we've essentially the the paradigm that we've essentially been implementing is this. If this shape over here represents the human or the user, they keep interacting with what the world generally calls a view. A view is the term of art that just describes the user interface, aka view. But that view is generated by a certain type of code, namely controller logic. So app.ie is technically what the world would call controller logic or business logic, to use an industry term, and that. Controller code, aka app.ie is generating one or more views. So the views that we're referring to here is like everything in your templates. Those are your views. But there's a third piece of the puzzle that we just introduced which is generally called a model, and initially my model was just a stupidly simple dictionary in memory, and that evolved eventually into Frosh IMs.db. So your model is generally your persistent data like where you're storing data related to the application. And even though the picture doesn't lend itself to pronouncing it in the right order. This is what's known as the MVC paradigm model view controller, and it's a very common way of developing web apps by just thinking about the different problems you need to solve with this kind of nomenclature. Like I've got to implement my controller which does all of the logic, all of the variables, functions, conditionals, loops, and so forth. I've got to implement the view which contains everything the user sees and interacts with like the HTML, and I've got to eventually implement the model which is like all of the back end data space and such. The catch though is that this is not a clean line because clearly in views, we've seen variables, we've seen loops, we've seen conditionals, so this is just a general mindset to have and then in the real world, if you ever uh explore web apps again. You are henceforth familiar with what's known as this MVC model. But now let's solve some other real world problems. So here's what you see on the occasion that you sign into something like Gmail or really any other website that asks for a username and then eventually a password or some such thing. This is just a web form. It looks a lot prettier than mine because they're using some fancy CSS to make things blue and nicely indented and so forth, but it's just HTML underneath the hood. with probably an input type equals text to give me this text box. Of course, when you log into Gmail after providing your password, somehow Gmail remembers often for days, weeks even that you have logged in already. Now how is that actually working? Well, when you first log into a site like Gmail and click submit or the next button in this case, presumably the browser is submitting in an envelope, so to speak, a message like this to Google's servers, post slash something to accounts.google.com, which happens to be the URL that Google typically uses for this. And

inside of this, there is your username and password and anything else that might be submitted to the server. Ideally, the server responds to you with 200 OK, like, here is your inbox. OK, you logged in successfully, but it also underneath the hood. Every time you've been logging into Gmail has been planting a cookie on your computer, and you might be generally familiar with cookies. They have kind of a bad rap because they're often used and are used quite frequently for tracking, for advertising, and really kind of keeping eyes on you in some way. But in their basic form they're just a feature of HTTP, which is wonderfully useful because it solves some typical problems. This is. Another HTTP header that is usually inside of those virtual envelopes that come back from servers to browsers. In addition to telling the browser what the type of content is in the envelope, it might tell the browser, Please set the following cookie. A cookie is just a key value pair. It might be something like session literally equals some value, and that value is usually a random string that might be 123456, or something like that, but it's a unique identifier. Or naively, if Google implemented cookies poorly, they could technically tell your browser to store a cookie on your computer containing your username and a password so that tomorrow when you open up Gmail, you're not prompted again with the stupid form to log in. It already knows your browser that you're logged in, and your browser can do that by just sending the same cookie it got yesterday to the server. Now this is bad to use cookies to store usernames and passwords. because it's putting very precious data in the browser's memory and any sibling or roommate who walks over to your browser can now find your username and password by just poking around your cookies. So generally what browsers do is more like this screenshot here whereby all the server does is it puts a big random value on your computer somewhere, essentially a text file containing a big random value, and that is equivalent essentially to a sort of a handstamp like if you go. To a bar or a club or an amusement park, generally you show your ticket once when you go in and then thereafter you just show your hand if you want to be able to come and go again and again. So right now my hand has not yet been stamped. We have this nice little smiley face sticker. I might have a smiley face now on my hand any time I want to go back into the bar or club or amusement park because they now know we already checked who you are, presumably the very first time that you came in. That's all cookies are. Effectively doing is it's putting a virtual hand stamp in your browser because the browser, the next time you go to Gmail and click on a link or click on an email, your browser, unbeknownst to you, will send a get request that looks like this but also contains a line like Cookie colon, and then that same key value pair. It's like presenting your hand stamp again and again every time you open an email or click on a link in Gmail. This cookie header is what the browser sends. This set cookie header is what the server sends. So this is the act of stamping your hand. This is the act of presenting your hand, and that effectively is how browsers and servers remember who you are. This is how advertisers generally remember who you are because at one point or other they put a cookie on your computer and unbeknownst to you, you're going to this website, this website, this website, and your browser has been presenting this hand stamp all. All this time, so advertisers know, oh, that's David again. That's David again, that's David again because they're seeing the same hand stamp. And so one of the reasons why last week, for instance, I kept opening things in Incognito mode, which you might use generally if you want to do something private and not have it be saved in the computer's memory, is also because Incognito mode gets rid of all of your cookies when you close the window, effectively like wiping off the hand stamp the next time you go to that same website. So

that's all a cookie is. It's a key value pair that can be planted on your computer, but it's a wonderfully powerful mechanism for implementing, and this is the juiciest idea for today, I'd argue what are called sessions. Sessions are this feature whereby browsers and servers have a persistent connection to each other even though HTTP is what we'll call stateless. So stateless just means that you don't have a constant. Connection to the server when you are using a website and that's not always true and nowadays you sometimes do have a consistent, a persistent connection, but cookies allow you to close your laptop, even shut down your computer, come back the next day and still have the illusion of being connected just as you were the previous day because of this virtual presentation of hand stamps. So a session more concretely, you can think of in Python as a dictionary of key value. Pairs that you can associate with each and every user. That is to say, when I log into a website that is using sessions implemented with cookies, they can store any number of key value pairs about me in the server's memory and my presentation of the hand stamp will ensure that they keep, they know which key value pairs to assign to me. Let me go back into VS code here and let me CD into a directory with which I came, which. Called Login, which is just going to be a relatively simple flask application that demonstrates how you can implement the ability to log into a website, and we'll keep it super simple with just user names, no passwords, but as you'll see in Problems at 9, we'll add some passwords to the mix as well. If I type LS inside of this login directory, you'll see some familiar friends app.ie, requirements.text, and templates. But let me draw our attention to one other library we're going to now start using called Flask Session. So Flask session is just a third party library that gives us the ability to use cookies in our application and not have to know or understand any of the screenshots we just saw of HDP requests. It sort of suffices to stipulate, OK, someone figured out how cookies works. I just want to use them now as a feature so that when a user uses my website, I can associate data with them like who they are, what their username is, and therefore that they've logged in. So let's go ahead and close requirements.text and open up. App.ie in this case. Here is an implementation of a program whose purpose in life is to enable me to log in. And in fact, before we before we walk through the code, let me do this in this terminal. Let's do flask run and I already hit control C on my other terminal window a moment ago. Uh, let me now go into my other tab up here and reload the slash route, which is now going to be this login route instead of Frosh IMs. All this website does by default is it tells me first, you are not logged in, but here's a link to log in. It's a little small, but if you look in the bottom left hand corner of my browser right now, it's a URL that ends with slash login. And in fact, I can see that more clearly if I view page source in the browser. Here is the only thing I'm really seeing. In this web app so far, but notice what happens now. If I click on log in, the route in my URL just changed to slash login. I'm again keeping it simple with just usernames, no passwords, but I'm going to log in as David and click log in. But first, let me show you the code. In View page source, I have a form that submits to slash login using the post method. The only thing about this button that's the form that's interesting is it's got a text box and a login button. Same as we've seen before. So let's click it. Now I click log in and notice I get whisked away back to the original route, the slash route, even though Chrome is hiding the slash from me, but the website somehow knows that I'm logged in as David. In fact, if I open up my page source in the browser, I'll see that now. It doesn't say you are not logged in. It says I am logged in as David, and it's now giving me apparently conditionally a logout link. So I argue this is representative now of any website that lets you log in and out of it. So how does this

work? Well, in my login account, in my login app here. What do we have in Aie the following. I've got from Flask import flask redirect, render template, request, and a new one session, which you can essentially think of as a dictionary where you can store key value pairs for each and every user, and Flask will make sure that your code. has a different copy of session for every user that visits. You can just treat it as though you only have one user, but Flask will ensure that when a user visits, they get their own copy of session, their own copy of session, their own copy of session, essentially to store whatever you want. This next line here I just need to copy paste from Flask session import capital session. This line is the same. Turn this file into a flask app. This stuff is new and find a copy paste. This just says configure this app to use sessions by storing the cookies on the server as files instead of in a database or somewhere else, but this is the default that we use for our examples. All right, what's going on here? Well, in my. Slash route I've got an index function whose purpose in life seems to be to render a template called index. HTML and then pass in a name placeholder which is the value of session.get.name. So whatever name is stored in the session, if any, that gets passed into the template. So let's go down this rabbit hole. Let me open up index. HTML. Interesting. So here is the logic that implemented those two different versions of the home page that we saw. If the name has a value, so if it's not empty, we saw you are logged in as such and such. Here's a logout link. If though there was no name, as happens by default before you even log in, you see you are not logged in. Here's a link to log in. So that's all the home page is. It's conditional logic checking if there is in fact a user logged in. All right, well, let's go back to A.ie. How does the login work? Well, if you find your way to the login route, then I'm asking a question. If the user got here via post, they probably got here by clicking the login button that I gave them. So let's store in the session dictionary the word name. And make the value of that key this value here where what I've just highlighted is whatever the user typed into the form, whether it's David, Kelly, John, or anyone else, that's what comes back from the form and I'm just storing that in the session, which again is like this special global variable that you get one per user and it's implemented underneath the hood by way of cookies or these hand stamps. Then I'm just redirected to the slash route. Otherwise if the request method wasn't post, that means the user just manually. Visited example.com or whatever my website is. That's why I show them login. HTML. All right, let's go down that rabbit hole. Let's open up login. HTML. It's pretty simple. It's just a stupid form that has a text box and a submit button, but the most important part is that as we saw in the browser, it submits to slash login, the route we just saw. All right, if I go back to here, how do you log out? Well, we didn't actually click this, but here is how you can delete the contents of the. Session and actually log the user out, you just call session.lear. And so in fact, if I go back over here and click log out, how does the server know that I've logged out? Well, that route very quickly you didn't even see the URL bar change logged me out by clearing the whole session. And so the cookie that was planted on my computer was essentially deleted at this point in time or really the server side data that's associated with that cookie was deleted, so I'm no longer seeing it at all. So that's kind of it, like if you log into a website, whether it's Facebook or Gmail or Outlook or anything else like that's effectively how they're logging you in, but of course they're adding into the mix some uh passwords and other security as well. All right, how about one other example? Let me go back into VS code here and let me go into my first terminal, hit Control C to kill this login example. Let me hit CD to go back and then CD store to implement the simplest of web stores like some kind of e-commerce site that has an actual shopping cart

implemented. Let me do flask run inside of this directory. Open up my other terminal window, and in my other terminal window I'm going to go CD to go back and then go into store here where I'm going to see some familiar files, namely app.py, requirements.txt, but a database file this time in addition to my templates. Well, let's see what's inside of that database. Let me go ahead and run SQLite 3 of store.db. Schema to see what's in the database. Ah, this is like a bookstore, like the very first version of Amazon.com. If you will, and the table has two columns an ID column and a title column for all of the books that this store shall sell. Well, what are those books? Select star from books, semicolon. OK, so this is a bookstore that sells only 5 books, among them The Hitchhiker's Guide to the Galaxy and sequels. All right, so wouldn't it be nice if we have a website that displays everything in this catalog and lets me Add thanks to my cart. And in fact here is maybe the better metaphor for what a session is. A session essentially gives you the ability to implement a shopping cart like this, where the shopping cart, of course, in the real world is specific to each user. Like if I'm on Amazon.com and Kelly's on Amazon.com and both logged in, we obviously don't see the contents of each other's carts, and that's because we have separate cookies on our hands and so flask or whatever Amazon is using creates the illusion. In that we each have our own global dictionary called session in which Amazon can store any key value pairs it wants like what's in our shopping cart. So let's try this. Let me go back to my other browser and reload, so I'll now see not the login example, but the bookstore example, and it's super ugly because I whipped it up using the simplest of HTML, but you'll see here every one of the books in the database plus an add to cart button. And even if again you're sort of new to all this web programming, there's not all that much you can do with HTML except use forms maybe with some hidden elements to achieve this result. So here we have the H1 tag with books. Here's an H2, which is big and bold but not quite as big. Here's the, here's the here's the button for the Hitchhiker's Guide to the Galaxy. As an aside, because there's like a curly quote or an apostrophe in the book's name. This is just an HTML entity that Flask is outputting for me, even though it's not there. Uh, visually in the database. So what does the button do for Hitchhiker's Guide to the Galaxy? Well, it's a form whose action is slash cart, presumably because I want to add it to my cart using the post method. I've got an input name equals ID, the type of which is hidden, the value of which is 1, and fast forward 234. So just like the deregister example for Kelly, similarly is each book going to be addable to a cart instead of removable by using that unique ID, and indeed every form has an add to cart button. So what's happening then on the server? Well, let's take a look at the other tab here. If I go back into uh VS code, and if I go into my, let's say, let's minimize the terminal window here and let's open up inside of store, let's open up our template for index. HTML, which is sort of the entry, which is not that, uh, let's open up app.py first and figure out what's going on. So at the top we have some imports including our SQL library. We have an app variable being created, a DB variable being created using that same store.db. We've got this boilerplate code which just again enables cookies and stores the contents on the local file system instead of in a database. Ah, here's the interesting beginning point. How did I see that big page with all the books and the buttons? Well, for the slash route we've got this function that first uses some SQL to get all of the books from the database. Select Star from books, and then, ah, there's no index. HTML because I called it books. HTML in this case just because, and I set the book's placeholder equal to the value of the book's variable. All right, let's go down this rabbit hole now. Let's open up the templates folders, books. HTML file. OK, so here we have that H1 with books and then

we have a 4 loop which is going to output for every book in H2 tag and a form tag, a form tag again and again and again, each of which has a value that equals the current book's ID, but the title in the H2, of course, is the title of the book, which is more human friendly. So what happens when I actually click on add to cart for The Hitchhiker's Guide to the Galaxy? Well, I should indeed see that now that one book has been added, and if I go back. At another like the restaurant at the end of the universe, I now have two books in my cart. So where is that data actually being stored? Well, if we go back to VS code here, hide the terminal, and focus on the cart route, the cart route, because it supports post in addition to get, also is doing this for me. Well, first, it's checking with some logic here. If there is no cart in the session, go ahead and create a key called cart and set it equal to an empty list. In other words, I can put any key value pairs into the session that I want. So if I want my shopping cart to effectively be a list of all of the books that the user has added to their cart, it stands to reason that my cart by default should just be an empty list when they first arrive. However, if the user has clicked submit in order to get here, well, I'm going to do this. I'm going to get the ID of the book that they've submitted via that form, and if it indeed exists and it's not someone like Kelly messing around and sending me invalid parameters, I am going to append to the cart list in the session. The book ID and then I'm just going to redirect the user to the cart and anytime you do a redirect that always is using get not post. And so when I come back to this cart route later, I'm not going to be using post. I'm going to be using get, which means this. chunk of code here is executed. I have a variable called books set at equal to the results of doing select star from books where ID in the following parenthesis list of IDs recall that in is the preposition that gives me back multiple IDs if I so choose, and then I'm rendering cart.html. With those their books and if I go back to the application, the reason why I'm seeing two elements here and indeed if I go to my developer tools or view page source rather, I'll see two list items inside of an ordered list or numbered list containing the contents then of that shopping cart. All right, so if we now have the ability to use sessions to remember who has logged in and we have the ability with sessions to remember what someone has added to their shopping cart, what else can we do with web applications more generally, even if not using sessions? Well, let me go ahead and close this tab here. Let me go back to VS code here, close out these two examples, and let's do a final set of examples that demonstrate what we can do with some real world data and a web application. I have lastly a directory called shows which is evocative of our use of IMDB in the past, and I'm going to go ahead into my first terminal window, hit Control C and call your attention to one thing before we move on. Every time I have executed a SQL query inside of my code in my first terminal window where Flask is running, you'll see either in green for success or yellow or red for some issues. The Actual SQL code, SQL commands that are being sent to your database. This is useful if you mess something up at some point related to a database query, you can actually see in your terminal where you're running flask run, actually what SQL command was sent to the server to try to troubleshoot errors that way. Otherwise you're just flying blind when actually interacting only with the web browser. But for now, let me go ahead and clear that away. And CD back to my default directory and CD now into shows where if I type LS we'll see a whole bunch of files A.ie, requirements.text, and this time shows.db, which is the very same database that we had in past weeks when we played with some of the very large number of shows in the internet movie database. And what does A.ie do here? Well, it implements the simplest of programs. This gives me access first to shows. with some boilerplate up top. If I scroll down here, you'll see that

there's an index. HTML template that's rendered by default, and then apparently there's a search route which is akin to what Google does for us when we searched for cats and dogs in the past. But for the first time I'm implementing my own search engine for TV shows, not for dogs and cats. But what does this search route do? Well, it uses a shows variable and it executes the sequel, so Star from shows where title equals question mark, and it passes in just like Google does the Q parameter for query and then it renders a template called Search. HTML passing in those shows as a placeholder. In other words, what does this do? Well, let me go back over to the store, uh, to the store tab here, change the URL to just slash, and because I'm now running, uh, I'm no longer running the store, I do want to go ahead and run in my first terminal window, flaskun. To start off the show's application instead. So if I now go back to that tab because no servers running, what I see here now is the simplest of search boxes like our Google example asking for a query, but this time I can search for things with which I'm more familiar like The Office capital T, capital O, search, and what I get back, not that enlighteningly, but is the title of every show that matches exactly that. If I go. Head and view page source you'll see that what was generated was an unordered list of offices that are in the database and recall there's the British one, the American one, and a bunch of others as well. However, this form does not work if I type in something like the office search. I get no results in that case, which isn't so much a well it's just a lack of features here. And so let me actually go into VS code here. And let me propose that we come up with a better version of this code. So in fact, I'm gonna go into the premade examples with which I came today. I'm going to go into the next version of shows here, run flask run here, reload the application over here, and now show you that the office in lower case does actually work. Moreover, it searches for anything that mentions the office. So if you had to guess, how might this be implemented underneath the hood, well, if I open up my other terminal window and go into that same directory. Shows one and open up this version of A.ie you'll see that instead of using a simple query like before, I'm now using the like keyword here because I'm checking that it is like the office and notice this is a bit clever here or a bit confusing at first glance. The placeholder I want is question mark, but I don't want to just search for the user's input. I want to tolerate zero or more characters to the left via the SQL wildcard and zero or more characters to the right. So I'm concatenating onto the user's input, a percent sign here, a percent sign here, because we recall from our week 7 with SQL. This just means look for anything case insensitively that has THE space all F F I C E in it. No matter where that string is in the text. How did it know to render that though as this bulleted list of all of these offices? Well, let me go into my terminal here and open up. Uh, search. HTML, which is the template that the search route is using, and you'll see that I'm just iterating over with a gina 4 loop each of those shows and then outputting a list item for each of those matches effectively just as I did before. But there's this other technique I can use altogether and it's generally going to open up more possibilities for us in final projects, if not beyond, of creating essentially my own AP. rather than just make a web app that spits out the entire HTML page that I want the user to see, wouldn't it be nice if I could just start to create routes that spit out the data that I want, and then I, or even some third party making a website with the same data can integrate my application into their own. And indeed an API is an application programming interface and it's essentially web-based function. You can call to get data from someone else's services generally using HTTP, and you can return the data in any number of formats in text format, in HTML format, or in something called JSON format, which is short for JavaScript object notation,

which looks a little something like this, which is quite like Python arrays and dictionaries combined. But notice here with the wave of the hand, there's a whole bunch of Key value pairs in this particular example of all of the offices that are in IMDB's database and so I wanted to show us these final versions of the same shows application that works a little bit differently. If I go into say shows 2 example here now run oops, and let's go ahead and exit out of the previous flask copy and run shows 2 inside of which is flask run, notice here that if I go back to this web form now. Notice that there is no more search button because this is meant to be highly interactive and I can search for THE space O F F I C E and you'll notice that this is effectively autocomplete, which we saw a taste of last week with JavaScript, which I am in fact using here. But how is this working? Well, let me reload and open up my developer tools and in Developer Tools, let's watch the network tab this time because when I. Type in something like T you'll see that my web page suddenly made a request to my own slash search route and if I click on my developer tools and look at the response that came back, you'll see that the slash search route spit out not a full web page but just a whole bunch of LI tags. Now why is that? Well, let me go back to VS code and open up in my other terminal. Uh app.ie and in app.ie scrolling down to search, you'll see that when I get shows from the database, I'm still using Search. HTML, which previously extended my layout and plugged in that whole ordered unordered list. But this time, if I go into this version of search. HTML, you'll see that I'm only spitting out raw HTML because I'm assuming that maybe someone, myself included, wants to use slash search to just get a Whole bunch of list items that they can put into their own unordered list or UL tag. And so what's effectively happening over here is every time I type a letter, notice at bottom left, another HTTP request goes across the internet, another HTTP request, and each of those is returning the set of LI elements that line up with the query that I've typed in. But this is a little sloppy, arguably, and so far as I'm returning a chunk of HTML but out of context and I'm. Dictating to the user that they have to use list items, wouldn't it be nice to just send the raw data, and I can do that too. Let me go back into VS code here and look at our final example, shows 3, inside of which is a version of this code that now returns that so-called JavaScript object notation. And if I go into shows 3, run flaskun, go back over now to my browser tab and click reload, I'll see now when I search for say T. And click on that row. Notice now in the response tab of my developer tools I'm getting back a whole bunch of juicy information, a massive JavaScript object notation chunk of data. Notice the square bracket means here comes a list or an array. Here comes a dictionary or dict, and indeed that's what I'm seeing. This looks like Python, but it's technically JavaScript and it's technically JavaScript's object notation. This just means this is the juicy data I'm getting back. From the server and if you now think way back to week 0 and even in our family weekend lecture on AI lecture on AI where I was writing code that talked to open AIs, so-called API, to get responses from our server side cat, they were sending us JavaScript object notation like this, and I was just grabbing the data that I actually cared about, namely the cat's actual response. And so in this case, if I open up. And my other terminal window here app.ie, you'll see in my search route that instead of returning a template, I'm using a crazy named function called Jasonify, which is just another function that comes with Flask itself that has the effect of taking the list of Python dictionaries that came back from my SQL database, Jasonifying it in such a way that I then can. Uh, serve it to anyone on the internet, myself included, as a service so that I and they can use my own data to implement their own web web applications. So that's sort of it for web programming. Ultimately you now have all of the building blocks from week zero onward

to make your own web applications, and if you so choose for final projects, your own mobile applications, even if this too, like every. Else has felt like a bit of a fire hose. It is in the process of your final project of specking out and proposing and executing your own final project that will make all of this feel much more comfortable and familiar, and you'll look back on so many of the past weeks as useful building blocks, but this then was your CS 50 education week 0 through 9. We have just one more left next week, so we'll see you then.

Alright, this is CS 50 week 10. The very end and we will end today's class just as we ended week zero, which is a little bit of cake outside in the transept. But over these past 10+ weeks, if you've been feeling like it was that proverbial fire hose sort of hitting you in the face with so much new content, so many new skills, so many new challenges, um, realize that you're in very good company and we can officially declare nonetheless that if you started the class among those less comfortable, you are officially after today no longer less comfortable. You're at least somewhere in between and if you were in between. You're more comfortable and if you were more comfortable, you're perhaps now most comfortable among those here. But keep in mind as for CS 50 syllabus, what does ultimately matter in this course is not so much where you end up relative to your classmates, but where you end up relative to where you yourself began, and that's taken into account come final projects, come final grades, but most importantly, that's really what's most important educationally in general is that Delta from week 0 to, in our case here now, week 10. So if it's any reassurance. And something I like to bring up around this time is just how badly I did in CS 50 and like the very first problem set. Like I didn't even get Hello world right somehow in the fall of 1996. So here's a photograph of my homework assignment for assignment one. It was a program to print Hello World on the screen. I was incredibly detailed with my comments, even commenting that Maine is Maine, which is not the way you're supposed to program, even telling the TF where my file ended, which is not really necessary, and I got -2 for not even following directions. Correctly, so take some comfort in that even if by problems at 9 you're still getting points off, you're hopefully, at least in my case, in some very good company. It only gets better and easier and faster in time. But the whole course ultimately has really been about this picture, right? Problem solving is computer science and you have inputs, which is the problem to be solved. You have the outputs that you want to get to, which is presumably the solutions there too. And inside of that proverbial black box are these algorithms, step by step instruction. For solving some problem, and I pulled up my own notes from CS 50's first lecture some 25 plus years ago too, where I wrote down this in my horrible writing, handwriting to this day, but I noted that what an algorithm is is a precise sequence of steps for getting something done, which is pretty much what we now say. I noted that programming itself, as we have for weeks now, is the process of taking an algorithm and putting it into a language that computer can process, and that's what you've done in Scratch and C, and Python and SQL and Java. and anything in between, and most important, at least my takeaway that day when it comes to algorithms is precision and correctness, and indeed those are points we've made perhaps not as emphatically over the past several weeks as well. But we thought we'd see just how much those two lessons in particular have sunk in, by doing a bit of an exercise, some CS 50 pictiography in this, our last lecture altogether this term, um, for which to begin we need one brave volunteer to come on upstage. Who would like to volunteer? Who, how about, OK, over here, we never call from the middle of the section, come on up, come on up, a round of applause for being so brave. Nice. All right, come on over. And In just a moment, let's go ahead

and do introductions. First, if you want to come up over to the middle of the stage and introduce yourself to the world. Hi, I'm Jia. I'm a freshman. All right, nice, nice to meet you. Thank you for joining us. So what we're about to do is Gia's going to look at my screen where there's going to be a picture on a white screen. All of you presumably have a white sheet of paper in front of you that you grabbed on the way in. If you don't. Just grab one from a friend or your binder or the like, and if you really don't, that's OK too, but hopefully everyone has a pen or pencil or someone near you does. And what we're going to ask you to do is program the audience to draw what it is you see on the screen. You can say anything you want, but you may not use any physical gestures or the like, verbal programming only. OK. All right, come on over to the lectern. And in just a moment, GN only Gia will see what is actually here on the screen. So. Step 1 for your audience. OK, so the first thing that you need to do is draw two lines right next to each other, 2 vertical lines. OK. OK. Step 2, once you have done that, you need to draw 3 dots, one on above those two vertical lines, one right in the middle between those two vertical lines, and one on the bottom of these 3 vertical lines beneath those two vertical lines. Yeah, so 3 dots. OK. Step 3. Step 3 is on the top of the left vertical line, you're gonna connect a line from that position to the top dot that you drew and then on the top of the right vertical line, you're gonna connect that position to the top dot that you drew. All right, step 4 is, remember that top left position? You're going to connect that to the middle dot that you drew. And then the top right of the vertical line, uh, the, yes, you're going to connect that to the middle dot of the line that you drew. Mhm. And then step 5 on the bottom left of your left vertical line, you're gonna connect that position to the bottom dot that you drew and then on the bottom right of the right vertical line you're gonna connect that position to the bottom dot that you drew. And now from the middle dot to the bottom dot you should have no line in between that, and you can now draw a line between those two dots. Step 6 and the last. I think you should be done. All right, a round of applause for our programmer. Let me give you a little something. If you want to take a seat, well, now what Kelly and I are going to do is very quickly collect your execution of this program, and we'll see just how it went with Gia as the programmer. If you want to just reach out and hand me or Kelly over there any of your handwritings, we don't need all of them, just a representative sample will suffice. If you're proud of your work, extend your hand quite a bit. OK, very proud. OK. OK. OK, OK, one more, one more, that's OK. All right, I'm gonna run back to the stage. OK. It's OK if we didn't grab yours. All right. All right, thank you to Kelly for grabbing these as well. So, without having seen any of these, here is how you all interpreted Gia's instructions. So here's one interpretation. OK. Perhaps similar or different from your own. Uh, here's another several. Vertical vertical line, question mark, OK. He is. Very narrow one. All right, and And let's see if we got any other variants thereof. Actually, the rest of them are pretty consistent. So, gee, if it's any reassurance, I'm seeing a lot of ones that look like this. Here's another that looks like this and here's yet another that looks like this. So if you're wondering where we're going with this, if I go ahead and reveal what it was Gia was looking at on the screen, she was in fact having you draw this here cube. So some of the takeaways here. So suffice to say, not all of that went well, uh, but why was that? Well, I dare say it was very easy to get confused, I think, G, in some of your words because you had in your mind's eye exactly what it was you were drawing and. Of course it was right there on the screen, but we didn't leverage, at least in GSA instructions, any abstractions. I dare say it might have been a little bit easier for all of us if maybe she had just teed things up by saying, All right, everyone, we're going to draw a cube, for

instance, which is indeed an abstraction over these lower level details that she was focusing on. But perhaps there could have been another approach altogether, which is even more pedantic. For instance, a lot of the earliest drawing programs. And even worlds like Scratch sort of take for granted that you have a coordinate system like X's and Y's and you can go up, down, left and right. So an alternative to just saying, hey, I'll draw a cube, which could be subject to interpretation because there's a cube like this, is it like this rotated so we still would have needed more information than just a cube from Gia. But here maybe an alternative approach would have been to really get into the weeds and say put your pen at the top of the page and then draw a Straight line to the southwest, for instance, and then draw another line of the same distance to the south and then to the southeast or so forth and it could have been in terms of degrees, it could be directionally in that way, but it might not have been clear to anyone what it was we were drawing until enough of the lines suddenly appear on the screen and then voila, you see that we've been drawing a cube this whole time. So the degree to which we're precise and the layer of the level of abstraction that operate in is incredibly important whether it's for another human to understand us, for an AI to understand us nowadays, or anything in between. All right, why don't we go ahead and flip things around a bit um for this, why don't we go ahead and get one more volunteer to do something a little different here on stage? One more OK, how about here on the aisle? Come on down. Round of applause for this brave volunteer. Come on down. All right. So in this exercise, we're gonna flip things around. So you all will be giving the instructions verbally by just shouting them out and our volunteer whose name is Preston. Presley. Presley, Presley, you want to say a quick introduction? Yeah, uh, my name is Presley. I'm a freshman, uh, living in Stton House. Nice. Well, welcome. Come on over to the, the, uh, the easel here, and we have a black marker for Presley here. And the only thing that we ask is that you not look up or behind you because the answer is going to be Right there on the screen, but everyone else is welcome to look up or over to the TV screen. And if you want to go ahead and face the easel here and as you draw, just make sure to kind of open up after each stroke of the pen so that everyone can see what you have done. All right, so no looking up as of now because what the audience is about to do is to program you to draw this on the screen. Oh, way to encourage him. OK, so step one, feel free to just raise your hand and we'll shout them out. I heard draw circle over here, but not too big, I heard over here. Good abstraction, you're gonna end up drawing a stick figure. But we should probably be a little more helpful than that. So let's do the hand thing just so we can be more precise and not overwhelm Presley with the was the hand over here, yeah, and back. Draw a line down from the circle, Presley. From the bottom of the circle. OK, someone else? So actually, let me, let me rewind, sorry, say it again. Draw two diagonal lines from the line you just drew. Well, I don't think the audience likes this. Wait, let's, uh, mm. OK. OK, that's what we were told. Next step, someone else? So OK. Good one. OK. Extends the original vertical line to be about the same height as the circle. OK, yeah, that's good, good feedback. All right, someone else. Next step. Next step. Yes. Nice. Draw two diagonal lines from the bottom of that line that look like legs, good use of detail and abstraction. OK, nice. Next step. Anyone? We're close. Yeah, over here. On the left. So you're gonna draw a speech bubble to the left of the head with the word high, capital H with a short line. No bubble, just high. And you wanted to clarify one other detail. A line from high to the face, with space in between. OK, no, you're doing great, it's OK, Presley. Hang in there. OK, final step or two. Next step, Anyone at all Feel free to shout it out. Adjust the arms to make them look like they're

running. Good luck. Oh, I like that. Draw a perpendicular line from the left arm. To the bottom. OK. And lastly, one final step. No. Same side. Yeah, it's permanent. Uh. I think we need a final touch on the other arm, maybe, yes, one final step. Anyone? Draw a perpendicular line per uh diagonally to the left. Of the right arm Mm Just a little bit. All right, I think I've, I think we've withheld our applause long enough. Presley, if you want to take a step back and look at what you, they were trying to get you to draw a round of applause. So here too, let me, here you go, your dorm room if you would like. OK. And a little Super Mario as well. All right, so here too, um, I think you were the problem this time. Round of applause for Presley. And of course since it's, you know, permanent ink, it's easy to sort of go off the rails early on and make a mistake, but I think that was actually a nice mix of low level details like the directions of the lines and the lengths thereof and also some abstractions because I do dare say someone shouting out that it is to be a stick figure gave him a much more helpful mental model. So that might be sort of the comments on top of the function, but when we really got into the weeds of implementing that function, it was more akin to step by step instructions for solving this here. Particular problem. So my thanks to Presley for bearing with us with that one as well. So beyond this, where have we been up until now? So, uh, if we look back at the past several weeks, this is sort of the trajectory on which, uh, we've been. So we started with scratch from scratch, literally in the very first week, the goal of Vol was to introduce you to some of those procedural fundamentals like what a loop is and a conditional and Boolean expressions and variables which are pretty. Much recurred in different forms and different languages over the weeks since thereafter we transition to a more traditional language C, which many of you will never use again and admittedly even I only use it for like a month or two of the year during CS 50 itself, but the intent was to be this incredibly foundational language that so many other languages today are built on top of. Case in point, the interpreter that you might use for Python itself can be written in C and that speaks to how we sort of talked. Bootstrapping from one language to another, from low level to high level and beyond, arrays and algorithms, all of that and memory and data structures like all of that is sort of omnipresent in computing, in programming and the like, even though you might not need to in modern languages like Python, worry as much about managing your own memory because good programmers, better programmers have figured out how to solve those problems for you in the language itself or in the libraries that you're using. You can take for granted now that you at least know what a hash table is. What a linked list is, what the trade-offs are among those, what the running times are, and that's what computer scientists and software engineers think about and talk about and whiteboard about in the real world when trying to implement algorithms of their own to real world problems or implementing real world products. And then of course over the past few weeks we've sort of used that as a stepping stone to talk about very modern programming paradigms, most recently web programming, and even though we didn't use it explicitly in the class, mobile programming is increasingly based on HTML and CSS. And JavaScript, which might be something some of you will tackle for your own final projects, and you can't escape now using or seeing or leveraging somehow artificial intelligence. And among the goals for today is to at least point you in the direction of tools that now, having finished Problem set 9, you are welcome and encouraged to use for your final project so that you can build all the more and all the more successfully than even some of your predecessors just a few years ago could have now that your own work and your own know-how can be amplified by the impact. Of AI itself. This of course now brings us to today, the end, but wanted to give you a

sense of where you can go here on out. So with your final project, this really is the intent of the final project is to be the very first of hopefully many projects that you decide to spec out for yourself. Like every problem set thus far has been written by me and the team, and you've been following our instructions step by step. The Frontal Project takes all of those training wheels off, and even though you are welcome and encouraged to borrow, code. From say problem set 9 if you want to do something web-based or even earlier, if you want to do something that's more similar to past Pets, is to make it ultimately your own. And even if you want, start with a completely empty window and just a blinking prompt and build something of your own, um, setting out for yourself, as you've seen in the specification, a good goal which you intend to meet no matter what, a better goal, which is a bit more of a stretch and a best goal, which in practice rarely ever happens with software to this day, 25 years since. Taking CS 50 myself, um, or plus now, um, even I consistently underappreciate just how long it takes sometimes to solve problems, but that's beginning to go away, at least to some extent thanks to AI, where at least now you essentially have a junior colleague next to you who can help solve bugs for you, point you in the right direction, even tackle features as well. Um, all that we ask for this final project is that you build something of interest to you, that you solve an actual problem, that you input to campus, or that you. As we say in the spec, change the world and try to achieve something, try to create something that outlives the course itself over these final few weeks of the class and even continue on with it if you'd like in January and beyond. So what are the support structures in place and how might you go about this? So a long standing tradition of CS 50 is this epic late nighter, the CS 50 hackathon, where by around 7 p.m. we'll serve some pizza around. p.m. we'll serve some ice cream, and if you're still awake come midnight when the event will start to wrap, we'll get in a whole bunch of CS 50 shuttles, drive down the road to the International House of Pancakes, which is open 24/7, and have some breakfast together if you choose, with some pancakes together at this local IHOP. Here are just a few visuals from the past year's hackathon of students gathering in the engineering building across campus. Um, here having some of that same pizza and here still being awake at uh early morning hours as well. uh, to paint a picture of this too, the team a few years back kindly put together a video, a sort of sizzle video that paints a picture of what it might be like if you at least were a Muppett attending this year, uh, CS 50 hackathon. If we could go ahead and dim the lights, I think we can go ahead and play, uh, this teaser of what awaits later this week. Just one night all we got. Just one night, all we got. Just one night. Does Still On the weekend. All right, so what then is the CSMC hack upon itself? What really is the late night opportunity to dive into, to start, uh, to make progress on your final project? The teaching staff and I will be there on hand, not so much to answer any and all questions for you, but to help you help yourselves, point you at resources, point you at, uh, the requisite websites, the documentation, so as to empower you ultimately to solve this here problem, uh, for yourself without some of those same prior training wheels that we've had. On, uh, the goal ultimately is to prepare you for the end of semester exhibition, the so-called CS 50 fair, which will be an opportunity to present your final projects on your phones or laptops, uh, to passersby from the faculty, from students, from staff, from visitors all across campus here, uh, some emoji balloons will await you here, uh, a photo booth if you'd like to uh immortalize the the memory, uh, and the goal of the hack of the fair will be really just a Present your projects to others that come by, uh, delighting in what it is you have achieved over these past few months alone, and of course by the end of it you'll pick up your very own CS 50 t-shirt which says

hopefully proudly that you have now indeed taken CS 50. But let's just see in preparation for that how much has indeed sunk in. We did have that test a couple of weeks back, oh, which went great, but we thought we'd pluck some of these same review questions that you prepared over the past couple of weeks to challenge at least 6 of your classmates to come up. Uh, for now, this, the so-called CS 50 charades for which we need 2 teams of 3. So if you're sitting there in a group of 3 of friends total, or we'll form one up here live, so come on up as our first volunteer, needed 5 more volunteers. Feel free to volunteer the persons next to you 3 in a row. How about 2 more over here, 1 and how about 2 on the end. Come on up. All right, a round of applause for these 6 year volunteers and. All right, so let me give you one microphone. Let me give you a 2nd microphone and Kelly, if you want to come on up as well. I think these three seem to know each other already, so we'll have them be one team if you guys want to be another team as well. Come on up. Uh, let me just take one microphone actually for the other team. All right. And how about quick introductions to this team here and we first we need a team name from you all. You haven't had time to think about this? Team A. OK, so Team A is who? I'm Leah. I'm a first year and I'm in Hoorthy. Welcome. Uh, my name is Stephen. I'm a freshman in. I'm Charlotte. I'm a freshman and I'm also in Canada. All right, let's do introductions on the other team as well. You are going to be team. Awesome sauce. Awesome sauce, OK, versus Team A, uh, if you wanna go ahead and introduce yourselves here. Hi, my name is Jenny Pan. I'm a freshman in Hollis. Hi, my name is Noah. I'm a freshman in Holbut. And hi, my name is Marie, and I'm a freshman, sorry, I'm a freshman in Canada B. All right, welcome to both of our teams here. And among the goals now, let's leave one microphone with each team, uh, is to play a bit of charades whereby one of you in a moment is going to be responsible for acting out a word that you see on the screen. So we're going to put on this screen and the screen over here some term that relates to CS 50 somehow, and that person's goal over the course of 60 seconds. Going to be to act that out in such a way that their teammates can hopefully guess what the word is. We'll give you 60 seconds at a time. Kelly has kindly offered to keep score, um, and if you solve it in fewer than 60 seconds, we've got another word for you in another word, and we'll see how many points you can accrue over the course of those 60 seconds. And depending on how this goes, we'll do maybe 1 or 2 rounds in total. Questions. How many skips do you get? I guess you can skip uh as many as you want until we run out of questions. But try not to run through all of our questions. All right, any questions that'll be on that. All right, so if you guys want to step off stage over there, why don't we have team A begin? So one of you, Leah, if you're holding the mic, if you want to be the curator, let's go ahead and have you stand here so you can see the screen, and we only ask that you two not look up because the answer is going to be right there. All right, and you should just shout out, uh, the word that Lee is acting out. Question. Acting only charades speaking. Yeah, yeah, I can't speak because that would kind of defeat the point. So yes, just acting out, just acting out physically. All right, I'm gonna go over here. Give me just a moment to get the slides ready with your questions. And Leah, the first clue, oh, and Kelly's gonna be timing you 60 seconds to accrue as many points as you can. All right, here we go. Go Act that out. 00, that was weird, thank you. Sorry. Yes, act out, this is CS 50. All right. No, act this out, please. Go. Calling a recursion. Yes, 1 point. uh, in array linkless. Abstraction? Python, Python, Python. Duck, the duck Nice. Binary uh 10 binary digit. Bite 10, definitely binary. Asky. Want to pass They Linkless array. Yes, array. Loop loop. Time. All right. Very nicely done. All right. 5 is the score to beat. So if you guys want to step over here,

if, uh, one of you has the mic, go ahead and assume the same roles. 5 is the score to beat. Alright, 5 is the score to beat. All right, here we go, final round. First word and you guys just make sure you don't look up. Go. Head node algorithm input. Algorithm Yes Well. These are hard No. Sure, you have to act it out, act it out. Oh, they go. Run time, run time. 3. Yes, 3. Next one. Next one. Binary search binary bullying? No. A merge phone call. It was binary search wasn't it? Oh, that's right, but a round of applause for our awesome sauce. OK, we have some, some parting prizes for you, your very own Super Mario Pezs for you guys as well. I'm glad we squared away the, the ability to pass though on the question, so thank you for that. All right, so admittedly pretty hard. Our thanks to all of these volunteers for playing that out. Allow me to turn our attention back to here in just a moment where else uh we can go from here. So up until now, up until now. We've been using Visual Studio Code for CS 50 at the URL CS50.dv. Recall that this is just an adaptation of a commercial tool called GitHub CodeSpaces, which is like a cloud-based version of Visual Studio Code itself, or VS Code, which is an largely open source tool for Microsoft that's incredibly popular in industry, which is to say, even though we had the CS50 library in there and we turned off by default some of the menu options and we disabled AI, it is the tool that so many programmers around the world do use every day to write code. So you have been learning. All this time sort of industry standards in that sense. It is now time, if you so choose, but you are welcome to keep using this for your final project if feeling more comfortable with it, to drop the 4 CS 50 and actually install on your own Mac or PC if you so choose, Visual Studio Code itself. You can go to this URL here. It's fairly straightforward to install it, but invariably you'll run into probably some technical support headaches depending on the language that you're trying to use with it. For instance, if you're trying to use it with Python, you'll probably also have to download and install Python on. Your computer, at least if you want the latest version, and just know a priori that sometimes just stuff happens and it just doesn't work and you have to Google or ask Chat GPT, and that's fine. And honestly that's kind of normal. But this is also why we don't do any of this in week zero of the class so that we can focus on Hello world and Mario and cash and credit and get into the interesting parts of computing and programming and not frustrating, not frustrating you so with technical support challenges. But now, given that all of you are somewhere in between. are among those more comfortable, you're now ready to sort of deal with those same technical challenge yourself, but who knows, maybe it will go perfectly smoothly. You can go to CS 50's own documentation because if you want to be able to use all of the same software that CS50 has pre-installed, you can use a technology known as containerization with a tool called Docker and actually run a CS50 environment on your Mac or PC or even in the cloud, but still run VS code on your own Mac and PC, among the upsides of which are that you're not dependent necessarily on the. You can do everything offline, which is useful in general. You can do things more quickly sometimes if you're using the full capabilities of your own computer and not just a browser. So this is generally how programmers approach their code using something like VS code or alternative products. And in fact there's a bunch of others out there, but perhaps the trendiest right now are these 3 here, not just Visual Studio code itself, but a tool called Cursor, another one called Windsurf, and there's dozens of other text editors, often known as integrated. Development environments which tend to have even more features that you can download for free or commercially on your own Macs, PCs and the like, uh, but you can't go wrong transitioning from CS 50 to VS code on your own Mac or PC if only because you're already familiar with it. As for the command line, so those of

you with Macs might have found somewhere in your utilities folder a program called Terminal. Um, if not, poke around there later today and you'll see that all this time you've had a command line interface available to you on Mac OS. Windows has something similar as well. They they. Necessarily come with all of the same tools that we've been using within CS50. Dev, but if you're a Mac user and you go to this URL here or you're a Windows user and you go to this URL here, or if you're a Linux user, you probably know all of this already, so there's no URL for you there. You can install some of those same tools on your Mac and PC and feel all the more at home doing things in a command line as well. Um, Git, this is something that we actually in CS 50 abstract on top of. This is essentially the de facto standard nowadays for collaborating with other people. Using a central cloud server in order to share your code with it and in turn other people for versioning your code so that you keep track of multiple versions thereof and changes that you've made. Go to this URL here if you would like, and you'll see a tutorial by CS 50's own Brian Yu introducing you to actual Git because we've been sort of abstracting away this particular tool by just doing it all automatically for you. If you've ever gone through your timeline in CS50.dev being able to roll back to previous versions of your code, we're just using Git, but we're. Automatically running this command for you. If you want to collaborate with partners for your final project, you can use Git. However, I will encourage you to alternatively use Visual Studio Code's live share feature, which allows one of you to log into your code space, click some buttons, and then share access to your code space with your friend or your partner with whom you're working on the project, and you can both in real time, like Google Docs, edit the code or different files they're in using that one code space, a little easier than getting onboarded at least with. hosting a website if this proves of interest for your final project or even after the course, if it's a static website, two popular places to go, if only because they offer free tiers, is what's called GitHub pages, which you can use to just host HTML, CSS and JavaScript with no Python, no flask, no back end, or NetLFy is a popular company nowadays too that has an entry level account for which you can sign up for free. If you just want to have like a portfolio website, if you're an artist or a programmer or you just want to have static content that you write once and deploy, these. Good starting points, but not all of them. Hosting a web app, so this, this list gets even longer, and all of these recommendations are essentially curated by the teaching staff, so they're all opinionated, but these are perhaps the most common places you can go. Amazon, Microsoft, Google, Cloudflare, they all have student type accounts. So if you use your .edu email address, for instance, or some other form of proving your status as a current student, you can generally sign up for discounts and free access to a lot of these same services as well without. Having to pay while you're just learning along the way. GitHub has something similar called the Student Developer Pack, and then a couple of other companies for hosting web apps that have been popular are Hiroku, Versal, and bunches of others. So by web app we mean not just HTML CSS and JavaScript, but maybe some Python, maybe some JavaScript on the server, maybe Ruby, yet another language or any number of others when you actually need a back end in addition to the front end, maybe you need a database as well. This would be the place to start, whether it's at the CS 50 hack. or beyond, and nowadays this is a slide that didn't even need to exist a couple of years ago asking AI again for your final projects, you are welcome and encouraged to amplifying your own productivity with AI, not by having it do it for you, but moving away from the duck, which by design has been fairly limited and meant to be a good teacher, but not necessarily one that's going to be a good partner when it comes to

building your final project. So Chachi BT, Claw, Gemini, GitHub copilot, OpenAI Codex, vro, um, are all, uh. Popular tools right now that you might want to play around with. The easiest of these to use perhaps, if not familiar with say Chacha BT already, would be GitHub co-pilot only because you can enable it within your CS50 code space by following our own documentation at cs50.readthedocs.io where we'll tell you the sequence of steps via which you can re-enable AI now that you're allowed to for your final project and turn on all of those features that were disabled by default. Um, and then there's still humans out there, like what remains to be seen just. Popular these websites are in the years to come, for better or for worse, but among the places that programmers and technophiles have gone for years are Reddit, Stack Overflow, Server fault, where there's a rich history of questions and answers that ironically all of those AIs have been trained on, which unfortunately means some of these might be driven out of business eventually in some sense if we're all just turning only to AI. But when you actually want that human component, these are still good places to go. And then news, two of the many places you can go for news and technology, computing, computer. Science more broadly would be TechCrunch is still a good one. Hacker news, so to speak, and then you might have some of your own popular choices as well. Um, and then if, uh, with some bias, um, take other classes like CS 50 besides this undergraduate class has a rich history now over the past decade of creating all the more open courseware, so courses in more Python, more SQL, a language called R, cybersecurity, uh, game development, and more. All of those are linked at this URL here edX.org/cs50 where you need not pay or sign up beyond. the course and all of the content is freely available, something for winter break, for instance, if you want to dive a little more deeply into some subject for the sake of your final project, your professional aspirations, or even just prepare for spring term. And then over the coming weeks too, will CS50 itself be soliciting interest and applications for becoming a teaching fellow or TF, a course assistant, or CA? If you would like to get all the more involved as a teacher of CS 50 next fall, uh, do, uh, follow the application link that we will soon circulate via email. And do stay in touch too if you just enjoy answering other people's questions or seeing what the pulse of sort of computing is, at this URL here is a whole bunch of CS 50's own communities in social media largely via which you can follow along at home in the months and years to come too. So a few thanks before we do one final game altogether to all of the people who have been making this course possible. So our friends at Memorial Hall who make bringing us into this beautiful space and make it possible for us to have, of all things, a class in such a. Space, um, our friends at ESS who help with the audio each and every week in CS 50, um, the restaurant Changzhou down the road. We hope you'll continue to visit our friends there. Wesley Chen is a good friend of ours and the manager. Um, please tell him you're from CS 50 and I'm sure he'll be delighted to see you. Um, and then CS 50's own team, well, most of whom were in back there are sitting next to you with cameras, um, without whom the course wouldn't be possible. And of course CS 50's own teaching fellows in CAs, just a few of whom posed here for this photo. If I could invite you to all give everyone here a round of applause, my thanks to all of them. So, um, and then of course, the CS 50 duck should be thanked as well. OK. Thanks to CSFT's own Rong Xin Lu and some of our own former teaching fellows and students who have been behind the development of that their duck that you've gotten to know over these past several months. All right, if Kelly could join me again on screen, the only thing between us and Cake is a final game, namely a quiz show in which all of you can partake. Here we go. Question one. What is the largest number in 8 bit unsigned

binary digit can represent? 256, 128, 255, or 1? Starting strong and keep in mind all of these questions came from you all, because we asked you recently for review questions that are now on the screen. Again, the timer is clicking and most popular answer was 255, which I think if we click once more, we'll confirm was in fact the correct answer. So, why is that and why is it not 256? Well, if we start counting from zero, as we always have. That's consuming one of the 256 possibilities, so the largest number that we can represent with that's 8 bit and unsigned, which means no negative numbers involved, is indeed going to be 255. Treasure, that information now always. All right, next question from Kelly. Which issue is at the center of the year 2038 problem, which hopefully you added to your Google calendars a few weeks back, entered your overflow, malicious inputs, SQL injection attacks, or memory leak? Which of those is at the core of the year 2038 problem? All right, let's go ahead and reveal the number one answer with 92% of you saying integer overflow is in fact correct because we're still in the habit of using 32 bit integers to keep track of time from the so-called epoch, which was January 1st, 1970, and unfortunately we humans aren't great at sort of planning ahead and so we're going to run out of permutations of 32 bits. By a certain date in the year 2038 unless everyone upgrades their computers to 64 bit counters, which thankfully most every piece of modern hardware nowadays is using already your Macs, your PCs, and your phones. So hopefully this will be really a non-event, but hopefully you'll think of us in CS 50 in, uh, you know, 10 plus years when your Google Calendar reminder goes off. Question three, which of the following is not a step of compiling, linking, pre-processing, assembling, or interpreting? A bit more of a challenge, which of these is not a step of compiling. Alright, almost 200 responses coming in. All right, why don't we go ahead and reveal the most popular answer with 54% of you saying interpreting is in fact correct. Recall that we, we talked about compiling. Compiling itself is just one of several steps. There is in fact the preprocessing step which takes care of any of the hash symbols in C that start. With hash include hashdefine, and the like, that's preprocessing. There was then assembling or there was then compiling, which actually compiled your code into assembly code. There was then the assembler which would actually take it down further to machine code and then linking. 29 this is for 29% of you. The linking step recall was taking your zeros and ones and combining them with say CS 50's libraries zeros and ones and maybe the standard IO library zeros and ones linking them all together to give you one executable program like hell itself. All right, next question. What does a pointer store, the name of a variable, the memory addresses of a value, the size of a value, or the value of a variable? Think for a moment. What does a pointer store? Alright, about 200 responses in and yes, the memory address of a variable with 96% of you confirming as much. That is correct. Question 5. What is the running time of linear search? Big O of 1, big O of N, big O of N squared, or big O of N log N? Linear search running time. I recall that with something like search you could get lucky, but if big O is the upper bound on our running time you might not you might hit the end of the list that you're searching and so the running time of linear search is of course big O of N. And it might be omega of 1, but not big O of one, at least if we're considering what the worst case scenarios might be. All right, on to question six. What data structure follows the first in, first out principle? A queue, a link list, a stack, or a hash table? first in, first out, AKA FIFO. Which of these is FIFO? All right, first in, first out is in fact a queue, as you would hope if you're getting in line for a restaurant, for a store, you'd hope that if you're the first one in line, you're going to be the first one out, equitably speaking, and so it is in fact a queue. The opposite of that in some sense then would have been a stack whereby when

you think about the cafeteria trays, the sort of first one in is actually the last one out, so LIFO instead of a stack. All right, question 7, which operator returns the memory address of a variable? An asterisk, a dollar sign, an ampersand, or a hyphen, and a greater than sign. Presumably in C. Which returns the memory address. Of a variable All right, let's see what everyone thinks. So the most popular and correct answer is the ampersand. This is the address of operator. The asterisk recall in most contexts is the opposite of that, that's the reference operators actually go to an address, um. This is not a thing in C. This though is similar in spirit to a combination of the star operator and the dot operator, which means to de-reference and follow a pointer to something inside of a struct typically. All right, question 8, which SQL command is used to remove duplicate rows from a result set? Remove unique, distinct, or clean. We didn't spend a huge amount of time on these keywords, but only one of them applies here. A result set is just the answers that you get back when doing your select. And if you want to filter out duplicates, you can in fact say. Distinct is correct. Unique is also a keyword in SQL, but that is when you want to define in your schema that a column's values are going to be unique like an email address column. Instead, distinct is how you filter out duplicates in your selects. All right, question 9, we're past the halfway mark. What does an HTTP code of 418 signify? Not found, I'm a teapot, forbidden, unauthorized. 4:18. This too, if you know this one moving forward, you'll be considered among the CS. Elite Answers are coming in a little slower, but I'm a teapot is correct, which is not actually a thing or useful technology. It was in fact an April Fool's joke years ago where a bunch of computer scientists got together in a room and wrote out an entire specification for what it means for a server to return 418, I'm a teapot. All right, number 10. Where does Mao dynamically allocate memory from? The heap, the stack, global variables, or assembly? All right. Heap is in fact correct. That's the sort of top part of the memory, even though top and bottom make no actual technical sense. It's just our artist's rendition thereof. The stack recall is what is used when functions are being called. Every time a function is called, it gets a so-called frame on the stack. That's where your local variables and your arguments get put. But if in C you use Mao, it does in fact end up on the heap. In C, if you allocate memory with Mao but forget to call free, what problem can occur a memory leak, segmentation fault, stack overflow, or all of the above? If you allocate memory with Mao, but forget to call free, what problem can occur? All right, most popular answer is in fact memory leak, which is correct. Um, you could imagine scenarios in which you also get a segmentation fault and or a stack overflow, but those aren't direct consequences of not calling free. That's generally. Consequence of using too much memory, for instance, or in this case doing something wrong with your memory. So interrelated, yes, but in terms of not calling free for each Mallock, this is what's going to happen by definition. All right, well done there. Next question, which is 12, what does this domain name give the web page of Safetyschool.org? Is it Harvard University? Is it Princeton University? Is it Yale University or Columbia University? All right, recall that this was in the context of our HTTP redirections. Yes, interesting. Yes, in fact, uh, Yale University, some alum has been paying like \$10 a year for like 20 years for this joke. Safetyschool.org, if you visit it, returns an HDP 301 HDP header, which says the location of it is in fact yale.edu. All right, 13, 3 to go. What is the purpose of DNS? Uh, to encrypt data sent over the dark web. To find the nearest coffee shop for you, to protect your location against hackers or to translate domain names into IP addresses. What is the purpose of DNS? If helpful, domain name system. Alright, about at the 200 mark, and the correct answer is indeed domain names into IP addresses. That is a server that is on

your home network, on your ISP's network, on your campus's network, your corporate network that just answers questions like that for you. All right, second to last question, which of the following is not a built-in SQL feature to tackle race conditions? Begin transaction, commit, roll back, or enroll. We talked ever so briefly about this in the context of ending up with too much milk, recall. And the correct answer is. Indeed, enroll. All three of those, even though you didn't have to use them for problem sets, 7 or 9 are indeed, uh, features of SQL, uh, but enroll is not a thing. All right. And the very last question, try to answer this as quickly as you can. What does Professor Mayland say at the beginning of every CS 50 lecture? Welcome to Harvard's Computer Science class. Hello everyone, ready to code? All right, this is CS 50 or let's get started with some programming. All of these questions were in fact written by you all. All right. And the correct answer, I'm pretty sure with 98% of you saying so is all right. This is CS 50 and all right, this was CS 50, cake is now served.